JavaOOP面试题

1、什么是B/S架构？什么是C/S架构

1. B/S(Browser/Server)，浏览器/服务器程序
2. C/S(Client/Server)，客户端/服务端，桌面应用程序

2.C/S(Client/Server)，客户端/服务端，桌面应用程序

2、Java都有那些开发平台？

1. JAVA SE：主要用在客户端开发
2. JAVA EE：主要用在web应用程序开发
3. JAVA ME：主要用在嵌入式应用程序开发

3、什么是JDK？什么是JRE?

1. JDK：java development kit：java开发工具包，是开发人员所需要安装的环境
2. JRE：java runtime environment：java运行环境，java程序运行所需要安装的环境

4、Java语言有哪些特点

1. 简单易学、有丰富的类库
2. 面向对象（Java最重要的特性，让程序耦合度更低，内聚性更高）
3. 与平台无关性（JVM是Java跨平台使用的根本）
4. 可靠安全
5. 支持多线程

5、面向对象和面向过程的区别

1. 面向过程：

   一种较早的编程思想，顾名思义就是该思想是站着过程的角度思考问题，强调的就是功能行为，功能的执行过程，即先后顺序，而每一个功能我们都使用函数（类似于方法）把这些步骤一步一步实现。使用的时候依次调用函数就可以了。

2. 面向对象：
   一种基于面向过程的新编程思想，顾名思义就是该思想是站在对象的角度思考问题，我们把多个功能合理放到不同对象里，强调的是具备某些功能的对象。

   具备某种功能的实体，称为对象。面向对象最小的程序单元是：类。面向对象更加符合常规的思维方式，稳定性好，可重用性强，易于开发大型软件产品，有良好的可维护性。

   在软件工程上，面向对象可以使工程更加模块化，实现更低的耦合和更高的内聚。

6、什么是数据结构？

计算机保存，组织数据的方式

7、Java的数据结构有那些？

1.线性表（ArrayList）
2.链表（LinkedList）
3.栈（Stack）
4.队列（Queue）
5.图（Map）
6.树（Tree）

8、什么是OOP?

面向对象编程

9、类与对象的关系?

类是对象的抽象，对象是类的具体，类是对象的模板，对象是类的实例

10、Java中有几种数据类型

整形：byte,short,int,long
浮点型：float,double
字符型：char
布尔型：boolean

11、标识符的命名规则。

1. 标识符的含义：
   是指在程序中，我们自己定义的内容，譬如，类的名字，方法名称以及变量名称等等，都是标识符。
2. 命名规则：（硬性要求）
   标识符可以包含英文字母，0-9的数字，$以及_
   标识符不能以数字开头
   标识符不是关键字
3. 命名规范：（非硬性要求）
   类名规范：首字符大写，后面每个单词首字母大写（大驼峰式）。
   变量名规范：首字母小写，后面每个单词首字母大写（小驼峰式）。
   方法名规范：同变量名。

12、instanceof关键字的作用

instanceof 严格来说是Java中的一个双目运算符，用来测试一个对象是否为一个类的实例，用法为：

```
boolean result = obj instanceof Class
```

其中 obj 为一个对象，Class 表示一个类或者一个接口，当 obj 为 Class 的对象，或者是其直接或

间接子类，或者是其接口的实现类，结果result 都返回 true，否则返回false。

注意：编译器会检查 obj 是否能转换成右边的class类型，如果不能转换则直接报错，如果不能确定

类型，则通过编译，具体看运行时定。

```
inti=0;

System.out.println(i instanceof Integer);//编译不通过i必须是引用类型，不能是基本类型

System.out.println(i instanceof Object);//编译不通过

Integer integer=newInteger(1);

System.out.println(integer instanceof Integer);//true
//false,在JavaSE规范中对instanceof运算符的规定就是：如果obj为null，那么将返回false。
System.out.println(nullinstanceofObject);
```

13、什么是隐式转换，什么是显式转换

显示转换就是类型强转，把一个大类型的数据强制赋值给小类型的数据；隐式转换就是大范围的变量能够接受小范围的数据；隐式转换和显式转换其实就是自动类型转换和强制类型转换。

14、Char类型能不能转成int类型？能不能转化成string类型，能不能转成double类型

Char在java中也是比较特殊的类型，它的int值从1开始，一共有2的16次方个数据；

Char<int<long<float<double；Char类型可以隐式转成int,double类型，但是不能隐式转换成string；如果char类型转成byte，short类型的时候，需要强转。

15、什么是拆装箱？

1. 装箱就是自动将基本数据类型转换为包装器类型（int-->Integer）；调用方法：Integer的 **valueOf(int) 方法**

   拆箱就是自动将包装器类型转换为基本数据类型（Integer-->int）。调用方法：Integer的intValue方 法

   在Java SE5之前，如果要生成一个数值为10的Integer对象，必须这样进行：

   ```
   Integer i = new Integer(10);
   ```

   而在从Java SE5开始就提供了自动装箱的特性，如果要生成一个数值为10的Integer对象，只需要这样就可以了：

   ```
   Integer i = 10;
   ```

   面试题**1**： 以下代码会输出什么？

   ```
   public class Main {

       public static void main(String[] args) {

           Integer i1 = 100;

           Integer i2 = 100;

           Integer i3 = 200;

           Integer i4 = 200;

           System.out.println(i1==i2);

           System.out.println(i3==i4);

       }
   }
   ```

   结果：

   ```
   true

   false
   ```

16、Java中的包装类都是那些？

byte：Byte，short：Short，int：Integer，long：Long，float：Float，double：Double，char：Character ，boolean：Boolean

17、一个java类中包含那些内容？

属性、方法、内部类、构造方法、代码块。

18、那针对浮点型数据运算出现的误差的问题，你怎么解决？

使用Bigdecimal类进行浮点型数据的运算

19、面向对象的特征有哪些方面?

抽象:

抽象是将一类对象的共同特征总结出来构造类的过程, 包括数据抽象和行为抽象两方面。抽象只关注对象有哪些属性和行为,并不关注这些行为的细节是什么。

继承:

继承是从已有类得到继承信息创建新类的过程.提供继承信息的类被称为父类(超类、基类) ;得到继承信息的类被称为子类(派生类)。继承让变化中的软件系统有了一定的延续性 ,同时继承也是封装程序中可变因素的重要手段(如果

不能理解请阅读阎宏博士的《Java 与模式》或《设计模式精解》中.关于桥梁模式的部分)。

封装：

通常认为封装是把数据和操作数据的方法绑定起来，对数据的访问

只能通过已定义的接口。面向对象的本质就是将现实世界描绘成一系列完全自

治、封闭的对象。我们在类中编写的方法就是对实现细节的一种封装；我们编写

一个类就是对数据和数据操作的封装。可以说，封装就是隐藏一切可隐藏的东西，

只向外界提供最简单的编程接口（可以想想普通洗衣机和全自动洗衣机的差别，

明显全自动洗衣机封装更好因此操作起来更简单；我们现在使用的智能手机也是

封装得足够好的，因为几个按键就搞定了所有的事情）。

多态性：

多态性是指允许不同子类型的对象对同一消息作出不同的响应。

简单的说就是用同样的对象引用调用同样的方法但是做了不同的事情。多态性分

为编译时的多态性和运行时的多态性。如果将对象的方法视为对象向外界提供的

服务，那么运行时的多态性可以解释为：当 A 系统访问 B 系统提供的服务时，B

系统有多种提供服务的方式，但一切对 A 系统来说都是透明的（就像电动剃须

刀是 A 系统，它的供电系统是 B 系统，B 系统可以使用电池供电或者用交流电，

甚至还有可能是太阳能，A 系统只会通过 B 类对象调用供电的方法，但并不知道

供电系统的底层实现是什么，究竟通过何种方式获得了动力）。方法重载

（overload）实现的是编译时的多态性（也称为前绑定），而方法重写（override）

实现的是运行时的多态性（也称为后绑定）。运行时的多态是面向对象最精髓的

东西，要实现多态需要做两件事：1). 方法重写（子类继承父类并重写父类中已

有的或抽象的方法）；2). 对象造型（用父类型引用引用子类型对象，这样同样

的引用调用同样的方法就会根据子类对象的不同而表现出不同的行为）。

20、访问修饰符 public,private,protected,以及不写（默认）时的区别？

修饰符	当前类	同 包	子 类	其他包
public	能	能	能	能
protected	能	能	能	不能
default	能	能	不能	不能
private	能	不能	不能	不能

类的成员不写访问修饰时默认为 default。默认对于同一个包中的其他类相当于公 开（public），对于不是同一个包中的其他类相当于私有（private）。受保护（protected）对子类相当于公开，对不是同一包中的没有父子关系的类相当于私 有。Java 中，外部类的修饰符只能是 public 或默认，类的成员（包括内部类）的 修饰符可以是以上四种。

21、String 是最基本的数据类型吗？

不是。Java 中的基本数据类型只有 8 个：byte、short、int、long、float、double、 char、boolean；除了基本类型（primitive type），剩下的都是引用类型（reference type），Java 5 以后引入的枚举类型也算是一种比较特殊的引用类型。

22、float f=3.4;是否正确？

答:不正确。3.4 是双精度数，将双精度型（double）赋值给浮点型（float）属于 下转型（down-casting，也称为窄化）会造成精度损失，因此需要强制类型转换 float f =(float)3.4; 或者写成 float f =3.4F;。

23、short s1 = 1; s1 = s1 + 1;有错吗?short s1 = 1; s1 += 1; 有错吗？

对于 short s1 = 1; s1 = s1 + 1;由于 1 是 int 类型，因此 s1+1 运算结果也是 int 型，需要强制转换类型才能赋值给 short 型。而 short s1 = 1; s1 += 1;可以正确 编译，因为 s1+= 1;相当于 s1 = (short)(s1 + 1);其中有隐含的强制类型转换。

24、重载和重写的区别

重写**(Override)**

从字面上看，重写就是 重新写一遍的意思。其实就是在子类中把父类本身有的方法重新写一遍。子类继承了父类

原有的方法，但有时子类并不想原封不动的继承父类中的某个方法，所以在方法名，参数列表，返回类型(除过子

类中方法的返回值是父类中方法返回值的子类时)都相同的情况下， 对方法体进行修改或重写，这就是重写。但要注意子类函数的访问修饰权限不能少于父类的。

```
public class Father {

    public static void main(String[] args) {

        // TODO Auto-generated method stub

        Son s = new Son();

        s.sayHello();

    }

    public void sayHello() {

        System.out.println("Hello");
    }
}

class Son extends Father{

    @Override

    public void sayHello() {

        // TODO Auto-generated method stub

        System.out.println("hello by ");

    }

}
```

原因： 在某个范围内的整型数值的个数是有限的，而浮点数却不是。

重写 总结：

1.发生在父类与子类之间

2.方法名，参数列表，返回类型（除过子类中方法的返回类型是父类中返回类型的子类）必须相同

3.访问修饰符的限制一定要大于被重写方法的访问修饰符（public>protected>default>private)

4.重写方法一定不能抛出新的检查异常或者比被重写方法申明更加宽泛的检查型异常

重载（Overload）

在一个类中，同名的方法如果有不同的参数列表（**参数类型不同、参数个数不同甚至是参数顺序不同**）则视为重载。同时，重载对返回类型没有要求，可以相同也可以不同，但**不能通过返回类型是否相同来判断重载。**

```
public class Father {

    public static void main(String[] args) {

        // TODO Auto-generated method stub

        Father s = new Father();

        s.sayHello();

        s.sayHello("wintershii");

      }

    public void sayHello() {

        System.out.println("Hello");

    }

    public void sayHello(String name) {

        System.out.println("Hello" + " " + name);

    }

}
```

重载总结：

1.重载Overload是一个类中多态性的一种表现

2.重载要求同名方法的参数列表不同(参数类型，参数个数甚至是参数顺序)

3.重载的时候，返回值类型可以相同也可以不相同。无法以返回型别作为重载函数的区分标准

25、equals与==的区别

==：

== 比较的是变量(栈)内存中存放的对象的(堆)内存地址，用来判断两个对象的地址是否相同，即是否是指相同一个对象。比较的是真正意义上的指针操作。

1、比较的是操作符两端的操作数是否是同一个对象。

2、两边的操作数必须是同一类型的（可以是父子类之间）才能编译通过。

3、比较的是地址，如果是具体的阿拉伯数字的比较，值相等则为true，如：

int a=10 与 long b=10L 与 double c=10.0都是相同的（为true），因为他们都指向地址为10的堆。

equals：

equals用来比较的是两个对象的内容是否相等，由于所有的类都是继承自java.lang.Object类的，所以适用于所有对象，如果没有对该方法进行覆盖的话，调用的仍然是Object类中的方法，而Object中的equals方法返回的却是==的判断。

总结：

所有比较是否相等时，都是用equals 并且在对常量相比较时，把常量写在前面，因为使用object的equals object可能为null 则空指针

在阿里的代码规范中只使用equals ，阿里插件默认会识别，并可以快速修改，推荐安装阿里插件来排查老代码使用“==”，替换成equals

36、++i与i++的区别

i++：先赋值，后计算
++i：先计算，后赋值

37、程序的结构有那些？

```
顺序结构
选择结构
循环结构
```

38、数组实例化有几种方式？

静态实例化：创建数组的时候已经指定数组中的元素,

```
int [] a= new int[]{ 1 , 3 , 3}
```

动态实例化：实例化数组的时候，只指定了数组程度，数组中所有元素都是数组类型的默认值

39、Java中各种数据默认值

Byte,short,int,long默认是都是0

Boolean默认值是false

Char类型的默认值是”

Float与double类型的默认是0.0

对象类型的默认值是null

40、Java常用包有那些？

```
Java.lang
Java.io
Java.sql
Java.util
Java.awt
Java.net
Java.math
```

41、Object类常用方法有那些？

```
Equals
Hashcode
toString
wait
notify
clone
getClass
```

42、java中有没有指针？

有指针，但是隐藏了，开发人员无法直接操作指针，由jvm来操作指针

43、java中是值传递引用传递？

理论上说，java都是引用传递，对于基本数据类型，传递是值的副本，而不是值本身。对于对象类型，传递是对象的引用，当在一个方法操作操作参数的时候，其实操作的是引用所指向的对象。

44、实例化数组后，能不能改变数组长度呢？

不能，数组一旦实例化，它的长度就是固定的

45、假设数组内有5个元素，如果对数组进行反序，该如何做？

创建一个新数组，从后到前循环遍历每个元素，将取出的元素依次顺序放入新数组中

46、形参与实参区别

实参(argument)：

全称为"实际参数"是在调用时传递给函数的参数. 实参可以是常量、变量、表达式、函数等， 无论实参是何种类型的量，在进行函数调用时，它们都必须具有确定的值， 以便把这些值传送给形参。 因此应预先用赋值，输入等办法使实参获得确定值。

形参(parameter)：

全称为"形式参数" 由于它不是实际存在变量，所以又称虚拟变量。是在定义函数名和函数体的时候使用的参数,目的是用来接收调用该函数时传入的参数.在调用函数时，实参将赋值给形参。因而，必须注意实参的个数，类型应与形参一一对应，并且实参必须要有确定的值。

形参出现在**函数定义**中，在整个函数体内都可以使用， 离开该函数则不能使用。

实参出现在**主调函数中，进入被调函数后，实参变量也不能使用。**

形参和实参的功能是作数据传送。发生函数调用时， **主调函数把实参的值传送给被调函数的形参从而实现主调函数向被调函数的数据传送。**

1.形参变量只有在被调用时才分配内存单元，**在调用结束时， 即刻释放所分配的内存单元。**因此，形参只有在函数内部有效。 函数调用结束返回

主调函数后则不能再使用该形参变量。

2.实参可以是常量、变量、表达式、函数等， 无论实参是何种类型的量，在进行函数调用时，它们都必须具有确定的值， 以便把这些值传送给形

参。 因此应预先用赋值，输入等办法使实参获得确定值。

3.实参和形参在数量上，类型上，顺序上应严格一致， 否则会发生"类型不匹配"的错误。

4.**函数调用中发生的数据传送是单向的。** 即只能把实参的值传送给形参，而不能把形参的值反向地传送给实参。 因此在函数调用过程中，形参的值

发生改变，而实参中的值不会变化。

5.当形参和实参不是指针类型时，在该函数运行时，**形参和实参是不同的变量，他们在内存中位于不同的位置，形参将实参的内容复制一份，在该**

函数运行结束的时候形参被释放，而实参内容不会改变。

而**如果函数的参数是指针类型变量,在调用该函数的过程中，传给函数的是实参的地址，在函数体内部使用的也是实参的地址，即使用的就是实参**

本身。所以在函数体内部可以改变实参的值。

47、构造方法能不能显式调用？

不能，构造方法当成普通方法调用，只有在创建对象的时候它才会被系统调用

48、什么是方法重载？

方法的重载就是在同一个类中允许同时存在一个以上的同名方法，只要它们的参数个数或者类型不同即可。在这种情况下，该方法就叫被重载了，这个过程称为方法的重载（override）

49、构造方法能不能重写？能不能重载？

可以重载，但不能重写。

50、内部类与静态内部类的区别？

静态内部类相对与外部类是独立存在的，在静态内部类中无法直接访问外部类中变量、方法。如果要访问的话，必须要new一个外部类的对象，使用new出来的对象来访问。但是可以直接访问静态的变量、调用静态的方法；

普通内部类作为外部类一个成员而存在，在普通内部类中可以直接访问外部类属性，调用外部类的方法。

如果外部类要访问内部类的属性或者调用内部类的方法，必须要创建一个内部类的对象，使用该对象访问属性或者调用方法。

如果其他的类要访问普通内部类的属性或者调用普通内部类的方法，必须要在外部类中创建一个普通内部类的对象作为一个属性，外同类可以通过该属性调用普通内部类的方法或者访问普通内部类的属性

如果其他的类要访问静态内部类的属性或者调用静态内部类的方法，直接创建一个静态内部类对象即可。

51、Static关键字有什么作用？

Static可以修饰内部类、方法、变量、代码块

Static修饰的类是静态内部类

Static修饰的方法是静态方法，表示该方法属于当前类的，而不属于某个对象的，静态方法也不能被重写，可以直接使用类名来调用。在static方法中不能使用this或者super关键字。

Static修饰变量是静态变量或者叫类变量，静态变量被所有实例所共享，不会依赖于对象。静态变量在内存中只有一份拷贝，在JVM加载类的时候，只为静态分配一次内存。

Static修饰的代码块叫静态代码块，通常用来做程序优化的。静态代码块中的代码在整个类加载的时候只会执行一次。静态代码块可以有多个，如果有多个，按照先后顺序依次执行。

52、final在java中的作用，有哪些用法?

final也是很多面试喜欢问的地方,但我觉得这个问题很无聊,通常能回答下以下5点就不错了:

1. 被fifinal修饰的类不可以被继承
2. 被fifinal修饰的方法不可以被重写
3. 被fifinal修饰的变量不可以被改变.如果修饰引用,那么表示引用不可变,引用指向的内容可变.
4. 被fifinal修饰的方法,JVM会尝试将其内联,以提高运行效率
5. 被fifinal修饰的常量,在编译阶段会存入常量池中.

除此之外,编译器对fifinal域要遵守的两个重排序规则更好:

在构造函数内对一个fifinal域的写入,与随后把这个被构造对象的引用赋值给一个引用变量,这两个操作之间不能重排序

初次读一个包含fifinal域的对象的引用,与随后初次读这个fifinal域,这两个操作之间不能重排序

53、StringString StringBuffffer 和 StringBuilder 的区别是什么？

String是只读字符串，它并不是基本数据类型，而是一个对象。从底层源码来看是一个fifinal类型的字符

数组，所引用的字符串不能被改变，一经定义，无法再增删改。每次对String的操作都会生成新的

String对象

```
private final char value[];
```

每次+操作 ： 隐式在堆上new了一个跟原字符串相同的StringBuilder对象，再调用append方法 拼接

+后面的字符。

StringBuffer与StringBuilder都继承了AbstractStringBulder类，而AbtractStringBuilder又实现了CharSequence接口，两个类都是用来进行字符串操作的。

在做字符串拼接修改删除替换时，效率比string更高。

StringBuffer是线程安全的，Stringbuilder是非线程安全的。所以Stringbuilder比stringbuffer效率更高，StringBuffer的方法大多都加了synchronized关键字

54、String str=”aaa”,与String str=new String(“aaa”)一样吗？

一共有两个引用，三个对象。因为”aa”与”bb”都是常量，常量的值不能改变，当执行字符串拼接时候，会创建一个新的常量是” aabbb”,有将其存到常量池中。

55、讲下java中的math类有那些常用方法？

Pow()：幂运算
Sqrt()：平方根
Round()：四舍五入
Abs()：求绝对值
Random()：生成一个0-1的随机数，包括0不包括1

56、String类的常用方法有那些？

charAt：返回指定索引处的字符
indexOf()：返回指定字符的索引
replace()：字符串替换
trim()：去除字符串两端空白
split()：分割字符串，返回一个分割后的字符串数组
getBytes()：返回字符串的byte类型数组
length()：返回字符串长度
toLowerCase()：将字符串转成小写字母
toUpperCase()：将字符串转成大写字符
substring()：截取字符串
format()：格式化字符串
equals()：字符串比较

57、Java中的继承是单继承还是多继承

Java中既有单继承，又有多继承。对于java类来说只能有一个父类，对于接口来说可以同时继承多个接口

58、Super与this表示什么？

Super表示当前类的父类对象
This表示当前类的对象

59、普通类与抽象类有什么区别？

普通类不能包含抽象方法，抽象类可以包含抽象方法
抽象类不能直接实例化，普通类可以直接实例化

60、什么是接口？为什么需要接口？

接口就是某个事物对外提供的一些功能的声明，是一种特殊的java类，接口弥补了java单继承的缺点

61、接口有什么特点？

接口中声明全是public static final修饰的常量
接口中所有方法都是抽象方法
接口是没有构造方法的
接口也不能直接实例化
接口可以多继承

62、抽象类和接口的区别?

抽象类：

1. 抽象方法，只有行为的概念，没有具体的行为实现。使用abstract关键字修饰，没有方法体。子类必须重写这些抽象方法。
2. 包含抽象方法的类，一定是抽象类。
3. 抽象类只能被继承，一个类只能继承一个抽象类。

接口：

1. 全部的方法都是抽象方法，属性都是常量
2. 不能实例化，可以定义变量。
3. 接口变量可以引用具体实现类的实例
4. 接口只能被实现，一个具体类实现接口，必须实现全部的抽象方法
5. 接口之间可以多实现
6. 一个具体类可以实现多个接口，实现多继承现象

63、Hashcode的作用

java的集合有两类，一类是List，还有一类是Set。前者有序可重复，后者无序不重复。当我们在set中插入的时候怎么判断是否已经存在该元素呢，可以通过equals方法。但是如果元素太多，用这样的方法就会比较满。

于是有人发明了哈希算法来提高集合中查找元素的效率。 这种方式将集合分成若干个存储区域，每个对象可以计算出一个哈希码，可以将哈希码分组，每组分别对应某个存储区域，根据一个对象的哈希码就可以确定该对象应该存储的那个区域。

hashCode方法可以这样理解：它返回的就是根据对象的内存地址换算出的一个值。这样一来，当集合要添加新的元素时，先调用这个元素的hashCode方法，就一下子能定位到它应该放置的物理位置上。如果这个位置上没有元素，它就可以直接存储在这个位置上，不用再进行任何比较了；如果这个位置上已经有元素了，就调用它的equals方法与新元素进行比较，相同的话就不存了，不相同就散列其它的地址。这样一来实际调用equals方法的次数就大大降低了，几乎只需要一两次。

64、 Java的四种引用，强弱软虚

强引用

强引用是平常中使用最多的引用，强引用在程序内存不足（OOM）的时候也不会被回收，使用方式：

```
String str = new String("str");
```

软引用

软引用在程序内存不足时，会被回收，使用方式：

```
// 注意：wrf这个引用也是强引用，它是指向SoftReference这个对象的，

// 这里的软引用指的是指向new String("str")的引用，也就是SoftReference类中T

SoftReference<String> wrf = new SoftReference<String>(new String("str"));
```

可用场景： 创建缓存的时候，创建的对象放进缓存中，当内存不足时，JVM就会回收早先创建的对象。

弱引用

弱引用就是只要JVM垃圾回收器发现了它，就会将之回收，使用方式：

```
WeakReference<String>wrf=newWeakReference<String>(str);
```

可用场景：Java源码中的java.util.WeakHashMap中的key就是使用弱引用，我的理解就是，

一旦我不需要某个引用，JVM会自动帮我处理它，这样我就不需要做其它操作。

虚引用

虚引用的回收机制跟弱引用差不多，但是它被回收之前，会被放入ReferenceQueue中。注意哦，其它引用是被JVM回收后才被传入ReferenceQueue中的。由于这个机制，所以虚引用大多被用于引用销毁前的处理工作。还有就是，虚引用创建的时候，必须带有ReferenceQueue，使用

例子：

```
PhantomReference<String>prf=newPhantomReference<String>(new

String("str"),newReferenceQueue<>());
```

可用场景： 对象销毁前的一些操作，比如说资源释放等。** Object.finalize() 虽然也可以做这类动作，但是这个方式即不安全又低效

上诉所说的几类引用，都是指对象本身的引用，而不是指 Reference **的四个子类的引用**

(SoftReference 等)。

65、Java创建对象有几种方式？

java中提供了以下四种创建对象的方式:

1. new创建新对象
2. 通过反射机制
3. 采用clone机制
4. 通过序列化机制

66、有没有可能两个不相等的对象有相同的hashcode

有可能.在产生hash冲突时,两个不相等的对象就会有相同的 hashcode 值.当hash冲突产生时,一般有以

下几种方式来处理:

1. 拉链法:每个哈希表节点都有一个next指针,多个哈希表节点可以用next指针构成一个单向链表，被分配到同一个索引上的多个节点可以用这个单向链表进行存储.
2. 开放定址法:一旦发生了冲突,就去寻找下一个空的散列地址,只要散列表足够大,空的散列地址总能找到,并将记录存入
3. 再哈希:又叫双哈希法,有多个不同的Hash函数.当发生冲突时,使用第二个,第三个....等哈希函数计算地址,直到无冲突.

67、拷贝和浅拷贝的区别是什么?

浅拷贝:

被复制对象的所有变量都含有与原来的对象相同的值,而所有的对其他对象的引用仍然指向原来的对象.换言之,浅拷贝仅仅复制所考虑的对象,而不复制它所引用的对象.

深拷贝:

被复制对象的所有变量都含有与原来的对象相同的值.而那些引用其他对象的变量将指向被复制过的新对象.而不再是原有的那些被引用的对象.换言之.深拷贝把要复制的对象所引用的对象都复制了一遍.

68、static都有哪些用法?

所有的人都知道static关键字这两个基本的用法:静态变量和静态方法.也就是被static所修饰的变量/方法都属于类的静态资源,类实例所共享.

除了静态变量和静态方法之外,static也用于静态块,多用于初始化操作：

```
public calss PreCache{

    static{

    //执行相关操作

    }

}
```

此外static也多用于修饰内部类,此时称之为静态内部类.

最后一种用法就是静态导包,即 import static .import static是在JDK 1.5之后引入的新特性,可以用来指定导入某个类中的静态资源,并且不需要使用类名,可以直接使用资源名,比如:

```
import static java.lang.Math.*;

public class Test{

    public static void main(String[] args){

        //System.out.println(Math.sin(20));传统做法

        System.out.println(sin(20));

    }

}
```

69、a=a+b与a+=b有什么区别吗?

+= 操作符会进行隐式自动类型转换,此处a+=b隐式的将加操作的结果类型强制转换为持有结果的类型, 而a=a+b则不会自动进行类型转换.

如：

```
byte a = 127;

byte b = 127;

b = a + b; // 报编译错误:cannot convert from int to byte

b += a;
```

以下代码是否有错,有的话怎么改？

```
short s1= 1;

s1 = s1 + 1;
```

有错误.short类型在进行运算时会自动提升为int类型,也就是说 s1+1 的运算结果是int类型,而s1是short

类型,此时编译器会报错.

正确写法：

```
short s1= 1;

s1 += 1;
```

+=操作符会对右边的表达式结果强转匹配左边的数据类型,所以没错.

70、final、finalize()、finally

性质不同

1. final为关键字；
2. finalize()为方法；
3. finally为区块标志，用于try语句中；

作用

1. final为用于标识常量的关键字，final标识的关键字存储在常量池中（在这里final常量的具体用法将在下面进行介绍）；
2. finalize()方法在Object中进行了定义，用于在对象“消失”时，由JVM进行调用用于对对象进行垃圾回收，类似于C++中的析构函数；用户自定义时，用于释放对象占用的资源（比如进行I/0操作）；
3. finally{}用于标识代码块，与try{}进行配合，不论try中的代码执行完或没有执行完（这里指有异常），该代码块之中的程序必定会进行；

71、JDBC操作的步骤

加载数据库驱动类
打开数据库连接
执行sql语句
处理返回结果
关闭资源

72、在使用jdbc的时候，如何防止出现sql注入的问题。

使用PreparedStatement类，而不是使用Statement类

73、怎么在JDBC内调用一个存储过程

使用CallableStatement

74、是否了解连接池，使用连接池有什么好处？

数据库连接是非常消耗资源的，影响到程序的性能指标。连接池是用来分配、管理、释放数据库连接的，可以使应用程序重复使用同一个数据库连接，而不是每次都创建一个新的数据库连接。通过释放空闲时间较长的数据库连接避免数据库因为创建太多的连接而造成的连接遗漏问题，提高了程序性能。

75、你所了解的数据源技术有那些？使用数据源有什么好处？

Dbcp,c3p0等，用的最多还是c3p0，因为c3p0比dbcp更加稳定，安全；通过配置文件的形式来维护数据库信息，而不是通过硬编码。当连接的数据库信息发生改变时，不需要再更改程序代码就实现了数据库信息的更新。

76、&和&&的区别

&是位运算符。&&是布尔逻辑运算符，在进行逻辑判断时用&处理的前面为false后面的内容仍需处理，用&&处理的前面为false不再处理后面的内容。

77、静态内部类如何定义

定义在类内部的静态类，就是静态内部类。

```
public class Out {

     private static int a;

     private int b;
```

```
    public static class Inner {

        public void print() {

        System.out.println(a);

        }

    }

}
```

1. 静态内部类可以访问外部类所有的静态变量和方法，即使是 private 的也一样。
2. 静态内部类和一般类一致，可以定义静态变量、方法，构造方法等。
3. 其它类使用静态内部类需要使用“外部类.静态内部类”方式，如下所示：Out.Inner inner = new Out.Inner();inner.print();
4. Java集合类HashMap内部就有一个静态内部类Entry。Entry是HashMap存放元素的抽象，HashMap 内部维护 Entry 数组用了存放元素，但是 Entry 对使用者是透明的。像这种和外部类关系密切的，且不依赖外部类实例的，都可以使用静态内部类。

78、什么是成员内部类

定义在类内部的非静态类，就是成员内部类。成员内部类不能定义静态方法和变量（final修饰的除外）。这是因为成员内部类是非静态的，类初始化的时候先初始化静态成员，如果允许成员内部类定义静态变量，那么成员内部类的静态变量初始化顺序是有歧义的。实例：

```
public class Out {

    private static int a;

    private int b;

    public class Inner {

        public void print() {

            System.out.println(a);

            System.out.println(b);

        }

    }

}
```

79、Static Nested Class 和 Inner Class的不同

Nested Class（一般是C++的说法），Inner Class (一般是JAVA的说法)。Java内部类与C++嵌套类最大的不同就在于是否有指向外部的引用上。注： 静态内部类（Inner Class）意味着1创建一个static内部类的对象，不需要一个外部类对象，2不能从一个static内部类的一个对象访问一个外部类对象

80、什么时候用assert

assertion(断言)在软件开发中是一种常用的调试方式，很多开发语言中都支持这种机制。在实现中，assertion就是在程序中的一条语句，它对一个boolean表达式进行检查，一个正确程序必须保证这个boolean表达式的值为true；如果该值为false，说明程序已经处于不正确的状态下，系统将给出警告或退出。一般来说，assertion用于保证程序最基本、关键的正确性。assertion检查通常在开发和测试时开启。为了提高性能，在软件发布后，assertion检查通常是关闭的

81、Java有没有goto

java中的保留字，现在没有在java中使用

82、数组有没有length()这个方法? String有没有length()这个方法

数组没有length()这个方法，有length的属性。String有有length()这个方法

83、用最有效率的方法算出2乘以8等於几

2 << 3

84、float型float f=3.4是否正确?

不正确。精度不准确,应该用强制类型转换，如下所示：float f=(float)3.4

85、排序都有哪几种方法？请列举

排序的方法有：插入排序（直接插入排序、希尔排序），交换排序（冒泡排序、快速排序），选择排序（直接选择排序、堆排序），归并排序，分配排序（箱排序、基数排序）快速排序的伪代码。/ /使用快速排序方法对a[0 :n- 1]排序从a[0 :n- 1]中选择一个元素作为m i d d l e，该元素为支点把余下的元素分割为两段left 和r i g h t，使得l e f t中的元素都小于等于支点，而right 中的元素都大于等于支点递归地使用快速排序方法对left 进行排序递归地使用快速排序方法对right 进行排序所得结果为l e f t + m i d d l e + r i g h t

86、静态变量和实例变量的区别？

static i = 10; //常量 class A a; a.i =10;//可变

87、说出一些常用的类，包，接口，请各举5个

常用的类：BufferedReader BufferedWriter FileReader FileWirter String Integer常用的包：java.lang java.awt java.io java.util java.sql常用的接口：Remote List Map Document NodeList

88、a.hashCode() 有什么用？与 a.equals(b) 有什么关系？

hashCode() 方法是相应对象整型的 hash 值。它常用于基于 hash 的集合类，如 Hashtable、HashMap、LinkedHashMap 等等。它与 equals() 方法关系特别紧密。根据 Java 规范，两个使用 equal() 方法来判断相等的对象，必须具有相同的 hash code。

89、Java 中的编译期常量是什么？使用它又什么风险？

公共静态不可变（public static final ）变量也就是我们所说的编译期常量，这里的 public 可选的。实际上这些变量在编译时会被替换掉，因为编译器知道这些变量的值，并且知道这些变量在运行时不能改变。这种方式存在的一个问题是你使用了一个内部的或第三方库中的公有编译时常量，但是这个值后面被其他人改变了，但是你的客户端仍然在使用老的值，甚至你已经部署了一个新的 jar。为了避免这种情况，当你在更新依赖 JAR 文件时，确保重新编译你的程序

90、在 Java 中，如何跳出当前的多重嵌套循环？

在最外层循环前加一个标记如 A，然后用 break A;可以跳出多重循环。（Java 中支持带标签的 break 和 continue 语句，作用有点类似于 C 和 C++中的 goto 语句，但是就像要避免使用 goto 一样，应该避免使用带标签的 break 和 continue，因为它不会让你的程序变得更优雅，很多时候甚至有相反的作用，所以这种语法其实不知道更好）

91、构造器（constructor）是否可被重写（override）？

构造器不能被继承，因此不能被重写，但可以被重载。

92、两个对象值相同(x.equals(y) == true)，但却可有不同的hash code，这句话对不对？

不对，如果两个对象 x 和 y 满足 x.equals(y) == true，它们的哈希码（hash code）应当相同。Java 对于 eqauls 方法和 hashCode 方法是这样规定的：

(1)如果两个 对象相同（equals 方法返回 true），那么它们的 hashCode 值一定要相同；

(2)如果两个对象的 hashCode 相同，它们并不一定相同。当然，你未必要按照要求去做，但是如果你违背了上述原则就会发现在使用容器时，相同的对象可以出现在 Set 集合中，同时增加新元素的效率会大大下降（对于使用哈希存储的系统，如果哈希码频繁的冲突将会造成存取性能急剧下降）。

93、是否可以继承 String 类？

String 类是 final 类，不可以被继承，继承 String 本身就是一个错误的行为，对 String 类型最好的重用方式是关联关系（Has-A）和依赖关系（Use-A）而不是继承关系（Is-A）。

94、当一个对象被当作参数传递到一个方法后，此方法可改变这个对象的属性，并可返回变化后的结果，那么这里到底是值传递还是引用传递？

是值传递。Java 语言的方法调用只支持参数的值传递。当一个对象实例作为一个参数被传递到方法中时，参数的值就是对该对象的引用。对象的属性可以在被调用过程中被改变，但对对象引用的改变是不会影响到调用者的。C++和 C#中可以通过传引用或传输出参数来改变传入的参数的值。在 C#中可以编写如下所示的代码，但是在 Java 中却做不到。

```
using System;
namespace CS01 {
    class Program {
        public static void swap(ref int x, ref int y) {
```

```
            int temp = x;
            x = y;
            y = temp;
        } p
        ublic static void Main (string[] args) {
            int a = 5, b = 10;
            swap (ref a, ref b);
// a = 10, b = 5;
            Console.WriteLine ("a = {0}, b = {1}", a, b);
        }
    }
}
```

说明：Java 中没有传引用实在是非常的不方便，这一点在 Java 8 中仍然没有得到改进，正是如此在 Java 编写的代码中才会出现大量的 Wrapper 类（将需要通过方法调用修改的引用置于一个 Wrapper 类中，再将 Wrapper 对象传入方法），这样的做法只会让代码变得臃肿，尤其是让从 C 和 C++转型为 Java 程序员的开发者无法容忍。

95、String 和 StringBuilder、StringBuffer 的区别？

Java 平台提供了两种类型的字符串：String 和 StringBuffer/StringBuilder，它们可以储存和操作字符串。其中 String 是只读字符串，也就意味着 String 引用的字符串内容是不能被改变的。而 StringBuffer/StringBuilder 类表示的字符串对象可以直接进行修改。StringBuilder 是 Java 5 中引入的，它和 StringBuffer 的方法完全相同，区别在于它是在单线程环境下使用的，因为它的所有方面都没有被synchronized 修饰，因此它的效率也比 StringBuffer 要高。

96、重载（Overload）和重写（Override）的区别。重载的方法能否根据返回类型进行区分？

方法的重载和重写都是实现多态的方式，区别在于前者实现的是编译时的多态性，而后者实现的是运行时的多态性。重载发生在一个类中，同名的方法如果有不同的参数列表（参数类型不同、参数个数不同或者二者都不同）则视为重载；重写发生在子类与父类之间，重写要求子类被重写方法与父类被重写方法有相同的返回类型，比父类被重写方法更好访问，不能比父类被重写方法声明更多的异常（里氏代换原则）。重载对返回类型没有特殊的要求。

97、char 型变量中能不能存贮一个中文汉字，为什么？

char 类型可以存储一个中文汉字，因为 Java 中使用的编码是 Unicode（不选择任何特定的编码，直接使用字符在字符集中的编号，这是统一的唯一方法），一个 char 类型占 2 个字节（16 比特），所以放一个中文是没问题的。

补充：使用 Unicode 意味着字符在 JVM 内部和外部有不同的表现形式，在 JVM内部都是 Unicode，当这个字符被从 JVM 内部转移到外部时（例如存入文件系统中），需要进行编码转换。所以 Java 中有字节流和字符流，以及在字符流和字节流之间进行转换的转换流，如 InputStreamReader 和 OutputStreamReader，这两个类是字节流和字符流之间的适配器类，承担了编码转换的任务；对于 C 程序员来说，要完成这样的编码转换恐怕要依赖于 union（联合体/共用体）共享内存的特征来实现了。

98、抽象类（abstract class）和接口（interface）有什么异同 ？

抽象类和接口都不能够实例化，但可以定义抽象类和接口类型的引用。一个类如果继承了某个抽象类或者实现了某个接口都需要对其中的抽象方法全部进行实现，否则该类仍然需要被声明为抽象类。接口比抽象类更加抽象，因为抽象类中可以定义构造器，可以有抽象方法和具体方法，而接口中不能定义构造器而且其中的方法全部都是抽象方法。抽象类中的成员可以是 private、默认、protected、public 的，而接口中的成员全都是 public 的。抽象类中可以定义成员变量，而接口中定义的成员变量实际上都是常量。有抽象方法的类必须被声明为抽象类，而抽象类未必要有抽象方法。

99、静态嵌套类(Static Nested Class)和内部类（Inner Class）的不同？

Static Nested Class 是被声明为静态（static）的内部类，它可以不依赖于外部类实例被实例化。而通常的内部类需要在外部类实例化后才能实例化，其语法看起来挺诡异的，如下所示

```
/**
* 扑克类（一副扑克）
* @author 骆昊
*/
public class Poker {
    private static String[] suites = {"黑桃", "红桃", "草花", "方块"};
    private static int[] faces = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13};
    private Card[] cards;
    /**
    * 构造器
    * */
    public Poker() {
        cards = new Card[52];
        for(int i = 0; i < suites.length; i++) {
            for(int j = 0; j < faces.length; j++) {
                cards[i * 13 + j] = new Card(suites[i], faces[j]);
            }
        }
```

```
    }
    /**
    * 洗牌 （随机乱序）
    * */
    public void shuffle() {
        for(int i = 0, len = cards.length; i < len; i++) {
            int index = (int) (Math.random() * len);
            Card temp = cards[index];
            cards[index] = cards[i];
            cards[i] = temp;
        }
    }
    /**
    * 发牌
    * @param index 发牌的位置
    * */
    public Card deal(int index) {
        return cards[index];
    }
    /**
    * 卡片类（一张扑克）
    * [内部类]
    * @author 骆昊
    * */
    public class Card {
        private String suite; // 花色
        private int face; // 点数
        public Card(String suite, int face) {
            this.suite = suite;
            this.face = face;
        }
        @Override
        public String toString() {
            String faceStr = "";
            switch(face) {
                case 1: faceStr = "A"; break;
                case 11: faceStr = "J"; break;
                case 12: faceStr = "Q"; break;
                case 13: faceStr = "K"; break;
                default: faceStr = String.valueOf(face);
            } return suite + faceStr;
        }
    }
}

//测试类
class PokerTest {
    public static void main(String[] args) {
        Poker poker = new Poker();
        poker.shuffle(); // 洗牌
        Poker.Card c1 = poker.deal(0); // 发第一张牌
        // 对于非静态内部类 Card
        // 只有通过其外部类 Poker 对象才能创建 Card 对象
        Poker.Card c2 = poker.new Card("红心", 1); // 自己创建一张牌
        System.out.println(c1); // 洗牌后的第一张
        System.out.println(c2); // 打印: 红心 A
    }
}
```

100、Java 中会存在内存泄漏吗，请简单描述。

理论上Java因为有垃圾回收机制（GC）不会存在内存泄露问题（这也是Java被广泛使用于服务器端编程的一个重要原因）；然而在实际开发中，可能会存在无用但可达的对象，这些对象不能被GC回收，因此也会导致内存泄露的发生。

例如Hibernate的Session（一级缓存）中的对象属于持久态，垃圾回收器是不会回收这些对象的，然而这些对象中可能存在无用的垃圾对象，如果不及时关闭（close）或清空（flush）一级缓存就可能导致内存泄露。下面例子中的代码也会导致内存泄露

```
import java.util.Arrays;
import java.util.EmptyStackException;
public class MyStack<T> {
    private T[] elements;
    private int size = 0;
    private static final int INIT_CAPACITY = 16;
    public MyStack() {
        elements = (T[]) new Object[INIT_CAPACITY];
    }
    public void push(T elem) {
        ensureCapacity();
        elements[size++] = elem;
    }
    public T pop() {
```

```
        if(size == 0)
            throw new EmptyStackException();
        return elements[--size];
    }
    private void ensureCapacity() {
        if(elements.length == size) {
            elements = Arrays.copyOf(elements, 2 * size + 1);
        }
    }
}
```

上面的代码实现了一个栈（先进后出（FILO））结构，乍看之下似乎没有什么明显的问题，它甚至可以通过你编写的各种单元测试。然而其中的 pop 方法却存在内存泄露的问题，当我们用 pop 方法弹出栈中的对象时，该对象不会被当作垃圾回收，即使使用栈的程序不再引用这些对象，因为栈内部维护着对这些对象的过期引用（obsolete reference）。在支持垃圾回收的语言中，内存泄露是很隐蔽的，这种内存泄露其实就是无意识的对象保持。如果一个对象引用被无意识的保留起来了，那么垃圾回收器不会处理这个对象，也不会处理该对象引用的其他对象，即使这样的对象只有少数几个，也可能会导致很多的对象被排除在垃圾回收之外，从而对性能造成重大影响，极端情况下会引发 Disk Paging（物理内存与硬盘的虚拟内存交换数据），甚至造成 OutOfMemoryError。

101、抽象的（abstract）方法是否可同时是静态的（static）,是否可同时是本地方法（native），是否可同时被 synchronized修饰？

都不能。抽象方法需要子类重写，而静态的方法是无法被重写的，因此二者是矛盾的。本地方法是由本地代码（如 C 代码）实现的方法，而抽象方法是没有实现的，也是矛盾的。synchronized 和方法的实现细节有关，抽象方法不涉及实现细节，因此也是相互矛盾的。

102、是否可以从一个静态（static）方法内部发出对非静态（non-static）方法的调用？

不可以，静态方法只能访问静态成员，因为非静态方法的调用要先创建对象，在调用静态方法时可能对象并没有被初始化。

103、如何实现对象克隆？

有两种方式：
1). 实现 Cloneable 接口并重写 Object 类中的 clone()方法；
2). 实现 Serializable 接口，通过对象的序列化和反序列化实现克隆，可以实现真正的深度克隆，代码如下。

```
import java.io.ByteArrayInputStream;
import java.io.ByteArrayOutputStream;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.Serializable;
public class MyUtil {
    private MyUtil() {
        throw new AssertionError();
    }
    @SuppressWarnings("unchecked")
    public static <T extends Serializable> T clone(T obj) throws
    Exception {
        ByteArrayOutputStream bout = new ByteArrayOutputStream();
        ObjectOutputStream oos = new ObjectOutputStream(bout);
        oos.writeObject(obj);
        ByteArrayInputStream bin = new
        ByteArrayInputStream(bout.toByteArray());
        ObjectInputStream ois = new ObjectInputStream(bin);
        return (T) ois.readObject();
// 说明：调用 ByteArrayInputStream 或 ByteArrayOutputStream对象的 close 方法没有任何意义
// 这两个基于内存的流只要垃圾回收器清理对象就能够释放资源，这一点不同于对外部资源（如文件流）的释放
    }
}
```

测试代码：

```
import java.io.Serializable;
class Person implements Serializable {
    private static final long serialVersionUID = -9102017020286042305L;
    private String name; // 姓名
    private int age; // 年龄
    private Car car; // 座驾
    public Person(String name, int age, Car car) {
        this.name = name;
        this.age = age;
        this.car = car;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
```

```
    }
    public int getAge() {
        return age;
    }
    public void setAge(int age) {
        this.age = age;
    }
    public Car getCar() {
        return car;
    }
    public void setCar(Car car) {
        this.car = car;
    }
    @Override
    public String toString() {
        return "Person [name=" + name + ", age=" + age + ", car=" +
        car + "]";
    }
}
```

```
class Car implements Serializable {
    private static final long serialVersionUID = -5713945027627603702L;
    private String brand; // 品牌
    private int maxSpeed; // 最高时速
    public Car(String brand, int maxSpeed) {
        this.brand = brand;
        this.maxSpeed = maxSpeed;
    }
    public String getBrand() {
        return brand;
    }
    public void setBrand(String brand) {
        this.brand = brand;
    }
    public int getMaxSpeed() {
        return maxSpeed;
    }
    public void setMaxSpeed(int maxSpeed) {
        this.maxSpeed = maxSpeed;
    }
    @Override
    public String toString() {
        return "Car [brand=" + brand + ", maxSpeed=" + maxSpeed +
        "]";
    }
}
class CloneTest {
    public static void main(String[] args) {
        try {
                Person p1 = new Person("Hao LUO", 33, new Car("Benz",
                    300));
                Person p2 = MyUtil.clone(p1); // 深度克隆
                p2.getCar().setBrand("BYD");
                // 修改克隆的 Person 对象 p2 关联的汽车对象的品牌属性
                // 原来的 Person 对象 p1 关联的汽车不会受到任何影响
                // 因为在克隆 Person 对象时其关联的汽车对象也被克隆了
                System.out.println(p1);
            } catch (Exception e) {
                e.printStackTrace();
            }
    }
}
```

注意：基于序列化和反序列化实现的克隆不仅仅是深度克隆，更重要的是通过泛型限定，可以检查出要克隆的对象是否支持序列化，这项检查是编译器完成的，不是在运行时抛出异常，这种是方案明显优于使用 Object 类的 clone 方法克隆对象。让问题在编译的时候暴露出来总是好过把问题留到运行时。

104、接口是否可继承（extends）接口？抽象类是否可实现（implements）接口？抽象类是否可继承具体类（concreteclass）？

接口可以继承接口，而且支持多重继承。抽象类可以实现(implements)接口，抽象类可继承具体类也可以继承抽象类。

105、一个".java"源文件中是否可以包含多个类（不是内部类）？有什么限制？

可以，但一个源文件中最多只能有一个公开类（public class）而且文件名必须和公开类的类名完全保持一致。

106、Anonymous Inner Class(匿名内部类)是否可以继承其它类？是否可以实现接口？

可以继承其他类或实现其他接口，在 Swing 编程和 Android 开发中常用此方式来实现事件监听和回调。

107、内部类可以引用它的包含类（外部类）的成员吗？有没有什么限制？

一个内部类对象可以访问创建它的外部类对象的成员，包括私有成员。

108、Java 中的 final 关键字有哪些用法？

(1)修饰类：表示该类不能被继承；

(2)修饰方法：表示方法不能被重写；

(3)修饰变量：表示变量只能一次赋值以后值不能被修改（常量）。

Java集合/泛型面试题

1、ArrayList和linkedList的区别

Array(数组）是基于索引**(index)**的数据结构，它使用索引在数组中搜索和读取数据是很快的。

Array获取数据的时间复杂度是O(1),但是要删除数据却是开销很大，因为这需要重排数组中的所有数据,

(因为删除数据以后, 需要把后面所有的数据前移)

缺点: 数组初始化必须指定初始化的长度, 否则报错

例如:

```
int[] a = new int[4];//推荐使用int[] 这种方式初始化

int c[] = {23,43,56,78};//长度：4，索引范围：[0,3]
```

List—是一个有序的集合，可以包含重复的元素，提供了按索引访问的方式，它继承Collection。

List有两个重要的实现类：ArrayList和LinkedList

ArrayList: 可以看作是能够自动增长容量的数组

ArrayList的toArray方法返回一个数组

ArrayList的asList方法返回一个列表

ArrayList底层的实现是Array, 数组扩容实现

LinkList是一个双链表,在添加和删除元素时具有比ArrayList更好的性能.但在get与set方面弱于

ArrayList.当然,这些对比都是指数据量很大或者操作很频繁。

2、 HashMap和HashTable的区别

1、两者父类不同

HashMap是继承自AbstractMap类，而Hashtable是继承自Dictionary类。不过它们都实现了同时实现

了map、Cloneable（可复制）、Serializable（可序列化）这三个接口。

2、对外提供的接口不同

Hashtable比HashMap多提供了elments() 和contains() 两个方法。

elments() 方法继承自Hashtable的父类Dictionnary。elements() 方法用于返回此Hashtable中的

value的枚举。

contains()方法判断该Hashtable是否包含传入的value。它的作用与containsValue()一致。事实上，contansValue() 就只是调用了一下contains() 方法。

3、对null的支持不同

Hashtable：key和value都不能为null。

HashMap：key可以为null，但是这样的key只能有一个，因为必须保证key的唯一性；可以有多个key值对应的value为null。

4、安全性不同

HashMap是线程不安全的，在多线程并发的环境下，可能会产生死锁等问题，因此需要开发人员自己处理多线程的安全问题。

Hashtable是线程安全的，它的每个方法上都有synchronized 关键字，因此可直接用于多线程中。

虽然HashMap是线程不安全的，但是它的效率远远高于Hashtable，这样设计是合理的，因为大部分的使用场景都是单线程。当需要多线程操作的时候可以使用线程安全的ConcurrentHashMap。

ConcurrentHashMap虽然也是线程安全的，但是它的效率比Hashtable要高好多倍。因为ConcurrentHashMap使用了分段锁，并不对整个数据进行锁定。

5、初始容量大小和每次扩充容量大小不同

6、计算hash值的方法不同

3、Collection包结构，与Collections的区别

Collection是集合类的上级接口，子接口有 Set、List、LinkedList、ArrayList、Vector、Stack、Set；

Collections是集合类的一个帮助类， 它包含有各种有关集合操作的静态多态方法，用于实现对各种集合的搜索、排序、线程安全化等操作。此类不能实例化，就像一个工具类，服务于Java的Collection框架。

4、泛型常用特点（待补充）

泛型是Java SE 1.5之后的特性，《Java 核心技术》中对泛型的定义是：

> “泛型”意味着编写的代码可以被不同类型的对象所重用。

“泛型”，顾名思义，“泛指的类型”。我们提供了泛指的概念，但具体执行的时候却可以有具体的规则来约束，比如我们用的非常多的ArrayList就是个泛型类，ArrayList作为集合可以存放各种元素，如Integer, String，自定义的各种类型等，但在我们使用的时候通过具体的规则来约束，如我们可以约束集合中只存放Integer类型的元素，如

```
List<Integer> iniData = new ArrayList<>()
```

使用泛型的好处？

以集合来举例，使用泛型的好处是我们不必因为添加元素类型的不同而定义不同类型的集合，如整型集合类，浮点型集合类，字符串集合类，我们可以定义一个集合来存放整型、浮点型，字符串型数据，而这并不是最重要的，因为我们只要把底层存储设置了Object即可，添加的数据全部都可向上转型为Object。 更重要的是我们可以通过规则按照自己的想法控制存储的数据类型。

5、说说List,Set,Map三者的区别

List(对付顺序的好帮手)： List接口存储一组不唯一（可以有多个元素引用相同的对象），有序的对象

Set(注重独一无二的性质):不允许重复的集合。不会有多个元素引用相同的对象。

Map(用Key来搜索的专): 使用键值对存储。Map会维护与Key有关联的值。两个Key可以引用相同的对象，但Key不能重复，典型的Key是String类型，但也可以是任何对象。

6、Array与ArrayList有什么不一样？

Array与ArrayList都是用来存储数据的集合。ArrayList底层是使用数组实现的，但是arrayList对数组进行了封装和功能扩展，拥有许多原生数组没有的一些功能。我们可以理解成ArrayList是Array的一个升级版。

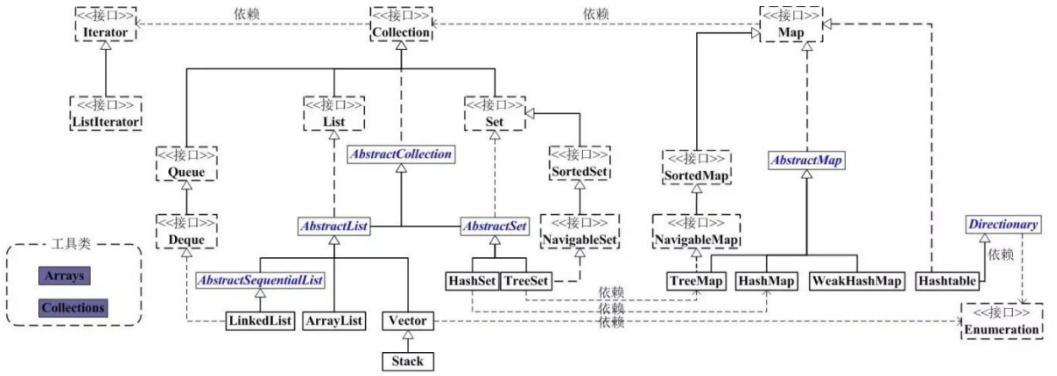
7、Map有什么特点

以键值对存储数据
元素存储循序是无序的
不允许出现重复键

8、集合类存放于 Java.util 包中， 主要有几 种接口

主要包含set(集）、 list(列表包含 Queue ）和 map(映射)。

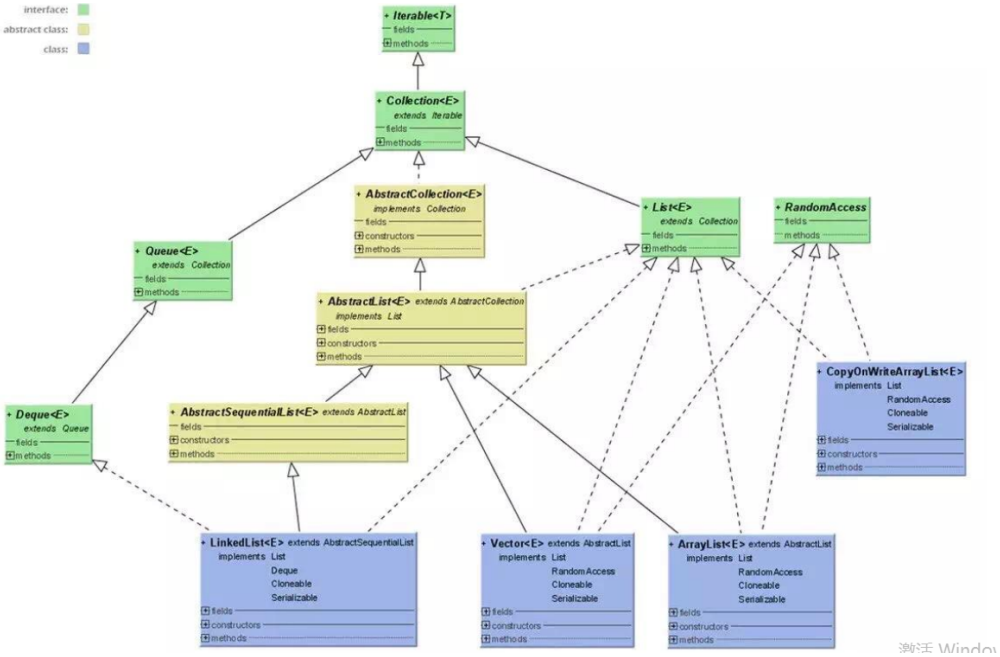
1. Collection： Collection 是集合 List、 Set、 Queue 的最基本的接口。
2. Iterator：迭代器，可以通过迭代器遍历集合中的数据
3. Map：是映射表的基础接口



9、什么是list接口

Java 的 List 是非常常用的数据类型。 List 是有序的 Collection。 Java List 一共三个实现类：
分别是 ArrayList、 Vector 和 LinkedList 。

list接口结构图



10、说说ArrayList（数组）

ArrayList 是最常用的 List 实现类，内部是通过数组实现的，它允许对元素进行快速随机访问。数组的缺点是每个元素之间不能有间隔， 当数组大小不满足时需要增加存储能力，就要将已经有数组的数据复制到新的存储空间中。 当从 ArrayList 的中间位置插入或者删除元素时，需要对数组进行复制、移动、代价比较高。因此，它适合随机查找和遍历，不适合插入和删除。

11、Vector（ 数组实现、 线程同步）

Vector 与 ArrayList 一样，也是通过数组实现的，不同的是它支持线程的同步，即某一时刻只有一个线程能够写 Vector，避免多线程同时写而引起的不一致性，但实现同步需要很高的花费，因此，访问它比访问 ArrayList 慢 。

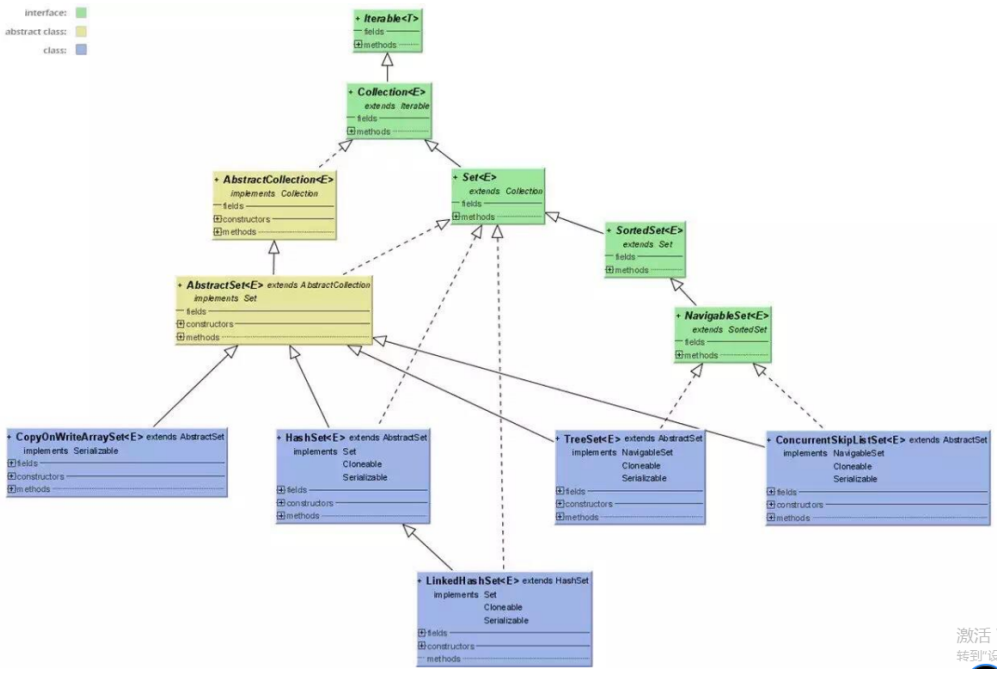
12、说说LinkList（链表）

LinkedList 是用链表结构存储数据的，很适合数据的动态插入和删除，随机访问和遍历速度比较慢。另外，他还提供了 List 接口中没有定义的方法，专门用于操作表头和表尾元素，可以当作堆栈、队列和双向队列使用

13、什么Set集合

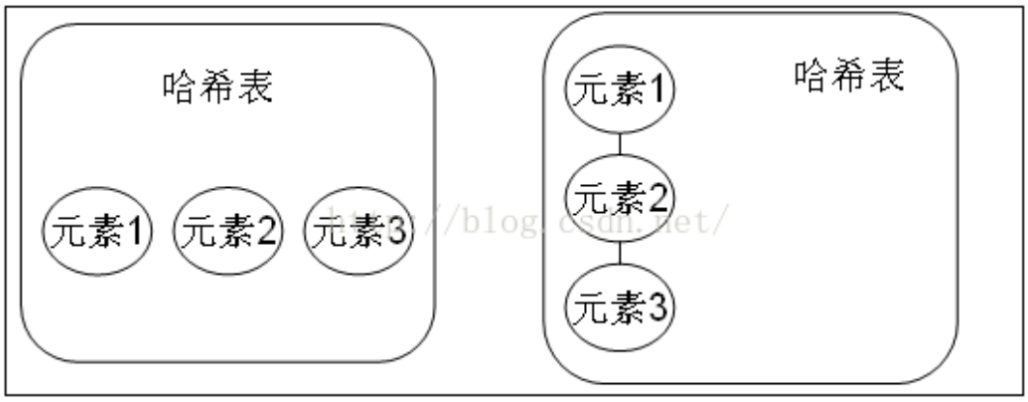
Set 注重独一无二的性质,该体系集合用于存储无序(存入和取出的顺序不一定相同)元素， 值不能重复。对象的相等性本质是对象 hashCode 值（java 是依据对象的内存地址计算出的此序号） 判断的， 如果想要让两个不同的对象视为相等的，就必须覆盖 Object 的 hashCode 方法和 equals 方法。

set结构结构图



14、HashSet（ Hash 表）

哈希表边存放的是哈希值。 HashSet 存储元素的顺序并不是按照存入时的顺序（和 List 显然不同）而是按照哈希值来存的所以取数据也是按照哈希值取得。元素的哈希值是通过元素的 hashcode 方法来获取的, HashSet 首先判断两个元素的哈希值，如果哈希值一样，接着会比较 equals 方法 如果 equls 结果为 true ， HashSet 就视为同一个元素。如果 equals 为 false 就不是同一个元素。
哈希值相同 equals 为 false 的元素是怎么存储呢,就是在同样的哈希值下顺延（可以认为哈希值相同的元素放在一个哈希桶中）。也就是哈希一样的存一列。 如图 1 表示 hashCode 值不相同的情况； 图 2 表示 hashCode 值相同，但 equals 不相同的情况。



HashSet 通过 hashCode 值来确定元素在内存中的位置。 一个 hashCode 位置上可以存放多个元素。

15、什么是TreeSet（二叉树）

1. TreeSet()是使用二叉树的原理对新 add()的对象按照指定的顺序排序（升序、降序），每增加一个对象都会进行排序，将对象插入的二叉树指定的位置。
2. Integer 和 String 对象都可以进行默认的 TreeSet 排序，而自定义类的对象是不可以的， 自己定义的类必须实现 Comparable 接口，并且覆写相应的 compareTo()函数，才可以正常使用。
3. 在覆写 compare()函数时，要返回相应的值才能使 TreeSet 按照一定的规则来排序
4. 比较此对象与指定对象的顺序。如果该对象小于、等于或大于指定对象，则分别返回负整数、零或正整数

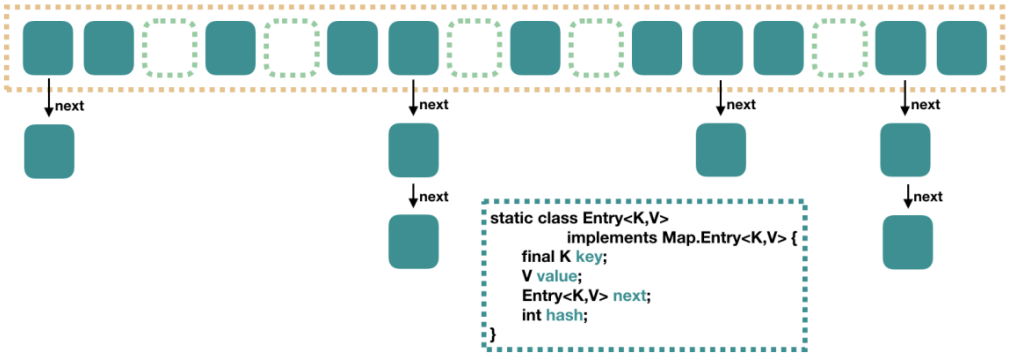
16、说说LinkHashSet（ HashSet+LinkedHashMap ）

对于 LinkedHashSet 而言，它继承与 HashSet、又基于 LinkedHashMap 来实现的。
LinkedHashSet 底层使用 LinkedHashMap 来保存所有元素，它继承与 HashSet，其所有的方法操作上又与 HashSet 相同，因此 LinkedHashSet 的实现上非常简单，只提供了四个构造方法，并通过传递一个标识参数，调用父类的构造器，底层构造一个 LinkedHashMap 来实现，在相关操作上与父类 HashSet 的操作相同，直接调用父类 HashSet 的方法即可。

17、HashMap（数组+链表+红黑树）

HashMap 根据键的 hashCode 值存储数据，大多数情况下可以直接定位到它的值，因而具有很快的访问速度，但遍历顺序却是不确定的。 HashMap 最多只允许一条记录的键为 null，允许多条记录的值为 null。 HashMap 非线程安全，即任一时刻可以有多个线程同时写 HashMap，可能会导致数据的不一致。如果需要满足线程安全，可以用 Collections 的 synchronizedMap 方法使 HashMap 具有线程安全的能力，或者使用 ConcurrentHashMap。 我们用下面这张图来介绍 HashMap 的结构。

Java7 HashMap 结构

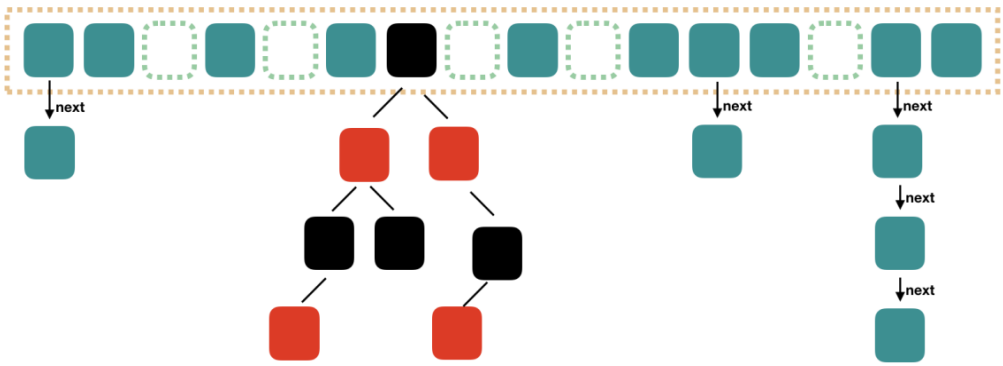


大方向上， HashMap 里面是一个数组，然后数组中每个元素是一个单向链表。上图中，每个绿色的实体是嵌套类 Entry 的实例， Entry 包含四个属性： key, value, hash 值和用于单向链表的 next。

1. capacity：当前数组容量，始终保持 2^n，可以扩容，扩容后数组大小为当前的 2 倍。
2. oadFactor：负载因子，默认为 0.75。
3. threshold：扩容的阈值，等于 capacity * loadFactor

Java8 对 HashMap 进行了一些修改， 最大的不同就是利用了红黑树，所以其由 数组+链表+红黑树 组成。
根据 Java7 HashMap 的介绍，我们知道，查找的时候，根据 hash 值我们能够快速定位到数组的具体下标，但是之后的话， 需要顺着链表一个个比较下去才能找到我们需要的，时间复杂度取决于链表的长度，为 O(n)。为了降低这部分的开销，在 Java8 中， 当链表中的元素超过了 8 个以后，会将链表转换为红黑树，在这些位置进行查找的时候可以降低时间复杂度为 O(logN)。

Java8 HashMap 结构



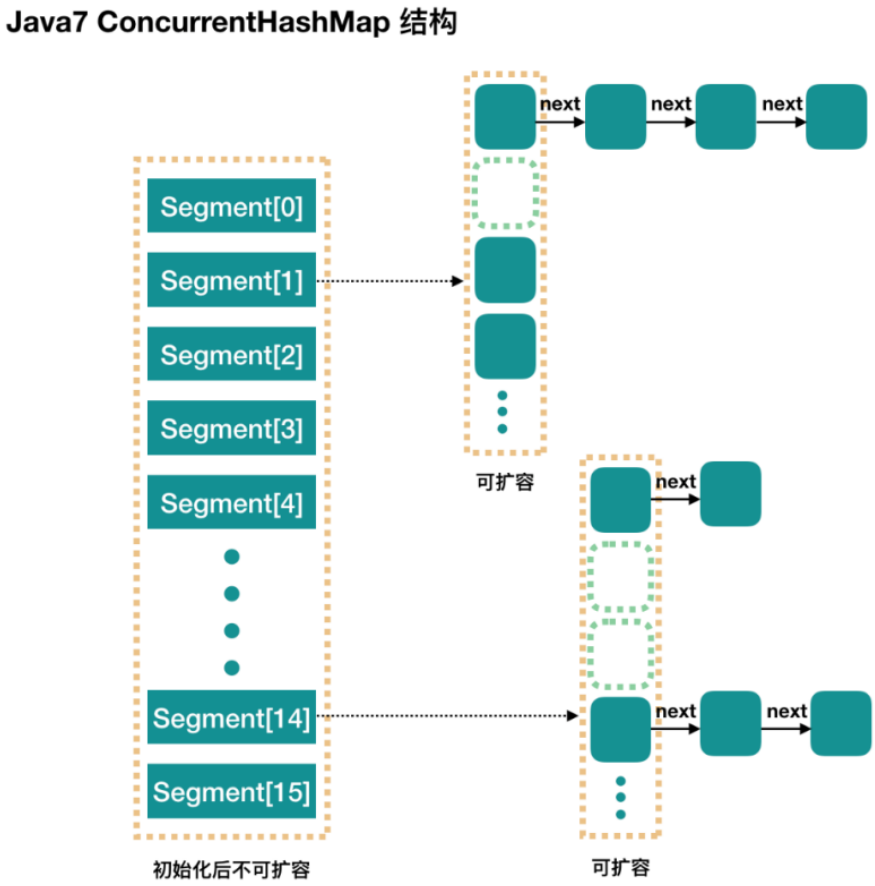
18、说说ConcurrentHashMap

Segment 段

ConcurrentHashMap 和 HashMap 思路是差不多的，但是因为它支持并发操作，所以要复杂一些。整个 ConcurrentHashMap 由一个个 Segment 组成， Segment 代表”部分“或”一段“的意思，所以很多地方都会将其描述为分段锁。注意，行文中，我很多地方用了”槽”来代表一个 segment。

线程安全（ Segment 继承 ReentrantLock 加锁 ）

简单理解就是， ConcurrentHashMap 是一个 Segment 数组， Segment 通过继承 ReentrantLock 来进行加锁，所以每次需要加锁的操作锁住的是一个 segment，这样只要保证每个 Segment 是线程安全的，也就实现了全局的线程安全

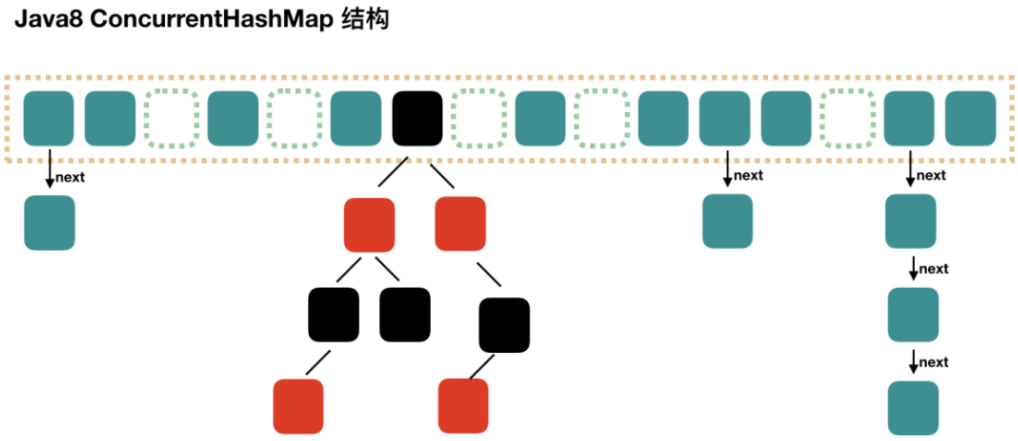


并行度（默认 16）

concurrencyLevel：并行级别、并发数、 Segment 数，怎么翻译不重要，理解它。默认是 16，也就是说 ConcurrentHashMap 有 16 个 Segments，所以理论上， 这个时候，最多可以同时支持 16 个线程并发写，只要它们的操作分别分布在不同的 Segment 上。这个值可以在初始化的时候设置为其他值，但是一旦初始化以后，它是不可以扩容的。再具体到每个 Segment 内部，其实每个 Segment 很像之前介绍的 HashMap，不过它要保证线程安全，所以处理起来要麻烦些。

Java8 实现 （引入了红黑树）

Java8 对 ConcurrentHashMap 进行了比较大的改动,Java8 也引入了红黑树。



19、HashTable（线程安全）

Hashtable 是遗留类，很多映射的常用功能与 HashMap 类似，不同的是它承自 Dictionary 类，并且是线程安全的，任一时间只有一个线程能写 Hashtable，并发性不如 ConcurrentHashMap，因为 ConcurrentHashMap 引入了分段锁。 Hashtable 不建议在新代码中使用，不需要线程安全的场合可以用 HashMap 替换，需要线程安全的场合可以用 ConcurrentHashMap 替换

20、TreeMap（可排序）

TreeMap 实现 SortedMap 接口，能够把它保存的记录根据键排序，默认是按键值的升序排序，
也可以指定排序的比较器，当用 Iterator 遍历 TreeMap 时，得到的记录是排过序的。
如果使用排序的映射，建议使用 TreeMap。
在使用 TreeMap 时， key 必须实现 Comparable 接口或者在构造 TreeMap 传入自定义的
Comparator，否则会在运行时抛出 java.lang.ClassCastException 类型的异常。
参考： https://www.ibm.com/developerworks/cn/java/j-lo-tree/index.html

21、LinkHashMap（记录插入顺序）

LinkedHashMap 是 HashMap 的一个子类，保存了记录的插入顺序，在用 Iterator 遍历
LinkedHashMap 时，先得到的记录肯定是先插入的，也可以在构造时带参数，按照访问次序排序。
参考 1： http://www.importnew.com/28263.html
参考 2： http://www.importnew.com/20386.html#comment-648123

22、泛型类

泛型类的声明和非泛型类的声明类似，除了在类名后面添加了类型参数声明部分。和泛型方法一样，泛型类的类型参数声明部分也包含一个或多个类型参数，参数间用逗号隔开。一个泛型参数，也被称为一个类型变量，是用于指定一个泛型类型名称的标识符。因为他们接受一个或多个参数，这些类被称为参数化的类或参数化的类型。

```
public class Box<T> {
    private T t;
    public void add(T t) {
        this.t = t;
    }
    public T get() {
        return t;
    }
}
```

23、类型通配符?

类型通配符一般是使用?代替具体的类型参数。例如 List<?> 在逻辑上是List,List 等所有 List<具体类型实参>的父类。

24、类型擦除

Java 中的泛型基本上都是在编译器这个层次来实现的。在生成的 Java 字节代码中是不包含泛型中的类型信息的。使用泛型的时候加上的类型参数，会被编译器在编译的时候去掉。这个过程就称为类型擦除。

如在代码中定义的 List和 List等类型，在编译之后都会变成 List。 JVM 看到的只是 List，而由泛型附加的类型信息对 JVM 来说是不可见的。

类型擦除的基本过程也比较简单，首先是找到用来替换类型参数的具体类。这个具体类一般是 Object。如果指定了类型参数的上界的话，则使用这个上界。把代码中的类型参数都替换成具体的类。

Java异常面试题

1、Java中异常分为哪两种？

编译时异常
运行时异常

2、异常的处理机制有几种？

异常捕捉：try...catch...finally，异常抛出：throws。

3、如何自定义一个异常

继承一个异常类，通常是RumtimeException或者Exception

4、try catch fifinally，try里有return，finally还执行么？

执行，并且finally的执行早于try里面的return

结论：

1、不管有木有出现异常，finally块中代码都会执行；

2、当try和catch中有return时，finally仍然会执行；

3、finally是在return后面的表达式运算后执行的（此时并没有返回运算后的值，而是先把要返回的值保存起来，管finally中的代码怎么样，返回的值都不会改变，任然是之前保存的值），所以函数返回值是在finally执行前确定的；

4、finally中最好不要包含return，否则程序会提前退出，返回值不是try或catch中保存的返回值。

5、 Excption与Error包结构

Java可抛出(Throwable)的结构分为三种类型：被检查的异常(CheckedException)，运行时异常

(RuntimeException)，错误(Error)。

1、运行时异常

定义:RuntimeException及其子类都被称为运行时异常。

特点:Java编译器不会检查它。也就是说，当程序中可能出现这类异常时，倘若既"没有通过throws声明抛出它"，也"没有用try-catch语句捕获它"，还是会编译通过。例如，除数为零时产生的ArithmeticException异常，数组越界时产生的IndexOutOfBoundsException异常，fail-fast机制产生的ConcurrentModi?cationException异常（java.util包下面的所有的集合类都是快速失败的，“快速失败”也就是fail-fast，它是Java集合的一种错误检测机制。当多个线程对集合进行结构上的改变的操作时，有可能会产生fail-fast机制。记住是有可能，而不是一定。例如：假设存在两个线程（线程1、线程

2），线程1通过Iterator在遍历集合A中的元素，在某个时候线程2修改了集合A的结构（是结构上面的修改，而不是简单的修改集合元素的内容），那么这个时候程序就会抛出ConcurrentModi?cationException 异常，从而产生fail-fast机制，这个错叫并发修改异常。Fail-safe，java.util.concurrent包下面的所有的类都是安全失败的，在遍历过程中，如果已经遍历的数组上的内容变化了，迭代器不会抛出ConcurrentModi?cationException异常。如果未遍历的数组上的内容发生了变化，则有可能反映到迭代过程中。这就是ConcurrentHashMap迭代器弱一致的表现。ConcurrentHashMap的弱一致性主要是为了提升效率，是一致性与效率之间的一种权衡。要成为强一致性，就得到处使用锁，甚至是全局锁，这就与Hashtable和同步的HashMap一样了。）等，都属于运行时异常。

常见的五种运行时异常：

ClassCastException（类转换异常）

IndexOutOfBoundsException（数组越界）

NullPointerException（空指针异常）

ArrayStoreException（数据存储异常，操作数组是类型不一致）

Bu?erOver?owException

2、被检查异常

定义:Exception类本身，以及Exception的子类中除了"运行时异常"之外的其它子类都属于被检查异常。特点：Java编译器会检查它。此类异常，要么通过throws进行声明抛出，要么通过try-catch进行捕获处理，否则不能通过编译。例如，CloneNotSupportedException就属于被检查异常。当通过clone()接口去克隆一个对象，而该对象对应的类没有实现Cloneable接口，就会抛出CloneNotSupportedException异常。被检查异常通常都是可以恢复的。

如：

IOException

FileNotFoundException

SQLException

被检查的异常适用于那些不是因程序引起的错误情况，比如：读取文件时文件不存在引发的FileNotFoundException 。然而，不被检查的异常通常都是由于糟糕的编程引起的，比如：在对象引用时没有确保对象非空而引起的 NullPointerException 。

3、错误

定义：Error类及其子类。

特点：和运行时异常一样，编译器也不会对错误进行检查。当资源不足、约束失败、或是其它程序无法继续运行的条件发生时，就产生错误。程序本身无法修复这些错误的。例如，VirtualMachineError就属于错误。出现这种错误会导致程序终止运行。OutOfMemoryError、ThreadDeath。

Java虚拟机规范规定JVM的内存分为了好几块，比如堆，栈，程序计数器，方法区等

6、Thow与thorws区别

位置不同

1. throws 用在函数上，后面跟的是异常类，可以跟多个；而 throw 用在函数内，后面跟的

是异常对象。

功能不同：

2. throws 用来声明异常，让调用者只知道该功能可能出现的问题，可以给出预先的处理方

式；throw 抛出具体的问题对象，执行到 throw，功能就已经结束了，跳转到调用者，并

将具体的问题对象抛给调用者。也就是说 throw 语句独立存在时，下面不要定义其他语

句，因为执行不到。

3. throws 表示出现异常的一种可能性，并不一定会发生这些异常；throw 则是抛出了异常，

执行 throw 则一定抛出了某种异常对象。

4. 两者都是消极处理异常的方式，只是抛出或者可能抛出异常，但是不会由函数去处理异

常，真正的处理异常由函数的上层调用处理。

7、Error与Exception区别？

Error和Exception都是java错误处理机制的一部分，都继承了Throwable类。

Exception表示的异常，异常可以通过程序来捕捉，或者优化程序来避免。

Error表示的是系统错误，不能通过程序来进行错误处理。

8、error和exception有什么区别

error 表示恢复不是不可能但很困难的情况下的一种严重问题。比如说内存溢出。不可能指望程序能处理这样的情况 exception 表示一种设计或实现问题。也就是说，它表示如果程序运行正常，从不会发生的情况

Java中的IO与NIO面试题

1、Java 中 IO 流？

Java 中 IO 流分为几种?

1. 按照流的流向分，可以分为输入流和输出流；
2. 按照操作单元划分，可以划分为字节流和字符流；
3. 按照流的角色划分为节点流和处理流。

Java Io 流共涉及 40 多个类，这些类看上去很杂乱，但实际上很有规则，而且彼此之间存在非常紧密的

联系， Java I0 流的 40 多个类都是从如下 4 个抽象类基类中派生出来的。

1. InputStream/Reader: 所有的输入流的基类，前者是字节输入流，后者是字符输入流。
2. OutputStream/Writer: 所有输出流的基类，前者是字节输出流，后者是字符输出流。

2、 Java IO与 NIO的区别

NIO即New IO，这个库是在JDK1.4中才引入的。NIO和IO有相同的作用和目的，但实现方式不同，NIO 主要用到的是块，所以NIO的效率要比IO高很多。在Java API中提供了两套NIO，一套是针对标准输入输出NIO，另一套就是网络编程NIO。

3、常用io类有那些

```
File
FileInputSteam，FileOutputStream
BufferInputStream，BufferedOutputSream
PrintWrite
FileReader，FileWriter
BufferReader，BufferedWriter
ObjectInputStream，ObjectOutputSream
```

4、字节流与字符流的区别

以字节为单位输入输出数据，字节流按照8位传输
以字符为单位输入输出数据，字符流按照16位传输

5、阻塞 IO 模型

最传统的一种 IO 模型，即在读写数据过程中会发生阻塞现象。当用户线程发出 IO 请求之后，内核会去查看数据是否就绪，如果没有就绪就会等待数据就绪，而用户线程就会处于阻塞状态，用户线程交出 CPU。当数据就绪之后，内核会将数据拷贝到用户线程，并返回结果给用户线程，用 户线程才解除 block 状态。典型的阻塞 IO 模型的例子为： data = socket.read();如果数据没有就绪，就会一直阻塞在 read 方法

6、非阻塞 IO 模型

当用户线程发起一个 read 操作后，并不需要等待，而是马上就得到了一个结果。 如果结果是一个error 时，它就知道数据还没有准备好，于是它可以再次发送 read 操作。一旦内核中的数据准备好了，并且又再次收到了用户线程的请求，那么它马上就将数据拷贝到了用户线程，然后返回。所以事实上，在非阻塞 IO 模型中，用户线程需要不断地询问内核数据是否就绪，也就说非阻塞 IO不会交出 CPU，而会一直占用 CPU。 典型的非阻塞 IO 模型一般如下：

```
while(true){
    data = socket.read();
    if(data!= error){
        //处理数据
        break;
    }
}
```

但是对于非阻塞 IO 就有一个非常严重的问题， 在 while 循环中需要不断地去询问内核数据是否就绪，这样会导致 CPU 占用率非常高，因此一般情况下很少使用 while 循环这种方式来读取数据。

7、多路复用 IO 模型

多路复用 IO 模型是目前使用得比较多的模型。 Java NIO 实际上就是多路复用 IO。在多路复用 IO模型中，会有一个线程不断去轮询多个 socket 的状态，只有当 socket 真正有读写事件时，才真正调用实际的 IO 读写操作。因为在多路复用 IO 模型中，只需要使用一个线程就可以管理多个socket，系统不需要建立新的进程或者线程，也不必维护这些线程和进程，并且只有在真正有socket 读写事件进行时，才会使用 IO 资源，所以它大大减少了资源占用。在 Java NIO 中，是通过 selector.select()去查询每个通道是否有到达事件，如果没有事件，则一直阻塞在那里，因此这种方式会导致用户线程的阻塞。多路复用 IO 模式，通过一个线程就可以管理多个 socket，只有当 socket 真正有读写事件发生才会占用资源来进行实际的读写操作。因此，多路复用 IO 比较适合连接数比较多的情况。

另外多路复用 IO 为何比非阻塞 IO 模型的效率高是因为在非阻塞 IO 中，不断地询问 socket 状态时通过用户线程去进行的，而在多路复用 IO 中，轮询每个 socket 状态是内核在进行的，这个效率要比用户线程要高的多。

不过要注意的是，多路复用 IO 模型是通过轮询的方式来检测是否有事件到达，并且对到达的事件逐一进行响应。因此对于多路复用 IO 模型来说， 一旦事件响应体很大，那么就会导致后续的事件迟迟得不到处理，并且会影响新的事件轮询。

8、信号驱动 IO 模型

在信号驱动 IO 模型中，当用户线程发起一个 IO 请求操作，会给对应的 socket 注册一个信号函数，然后用户线程会继续执行，当内核数据就绪时会发送一个信号给用户线程，用户线程接收到信号之后，便在信号函数中调用 IO 读写操作来进行实际的 IO 请求操作。

9、异步 IO 模型

异步 IO 模型才是最理想的 IO 模型，在异步 IO 模型中，当用户线程发起 read 操作之后，立刻就可以开始去做其它的事。而另一方面，从内核的角度，当它受到一个 asynchronous read 之后，它会立刻返回，说明 read 请求已经成功发起了，因此不会对用户线程产生任何 block。然后，内核会等待数据准备完成，然后将数据拷贝到用户线程，当这一切都完成之后，内核会给用户线程发送一个信号，告诉它 read 操作完成了。也就说用户线程完全不需要实际的整个 IO 操作是如何进行的， 只需要先发起一个请求，当接收内核返回的成功信号时表示 IO 操作已经完成，可以直接去使用数据了。

也就说在异步 IO 模型中， IO 操作的两个阶段都不会阻塞用户线程，这两个阶段都是由内核自动完成，然后发送一个信号告知用户线程操作已完成。用户线程中不需要再次调用 IO 函数进行具体的读写。这点是和信号驱动模型有所不同的，在信号驱动模型中，当用户线程接收到信号表示数据已经就绪，然后需要用户线程调用 IO 函数进行实际的读写操作；而在异步 IO 模型中，收到信号表示 IO 操作已经完成，不需要再在用户线程中调用 IO 函数进行实际的读写操作。

注意，异步 IO 是需要操作系统的底层支持，在 Java 7 中，提供了 Asynchronous IO。
更多参考： http://www.importnew.com/19816.html

10、JAVA NIO

NIO 主要有三大核心部分： Channel(通道)， Buffer(缓冲区), Selector。传统 IO 基于字节流和字符流进行操作， 而 NIO 基于 Channel 和 Buffer(缓冲区)进行操作，数据总是从通道读取到缓冲区中，或者从缓冲区写入到通道中。 Selector(选择区)用于监听多个通道的事件（比如：连接打开，数据到达）。因此，单个线程可以监听多个数据通道。 NIO 和传统 IO 之间第一个最大的区别是， IO 是面向流的， NIO 是面向缓冲区的。

11、NIO 的缓冲区

Java IO 面向流意味着每次从流中读一个或多个字节，直至读取所有字节，它们没有被缓存在任何地方。此外，它不能前后移动流中的数据。如果需要前后移动从流中读取的数据， 需要先将它缓存到一个缓冲区。 NIO 的缓冲导向方法不同。数据读取到一个它稍后处理的缓冲区，需要时可在缓冲区中前后移动。这就增加了处理过程中的灵活性。但是，还需要检查是否该缓冲区中包含所有您需要处理的数据。而且，需确保当更多的数据读入缓冲区时，不要覆盖缓冲区里尚未处理的数据。

12、NIO 的非阻塞

IO 的各种流是阻塞的。这意味着，当一个线程调用 read() 或 write()时，该线程被阻塞，直到有一些数据被读取，或数据完全写入。该线程在此期间不能再干任何事情了。 NIO 的非阻塞模式，使一个线程从某通道发送请求读取数据，但是它仅能得到目前可用的数据，如果目前没有数据可用时，就什么都不会获取。而不是保持线程阻塞，所以直至数据变的可以读取之前，该线程可以继续做其他的事情。 非阻塞写也是如此。一个线程请求写入一些数据到某通道，但不需要等待它完全写入，这个线程同时可以去做别的事情。 线程通常将非阻塞 IO 的空闲时间用于在其它通道上执行 IO 操作，所以一个单独的线程现在可以管理多个输入和输出通道（channel）。

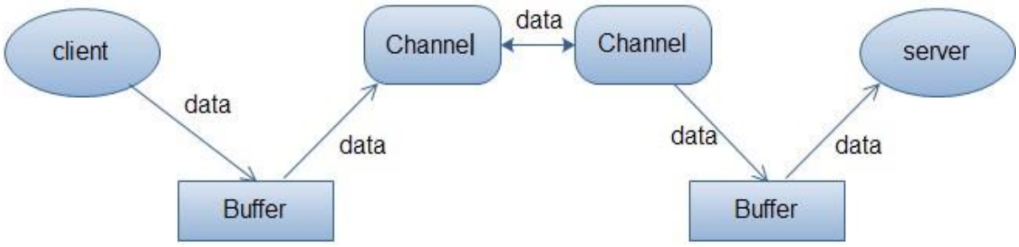
13、Channel

首先说一下 Channel，国内大多翻译成“通道”。 Channel 和 IO 中的 Stream(流)是差不多一个等级的。 只不过 Stream 是单向的，譬如： InputStream, OutputStream， 而 Channel 是双向的，既可以用来进行读操作，又可以用来进行写操作。NIO 中的 Channel 的主要实现有：

1. FileChannel
2. DatagramChannel
3. SocketChannel
4. ServerSocketChannel
   这里看名字就可以猜出个所以然来：分别可以对应文件 IO、 UDP 和 TCP（Server 和 Client）。
   下面演示的案例基本上就是围绕这 4 个类型的 Channel 进行陈述的。

14、Buffer

Buffer，故名思意， 缓冲区，实际上是一个容器，是一个连续数组。 Channel 提供从文件、网络读取数据的渠道，但是读取或写入的数据都必须经由 Buffer。



上面的图描述了从一个客户端向服务端发送数据，然后服务端接收数据的过程。客户端发送数据时，必须先将数据存入 Buffer 中，然后将 Buffer 中的内容写入通道。服务端这边接收数据必须通过 Channel 将数据读入到 Buffer 中，然后再从 Buffer 中取出数据来处理。

在 NIO 中， Buffer 是一个顶层父类，它是一个抽象类，常用的 Buffer 的子类有：ByteBuffer、 IntBuffer、 CharBuffer、 LongBuffer、 DoubleBuffer、 FloatBuffer、ShortBuffer

15、Selector

Selector 类是 NIO 的核心类， Selector 能够检测多个注册的通道上是否有事件发生，如果有事件发生，便获取事件然后针对每个事件进行相应的响应处理。这样一来，只是用一个单线程就可以管理多个通道，也就是管理多个连接。这样使得只有在连接真正有读写事件发生时，才会调用函数来进行读写，就大大地减少了系统开销，并且不必为每个连接都创建一个线程，不用去维护多个线程，并且避免了多线程之间的上下文切换导致的开销。

Java反射面试题

1、除了使用new创建对象之外，还可以用什么方法创建对象？

使用Java反射可以创建对象!

2、Java反射创建对象效率高还是通过new创建对象的效率高？

通过new创建对象的效率比较高。通过反射时，先找查找类资源，使用类加载器创建，过程比较繁琐，所以效率较低

3、java反射的作用

反射机制是在运行时，对于任意一个类，都能够知道这个类的所有属性和方法；对于任意个对象，都能够调用它的任意一个方法。在java中，只要给定类的名字，就可以通过反射机制来获得类的所有信息。

> 这种动态获取的信息以及动态调用对象的方法的功能称为Java语言的反射机制。

4、哪里会用到反射机制？

jdbc就是典型的反射

```
Class.forName('com.mysql.jdbc.Driver.class');//加载MySQL的驱动类
```

这就是反射。如hibernate，struts等框架使用反射实现的。

5、反射的实现方式：

第一步：获取Class对象，有4中方法：

1）Class.forName(“类的路径”)；

2）类名.class

3）对象名.getClass()

4）基本类型的包装类，可以调用包装类的Type属性来获得该包装类的Class对象

6、实现Java反射的类：

1）Class：表示正在运行的Java应用程序中的类和接口

注意： 所有获取对象的信息都需要Class类来实现。

2）Field：提供有关类和接口的属性信息，以及对它的动态访问权限。

3）Constructor：提供关于类的单个构造方法的信息以及它的访问权限

4）Method：提供类或接口中某个方法的信息

7、反射机制的优缺点：

优点：

1）能够运行时动态获取类的实例，提高灵活性；

2）与动态编译结合

缺点：

1）使用反射性能较低，需要解析字节码，将内存中的对象进行解析。

解决方案：

1、通过setAccessible(true)关闭JDK的安全检查来提升反射速度；

2、多次创建一个类的实例时，有缓存会快很多

3、ReflflectASM工具类，通过字节码生成的方式加快反射速度

2）相对不安全，破坏了封装性（因为通过反射可以获得私有方法和属性）

8、Java 反射 API

反射 API 用来生成 JVM 中的类、接口或则对象的信息。

1. Class 类：反射的核心类，可以获取类的属性，方法等信息。
2. Field 类：Java.lang.reflec 包中的类，表示类的成员变量，可以用来获取和设置类之中的属性

值。

3. Method 类： Java.lang.reflec 包中的类，表示类的方法，它可以用来获取类中的方法信息或

者执行方法。

4. Constructor 类： Java.lang.reflec 包中的类，表示类的构造方法。

9、反射使用步骤（获取 Class 对象、调用对象方法）

1. 获取想要操作的类的 Class 对象，他是反射的核心，通过 Class 对象我们可以任意调用类的方法。
2. 调用 Class 类中的方法，既就是反射的使用阶段。
3. 使用反射 API 来操作这些信息。

10、获取 Class 对象有几种方法

调用某个对象的 getClass()方法

```
Person p=new Person();

Class clazz=p.getClass();
```

调用某个类的 class 属性来获取该类对应的 Class 对象

```
Class clazz=Person.class;
```

使用 Class 类中的 forName()静态方法(最安全/性能最好)

```
Class clazz=Class.forName("类的全路径"); (最常用)
```

当我们获得了想要操作的类的 Class 对象后，可以通过 Class 类中的方法获取并查看该类中的方法

和属性。

```
//获取 Person 类的 Class 对象

 Class clazz=Class.forName("reflection.Person");
//获取 Person 类的所有方法信息
 Method[] method=clazz.getDeclaredMethods();
 for(Method m:method){
    System.out.println(m.toString());
 }
 //获取 Person 类的所有成员属性信息
 Field[] field=clazz.getDeclaredFields();
 for(Field f:field){
    System.out.println(f.toString());
 }
 //获取 Person 类的所有构造方法信息
 Constructor[] constructor=clazz.getDeclaredConstructors();
 for(Constructor c:constructor){
    System.out.println(c.toString());
 }
```

11、利用反射动态创建对象实例

Class 对象的 newInstance()

1. 使用 Class 对象的 newInstance()方法来创建该 Class 对象对应类的实例，但是这种方法要求

该 Class 对象对应的类有默认的空构造器。

调用 Constructor 对象的 newInstance()

2. 先使用 Class 对象获取指定的 Constructor 对象，再调用 Constructor 对象的 newInstance()

方法来创建 Class 对象对应类的实例,通过这种方法可以选定构造方法创建实例。

```
 //获取 Person 类的 Class 对象

 Class clazz=Class.forName("reflection.Person");

 //使用.newInstane 方法创建对象

 Person p=(Person) clazz.newInstance();

//获取构造方法并创建对象

 Constructor c=clazz.getDeclaredConstructor(String.class,String.class,int.class);

 //创建对象并设置属性13/04/2018

 Person p1=(Person) c.newInstance("李四","男",20);
```

Java序列化面试题

1、什么是java序列化，如何实现java序列化？

序列化就是一种用来处理对象流的机制，所谓对象流也就是将对象的内容进行流化。可以对流化后的对象进行读写操作，也可将流化后的对象传输于网络之间。序列化是为了解决在对对象流进行读写操作时所引发的问题。序列化的实现：将需要被序列化的类实现Serializable接口，该接口没有需要实现的方法，implements Serializable只是为了标注该对象是可被序列化的，然后使用一个输出流(如：FileOutputStream)来构造一个ObjectOutputStream(对象流)对象，接着，使用ObjectOutputStream对象的writeObject(Object obj)方法就可以将参数为obj的对象写出(即保存其状态)，要恢复的话则用输入流。

2、保存(持久化)对象及其状态到内存或者磁盘

Java 平台允许我们在内存中创建可复用的 Java 对象，但一般情况下，只有当 JVM 处于运行时，这些对象才可能存在，即，这些对象的生命周期不会比 JVM 的生命周期更长。 但在现实应用中，就可能要求在JVM停止运行之后能够保存(持久化)指定的对象，并在将来重新读取被保存的对象。Java 对象序列化就能够帮助我们实现该功能。

3、序列化对象以字节数组保持-静态成员不保存

使用 Java 对象序列化， 在保存对象时，会把其状态保存为一组字节，在未来， 再将这些字节组装成对象。必须注意地是， 对象序列化保存的是对象的"状态"，即它的成员变量。由此可知，对象序列化不会关注类中的静态变量。

4、序列化用户远程对象传输

除了在持久化对象时会用到对象序列化之外，当使用 RMI(远程方法调用)，或在网络中传递对象时，都会用到对象序列化。 Java序列化API为处理对象序列化提供了一个标准机制，该API简单易用。

5、Serializable 实现序列化

在 Java 中， 只要一个类实现了 java.io.Serializable 接口，那么它就可以被序列化。ObjectOutputStream 和 ObjectInputStream 对对象进行序列化及反序列化通过 ObjectOutputStream 和 ObjectInputStream 对对象进行序列化及反序列化。

6、writeObject 和 readObject 自定义序列化策略

在类中增加 writeObject 和 readObject 方法可以实现自定义序列化策略。

7、序列化 ID

虚拟机是否允许反序列化，不仅取决于类路径和功能代码是否一致，一个非常重要的一点是两个类的序列化 ID 是否一致（就是 private static final long serialVersionUID

8、序列化并不保存静态变量

序列化子父类说明
要想将父类对象也序列化，就需要让父类也实现 Serializable 接口。

9、Transient 关键字阻止该变量被序列化到文件中

1. 在变量声明前加上 Transient 关键字，可以阻止该变量被序列化到文件中，在被反序列化后， transient 变量的值被设为初始值，如 int 型的是 0，对象型的是 null。
2. 服务器端给客户端发送序列化对象数据，对象中有一些数据是敏感的，比如密码字符串等，希望对该密码字段在序列化时，进行加密，而客户端如果拥有解密的密钥，只有在客户端进行反序列化时，才可以对密码进行读取，这样可以一定程度保证序列化对象的数据安全。

10、序列化（深 clone 一中实现）

在 Java 语言里深复制一个对象，常常可以先使对象实现 Serializable 接口，然后把对象（实际上只是对象的一个拷贝）写到一个流里，再从流里读出来，便可以重建对象。

Java注解面试题

1、4种标准元注解是哪四种？

元注解的作用是负责注解其他注解。 Java5.0 定义了 4 个标准的 meta-annotation 类型，它们被用来提供对其它 annotation 类型作说明。

@Target 修饰的对象范围

@Target说明了Annotation所修饰的对象范围： Annotation可被用于 packages、types（类、接口、枚举、Annotation 类型）、类型成员（方法、构造方法、成员变量、枚举值）、方法参数和本地变量（如循环变量、catch 参数）。在 Annotation 类型的声明中使用了 target 可更加明晰其修饰的目标

@Retention 定义 被保留的时间长短

Retention 定义了该 Annotation 被保留的时间长短：表示需要在什么级别保存注解信息，用于描述注解的生命周期（即：被描述的注解在什么范围内有效），取值（RetentionPoicy）由：

1. SOURCE:在源文件中有效（即源文件保留）
2. CLASS:在 class 文件中有效（即 class 保留）
3. RUNTIME:在运行时有效（即运行时保留）
4.

@Documented 描述-javadoc

@ Documented 用于描述其它类型的 annotation 应该被作为被标注的程序成员的公共 API，因

此可以被例如 javadoc 此类的工具文档化。

@Inherited 阐述了某个被标注的类型是被继承的

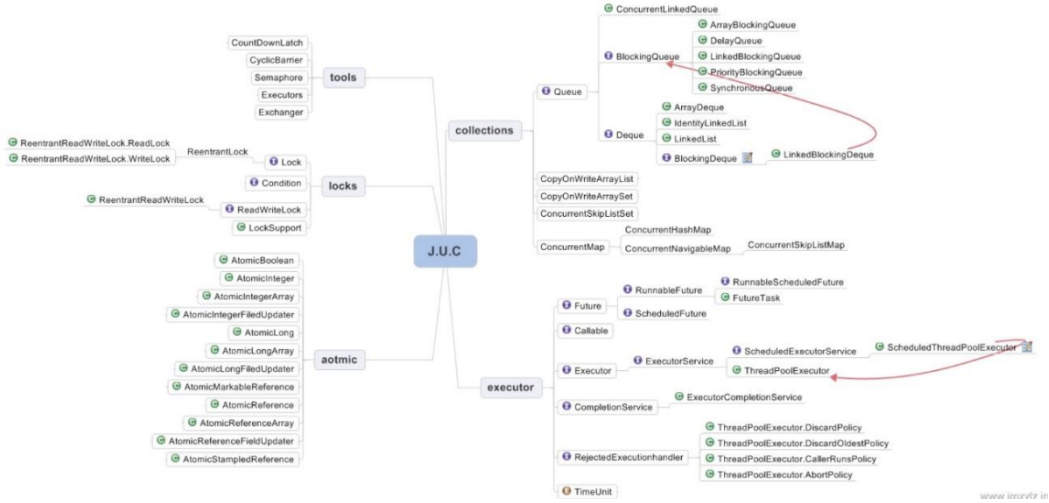
@Inherited 元注解是一个标记注解，@Inherited 阐述了某个被标注的类型是被继承的。如果一个使用了@Inherited 修饰的 annotation 类型被用于一个 class，则这个 annotation 将被用于该 class 的子类。

2、注解是什么？

Annotation（注解）是 Java 提供的一种对元程序中元素关联信息和元数据（metadata）的途径和方法。 Annatation(注解)是一个接口，程序可以通过反射来获取指定程序中元素的 Annotation对象，然后通过该 Annotation 对象来获取注解中的元数据信息。

多线程&并发面试题

JAVA 并发知识库



1、Java中实现多线程有几种方法

继承Thread类；
实现Runnable接口；
实现Callable接口通过FutureTask包装器来创建Thread线程；
使用ExecutorService、Callable、Future实现有返回结果的多线程（也就是使用了ExecutorService来管理前面的三种方式）。

2、继承 Thread 类

Thread 类本质上是实现了 Runnable 接口的一个实例，代表一个线程的实例。 启动线程的唯一方法就是通过 Thread 类的 start()实例方法。 start()方法是一个 native 方法，它将启动一个新线程，并执行 run()方法。

```
public class MyThread extends Thread {
    public void run() {
        System.out.println("MyThread.run()");
    }
}
MyThread myThread1 = new MyThread();
myThread1.start();
```

3、实现 Runnable 接口。

如果自己的类已经 extends 另一个类，就无法直接 extends Thread，此时，可以实现一个 Runnable 接口。

```
public class MyThread extends OtherClass implements Runnable {
    public void run() {
        System.out.println("MyThread.run()");
    }
}
//启动 MyThread，需要首先实例化一个 Thread，并传入自己的 MyThread 实例：
MyThread myThread = new MyThread();
Thread thread = new Thread(myThread);
thread.start();
//事实上，当传入一个 Runnable target 参数给 Thread 后， Thread 的 run()方法就会调用
target.run()
public void run() {
    if (target != null) {
        target.run();
    }
}
```

4、ExecutorService、 Callable、 Future 有返回值线程

有返回值的任务必须实现 Callable 接口，类似的，无返回值的任务必须 Runnable 接口。执行 Callable 任务后，可以获取一个 Future 的对象，在该对象上调用 get 就可以获取到 Callable 任务返回的 Object 了，再结合线程池接口 ExecutorService 就可以实现传说中有返回结果的多线程了。

```
//创建一个线程池
ExecutorService pool = Executors.newFixedThreadPool(taskSize);
// 创建多个有返回值的任务
List<Future> list = new ArrayList<Future>();
for (int i = 0; i < taskSize; i++) {
    Callable c = new MyCallable(i + " ");
// 执行任务并获取 Future 对象
    Future f = pool.submit(c);
    list.add(f);
}
// 关闭线程池
pool.shutdown();
// 获取所有并发任务的运行结果
for (Future f : list) {
// 从 Future 对象上获取任务的返回值，并输出到控制台
    System.out.println("res：  " + f.get().toString());
}
```

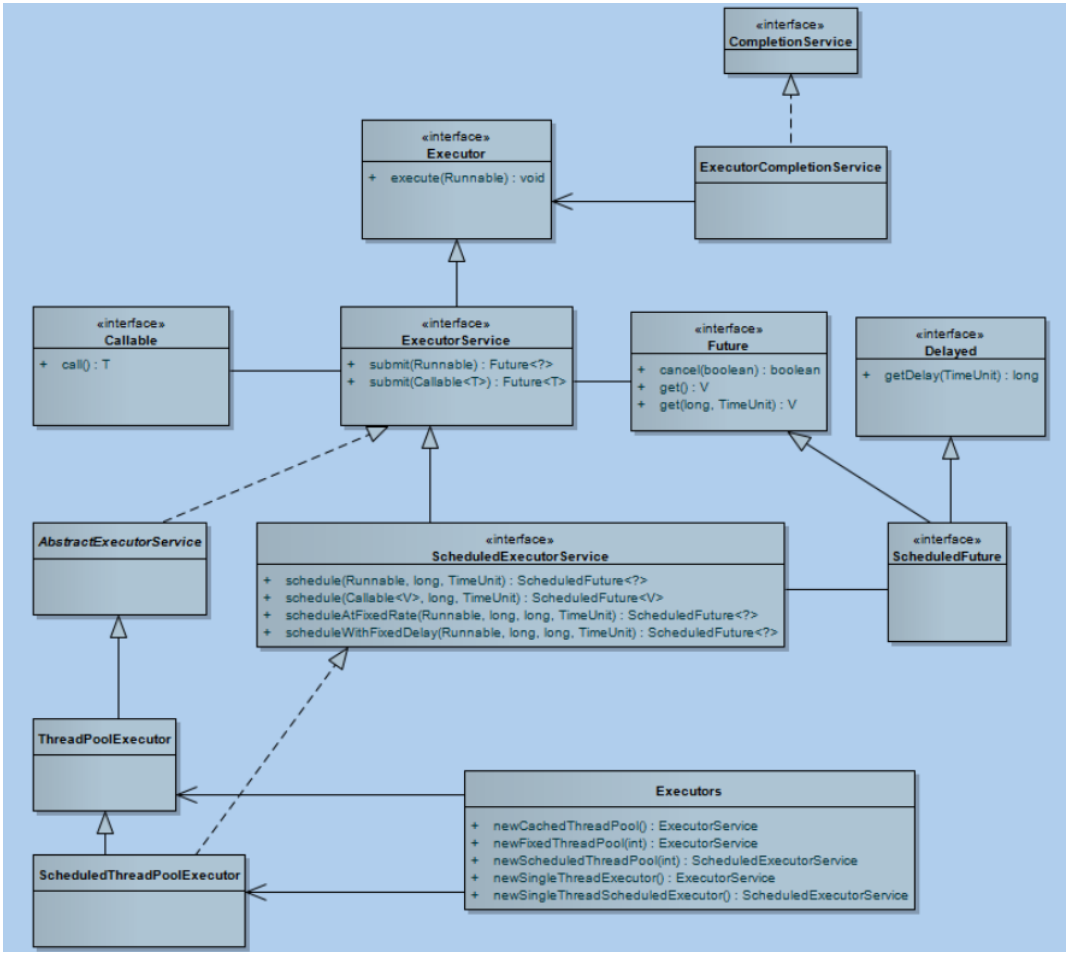
5、基于线程池的方式

线程和数据库连接这些资源都是非常宝贵的资源。那么每次需要的时候创建，不需要的时候销毁，是非常浪费资源的。那么我们就可以使用缓存的策略，也就是使用线程池。

```
// 创建线程池
ExecutorService threadPool = Executors.newFixedThreadPool(10);
while(true) {
    threadPool.execute(new Runnable() { // 提交多个线程任务，并执行
                        @Override
                        public void run() {
                            System.out.println(Thread.currentThread().getName() + " is running ..");
                            try {
                                Thread.sleep(3000);
                            } catch (InterruptedException e) {
                                e.printStackTrace();
                            }
                        }
                    });
    }
}
```

6、4 种线程池

Java 里面线程池的顶级接口是 Executor，但是严格意义上讲 Executor 并不是一个线程池，而只是一个执行线程的工具。真正的线程池接口是 ExecutorService。



newCachedThreadPool

创建一个可根据需要创建新线程的线程池，但是在以前构造的线程可用时将重用它们。对于执行很多短期异步任务的程序而言，这些线程池通常可提高程序性能。 调用 execute 将重用以前构造的线程（如果线程可用）。如果现有线程没有可用的，则创建一个新线程并添加到池中。终止并从缓存中移除那些已有 60 秒钟未被使用的线程。 因此，长时间保持空闲的线程池不会使用任何资源。

newFixedThreadPool

创建一个可重用固定线程数的线程池，以共享的无界队列方式来运行这些线程。在任意点，在大多数 nThreads 线程会处于处理任务的活动状态。如果在所有线程处于活动状态时提交附加任务，则在有可用线程之前，附加任务将在队列中等待。如果在关闭前的执行期间由于失败而导致任何线程终止，那么一个新线程将代替它执行后续的任务（如果需要）。在某个线程被显式地关闭之前，池中的线程将一直存在。

newScheduledThreadPool

创建一个线程池，它可安排在给定延迟后运行命令或者定期地执行。

```
ScheduledExecutorService scheduledThreadPool= Executors.newScheduledThreadPool(3);
scheduledThreadPool.schedule(newRunnable(){
    @Override
    public void run() {
        System.out.println("延迟三秒");
    }
}, 3, TimeUnit.SECONDS);
scheduledThreadPool.scheduleAtFixedRate(newRunnable(){
    @Override
    public void run() {
        System.out.println("延迟 1 秒后每三秒执行一次");
    }
},1,3,TimeUnit.SECONDS);
```

newSingleThreadExecutor

Executors.newSingleThreadExecutor()返回一个线程池（这个线程池只有一个线程），这个线程池可以在线程死后（或发生异常时）重新启动一个线程来替代原来的线程继续执行下去！

7、如何停止一个正在运行的线程

1、使用退出标志，使线程正常退出，也就是当run方法完成后线程终止。
2、使用stop方法强行终止，但是不推荐这个方法，因为stop和suspend及resume一样都是过期作废的方法。
3、使用interrupt方法中断线程。

```
class MyThread extends Thread {
    volatile boolean stop = false;
        public void run() {
            while (!stop) {
                System.out.println(getName() + " is running");
                try {
                    sleep(1000);
                } catch (InterruptedException e) {
                    System.out.println("week up from blcok...");
                    stop = true; // 在异常处理代码中修改共享变量的状态
                }
        }
    System.out.println(getName() + " is exiting...");
    }
}

class InterruptThreadDemo3 {
    public static void main(String[] args) throws InterruptedException {
        MyThread m1 = new MyThread();
        System.out.println("Starting thread...");
        m1.start();
        Thread.sleep(3000);
        System.out.println("Interrupt thread...: " + m1.getName());
        m1.stop = true; // 设置共享变量为true
        m1.interrupt(); // 阻塞时退出阻塞状态
        Thread.sleep(3000); // 主线程休眠3秒以便观察线程m1的中断情况
        System.out.println("Stopping application...");
    }
}
```

8、notify()和notifyAll()有什么区别？

notify可能会导致死锁，而notifyAll则不会

任何时候只有一个线程可以获得锁，也就是说只有一个线程可以运行synchronized 中的代码
使用notifyall,可以唤醒

所有处于wait状态的线程，使其重新进入锁的争夺队列中，而notify只能唤醒一个。

wait() 应配合while循环使用，不应使用if，务必在wait()调用前后都检查条件，如果不满足，必须调用notify()唤醒另外的线程来处理，自己继续wait()直至条件满足再往下执行。

notify() 是对notifyAll()的一个优化，但它有很精确的应用场景，并且要求正确使用。不然可能导致死锁。正确的场景应该是 WaitSet中等待的是相同的条件，唤醒任一个都能正确处理接下来的事项，如果唤醒的线程无法正确处理，务必确保继续notify()下一个线程，并且自身需要重新回到WaitSet中.

9、sleep()和wait() 有什么区别？

1. 对于 sleep()方法，我们首先要知道该方法是属于 Thread 类中的。而 wait()方法，则是属于 Object 类中的。
2. sleep()方法导致了程序暂停执行指定的时间，让出 cpu 该其他线程，但是他的监控状态依然保持者，当指定的时间到了又会自动恢复运行状态
3. 在调用 sleep()方法的过程中， 线程不会释放对象锁。
4. 而当调用 wait()方法的时候，线程会放弃对象锁，进入等待此对象的等待锁定池，只有针对此对象调用 notify()方法后本线程才进入对象锁定池准备获取对象锁进入运行状态。

10、volatile 是什么?可以保证有序性吗?

一旦一个共享变量（类的成员变量、类的静态成员变量）被volatile修饰之后，那么就具备了两层语义：

1）保证了不同线程对这个变量进行操作时的可见性，即一个线程修改了某个变量的值，这新值对其他线程来说是立即可见的,volatile关键字会强制将修改的值立即写入主存。

2）禁止进行指令重排序。
volatile 不是原子性操作
什么叫保证部分有序性?

当程序执行到volatile变量的读操作或者写操作时，在其前面的操作的更改肯定全部已经进行，且结果已经对后面的操作可见；在其后面的操作肯定还没有进行；

```
x = 2;        //语句1
y = 0;        //语句2
flag = true; //语句3
x = 4;        //语句4
y = -1;       //语句5
```

由于flag变量为volatile变量，那么在进行指令重排序的过程的时候，不会将语句3放到语句1、语句2前面，也不会讲语句3放到语句4、语句5后面。但是要注意语句1和语句2的顺序、语句4和语句5的顺序是不作任何保证的。
使用 Volatile 一般用于 状态标记量 和 单例模式的双检锁

11、Thread 类中的start() 和 run() 方法有什么区别？

start()方法被用来启动新创建的线程，而且start()内部调用了run()方法，这和直接调用run()方法的效果不一样。当你调用run()方法的时候，只会是在原来的线程中调用，没有新的线程启动，start()方法才会启动新线程 。

12、为什么wait, notify 和 notifyAll这些方法不在thread类里面？

明显的原因是JAVA提供的锁是对象级的而不是线程级的，每个对象都有锁，通过线程获得。如果线程需要等待某些锁那么调用对象中的wait()方法就有意义了。如果wait()方法定义在Thread类中，线程正在等待的是哪个锁就不明显了。简单的说，由于wait，notify和notifyAll都是锁级别的操作，所以把他们定义在Object类中因为锁属于对象 。

13、为什么wait和notify方法要在同步块中调用？

1. 只有在调用线程拥有某个对象的独占锁时，才能够调用该对象的wait(),notify()和notifyAll()方法。
2. 如果你不这么做，你的代码会抛出IllegalMonitorStateException异常。
3. 还有一个原因是为了避免wait和notify之间产生竞态条件。

wait()方法强制当前线程释放对象锁。这意味着在调用某对象的wait()方法之前，当前线程必须已经获得该对象的锁。因此，线程必须在某个对象的同步方法或同步代码块中才能调用该对象的wait()方法。

在调用对象的notify()和notifyAll()方法之前，调用线程必须已经得到该对象的锁。因此，必须在某个对象的同步方法或同步代码块中才能调用该对象的notify()或notifyAll()方法。

调用wait()方法的原因通常是，调用线程希望某个特殊的状态(或变量)被设置之后再继续执行。调用notify()或notifyAll()方法的原因通常是，调用线程希望告诉其他等待中的线程:"特殊状态已经被设置"。这个状态作为线程间通信的通道，它必须是一个可变的共享状态(或变量)。

14、Java中interrupted 和 isInterruptedd方法的区别？

interrupted() 和 isInterrupted()的主要区别是前者会将中断状态清除而后者不会。Java多线程的中断机制是用内部标识来实现的，调用Thread.interrupt()来中断一个线程就会设置中断标识为true。

当中断线程调用静态方法Thread.interrupted()来检查中断状态时，中断状态会被清零。

而非静态方法isInterrupted()用来查询其它线程的中断状态且不会改变中断状态标识。简单的说就是任何抛出InterruptedException异常的方法都会将中断状态清零。无论如何，一个线程的中断状态有有可能被其它线程调用中断来改变 。

15、Java中synchronized 和 ReentrantLock 有什么不同？

相似点：
这两种同步方式有很多相似之处，它们都是加锁方式同步，而且都是阻塞式的同步，也就是说当如果一个线程获得了对象锁，进入了同步块，其他访问该同步块的线程都必须阻塞在同步块外面等待，而进行线程阻塞和唤醒的代价是比较高的.

区别：
这两种方式最大区别就是对于Synchronized来说，它是java语言的关键字，是原生语法层面的互斥，需要jvm实现。而ReentrantLock它是JDK 1.5之后提供的API层面的互斥锁，需要lock()和unlock()方法配合try/finally语句块来完成。

Synchronized进过编译，会在同步块的前后分别形成monitorenter和monitorexit这个两个字节码指令。在执行monitorenter指令时，首先要尝试获取对象锁。如果这个对象没被锁定，或者当前线程已经拥有了那个对象锁，把锁的计算器加1，相应的，在执行monitorexit指令时会将锁计算器就减1，当计算器为0时，锁就被释放了。如果获取对象锁失败，那当前线程就要阻塞，直到对象锁被另一个线程释放为止 。

由于ReentrantLock是java.util.concurrent包下提供的一套互斥锁，相比Synchronized，ReentrantLock类提供了一些高级功能，主要有以下3项：
1.等待可中断，持有锁的线程长期不释放的时候，正在等待的线程可以选择放弃等待，这相当于Synchronized来说可以避免出现死锁的情况。

2.公平锁，多个线程等待同一个锁时，必须按照申请锁的时间顺序获得锁，Synchronized锁非公平锁，ReentrantLock默认的构造函数是创建的非公平锁，可以通过参数true设为公平锁，但公平锁表现的性能不是很好。

3.锁绑定多个条件，一个ReentrantLock对象可以同时绑定对个对象 。

16、有三个线程T1,T2,T3,如何保证顺序执行？

在多线程中有多种方法让线程按特定顺序执行，你可以用线程类的join()方法在一个线程中启动另一个线程，另外一个线程完成该线程继续执行。为了确保三个线程的顺序你应该先启动最后一个(T3调用T2，T2调用T1)，这样T1就会先完成而T3最后完成。

实际上先启动三个线程中哪一个都行，
因为在每个线程的run方法中用join方法限定了三个线程的执行顺序

```
public class JoinTest2 {

 // 1.现在有T1、T2、T3三个线程，你怎样保证T2在T1执行完后执行，T3在T2执行完后执行
```

```
    public static void main(String[] args) {
        final Thread t1 = new Thread(new Runnable() {
            @Override
            public void run() {
                System.out.println("t1");
            }
        });
        final Thread t2 = new Thread(new Runnable() {
            @Override
            public void run() {
                try {

// 引用t1线程，等待t1线程执行完
                    t1.join();
                } catch (InterruptedException e) {
                    e.printStackTrace();
                } S
                ystem.out.println("t2");
            }
        });
        Thread t3 = new Thread(new Runnable() {
            @Override
            public void run() {
                try {
// 引用t2线程，等待t2线程执行完
                    t2.join();
                } catch (InterruptedException e) {
                    e.printStackTrace();
                } S
                ystem.out.println("t3");
            }
        });
        t3.start();//这里三个线程的启动顺序可以任意，大家可以试下！
        t2.start();
        t1.start();
    }
}
```

17、SynchronizedMap和ConcurrentHashMap有什么区别？

SynchronizedMap()和Hashtable一样，实现上在调用map所有方法时，都对整个map进行同步。而ConcurrentHashMap的实现却更加精细，它对map中的所有桶加了锁。所以，只要有一个线程访问map，其他线程就无法进入map，而如果一个线程在访问ConcurrentHashMap某个桶时，其他线程，仍然可以对map执行某些操作。

所以，ConcurrentHashMap在性能以及安全性方面，明显比Collections.synchronizedMap()更加有优势。同时，同步操作精确控制到桶，这样，即使在遍历map时，如果其他线程试图对map进行数据修改，也不会抛出ConcurrentModificationException 。

18、什么是线程安全

线程安全就是说多线程访问同一代码，不会产生不确定的结果。

在多线程环境中，当各线程不共享数据的时候，即都是私有（private）成员，那么一定是线程安全的。但这种情况并不多见，在多数情况下需要共享数据，这时就需要进行适当的同步控制了。

线程安全一般都涉及到synchronized， 就是一段代码同时只能有一个线程来操作 不然中间过程可能会产生不可预制的结果。

如果你的代码所在的进程中有多个线程在同时运行，而这些线程可能会同时运行这段代码。如果每次运行结果和单线程运行的结果是一样的，而且其他的变量的值也和预期的是一样的，就是线程安全的。

19、Thread类中的yield方法有什么作用？

Yield方法可以暂停当前正在执行的线程对象，让其它有相同优先级的线程执行。它是一个静态方法而且只保证当前线程放弃CPU占用而不能保证使其它线程一定能占用CPU，执行yield()的线程有可能在进入到暂停状态后马上又被执行。

20、Java线程池中submit() 和 execute()方法有什么区别？

两个方法都可以向线程池提交任务，execute()方法的返回类型是void，它定义在Executor接口中, 而submit()方法可以返回持有计算结果的Future对象，它定义在ExecutorService接口中，它扩展了Executor接口，其它线程池类像ThreadPoolExecutor和ScheduledThreadPoolExecutor都有这些方法 。

21、说一说自己对于 synchronized 关键字的了解

synchronized关键字解决的是多个线程之间访问资源的同步性，synchronized关键字可以保证被它修饰的方法或者代码块在任意时刻只能有一个线程执行。

另外，在 Java 早期版本中，synchronized属于重量级锁，效率低下，因为监视器锁（monitor）是依赖于底层的操作系统的 Mutex Lock 来实现的，Java 的线程是映射到操作系统的原生线程之上的。

如果要挂起或者唤醒一个线程，都需要操作系统帮忙完成，而操作系统实现线程之间的切换时需要从用户态转换到内核态，这个状态之间的转换需要相对比较长的时间，时间成本相对较高，这也是为什么早期的synchronized 效率低的原因。

庆幸的是在 Java 6 之后 Java 官方对从 JVM 层面对synchronized 较大优化，所以现在的 synchronized 锁效率也优化得很不错了。JDK1.6对锁的实现引入了大量的优化，如自旋锁、适应性自旋锁、锁消除、锁粗化、偏向锁、轻量级锁等技术来减少锁操作的开销 。

22、说说自己是怎么使用 synchronized 关键字，在项目中用到了吗synchronized关键字最主要的三种使用方式

修饰实例方法: 作用于当前对象实例加锁，进入同步代码前要获得当前对象实例的锁
修饰静态方法: 也就是给当前类加锁，会作用于类的所有对象实例，因为静态成员不属于任何一个实例对象，是类成员（ static 表明这是该类的一个静态资源，不管new了多少个对象，只有一份）。所以如果一个线程A调用一个实例对象的非静态 synchronized 方法，而线程B需要调用这个实例对象所属类的静态 synchronized 方法，是允许的，不会发生互斥现象，**因为访问静态 synchronized 方法占用的锁是当前类的锁，而访问非静态synchronized 方法占用的锁是当前实例对象锁。**
修饰代码块: 指定加锁对象，对给定对象加锁，进入同步代码库前要获得给定对象的锁。
总结： synchronized 关键字加到 static 静态方法和 synchronized(class)代码块上都是是给 Class 类上锁。synchronized 关键字加到实例方法上是给对象实例上锁。尽量不要使用 synchronized(String a) 因为JVM中，字符串常量池具有缓存功能

23、什么是线程安全？Vector是一个线程安全类吗？

如果你的代码所在的进程中有多个线程在同时运行，而这些线程可能会同时运行这段代码。如果每次运行结果和单线程运行的结果是一样的，而且其他的变量 的值也和预期的是一样的，就是线程安全的。一个线程安全的计数器类的同一个实例对象在被多个线程使用的情况下也不会出现计算失误。很显然你可以将集合类分 成两组，线程安全和非线程安全的。Vector 是用同步方法来实现线程安全的, 而和它相似的ArrayList不是线程安全的。

24、volatile关键字的作用？

一旦一个共享变量（类的成员变量、类的静态成员变量）被volatile修饰之后，那么就具备了两层语义：
保证了不同线程对这个变量进行操作时的可见性，即一个线程修改了某个变量的值，这新值对其他线程来说是立即可见的。

禁止进行指令重排序。

1. volatile本质是在告诉jvm当前变量在寄存器（工作内存）中的值是不确定的，需要从主存中读取；synchronized则是锁定当前变量，只有当前线程可以访问该变量，其他线程被阻塞住。
2. volatile仅能使用在变量级别；synchronized则可以使用在变量、方法、和类级别的。
3. volatile仅能实现变量的修改可见性，并不能保证原子性；synchronized则可以保证变量的修改可见性和原子性。
4. volatile不会造成线程的阻塞；synchronized可能会造成线程的阻塞。

volatile标记的变量不会被编译器优化；synchronized标记的变量可以被编译器优化

25、简述一下你对线程池的理解

如果问到了这样的问题，可以展开的说一下线程池如何用、线程池的好处、线程池的启动策略）合理利用线程池能够带来三个好处。

第一：降低资源消耗。通过重复利用已创建的线程降低线程创建和销毁造成的消耗。
第二：提高响应速度。当任务到达时，任务可以不需要等到线程创建就能立即执行。
第三：提高线程的可管理性。线程是稀缺资源，如果无限制的创建，不仅会消耗系统资源，还会降低系统的稳定性，使用线程池可以进行统一的分配，调优和监控

26、线程生命周期(状态)

当线程被创建并启动以后，它既不是一启动就进入了执行状态，也不是一直处于执行状态。在线程的生命周期中，它要经过新建(New)、就绪（Runnable）、运行（Running）、阻塞(Blocked)和死亡(Dead)5 种状态。尤其是当线程启动以后，它不可能一直"霸占"着 CPU 独自运行，所以 CPU 需要在多条线程之间切换，于是线程状态也会多次在运行、阻塞之间切换

27、新建状态（NEW）

当程序使用 new 关键字创建了一个线程之后，该线程就处于新建状态，此时仅由 JVM 为其分配内存，并初始化其成员变量的值

28、就绪状态（RUNNABLE）

当线程对象调用了 start()方法之后，该线程处于就绪状态。 Java 虚拟机会为其创建方法调用栈和程序计数器，等待调度运行。

29、运行状态（RUNNING）

如果处于就绪状态的线程获得了 CPU，开始执行 run()方法的线程执行体，则该线程处于运行状态。

30、阻塞状态（BLOCKED）

阻塞状态是指线程因为某种原因放弃了 cpu 使用权，也即让出了 cpu timeslice，暂时停止运行。直到线程进入可运行(runnable)状态，才有机会再次获得 cpu timeslice 转到运行(running)状态。阻塞的情况分三种：

等待阻塞（o.wait->等待对列）：
运行(running)的线程执行 o.wait()方法， JVM 会把该线程放入等待队列(waitting queue)中。

同步阻塞(lock->锁池)
运行(running)的线程在获取对象的同步锁时，若该同步锁被别的线程占用，则 JVM 会把该线程放入锁池(lock pool)中。

其他阻塞(sleep/join)
运行(running)的线程执行 Thread.sleep(long ms)或 t.join()方法，或者发出了 I/O 请求时，JVM 会把该线程置为阻塞状态。当 sleep()状态超时、 join()等待线程终止或者超时、或者 I/O处理完毕时，线程重新转入可运行(runnable)状态。

31、线程死亡（DEAD）

线程会以下面三种方式结束，结束后就是死亡状态。
正常结束

1. run()或 call()方法执行完成，线程正常结束。异常结束
2. 线程抛出一个未捕获的 Exception 或 Error。调用 stop
3. 直接调用该线程的 stop()方法来结束该线程—该方法通常容易导致死锁，不推荐使用。

32、终止线程 4 种方式

正常运行结束

程序运行结束，线程自动结束。

使用退出标志退出线程

一般 run()方法执行完，线程就会正常结束，然而，常常有些线程是伺服线程。它们需要长时间的运行，只有在外部某些条件满足的情况下，才能关闭这些线程。使用一个变量来控制循环，例如：最直接的方法就是设一个 boolean 类型的标志，并通过设置这个标志为 true 或 false 来控制 while循环是否退出，代码示例 ：

```
public class ThreadSafe extends Thread {
    public volatile boolean exit = false;
    public void run() {
        while (!exit){
//do something
        }
    }
}
```

定义了一个退出标志 exit，当 exit 为 true 时， while 循环退出， exit 的默认值为 false.在定义 exit时，使用了一个 Java 关键字 volatile，这个关键字的目的是使 exit 同步，也就是说在同一时刻只能由一个线程来修改 exit 的值。

Interrupt 方法结束线程

使用 interrupt()方法来中断线程有两种情况：

1.线程处于阻塞状态： 如使用了 sleep,同步锁的 wait,socket 中的 receiver,accept 等方法时，会使线程处于阻塞状态。当调用线程的 interrupt()方法时，会抛出 InterruptException 异常。阻塞中的那个方法抛出这个异常，通过代码捕获该异常，然后 break 跳出循环状态，从而让我们有机会结束这个线程的执行。 通常很多人认为只要调用 interrupt 方法线程就会结束，实际上是错的， 一定要先捕获 InterruptedException 异常之后通过 break 来跳出循环，才能正常结束 run 方法。

2.线程未处于阻塞状态： 使用 isInterrupted()判断线程的中断标志来退出循环。当使用interrupt()方法时，中断标志就会置 true，和使用自定义的标志来控制循环是一样的道理。

```
public class ThreadSafe extends Thread {
    public void run() {
        while (!isInterrupted()){ //非阻塞过程中通过判断中断标志来退出
            try{
                Thread.sleep(5*1000);//阻塞过程捕获中断异常来退出
            }catch(InterruptedException e){
                e.printStackTrace();
                break;//捕获到异常之后，执行 break 跳出循环
            }
        }
    }
}
```

stop 方法终止线程（线程不安全）

程序中可以直接使用 thread.stop()来强行终止线程，但是 stop 方法是很危险的，就象突然关闭计算机电源，而不是按正常程序关机一样，可能会产生不可预料的结果，不安全主要是：thread.stop()调用之后，创建子线程的线程就会抛出 ThreadDeatherror 的错误，并且会释放子线程所持有的所有锁。一般任何进行加锁的代码块，都是为了保护数据的一致性，如果在调用thread.stop()后导致了该线程所持有的所有锁的突然释放(不可控制)，那么被保护数据就有可能呈现不一致性，其他线程在使用这些被破坏的数据时，有可能导致一些很奇怪的应用程序错误。因
此，并不推荐使用 stop 方法来终止线程。

33、start 与 run 区别

1. start（ ）方法来启动线程，真正实现了多线程运行。这时无需等待 run 方法体代码执行完毕，可以直接继续执行下面的代码。
2. 通过调用 Thread 类的 start()方法来启动一个线程， 这时此线程是处于就绪状态， 并没有运行。
3. 方法 run()称为线程体，它包含了要执行的这个线程的内容，线程就进入了运行状态，开始运行 run 函数当中的代码。 Run 方法运行结束， 此线程终止。然后 CPU 再调度其它线程。

34、JAVA 后台线程

1. 定义：守护线程--也称"服务线程"， 他是后台线程， 它有一个特性，即为用户线程 提供 公共服务， 在没有用户线程可服务时会自动离开。
2. 优先级：守护线程的优先级比较低，用于为系统中的其它对象和线程提供服务。
3. 设置：通过 setDaemon(true)来设置线程为"守护线程"；将一个用户线程设置为守护线程的方式是在 线程对象创建 之前 用线程对象的 setDaemon 方法。
4. 在 Daemon 线程中产生的新线程也是 Daemon 的。
5. 线程则是 JVM 级别的，以 Tomcat 为例，如果你在 Web 应用中启动一个线程，这个线程的生命周期并不会和 Web 应用程序保持同步。也就是说，即使你停止了 Web 应用，这个线程依旧是活跃的。
6. example: 垃圾回收线程就是一个经典的守护线程，当我们的程序中不再有任何运行的Thread, 程序就不会再产生垃圾，垃圾回收器也就无事可做， 所以当垃圾回收线程是 JVM 上仅剩的线程时，垃圾回收线程会自动离开。它始终在低级别的状态中运行，用于实时监控和管理系统中的可回收资源。
7. 生命周期：守护进程（Daemon）是运行在后台的一种特殊进程。它独立于控制终端并且周期性地执行某种任务或等待处理某些发生的事件。也就是说守护线程不依赖于终端，但是依赖于系统，与系统"同生共死"。当 JVM 中所有的线程都是守护线程的时候， JVM 就可以退出了；如果还有一个或以上的非守护线程则 JVM 不会退出

35、什么是乐观锁

乐观锁是一种乐观思想，即认为读多写少，遇到并发写的可能性低，每次去拿数据的时候都认为别人不会修改，所以不会上锁，但是在更新的时候会判断一下在此期间别人有没有去更新这个数据，采取在写时先读出当前版本号，然后加锁操作（比较跟上一次的版本号，如果一样则更新），如果失败则要重复读-比较-写的操作。

java 中的乐观锁基本都是通过 CAS 操作实现的， CAS 是一种更新的原子操作， 比较当前值跟传入值是否一样，一样则更新，否则失败。

36、什么是悲观锁

悲观锁是就是悲观思想，即认为写多，遇到并发写的可能性高，每次去拿数据的时候都认为别人会修改，所以每次在读写数据的时候都会上锁，这样别人想读写这个数据就会 block 直到拿到锁。java中的悲观锁就是Synchronized,AQS框架下的锁则是先尝试cas乐观锁去获取锁，获取不到，才会转换为悲观锁，如 RetreenLock。

37、什么是自旋锁

自旋锁原理非常简单， 如果持有锁的线程能在很短时间内释放锁资源，那么那些等待竞争锁的线程就不需要做内核态和用户态之间的切换进入阻塞挂起状态，它们只需要等一等（自旋），等持有锁的线程释放锁后即可立即获取锁，这样就避免用户线程和内核的切换的消耗。

线程自旋是需要消耗 cup 的，说白了就是让 cup 在做无用功，如果一直获取不到锁，那线程也不能一直占用 cup 自旋做无用功，所以需要设定一个自旋等待的最大时间。

如果持有锁的线程执行的时间超过自旋等待的最大时间扔没有释放锁，就会导致其它争用锁的线程在最大等待时间内还是获取不到锁，这时争用线程会停止自旋进入阻塞状态。

自旋锁的优缺点
自旋锁尽可能的减少线程的阻塞，这对于锁的竞争不激烈，且占用锁时间非常短的代码块来说性能能大幅度的提升，因为自旋的消耗会小于线程阻塞挂起再唤醒的操作的消耗，这些操作会导致线程发生两次上下文切换！

但是如果锁的竞争激烈，或者持有锁的线程需要长时间占用锁执行同步块，这时候就不适合使用自旋锁了，因为自旋锁在获取锁前一直都是占用 cpu 做无用功，占着 XX 不 XX，同时有大量线程在竞争一个锁，会导致获取锁的时间很长，线程自旋的消耗大于线程阻塞挂起操作的消耗，其它需要 cup 的线程又不能获取到 cpu，造成 cpu 的浪费。所以这种情况下我们要关闭自旋锁；

自旋锁时间阈值（1.6 引入了适应性自旋锁）
自旋锁的目的是为了占着 CPU 的资源不释放，等到获取到锁立即进行处理。但是如何去选择自旋的执行时间呢？如果自旋执行时间太长，会有大量的线程处于自旋状态占用 CPU 资源，进而会影响整体系统的性能。因此自旋的周期选的额外重要！

JVM 对于自旋周期的选择， jdk1.5 这个限度是一定的写死的， 在 1.6 引入了适应性自旋锁，适应性自旋锁意味着自旋的时间不在是固定的了，而是由前一次在同一个锁上的自旋时间以及锁的拥有者的状态来决定，基本认为一个线程上下文切换的时间是最佳的一个时间，同时 JVM 还针对当前 CPU 的负荷情况做了较多的优化， 如果平均负载小于 CPUs 则一直自旋， 如果有超过(CPUs/2)个线程正在自旋，则后来线程直接阻塞， 如果正在自旋的线程发现 Owner 发生了变化则延迟自旋时间（自旋计数）或进入阻塞， 如果 CPU 处于节电模式则停止自旋， 自旋时间的最坏情况是 CPU的存储延迟（CPU A 存储了一个数据，到 CPU B 得知这个数据直接的时间差）， 自旋时会适当放弃线程优先级之间的差异。

自旋锁的开启
JDK1.6 中-XX:+UseSpinning 开启；
-XX:PreBlockSpin=10 为自旋次数；
JDK1.7 后，去掉此参数，由 jvm 控制；

38、Synchronized 同步锁

synchronized 它可以把任意一个非 NULL 的对象当作锁。 他属于独占式的悲观锁，同时属于可重入锁。 Synchronized 作用范围**

1. 作用于方法时，锁住的是对象的实例(this)；
2. 当作用于静态方法时，锁住的是Class实例，又因为Class的相关数据存储在永久带PermGen（jdk1.8 则是 metaspace），永久带是全局共享的，因此静态方法锁相当于类的一个全局锁，会锁所有调用该方法的线程；
3. synchronized 作用于一个对象实例时，锁住的是所有以该对象为锁的代码块。 它有多个队列，当多个线程一起访问某个对象监视器的时候，对象监视器会将这些线程存储在不同的容器中。

Synchronized 核心组件
1) Wait Set：哪些调用 wait 方法被阻塞的线程被放置在这里；
2) Contention List： 竞争队列，所有请求锁的线程首先被放在这个竞争队列中；
3) Entry List： Contention List 中那些有资格成为候选资源的线程被移动到 Entry List 中；
4) OnDeck：任意时刻， 最多只有一个线程正在竞争锁资源，该线程被成为 OnDeck；
5) Owner：当前已经获取到所资源的线程被称为 Owner；
6) !Owner：当前释放锁的线程。

Synchronized 实现

1. JVM 每次从队列的尾部取出一个数据用于锁竞争候选者（OnDeck），但是并发情况下，ContentionList 会被大量的并发线程进行 CAS 访问，为了降低对尾部元素的竞争， JVM 会将一部分线程移动到 EntryList 中作为候选竞争线程。
2. Owner 线程会在 unlock 时，将 ContentionList 中的部分线程迁移到 EntryList 中，并指定 EntryList 中的某个线程为 OnDeck 线程（一般是最先进去的那个线程）。
3. Owner 线程并不直接把锁传递给 OnDeck 线程，而是把锁竞争的权利交给 OnDeck，OnDeck 需要重新竞争锁。这样虽然牺牲了一些公平性，但是能极大的提升系统的吞吐量，在 JVM 中，也把这种选择行为称之为“竞争切换”。
4. OnDeck 线程获取到锁资源后会变为 Owner 线程，而没有得到锁资源的仍然停留在 EntryList 中。如果 Owner 线程被 wait 方法阻塞，则转移到 WaitSet 队列中，直到某个时刻通过 notify 或者 notifyAll 唤醒，会重新进去 EntryList 中。
5. 处于 ContentionList、 EntryList、 WaitSet 中的线程都处于阻塞状态，该阻塞是由操作系统来完成的（Linux 内核下采用 pthread_mutex_lock 内核函数实现的）。
6. Synchronized 是非公平锁。 Synchronized 在线程进入 ContentionList 时， 等待的线程会先尝试自旋获取锁，如果获取不到就进入 ContentionList，这明显对于已经进入队列的线程是不公平的，还有一个不公平的事情就是自旋获取锁的线程还可能直接抢占 OnDeck 线程的锁资源。
   参考： https://blog.csdn.net/zqz_zqz/article/details/70233767
7. 每个对象都有个 monitor 对象， 加锁就是在竞争 monitor 对象，代码块加锁是在前后分别加上 monitorenter 和 monitorexit 指令来实现的，方法加锁是通过一个标记位来判断的
8. synchronized 是一个重量级操作，需要调用操作系统相关接口，性能是低效的，有可能给线程加锁消耗的时间比有用操作消耗的时间更多。
9. Java1.6， synchronized 进行了很多的优化， 有适应自旋、锁消除、锁粗化、轻量级锁及偏向锁等，效率有了本质上的提高。在之后推出的 Java1.7 与 1.8 中，均对该关键字的实现机理做了优化。引入了偏向锁和轻量级锁。都是在对象头中有标记位，不需要经过操作系统加锁。
10. 锁可以从偏向锁升级到轻量级锁，再升级到重量级锁。这种升级过程叫做锁膨胀；
11. JDK 1.6 中默认是开启偏向锁和轻量级锁，可以通过-XX:-UseBiasedLocking 来禁用偏向锁。

39、ReentrantLock

ReentantLock 继承接口 Lock 并实现了接口中定义的方法， 他是一种可重入锁， 除了能完成 synchronized 所能完成的所有工作外，还提供了诸如可响应中断锁、可轮询锁请求、定时锁等避免多线程死锁的方法。
Lock 接口的主要方法

void lock(): 执行此方法时, 如果锁处于空闲状态, 当前线程将获取到锁. 相反, 如果锁已经被其他线程持有, 将禁用当前线程, 直到当前线程获取到锁.

boolean tryLock()： 如果锁可用, 则获取锁, 并立即返回 true, 否则返回 false. 该方法和 lock()的区别在于, tryLock()只是"试图"获取锁, 如果锁不可用, 不会导致当前线程被禁用, 当前线程仍然继续往下执行代码. 而 lock()方法则是一定要获取到锁, 如果锁不可用, 就一直等待, 在未获得锁之前,当前线程并不继续向下执行.

void unlock()：执行此方法时, 当前线程将释放持有的锁. 锁只能由持有者释放, 如果线程并不持有锁, 却执行该方法, 可能导致异常的发生.

Condition newCondition()： 条件对象，获取等待通知组件。该组件和当前的锁绑定，当前线程只有获取了锁，才能调用该组件的 await()方法，而调用后，当前线程将缩放锁。

getHoldCount() ： 查询当前线程保持此锁的次数，也就是执行此线程执行 lock 方法的次数。

getQueueLength（） ： 返回正等待获取此锁的线程估计数，比如启动 10 个线程， 1 个线程获得锁，此时返回的是 9

getWaitQueueLength： （Condition condition）返回等待与此锁相关的给定条件的线程估计数。比如 10 个线程，用同一个 condition 对象，并且此时这 10 个线程都执行了 condition 对象的 await 方法，那么此时执行此方法返回 10

hasWaiters(Condition condition)： 查询是否有线程等待与此锁有关的给定条件(condition)，对于指定 contidion 对象，有多少线程执行了 condition.await 方法

hasQueuedThread(Thread thread)： 查询给定线程是否等待获取此锁

hasQueuedThreads()： 是否有线程等待此锁

isFair()： 该锁是否公平锁

isHeldByCurrentThread()： 当前线程是否保持锁锁定，线程的执行 lock 方法的前后分别是 false 和 true

isLock()： 此锁是否有任意线程占用

lockInterruptibly（）： 如果当前线程未被中断，获取锁

tryLock（）： 尝试获得锁，仅在调用时锁未被线程占用，获得锁

tryLock(long timeout TimeUnit unit)： 如果锁在给定等待时间内没有被另一个线程保持，则获取该锁。

非公平锁
JVM 按随机、就近原则分配锁的机制则称为不公平锁， ReentrantLock 在构造函数中提供了是否公平锁的初始化方式，默认为非公平锁。 非公平锁实际执行的效率要远远超出公平锁，除非程序有特殊需要，否则最常用非公平锁的分配机制。

公平锁
公平锁指的是锁的分配机制是公平的，通常先对锁提出获取请求的线程会先被分配到锁，ReentrantLock 在构造函数中提供了是否公平锁的初始化方式来定义公平锁。

40、Condition 类和 Object 类锁方法区别区别

1. Condition 类的 awiat 方法和 Object 类的 wait 方法等效
2. Condition 类的 signal 方法和 Object 类的 notify 方法等效
3. Condition 类的 signalAll 方法和 Object 类的 notifyAll 方法等效
4. ReentrantLock 类可以唤醒指定条件的线程，而 object 的唤醒是随机的

41、tryLock 和 lock 和 lockInterruptibly 的区别

1. tryLock 能获得锁就返回 true，不能就立即返回 false， tryLock(long timeout,TimeUnit unit)，可以增加时间限制，如果超过该时间段还没获得锁，返回 false
2. lock 能获得锁就返回 true，不能的话一直等待获得锁
3. lock 和 lockInterruptibly，如果两个线程分别执行这两个方法，但此时中断这两个线程，lock 不会抛出异常，而 lockInterruptibly 会抛出异常。

42、Semaphore 信号量

Semaphore 是一种基于计数的信号量。它可以设定一个阈值，基于此，多个线程竞争获取许可信号，做完自己的申请后归还，超过阈值后，线程申请许可信号将会被阻塞。 Semaphore 可以用来构建一些对象池，资源池之类的， 比如数据库连接池

实现互斥锁（计数器为 1）
我们也可以创建计数为 1 的 Semaphore，将其作为一种类似互斥锁的机制，这也叫二元信号量，表示两种互斥状态。
代码实现

```
// 创建一个计数阈值为 5 的信号量对象
// 只能 5 个线程同时访问
Semaphore semp = new Semaphore(5);
try { // 申请许可
        semp.acquire();
        try {
    // 业务逻辑
        } catch (Exception e) {
        } finally {
    // 释放许可
            semp.release();
        }
    } catch (InterruptedException e) {
    }
```

43、Semaphore 与 ReentrantLock 区别

Semaphore 基本能完成 ReentrantLock 的所有工作，使用方法也与之类似，通过 acquire()与 release()方法来获得和释放临界资源。经实测， Semaphone.acquire()方法默认为可响应中断锁，与 ReentrantLock.lockInterruptibly()作用效果一致，也就是说在等待临界资源的过程中可以被 Thread.interrupt()方法中断。

此外， Semaphore 也实现了可轮询的锁请求与定时锁的功能，除了方法名 tryAcquire 与 tryLock 不同，其使用方法与 ReentrantLock 几乎一致。 Semaphore 也提供了公平与非公平锁的机制，也可在构造函数中进行设定。

Semaphore 的锁释放操作也由手动进行，因此与 ReentrantLock 一样，为避免线程因抛出异常而无法正常释放锁的情况发生，释放锁的操作也必须在 finally 代码块中完成。

44、可重入锁（递归锁）

本文里面讲的是广义上的可重入锁，而不是单指 JAVA 下的 ReentrantLock。 可重入锁，也叫做递归锁，指的是同一线程 外层函数获得锁之后 ，内层递归函数仍然有获取该锁的代码，但不受影响。在 JAVA 环境下 ReentrantLock 和 synchronized 都是 可重入锁。

45、公平锁与非公平锁

公平锁（Fair）
加锁前检查是否有排队等待的线程，优先排队等待的线程，先来先得
非公平锁（Nonfair）
加锁时不考虑排队等待问题，直接尝试获取锁，获取不到自动到队尾等待

1. 非公平锁性能比公平锁高 5~10 倍，因为公平锁需要在多核的情况下维护一个队列
2. Java 中的 synchronized 是非公平锁， ReentrantLock 默认的 lock()方法采用的是非公平锁。

46、ReadWriteLock 读写锁

为了提高性能， Java 提供了读写锁，在读的地方使用读锁，在写的地方使用写锁，灵活控制，如果没有写锁的情况下，读是无阻塞的,在一定程度上提高了程序的执行效率。 读写锁分为读锁和写锁，多个读锁不互斥，读锁与写锁互斥，这是由 jvm 自己控制的，你只要上好相应的锁即可。

读锁
如果你的代码只读数据，可以很多人同时读，但不能同时写，那就上读锁

写锁
如果你的代码修改数据，只能有一个人在写，且不能同时读取，那就上写锁。总之，读的时候上读锁，写的时候上写锁！

Java 中读写锁有个接口 java.util.concurrent.locks.ReadWriteLock ，也有具体的实现 ReentrantReadWriteLock。

47、共享锁和独占锁

java 并发包提供的加锁模式分为独占锁和共享锁。

独占锁
独占锁模式下，每次只能有一个线程能持有锁， ReentrantLock 就是以独占方式实现的互斥锁。独占锁是一种悲观保守的加锁策略，它避免了读/读冲突，如果某个只读线程获取锁，则其他读线程都只能等待，这种情况下就限制了不必要的并发性，因为读操作并不会影响数据的一致性。

共享锁
共享锁则允许多个线程同时获取锁，并发访问 共享资源，如： ReadWriteLock。 共享锁则是一种乐观锁，它放宽了加锁策略，允许多个执行读操作的线程同时访问共享资源。

1. AQS 的内部类 Node 定义了两个常量 SHARED 和 EXCLUSIVE，他们分别标识 AQS 队列中等待线程的锁获取模式。
2. java 的并发包中提供了 ReadWriteLock，读-写锁。它允许一个资源可以被多个读操作访问，或者被一个 写操作访问，但两者不能同时进行。

48、重量级锁（Mutex Lock）

Synchronized 是通过对象内部的一个叫做监视器锁（monitor）来实现的。但是监视器锁本质又是依赖于底层的操作系统的 Mutex Lock 来实现的。

而操作系统实现线程之间的切换这就需要从用户态转换到核心态，这个成本非常高，状态之间的转换需要相对比较长的时间，这就是为什么 Synchronized 效率低的原因。

因此， 这种依赖于操作系统 Mutex Lock 所实现的锁我们称之为“重量级锁”。 JDK 中对 Synchronized 做的种种优化，其核心都是为了减少这种重量级锁的使用。

JDK1.6 以后，为了减少获得锁和释放锁所带来的性能消耗，提高性能，引入了“轻量级锁”和“偏向锁”。

49、轻量级锁

锁的状态总共有四种：无锁状态、偏向锁、轻量级锁和重量级锁。
锁升级
随着锁的竞争，锁可以从偏向锁升级到轻量级锁，再升级的重量级锁（但是锁的升级是单向的，也就是说只能从低到高升级，不会出现锁的降级）。

“轻量级”是相对于使用操作系统互斥量来实现的传统锁而言的。但是，首先需要强调一点的是，轻量级锁并不是用来代替重量级锁的，它的本意是在没有多线程竞争的前提下，减少传统的重量级锁使用产生的性能消耗。

在解释轻量级锁的执行过程之前， 先明白一点，轻量级锁所适应的场景是线程交替执行同步块的情况，如果存在同一时间访问同一锁的情况，就会导致轻量级锁膨胀为重量级锁

50、偏向锁

Hotspot 的作者经过以往的研究发现大多数情况下锁不仅不存在多线程竞争，而且总是由同一线程多次获得。 偏向锁的目的是在某个线程获得锁之后，消除这个线程锁重入（CAS）的开销，看起来让这个线程得到了偏护。

引入偏向锁是为了在无多线程竞争的情况下尽量减少不必要的轻量级锁执行路径，因为轻量级锁的获取及释放依赖多次 CAS 原子指令， 而偏向锁只需要在置换ThreadID 的时候依赖一次 CAS 原子指令（由于一旦出现多线程竞争的情况就必须撤销偏向锁，所以偏向锁的撤销操作的性能损耗必须小于节省下来的 CAS 原子指令的性能消耗）。

上面说过， 轻量级锁是为了在线程交替执行同步块时提高性能， 而偏向锁则是在只有一个线程执行同步块时进一步提高性能

51、分段锁

分段锁也并非一种实际的锁，而是一种思想 ConcurrentHashMap 是学习分段锁的最好实践

52、锁优化

减少锁持有时间
只用在有线程安全要求的程序上加锁

减小锁粒度
将大对象（这个对象可能会被很多线程访问），拆成小对象，大大增加并行度，降低锁竞争。降低了锁的竞争，偏向锁，轻量级锁成功率才会提高。最最典型的减小锁粒度的案例就是ConcurrentHashMap。

锁分离
最常见的锁分离就是读写锁 ReadWriteLock，根据功能进行分离成读锁和写锁，这样读读不互斥，读写互斥，写写互斥，即保证了线程安全，又提高了性能，具体也请查看[高并发 Java 五] JDK 并发包 1。读写分离思想可以延伸，只要操作互不影响，锁就可以分离。比如 LinkedBlockingQueue 从头部取出，从尾部放数据

锁粗化
通常情况下，为了保证多线程间的有效并发，会要求每个线程持有锁的时间尽量短，即在使用完公共资源后，应该立即释放锁。但是，凡事都有一个度， 如果对同一个锁不停的进行请求、同步和释放，其本身也会消耗系统宝贵的资源，反而不利于性能的优化 。

锁消除
锁消除是在编译器级别的事情。 在即时编译器时，如果发现不可能被共享的对象，则可以消除这些对象的锁操作，多数是因为程序员编码不规范引起。
参考： https://www.jianshu.com/p/39628e1180a9

53、线程基本方法

线程相关的基本方法有 wait， notify， notifyAll， sleep， join， yield 等。

54、线程等待（wait）

调用该方法的线程进入 WAITING 状态，只有等待另外线程的通知或被中断才会返回，需要注意的是调用 wait()方法后， 会释放对象的锁。因此， wait 方法一般用在同步方法或同步代码块中。

55、线程睡眠（sleep）

sleep 导致当前线程休眠，与 wait 方法不同的是 sleep 不会释放当前占有的锁,sleep(long)会导致线程进入 TIMED-WATING 状态，而 wait()方法会导致当前线程进入 WATING 状态

56、线程让步（yield）

yield 会使当前线程让出 CPU 执行时间片，与其他线程一起重新竞争 CPU 时间片。一般情况下，优先级高的线程有更大的可能性成功竞争得到 CPU 时间片， 但这又不是绝对的，有的操作系统对线程优先级并不敏感。

57、线程中断（interrupt）

中断一个线程，其本意是给这个线程一个通知信号，会影响这个线程内部的一个中断标识位。 这个线程本身并不会因此而改变状态(如阻塞，终止等)。

1. 调用 interrupt()方法并不会中断一个正在运行的线程。也就是说处于 Running 状态的线程并不会因为被中断而被终止，仅仅改变了内部维护的中断标识位而已。
2. 若调用 sleep()而使线程处于 TIMED-WATING 状态，这时调用 interrupt()方法，会抛出 InterruptedException,从而使线程提前结束 TIMED-WATING 状态。
3. 许多声明抛出 InterruptedException 的方法(如 Thread.sleep(long mills 方法))，抛出异常前，都会清除中断标识位，所以抛出异常后，调用 isInterrupted()方法将会返回 false。
4. 中断状态是线程固有的一个标识位，可以通过此标识位安全的终止线程。比如,你想终止一个线程 thread 的时候，可以调用 thread.interrupt()方法，在线程的 run 方法内部可以根据 thread.isInterrupted()的值来优雅的终止线程。

58、Join 等待其他线程终止

join() 方法，等待其他线程终止，在当前线程中调用一个线程的 join() 方法，则当前线程转为阻塞状态，回到另一个线程结束，当前线程再由阻塞状态变为就绪状态，等待 cpu 的宠幸。

59、为什么要用 join()方法？

很多情况下，主线程生成并启动了子线程，需要用到子线程返回的结果，也就是需要主线程需要在子线程结束后再结束，这时候就要用到 join() 方法 。

```
System.out.println(Thread.currentThread().getName() + "线程运行开始!");
Thread6 thread1 = new Thread6();
thread1.setName("线程 B");
thread1.join();
System.out.println("这时 thread1 执行完毕之后才能执行主线程");
```

60、线程唤醒（notify）

Object 类中的 notify() 方法， 唤醒在此对象监视器上等待的单个线程，如果所有线程都在此对象上等待，则会选择唤醒其中一个线程，选择是任意的，并在对实现做出决定时发生，线程通过调用其中一个 wait() 方法，在对象的监视器上等待， 直到当前的线程放弃此对象上的锁定，才能继续执行被唤醒的线程，被唤醒的线程将以常规方式与在该对象上主动同步的其他所有线程进行竞争。类似的方法还有 notifyAll() ，唤醒再次监视器上等待的所有线程。

61、线程其他方法

1. sleep()：强迫一个线程睡眠N毫秒。
2. isAlive()： 判断一个线程是否存活。
3. join()： 等待线程终止。
4. activeCount()： 程序中活跃的线程数。
5. enumerate()： 枚举程序中的线程。
6. currentThread()： 得到当前线程。
7. isDaemon()： 一个线程是否为守护线程。
8. setDaemon()： 设置一个线程为守护线程。 (用户线程和守护线程的区别在于，是否等待主线程依赖于主线程结束而结束)
9. setName()： 为线程设置一个名称。
10. wait()： 强迫一个线程等待。
    11.notify()： 通知一个线程继续运行。
11. setPriority()： 设置一个线程的优先级。
12. getPriority()::获得一个线程的优先级。

62、进程

（有时候也称做任务）是指一个程序运行的实例。在 Linux 系统中，线程就是能并行运行并且与他们的父进程（创建他们的进程）共享同一地址空间（一段内存区域）和其他资源的轻量级的进程。

63、上下文

是指某一时间点 CPU 寄存器和程序计数器的内容。

64、寄存器

是 CPU 内部的数量较少但是速度很快的内存（与之对应的是 CPU 外部相对较慢的 RAM 主内存）。寄存器通过对常用值（通常是运算的中间值）的快速访问来提高计算机程序运行的速度。

65、程序计数器

是一个专用的寄存器， 用于表明指令序列中 CPU 正在执行的位置，存的值为正在执行的指令的位置或者下一个将要被执行的指令的位置，具体依赖于特定的系统。

66、PCB-“切换桢”

上下文切换可以认为是内核（操作系统的核心）在 CPU 上对于进程（包括线程）进行切换，上下文切换过程中的信息是保存在进程控制块（PCB, process control block）中的。 PCB 还经常被称作“切换桢”（switchframe）。 信息会一直保存到 CPU 的内存中，直到他们被再次使用。

67、上下文切换的活动

1.挂起一个进程，将这个进程在 CPU 中的状态（上下文）存储于内存中的某处。

2.在内存中检索下一个进程的上下文并将其在 CPU 的寄存器中恢复。

3.跳转到程序计数器所指向的位置（即跳转到进程被中断时的代码行），以恢复该进程在程序中。

68、引起线程上下文切换的原因

1. 当前执行任务的时间片用完之后，系统 CPU 正常调度下一个任务；
2. 当前执行任务碰到 IO 阻塞，调度器将此任务挂起，继续下一任务；
3. 多个任务抢占锁资源，当前任务没有抢到锁资源，被调度器挂起，继续下一任务；
4. 用户代码挂起当前任务，让出 CPU 时间；
5. 硬件中断；

69、同步锁

当多个线程同时访问同一个数据时，很容易出现问题。为了避免这种情况出现，我们要保证线程同步互斥，就是指并发执行的多个线程，在同一时间内只允许一个线程访问共享数据。 Java 中可以使用 synchronized 关键字来取得一个对象的同步锁。

70、死锁

何为死锁，就是多个线程同时被阻塞，它们中的一个或者全部都在等待某个资源被释放。

71、线程池原理

线程池做的工作主要是控制运行的线程的数量，处理过程中将任务放入队列，然后在线程创建后启动这些任务，如果线程数量超过了最大数量超出数量的线程排队等候，等其它线程执行完毕，再从队列中取出任务来执行。 他的主要特点为： 线程复用； 控制最大并发数； 管理线程。

72、线程复

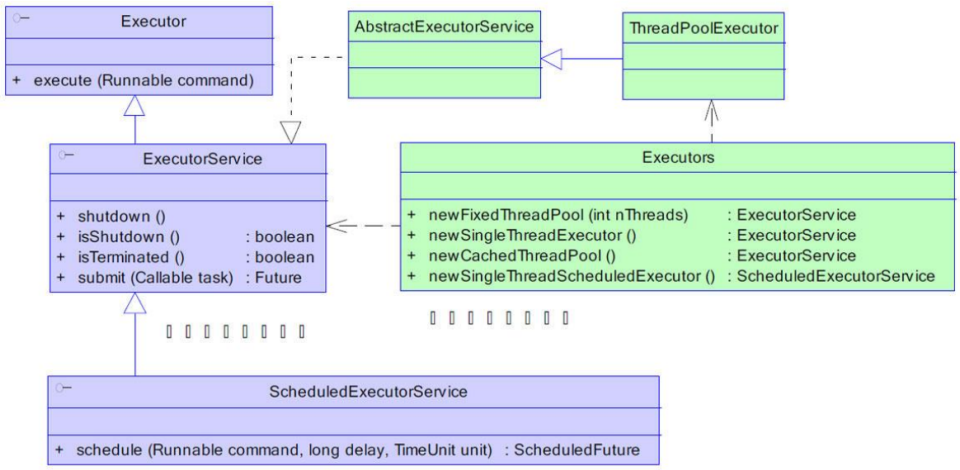
每一个 Thread 的类都有一个 start 方法。 当调用 start 启动线程时 Java 虚拟机会调用该类的 run 方法。 那么该类的 run() 方法中就是调用了 Runnable 对象的 run() 方法。 我们可以继承重写 Thread 类，在其 start 方法中添加不断循环调用传递过来的 Runnable 对象。 这就是线程池的实现原理。 循环方法中不断获取 Runnable 是用 Queue 实现的，在获取下一个 Runnable 之前可以是阻塞的 。

73、线程池的组成

一般的线程池主要分为以下 4 个组成部分：

1. 线程池管理器：用于创建并管理线程池
2. 工作线程：线程池中的线程
3. 任务接口：每个任务必须实现的接口，用于工作线程调度其运行
4. 任务队列：用于存放待处理的任务，提供一种缓冲机制

Java 中的线程池是通过 Executor 框架实现的，该框架中用到了 Executor， Executors，ExecutorService， ThreadPoolExecutor ， Callable 和 Future、 FutureTask 这几个类。



ThreadPoolExecutor 的构造方法如下：

```
public ThreadPoolExecutor(int corePoolSize,int maximumPoolSize, long keepAliveTime,
    TimeUnit unit, BlockingQueue<Runnable> workQueue) {

    this(corePoolSize, maximumPoolSize, keepAliveTime, unit, workQueue,
    Executors.defaultThreadFactory(), defaultHandler);
}
```

1. corePoolSize：指定了线程池中的线程数量。
2. maximumPoolSize：指定了线程池中的最大线程数量。
3. keepAliveTime：当前线程池数量超过 corePoolSize 时，多余的空闲线程的存活时间，即多次时间内会被销毁。
4. unit： keepAliveTime 的单位。
5. workQueue：任务队列，被提交但尚未被执行的任务。
6. threadFactory：线程工厂，用于创建线程，一般用默认的即可。
7. handler：拒绝策略，当任务太多来不及处理，如何拒绝任务。

74、拒绝策略

线程池中的线程已经用完了，无法继续为新任务服务，同时，等待队列也已经排满了，再也塞不下新任务了。这时候我们就需要拒绝策略机制合理的处理这个问题。
JDK 内置的拒绝策略如下：

1. AbortPolicy ： 直接抛出异常，阻止系统正常运行。
2. CallerRunsPolicy ： 只要线程池未关闭，该策略直接在调用者线程中，运行当前被丢弃的任务。显然这样做不会真的丢弃任务，但是，任务提交线程的性能极有可能会急剧下降。
3. DiscardOldestPolicy ： 丢弃最老的一个请求，也就是即将被执行的一个任务，并尝试再次提交当前任务。
4. DiscardPolicy ： 该策略默默地丢弃无法处理的任务，不予任何处理。如果允许任务丢失，这是最好的一种方案。
   以上内置拒绝策略均实现了 RejectedExecutionHandler 接口，若以上策略仍无法满足实际需要，完全可以自己扩展 RejectedExecutionHandler 接口。

75、Java 线程池工作过程

1. 线程池刚创建时，里面没有一个线程。任务队列是作为参数传进来的。不过，就算队列里面有任务，线程池也不会马上执行它们。
2. 当调用 execute() 方法添加一个任务时，线程池会做如下判断：
   a) 如果正在运行的线程数量小于 corePoolSize，那么马上创建线程运行这个任务；
   b) 如果正在运行的线程数量大于或等于 corePoolSize，那么将这个任务放入队列；
   c) 如果这时候队列满了，而且正在运行的线程数量小于 maximumPoolSize，那么还是要创建非核心线程立刻运行这个任务；
   d) 如果队列满了，而且正在运行的线程数量大于或等于 maximumPoolSize，那么线程池会抛出异常 RejectExecutionException。
3. 当一个线程完成任务时，它会从队列中取下一个任务来执行。
4. 当一个线程无事可做，超过一定的时间（keepAliveTime）时，线程池会判断，如果当前运行的线程数大于 corePoolSize，那么这个线程就被停掉。所以线程池的所有任务完成后，它最终会收缩到 corePoolSize 的大小。

76、JAVA 阻塞队列原理

阻塞队列，关键字是阻塞，先理解阻塞的含义，在阻塞队列中，线程阻塞有这样的两种情况：

1. 当队列中没有数据的情况下，消费者端的所有线程都会被自动阻塞（挂起），直到有数据放入队列。
2. 当队列中填满数据的情况下，生产者端的所有线程都会被自动阻塞（挂起），直到队列中有空的位置，线程被自动唤醒。

阻塞队列的主要方法：

方法类型	抛出异常	特殊值	阻塞	超时
插入	add(e)	offer(e)	put(e)	offer(e,time,unit)
移除	remove()	poll()	take()	poll(time,unit)
检查	element()	peek()	不可用	不可用

抛出异常：抛出一个异常；
特殊值：返回一个特殊值（null 或 false,视情况而定）
则塞：在成功操作之前，一直阻塞线程
超时：放弃前只在最大的时间内阻塞

插入操作

1： public abstract boolean add(E paramE)： 将指定元素插入此队列中（如果立即可行且不会违反容量限制），成功时返回 true，如果当前没有可用的空间，则抛出 IllegalStateException。如果该元素是 NULL，则会抛出 NullPointerException 异常。

2： public abstract boolean offer(E paramE)： 将指定元素插入此队列中（如果立即可行且不会违反容量限制），成功时返回 true，如果当前没有可用的空间，则返回 false。

3： public abstract void put(E paramE) throws InterruptedException： 将指定元素插入此队列中，将等待可用的空间（如果有必要）

```
public void put(E paramE) throws InterruptedException {
    checkNotNull(paramE);
    ReentrantLock localReentrantLock = this.lock;
    localReentrantLock.lockInterruptibly();
    try {
            while (this.count == this.items.length)
            this.notFull.await();//如果队列满了，则线程阻塞等待
            enqueue(paramE);
            localReentrantLock.unlock();
        } finally {
            localReentrantLock.unlock();
        }
}
```

4： offer(E o, long timeout, TimeUnit unit)： 可以设定等待的时间， 如果在指定的时间内， 还不能往队列中加入 BlockingQueue， 则返回失败。

获取数据操作：

1： poll(time):取走 BlockingQueue 里排在首位的对象,若不能立即取出,则可以等 time 参数规定的时间,取不到时返回 null;

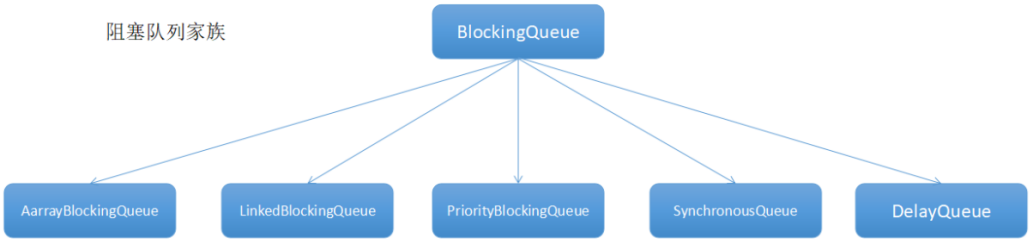
2： poll(long timeout, TimeUnit unit)： 从 BlockingQueue 取出一个队首的对象， 如果在指定时间内， 队列一旦有数据可取， 则立即返回队列中的数据。否则直到时间超时还没有数据可取，返回失败。

3： take():取走 BlockingQueue 里排在首位的对象,若 BlockingQueue 为空,阻断进入等待状态直到 BlockingQueue 有新的数据被加入。

4.drainTo():一次性从 BlockingQueue 获取所有可用的数据对象（还可以指定获取数据的个数），通过该方法，可以提升获取数据效率；不需要多次分批加锁或释放锁。

77、Java 中的阻塞队列

1. ArrayBlockingQueue ：由数组结构组成的有界阻塞队列。
2. LinkedBlockingQueue ：由链表结构组成的有界阻塞队列。
3. PriorityBlockingQueue ：支持优先级排序的无界阻塞队列。
4. DelayQueue：使用优先级队列实现的无界阻塞队列。
5. SynchronousQueue：不存储元素的阻塞队列。
6. LinkedTransferQueue：由链表结构组成的无界阻塞队列。
7. LinkedBlockingDeque：由链表结构组成的双向阻塞队列



78、ArrayBlockingQueue（公平、非公平）

用数组实现的有界阻塞队列。此队列按照先进先出（FIFO）的原则对元素进行排序。 默认情况下不保证访问者公平的访问队列，所谓公平访问队列是指阻塞的所有生产者线程或消费者线程，当队列可用时，可以按照阻塞的先后顺序访问队列，即先阻塞的生产者线程，可以先往队列里插入元素，先阻塞的消费者线程，可以先从队列里获取元素。通常情况下为了保证公平性会降低吞吐量。我们可以使用以下代码创建一个公平的阻塞队列

```
ArrayBlockingQueue fairQueue = new ArrayBlockingQueue(1000,true);
```

79、LinkedBlockingQueue（两个独立锁提高并发）

基于链表的阻塞队列，同 ArrayListBlockingQueue 类似，此队列按照先进先出（FIFO）的原则对元素进行排序。而 LinkedBlockingQueue 之所以能够高效的处理并发数据，还因为其对于生产者端和消费者端分别采用了独立的锁来控制数据同步，这也意味着在高并发的情况下生产者和消费者可以并行地操作队列中的数据，以此来提高整个队列的并发性能。LinkedBlockingQueue 会默认一个类似无限大小的容量（Integer.MAX_VALUE）

80、PriorityBlockingQueue（compareTo 排序实现优先）

是一个支持优先级的无界队列。默认情况下元素采取自然顺序升序排列。 可以自定义实现 compareTo()方法来指定元素进行排序规则，或者初始化 PriorityBlockingQueue 时，指定构造参数 Comparator 来对元素进行排序。需要注意的是不能保证同优先级元素的顺序。

81、DelayQueue（缓存失效、定时任务 ）

是一个支持延时获取元素的无界阻塞队列。队列使用 PriorityQueue 来实现。队列中的元素必须实现 Delayed 接口，在创建元素时可以指定多久才能从队列中获取当前元素。只有在延迟期满时才能从队列中提取元素。我们可以将 DelayQueue 运用在以下应用场景：

1. 缓存系统的设计：可以用 DelayQueue 保存缓存元素的有效期，使用一个线程循环查询 DelayQueue，一旦能从 DelayQueue 中获取元素时，表示缓存有效期到了。
2. 定时任务调度：使用 DelayQueue 保存当天将会执行的任务和执行时间，一旦从 DelayQueue 中获取到任务就开始执行，从比如 TimerQueue 就是使用 DelayQueue 实现的

82、SynchronousQueue（不存储数据、可用于传递数据）

是一个不存储元素的阻塞队列。每一个 put 操作必须等待一个 take 操作，否则不能继续添加元素。SynchronousQueue 可以看成是一个传球手，负责把生产者线程处理的数据直接传递给消费者线程。队列本身并不存储任何元素，非常适合于传递性场景,比如在一个线程中使用的数据，传递给另外一个线程使用，SynchronousQueue 的吞吐量高于 LinkedBlockingQueue 和 ArrayBlockingQueue。

83、LinkedTransferQueue

是一个由链表结构组成的无界阻塞 TransferQueue 队列。相对于其他阻塞队列，LinkedTransferQueue 多了 tryTransfer 和 transfer 方法。

1. transfer 方法：如果当前有消费者正在等待接收元素（消费者使用 take()方法或带时间限制的 poll()方法时），transfer 方法可以把生产者传入的元素立刻 transfer（传输）给消费者。如果没有消费者在等待接收元素，transfer 方法会将元素存放在队列的 tail 节点，并等到该元素被消费者消费了才返回。
2. tryTransfer 方法。则是用来试探下生产者传入的元素是否能直接传给消费者。如果没有消费者等待接收元素，则返回 false。和 transfer 方法的区别是 tryTransfer 方法无论消费者是否接收，方法立即返回。而 transfer 方法是必须等到消费者消费了才返回。
   对于带有时间限制的 tryTransfer(E e, long timeout, TimeUnit unit)方法，则是试图把生产者传入的元素直接传给消费者，但是如果没有消费者消费该元素则等待指定的时间再返回，如果超时还没消费元素，则返回 false，如果在超时时间内消费了元素，则返回 true。

84、LinkedBlockingDeque

是一个由链表结构组成的双向阻塞队列。所谓双向队列指的你可以从队列的两端插入和移出元素。双端队列因为多了一个操作队列的入口，在多线程同时入队时，也就减少了一半的竞争。相比其他的阻塞队列，LinkedBlockingDeque 多了 addFirst，addLast，offerFirst，offerLast，peekFirst，peekLast 等方法，以 First 单词结尾的方法，表示插入，获取（peek）或移除双端队列的第一个元素。以 Last 单词结尾的方法，表示插入，获取或移除双端队列的最后一个元素。另外插入方法 add 等同于 addLast，移除方法 remove 等效于 removeFirst。但是 take 方法却等同于 takeFirst，不知道是不是 Jdk 的 bug，使用时还是用带有 First 和 Last 后缀的方法更清楚。
在初始化 LinkedBlockingDeque 时可以设置容量防止其过渡膨胀。另外双向阻塞队列可以运用在“工作窃取”模式中。

85、在 java 中守护线程和本地线程区别

java 中的线程分为两种：守护线程（Daemon）和用户线程（User）。

任何线程都可以设置为守护线程和用户线程，通过方法 Thread.setDaemon(boolon)；true 则把该线程设置为守护线程，反之则为用户线程。Thread.setDaemon()必须在 Thread.start()之前调用，否则运行时会抛出异常。

两者的区别：
唯一的区别是判断虚拟机(JVM)何时离开，Daemon 是为其他线程提供服务，如果全部的 User Thread 已经撤离，Daemon 没有可服务的线程，JVM 撤离。也可以理解为守护线程是 JVM 自动创建的线程（但不一定），用户线程是程序创建的线程；比如 JVM 的垃圾回收线程是一个守护线程，当所有线程已经撤离，不再产生垃圾，守护线程自然就没事可干了，当垃圾回收线程是 Java 虚拟机上仅剩的线程时，Java 虚拟机会自动离开。

扩展：

Thread Dump 打印出来的线程信息，含有 daemon 字样的线程即为守护进程，可能会有：服务守护进程、编译守护进程、windows 下的监听 Ctrl+break 的守护进程、Finalizer 守护进程、引用处理守护进程、GC 守护进程。

86、线程与进程的区别？

进程是操作系统分配资源的最小单元，线程是操作系统调度的最小单元。
一个程序至少有一个进程,一个进程至少有一个线程。

87、什么是多线程中的上下文切换？

多线程会共同使用一组计算机上的 CPU，而线程数大于给程序分配的 CPU 数量时，为了让各个线程都有执行的机会，就需要轮转使用 CPU。不同的线程切换使用 CPU 发生的切换数据等就是上下文切换。

88、死锁与活锁的区别，死锁与饥饿的区别？

死锁：是指两个或两个以上的进程（或线程）在执行过程中，因争夺资源而造成的一种互相等待的现象，若无外力作用，它们都将无法推进下去。

产生死锁的必要条件：
1、互斥条件：所谓互斥就是进程在某一时间内独占资源。
2、请求与保持条件：一个进程因请求资源而阻塞时，对已获得的资源保持不放。
3、不剥夺条件:进程已获得资源，在未使用完之前，不能强行剥夺。
4、循环等待条件:若干进程之间形成一种头尾相接的循环等待资源关系。

活锁：任务或者执行者没有被阻塞，由于某些条件没有满足，导致一直重复尝试，失败，尝试，失败。
活锁和死锁的区别在于，处于活锁的实体是在不断的改变状态，所谓的“活”， 而处于死锁的实体表现为等待；活锁有可能自行解开，死锁则不能。

饥饿：一个或者多个线程因为种种原因无法获得所需要的资源，导致一直无法执行的状态。

Java 中导致饥饿的原因：
1、高优先级线程吞噬所有的低优先级线程的 CPU 时间。
2、线程被永久堵塞在一个等待进入同步块的状态，因为其他线程总是能在它之前持续地对该同步块进行访问。
3、线程在等待一个本身也处于永久等待完成的对象(比如调用这个对象的 wait 方法)，因为其他线程总是被持续地获得唤醒。

89、Java 中用到的线程调度算法是什么？

采用时间片轮转的方式。可以设置线程的优先级，会映射到下层的系统上面的优先级上，如非特别需要，尽量不要用，防止线程饥饿。

90、什么是线程组，为什么在 Java 中不推荐使用？

ThreadGroup 类，可以把线程归属到某一个线程组中，线程组中可以有线程对象，也可以有线程组，组中还可以有线程，这样的组织结构有点类似于树的形式。
为什么不推荐使用？因为使用有很多的安全隐患吧，没有具体追究，如果需要使用，推荐使用线程池。

91、为什么使用 Executor 框架？

每次执行任务创建线程 new Thread()比较消耗性能，创建一个线程是比较耗时、耗资源的。

调用 new Thread()创建的线程缺乏管理，被称为野线程，而且可以无限制的创建，线程之间的相互竞争会导致过多占用系统资源而导致系统瘫痪，还有线程之间的频繁交替也会消耗很多系统资源。

接使用 new Thread() 启动的线程不利于扩展，比如定时执行、定期执行、定时定期执行、线程中断等都不便实现。

92、在 Java 中 Executor 和 Executors 的区别？

Executors 工具类的不同方法按照我们的需求创建了不同的线程池，来满足业务的需求。
Executor 接口对象能执行我们的线程任务。
ExecutorService 接口继承了 Executor 接口并进行了扩展，提供了更多的方法我们能获得任务执行的状态并且可以获取任务的返回值。
使用 ThreadPoolExecutor 可以创建自定义线程池。

Future 表示异步计算的结果，他提供了检查计算是否完成的方法，以等待计算的完成，并可以使用 get()方法获取计算的结果。

93、如何在 Windows 和 Linux 上查找哪个线程使用的 CPU 时间最长？

参考：
http://daiguahub.com/2016/07/31/使用 jstack 找出消耗 CPU 最多的线程代码/

94、什么是原子操作？在 Java Concurrency API 中有哪些原子类(atomic classes)？

原子操作（atomic operation）意为"不可被中断的一个或一系列操作"。处理器使用基于对缓存加锁或总线加锁的方式来实现多处理器之间的原子操作。在 Java 中可以通过锁和循环 CAS 的方式来实现原子操作。 CAS 操作——
Compare & Set，或是 Compare & Swap，现在几乎所有的 CPU 指令都支持 CAS的原子操作。

原子操作是指一个不受其他操作影响的操作任务单元。原子操作是在多线程环境下避免数据不一致必须的手段。

int++并不是一个原子操作，所以当一个线程读取它的值并加 1 时，另外一个线程有可能会读到之前的值，这就会引发错误。

为了解决这个问题，必须保证增加操作是原子的，在 JDK1.5 之前我们可以使用同步技术来做到这一点。到 JDK1.5，java.util.concurrent.atomic 包提供了 int 和long 类型的原子包装类，它们可以自动的保证对于他们的操作是原子的并且不需要使用同步。

java.util.concurrent 这个包里面提供了一组原子类。其基本的特性就是在多线程环境下，当有多个线程同时执行这些类的实例包含的方法时，具有排他性，即当某个线程进入方法，执行其中的指令时，不会被其他线程打断，而别的线程就像自旋锁一样，一直等到该方法执行完成，才由 JVM 从等待队列中选择一个另一个线程进入，这只是一种逻辑上的理解。

原子类：AtomicBoolean，AtomicInteger，AtomicLong，AtomicReference
原子数组：AtomicIntegerArray，AtomicLongArray，AtomicReferenceArray
原子属性更新器：AtomicLongFieldUpdater，AtomicIntegerFieldUpdater，AtomicReferenceFieldUpdater
解决 ABA 问题的原子类：AtomicMarkableReference（通过引入一个 boolean来反映中间有没有变过），AtomicStampedReference（通过引入一个 int 来累加来反映中间有没有变过）

95、Java Concurrency API 中的 Lock 接口(Lock interface)是什么？对比同步它有什么优势？

Lock 接口比同步方法和同步块提供了更具扩展性的锁操作。他们允许更灵活的结构，可以具有完全不同的性质，并且可以支持多个相关类的条件对象。

它的优势有：
可以使锁更公平
可以使线程在等待锁的时候响应中断
可以让线程尝试获取锁，并在无法获取锁的时候立即返回或者等待一段时间
可以在不同的范围，以不同的顺序获取和释放锁

整体上来说 Lock 是 synchronized 的扩展版，Lock 提供了无条件的、可轮询的(tryLock 方法)、定时的(tryLock 带参方法)、可中断的(lockInterruptibly)、可多条件队列的(newCondition 方法)锁操作。另外 Lock 的实现类基本都支持非公平锁(默认)和公平锁，synchronized 只支持非公平锁，当然，在大部分情况下，非公平锁是高效的选择。

96、什么是 Executors 框架？

Executor 框架是一个根据一组执行策略调用，调度，执行和控制的异步任务的框架。无限制的创建线程会引起应用程序内存溢出。所以创建一个线程池是个更好的的解决方案，因为可以限制线程的数量并且可以回收再利用这些线程。利用Executors 框架可以非常方便的创建一个线程池。

97、什么是阻塞队列？阻塞队列的实现原理是什么？如何使用阻塞队列来实现生产者-消费者模型？

阻塞队列（BlockingQueue）是一个支持两个附加操作的队列。这两个附加的操作是：在队列为空时，获取元素的线程会等待队列变为非空。当队列满时，存储元素的线程会等待队列可用。阻塞队列常用于生产者和消费者的场景，生产者是往队列里添加元素的线程，消费者是从队列里拿元素的线程。阻塞队列就是生产者存放元素的容器，而消费者也只从容器里拿元素。
JDK7 提供了 7 个阻塞队列。分别是：

ArrayBlockingQueue ：一个由数组结构组成的有界阻塞队列。
LinkedBlockingQueue ：一个由链表结构组成的有界阻塞队列。
PriorityBlockingQueue ：一个支持优先级排序的无界阻塞队列。
DelayQueue：一个使用优先级队列实现的无界阻塞队列。
SynchronousQueue：一个不存储元素的阻塞队列。
LinkedTransferQueue：一个由链表结构组成的无界阻塞队列。
LinkedBlockingDeque：一个由链表结构组成的双向阻塞队列。

Java 5 之前实现同步存取时，可以使用普通的一个集合，然后在使用线程的协作和线程同步可以实现生产者，消费者模式，主要的技术就是用好，wait ,notify,notifyAll,sychronized 这些关键字。而在 java 5 之后，可以使用阻
塞队列来实现，此方式大大简少了代码量，使得多线程编程更加容易，安全方面也有保障。

BlockingQueue 接口是 Queue 的子接口，它的主要用途并不是作为容器，而是作为线程同步的的工具，因此他具有一个很明显的特性，当生产者线程试图向BlockingQueue 放入元素时，如果队列已满，则线程被阻塞，当消费者线程试图从中取出一个元素时，如果队列为空，则该线程会被阻塞，正是因为它所具有这个特性，所以在程序中多个线程交替向 BlockingQueue 中放入元素，取出元素，它可以很好的控制线程之间的通信。

阻塞队列使用最经典的场景就是 socket 客户端数据的读取和解析，读取数据的线程不断将数据放入队列，然后解析线程不断从队列取数据解析。

98、什么是 Callable 和 Future?

Callable 接口类似于 Runnable，从名字就可以看出来了，但是 Runnable 不会返回结果，并且无法抛出返回结果的异常，而 Callable 功能更强大一些，被线程执行后，可以返回值，这个返回值可以被 Future 拿到，也就是说，Future 可以拿到异步执行任务的返回值。可以认为是带有回调的 Runnable。

Future 接口表示异步任务，是还没有完成的任务给出的未来结果。所以说 Callable
用于产生结果，Future 用于获取结果。

99、什么是 FutureTask?使用 ExecutorService 启动任务。

在 Java 并发程序中 FutureTask 表示一个可以取消的异步运算。它有启动和取消运算、查询运算是否完成和取回运算结果等方法。只有当运算完成的时候结果才能取回，如果运算尚未完成 get 方法将会阻塞。一个 FutureTask 对象可以对调用了 Callable 和 Runnable 的对象进行包装，由于 FutureTask 也是调用了 Runnable接口所以它可以提交给 Executor 来执行。

100、什么是并发容器的实现？

何为同步容器：可以简单地理解为通过 synchronized 来实现同步的容器，如果有多个线程调用同步容器的方法，它们将会串行执行。比如 Vector，Hashtable，以及 Collections.synchronizedSet，synchronizedList 等方法返回的容器。可以通过查看 Vector，Hashtable 等这些同步容器的实现代码，可以看到这些容器实现线程安全的方式就是将它们的状态封装起来，并在需要同步的方法上加上关键字 synchronized。

并发容器使用了与同步容器完全不同的加锁策略来提供更高的并发性和伸缩性，例如在 ConcurrentHashMap 中采用了一种粒度更细的加锁机制，可以称为分段锁，在这种锁机制下，允许任意数量的读线程并发地访问 map，并且执行读操作的线程和写操作的线程也可以并发的访问 map，同时允许一定数量的写操作线程并发地修改 map，所以它可以在并发环境下实现更高的吞吐量。

101、多线程同步和互斥有几种实现方法，都是什么？

线程同步是指线程之间所具有的一种制约关系，一个线程的执行依赖另一个线程的消息，当它没有得到另一个线程的消息时应等待，直到消息到达时才被唤醒。线程互斥是指对于共享的进程系统资源，在各单个线程访问时的排它性。当有若干个线程都要使用某一共享资源时，任何时刻最多只允许一个线程去使用，其它要使用该资源的线程必须等待，直到占用资源者释放该资源。线程互斥可以看成是一种特殊的线程同步。

线程间的同步方法大体可分为两类：用户模式和内核模式。顾名思义，内核模式就是指利用系统内核对象的单一性来进行同步，使用时需要切换内核态与用户态，而用户模式就是不需要切换到内核态，只在用户态完成操作。用户模式下的方法有：原子操作（例如一个单一的全局变量），临界区。内核模式下的方法有：事件，信号量，互斥量。

102、什么是竞争条件？你怎样发现和解决竞争？

当多个进程都企图对共享数据进行某种处理，而最后的结果又取决于进程运行的顺序时，则我们认为这发生了竞争条件（race condition）。

103、为什么我们调用 start()方法时会执行 run()方法，为什么我们不能直接调用 run()方法？

当你调用 start()方法时你将创建新的线程，并且执行在 run()方法里的代码。但是如果你直接调用 run()方法，它不会创建新的线程也不会执行调用线程的代码，只会把 run 方法当作普通方法去执行。

104、Java 中你怎样唤醒一个阻塞的线程？

在 Java 发展史上曾经使用 suspend()、resume()方法对于线程进行阻塞唤醒，但随之出现很多问题，比较典型的还是死锁问题。

解决方案可以使用以对象为目标的阻塞，即利用 Object 类的 wait()和 notify()方法实现线程阻塞。

首先，wait、notify 方法是针对对象的，调用任意对象的 wait()方法都将导致线程阻塞，阻塞的同时也将释放该对象的锁，相应地，调用任意对象的 notify()方法则将随机解除该对象阻塞的线程，但它需要重新获取改对象的锁，直到获取成功才能往下执行；其次，wait、notify 方法必须在 synchronized 块或方法中被调用，并且要保证同步块或方法的锁对象与调用 wait、notify 方法的对象是同一个，如此一来在调用 wait 之前当前线程就已经成功获取某对象的锁，执行 wait 阻塞后当前线程就将之前获取的对象锁释放。

105、在 Java 中 CycliBarriar 和 CountdownLatch 有什么区别？

CyclicBarrier 可以重复使用，而 CountdownLatch 不能重复使用。 Java 的 concurrent 包里面的 CountDownLatch 其实可以把它看作一个计数器，只不过这个计数器的操作是原子操作，同时只能有一个线程去操作这个计数器，也就是同时只能有一个线程去减这个计数器里面的值。你可以向 CountDownLatch 对象设置一个初始的数字作为计数值，任何调用这个对象上的 await()方法都会阻塞，直到这个计数器的计数值被其他的线程减为 0 为止。

所以在当前计数到达零之前，await 方法会一直受阻塞。之后，会释放所有等待的线程，await 的所有后续调用都将立即返回。这种现象只出现一次——计数无法被重置。如果需要重置计数，请考虑使用 CyclicBarrier。

CountDownLatch 的一个非常典型的应用场景是：有一个任务想要往下执行，但必须要等到其他的任务执行完毕后才可以继续往下执行。假如我们这个想要继续往下执行的任务调用一个 CountDownLatch 对象的 await()方法，其他的任务执行完自己的任务后调用同一个 CountDownLatch 对象上的 countDown()方法，这个调用 await()方法的任务将一直阻塞等待，直到这个 CountDownLatch 对象的计数值减到 0 为止。

CyclicBarrier 一个同步辅助类，它允许一组线程互相等待，直到到达某个公共屏障点 (common barrier point)。在涉及一组固定大小的线程的程序中，这些线程必须不时地互相等待，此时 CyclicBarrier 很有用。因为该 barrie在释放等待线程后可以重用，所以称它为循环 的 barrier。

106、什么是不可变对象，它对写并发应用有什么帮助

不可变对象(Immutable Objects)即对象一旦被创建它的状态（对象的数据，也即对象属性值）就不能改变，反之即为可变对象(Mutable Objects)。不可变对象的类即为不可变类(Immutable Class)。Java 平台类库中包含许多不可变类，如 String、基本类型的包装类、BigInteger 和 BigDecimal 等。不可变对象天生是线程安全的。它们的常量（域）是在构造函数中创建的。既然它们的状态无法修改，这些常量永远不会变。

不可变对象永远是线程安全的。
只有满足如下状态，一个对象才是不可变的；
它的状态不能在创建后再被修改；
所有域都是 final 类型；并且，
它被正确创建（创建期间没有发生 this 引用的逸出）。

107、Java 中用到的线程调度算法是什么？

计算机通常只有一个 CPU,在任意时刻只能执行一条机器指令,每个线程只有获得CPU 的使用权才能执行指令.所谓多线程的并发运行,其实是指从宏观上看,各个线程轮流获得 CPU 的使用权,分别执行各自的任务.在运行池中,会有多个处于就绪状态的线程在等待 CPU,JAVA 虚拟机的一项任务就是负责线程的调度,线程调度是指按照特定机制为多个线程分配 CPU 的使用权.

有两种调度模型：分时调度模型和抢占式调度模型。分时调度模型是指让所有的线程轮流获得 cpu 的使用权,并且平均分配每个线程占用的 CPU 的时间片这个也比较好理解。 java 虚拟机采用抢占式调度模型，是指优先让可运行池中优先级高的线程占用CPU，如果可运行池中的线程优先级相同，那么就随机选择一个线程，使其占用CPU。处于运行状态的线程会一直运行，直至它不得不放弃 CPU。

108、什么是线程组，为什么在 Java 中不推荐使用？

线程组和线程池是两个不同的概念，他们的作用完全不同，前者是为了方便线程的管理，后者是为了管理线程的生命周期，复用线程，减少创建销毁线程的开销。

JVM面试题

1、java中会存在内存泄漏吗，请简单描述。

会。自己实现堆载的数据结构时有可能会出现内存泄露，可参看effective java.

2、64 位 JVM 中，int 的长度是多数？

Java 中，int 类型变量的长度是一个固定值，与平台无关，都是 32 位。意思就是说，在 32 位 和 64 位 的 Java 虚拟机中，int 类型的长度是相同的。

3、Serial 与 Parallel GC 之间的不同之处？

Serial 与 Parallel 在 GC 执行的时候都会引起 stop-the-world。它们之间主要不同 serial 收集器是默认的复制收集器，执行 GC 的时候只有一个线程，而parallel 收集器使用多个 GC 线程来执行。

4、32 位和 64 位的 JVM，int 类型变量的长度是多数？

32 位和 64 位的 JVM 中，int 类型变量的长度是相同的，都是 32 位或者 4个字节。

5、Java 中 WeakReference 与 SoftReference 的区别？

虽然 WeakReference 与 SoftReference 都有利于提高 GC 和 内存的效率，但是 WeakReference ，一旦失去最后一个强引用，就会被 GC 回收，而软引用虽然不能阻止被回收，但是可以延迟到 JVM 内存不足的时候。

6、JVM 选项 -XX:+UseCompressedOops 有什么作用？为什么要使用

当你将你的应用从 32 位的 JVM 迁移到 64 位的 JVM 时，由于对象的指针从32 位增加到了 64 位，因此堆内存会突然增加，差不多要翻倍。这也会对 CPU缓存（容量比内存小很多）的数据产生不利的影响。因为，迁移到 64 位的 JVM主要动机在于可以指定最大堆大小，通过压缩 OOP 可以节省一定的内存。通过-XX:+UseCompressedOops 选项，JVM 会使用 32 位的 OOP，而不是 64 位的 OOP。

7、怎样通过 Java 程序来判断 JVM 是 32 位 还是 64位？

你可以检查某些系统属性如 sun.arch.data.model 或 os.arch 来获取该信息。

8、32 位 JVM 和 64 位 JVM 的最大堆内存分别是多数？

理论上说上 32 位的 JVM 堆内存可以到达 2^32，即 4GB，但实际上会比这个小很多。不同操作系统之间不同，如 Windows 系统大约 1.5 GB，Solaris 大约3GB。64 位 JVM 允许指定最大的堆内存，理论上可以达到 2^64，这是一个非常大的数字，实际上你可以指定堆内存大小到 100GB。甚至有的 JVM，如 Azul，堆内存到 1000G 都是可能的。

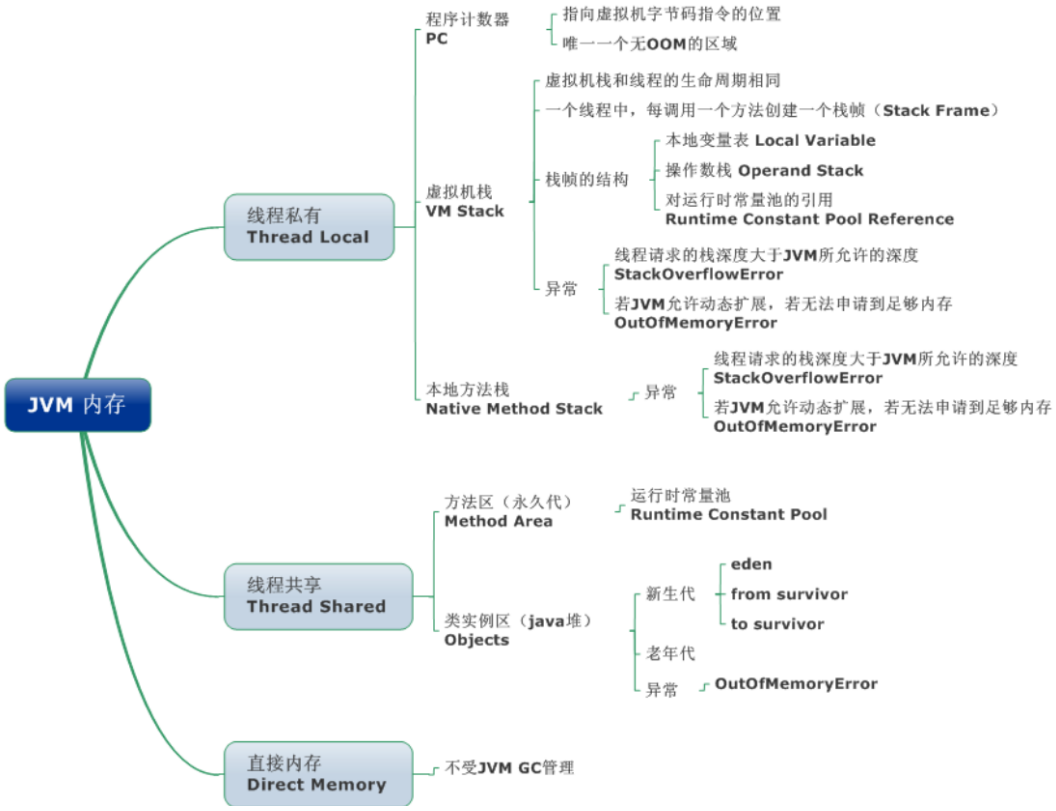
9、JRE、JDK、JVM 及 JIT 之间有什么不同？

JRE 代表 Java 运行时（Java run-time），是运行 Java 引用所必须的。JDK 代表 Java 开发工具（Java development kit），是 Java 程序的开发工具，如 Java编译器，它也包含 JRE。JVM 代表 Java 虚拟机（Java virtual machine），它的责任是运行 Java 应用。JIT 代表即时编译（Just In Time compilation），当代码执行的次数超过一定的阈值时，会将 Java 字节码转换为本地代码，如，主要的热点代码会被准换为本地代码，这样有利大幅度提高 Java 应用的性能。

10、解释 Java 堆空间及 GC？

当通过 Java 命令启动 Java 进程的时候，会为它分配内存。内存的一部分用于创建堆空间，当程序中创建对象的时候，就从对空间中分配内存。GC 是 JVM 内部的一个进程，回收无效对象的内存用于将来的分配。

11、JVM 内存区域

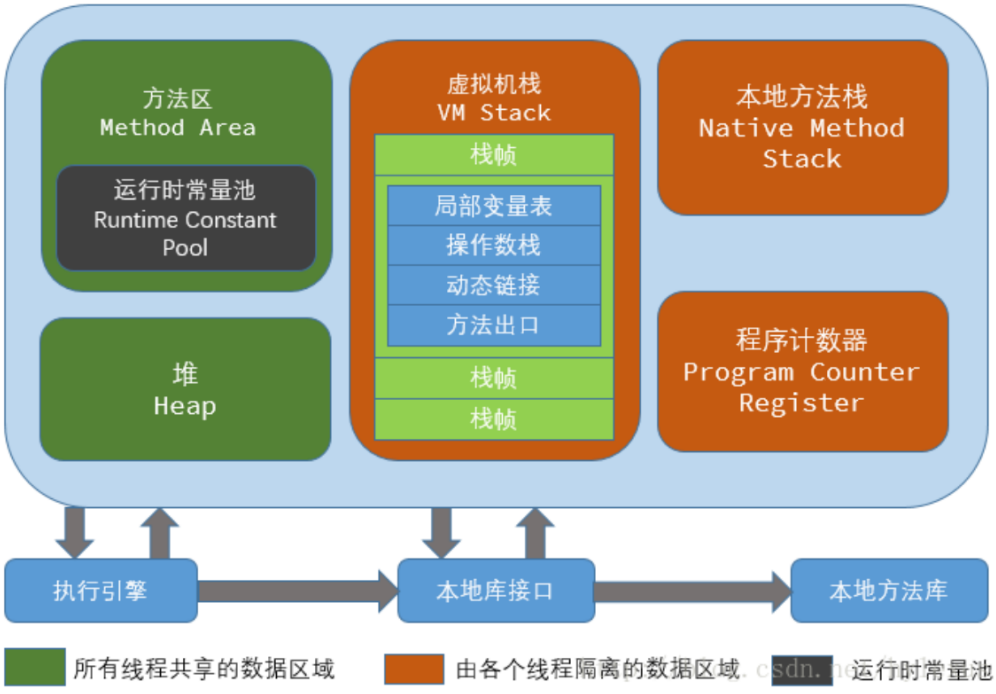


JVM 内存区域主要分为线程私有区域【程序计数器、虚拟机栈、本地方法区】、线程共享区域【JAVA 堆、方法区】、直接内存。

线程私有数据区域生命周期与线程相同, 依赖用户线程的启动/结束 而 创建/销毁(在 Hotspot VM 内, 每个线程都与操作系统的本地线程直接映射, 因此这部分内存区域的存/否跟随本地线程的生/死对应)。

线程共享区域随虚拟机的启动/关闭而创建/销毁。

直接内存并不是 JVM 运行时数据区的一部分, 但也会被频繁的使用: 在 JDK 1.4 引入的 NIO 提供了基于 Channel 与 Buffer 的 IO 方式, 它可以使用 Native 函数库直接分配堆外内存, 然后使用DirectByteBuffer 对象作为这块内存的引用进行操作(详见: Java I/O 扩展), 这样就避免了在 Java堆和 Native 堆中来回复制数据, 因此在一些场景中可以显著提高性能。



12、程序计数器(线程私有)

一块较小的内存空间, 是当前线程所执行的字节码的行号指示器，每条线程都要有一个独立的程序计数器，这类内存也称为“线程私有” 的内存。

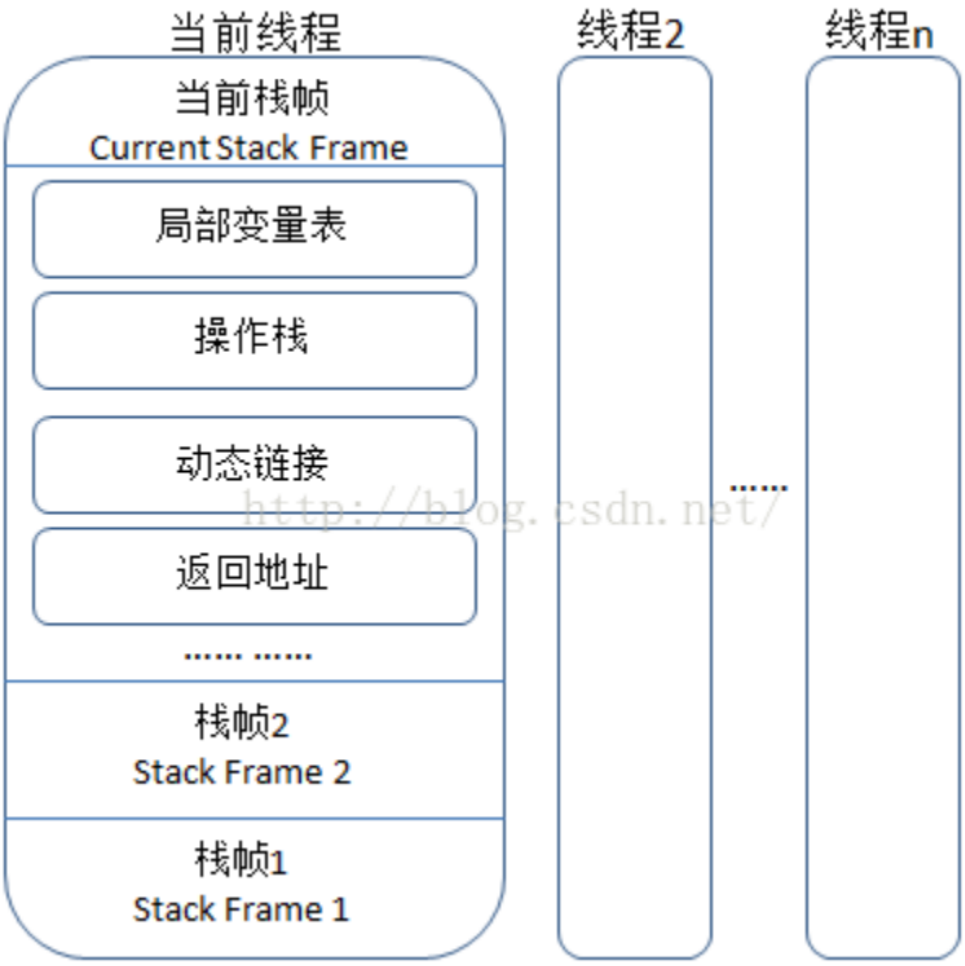
正在执行 java 方法的话，计数器记录的是虚拟机字节码指令的地址（当前指令的地址）。如果还是 Native 方法，则为空。

这个内存区域是唯一一个在虚拟机中没有规定任何 OutOfMemoryError 情况的区域。

13、虚拟机栈(线程私有)

是描述java方法执行的内存模型，每个方法在执行的同时都会创建一个栈帧（Stack Frame）用于存储局部变量表、操作数栈、动态链接、方法出口等信息。 每一个方法从调用直至执行完成的过程，就对应着一个栈帧在虚拟机栈中入栈到出栈的过程。

栈帧（ Frame）是用来存储数据和部分过程结果的数据结构，同时也被用来处理动态链接(Dynamic Linking)、 方法返回值和异常分派（Dispatch Exception）。 栈帧随着方法调用而创建，随着方法结束而销毁——无论方法是正常完成还是异常完成（抛出了在方法内未被捕获的异常）都算作方法结束。



14、本地方法区(线程私有)

本地方法区和 Java Stack 作用类似, 区别是虚拟机栈为执行 Java 方法服务, 而本地方法栈则为Native 方法服务, 如果一个 VM 实现使用 C-linkage 模型来支持 Native 调用, 那么该栈将会是一个C 栈，但 HotSpot VM 直接就把本地方法栈和虚拟机栈合二为一 。

15、你能保证 GC 执行吗？

不能，虽然你可以调用 System.gc() 或者 Runtime.gc()，但是没有办法保证 GC的执行。

16、怎么获取 Java 程序使用的内存？堆使用的百分比？

可以通过 java.lang.Runtime 类中与内存相关方法来获取剩余的内存，总内存及最大堆内存。通过这些方法你也可以获取到堆使用的百分比及堆内存的剩余空间。Runtime.freeMemory() 方法返回剩余空间的字节数，Runtime.totalMemory()方法总内存的字节数，Runtime.maxMemory() 返回最大内存的字节数。

17、Java 中堆和栈有什么区别？

JVM 中堆和栈属于不同的内存区域，使用目的也不同。栈常用于保存方法帧和局部变量，而对象总是在堆上分配。栈通常都比堆小，也不会在多个线程之间共享，而堆被整个 JVM 的所有线程共享。

18、描述一下 JVM 加载 class 文件的原理机制

JVM 中类的装载是由类加载器（ClassLoader）和它的子类来实现的，Java 中的类加载器是一个重要的 Java 运行时系统组件，它负责在运行时查找和装入类文件中的类。

由于 Java 的跨平台性，经过编译的 Java 源程序并不是一个可执行程序，而是一个或多个类文件。当 Java 程序需要使用某个类时，JVM 会确保这个类已经被加载、连接（验证、准备和解析）和初始化。类的加载是指把类的.class 文件中的数据读入到内存中，通常是创建一个字节数组读入.class 文件，然后产生与所加载类对应
的 Class 对象。

加载完成后，Class 对象还不完整，所以此时的类还不可用。当类被加载后就进入连接阶段，这一阶段包括验证、准备（为静态变量分配内存并设置默认的初始值）和解析（将符号引用替换为直接引用）三个步骤。最后 JVM 对
类进行初始化，包括：1)如果类存在直接的父类并且这个类还没有被初始化，那么就先初始化父类；2)如果类中存在初始化语句，就依次执行这些初始化语句。

类的加载是由类加载器完成的，类加载器包括：根加载器（BootStrap）、扩展加载器（Extension）、系统加载器（System）和用户自定义类加载器（java.lang.ClassLoader 的子类）。

从 Java 2（JDK 1.2）开始，类加载过程采取了父亲委托机制（PDM）。PDM 更好的保证了 Java 平台的安全性，在该机制中，JVM 自带的 Bootstrap 是根加载器，其他的加载器都有且仅有一个父类加载器。类的加载首先请求父类加载器加载，父类加载器无能为力时才由其子类加载器自行加载。JVM 不会向 Java 程序提供对 Bootstrap 的引用。下面是关于几个类
加载器的说明：

1. Bootstrap：一般用本地代码实现，负责加载 JVM 基础核心类库（rt.jar）；
2. Extension：从 java.ext.dirs 系统属性所指定的目录中加载类库，它的父加载器是 Bootstrap；
3. System：又叫应用类加载器，其父类是 Extension。它是应用最广泛的类加载器。它从环境变量 classpath 或者系统属性 java.class.path 所指定的目录中记载类，是用户自定义加载器的默认父加载器。

19、GC 是什么？为什么要有 GC？

GC 是垃圾收集的意思，内存处理是编程人员容易出现问题的地方，忘记或者错误的内存回收会导致程序或系统的不稳定甚至崩溃，Java 提供的 GC 功能可以自动监测对象是否超过作用域从而达到自动回收内存的目的，Java 语言没有提供释放已分配内存的显示操作方法。Java 程序员不用担心内存管理，因为垃圾收集器会自动进行管理。要请求垃圾收集，可以调用下面的方法之一：System.gc() 或Runtime.getRuntime().gc()，但 JVM 可以屏蔽掉显示的垃圾回收调用。

垃圾回收可以有效的防止内存泄露，有效的使用可以使用的内存。垃圾回收器通常是作为一个单独的低优先级的线程运行，不可预知的情况下对内存堆中已经死亡的或者长时间没有使用的对象进行清除和回收，程序员不能实时的调用垃圾回收器对某个对象或所有对象进行垃圾回收。在 Java 诞生初期，垃圾回收是 Java最大的亮点之一，因为服务器端的编程需要有效的防止内存泄露问题，然而时过境迁，如今 Java 的垃圾回收机制已经成为被诟病的东。移动智能终端用户通常觉得 iOS 的系统比 Android 系统有更好的用户体验，其中一个深层次的原因就在于 Android 系统中垃圾回收的不可预知性。

20、堆（Heap-线程共享）-运行时数据区

是被线程共享的一块内存区域，创建的对象和数组都保存在 Java 堆内存中，也是垃圾收集器进行垃圾收集的最重要的内存区域。 由于现代 VM 采用分代收集算法, 因此 Java 堆从 GC 的角度还可以细分为: 新生代(Eden 区、 From Survivor 区和 To Survivor 区)和老年代。

21、方法区/永久代（线程共享）

即我们常说的永久代(Permanent Generation), 用于存储被 JVM 加载的类信息、 常量、 静态变量、 即时编译器编译后的代码等数据. HotSpot VM把GC分代收集扩展至方法区, 即使用Java堆的永久代来实现方法区, 这样 HotSpot 的垃圾收集器就可以像管理 Java 堆一样管理这部分内存,而不必为方法区开发专门的内存管理器(永久带的内存回收的主要目标是针对常量池的回收和类型的卸载, 因此收益一般很小）。

运行时常量池（Runtime Constant Pool）是方法区的一部分。 Class 文件中除了有类的版本、字段、方法、接口等描述等信息外，还有一项信息是常量池 （Constant Pool Table），用于存放编译期生成的各种字面量和符号引用，这部分内容将在类加载后存放到方法区的运行时常量池中。 Java 虚拟机对 Class 文件的每一部分（自然也包括常量池）的格式都有严格的规定，每一个字节用于存储哪种数据都必须符合规范上的要求，这样才会被虚拟机认可、装载和执行。

22、JVM 运行时内存

Java 堆从 GC 的角度还可以细分为: 新生代(Eden 区、 From Survivor 区和 To Survivor 区)和老年代。

23、新生代

是用来存放新生的对象。一般占据堆的 1/3 空间。由于频繁创建对象，所以新生代会频繁触发MinorGC 进行垃圾回收。新生代又分为 Eden 区、 ServivorFrom、 ServivorTo 三个区。

Eden 区
Java 新对象的出生地（如果新创建的对象占用内存很大，则直接分配到老年代）。当 Eden 区内存不够的时候就会触发 MinorGC，对新生代区进行一次垃圾回收。

ServivorFrom

上一次 GC 的幸存者，作为这一次 GC 的被扫描者。

ServivorTo

保留了一次 MinorGC 过程中的幸存者。

MinorGC 的过程（复制->清空->互换）

MinorGC 采用复制算法。

1： eden、 servicorFrom 复制到 ServicorTo，年龄+1
首先，把 Eden 和 ServivorFrom 区域中存活的对象复制到 ServicorTo 区域（如果有对象的年龄以及达到了老年的标准，则赋值到老年代区），同时把这些对象的年龄+1（如果 ServicorTo 不够位置了就放到老年区）；

2： 清空 eden、 servicorFrom
然后，清空 Eden 和 ServicorFrom 中的对象；

3： ServicorTo 和 ServicorFrom 互换
最后， ServicorTo 和 ServicorFrom 互换，原 ServicorTo 成为下一次 GC 时的 ServicorFrom区。

24、老年代

主要存放应用程序中生命周期长的内存对象。

老年代的对象比较稳定，所以 MajorGC 不会频繁执行。在进行 MajorGC 前一般都先进行了一次 MinorGC，使得有新生代的对象晋身入老年代，导致空间不够用时才触发。当无法找到足够大的连续空间分配给新创建的较大对象时也会提前触发一次 MajorGC 进行垃圾回收腾出空间。

MajorGC 采用标记清除算法：首先扫描一次所有老年代，标记出存活的对象，然后回收没有标记的对象。 ajorGC 的耗时比较长，因为要扫描再回收。 MajorGC 会产生内存碎片，为了减少内存损耗，我们一般需要进行合并或者标记出来方便下次直接分配。当老年代也满了装不下的时候，就会抛出 OOM（Out of Memory）异常。

25、永久代

指内存的永久保存区域，主要存放 Class 和 Meta（元数据）的信息,Class 在被加载的时候被放入永久区域， 它和和存放实例的区域不同,GC 不会在主程序运行期对永久区域进行清理。所以这也导致了永久代的区域会随着加载的 Class 的增多而胀满，最终抛出 OOM 异常。

26、JAVA8 与元数据

在 Java8 中， 永久代已经被移除，被一个称为"元数据区"（元空间）的区域所取代。元空间的本质和永久代类似，元空间与永久代之间最大的区别在于： 元空间并不在虚拟机中，而是使用本地内存。因此，默认情况下，元空间的大小仅受本地内存限制。 类的元数据放入 nativememory, 字符串池和类的静态变量放入 java 堆中， 这样可以加载多少类的元数据就不再由MaxPermSize 控制, 而由系统的实际可用空间来控制。

27、引用计数法

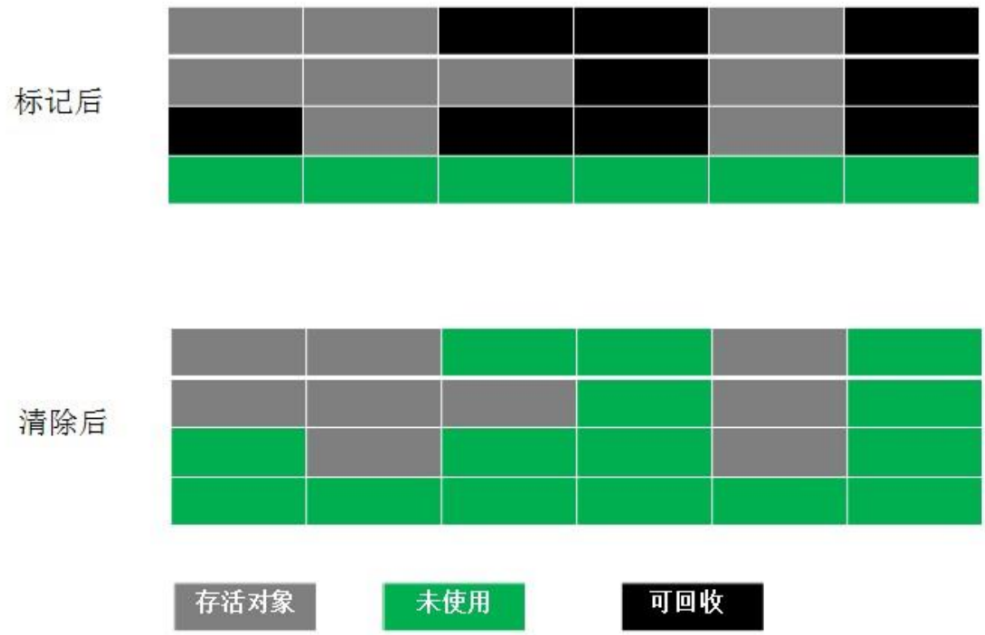
在 Java 中，引用和对象是有关联的。如果要操作对象则必须用引用进行。因此，很显然一个简单的办法是通过引用计数来判断一个对象是否可以回收。简单说，即一个对象如果没有任何与之关联的引用， 即他们的引用计数都不为 0， 则说明对象不太可能再被用到，那么这个对象就是可回收对象。

28、可达性分析

为了解决引用计数法的循环引用问题， Java 使用了可达性分析的方法。通过一系列的“GC roots”对象作为起点搜索。如果在“GC roots”和一个对象之间没有可达路径，则称该对象是不可达的。要注意的是，不可达对象不等价于可回收对象， 不可达对象变为可回收对象至少要经过两次标记过程。两次标记后仍然是可回收对象，则将面临回收。

29、标记清除算法（ Mark-Sweep ）

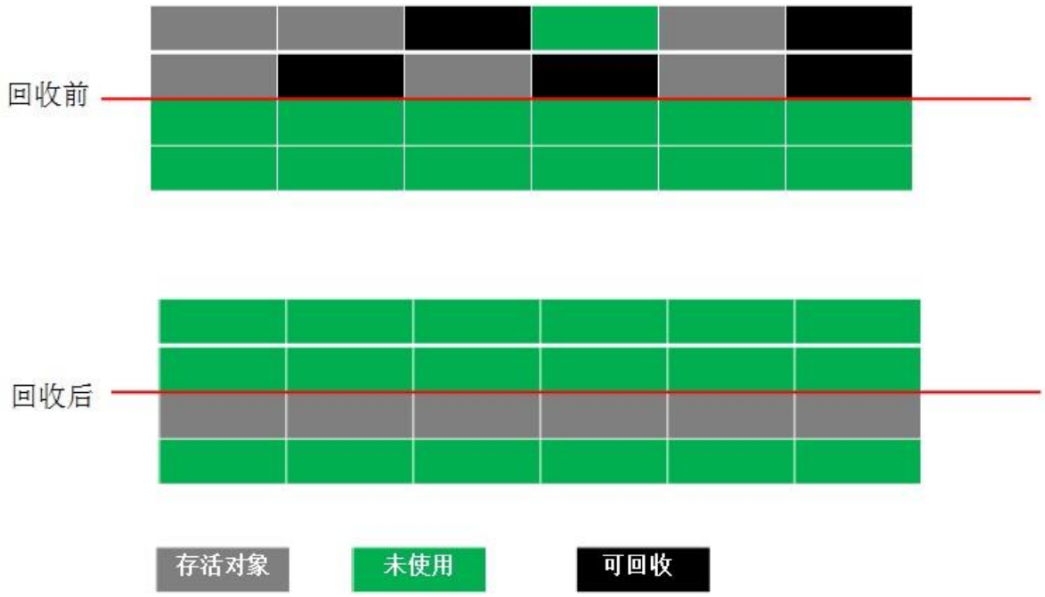
最基础的垃圾回收算法，分为两个阶段，标注和清除。标记阶段标记出所有需要回收的对象，清除阶段回收被标记的对象所占用的空间。如图



从图中我们就可以发现，该算法最大的问题是内存碎片化严重，后续可能发生大对象不能找到可利用空间的问题。

30、复制算法（copying）

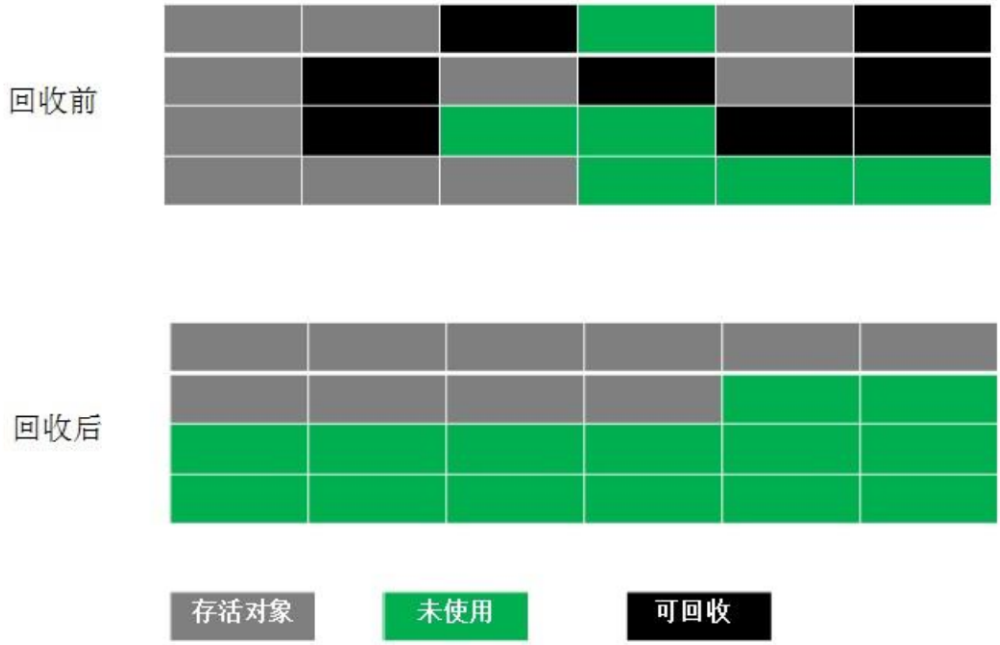
为了解决 Mark-Sweep 算法内存碎片化的缺陷而被提出的算法。按内存容量将内存划分为等大小的两块。每次只使用其中一块，当这一块内存满后将尚存活的对象复制到另一块上去，把已使用的内存清掉，如图：



这种算法虽然实现简单，内存效率高，不易产生碎片，但是最大的问题是可用内存被压缩到了原本的一半。且存活对象增多的话， Copying 算法的效率会大大降低。

31、标记整理算法(Mark-Compact)

结合了以上两个算法，为了避免缺陷而提出。标记阶段和 Mark-Sweep 算法相同， 标记后不是清理对象，而是将存活对象移向内存的一端。然后清除端边界外的对象。如图：

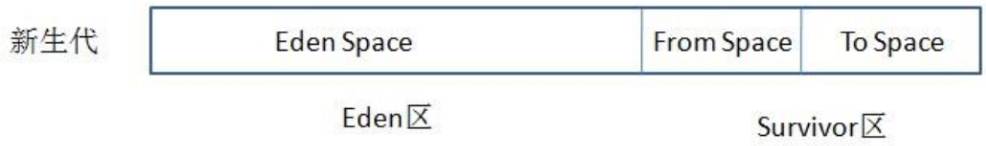


32、分代收集算法

分代收集法是目前大部分 JVM 所采用的方法，其核心思想是根据对象存活的不同生命周期将内存划分为不同的域，一般情况下将 GC 堆划分为老生代(Tenured/Old Generation)和新生代(YoungGeneration)。老生代的特点是每次垃圾回收时只有少量对象需要被回收，新生代的特点是每次垃圾回收时都有大量垃圾需要被回收，因此可以根据不同区域选择不同的算法。

33、新生代与复制算法

目前大部分 JVM 的 GC 对于新生代都采取 Copying 算法，因为新生代中每次垃圾回收都要回收大部分对象，即要复制的操作比较少，但通常并不是按照 1： 1 来划分新生代。一般将新生代划分为一块较大的 Eden 空间和两个较小的 Survivor 空间(From Space, To Space)，每次使用Eden 空间和其中的一块 Survivor 空间，当进行回收时，将该两块空间中还存活的对象复制到另一块 Survivor 空间中。



34、老年代与标记复制算法

而老年代因为每次只回收少量对象，因而采用 Mark-Compact 算法。

1. JAVA 虚拟机提到过的处于方法区的永生代(Permanet Generation)， 它用来存储 class 类，常量，方法描述等。对永生代的回收主要包括废弃常量和无用的类。
2. 对象的内存分配主要在新生代的 Eden Space 和 Survivor Space 的 From Space(Survivor 目前存放对象的那一块)，少数情况会直接分配到老生代。
3. 当新生代的 Eden Space 和 From Space 空间不足时就会发生一次 GC，进行 GC 后， EdenSpace 和 From Space 区的存活对象会被挪到 To Space，然后将 Eden Space 和 FromSpace 进行清理。
4. 如果 To Space 无法足够存储某个对象，则将这个对象存储到老生代。
5. 在进行 GC 后，使用的便是 Eden Space 和 To Space 了，如此反复循环。
6. 当对象在 Survivor 区躲过一次 GC 后，其年龄就会+1。 默认情况下年龄到达 15 的对象会被移到老生代中。

35、JAVA 强引用

在 Java 中最常见的就是强引用， 把一个对象赋给一个引用变量，这个引用变量就是一个强引用。当一个对象被强引用变量引用时，它处于可达状态，它是不可能被垃圾回收机制回收的，即使该对象以后永远都不会被用到 JVM 也不会回收。因此强引用是造成 Java 内存泄漏的主要原因之一。

36、JAVA软引用

软引用需要用 SoftReference 类来实现，对于只有软引用的对象来说，当系统内存足够时它不会被回收，当系统内存空间不足时它会被回收。软引用通常用在对内存敏感的程序中。

37、JAVA弱引用

弱引用需要用 WeakReference 类来实现，它比软引用的生存期更短，对于只有弱引用的对象来说，只要垃圾回收机制一运行，不管 JVM 的内存空间是否足够，总会回收该对象占用的内存。

38、JAVA虚引用

虚引用需要 PhantomReference 类来实现，它不能单独使用，必须和引用队列联合使用。 虚引用的主要作用是跟踪对象被垃圾回收的状态。

39、分代收集算法

当前主流 VM 垃圾收集都采用"分代收集" (Generational Collection)算法, 这种算法会根据对象存活周期的不同将内存划分为几块, 如 JVM 中的 新生代、老年代、永久代， 这样就可以根据各年代特点分别采用最适当的 GC 算法

40、在新生代-复制算法

每次垃圾收集都能发现大批对象已死, 只有少量存活. 因此选用复制算法, 只需要付出少量存活对象的复制成本就可以完成收集

41、在老年代-标记整理算法

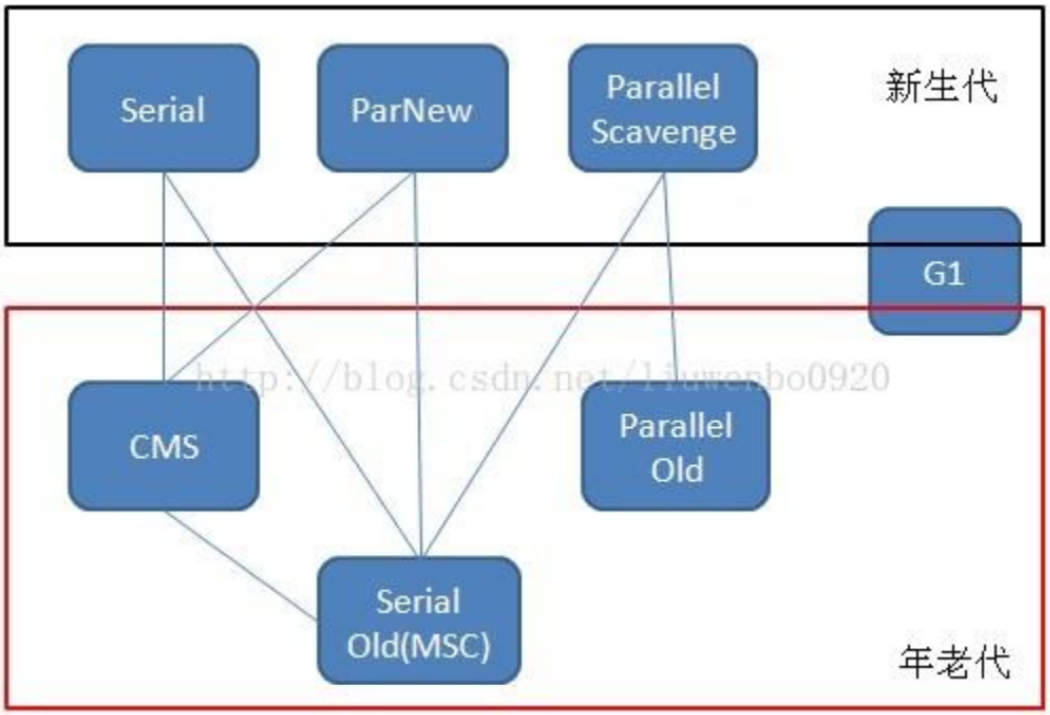
因为对象存活率高、没有额外空间对它进行分配担保, 就必须采用"标记—清理"或"标记—整理" 算法来进行回收, 不必进行内存复制, 且直接腾出空闲内存。

42、分区收集算法

分区算法则将整个堆空间划分为连续的不同小区间, 每个小区间独立使用, 独立回收. 这样做的好处是可以控制一次回收多少个小区间 , 根据目标停顿时间, 每次合理地回收若干个小区间(而不是整个堆), 从而减少一次 GC 所产生的停顿。

43、GC 垃圾收集器

Java 堆内存被划分为新生代和年老代两部分，新生代主要使用复制和标记-清除垃圾回收算法；年老代主要使用标记-整理垃圾回收算法，因此 java 虚拟中针对新生代和年老代分别提供了多种不同的垃圾收集器， JDK1.6 中 Sun HotSpot 虚拟机的垃圾收集器如下：



44、Serial 垃圾收集器（单线程、 复制算法）

Serial（英文连续）是最基本垃圾收集器，使用复制算法，曾经是JDK1.3.1 之前新生代唯一的垃圾收集器。 Serial 是一个单线程的收集器，它不但只会使用一个 CPU 或一条线程去完成垃圾收集工作，并且在进行垃圾收集的同时，必须暂停其他所有的工作线程，直到垃圾收集结束。

Serial 垃圾收集器虽然在收集垃圾过程中需要暂停所有其他的工作线程，但是它简单高效，对于限定单个 CPU 环境来说，没有线程交互的开销，可以获得最高的单线程垃圾收集效率，因此 Serial垃圾收集器依然是 java 虚拟机运行在 Client 模式下默认的新生代垃圾收集器。

45、ParNew 垃圾收集器（Serial+多线程）

ParNew 垃圾收集器其实是 Serial 收集器的多线程版本，也使用复制算法，除了使用多线程进行垃圾收集之外，其余的行为和 Serial 收集器完全一样， ParNew 垃圾收集器在垃圾收集过程中同样也要暂停所有其他的工作线程。

ParNew 收集器默认开启和 CPU 数目相同的线程数，可以通过-XX:ParallelGCThreads 参数来限制垃圾收集器的线程数。【Parallel：平行的】
ParNew 虽然是除了多线程外和Serial 收集器几乎完全一样，但是ParNew垃圾收集器是很多 java虚拟机运行在 Server 模式下新生代的默认垃圾收集器。

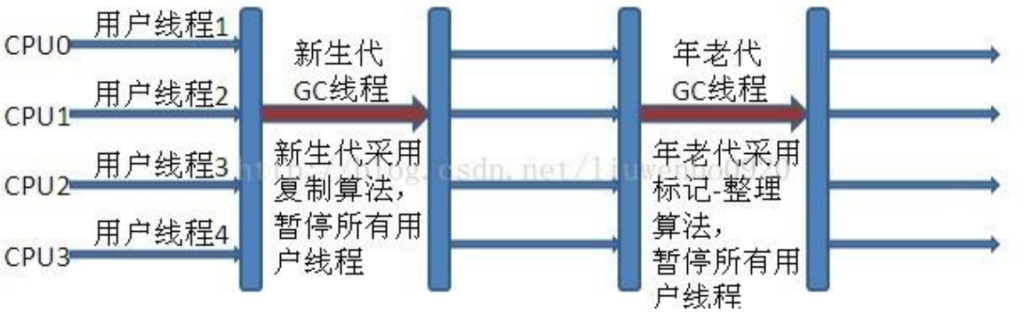
46、Parallel Scavenge 收集器（多线程复制算法、高效）

Parallel Scavenge 收集器也是一个新生代垃圾收集器，同样使用复制算法，也是一个多线程的垃圾收集器， 它重点关注的是程序达到一个可控制的吞吐量（Thoughput， CPU 用于运行用户代码的时间/CPU 总消耗时间，即吞吐量=运行用户代码时间/(运行用户代码时间+垃圾收集时间)），高吞吐量可以最高效率地利用 CPU 时间，尽快地完成程序的运算任务，主要适用于在后台运算而不需要太多交互的任务。 自适应调节策略也是 ParallelScavenge 收集器与 ParNew 收集器的一个重要区别。

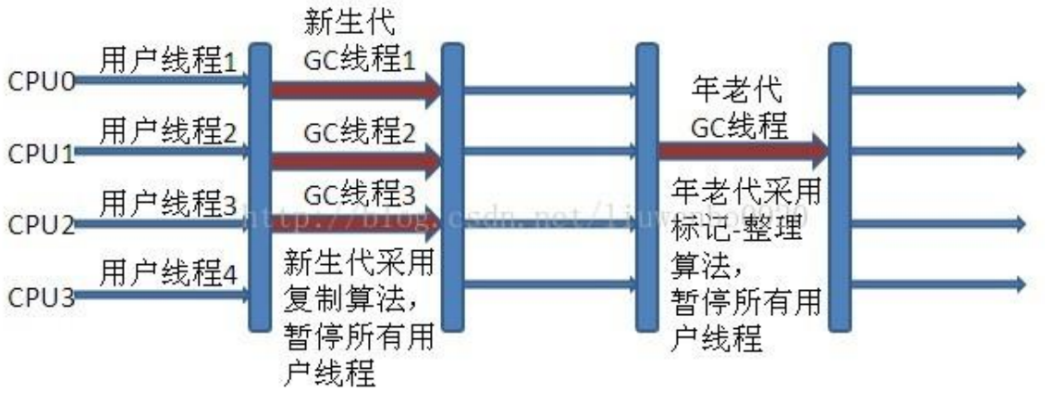
57、Serial Old 收集器（单线程标记整理算法 ）

Serial Old 是 Serial 垃圾收集器年老代版本，它同样是个单线程的收集器，使用标记-整理算法，这个收集器也主要是运行在 Client 默认的 java 虚拟机默认的年老代垃圾收集器。在 Server 模式下，主要有两个用途：

1. 在 JDK1.5 之前版本中与新生代的 Parallel Scavenge 收集器搭配使用。
2. 作为年老代中使用 CMS 收集器的后备垃圾收集方案。新生代 Serial 与年老代 Serial Old 搭配垃圾收集过程图：

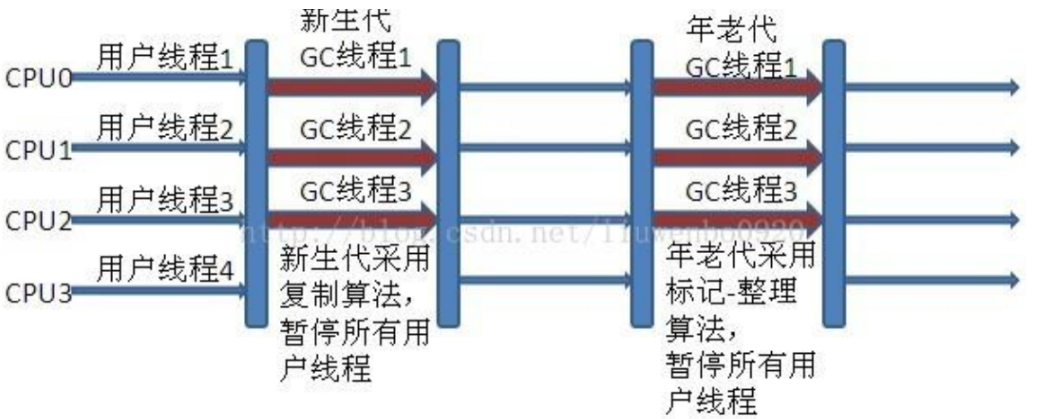


新生代 Parallel Scavenge 收集器与 ParNew 收集器工作原理类似，都是多线程的收集器，都使用的是复制算法，在垃圾收集过程中都需要暂停所有的工作线程。新生代 ParallelScavenge/ParNew 与年老代 Serial Old 搭配垃圾收集过程图：



58、Parallel Old 收集器（多线程标记整理算法）

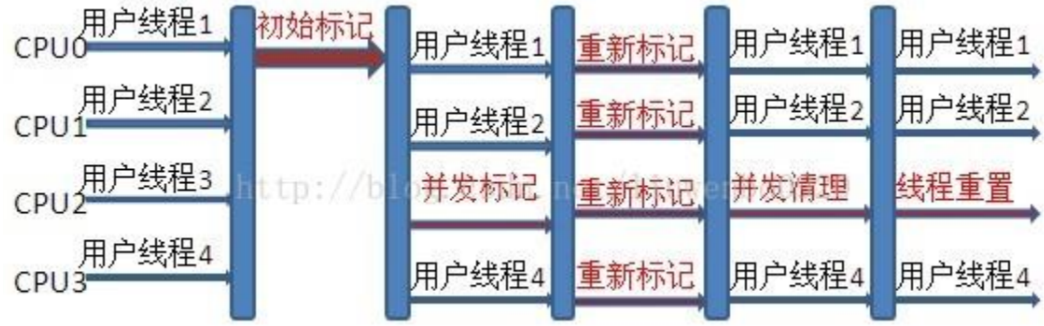
Parallel Old 收集器是Parallel Scavenge的年老代版本，使用多线程的标记-整理算法，在 JDK1.6才开始提供。
在 JDK1.6 之前，新生代使用 ParallelScavenge 收集器只能搭配年老代的 Serial Old 收集器，只能保证新生代的吞吐量优先，无法保证整体的吞吐量， Parallel Old 正是为了在年老代同样提供吞吐量优先的垃圾收集器， 如果系统对吞吐量要求比较高，可以优先考虑新生代 Parallel Scavenge和年老代 Parallel Old 收集器的搭配策略。
新生代 Parallel Scavenge 和年老代 Parallel Old 收集器搭配运行过程图



59、CMS 收集器（多线程标记清除算法）

Concurrent mark sweep(CMS)收集器是一种年老代垃圾收集器，其最主要目标是获取最短垃圾回收停顿时间， 和其他年老代使用标记-整理算法不同，它使用多线程的标记-清除算法。最短的垃圾收集停顿时间可以为交互比较高的程序提高用户体验。CMS 工作机制相比其他的垃圾收集器来说更复杂。整个过程分为以下 4 个阶段：

初始标记
只是标记一下 GC Roots 能直接关联的对象，速度很快，仍然需要暂停所有的工作线程。
并发标记
进行 GC Roots 跟踪的过程，和用户线程一起工作，不需要暂停工作线程。
重新标记
为了修正在并发标记期间，因用户程序继续运行而导致标记产生变动的那一部分对象的标记记录，仍然需要暂停所有的工作线程。
并发清除
清除 GC Roots 不可达对象，和用户线程一起工作，不需要暂停工作线程。由于耗时最长的并发标记和并发清除过程中，垃圾收集线程可以和用户现在一起并发工作， 所以总体上来看CMS 收集器的内存回收和用户线程是一起并发地执行。CMS 收集器工作过程



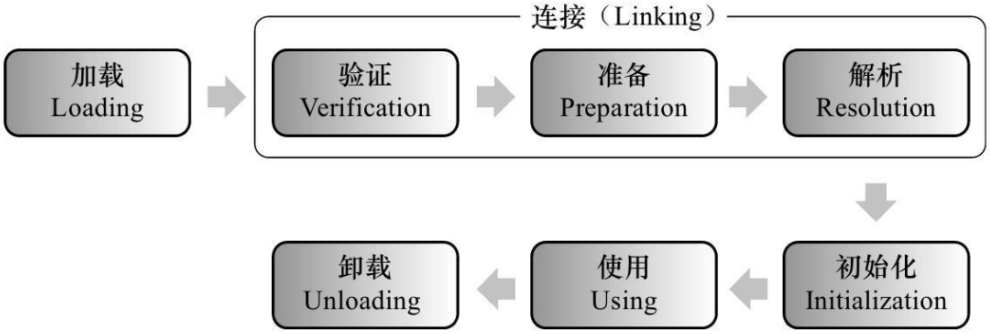
60、G1 收集器

Garbage first 垃圾收集器是目前垃圾收集器理论发展的最前沿成果，相比与 CMS 收集器， G1 收集器两个最突出的改进是：

1. 基于标记-整理算法，不产生内存碎片。
2. 可以非常精确控制停顿时间，在不牺牲吞吐量前提下，实现低停顿垃圾回收。G1 收集器避免全区域垃圾收集，它把堆内存划分为大小固定的几个独立区域，并且跟踪这些区域的垃圾收集进度，同时在后台维护一个优先级列表，每次根据所允许的收集时间， 优先回收垃圾最多的区域。区域划分和优先级区域回收机制，确保 G1 收集器可以在有限时间获得最高的垃圾收集效率

61、JVM 类加载机制

JVM 类加载机制分为五个部分：加载，验证，准备，解析，初始化，下面我们就分别来看一下这五个过程。



加载
加载是类加载过程中的一个阶段， 这个阶段会在内存中生成一个代表这个类的 java.lang.Class 对象， 作为方法区这个类的各种数据的入口。注意这里不一定非得要从一个 Class 文件获取，这里既可以从 ZIP 包中读取（比如从 jar 包和 war 包中读取），也可以在运行时计算生成（动态代理），也可以由其它文件生成（比如将 JSP 文件转换成对应的 Class 类）。

验证
这一阶段的主要目的是为了确保 Class 文件的字节流中包含的信息是否符合当前虚拟机的要求，并且不会危害虚拟机自身的安全。

准备
准备阶段是正式为类变量分配内存并设置类变量的初始值阶段，即在方法区中分配这些变量所使用的内存空间。注意这里所说的初始值概念，比如一个类变量定义为：

实际上变量 v 在准备阶段过后的初始值为 0 而不是 8080， 将 v 赋值为 8080 的 put static 指令是
程序被编译后， 存放于类构造器方法之中。
但是注意如果声明为：
public static final int v = 8080;
在编译阶段会为 v 生成 ConstantValue 属性，在准备阶段虚拟机会根据 ConstantValue 属性将 v
赋值为 8080。
解析

解析阶段是指虚拟机将常量池中的符号引用替换为直接引用的过程。符号引用就是 class 文件中
的：

```
public static int v = 8080;
```

实际上变量 v 在准备阶段过后的初始值为 0 而不是 8080， 将 v 赋值为 8080 的 put static 指令是程序被编译后， 存放于类构造器方法之中。但是注意如果声明为：

在编译阶段会为 v 生成 ConstantValue 属性，在准备阶段虚拟机会根据 ConstantValue 属性将 v
赋值为 8080。
解析
解析阶段是指虚拟机将常量池中的符号引用替换为直接引用的过程。符号引用就是 class 文件中
的：

```
public static final int v = 8080;
```

在编译阶段会为 v 生成 ConstantValue 属性，在准备阶段虚拟机会根据 ConstantValue 属性将 v赋值为 8080。

解析
解析阶段是指虚拟机将常量池中的符号引用替换为直接引用的过程。符号引用就是 class 文件中的：

1. CONSTANT_Class_info
2. CONSTANT_Field_info
3. CONSTANT_Method_info

等类型的常量。

符号引用

符号引用与虚拟机实现的布局无关， 引用的目标并不一定要已经加载到内存中。 各种虚拟机实现的内存布局可以各不相同，但是它们能接受的符号引用必须是一致的，因为符号引用的字面量形式明确定义在 Java 虚拟机规范的 Class 文件格式中。

直接引用

直接引用可以是指向目标的指针，相对偏移量或是一个能间接定位到目标的句柄。如果有了直接引用，那引用的目标必定已经在内存中存在。

初始化

初始化阶段是类加载最后一个阶段，前面的类加载阶段之后，除了在加载阶段可以自定义类加载器以外，其它操作都由 JVM 主导。到了初始阶段，才开始真正执行类中定义的 Java 程序代码。

类构造器

初始化阶段是执行类构造器方法的过程。 方法是由编译器自动收集类中的类变量的赋值操作和静态语句块中的语句合并而成的。虚拟机会保证子方法执行之前，父类的方法已经执行完毕， 如果一个类中没有对静态变量赋值也没有静态语句块，那么编译器可以不为这个类生成()方法。注意以下几种情况不会执行类初始化：

1. 通过子类引用父类的静态字段，只会触发父类的初始化，而不会触发子类的初始化。
2. 定义对象数组，不会触发该类的初始化。
3. 常量在编译期间会存入调用类的常量池中，本质上并没有直接引用定义常量的类，不会触
   发定义常量所在的类。
4. 通过类名获取 Class 对象，不会触发类的初始化。
5. 通过 Class.forName 加载指定类时，如果指定参数 initialize 为 false 时，也不会触发类初
   始化，其实这个参数是告诉虚拟机，是否要对类进行初始化。
6. 通过 ClassLoader 默认的 loadClass 方法，也不会触发初始化动作。

62、类加载器

虚拟机设计团队把加载动作放到 JVM 外部实现，以便让应用程序决定如何获取所需的类， JVM 提供了 3 种类加载器：

启动类加载器(Bootstrap ClassLoader)

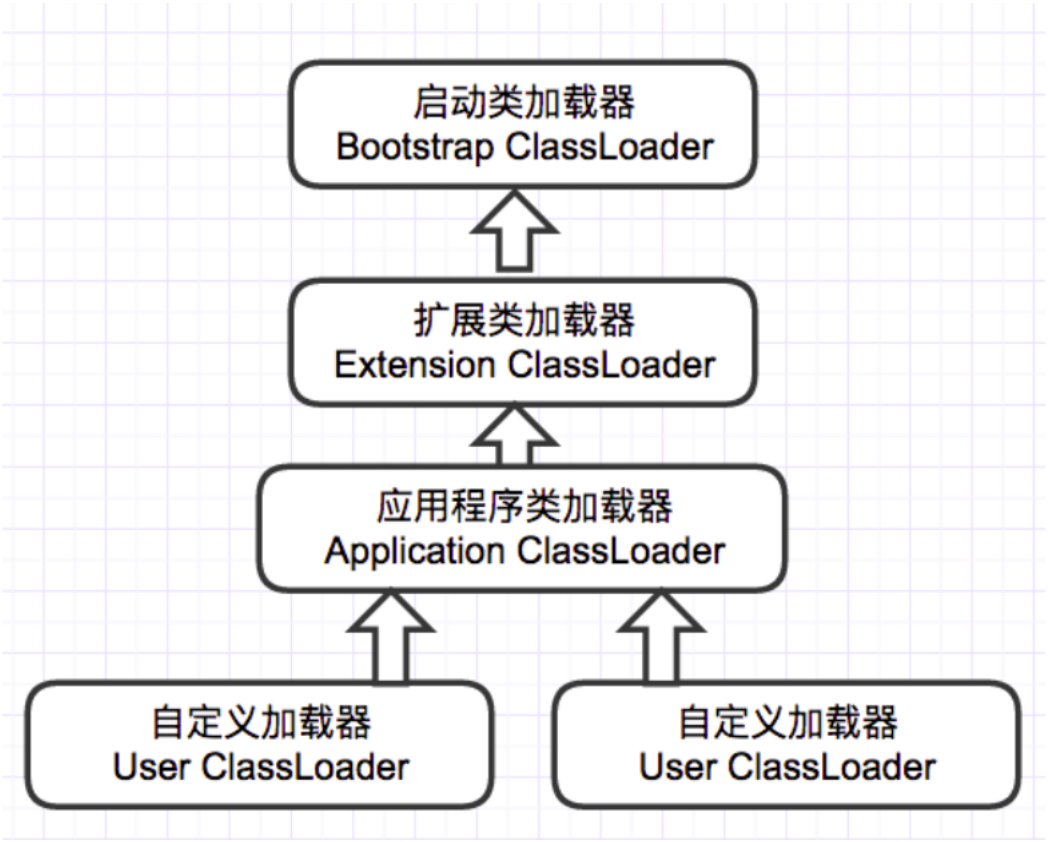
负责加载 JAVA_HOME\lib 目录中的， 或通过-Xbootclasspath 参数指定路径中的， 且被虚拟机认可（按文件名识别， 如 rt.jar ）的类。

扩展类加载器(Extension ClassLoader)

负责加载 JAVA_HOME\lib\ext 目录中的，或通过 java.ext.dirs 系统变量指定路径中的类库。

应用程序类加载器(Application ClassLoader)：

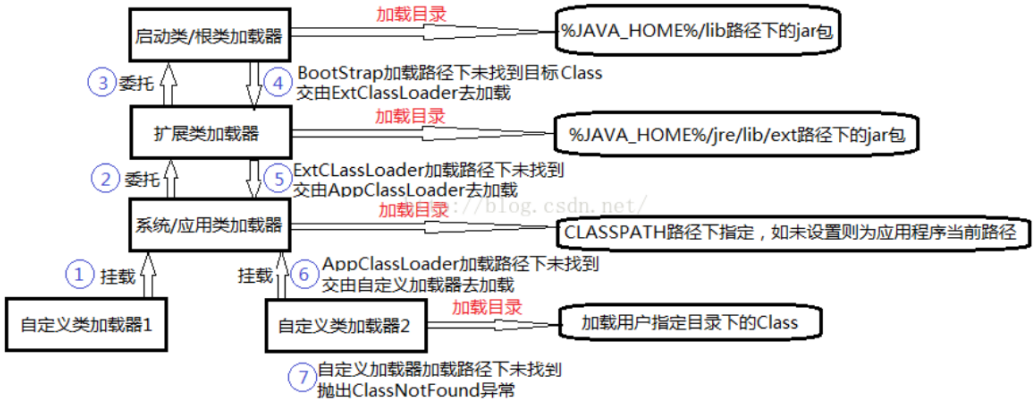
负责加载用户路径（classpath ）上的类库。JVM 通过双亲委派模型进行类的加载， 当然我们也可以通过继承 java.lang.ClassLoader实现自定义的类加载器。



63、双亲委派

当一个类收到了类加载请求，他首先不会尝试自己去加载这个类，而是把这个请求委派给父类去完成，每一个层次类加载器都是如此，因此所有的加载请求都应该传送到启动类加载其中，只有当父类加载器反馈自己无法完成这个请求的时候（在它的加载路径下没有找到所需加载的Class），子类加载器才会尝试自己去加载。

采用双亲委派的一个好处是比如加载位于 rt.jar 包中的类 java.lang.Object，不管是哪个加载器加载这个类，最终都是委托给顶层的启动类加载器进行加载，这样就保证了使用不同的类加载器最终得到的都是同样一个 Object 对象



64、OSGI（ 动态模型系统 ）

OSGi(Open Service Gateway Initiative)，是面向 Java 的动态模型系统，是 Java 动态化模块化系统的一系列规范。

65、动态改变构造

OSGi 服务平台提供在多种网络设备上无需重启的动态改变构造的功能。为了最小化耦合度和促使这些耦合度可管理， OSGi 技术提供一种面向服务的架构，它能使这些组件动态地发现对方。

66、模块化编程与热插拔

OSGi 旨在为实现 Java 程序的模块化编程提供基础条件，基于 OSGi 的程序很可能可以实现模块级的热插拔功能，当程序升级更新时，可以只停用、重新安装然后启动程序的其中一部分，这对企业级程序开发来说是非常具有诱惑力的特性。

OSGi 描绘了一个很美好的模块化开发目标，而且定义了实现这个目标的所需要服务与架构，同时也有成熟的框架进行实现支持。但并非所有的应用都适合采用 OSGi 作为基础架构，它在提供强大功能同时，也引入了额外的复杂度，因为它不遵守了类加载的双亲委托模型。

67、JVM内存模型

线程独占:栈,本地方法栈,程序计数器
线程共享:堆,方法区

68、栈

又称方法栈,线程私有的,线程执行方法是都会创建一个栈阵,用来存储局部变量表,操作栈,动态链接,方法出口等信息.调用方法时执行入栈,方法返回式执行出栈.

69、本地方法栈

与栈类似,也是用来保存执行方法的信息.执行Java方法是使用栈,执行Native方法时使用本地方法栈.

70、程序计数器

保存着当前线程执行的字节码位置,每个线程工作时都有独立的计数器,只为执行Java方法服务,执行Native方法时,程序计数器为空.

71、堆

JVM内存管理最大的一块,对被线程共享,目的是存放对象的实例,几乎所欲的对象实例都会放在这里,当堆没有可用空间时,会抛出OOM异常.根据对象的存活周期不同,JVM把对象进行分代管理,由垃圾回收器进行垃圾的回收管理

72、方法区

又称非堆区,用于存储已被虚拟机加载的类信息,常量,静态变量,即时编译器优化后的代码等数据.1.7的永久代和1.8的元空间都是方法区的一种实现。

73、分代回收

分代回收基于两个事实:大部分对象很快就不使用了,还有一部分不会立即无用,但也不会持续很长时间

堆分代			
年轻代	Dden	Survivor1	Survivor2
老年代	Tenured	Tenured	Tenured
永久代	PremGen/MetaSpace	PremGen/MetaSpace	PremGen/MetaSpace

年轻代->标记-复制
老年代->标记-清除

74、堆和栈的区别

栈是运行时单位，代表着逻辑，内含基本数据类型和堆中对象引用，所在区域连续，没有碎片；堆是存储单位，代表着数据，可被多个栈共享（包括成员中基本数据类型、引用和引用对象），所在区域不连续，会有碎片。
1、功能不同
栈内存用来存储局部变量和方法调用，而堆内存用来存储Java中的对象。无论是成员变量，局部变量，还是类变量，它们指向的对象都存储在堆内存中。
2、共享性不同
栈内存是线程私有的。
堆内存是所有线程共有的。
3、异常错误不同
如果栈内存或者堆内存不足都会抛出异常。
栈空间不足：java.lang.StackOverFlowError。
堆空间不足：java.lang.OutOfMemoryError。
4、空间大小
栈的空间大小远远小于堆的

75、什么时候会触发FullGC

除直接调用System.gc外，触发Full GC执行的情况有如下四种。

1. **旧生代空间不足**
   旧生代空间只有在新生代对象转入及创建为大对象、大数组时才会出现不足的现象，当执行Full GC后空间仍然不足，则抛出如下错误：
   java.lang.OutOfMemoryError: Java heap space
   为避免以上两种状况引起的FullGC，调优时应尽量做到让对象在Minor GC阶段被回收、让对象在新生代多存活一段时间及不要创建过大的对象及数组。
2. **Permanet Generation空间满**
   PermanetGeneration中存放的为一些class的信息等，当系统中要加载的类、反射的类和调用的方法较多时，Permanet Generation可能会被占满，在未配置为采用CMS GC的情况下会执行Full GC。如果经过Full GC仍然回收不了，那么JVM会抛出如下错误信息：
   java.lang.OutOfMemoryError: PermGen space
   为避免Perm Gen占满造成Full GC现象，可采用的方法为增大Perm Gen空间或转为使用CMS GC。
3. **CMS GC时出现promotion failed和concurrent mode failure**
   对于采用CMS进行旧生代GC的程序而言，尤其要注意GC日志中是否有promotion failed和concurrent mode failure两种状况，当这两种状况出现时可能会触发Full GC。

   promotionfailed是在进行Minor GC时，survivor space放不下、对象只能放入旧生代，而此时旧生代也放不下造成的；concurrent mode failure是在执行CMS GC的过程中同时有对象要放入旧生代，而此时旧生代空间不足造成的。

   应对措施为：增大survivorspace、旧生代空间或调低触发并发GC的比率，但在JDK 5.0+、6.0+的版本中有可能会由于JDK的bug29导致CMS在remark完毕后很久才触发sweeping动作。对于这种状况，可通过设置-XX:CMSMaxAbortablePrecleanTime=5（单位为ms）来避免。
4. **统计得到的Minor GC晋升到旧生代的平均大小大于旧生代的剩余空间**
   这是一个较为复杂的触发情况，Hotspot为了避免由于新生代对象晋升到旧生代导致旧生代空间不足的现象，在进行Minor GC时，做了一个判断，如果之前统计所得到的Minor GC晋升到旧生代的平均大小大于旧生代的剩余空间，那么就直接触发Full GC。
   例如程序第一次触发MinorGC后，有6MB的对象晋升到旧生代，那么当下一次Minor GC发生时，首先检查旧生代的剩余空间是否大于6MB，如果小于6MB，则执行Full GC。
   当新生代采用PSGC时，方式稍有不同，PS GC是在Minor GC后也会检查，例如上面的例子中第一次Minor GC后，PS GC会检查此时旧生代的剩余空间是否大于6MB，如小于，则触发对旧生代的回收。除了以上4种状况外，对于使用RMI来进行RPC或管理的Sun JDK应用而言，默认情况下会一小时执行一次Full GC。可通过在启动时通过- java-Dsun.rmi.dgc.client.gcInterval=3600000来设置Full GC执行的间隔时间或通过-XX:+ DisableExplicitGC来禁止RMI调用System.gc

76、什么是Java虚拟机？为什么Java被称作是“平台无关的编程语言”？

Java虚拟机是一个可以执行Java字节码的虚拟机进程。Java源文件被编译成能被Java虚拟机执行的字节码文件。 Java被设计成允许应用程序可以运行在任意的平台，而不需要程序员为每一个平台单独重写或者是重新编译。Java虚拟机让这个变为可能，因为它知道底层硬件平台的指令长度和其他特性。

77、对象分配规则

1. 对象优先分配在Eden区，如果Eden区没有足够的空间时，虚拟机执行一次Minor GC。
2. 大对象直接进入老年代（大对象是指需要大量连续内存空间的对象）。这样做的目的是避免在Eden区和两个Survivor区之间发生大量的内存拷贝（新生代采用复制算法收集内存）。
3. 长期存活的对象进入老年代。虚拟机为每个对象定义了一个年龄计数器，如果对象经过了1次Minor GC那么对象会进入Survivor区，之后每经过一次Minor GC那么对象的年龄加1，知道达到阀值对象进入老年区。
4. 动态判断对象的年龄。如果Survivor区中相同年龄的所有对象大小的总和大于Survivor空间的一半，年龄大于或等于该年龄的对象可以直接进入老年代。
5. 空间分配担保。每次进行Minor GC时，JVM会计算Survivor区移至老年区的对象的平均大小，如果这个值大于老年区的剩余值大小则进行一次Full GC，如果小于检查HandlePromotionFailure设置，如果true则只进行Monitor GC,如果false则进行Full GC

78、描述一下JVM加载class文件的原理机制？

JVM中类的装载是由类加载器（ClassLoader）和它的子类来实现的，Java中的类加载器是一个重要的Java运行时系统组件，它负责在运行时查找和装入类文件中的类。

由于Java的跨平台性，经过编译的Java源程序并不是一个可执行程序，而是一个或多个类文件。当Java程序需要使用某个类时，JVM会确保这个类已经被加载、连接（验证、准备和解析）和初始化。

类的加载是指把类的.class文件中的数据读入到内存中，通常是创建一个字节数组读入.class文件，然后产生与所加载类对应的Class对象。加载完成后，Class对象还不完整，所以此时的类还不可用。

当类被加载后就进入连接阶段，这一阶段包括验证、准备（为静态变量分配内存并设置默认的初始值）和解析（将符号引用替换为直接引用）三个步骤。最后JVM对类进行初始化，

包括：

1)如果类存在直接的父类并且这个类还没有被初始化，那么就先初始化父类；

2)如果类中存在初始化语句，就依次执行这些初始化语句。 类的加载是由类加载器完成的，类加载器包括：根加载器（BootStrap）、扩展加载器（Extension）、系统加载器（System）和用户自定义类加载器（java.lang.ClassLoader的子类）。

从Java 2（JDK 1.2）开始，类加载过程采取了父亲委托机制（PDM）。PDM更好的保证了Java平台的安全性，在该机制中，JVM自带的Bootstrap是根加载器，其他的加载器都有且仅有一个父类加载器。类的加载首先请求父类加载器加载，父类加载器无能为力时才由其子类加载器自行加载。JVM不会向Java程序提供对Bootstrap的引用。下面是关于几个类加载器的说明

Bootstrap：一般用本地代码实现，负责加载JVM基础核心类库（rt.jar）；
Extension：从java.ext.dirs系统属性所指定的目录中加载类库，它的父加载器是Bootstrap；
System：又叫应用类加载器，其父类是Extension。它是应用最广泛的类加载器。它从环境变量classpath或者系统属性java.class.path所指定的目录中记载类，是用户自定义加载器的默认父加载器。

79、Java对象创建过程

1.JVM遇到一条新建对象的指令时首先去检查这个指令的参数是否能在常量池中定义到一个类的符号引用。然后加载这个类（类加载过程在后边讲）
2.为对象分配内存。一种办法"指针碰撞"、一种办法"空闲列表"，最终常用的办法"本地线程缓冲分配(TLAB)"
3.将除对象头外的对象内存空间初始化为0
4.对对象头进行必要设置

80、简述Java的对象结构

Java对象由三个部分组成：对象头、实例数据、对齐填充。
对象头由两部分组成，第一部分存储对象自身的运行时数据：哈希码、GC分代年龄、锁标识状态、线程持有的锁、偏向线程ID（一般占32/64 bit）。第二部分是指针类型，指向对象的类元数据类型（即对象代表哪个类）。如果是数组对象，则对象头中还有一部分用来记录数组长度。
实例数据用来存储对象真正的有效信息（包括父类继承下来的和自己定义的）
对齐填充：JVM要求对象起始地址必须是8字节的整数倍（8字节对齐）

81、如何判断对象可以被回收

判断对象是否存活一般有两种方式：
引用计数：每个对象有一个引用计数属性，新增一个引用时计数加1，引用释放时计数减1，计数为0时可以回收。此方法简单，无法解决对象相互循环引用的问题。

可达性分析（Reachability Analysis）：从GC Roots开始向下搜索，搜索所走过的路径称为引用链。当一个对象到GC Roots没有任何引用链相连时，则证明此对象是不可用的，不可达对象。

82、JVM的永久代中会发生垃圾回收么

垃圾回收不会发生在永久代，如果永久代满了或者是超过了临界值，会触发完全垃圾回收(Full GC)。如果你仔细查看垃圾收集器的输出信息，就会发现永久代也是被回收的。这就是为什么正确的永久代大小对避免Full GC是非常重要的原因。请参考下Java8：从永久代到元数据区 (注：Java8中已经移除了永久代，新加了一个叫做元数据区的native内存区)

83、垃圾收集算法

GC最基础的算法有三种： 标记 -清除算法、复制算法、标记-压缩算法，我们常用的垃圾回收器一般都采用分代收集算法。

标记 -清除算法，"标记-清除"（Mark-Sweep）算法，如它的名字一样，算法分为"标记"和"清除"两个阶段：首先标记出所有需要回收的对象，在标记完成后统一回收掉所有被标记的对象。

复制算法，"复制"（Copying）的收集算法，它将可用内存按容量划分为大小相等的两块，每次只使用其中的一块。当这一块的内存用完了，就将还存活着的对象复制到另外一块上面，然后再把已使用过的内存空间一次清理掉。

标记-压缩算法，标记过程仍然与"标记-清除"算法一样，但后续步骤不是直接对可回收对象进行清理，而是让所有存活的对象都向一端移动，然后直接清理掉端边界以外的内存

分代收集算法，"分代收集"（Generational Collection）算法，把Java堆分为新生代和老年代，这样就可以根据各个年代的特点采用最适当的收集算法

84、调优命令有哪些？

Sun JDK监控和故障处理命令有jps jstat jmap jhat jstack jinfo

1. jps，JVM Process Status Tool,显示指定系统内所有的HotSpot虚拟机进程。
2. jstat，JVM statistics Monitoring是用于监视虚拟机运行时状态信息的命令，它可以显示出虚拟机进程中的类装载、内存、垃圾收集、JIT编译等运行数据。
3. jmap，JVM Memory Map命令用于生成heap dump文件
4. jhat，JVM Heap Analysis Tool命令是与jmap搭配使用，用来分析jmap生成的dump，jhat内置了一个微型的HTTP/HTML服务器，生成dump的分析结果后，可以在浏览器中查看
5. jstack，用于生成java虚拟机当前时刻的线程快照。
6. jinfo，JVM Configuration info 这个命令作用是实时查看和调整虚拟机运行参数

85、调优工具

常用调优工具分为两类,jdk自带监控工具：jconsole和jvisualvm，第三方有：MAT(Memory Analyzer Tool)、GChisto。

1. jconsole，Java Monitoring and Management Console是从java5开始，在JDK中自带的java监控和管理控制台，用于对JVM中内存，线程和类等的监控

2. jvisualvm，jdk自带全能工具，可以分析内存快照、线程快照；监控内存变化、GC变化等。
3. MAT，Memory Analyzer Tool，一个基于Eclipse的内存分析工具，是一个快速、功能丰富的Javaheap分析工具，它可以帮助我们查找内存泄漏和减少内存消耗
4. GChisto，一款专业分析gc日志的工具

86、Minor GC与Full GC分别在什么时候发生？

新生代内存不够用时候发生MGC也叫YGC，JVM内存不够的时候发生FGC

87、你知道哪些JVM性能调优

设定堆内存大小
-Xmx：堆内存最大限制。
设定新生代大小。 新生代不宜太小，否则会有大量对象涌入老年代
-XX:NewSize：新生代大小
-XX:NewRatio 新生代和老生代占比
-XX:SurvivorRatio：伊甸园空间和幸存者空间的占比
设定垃圾回收器 年轻代用 -XX:+UseParNewGC 年老代用-XX:+UseConcMarkSweepGC

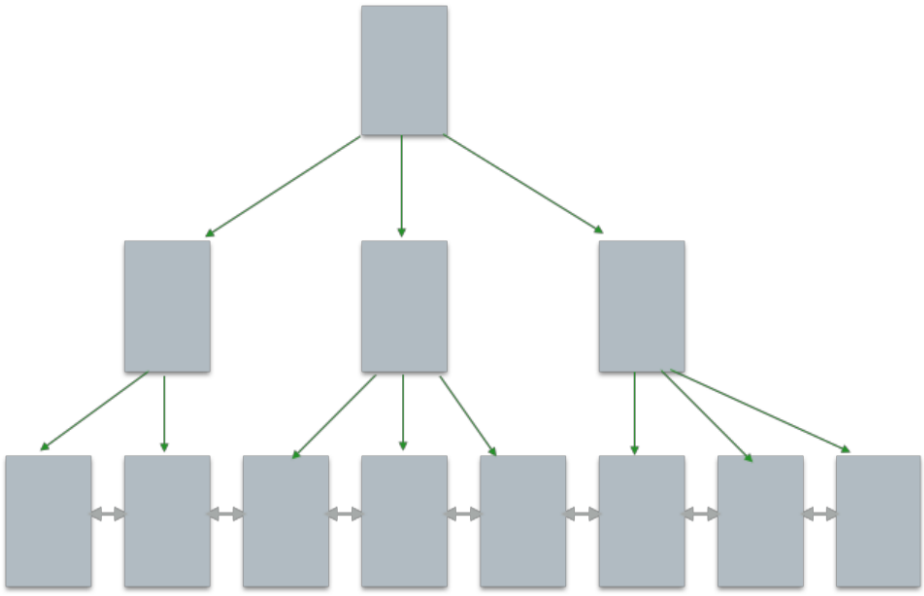
Mysql面试题

1、数据库存储引擎

数据库存储引擎是数据库底层软件组织，数据库管理系统（DBMS）使用数据引擎进行创建、查询、更新和删除数据。不同的存储引擎提供不同的存储机制、索引技巧、锁定水平等功能，使用不同的存储引擎，还可以 获得特定的功能。现在许多不同的数据库管理系统都支持多种不同的数据引擎。存储引擎主要有： 1. MyIsam , 2. InnoDB, 3. Memory, 4. Archive, 5. Federated 。

2、InnoDB（B+树）

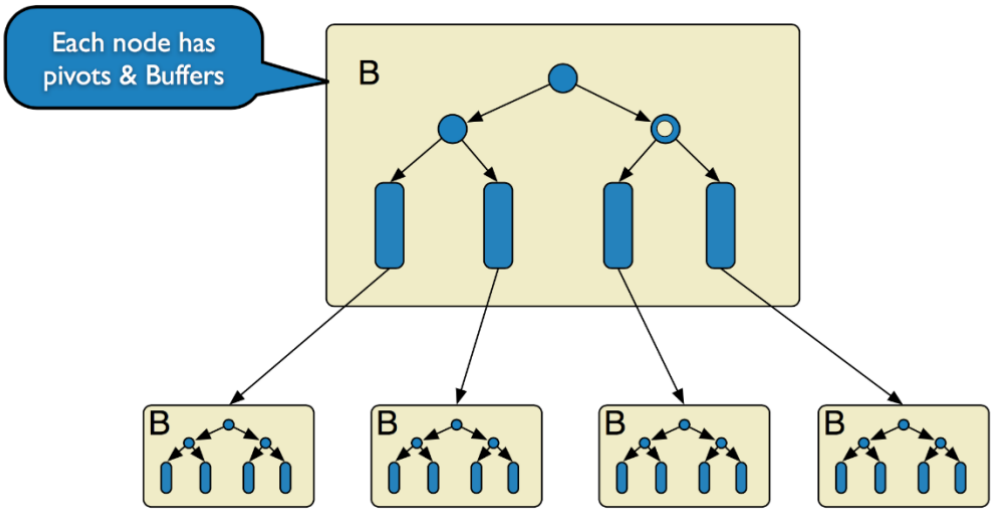
InnoDB 底层存储结构为B+树， B树的每个节点对应innodb的一个page， page大小是固定的，一般设为 16k。其中非叶子节点只有键值，叶子节点包含完成数据



适用场景：
1）经常更新的表，适合处理多重并发的更新请求。
2）支持事务。
3）可以从灾难中恢复（通过 bin-log 日志等）。
4）外键约束。只有他支持外键。
5）支持自动增加列属性 auto_increment。

2、TokuDB（ Fractal Tree-节点带数据 ）

TokuDB 底层存储结构为 Fractal Tree,Fractal Tree 的结构与 B+树有些类似, 在 Fractal Tree中， 每一个 child 指针除了需要指向一个 child 节点外，还会带有一个 Message Buffer ，这个Message Buffer 是一个 FIFO 的队列，用来缓存更新操作。
例如，一次插入操作只需要落在某节点的 Message Buffer 就可以马上返回了，并不需要搜索到叶子节点。这些缓存的更新会在查询时或后台异步合并应用到对应的节点中。



TokuDB 在线添加索引，不影响读写操作, 非常快的写入性能， Fractal-tree 在事务实现上有优势。 他主要适用于访问频率不高的数据或历史数据归档

3、MyIASM

MyIASM是 MySQL默认的引擎，但是它没有提供对数据库事务的支持，也不支持行级锁和外键，因此当 NSERT(插入)或 UPDATE(更新)数据时即写操作需要锁定整个表，效率便会低一些。

ISAM 执行读取操作的速度很快，而且不占用大量的内存和存储资源。在设计之初就预想数据组织成有固定长度的记录，按顺序存储的。 ---ISAM 是一种静态索引结构。

缺点是它不 支持事务处理。

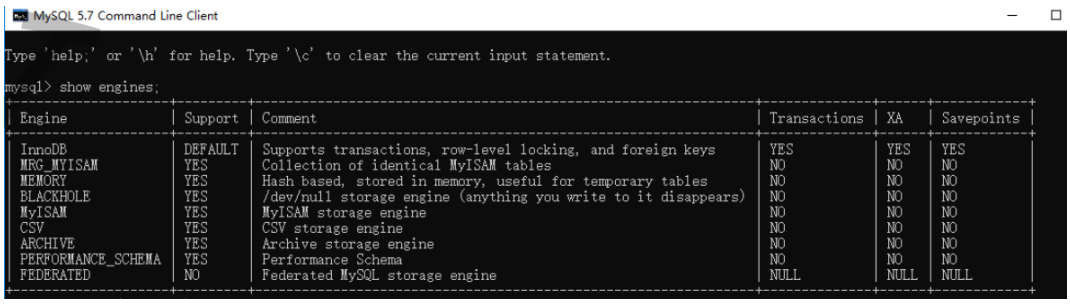
4、Memory

Memory（也叫 HEAP）堆内存：使用存在内存中的内容来创建表。每个 MEMORY 表只实际对应一个磁盘文件。 MEMORY 类型的表访问非常得快，因为它的数据是放在内存中的，并且默认使用HASH 索引。但是一旦服务关闭，表中的数据就会丢失掉。 Memory 同时支持散列索引和 B 树索引， B树索引可以使用部分查询和通配查询，也可以使用<,>和>=等操作符方便数据挖掘，散列索引相等的比较快但是对于范围的比较慢很多

5、数据库引擎有哪些

如何查看mysql提供的所有存储引擎

```
mysql> show engines;
```



mysql常用引擎包括：MYISAM、Innodb、Memory、MERGE

1. MYISAM：全表锁，拥有较高的执行速度，不支持事务，不支持外键，并发性能差，占用空间相对较小，对事务完整性没有要求，以select、insert为主的应用基本上可以使用这引擎
2. Innodb:行级锁，提供了具有提交、回滚和崩溃回复能力的事务安全，支持自动增长列，支持外键约束，并发能力强，占用空间是MYISAM的2.5倍，处理效率相对会差一些
3. Memory:全表锁，存储在内容中，速度快，但会占用和数据量成正比的内存空间且数据在mysql重启时会丢失，默认使用HASH索引，检索效率非常高，但不适用于精确查找，主要用于那些内容变化不频繁的代码表
4. MERGE：是一组MYISAM表的组合

6、InnoDB与MyISAM的区别

1. InnoDB支持事务，MyISAM不支持，对于InnoDB每一条SQL语言都默认封装成事务，自动提交，这样会影响速度，所以最好把多条SQL语言放在begin和commit之间，组成一个事务；
2. InnoDB支持外键，而MyISAM不支持。对一个包含外键的InnoDB表转为MYISAM会失败；
3. InnoDB是聚集索引，数据文件是和索引绑在一起的，必须要有主键，通过主键索引效率很高。但是辅助索引需要两次查询，先查询到主键，然后再通过主键查询到数据。因此，主键不应该过大，因为主键太大，其他索引也都会很大。而MyISAM是非聚集索引，数据文件是分离的，索引保存的是数据文件的指针。主键索引和辅助索引是独立的。
4. InnoDB不保存表的具体行数，执行select count(*) from table时需要全表扫描。而MyISAM用一个变量保存了整个表的行数，执行上述语句时只需要读出该变量即可，速度很快；
5. Innodb不支持全文索引，而MyISAM支持全文索引，查询效率上MyISAM要高

7、索引

索引（Index）是帮助 MySQL 高效获取数据的数据结构。 常见的查询算法,顺序查找,二分查找,二叉排序树查找,哈希散列法,分块查找,平衡多路搜索树 B 树（B-tree） ，索引是对数据库表中一个或多个列的值进行排序的结构，建立索引有助于快速获取信息。

你也可以这样理解：索引就是加快检索表中数据的方法。数据库的索引类似于书籍的索引。在书籍中，索引允许用户不必翻阅完整个书就能迅速地找到所需要的信息。在数据库中，索引也允许数据库程序迅速地找到表中的数据，而不必扫描整个数据库

mysql 有4种不同的索引：
主键索引（PRIMARY）
唯一索引（UNIQUE）
普通索引（INDEX）
全文索引（FULLTEXT）

索引并非是越多越好，创建索引也需要耗费资源，一是增加了数据库的存储空间，二是在插入和删除时要花费较多的时间维护索引
索引加快数据库的检索速度
索引降低了插入、删除、修改等维护任务的速度
唯一索引可以确保每一行数据的唯一性
通过使用索引，可以在查询的过程中使用优化隐藏器，提高系统的性能
索引需要占物理和数据空间

8、常见索引原则有

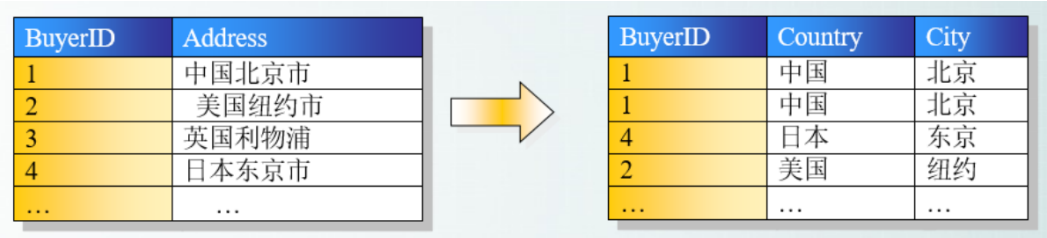
1. 选择唯一性索引，唯一性索引的值是唯一的，可以更快速的通过该索引来确定某条记录。
2. 为经常需要排序、分组和联合操作的字段建立索引。
3. 为常用作为查询条件的字段建立索引。
4. 限制索引的数目：
   越多的索引，会使更新表变得很浪费时间。尽量使用数据量少的索引
5. 如果索引的值很长，那么查询的速度会受到影响。尽量使用前缀来索引
6. 如果索引字段的值很长，最好使用值的前缀来索引。
7. 删除不再使用或者很少使用的索引
8. 最左前缀匹配原则，非常重要的原则。
9. 尽量选择区分度高的列作为索引区分度的公式是表示字段不重复的比例
10. 索引列不能参与计算，保持列“干净”：带函数的查询不参与索引。
11. 尽量的扩展索引，不要新建索引

9、数据库的三范式是什么

第一范式：列不可再分
第二范式：行可以唯一区分，主键约束
第三范式：表的非主属性不能依赖与其他表的非主属性 外键约束
且三大范式是一级一级依赖的，第二范式建立在第一范式上，第三范式建立第一第二范式上 。

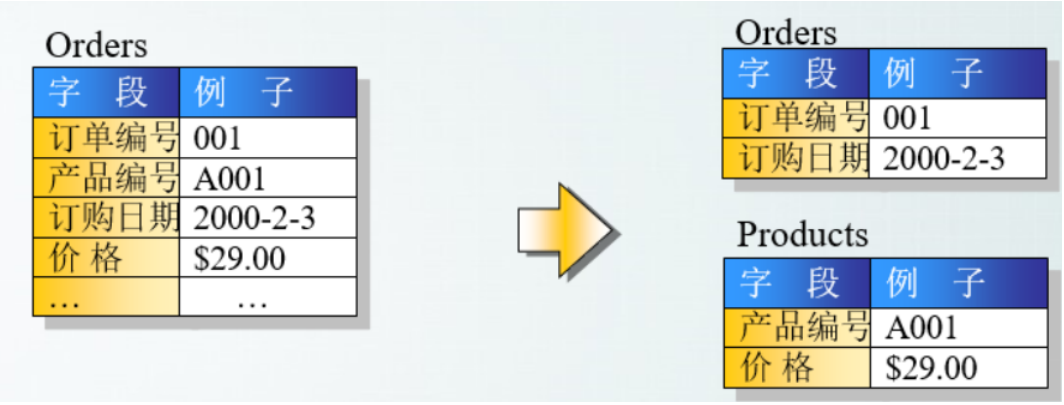
10、第一范式(1st NF - 列都是不可再分)

第一范式的目标是确保每列的原子性:如果每列都是不可再分的最小数据单元（也称为最小的原子单元），则满足第一范式（1NF）



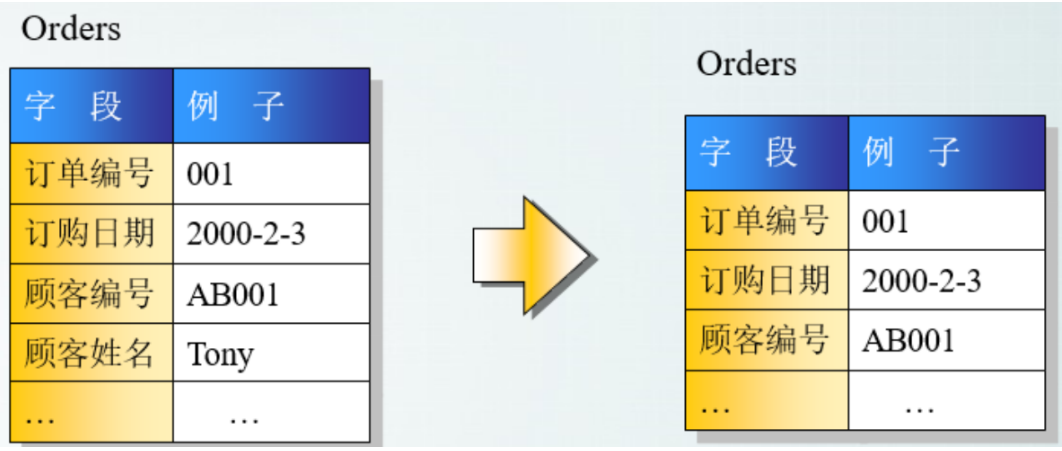
11、第二范式(2nd NF - 每个表只描述一件事情)

首先满足第一范式，并且表中非主键列不存在对主键的部分依赖。 第二范式要求每个表只描述一件事情。



12、第三范式(3rd NF - 不存在对非主键列的传递依赖)

第三范式定义是，满足第二范式，并且表中的列不存在对非主键列的传递依赖。 除了主键订单编号外，顾客姓名依赖于非主键顾客编号。



13、数据库是事务

事务(TRANSACTION)是作为单个逻辑工作单元执行的一系列操作， 这些操作作为一个整体一起向系统提交，要么都执行、要么都不执行 。事务是一个不可分割的工作逻辑单元事务必须具备以下四个属性，简称 ACID 属性：

原子性（Atomicity）

1. 事务是一个完整的操作。事务的各步操作是不可分的（原子的）；要么都执行，要么都不执行。

   一致性（Consistency）

2. 当事务完成时，数据必须处于一致状态。

隔离性（Isolation）

3. 对数据进行修改的所有并发事务是彼此隔离的， 这表明事务必须是独立的，它不应以任何方式依赖于或影响其他事务。

   永久性（Durability）

4. 事务完成后，它对数据库的修改被永久保持，事务日志能够保持事务的永久性

14、SQL优化

1、查询语句中不要使用select *
2、尽量减少子查询，使用关联查询（left join,right join,inner join）替代
3、减少使用IN或者NOT IN ,使用exists，not exists或者关联查询语句替代
4、or 的查询尽量用 union或者union all 代替(在确认没有重复数据或者不用剔除重复数据时，union all会更好)
5、应尽量避免在 where 子句中使用!=或<>操作符，否则将引擎放弃使用索引而进行全表扫描。

6、应尽量避免在 where 子句中对字段进行 null 值判断，否则将导致引擎放弃使用索引而进行全表扫描，如： select id from t where num is null 可以在num上设置默认值0，确保表中num列没有null值，然后这样查询： select id from t where num=0

15、简单说一说drop、delete与truncate的区别

SQL中的drop、delete、truncate都表示删除，但是三者有一些差别
delete和truncate只删除表的数据不删除表的结构
速度,一般来说: drop> truncate >delete
delete语句是dml,这个操作会放到rollback segement中,事务提交之后才生效;
如果有相应的trigger,执行的时候将被触发. truncate,drop是ddl, 操作立即生效,原数据不放到rollbacksegment中,不能回滚. 操作不触发trigger

16、什么是视图

视图是一种虚拟的表，具有和物理表相同的功能。可以对视图进行增，改，查，操作，试图通常是有一个表或者多个表的行或列的子集。对视图的修改不影响基本表。它使得我们获取数据更容易，相比多表查询

17、什么是内联接、左外联接、右外联接？

内联接（Inner Join）：匹配2张表中相关联的记录。
左外联接（Left Outer Join）：除了匹配2张表中相关联的记录外，还会匹配左表中剩余的记录，右表中未匹配到的字段用NULL表示。
右外联接（Right Outer Join）：除了匹配2张表中相关联的记录外，还会匹配右表中剩余的记录，左表中未匹配到的字段用NULL表示。在判定左表和右表时，要根据表名出现在Outer Join的左右位置关系

18、并发事务带来哪些问题?

在典型的应用程序中，多个事务并发运行，经常会操作相同的数据来完成各自的任务（多个用户对同一
数据进行操作）。并发虽然是必须的，但可能会导致以下的问题。

脏读（Dirty read）: 当一个事务正在访问数据并且对数据进行了修改，而这种修改还没有提交到数据库中，这时另外一个事务也访问了这个数据，然后使用了这个数据。因为这个数据是还没有提交的数据，那么另外一个事务读到的这个数据是“脏数据”，依据“脏数据”所做的操作可能是不正确的。

丢失修改（Lost to modify）: 指在一个事务读取一个数据时，另外一个事务也访问了该数据，那么在第一个事务中修改了这个数据后，第二个事务也修改了这个数据。这样第一个事务内的修改结果就被丢失，因此称为丢失修。 例如：事务1读取某表中的数据A=20，事务2也读取A=20，事务1修改A=A-1，事务2也修改A=A-1，最终结果A=19，事务1的修改被丢失。

不可重复读（Unrepeatableread）: 指在一个事务内多次读同一数据。在这个事务还没有结束时，另一个事务也访问该数据。那么，在第一个事务中的两次读数据之间，由于第二个事务的修改导致第一个事务两次读取的数据可能不太一样。这就发生了在一个事务内两次读到的数据是不一样的情况，因此称为不可重复读。

幻读（Phantom read）: 幻读与不可重复读类似。它发生在一个事务（T1）读取了几行数据，接着另一个并发事务（T2）插入了一些数据时。在随后的查询中，第一个事务（T1）就会发现多了一些原本不存在的记录，就好像发生了幻觉一样，所以称为幻读。
不可重复读和幻读区别：
不可重复读的重点是修改比如多次读取一条记录发现其中某些列的值被修改，幻读的重点在于新增或者删除比如多次读取一条记录发现记录增多或减少了

19、事务隔离级别有哪些?MySQL的默认隔离级别是?

SQL 标准定义了四个隔离级别：

READ-UNCOMMITTED(读取未提交)： 最低的隔离级别，允许读取尚未提交的数据变更，可能会导致脏读、幻读或不可重复读。

READ-COMMITTED(读取已提交)： 允许读取并发事务已经提交的数据，可以阻止脏读，但是幻读或不可重复读仍有可能发生。

REPEATABLE-READ(可重复读)： 对同一字段的多次读取结果都是一致的，除非数据是被本身事务自己所修改，可以阻止脏读和不可重复读，但幻读仍有可能发生

SERIALIZABLE(可串行化)： 最高的隔离级别，完全服从ACID的隔离级别。所有的事务依次逐个执行，这样事务之间就完全不可能产生干扰，也就是说，该级别可以防止脏读、不可重复读以及幻读

隔离级别	脏读	不可重复读	幻影读
READ-UNCOMMITTED	√	√	√
READ-COMMITTED	×	√	√
REPEATABLE-READ	×	×	√
SERIALIZABLE	×	×	×

MySQL InnoDB 存储引擎的默认支持的隔离级别是 REPEATABLE-READ（可重读）。我们可以通过
SELECT @@tx_isolation; 命令来查看

```
mysql> SELECT @@tx_isolation;
+-----------------+
| @@tx_isolation |
+-----------------+
| REPEATABLE-READ |
+-----------------+
```

这里需要注意的是：与 SQL 标准不同的地方在于 InnoDB 存储引擎在 REPEATABLE-READ（可重读）事务隔离级别下使用的是Next-Key Lock 锁算法，因此可以避免幻读的产生，这与其他数据库系统(如SQL Server) 是不同的。所以说InnoDB 存储引擎的默认支持的隔离级别是 REPEATABLE-READ（可重读） 已经可以完全保证事务的隔离性要求，即达到了 SQL标准的 SERIALIZABLE(可串行化) 隔离级别。因为隔离级别越低，事务请求的锁越少，所以大部分数据库系统的隔离级别都是 READCOMMITTED(读取提交内容) ，但是你要知道的是InnoDB 存储引擎默认使用 REPEAaTABLEREAD（可重读） 并不会有任何性能损失

InnoDB 存储引擎在 分布式事务 的情况下一般会用到 SERIALIZABLE(可串行化) 隔离级别。

20、大表如何优化？

当MySQL单表记录数过大时，数据库的CRUD性能会明显下降，一些常见的优化措施如下：

限定数据的范围

务必禁止不带任何限制数据范围条件的查询语句。比如：我们当用户在查询订单历史的时候，我们可以控制在一个月的范围内；

读/写分离

经典的数据库拆分方案，主库负责写，从库负责读；

垂直分区

根据数据库里面数据表的相关性进行拆分。 例如，用户表中既有用户的登录信息又有用户的基本信息，可以将用户表拆分成两个单独的表，甚至放到单独的库做分库。

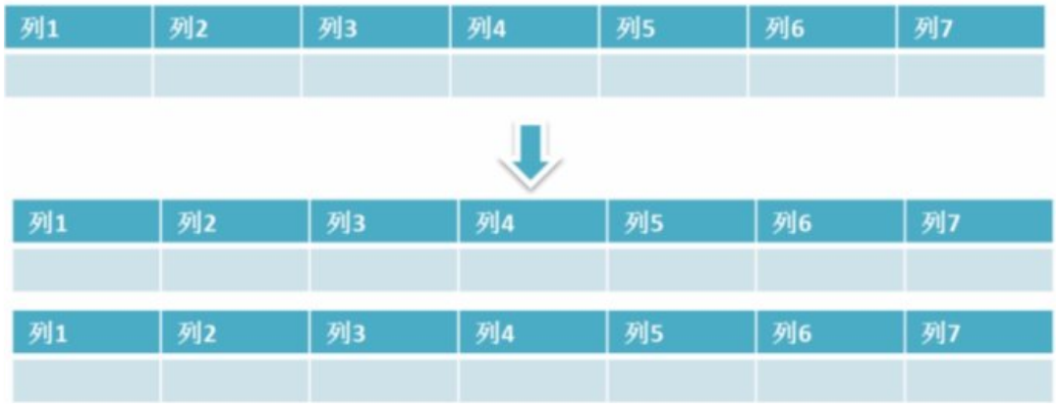
简单来说垂直拆分是指数据表列的拆分，把一张列比较多的表拆分为多张表。 如下图所示，这样来说大家应该就更容易理解了

垂直拆分的优点： 可以使得列数据变小，在查询时减少读取的Block数，减少I/O次数。此外，垂直分区可以简化表的结构，易于维护。

垂直拆分的缺点： 主键会出现冗余，需要管理冗余列，并会引起Join操作，可以通过在应用层进行Join来解决。此外，垂直分区会让事务变得更加复杂；

21、水平分区

保持数据表结构不变，通过某种策略存储数据分片。这样每一片数据分散到不同的表或者库中，达到了分布式的目的。 水平拆分可以支撑非常大的数据量。
水平拆分是指数据表行的拆分，表的行数超过200万行时，就会变慢，这时可以把一张的表的数据拆成多张表来存放。举个例子：我们可以将用户信息表拆分成多个用户信息表，这样就可以避免单一表数据量过大对性能造成影响。



水平拆分可以支持非常大的数据量。需要注意的一点是：分表仅仅是解决了单一表数据过大的问题，但由于表的数据还是在同一台机器上，其实对于提升MySQL并发能力没有什么意义，所以 水平拆分最好分库 。

水平拆分能够 支持非常大的数据量存储，应用端改造也少，但 分片事务难以解决 ，跨节点Join性能较差，逻辑复杂。《Java工程师修炼之道》的作者推荐 尽量不要对数据进行分片，因为拆分会带来逻辑、部署、运维的各种复杂度 ，一般的数据表在优化得当的情况下支撑千万以下的数据量是没有太大问题的。如果实在要分片，尽量选择客户端分片架构，这样可以减少一次和中间件的网络I/O。

1. **下面补充一下数据库分片的两种常见方案：**
   客户端代理： 分片逻辑在应用端，封装在jar包中，通过修改或者封装JDBC层来实现。 当当网的Sharding-JDBC 、阿里的TDDL是两种比较常用的实现。
2. 中间件代理： 在应用和数据中间加了一个代理层。分片逻辑统一维护在中间件服务中。 我们现在谈的 Mycat 、360的Atlas、网易的DDB等等都是这种架构的实现。

详细内容可以参考： MySQL大表优化方案: https://segmentfault.com/a/1190000006158186

22、分库分表之后,id 主键如何处理

因为要是分成多个表之后，每个表都是从 1 开始累加，这样是不对的，我们需要一个全局唯一的 id 来支持。

生成全局 id 有下面这几种方式：
UUID：不适合作为主键，因为太长了，并且无序不可读，查询效率低。比较适合用于生成唯一的名字的标示比如文件的名字。
数据库自增 id：两台数据库分别设置不同步长，生成不重复ID的策略来实现高可用。这种方式生成的 id 有序，但是需要独立部署数据库实例，成本高，还会有性能瓶颈。
利用 redis 生成 id：性能比较好，灵活方便，不依赖于数据库。但是，引入了新的组件造成系统更加复杂，可用性降低，编码更加复杂，增加了系统成本。

Twitter的snowflake算法 ：Github 地址：https://github.com/twitter-archive/snowflake。

美团的Leaf分布式ID生成系统 ：Leaf 是美团开源的分布式ID生成器，能保证全局唯一性、趋势递增、单调递增、信息安全，里面也提到了几种分布式方案的对比，但也需要依赖关系数据库、Zookeeper等中间件。感觉还不错。美团技术团队的一篇文章：https://tech.meituan.com/2017/04/21/mt-leaf.html

23、存储过程(特定功能的 SQL 语句集)

一组为了完成特定功能的 SQL 语句集，存储在数据库中，经过第一次编译后再次调用不需要再次编译，用户通过指定存储过程的名字并给出参数（如果该存储过程带有参数）来执行它。存储过程是数据库中的一个重要对象。

24、存储过程优化思路

1. 尽量利用一些 sql 语句来替代一些小循环，例如聚合函数，求平均函数等。
2. 中间结果存放于临时表，加索引。
3. 少使用游标。 sql 是个集合语言，对于集合运算具有较高性能。而 cursors 是过程运算。比如对一个 100 万行的数据进行查询。游标需要读表 100 万次，而不使用游标则只需要少量几次读取。
4. 事务越短越好。 sqlserver 支持并发操作。如果事务过多过长，或者隔离级别过高，都会造成并发操作的阻塞，死锁。导致查询极慢，cpu 占用率极地。
5. 使用 try-catch 处理错误异常。
6. 查找语句尽量不要放在循环内

25、触发器(一段能自动执行的程序)

触发器是一段能自动执行的程序，是一种特殊的存储过程， 触发器和普通的存储过程的区别是：触发器是当对某一个表进行操作时触发。诸如： update、 insert、 delete 这些操作的时候，系统会自动调用执行该表上对应的触发器。 SQL Server 2005 中触发器可以分为两类：DML 触发器和DDL 触发器，其中 DDL 触发器它们会影响多种数据定义语言语句而激发，这些语句有 create、alter、 drop 语句。

26、数据库并发策略

并发控制一般采用三种方法，分别是乐观锁和悲观锁以及时间戳。

27、MySQL 中有哪几种锁？

1、表级锁：开销小，加锁快；不会出现死锁；锁定粒度大，发生锁冲突的概率最高，并发度最低。
2、行级锁：开销大，加锁慢；会出现死锁；锁定粒度最小，发生锁冲突的概率最低，并发度也最高。
3、页面锁：开销和加锁时间界于表锁和行锁之间；会出现死锁；锁定粒度界于表锁和行锁之间，并发度一般

28、MySQL 中有哪些不同的表格？

共有 5 种类型的表格：
1、MyISAM
2、Heap
3、Merge
4、INNODB
5、ISAM

29、简述在 MySQL 数据库中 MyISAM 和 InnoDB 的区别

MyISAM：

不支持事务，但是每次查询都是原子的；
支持表级锁，即每次操作是对整个表加锁；
存储表的总行数；

一个 MYISAM 表有三个文件：索引文件、表结构文件、数据文件；
采用非聚集索引，索引文件的数据域存储指向数据文件的指针。辅索引与主索引基本一致，但是辅索引不用保证唯一性。

InnoDb：
支持 ACID 的事务，支持事务的四种隔离级别；
支持行级锁及外键约束：因此可以支持写并发；

不存储总行数：
一个 InnoDb 引擎存储在一个文件空间（共享表空间，表大小不受操作系统控制，一个表可能分布在多个文件里），也有可能为多个（设置为独立表空，表大小受操作系统文件大小限制，一般为 2G），受操作系统文件大小的限制；

主键索引采用聚集索引（索引的数据域存储数据文件本身），辅索引的数据域存储主键的值；因此从辅索引查找数据，需要先通过辅索引找到主键值，再访问辅索引；最好使用自增主键，防止插入数据时，为维持 B+树结构，文件的大调整。

30、MySQL 中 InnoDB 支持的四种事务隔离级别名称，以及逐级之间的区别？

SQL 标准定义的四个隔离级别为：
1、read uncommited ：读到未提交数据
2、read committed：脏读，不可重复读
3、repeatable read：可重读
4、serializable ：串行事物

31、CHAR 和 VARCHAR 的区别？

1、CHAR 和 VARCHAR 类型在存储和检索方面有所不同
2、CHAR 列长度固定为创建表时声明的长度，长度值范围是 1 到 255 当 CHAR值被存储时，它们被用空格填充到特定长度，检索 CHAR 值时需删除尾随空格。

32、主键和候选键有什么区别？

表格的每一行都由主键唯一标识,一个表只有一个主键。
主键也是候选键。按照惯例，候选键可以被指定为主键，并且可以用于任何外键引用。

33、myisamchk 是用来做什么的？

它用来压缩 MyISAM 表，这减少了磁盘或内存使用。

34、MyISAM Static 和 MyISAM Dynamic 有什么区别？

在 MyISAM Static 上的所有字段有固定宽度。动态 MyISAM 表将具有像 TEXT，BLOB 等字段，以适应不同长度的数据类型。
MyISAM Static 在受损情况下更容易恢复。

35、如果一个表有一列定义为 TIMESTAMP，将发生什么？

每当行被更改时，时间戳字段将获取当前时间戳。

列设置为 AUTO INCREMENT 时，如果在表中达到最大值，会发生什么情况？
它会停止递增，任何进一步的插入都将产生错误，因为密钥已被使用。

怎样才能找出最后一次插入时分配了哪个自动增量？
LAST_INSERT_ID 将返回由 Auto_increment 分配的最后一个值，并且不需要指定表名称

36、你怎么看到为表格定义的所有索引？

索引是通过以下方式为表格定义的：

```
SHOW INDEX FROM <tablename>;
```

37、LIKE 声明中的%和_是什么意思？

%对应于 0 个或更多字符，_只是 LIKE 语句中的一个字符

如何在 Unix 和 MySQL 时间戳之间进行转换？

UNIX_TIMESTAMP 是从 MySQL 时间戳转换为 Unix 时间戳的命令
FROM_UNIXTIME 是从 Unix 时间戳转换为 MySQL 时间戳的命令

38、列对比运算符是什么？

在 SELECT 语句的列比较中使用=，<>，<=，<，>=，>，<<，>>，<=>，AND，OR 或 LIKE 运算符。

39、BLOB 和 TEXT 有什么区别？

BLOB 是一个二进制对象，可以容纳可变数量的数据。TEXT 是一个不区分大小写的 BLOB。

BLOB 和 TEXT 类型之间的唯一区别在于对 BLOB 值进行排序和比较时区分大小写，对 TEXT 值不区分大小写。

40、MySQL_fetch_array 和 MySQL_fetch_object 的区别是什么？

以下是 MySQL_fetch_array 和 MySQL_fetch_object 的区别：
MySQL_fetch_array（） – 将结果行作为关联数组或来自数据库的常规数组返回。
MySQL_fetch_object – 从数据库返回结果行作为对象。

41、MyISAM 表格将在哪里存储，并且还提供其存储格式？

每个 MyISAM 表格以三种格式存储在磁盘上：
·“.frm”文件存储表定义
·数据文件具有“.MYD”（MYData）扩展名
索引文件具有“.MYI”（MYIndex）扩展名

42、MySQL 如何优化 DISTINCT？

DISTINCT 在所有列上转换为 GROUP BY，并与 ORDER BY 子句结合使用。
SELECT DISTINCT t1.a FROM t1,t2 where t1.a=t2.a;

43、如何显示前 50 行？

在 MySQL 中，使用以下代码查询显示前 50 行：

```
SELECT*FROM  LIMIT 0,50;
```

44、可以使用多少列创建索引？

任何标准表最多可以创建 16 个索引列 。

45、NOW（）和 CURRENT_DATE（）有什么区别？

NOW（）命令用于显示当前年份，月份，日期，小时，分钟和秒。
CURRENT_DATE（）仅显示当前年份，月份和日期。

46、什么是非标准字符串类型？

1、TINYTEXT
2、TEXT
3、MEDIUMTEXT
4、LONGTEXT

47、什么是通用 SQL 函数？

1、CONCAT(A, B) – 连接两个字符串值以创建单个字符串输出。通常用于将两个或多个字段合并为一个字段。

2、FORMAT(X, D)- 格式化数字 X 到 D 有效数字。
3、CURRDATE(), CURRTIME()- 返回当前日期或时间。
4、NOW（ ）– 将当前日期和时间作为一个值返回。
5、MONTH（ ），DAY（ ），YEAR（ ），WEEK（ ），WEEKDAY（ ）– 从日期值中提取给定数据。
6、HOUR（ ），MINUTE（ ），SECOND（ ）– 从时间值中提取给定数据。
7、DATEDIFF（A，B）– 确定两个日期之间的差异，通常用于计算年龄
8、SUBTIMES（A，B）– 确定两次之间的差异。
9、FROMDAYS（INT）– 将整数天数转换为日期值

48、MySQL 支持事务吗？

在缺省模式下，MySQL 是 autocommit 模式的，所有的数据库更新操作都会即时提交，所以在缺省情况下，MySQL 是不支持事务的。
但是如果你的 MySQL 表类型是使用 InnoDB Tables 或 BDB tables 的话，你的MySQL 就可以使用事务处理,使用 SET AUTOCOMMIT=0 就可以使 MySQL 允许在非 autocommit 模式，在非autocommit 模式下，你必须使用 COMMIT 来提交你的更改，或者用 ROLLBACK来回滚你的更改。

49、MySQL 里记录货币用什么字段类型好

NUMERIC 和 DECIMAL 类型被 MySQL 实现为同样的类型，这在 SQL92 标准允许。他们被用于保存值，该值的准确精度是极其重要的值，例如与金钱有关的数据。当声明一个类是这些类型之一时，精度和规模的能被(并且通常是)指定。
例如：

在这个例子中，9(precision)代表将被用于存储值的总的小数位数，而 2(scale)代表将被用于存储小数点后的位数。因此，在这种情况下，能被存储在 salary 列中的值的范围是从-9999999.99 到
9999999.99。

```
salary DECIMAL(9,2)
```

在这个例子中，9(precision)代表将被用于存储值的总的小数位数，而 2(scale)代表将被用于存储小数点后的位数。因此，在这种情况下，能被存储在 salary 列中的值的范围是从-9999999.99 到
9999999.99。

50、MySQL 有关权限的表都有哪几个？

MySQL 服务器通过权限表来控制用户对数据库的访问，权限表存放在 MySQL 数据库里，由 MySQL_install_db 脚本初始化。这些权限表分别 user，db，table_priv，columns_priv 和 host。

51、列的字符串类型可以是什么？

字符串类型是：
1、SET

2、BLOB
3、ENUM
4、CHAR
5、TEXT

52、MySQL 数据库作发布系统的存储，一天五万条以上的增量，预计运维三年,怎么优化？

1、设计良好的数据库结构，允许部分数据冗余，尽量避免 join 查询，提高效率。
2、选择合适的表字段数据类型和存储引擎，适当的添加索引。
3、MySQL 库主从读写分离。
4、找规律分表，减少单表中的数据量提高查询速度。
5、添加缓存机制，比如 memcached，apc 等。
6、不经常改动的页面，生成静态页面。
7、书写高效率的 SQL。比如 SELECT * FROM TABEL 改为 SELECT field_1,field_2, field_3 FROM TABLE.

53、锁的优化策略

1、读写分离
2、分段加锁
3、减少锁持有的时间
4.多个线程尽量以相同的顺序去获取资源不能将锁的粒度过于细化，不然可能会出现线程的加锁和释放次数过多，反而效率不如一次加一把大锁。

54、索引的底层实现原理和优化

B+树，经过优化的 B+树
主要是在所有的叶子结点中增加了指向下一个叶子节点的指针，因此 InnoDB 建议为大部分表使用默认自增的主键作为主索引。

55、什么情况下设置了索引但无法使用

1、以“%”开头的 LIKE 语句，模糊匹配
2、OR 语句前后没有同时使用索引
3、数据类型出现隐式转化（如 varchar 不加单引号的话可能会自动转换为 int 型）

56、实践中如何优化 MySQL

最好是按照以下顺序优化：
1、SQL 语句及索引的优化
2、数据库表结构的优化
3、系统配置的优化
4、硬件的优化
详细可以查看 阿里 P8 架构师谈：MySQL 慢查询优化、索引优化、以及表等优化总结

57、优化数据库的方法

1、选取最适用的字段属性，尽可能减少定义字段宽度，尽量把字段设置 NOTNULL，例如‘省份’、‘性别’最好适用 ENUM
2、使用连接(JOIN)来代替子查询
3、适用联合(UNION)来代替手动创建的临时表
4、事务处理
5、锁定表、优化事务处理
6、适用外键，优化锁定表
7、建立索引
8、优化查询语句

58、简单描述 MySQL 中，索引，主键，唯一索引，联合索引的区别，对数据库的性能有什么影响（从读写两方面）

索引是一种特殊的文件(InnoDB 数据表上的索引是表空间的一个组成部分)，它们包含着对数据表里所有记录的引用指针。

普通索引(由关键字 KEY 或 INDEX 定义的索引)的唯一任务是加快对数据的访问速度。

普通索引允许被索引的数据列包含重复的值。如果能确定某个数据列将只包含彼此各不相同的值，在为这个数据列创建索引的时候就应该用关键字 UNIQUE 把它定义为一个唯一索引。也就是说，唯一索引可以保证数据记录的唯一性。

主键，是一种特殊的唯一索引，在一张表中只能定义一个主键索引，主键用于唯一标识一条记录，使用关键字 PRIMARY KEY 来创建。

索引可以覆盖多个数据列，如像 INDEX(columnA, columnB)索引，这就是联合索引。

索引可以极大的提高数据的查询速度，但是会降低插入、删除、更新表的速度，因为在执行这些写操作时，还要操作索引文件

59、数据库中的事务是什么?

事务（transaction）是作为一个单元的一组有序的数据库操作。如果组中的所有操作都成功，则认为事务成功，即使只有一个操作失败，事务也不成功。如果所有操作完成，事务则提交，其修改将作用于所有其他数据库进程。如果一个操作失败，则事务将回滚，该事务所有操作的影响都将取消。

事务特性：
1、原子性：即不可分割性，事务要么全部被执行，要么就全部不被执行。

2、一致性或可串性。事务的执行使得数据库从一种正确状态转换成另一种正确状态

3、隔离性。在事务正确提交之前，不允许把该事务对数据的任何改变提供给任何其他事物
4、持久性。事务正确提交后，其结果将永久保存在数据库中，即使在事务提交后有了其他故障，事务的处理结果也会得到保存。

或者这样理解：
事务就是被绑定在一起作为一个逻辑工作单元的 SQL 语句分组，如果任何一个语句操作失败那么整个操作就被失败，以后操作就会回滚到操作前状态，或者是上有个节点。为了确保要么执行，要么不执行，就可以使用事务。要将有组语句作为事务考虑，就需要通过 ACID 测试，即原子性，一致性，隔离性和持久性。

60、SQL 注入漏洞产生的原因？如何防止？

SQL 注入产生的原因：程序开发过程中不注意规范书写 sql 语句和对特殊字符进行过滤，导致客户端可以通过全局变量 POST 和 GET 提交一些 sql 语句正常执行。
防止 SQL 注入的方式：
开启配置文件中的 magic_quotes_gpc 和 magic_quotes_runtime 设置 执行 sql 语句时使用 addslashes 进行 sql 语句转换

Sql 语句书写尽量不要省略双引号和单引号。

过滤掉 sql 语句中的一些关键词：update、insert、delete、select、 * 。

提高数据库表和字段的命名技巧，对一些重要的字段根据程序的特点命名，取不易被猜到的。

61、为表中得字段选择合适得数据类型

字段类型优先级: 整形>date,time>enum,char>varchar>blob,text优先考虑数字类型，其次是日期或者二进制类型，最后是字符串类型，同级别得数据类型，应该优先选择占用空间小的数据类型

62、存储时期

Datatime:以 YYYY-MM-DD HH:MM:SS 格式存储时期时间，精确到秒，占用 8 个字节得存储空间，datatime 类型与时区无关

Timestamp:以时间戳格式存储，占用 4 个字节，范围小 1970-1-1 到 2038-1-19，显示依赖于所指定得时区，默认在第一个列行的数据修改时可以自动得修改

timestamp 列得值
Date:（生日）占用得字节数比使用字符串.datatime.int 储存要少，使用 date 只需要 3 个字节，存储日期月份，还可以利用日期时间函数进行日期间得计算

Time:存储时间部分得数据
注意:不要使用字符串类型来存储日期时间数据（通常比字符串占用得储存空间小，在进行查找过滤可以利用日期得函数）

使用 int 存储日期时间不如使用 timestamp 类型

63、对于关系型数据库而言，索引是相当重要的概念，请回答有关索引的几个问题

1、索引的目的是什么？
快速访问数据表中的特定信息，提高检索速度

创建唯一性索引，保证数据库表中每一行数据的唯一性。

加速表和表之间的连接

使用分组和排序子句进行数据检索时，可以显著减少查询中分组和排序的时间

2、索引对数据库系统的负面影响是什么？
负面影响：
创建索引和维护索引需要耗费时间，这个时间随着数据量的增加而增加；索引需要占用物理空间，不光是表需要占用数据空间，每个索引也需要占用物理空间；当对表进行增、删、改、的时候索引也要动态维护，这样就降低了数据的维护速度。

3、为数据表建立索引的原则有哪些？
在最频繁使用的、用以缩小查询范围的字段上建立索引。

在频繁使用的、需要排序的字段上建立索引

4、什么情况下不宜建立索引？

对于查询中很少涉及的列或者重复值比较多的列，不宜建立索引。

对于一些特殊的数据类型，不宜建立索引，比如文本字段（text）等

64、解释 MySQL 外连接、内连接与自连接的区别

先说什么是交叉连接: 交叉连接又叫笛卡尔积，它是指不使用任何条件，直接将一个表的所有记录和另一个表中的所有记录一一匹配。

内连接 则是只有条件的交叉连接，根据某个条件筛选出符合条件的记录，不符合条件的记录不会出现在结果集中，即内连接只连接匹配的行。

外连接 其结果集中不仅包含符合连接条件的行，而且还会包括左表、右表或两个表中的所有数据行，这三种情况依次称之为左外连接，右外连接，和全外连接。

左外连接，也称左连接，左表为主表，左表中的所有记录都会出现在结果集中，对于那些在右表中并没有匹配的记录，仍然要显示，右边对应的那些字段值以NULL 来填充。右外连接，也称右连接，右表为主表，右表中的所有记录都会出现在结果集中。左连接和右连接可以互换，MySQL 目前还不支持全外连接。

65、Myql 中的事务回滚机制概述

事务是用户定义的一个数据库操作序列，这些操作要么全做要么全不做，是一个不可分割的工作单位，事务回滚是指将该事务已经完成的对数据库的更新操作撤销。

要同时修改数据库中两个不同表时，如果它们不是一个事务的话，当第一个表修改完，可能第二个表修改过程中出现了异常而没能修改，此时就只有第二个表依旧是未修改之前的状态，而第一个表已经被修改完毕。而当你把它们设定为一个事务的时候，当第一个表修改完，第二表修改出现异常而没能修改，第一个表和第二个表都要回到未修改的状态，这就是所谓的事务回滚

66、SQL 语言包括哪几部分？每部分都有哪些操作关键

SQL 语言包括数据定义(DDL)、数据操纵(DML),数据控制(DCL)和数据查询（DQL）四个部分。
数据定义：Create Table,Alter Table,Drop Table, Craete/Drop Index 等
数据操纵：Select ,insert,update,delete,
数据控制：grant,revoke
数据查询：select

67、完整性约束包括哪些？

数据完整性(Data Integrity)是指数据的精确(Accuracy)和可靠性(Reliability)。

分为以下四类：
1、实体完整性：规定表的每一行在表中是惟一的实体。
2、域完整性：是指表中的列必须满足某种特定的数据类型约束，其中约束又包括取值范围、精度等规定。
3、参照完整性：是指两个表的主关键字和外关键字的数据应一致，保证了表之间的数据的一致性，防止了数据丢失或无意义的数据在数据库中扩散。

4、用户定义的完整性：不同的关系数据库系统根据其应用环境的不同，往往还需要一些特殊的约束条件。用户定义的完整性即是针对某个特定关系数据库的约束条件，它反映某一具体应用必须满足的语义要求。与表有关的约束：包括列约束(NOT NULL（非空约束）)和表约束(PRIMARY KEY、foreign key、check、UNIQUE)。

68、什么是锁？

答：数据库是一个多用户使用的共享资源。当多个用户并发地存取数据时，在数据库中就会产生多个事务同时存取同一数据的情况。若对并发操作不加控制就可能会读取和存储不正确的数据，破坏数据库的一致性。

加锁是实现数据库并发控制的一个非常重要的技术。当事务在对某个数据对象进行操作前，先向系统发出请求，对其加锁。加锁后事务就对该数据对象有了一定的控制，在该事务释放锁之前，其他的事务不能对此数据对象进行更新操作。

基本锁类型：锁包括行级锁和表级锁

69、什么叫视图？游标是什么？

视图是一种虚拟的表，具有和物理表相同的功能。可以对视图进行增，改，查，操作，视图通常是有一个表或者多个表的行或列的子集。对视图的修改不影响基本表。它使得我们获取数据更容易，相比多表查询。

游标：是对查询出来的结果集作为一个单元来有效的处理。游标可以定在该单元中的特定行，从结果集的当前行检索一行或多行。可以对结果集当前行做修改。一般不使用游标，但是需要逐条处理数据的时候，游标显得十分重要。

70、什么是存储过程？用什么来调用？

答：存储过程是一个预编译的 SQL 语句，优点是允许模块化的设计，就是说只需创建一次，以后在该程序中就可以调用多次。如果某次操作需要执行多次 SQL，使用存储过程比单纯 SQL 语句执行要快。可以用一个命令对象来调用存储过程。

71、如何通俗地理解三个范式？

第一范式：1NF 是对属性的原子性约束，要求属性具有原子性，不可再分解；
第二范式：2NF 是对记录的惟一性约束，要求记录有惟一标识，即实体的惟一性；
第三范式：3NF 是对字段冗余性的约束，即任何字段不能由其他字段派生出来，它要求字段没有冗余。

范式化设计优缺点:
优点:
可以尽量得减少数据冗余，使得更新快，体积小

缺点:对于查询需要多个表进行关联，减少写得效率增加读得效率，更难进行索引
优化
反范式化:
优点:可以减少表得关联，可以更好得进行索引优化

缺点:数据冗余以及数据异常，数据得修改需要更多的成本

72、什么是基本表？什么是视图？

答：基本表是本身独立存在的表，在 SQL 中一个关系就对应一个表。 视图是从一个或几个基本表导出的表。视图本身不独立存储在数据库中，是一个虚表

73、试述视图的优点？

(1) 视图能够简化用户的操作

(2) 视图使用户能以多种角度看待同一数据；
(3) 视图为数据库提供了一定程度的逻辑独立性；

(4) 视图能够对机密数据提供安全保护

74、NULL 是什么意思

NULL 这个值表示 UNKNOWN(未知):它不表示""(空字符串)。对 NULL 这个值的任何比较都会生产一个 NULL 值。您不能把任何值与一个 NULL 值进行比较，并在逻辑上希望获得一个答案。使用 IS NULL 来进行 NULL 判断

75、主键、外键和索引的区别？

主键、外键和索引的区别
定义 ：

主键–唯一标识一条记录，不能有重复的，不允许为空
外键–表的外键是另一表的主键, 外键可以有重复的, 可以是空值
索引–该字段没有重复值，但可以有一个空值
作用：
主键–用来保证数据完整性
外键–用来和其他表建立联系用的
索引–是提高查询排序的速度
个数：
主键–主键只能有一个
外键–一个表可以有多个外键
索引–一个表可以有多个唯一索引

76、你可以用什么来确保表格里的字段只接受特定范围里的值?

Check 限制，它在数据库表格里被定义，用来限制输入该列的值。触发器也可以被用来限制数据库表格里的字段能够接受的值，但是这种办法要求触发器在表格里被定义，这可能会在某些情况下影响到性能。

77、说说对 SQL 语句优化有哪些方法？（选择几条）

1、Where 子句中：where 表之间的连接必须写在其他 Where 条件之前，那些可以过滤掉最大数量记录的条件必须写在 Where 子句的末尾.HAVING 最后。

2、用 EXISTS 替代 IN、用 NOT EXISTS 替代 NOT IN。

3、 避免在索引列上使用计算

4、避免在索引列上使用 IS NULL 和 IS NOT NULL

5、对查询进行优化，应尽量避免全表扫描，首先应考虑在 where 及 order by 涉及的列上建立索引。

6、应尽量避免在 where 子句中对字段进行 null 值判断，否则将导致引擎放弃使用索引而进行全表扫描

7、应尽量避免在 where 子句中对字段进行表达式操作，这将导致引擎放弃使用索引而进行全表扫描

78、什么是乐观锁

乐观锁认为一个用户读数据的时候，别人不会去写自己所读的数据；悲观锁就刚好相反，觉得自己读数据库的时候，别人可能刚好在写自己刚读的数据，其实就是持一种比较保守的态度；时间戳就是不加锁，通过时间戳来控制并发出现的问题。

79、什么是悲观锁

悲观锁就是在读取数据的时候，为了不让别人修改自己读取的数据，就会先对自己读取的数据加锁，只有自己把数据读完了，才允许别人修改那部分数据，或者反过来说，就是自己修改某条数据的时候，不允许别人读取该数据，只有等自己的整个事务提交了，才释放自己加上的锁，才允许其他用户访问那部分数据。

80、什么是时间戳

时间戳就是在数据库表中单独加一列时间戳，比如"TimeStamp"， 每次读出来的时候，把该字段也读出来，当写回去的时候，把该字段加 1，提交之前 ，跟数据库的该字段比较一次，如果比数据库的值大的话，就允许保存，否则不允许保存，这种处理方法虽然不使用数据库系统提供的锁机制，但是这种方法可以大大提高数据库处理的并发量，以上悲观锁所说的加"锁"，其实分为几种锁，分别是： 排它锁（写锁）和共享锁（读锁）。

81、什么是行级锁

行级锁是一种排他锁，防止其他事务修改此行；在使用以下语句时， Oracle 会自动应用行级锁：

1. INSERT、 UPDATE、 DELETE、 SELECT ... FOR UPDATE [OF columns] [WAIT n | NOWAIT];
2. SELECT ... FOR UPDATE 语句允许用户一次锁定多条记录进行更新
3. 使用 COMMIT 或 ROLLBACK 语句释放锁。

82、什么是表级锁

表示对当前操作的整张表加锁，它实现简单，资源消耗较少，被大部分 MySQL 引擎支持。最常使用的 MYISAM 与 INNODB 都支持表级锁定。表级锁定分为表共享读锁（共享锁）与表独占写锁（排他锁）。

83、什么是页级锁

页级锁是 MySQL 中锁定粒度介于行级锁和表级锁中间的一种锁。表级锁速度快，但冲突多，行级冲突少，但速度慢。所以取了折衷的页级，一次锁定相邻的一组记录。 BDB 支持页级锁

Redis面试题

1、什么是 Redis?

Redis 是完全开源免费的，遵守 BSD 协议，是一个高性能的 key-value 数据库。

Redis 与其他 key - value 缓存产品有以下三个特点：
Redis 支持数据的持久化，可以将内存中的数据保存在磁盘中，重启的时候可以再次加载进行使用。

Redis 不仅仅支持简单的 key-value 类型的数据，同时还提供 list，set，zset，hash 等数据结构的存储。

Redis 支持数据的备份，即 master-slave 模式的数据备份。

Redis 优势
性能极高 – Redis 能读的速度是 110000 次/s,写的速度是 81000 次/s 。丰富的数据类型 – Redis 支持二进制案例的 Strings, Lists, Hashes, Sets 及Ordered Sets 数据类型操作。

原子 – Redis 的所有操作都是原子性的，意思就是要么成功执行要么失败完全不执行。单个操作是原子性的。多个操作也支持事务，即原子性，通过 MULTI 和 EXEC指令包起来。

丰富的特性 – Redis 还支持 publish/subscribe, 通知, key 过期等等特性。

2、Redis 与其他 key-value 存储有什么不同？

Redis 有着更为复杂的数据结构并且提供对他们的原子性操作，这是一个不同于其他数据库的进化路径。Redis 的数据类型都是基于基本数据结构的同时对程序员透明，无需进行额外的抽象。Redis 运行在内存中但是可以持久化到磁盘，所以在对不同数据集进行高速读写时需要权衡内存，因为数据量不能大于硬件内存。在内存数据库方面的另一个优点是，相比在磁盘上相同的复杂的数据结构，在内存中操作起来非常简单，这样 Redis可以做很多内部复杂性很强的事情。同时，在磁盘格式方面他们是紧凑的以追加的方式产生的，因为他们并不需要进行随机访问。

3、Redis 的数据类型？

Redis 支持五种数据类型：string（字符串），hash（哈希），list（列表），set（集合）及 zsetsorted set：有序集合)。

我们实际项目中比较常用的是 string，hash 如果你是 Redis 中高级用户，还需要加上下面几种数据结构 HyperLogLog、Geo、Pub/Sub。

如果你说还玩过 Redis Module，像 BloomFilter，RedisSearch，Redis-ML，面试官得眼睛就开始发亮了。

4、使用 Redis 有哪些好处？

1、速度快，因为数据存在内存中，类似于 HashMap，HashMap 的优势就是查找和操作的时间复杂度都是 O1)
2、支持丰富数据类型，支持 string，list，set，Zset，hash 等
3、支持事务，操作都是原子性，所谓的原子性就是对数据的更改要么全部执行，要么全部不执行
4、丰富的特性：可用于缓存，消息，按 key 设置过期时间，过期后将会自动删除

5、Redis 相比 Memcached 有哪些优势？

1、Memcached 所有的值均是简单的字符串，redis 作为其替代者，支持更为丰富的数据类
2、Redis 的速度比 Memcached 快很
3、Redis 可以持久化其数据

6、Memcache 与 Redis 的区别都有哪些？

1、存储方式 Memecache 把数据全部存在内存之中，断电后会挂掉，数据不能超过内存大小。 Redis 有部份存在硬盘上，这样能保证数据的持久性。

2、数据支持类型 Memcache 对数据类型支持相对简单。 Redis 有复杂的数据类型。

3、使用底层模型不同 它们之间底层实现方式 以及与客户端之间通信的应用协议不一样。 Redis 直接自己构建了 VM 机制 ，因为一般的系统调用系统函数的话，会浪费一定的时间去移动和请求。

7、Redis 是单进程单线程的？

Redis 是单进程单线程的，redis 利用队列技术将并发访问变为串行访问，消除了传统数据库串行控制的开销。

8、一个字符串类型的值能存储最大容量是多少？

512M

9、Redis持久化机制

Redis是一个支持持久化的内存数据库，通过持久化机制把内存中的数据同步到硬盘文件来保证数据持久化。当Redis重启后通过把硬盘文件重新加载到内存，就能达到恢复数据的目的。

实现：单独创建fork()一个子进程，将当前父进程的数据库数据复制到子进程的内存中，然后由子进程写入到临时文件中，持久化的过程结束了，再用这个临时文件替换上次的快照文件，然后子进程退出，内存释放。

RDB是Redis默认的持久化方式。按照一定的时间周期策略把内存的数据以快照的形式保存到硬盘的二进制文件。即Snapshot快照存储，对应产生的数据文件为dump.rdb，通过配置文件中的save参数来定义快照的周期。（ 快照可以是其所表示的数据的一个副本，也可以是数据的一个复制品。 ）

AOF：Redis会将每一个收到的写命令都通过Write函数追加到文件最后，类似于MySQL的binlog。当Redis重启是会通过重新执行文件中保存的写命令来在内存中重建整个数据库的内容。当两种方式同时开启时，数据恢复Redis会优先选择AOF恢复。

10、缓存雪崩、缓存穿透、缓存预热、缓存更新、缓存降级等问题

一、缓存雪崩

我们可以简单的理解为：由于原有缓存失效，新缓存未到期间(例如：我们设置缓存时采用了相同的过期时间，在同一时刻出现大面积的缓存过期)，所有原本应该访问缓存的请求都去查询数据库了，而对数据库CPU和内存造成巨大压力，严重的会造成数据库宕机。从而形成一系列连锁反应，造成整个系统崩溃。

解决办法：
大多数系统设计者考虑用加锁（ 最多的解决方案）或者队列的方式保证来保证不会有大量的线程对数据库一次性进行读写，从而避免失效时大量的并发请求落到底层存储系统上。还有一个简单方案就时讲缓存失效时间分散开。

二、缓存穿透
缓存穿透是指用户查询数据，在数据库没有，自然在缓存中也不会有。这样就导致用户查询的时候，在缓存中找不到，每次都要去数据库再查询一遍，然后返回空（相当于进行了两次无用的查询）。这样请求就绕过缓存直接查数据库，这也是经常提的缓存命中率问题。

解决办法;
最常见的则是采用布隆过滤器，将所有可能存在的数据哈希到一个足够大的bitmap中，一个一定不存在的数据会被这个bitmap拦截掉，从而避免了对底层存储系统的查询压力。另外也有一个更为简单粗暴的方法，如果一个查询返回的数据为空（不管是数据不存在，还是系统故障），我们仍然把这个空结果进行缓存，但它的过期时间会很短，最长不超过五分钟。通过这个直接设置的默认值存放到缓存，这样第二次到缓冲中获取就有值了，而不会继续访问数据库，这种办法最简单粗暴。

5TB的硬盘上放满了数据，请写一个算法将这些数据进行排重。如果这些数据是一些32bit大小的数据该如何解决？如果是64bit的呢？

对于空间的利用到达了一种极致，那就是Bitmap和布隆过滤器(Bloom Filter)。

Bitmap： 典型的就是哈希表
缺点是，Bitmap对于每个元素只能记录1bit信息，如果还想完成额外的功能，恐怕只能靠牺牲更多的空间、时间来完成了

布隆过滤器（推荐）

就是引入了k(k>1)k(k>1)个相互独立的哈希函数，保证在给定的空间、误判率下，完成元素判重的过程。

它的优点是空间效率和查询时间都远远超过一般的算法，缺点是有一定的误识别率和删除困难。

Bloom-Filter算法的核心思想就是利用多个不同的Hash函数来解决“冲突”。

Hash存在一个冲突（碰撞）的问题，用同一个Hash得到的两个URL的值有可能相同。为了减少冲突，我们可以多引入几个Hash，如果通过其中的一个Hash值我们得出某元素不在集合中，那么该元素肯定不在集合中。只有在所有的Hash函数告诉我们该元素在集合中时，才能确定该元素存在于集合中。这便是Bloom-Filter的基本思想。

Bloom-Filter一般用于在大数据量的集合中判定某元素是否存在。

三、缓存预热

缓存预热这个应该是一个比较常见的概念，相信很多小伙伴都应该可以很容易的理解，缓存预热就是系统上线后，将相关的缓存数据直接加载到缓存系统。这样就可以避免在用户请求的时候，先查询数据库，然后再将数据缓存的问题！用户直接查询事先被预热的缓存数据！
解决思路：
1、直接写个缓存刷新页面，上线时手工操作下；
2、数据量不大，可以在项目启动的时候自动进行加载；
3、定时刷新缓存

四、缓存更新

除了缓存服务器自带的缓存失效策略之外（Redis默认的有6中策略可供选择），我们还可以根据具体的业务需求进行自定义的缓存淘汰，常见的策略有两种：

（1）定时去清理过期的缓存；
（2）当有用户请求过来时，再判断这个请求所用到的缓存是否过期，过期的话就去底层系统得到新数
据并更新缓存。

两者各有优劣，第一种的缺点是维护大量缓存的key是比较麻烦的，第二种的缺点就是每次用户请求过来都要判断缓存失效，逻辑相对比较复杂！具体用哪种方案，大家可以根据自己的应用场景来权衡

五、缓存降级

当访问量剧增、服务出现问题（如响应时间慢或不响应）或非核心服务影响到核心流程的性能时，仍然需要保证服务还是可用的，即使是有损服务。系统可以根据一些关键数据进行自动降级，也可以配置开关实现人工降级。

降级的最终目的是保证核心服务可用，即使是有损的。而且有些服务是无法降级的（如加入购物车、结算）。

以参考日志级别设置预案：
（1）一般：比如有些服务偶尔因为网络抖动或者服务正在上线而超时，可以自动降级；
（2）警告：有些服务在一段时间内成功率有波动（如在95~100%之间），可以自动降级或人工降级，并发送告警；
（3）错误：比如可用率低于90%，或者数据库连接池被打爆了，或者访问量突然猛增到系统能承受的最大阀值，此时可以根据情况自动降级或者人工降级；
（4）严重错误：比如因为特殊原因数据错误了，此时需要紧急人工降级。服务降级的目的，是为了防止Redis服务故障，导致数据库跟着一起发生雪崩问题。因此，对于不重要的缓存数据，可以采取服务降级策略，例如一个比较常见的做法就是，Redis出现问题，不去数据库查询，而是直接返回默认值给用户

11、热点数据和冷数据是什么

热点数据，缓存才有价值
对于冷数据而言，大部分数据可能还没有再次访问到就已经被挤出内存，不仅占用内存，而且价值不大。频繁修改的数据，看情况考虑使用缓存

对于上面两个例子，寿星列表、导航信息都存在一个特点，就是信息修改频率不高，读取通常非常高的场景。

对于热点数据，比如我们的某IM产品，生日祝福模块，当天的寿星列表，缓存以后可能读取数十万次。再举个例子，某导航产品，我们将导航信息，缓存以后可能读取数百万次。
数据更新前至少读取两次，缓存才有意义。这个是最基本的策略，如果缓存还没有起作用就失效了，那就没有太大价值了。

那存不存在，修改频率很高，但是又不得不考虑缓存的场景呢？有！比如，这个读取接口对数据库的压力很大，但是又是热点数据，这个时候就需要考虑通过缓存手段，减少数据库的压力，比如我们的某助手产品的，点赞数，收藏数，分享数等是非常典型的热点数据，但是又不断变化，此时就需要将数据同步保存到Redis缓存，减少数据库压力

12、单线程的redis为什么这么快

(一)纯内存操作
(二)单线程操作，避免了频繁的上下文切换
(三)采用了非阻塞I/O多路复用机制

13、redis的数据类型，以及每种数据类型的使用场景

一共五种
(一)String
这个其实没啥好说的，最常规的set/get操作，value可以是String也可以是数字。一般做一些复杂的计数功能的缓存。
(二)hash
这里value存放的是结构化的对象，比较方便的就是操作其中的某个字段。博主在做单点登录的时候，就是用这种数据结构存储用户信息，以cookieId作为key，设置30分钟为缓存过期时间，能很好的模拟出类似session的效果。
(三)list
使用List的数据结构，可以做简单的消息队列的功能。另外还有一个就是，可以利用lrange命令，做基于redis的分页功能，性能极佳，用户体验好。本人还用一个场景，很合适—取行情信息。就也是个生产者和消费者的场景。LIST可以很好的完成排队，先进先出的原则。
(四)set
因为set堆放的是一堆不重复值的集合。所以可以做全局去重的功能。为什么不用JVM自带的Set进行去重？因为我们的系统一般都是集群部署，使用JVM自带的Set，比较麻烦，难道为了一个做一个全局去重，再起一个公共服务，太麻烦了。
另外，就是利用交集、并集、差集等操作，可以计算共同喜好，全部的喜好，自己独有的喜好等功能。
(五)sorted set
sorted set多了一个权重参数score,集合中的元素能够按score进行排列。可以做排行榜应用，取TOP N操作

14、redis的过期策略以及内存淘汰机制

redis采用的是定期删除+惰性删除策略。

为什么不用定时删除策略?

定时删除,用一个定时器来负责监视key,过期则自动删除。虽然内存及时释放，但是十分消耗CPU资源。在大并发请求下，CPU要将时间应用在处理请求，而不是删除key,因此没有采用这一策略.

定期删除+惰性删除是如何工作的呢?
定期删除，redis默认每个100ms检查，是否有过期的key,有过期key则删除。需要说明的是，redis不是每个100ms将所有的key检查一次，而是随机抽取进行检查(如果每隔100ms,全部key进行检查，redis岂不是卡死)。因此，如果只采用定期删除策略，会导致很多key到时间没有删除。于是，惰性删除派上用场。也就是说在你获取某个key的时候，redis会检查一下，这个key如果设置了过期时间那么是否过期了？如果过期了此时就会删除。

采用定期删除+惰性删除就没其他问题了么?
不是的，如果定期删除没删除key。然后你也没即时去请求key，也就是说惰性删除也没生效。这样，redis的内存会越来越高。那么就应该采用内存淘汰机制。在redis.conf中有一行配置

```
maxmemory-policy volatile-lru
```

该配置就是配内存淘汰策略的(什么，你没配过？好好反省一下自己)
volatile-lru：从已设置过期时间的数据集（server.db[i].expires）中挑选最近最少使用的数据淘汰
volatile-ttl：从已设置过期时间的数据集（server.db[i].expires）中挑选将要过期的数据淘汰
volatile-random：从已设置过期时间的数据集（server.db[i].expires）中任意选择数据淘汰
allkeys-lru：从数据集（server.db[i].dict）中挑选最近最少使用的数据淘汰
allkeys-random：从数据集（server.db[i].dict）中任意选择数据淘汰
no-enviction（驱逐）：禁止驱逐数据，新写入操作会报错
ps：如果没有设置 expire 的key, 不满足先决条件(prerequisites); 那么 volatile-lru, volatile-random 和volatile-ttl 策略的行为, 和 noeviction(不删除) 基本上一致

15、Redis 常见性能问题和解决方案？

(1) Master 最好不要做任何持久化工作，如 RDB 内存快照和 AOF 日志文件
(2) 如果数据比较重要，某个 Slave 开启 AOF 备份数据，策略设置为每秒同步一次
(3) 为了主从复制的速度和连接的稳定性， Master 和 Slave 最好在同一个局域网内
(4) 尽量避免在压力很大的主库上增加从库
(5) 主从复制不要用图状结构，用单向链表结构更为稳定，即： Master <- Slave1 <- Slave2 <-Slave3...

16、为什么Redis的操作是原子性的，怎么保证原子性的？

对于Redis而言，命令的原子性指的是：一个操作的不可以再分，操作要么执行，要么不执行。
Redis的操作之所以是原子性的，是因为Redis是单线程的。
Redis本身提供的所有API都是原子操作，Redis中的事务其实是要保证批量操作的原子性。
多个命令在并发中也是原子性的吗？
不一定， 将get和set改成单命令操作，incr 。使用Redis的事务，或者使用Redis+Lua==的方式实现。

17、Redis事务

Redis事务功能是通过MULTI、EXEC、DISCARD和WATCH 四个原语实现的
Redis会将一个事务中的所有命令序列化，然后按顺序执行。
1.redis 不支持回滚“Redis 在事务失败时不进行回滚，而是继续执行余下的命令”， 所以 Redis 的内部可以保持简单且快速。
2.如果在一个事务中的命令出现错误，那么所有的命令都不会执行；
3.如果在一个事务中出现运行错误，那么正确的命令会被执行。
1）MULTI命令用于开启一个事务，它总是返回OK。 MULTI执行之后，客户端可以继续向服务器发送任意多条命令，这些命令不会立即被执行，而是被放到一个队列中，当EXEC命令被调用时，所有队列中的命令才会被执行。
2）EXEC：执行所有事务块内的命令。返回事务块内所有命令的返回值，按命令执行的先后顺序排列。当操作被打断时，返回空值 nil 。
3）通过调用DISCARD，客户端可以清空事务队列，并放弃执行事务， 并且客户端会从事务状态中退出。
4）WATCH 命令可以为 Redis 事务提供 check-and-set （CAS）行为。 可以监控一个或多个键，一旦其中有一个键被修改（或删除），之后的事务就不会执行，监控一直持续到EXEC命令

18、Redis 的持久化机制是什么？各自的优缺点？

Redis 提供两种持久化机制 RDB 和 AOF 机制:
1、RDBRedis DataBase)持久化方式： 是指用数据集快照的方式半持久化模式)记录 redis 数据库的所有键值对,在某个时间点将数据写入一个临时文件，持久化结束后，用这个临时文件替换上次持久化的文件，达到数据恢复。
优点：
1、只有一个文件 dump.rdb，方便持久化。
2、容灾性好，一个文件可以保存到安全的磁盘。
3、性能最大化，fork 子进程来完成写操作，让主进程继续处理命令，所以是 IO最大化。使用单独子进程来进行持久化，主进程不会进行任何 IO 操作，保证了 redis的高性能) 4.相对于数据集大时，比 AOF 的启动效率更高。
缺点：
1、数据安全性低。RDB 是间隔一段时间进行持久化，如果持久化之间 redis 发生故障，会发生数据丢失。所以这种方式更适合数据要求不严谨的时候)
2、AOFAppend-only file)持久化方式： 是指所有的命令行记录以 redis 命令请求协议的格式完全持久化存储)保存为 aof 文件。

优点：

1、数据安全，aof 持久化可以配置 appendfsync 属性，有 always，每进行一次命令操作就记录到 aof 文件中一次。
2、通过 append 模式写文件，即使中途服务器宕机，可以通过 redis-check-aof工具解决数据一致性问题。
3、AOF 机制的 rewrite 模式。AOF 文件没被 rewrite 之前（文件过大时会对命令进行合并重写），可以删除其中的某些命令（比如误操作的 flushall））

缺点：
1、AOF 文件比 RDB 文件大，且恢复速度慢。
2、数据集大的时候，比 rdb 启动效率低。

19、Redis 常见性能问题和解决方案：

1、Master 最好不要写内存快照，如果 Master 写内存快照，save 命令调度 rdbSave函数，会阻塞主线程的工作，当快照比较大时对性能影响是非常大的，会间断性暂停服务

2、如果数据比较重要，某个 Slave 开启 AOF 备份数据，策略设置为每秒同步一3、为了主从复制的速度和连接的稳定性，Master 和 Slave 最好在同一个局域网

4、尽量避免在压力很大的主库上增加从

5、主从复制不要用图状结构，用单向链表结构更为稳定，即：Master <- Slave1<- Slave2 <- Slave3…这样的结构方便解决单点故障问题，实现 Slave 对 Master的替换。如果 Master 挂了，可以立刻启用 Slave1 做 Master，其他不变。

20、redis 过期键的删除策略？

1、定时删除:在设置键的过期时间的同时，创建一个定时器 timer). 让定时器在键的过期时间来临时，立即执行对键的删除操作。

2、惰性删除:放任键过期不管，但是每次从键空间中获取键时，都检查取得的键是否过期，如果过期的话，就删除该键;如果没有过期，就返回该键。

3、定期删除:每隔一段时间程序就对数据库进行一次检查，删除里面的过期键。至于要删除多少过期键，以及要检查多少个数据库，则由算法决定。

21、Redis 的回收策略（淘汰策略）？

volatile-lru：从已设置过期时间的数据集（server.db[i].expires）中挑选最近最少使用的数据淘汰

volatile-ttl：从已设置过期时间的数据集（server.db[i].expires）中挑选将要过期的数据淘汰

volatile-random：从已设置过期时间的数据集（server.db[i].expires）中任意选择数据淘汰

allkeys-lru：从数据集（server.db[i].dict）中挑选最近最少使用的数据淘汰

allkeys-random：从数据集（server.db[i].dict）中任意选择数据淘汰

no-enviction（驱逐）：禁止驱逐数据

> 注意这里的 6 种机制，volatile 和 allkeys 规定了是对已设置过期时间的数据集淘汰数据还是从全部数据集淘汰数据，后面的 lru、ttl 以及 random 是三种不同的淘汰策略，再加上一种 no-enviction 永不回收的策略

使用策略规则：
1、如果数据呈现幂律分布，也就是一部分数据访问频率高，一部分数据访问频率低，则使用 allkeys-lru
2、如果数据呈现平等分布，也就是所有的数据访问频率都相同，则使用allkey

22、为什么 edis 需要把所有数据放到内存中？

Redis 为了达到最快的读写速度将数据都读到内存中，并通过异步的方式将数据写入磁盘。所以 redis 具有快速和数据持久化的特征。如果不将数据放在内存中，磁盘 I/O 速度为严重影响 redis 的性能。在内存越来越便宜的今，redis 将会越来越受欢迎。如果设置了最大使用的内存，则数据已有记录数达到内存限值后不能继续插入新值。

23、Redis 的同步机制了解么？

Redis 可以使用主从同步，从从同步。第一次同步时，主节点做一次 bgsave，并同时将后续修改操作记录到内存 buffer，待完成后将 rdb 文件全量同步到复制节点，复制节点接受完成后将 rdb 镜像加载到内存。加载完成后，再通知主节点将期间修改的操作记录同步到复制节点进行重放就完成了同步过程。

24、Pipeline 有什么好处，为什么要用 pipeline？

可以将多次 IO 往返的时间缩减为一次，前提是 pipeline 执行的指令之间没有因果相关性。使用 redis-benchmark 进行压测的时候可以发现影响 redis 的 QPS峰值的一个重要因素是 pipeline 批次指令的数目。

25、是否使用过 Redis 集群，集群的原理是什么？

1)、Redis Sentinal 着眼于高可用，在 master 宕机时会自动将 slave 提升为master，继续提供服务。
2)、Redis Cluster 着眼于扩展性，在单个 redis 内存不足时，使用 Cluster 进行分片存储。

26、Redis 集群方案什么情况下会导致整个集群不可用？

答：有 A，B，C 三个节点的集群,在没有复制模型的情况下,如果节点 B 失败了，那么整个集群就会以为缺少 5501-11000 这个范围的槽而不可用。

27、Redis 支持的 Java 客户端都有哪些？官方推荐用哪个？

Redisson、Jedis、lettuce 等等，官方推荐使用 Redisson。

28、Jedis 与 Redisson 对比有什么优缺点？

Jedis 是 Redis 的 Java 实现的客户端，其 API 提供了比较全面的 Redis 命令的支持；Redisson 实现了分布式和可扩展的 Java 数据结构，和 Jedis 相比，功能较为简单，不支持字符串操作，不支持排序、事务、管道、分区等 Redis 特性。Redisson 的宗旨是促进使用者对 Redis 的关注分离，从而让使用者能够将精力更集中地放在处理业务逻辑上

29、Redis 如何设置密码及验证密码？

设置密码：config set requirepass 123456
授权密码：auth 123456

30、说说 Redis 哈希槽的概念？

Redis 集群没有使用一致性 hash,而是引入了哈希槽的概念，Redis 集群有16384 个哈希槽，每个 key 通过 CRC16 校验后对 16384 取模来决定放置哪个槽，集群的每个节点负责一部分 hash 槽。

31、Redis 集群的主从复制模型是怎样的？

为了使在部分节点失败或者大部分节点无法通信的情况下集群仍然可用，所以集群使用了主从复制模型,每个节点都会有 N-1 个复制品.

32、Redis 集群会有写操作丢失吗？为什么？

Redis 并不能保证数据的强一致性，这意味这在实际中集群在特定的条件下可能会丢失写操作。

33、Redis 集群之间是如何复制的？

异步复制

34、Redis 集群最大节点个数是多少？

16384 个

35、Redis 集群如何选择数据库？

Redis 集群目前无法做数据库选择，默认在 0 数据库。

36、怎么测试 Redis 的连通性？

使用 ping 命令

37、怎么理解 Redis 事务？

1) 事务是一个单独的隔离操作：事务中的所有命令都会序列化、按顺序地执行。事务在执行的过程中，不会被其他客户端发送来的命令请求所打断。
2) 事务是一个原子操作：事务中的命令要么全部被执行，要么全部都不执行。

38、Redis 事务相关的命令有哪几个？

MULTI、EXEC、DISCARD、WATCH

39、Redis key 的过期时间和永久有效分别怎么设置？

EXPIRE 和 PERSIST 命令。

40、Redis 如何做内存优化？

尽可能使用散列表（hashes），散列表（是说散列表里面存储的数少）使用的内存非常小，所以你应该尽可能的将你的数据模型抽象到一个散列表里面。比如你的 web 系统中有一个用户对象，不要为这个用户的名称，姓氏，邮箱，密码设置单独的 key,而是应该把这个用户的所有信息存储到一张散列表里面.

41、Redis 回收进程如何工作的？

一个客户端运行了新的命令，添加了新的数据。Redi 检查内存使用情况，如果大于 maxmemory 的限制, 则根据设定好的策略进行回收。一个新的命令被执行，等等。所以我们不断地穿越内存限制的边界，通过不断达到边界然后不断地回收回到边界以下。如果一个命令的结果导致大量内存被使用（例如很大的集合的交集保存到一个新的键），不用多久内存限制就会被这个内存使用量超越。

42、都有哪些办法可以降低 Redis 的内存使用情况呢？

如果你使用的是 32 位的 Redis 实例，可以好好利用 Hash,list,sorted set,set等集合类型数据，因为通常情况下很多小的 Key-Value 可以用更紧凑的方式存放到一起。

43、Redis 的内存用完了会发生什么？

如果达到设置的上限，Redis 的写命令会返回错误信息（但是读命令还可以正常返回。）或者你可以将 Redis 当缓存来使用配置淘汰机制，当 Redis 达到内存上限时会冲刷掉旧的内容。

44、一个 Redis 实例最多能存放多少的 keys？List、Set、Sorted Set 他们最多能存放多少元素

理论上 Redis 可以处理多达 232 的 keys，并且在实际中进行了测试，每个实例至少存放了 2 亿 5 千万的 keys。我们正在测试一些较大的值。任何 list、set、和 sorted set 都可以放 232 个元素。换句话说，Redis 的存储极限是系统中的可用内存值。

45、MySQL 里有 2000w 数据，redis 中只存 20w 的数据，如何保证 redis 中的数据都是热点数据？

Redis 内存数据集大小上升到一定大小的时候，就会施行数据淘汰策略。相关知识：Redis 提供 6 种数据淘汰策略：
volatile-lru：从已设置过期时间的数据集（server.db[i].expires）中挑选最近最少使用的数据淘汰
volatile-ttl：从已设置过期时间的数据集（server.db[i].expires）中挑选将要过期的数据淘汰

volatile-random：从已设置过期时间的数据集（server.db[i].expires）中任意选择数据淘汰
allkeys-lru：从数据集（server.db[i].dict）中挑选最近最少使用的数据淘汰
allkeys-random：从数据集（server.db[i].dict）中任意选择数据淘汰
no-enviction（驱逐）：禁止驱逐数据

46、Redis 最适合的场景？

1、会话缓存（Session Cache）
最常用的一种使用 Redis 的情景是会话缓存（session cache）。用 Redis 缓存会话比其他存储（如 Memcached）的优势在于：Redis 提供持久化。当维护一个不是严格要求一致性的缓存时，如果用户的购物车信息全部丢失，大部分人都会不高兴的，现在，他们还会这样吗？幸运的是，随着 Redis 这些年的改进，很容
易找到怎么恰当的使用 Redis 来缓存会话的文档。甚至广为人知的商业平台Magento 也提供 Redis 的插件。

2、全页缓存（FPC）
除基本的会话 token 之外，Redis 还提供很简便的 FPC 平台。回到一致性问题，即使重启了 Redis 实例，因为有磁盘的持久化，用户也不会看到页面加载速度的下降，这是一个极大改进，类似 PHP 本地 FPC。 再次以 agento 为例，Magento提供一个插件来使用 Redis 作为全页缓存后端。 此外，对 WordPress 的用户来说，Pantheon 有一个非常好的插件 wp-redis，这个插件能帮助你以最快速度加载你曾浏览过的页面。

3、队列

Reids 在内存存储引擎领域的一大优点是提供 list 和 set 操作，这使得 Redis能作为一个很好的消息队列平台来使用。Redis 作为队列使用的操作，就类似于本地程序语言（如 Python）对 list 的 push/pop 操作。 如果你快速的在 Google中搜索“Redis queues”，你马上就能找到大量的开源项目，这些项目的目的就是利用 Redis 创建非常好的后端工具，以满足各种队列需求。例如，Celery 有一个后台就是使用 Redis 作为 broker，你可以从这里去查看。

4，排行榜/计数器
Redis 在内存中对数字进行递增或递减的操作实现的非常好。集合（Set）和有序集合（Sorted Set）也使得我们在执行这些操作的时候变的非常简单，Redis 只是正好提供了这两种数据结构。所以，我们要从排序集合中获取到排名最靠前的 10个用户–我们称之为“user_scores”，我们只需要像下面一样执行即可： 当然，这是假定你是根据你用户的分数做递增的排序。如果你想返回用户及用户的分数，你需要这样执行： ZRANGE user_scores 0 10 WITHSCORES Agora Games 就是一个很好的例子，用 Ruby 实现的，它的排行榜就是使用 Redis 来存储数据的，
你可以在这里看到。

5、发布/订阅
最后（但肯定不是最不重要的）是 Redis 的发布/订阅功能。发布/订阅的使用场景确实非常多。我已看见人们在社交网络连接中使用，还可作为基于发布/订阅的脚本触发器，甚至用 Redis 的发布/订阅功能来建立聊天系统！

47、假如 Redis 里面有 1 亿个 key，其中有 10w 个 key 是以某个固定的已知的前缀开头的，如果将它们全部找出来？

使用 keys 指令可以扫出指定模式的 key 列表。 对方接着追问：如果这个 redis 正在给线上的业务提供服务，那使用 keys 指令会**有什么问题？**
这个时候你要回答 redis 关键的一个特性：redis 的单线程的。keys 指令会导致线程阻塞一段时间，线上服务会停顿，直到指令执行完毕，服务才能恢复。这个时候可以使用 scan 指令，scan 指令可以无阻塞的提取出指定模式的 key 列表，但是会有一定的重复概率，在客户端做一次去重就可以了，但是整体所花费的时间会比直接用 keys 指令长。

48、如果有大量的 key 需要设置同一时间过期，一般需要注意什么？

如果大量的 key 过期时间设置的过于集中，到过期的那个时间点，redis 可能会出现短暂的卡顿现象。一般需要在时间上加一个随机值，使得过期时间分散一些。

49、使用过 Redis 做异步队列么，你是怎么用的？

一般使用 list 结构作为队列，rpush 生产消息，lpop 消费消息。当 lpop 没有消息的时候，要适当 sleep 一会再重试。

如果对方追问可不可以不用 sleep 呢？

list 还有个指令叫 blpop，在没有消息的时候，它会阻塞住直到消息到来。如果对方追问能不能生产一次消费多次呢？使用 pub/sub 主题订阅者模式，可以实现1:N 的消息队列。

如果对方追问 pub/sub 有什么缺点？
在消费者下线的情况下，生产的消息会丢失，得使用专业的消息队列如 RabbitMQ等。

如果对方追问 redis 如何实现延时队列？
我估计现在你很想把面试官一棒打死如果你手上有一根棒球棍的话，怎么问的这么详细。但是你很克制，然后神态自若的回答道：使用 sortedset，拿时间戳作为score，消息内容作为 key 调用 zadd 来生产消息，消费者用 zrangebyscore 指令获取 N 秒之前的数据轮询进行处理。到这里，面试官暗地里已经对你竖起了大拇指。但是他不知道的是此刻你却竖起了中指，在椅子背后。

50、使用过 Redis 分布式锁么，它是什么回事

先拿 setnx 来争抢锁，抢到之后，再用 expire 给锁加一个过期时间防止锁忘记了释放。

这时候对方会告诉你说你回答得不错，然后接着问如果在 setnx 之后执行 expire之前进程意外 crash 或者要重启维护了，那会怎么样？

这时候你要给予惊讶的反馈：唉，是喔，这个锁就永远得不到释放了。紧接着你需要抓一抓自己得脑袋，故作思考片刻，好像接下来的结果是你主动思考出来的，然后回答：我记得 set 指令有非常复杂的参数，这个应该是可以同时把 setnx 和expire 合成一条指令来用的！对方这时会显露笑容，心里开始默念：嗯，这小子还不错。

Memcached面试题

1、Memcached 是什么，有什么作用？

Memcached 是一个开源的，高性能的内存缀存软件，从名称上看 Mem 就是内存的意思，而 Cache 就是缓存的意思。Memcached 的作用：通过在事先规划好的内存空间中临时缀存数据库中的各类数据，以达到减少业务对数据库的直接高并发访问，从而达到提升数据库的访问性能，加速网站集群动态应用服务的能力。

2、memcached 服务在企业集群架构中有哪些应用场景？

一、作为数据库的前端缓存应用
a、完整缓存（易），静态缓存
例如：商品分类（京东），以及商品信息，可事先放在内存里，然后再对外提供数据访问，这种先放到内存，我们称之为预热，（先把数据存缓存中），用户访问时可以只读取 memcached 缓存，不读取数据库了。
b、执点缓存（难）
需要前端 web 程序配合，只缓存热点的数据，即缓存经常被访问的数据。先预热数据库里的基础数据，然后在动态更新，选读取缓存，如果缓存里没有对应的数据，程序再去读取数据库，然后程序把读取的新数据放入缓存存储。
特殊说明 ：
 如果碰到电商秒杀等高并发的业务，一定要事先预热，或者其它思想实现，例如：称杀只是获取资格，而不是瞬间秒杀到手商品。

那么什么是获取资格？

 就是在数据库中，把 0 标成 1.就有资格啦。再慢慢的去领取商品订单。因为秒杀过程太长会占用服务器资源。
 如果数据更新，同时触发缓存更新，防止给用户过期数据。
 对于持久化缓存存储系统，例如：redis，可以替代一部分数据库的存储，一些简单的数据业务，投票，统计，好友关注，商品分类等。
nosql= not onlysql

二、作业集群的 session 会话共享存储。
 Memcached 服务在不同企业业务应用场景中的工作流程

 当 web 程序需要访问后端数据库获取数据时会优先访问 Memcached 内存缓存，如果缓存中有数据就直接获取返回前端服务及用户，如果没有数据（没有命中），在由程序请求后端的数据库服务器，获取到对应的数据后，除了返回给前端服务及用户数据外，还会把数据放到 Memcached 内存中进行缓存，等待下次请求被访问，Memcache 内存始终是数据库的挡箭牌，从而大大的减轻数据库的访问压力，提高整个网站架构的响应速度，提升了用户体验。

 当程序更新，修改或删除数据库中已有的数据时，会同时发送请求通知Memcached 已经缓存的同一个 ID 内容的旧数据失效，从而保证 Memcache中数据和数据库中的数据一致。

 如果在高并发场合，除了通知 Memcached 过程的缓存失效外，还会通过相关机制，使得在用户访问新数据前，通过程序预先把更新过的数据推送到memcache 中缓存起来，这样可以减少数据库的访问压力，提升 Memcached中缓存命中率。

 数据库插件可以再写入更新数据库后，自动抛给 MC 缓存起来，自身不Cache.

2、Memcached 服务分布式集群如何实现？

特殊说明：Memcached 集群和 web 服务集群是不一样的，所有 Memcached 的数据总和才是数据库的数据。每台 Memcached 都是部分数据。（一台 memcached 的数据，就是一部分 mysql 数据库的数据）
a、程序端实现
程序加载所有 mc 的 ip 列表，通过对 key 做 hash (一致性哈希算法)

例如：web1 (key)===>对应 A,B,C,D,E,F,G.....若干台服务器。（通过哈希算法实现）
b、负载均衡器
通过对 key 做 hash (一致性哈希算法)一致哈希算法的目的是不但保证每个对象只请求一个对应的服务器，而且当节点宕机，缓存服务器的更新重新分配比例降到最低。

3、Memcached 服务特点及工作原理是什么？

a、完全基于内存缓存的
b、节点之间相互独立
c、C/S 模式架构，C 语言编写，总共 2000 行代码。
d、异步 I /O 模型，使用 libevent 作为事件通知机制。
e、被缓存的数据以 key/value 键值对形式存在的。
f、全部数据存放于内存中，无持久性存储的设计，重启服务器，内存里的数据会丢失。
g、当内存中缓存的数据容量达到启动时设定的内存值时，就自动使用 LRU 算法删除过期的缓存数据。
h、可以对存储的数据设置过期时间，这样过期后的数据自动被清除，服务本身不会监控过期，而是在访问的时候查看 key 的时间戳,判断是否过期。
j、memcache 会对设定的内存进行分块，再把块分组，然后再提供服务

4、简述 Memcached 内存管理机制原理？

早期的 Memcached 内存管理方式是通过 malloc 的分配的内存，使用完后通过free 来回收内存，这种方式容易产生内存碎片，并降低操作系统对内存的管理效率。加重操作系统内存管理器的负担，最坏的情况下，会导致操作系统比memcached 进程本身还慢，为了解决这个问题，Slab Allocation 内存分配机制就延生了。

现在 Memcached 利用 Slab Allocation 机制来分配和管理内存。

Slab Allocation 机制原理是按照预先规定的大小，将分配给 memcached 的内存分割成特定长度的内存块（chunk)，再把尺寸相同的内存块，分成组（chunks slab class),这些内存块不会释放，可以重复利用。
而且，slab allocator 还有重复使用已分配的内存的目的。 也就是说，分配到的内存不会释放，而是重复利用。
Slab Allocation 的主要术语Page分配给 Slab 的内存空间，默认是 1MB。分配给 Slab 之后根据 slab 的大小切分成chunk。
Chunk
用于缓存记录的内存空间。
SlabClass
特定大小的 chunk 的组

5、memcached 是怎么工作的？

Memcached 的神奇来自两阶段哈希（two-stage hash）。Memcached 就像一个巨大的、存储了很多<key,value>对的哈希表。通过 key，可以存储或查询任意的数据。
客户端可以把数据存储在多台 memcached 上。当查询数据时，客户端首先参考节点列表计算出 key 的哈希值（阶段一哈希），进而选中一个节点；客户端将请求发送给选中的节点，然后 memcached 节点通过一个内部的哈希算法（阶段二哈希），查找真正的数据（item）

6、memcached 最大的优势是什么？

Memcached 最大的好处就是它带来了极佳的水平可扩展性，特别是在一个巨大的系统中。由于客户端自己做了一次哈希，那么我们很容易增加大量 memcached到集群中。memcached 之间没有相互通信，因此不会增加 memcached 的负载；没有多播协议，不会网络通信量爆炸（implode）。memcached 的集群很好用。内存不够了？增加几台 memcached 吧；CPU 不够用了？再增加几台吧；有多余的内存？在增加几台吧，不要浪费了。
基于 memcached 的基本原则，可以相当轻松地构建出不同类型的缓存架构。除了这篇 FAQ，在其他地方很容易找到详细资料的。

7、memcached 和 MySQL 的 query

cache 相比，有什么优缺点？

把 memcached 引入应用中，还是需要不少工作量的。MySQL 有个使用方便的query cache，可以自动地缓存 SQL 查询的结果，被缓存的 SQL 查询可以被反复地快速执行。Memcached 与之相比，怎么样呢？MySQL 的 query cache 是集中式的，连接到该 query cache 的 MySQL 服务器都会受益。

□ 当您修改表时，MySQL 的 query cache 会立刻被刷新（flush）。存储一个 memcached item 只需要很少的时间，但是当写操作很频繁时，MySQL的 query cache 会经常让所有缓存数据都失效。

□ 在多核 CPU 上，MySQL 的 query cache 会遇到扩展问题（scalabilityissues）。在多核 CPU 上，query cache 会增加一个全局锁（global lock），由于需要刷新更多的缓存数据，速度会变得更慢。

□ 在 MySQL 的 query cache 中，我们是不能存储任意的数据的（只能是SQL 查询结果）。而利用 emcached，我们可以搭建出各种高效的缓存。比如，可以执行多个独立的查询，构建出一个用户对象（user object），然后将用户对象缓存到 memcached 中。而 query cache 是 SQL 语句级别的，不可能做到这一点。在小的网站中，query cache 会有所帮助，但随着网站规模的增加，query cache 的弊将大于利。

□ query cache能够利用的内存容量受到MySQL服务器空闲内存空间的限制。给数据库服务器增加更多的内存来缓存数据，固然是很好的。但是，有了memcached，只要您有空闲的内存，都可以用来增加 memcached 集群的规
模，然后您就可以缓存更多的数据。

8、memcached 和服务器的 local cache（比如 PHP 的 APC、mmap 文件等）相比，有什么优缺点？

首先，local cache 有许多与上面(query cache)相同的问题。local cache 能够利用的内存容量受到（单台）服务器空闲内存空间的限制。不过，local cache 有一 点比 memcached 和 query cache 都要 好， 那就 是它 不但 可以 存储 任意的 数据 ，而 且没 有网 络存 取的 延迟 。

□ local cache 的数据查询更快。考虑把 highly common 的数据放在 localcache 中吧。如果每个页面都需要加载一些数量较少的数据，考虑把它们放在local cached 吧。

□ local cache 缺少集体失效（groupinvalidation）的特性。在 memcached 集群中，删除或更新一个 key 会让所有的观察者觉察到。但是在 local cache 中, 我们只能通知所有的服务器刷新 cache（很慢，不具扩展性），或者仅仅依赖缓存超时失效机制。

□ local cache 面临着严重的内存限制，这一点上面已经提到。

9、memcached 的 cache 机制是怎样的？

Memcached 主要 的 cache 机制 是 LRU（最 近最 少用 ）算 法 +超时 失效 。当 您存数据 到 memcached 中， 可以 指定 该数 据在 缓存 中可 以呆 多久 Which is forever,or some time in the future。如 果 memcached 的内 存不 够用 了， 过期 的 slabs会优 先被 替换 ，接 着就 轮到 最老 的未 被使 用的 slabs。

10、memcached 如何实现冗余机制？

不实 现！我们 对这 个问 题感 到很 惊讶 。Memcached 应该 是应 用的 缓存 层 。它的 设计本 身就 不带 有任 何冗 余机 制。 如果 一个 memcached 节点 失去 了所 有数 据， 您应该 可以 从数 据源 （比 如数 据库 ）再 次获 取到 数据 。您 应该 特别 注意 ，您 的应 用应该 可以 容忍 节点 的失 效。 不要 写一 些糟 糕的 查询 代码 ，寄 希望 于 memcached 来保证一切！如果您担心节点失效会大大加重数据库的负担，那么您可以采取一些办法。比如您可以增加更多的节点（来减少丢失一个节点的影响），热备节点（在其他节点 down 了的时候接管 IP），等等

11、memcached 如何处理容错的？

不处理！ 在 memcached 节点失效的情况下，集群没有必要做任何容错处理。如果发生了节点失效，应对的措施完全取决于用户。节点失效时，下面列出几种方案供您选择：

□ 忽略它！ 在失效节点被恢复或替换之前，还有很多其他节点可以应对节点失效带来的影响。

□ 把失效的节点从节点列表中移除。做这个操作千万要小心！在默认情况下（余数式哈希算法），客户端添加或移除节点，会导致所有的缓存数据不可用！因为哈希参照的节点列表变化了，大部分 key 会因为哈希值的改变而被映射到（与原来）不同的节点上。

□ 启动热备节点，接管失效节点所占用的 IP。这样可以防止哈希紊乱（hashing chaos）。

□ 如果希望添加和移除节点，而不影响原先的哈希结果，可以使用一致性哈希算法（consistent hashing）。您可以百度一下一致性哈希算法。支持一致性哈希的客户端已经很成熟，而且被广泛使用。去尝试一下吧！

□ 两次哈希（reshing）。当客户端存取数据时，如果发现一个节点 down了，就再做一次哈希（哈希算法与前一次不同），重新选择另一个节点（需要注意的时，客户端并没有把 down 的节点从节点列表中移除，下次还是有可能先哈希到它）。如果某个节点时好时坏，两次哈希的方法就有风险了，好的节点和坏的节点上都可能存在脏数据（stale data）。

12、如何将 memcached 中 item 批量导入导出？

您不应该这样做！Memcached 是一个非阻塞的服务器。任何可能导致memcached 暂停或瞬时拒绝服务的操作都应该值得深思熟虑。向memcached中批量导入数据往往不是您真正想要的！想象看，如果缓存数据在导出导入之间发生了变化，您就需要处理脏数据了；

13、如果缓存数据在导出导入之间过期了，您又怎么处理这些数据呢？

因此，批量导出导入数据并不像您想象中的那么有用。不过在一个场景倒是很有用。如果您有大量的从不变化的数据，并且希望缓存很快热（warm）起来，批量导入缓存数据是很有帮助的。虽然这个场景并不典型，但却经常发生，因此我们会考虑在将来实现批量导出导入的功能。如果一个 memcached 节点 down 了让您很痛苦，那么您还会陷入其他很多麻烦。您的系统太脆弱了。您需要做一些优化工作。比如处理"惊群"问题（比如memcached 节点都失效了，反复的查询让您的数据库不堪重负...这个问题在 FAQ的其他提到过），或者优化不好的查询。记住，Memcached 并不是您逃避优化查询的借口。

14、memcached 是如何做身份验证的？

没有身份认证机制！memcached 是运行在应用下层的软件（身份验证应该是应用上层的职责）。memcached 的客户端和服务器端之所以是轻量级的，部分原因就是完全没有实现身份验证机制。这样，memcached 可以很快地创建新连接，服务器端也无需任何配置。如果您希望限制访问，您可以使用防火墙，或者让 memcached 监听 unix domain socket

15、memcached 的多线程是什么？如何使用它们？

线程就是定律（threads rule）！在 Steven Grimm 和 Facebook 的努力下，memcached 1.2 及更高版本拥有了多线程模式。多线程模式允许 memcached 能够充分利用多个 CPU，并在 CPU 之间共享所有的缓存数据。memcached 使用一种简单的锁机制来保证数据更新操作的互斥。相比在同一个物理机器上运行多个
memcached 实例，这种方式能够更有效地处理 multi gets。

如果您的系统负载并不重，也许您不需要启用多线程工作模式。如果您在运行一个拥有大规模硬件的、庞大的网站，您将会看到多线程的好处。

简单地总结一下：命令解析（memcached 在这里花了大部分时间）可以运行在多线程模式下。memcached 内部对数据的操作是基于很多全局锁的（因此这部分工作不是多线程的）。未来对多线程模式的改进，将移除大量的全局锁，提高memcached 在负载极高的场景下的性能。

16、memcached 能接受的 key 的最大长度是多少？

key 的最大长度是 250 个字符。需要注意的是，250 是 memcached 服务器端内部的限制，如果您使用的客户端支持"key 的前缀"或类似特性，那么 key（前缀+原始 key）的最大长度是可以超过 250 个字符的。我们推荐使用使用较短的 key，因为可以节省内存和带宽。

17、memcached 对 item 的过期时间有什么限制？

过期时间最大可以达到 30 天。memcached 把传入的过期时间（时间段）解释成时间点后，一旦到了这个时间点，memcached 就把 item 置为失效状态。这是一个简单但 obscure 的机制。

18、memcached 最大能存储多大的单个 item？

1MB。如果你的数据大于 1MB，可以考虑在客户端压缩或拆分到多个 key 中。

为什么单个 item 的大小被限制在 1M byte 之内？

啊...这是一个大家经常问的问题！

简单的回答：因为内存分配器的算法就是这样的。

详细的回答：Memcached 的内存存储引擎（引擎将来可插拔...），使用 slabs 来管理内存。内存被分成大小不等的 slabs chunks（先分成大小相等的 slabs，然后每个 slab 被分成大小相等 chunks，不同 slab 的 chunk 大小是不相等的）。chunk的大小依次从一个最小数开始，按某个因子增长，直到达到最大的可能值。

19、memcached 能够更有效地使用内存吗？

Memcache 客户端仅根据哈希算法来决定将某个 key 存储在哪个节点上，而不考虑节点的内存大小。因此，您可以在不同的节点上使用大小不等的缓存。但是一般都是这样做的：拥有较多内存的节点上可以运行多个 memcached 实例，每个实例使用的内存跟其他节点上的实例相同。

20、什么是二进制协议，我该关注吗？

关于二进制最好的信息当然是二进制协议规范：

二进制协议尝试为端提供一个更有效的、可靠的协议，减少客户端/服务器端因处理协议而产生的 CPU 时间。

根据 Facebook 的测试，解析 ASCII 协议是 memcached 中消耗 CPU 时间最多的环节。所以，我们为什么不改进 ASCII 协议呢？

21、memcached 的内存分配器是如何工作的？为什么不适用malloc/free！？为何要使用 slabs？

实际上，这是一个编译时选项。默认会使用内部的 slab 分配器。您确实确实应该使用内建的 slab 分配器。最早的时候，memcached 只使用 malloc/free 来管理内存。然而，这种方式不能与 OS 的内存管理以前很好地工作。反复地 malloc/free造成了内存碎片，OS 最终花费大量的时间去查找连续的内存块来满足 malloc 的请求，而不是运行 memcached 进程。如果您不同意，当然可以使用 malloc！

只是不要在邮件列表中抱怨啊

slab 分配器就是为了解决这个问题而生的。内存被分配并划分成 chunks，一直被重复使用。因为内存被划分成大小不等的 slabs，如果 item 的大小与被选择存放它的 slab 不是很合适的话，就会浪费一些内存。Steven Grimm 正在这方面已经做出了有效的改进。

22、memcached 是原子的吗？

所有的被发送到 memcached 的单个命令是完全原子的。如果您针对同一份数据同时发送了一个 set 命令和一个 get 命令，它们不会影响对方。它们将被串行化、先后执行。即使在多线程模式，所有的命令都是原子的，除非程序有 bug:)

命令序列不是原子的。如果您通过 get 命令获取了一个 item，修改了它，然后想把它 set 回 memcached，我们不保证这个 item 没有被其他进程（process，未必是操作系统中的进程）操作过。在并发的情况下，您也可能覆写了一个被其他进程 set 的 item。

memcached 1.2.5 以及更高版本，提供了 gets 和 cas 命令，它们可以解决上面的问题。如果您使用 gets 命令查询某个 key 的 item，memcached 会给您返回该 item 当前值的唯一标识。如果您覆写了这个 item 并想把它写回到 memcached中，您可以通过 cas 命令把那个唯一标识一起发送给 memcached。如果该 item存放在 memcached 中的唯一标识与您提供的一致，您的写操作将会成功。如果另一个进程在这期间也修改了这个 item，那么该 item 存放在 memcached 中的唯一标识将会改变，您的写操作就会失败

23、如何实现集群中的 session 共享存储？

Session 是运行在一台服务器上的，所有的访问都会到达我们的唯一服务器上，这样我们可以根据客户端传来的 sessionID，来获取 session，或在对应 Session 不存在的情况下（session 生命周期到了/用户第一次登录），创建一个新的 Session；但是，如果我们在集群环境下，假设我们有两台服务器 A，B，用户的请求会由
Nginx 服务器进行转发（别的方案也是同理），用户登录时，Nginx 将请求转发至服务器 A 上，A 创建了新的 session，并将 SessionID 返回给客户端，用户在浏览其他页面时，客户端验证登录状态，Nginx 将请求转发至服务器 B，由于 B 上并没有对应客户端发来 sessionId 的 session，所以会重新创建一个新的 session，并且再将这个新的 sessionID 返回给客户端，这样，我们可以想象一下，用户每一次操作都有 1/2 的概率进行再次的登录，这样不仅对用户体验特别差，还会让服务器上的 session 激增，加大服务器的运行压力。

为了解决集群环境下的 seesion 共享问题，共有 4 种解决方案：
1.粘性 session
粘性 session 是指 Ngnix 每次都将同一用户的所有请求转发至同一台服务器上，即将用户与服务器绑定。
2.服务器 session 复制
即每次 session 发生变化时，创建或者修改，就广播给所有集群中的服务器，使所有的服务器上的 session 相同。
3.session 共享
缓存 session，使用 redis， memcached。
4.session 持久化
将 session 存储至数据库中，像操作数据一样才做 session

24、memcached 与 redis 的区别

1、Redis 不仅仅支持简单的 k/v 类型的数据，同时还提供 list，set，zset，hash等数据结构的存储。而 memcache 只支持简单数据类型，需要客户端自己处理复杂对象

2、Redis 支持数据的持久化，可以将内存中的数据保持在磁盘中，重启的时候可以再次加载进行使用（PS：持久化在 rdb、aof）。

3、由于 Memcache 没有持久化机制，因此宕机所有缓存数据失效。Redis 配置为持久化，宕机重启后，将自动加载宕机时刻的数据到缓存系统中。具有更好的灾备机制。

4、Memcache 可以使用 Magent 在客户端进行一致性 hash 做分布式。Redis 支持在服务器端做分布式（PS:Twemproxy/Codis/Redis-cluster 多种分布式实现方式）

5、Memcached 的简单限制就是键（key）和 Value 的限制。最大键长为 250 个字符。可以接受的储存数据不能超过 1MB（可修改配置文件变大），因为这是典型 slab 的最大值，不适合虚拟机使用。而 Redis 的 Key 长度支持到 512k。

6、Redis 使用的是单线程模型，保证了数据按顺序提交。Memcache 需要使用cas 保证数据一致性。CAS（Check and Set）是一个确保并发一致性的机制，属于“乐观锁”范畴；原理很简单：拿版本号，操作，对比版本号，如果一致就操作，不一致就放弃任何操作cpu 利用。由于 Redis 只使用单核，而 Memcached 可以使用多核，所以平均每一个核上 Redis 在存储小数据时比 Memcached 性能更 高。而在 100k 以上的数据中，Memcached 性能要高于 Redis 。

7、memcache 内存管理：使用 Slab Allocation。原理相当简单，预先分配一系列大小固定的组，然后根据数据大小选择最合适的块存储。避免了内存碎片。（缺点：不能变长，浪费了一定空间）memcached 默认情况下下一个 slab 的最大值为前一个的 1.25 倍。

8、redis 内存管理： Redis 通过定义一个数组来记录所有的内存分配情况， Redis采用的是包装的 malloc/free，相较于 Memcached 的内存 管理方法来说，要简单很多。由于 malloc 首先以链表的方式搜索已管理的内存中可用的空间分配，导致内存碎片比较多

MongoDB面试题

1、mongodb是什么？

MongoDB 是由 C++语言编写的，是一个基于分布式文件存储的开源数据库系统。 在高负载的情况下，添加更多的节点，可以保证服务器性能。 MongoDB 旨在为 WEB 应用提供可扩展的高性能数据存储解决方案。
MongoDB 将数据存储为一个文档，数据结构由键值(key=>value)对组成。 MongoDB 文档类似于 JSON 对象。字段值可以包含其他文档，数组及文档数组。

2、mongodb有哪些特点？

1.MongoDB 是一个面向文档存储的数据库，操作起来比较简单和容易。

2.你可以在 MongoDB 记录中设置任何属性的索引 (如： FirstName="Sameer",Address="8 Gandhi Road")来实现更快的排序。

3.你可以通过本地或者网络创建数据镜像，这使得 MongoDB 有更强的扩展性。

4.如果负载的增加（需要更多的存储空间和更强的处理能力），它可以分布在计算机网络中的其他节点上这就是所谓的分片。

5.Mongo 支持丰富的查询表达式。查询指令使用 JSON 形式的标记，可轻易查询文档中内嵌的对象及数组。

6.MongoDb 使用 update()命令可以实现替换完成的文档（数据）或者一些指定的数据字段 。

7.Mongodb 中的 Map/reduce 主要是用来对数据进行批量处理和聚合操作。

8.Map 和 Reduce。 Map 函数调用 emit(key,value)遍历集合中所有的记录，将 key 与 value 传给 Reduce 函数进行处理。

9.Map 函数和 Reduce 函数是使用 Javascript 编写的，并可以通过 db.runCommand 或 mapreduce 命令来执行 MapReduce 操作。

10. GridFS 是 MongoDB 中的一个内置功能，可以用于存放大量小文件。

11. MongoDB 允许在服务端执行脚本， 可以用 Javascript 编写某个函数，直接在服务端执行，也
可以把函数的定义存储在服务端，下次直接调用即可。

3、你说的NoSQL数据库是什么意思?NoSQL与RDBMS直接有什么区别?为什么要使用和不使用NoSQL数据库?说一说NoSQL数据库的几个优点?

NoSQL是非关系型数据库，NoSQL = Not Only SQL。

关系型数据库采用的结构化的数据，NoSQL采用的是键值对的方式存储数据。

在处理非结构化/半结构化的大数据时；在水平方向上进行扩展时；随时应对动态增加的数据项时可以优先考虑使用NoSQL数据库。

在考虑数据库的成熟度；支持；分析和商业智能；管理及专业性等问题时，应优先考虑关系型数据库。

4、NoSQL数据库有哪些类型?

NoSQL数据库的类型

例如：MongoDB, Cassandra, CouchDB, Hypertable, Redis, Riak, Neo4j, HBASE, Couchbase, MemcacheDB, RevenDB and Voldemort are the examples of NoSQL databases.详细阅读。

5、MySQL与MongoDB之间最基本的差别是什么?

MySQL和MongoDB两者都是免费开源的数据库。MySQL和MongoDB有许多基本差别包括数据的表示(data representation)，查询，关系，事务，schema的设计和定义，标准化(normalization)，速度和性能。

通过比较MySQL和MongoDB，实际上我们是在比较关系型和非关系型数据库，即数据存储结构不同。详细阅读

6、你怎么比较MongoDB、CouchDB及CouchBase?

MongoDB和CouchDB都是面向文档的数据库。MongoDB和CouchDB都是开源NoSQL数据库的最典型代表。 除了都以文档形式存储外它们没有其他的共同点。MongoDB和CouchDB在数据模型实现、接口、对象存储以及复制方法等方面有很多不同。

细节可以参见下面的链接：

MongDB vs CouchDB

CouchDB vs CouchBase

7、MongoDB成为最好NoSQL数据库的原因是什么?

以下特点使得MongoDB成为最好的NoSQL数据库：

- 面向文件的
- 高性能
- 高可用性
- 易扩展性
- 丰富的查询语言

6.32位系统上有什么细微差别?

journaling会激活额外的内存映射文件。这将进一步抑制32位版本上的数据库大小。因此，现在journaling在32位系统上默认是禁用的。

8、journal回放在条目(entry)不完整时(比如恰巧有一个中途故障了)会遇到问题吗?

每个journal (group)的写操作都是一致的，除非它是完整的否则在恢复过程中它不会回放。

9、分析器在MongoDB中的作用是什么?

MongoDB中包括了一个可以显示数据库中每个操作性能特点的数据库分析器。通过这个分析器你可以找到比预期慢的查询(或写操作);利用这一信息，比如，可以确定是否需要添加索引。

10、名字空间(namespace)是什么?

MongoDB存储BSON对象在丛集(collection)中。数据库名字和丛集名字以句点连结起来叫做名字空间(namespace)。

11、 如果用户移除对象的属性，该属性是否从存储层中删除?

是的，用户移除属性然后对象会重新保存(re-save())。

12、能否使用日志特征进行安全备份?

是的。

13、允许空值null吗?

对于对象成员而言，是的。然而用户不能够添加空值(null)到数据库丛集(collection)因为空值不是对象。然而用户能够添加空对象{}。

14、更新操作立刻fsync到磁盘?

不会，磁盘写操作默认是延迟执行的。写操作可能在两三秒(默认在60秒内)后到达磁盘。例如，如果一秒内数据库收到一千个对一个对象递增的操作，仅刷新磁盘一次。(注意，尽管fsync选项在命令行和经过getLastError_old是有效的)(译者：也许是坑人的面试题??)。

15、如何执行事务/加锁?

MongoDB没有使用传统的锁或者复杂的带回滚的事务，因为它设计的宗旨是轻量，快速以及可预计的高性能。可以把它类比成MySQL MyISAM的自动提交模式。通过精简对事务的支持，性能得到了提升，特别是在一个可能会穿过多个服务器的系统里。

16、为什么我的数据文件如此庞大?

MongoDB会积极的预分配预留空间来防止文件系统碎片。

17、启用备份故障恢复需要多久?

从备份数据库声明主数据库宕机到选出一个备份数据库作为新的主数据库将花费10到30秒时间。这期间在主数据库上的操作将会失败--包括写入和强一致性读取(strong consistent read)操作。然而，你还能在第二数据库上执行最终一致性查询(eventually consistent query)(在slaveOk模式下)，即使在这段时间里。

18、什么是master或primary?

它是当前备份集群(replica set)中负责处理所有写入操作的主要节点/成员。在一个备份集群中，当失效备援(failover)事件发生时，一个另外的成员会变成primary。

19、什么是secondary或slave?

Seconday从当前的primary上复制相应的操作。它是通过跟踪复制oplog(local.oplog.rs)做到的。

20、我必须调用getLastError来确保写操作生效了么?

不用。不管你有没有调用getLastError(又叫"Safe Mode")服务器做的操作都一样。调用getLastError只是为了确认写操作成功提交了。当然，你经常想得到确认，但是写操作的安全性和是否生效不是由这个决定的。

21、我应该启动一个集群分片(sharded)还是一个非集群分片的 MongoDB 环境?

为开发便捷起见，我们建议以非集群分片(unsharded)方式开始一个 MongoDB 环境，除非一台服务器不足以存放你的初始数据集。从非集群分片升级到集群分片(sharding)是无缝的，所以在你的数据集还不是很大的时候没必要考虑集群分片(sharding)。

22、分片(sharding)和复制(replication)是怎样工作的?

每一个分片(shard)是一个分区数据的逻辑集合。分片可能由单一服务器或者集群组成，我们推荐为每一个分片(shard)使用集群。

23、数据在什么时候才会扩展到多个分片(shard)里?

MongoDB 分片是基于区域(range)的。所以一个集合(collection)中的所有的对象都被存放到一个块(chunk)中。只有当存在多余一个块的时候，才会有多个分片获取数据的选项。现在，每个默认块的大小是 64Mb，所以你需要至少 64 Mb 空间才可以实施一个迁移。

24、当我试图更新一个正在被迁移的块(chunk)上的文档时会发生什么?

更新操作会立即发生在旧的分片(shard)上，然后更改才会在所有权转移(ownership transfers)前复制到新的分片上。

25、如果在一个分片(shard)停止或者很慢的时候，我发起一个查询会怎样?

如果一个分片(shard)停止了，除非查询设置了“Partial”选项，否则查询会返回一个错误。如果一个分片(shard)响应很慢，MongoDB则会等待它的响应。

26、我可以把moveChunk目录里的旧文件删除吗?

没问题，这些文件是在分片(shard)进行均衡操作(balancing)的时候产生的临时文件。一旦这些操作已经完成，相关的临时文件也应该被删除掉。但目前清理工作是需要手动的，所以请小心地考虑再释放这些文件的空间。

27、我怎么查看 Mongo 正在使用的链接?

db._adminCommand("connPoolStats");

28、如果块移动操作(moveChunk)失败了，我需要手动清除部分转移的文档吗?

不需要，移动操作是一致(consistent)并且是确定性的(deterministic);一次失败后，移动操作会不断重试;当完成后，数据只会出现在新的分片里(shard)。

29、如果我在使用复制技术(replication)，可以一部分使用日志(journaling)而其他部分则不使用吗?

可以。

30、当更新一个正在被迁移的块（Chunk）上的文档时会发生什么？

更新操作会立即发生在旧的块（Chunk）上，然后更改才会在所有权转移前复制到新的分片上。

31、MongoDB在A:{B,C}上建立索引，查询A:{B,C}和A:{C,B}都会使用索引吗？

不会，只会在A:{B,C}上使用索引。

32、如果一个分片（Shard）停止或很慢的时候，发起一个查询会怎样？

如果一个分片停止了，除非查询设置了"Partial"选项，否则查询会返回一个错误。如果一个分片响应很慢，MongoDB会等待它的响应。

33、MongoDB支持存储过程吗？如果支持的话，怎么用？

MongoDB支持存储过程，它是javascript写的，保存在db.system.js表中。

34、如何理解MongoDB中的GridFS机制，MongoDB为何使用GridFS来存储文件？

GridFS是一种将大型文件存储在MongoDB中的文件规范。使用GridFS可以将大文件分隔成多个小文档存放，这样我们能够有效的保存大文档，而且解决了BSON对象有限制的问题。

35、什么是NoSQL数据库？NoSQL和RDBMS有什么区别？在哪些情况下使用和不使用NoSQL数据库？

NoSQL是非关系型数据库，NoSQL = Not Only SQL。

关系型数据库采用的结构化的数据，NoSQL采用的是键值对的方式存储数据。

在处理非结构化/半结构化的大数据时；在水平方向上进行扩展时；随时应对动态增加的数据项时可以优先考虑使用NoSQL数据库。

在考虑数据库的成熟度；支持；分析和商业智能；管理及专业性等问题时，应优先考虑关系型数据库。

36、MongoDB支持存储过程吗？如果支持的话，怎么用？

MongoDB支持存储过程，它是javascript写的，保存在db.system.js表中。

37、如何理解MongoDB中的GridFS机制，MongoDB为何使用GridFS来存储文件？

GridFS是一种将大型文件存储在MongoDB中的文件规范。使用GridFS可以将大文件分隔成多个小文档存放，这样我们能够有效的保存大文档，而且解决了BSON对象有限制的问题。

38、为什么MongoDB的数据文件很大？

MongoDB采用的预分配空间的方式来防止文件碎片。

39、当更新一个正在被迁移的块（Chunk）上的文档时会发生什么？

更新操作会立即发生在旧的块（Chunk）上，然后更改才会在所有权转移前复制到新的分片上。

40、MongoDB在A:{B,C}上建立索引，查询A:{B,C}和A:{C,B}都会使用索引吗？

不会，只会在A:{B,C}上使用索引。

41、如果一个分片（Shard）停止或很慢的时候，发起一个查询会怎样？

如果一个分片停止了，除非查询设置了“Partial”选项，否则查询会返回一个错误。如果一个分片响应很慢，MongoDB会等待它的响应。

42、分析器在MongoDB中的作用是什么?

分析器就是explain 显示每次操作性能特点的数据库分析器。通过分析器可能查找比预期慢的操作

43、如果用户移除对象的属性，该属性是否从存储层中删除？

是的，用户移除属性然后对象会重新保存（re-save()）。

44、能否使用日志特征进行安全备份？

是的

45、更新操作立刻fsync到磁盘？

一般磁盘的写操作都是延迟执行的

46、如何执行事务/加锁？

因为mongodb设计就是轻量高性能，所以没有传统的锁和复杂的事务的回滚

47、什么是master或primary？

当前备份集群负责所有的写入操作的主要节点，在集群中，当主节点（master）失效，另一个成员会变为master

48、getLastError的作用

调用getLastError 可以确认当前的写操作是否成功的提交

49、分片（sharding）和复制（replication）是怎样工作的？

分片可能是单一的服务器或者集群组成，推荐使用集群

50、数据在什么时候才会扩展到多个分片（shard）里？

mongodb分片是基于区域的，所以一个集合的所有对象都放置在同一个块中，只有当存在多余一个块的时候，才会有多个分片获取数据的选项

51、 当我试图更新一个正在被迁移的块（chunk）上的文档时会发生什么？

会立即更新旧的分片，然后更改才会在所有权转移前复制到新的分片上

52、 我怎么查看 Mongo 正在使用的链接？

```
db._adminCommand("connPoolStats");
```

53、mongodb的结构介绍

数据库中存储的对象设计bson，一种类似json的二进制文件，由键值对组成

54、数据库的整体结构

键值对-》文档-》集合-》数据库

55、MongoDB是由哪种语言写的

MongoDB用c++编写的,流行的开源数据库MySQL也是用C++开发的。**C++**1983年发行是一种使用广泛的计算机程序设计语言。它是一种通用程序设计语言，支持多重编程模式。

56、MongoDB的优势有哪些

- 面向文档的存储：以 JSON 格式的文档保存数据。
- 任何属性都可以建立索引。
- 复制以及高可扩展性。
- 自动分片。
- 丰富的查询功能。
- 快速的即时更新。
- 来自 MongoDB 的专业支持。、

57、什么是集合

集合就是一组 MongoDB 文档。它相当于关系型数据库（RDBMS）中的表这种概念。集合位于单独的一个数据库中。一个集合内的多个文档可以有多个不同的字段。一般来说，集合中的文档都有着相同或相关的目的。

58、什么是文档

文档由一组key value组成。文档是动态模式,这意味着同一集合里的文档不需要有相同的字段和结构。在关系型数据库中table中的每一条记录相当于MongoDB中的一个文档。

59、什么是”mongod“

mongod是处理MongoDB系统的主要进程。它处理数据请求，管理数据存储，和执行后台管理操作。当我们运行mongod命令意味着正在启动MongoDB进程,并且在后台运行。

60、"mongod"参数有什么

- 传递数据库存储路径，默认是"/data/db"
- 端口号 默认是 "27017"

61、什么是"mongo"

它是一个命令行工具用于连接一个特定的mongod实例。当我们没有带参数运行mongo命令它将使用默认的端口号和localhost连接

62、MongoDB哪个命令可以切换数据库

MongoDB 用`use`+**数据库名称**的方式来创建数据库。`use`会创建一个新的数据库，如果该数据库存在，则返回这个数据库。

```
>use database_name
```

63、什么是非关系型数据库

非关系型数据库是对不同于传统关系型数据库的统称。非关系型数据库的显著特点是不使用SQL作为查询语言，数据存储不需要特定的表格模式。由于简单的设计和非常好的性能所以被用于大数据和Web Apps等

64、非关系型数据库有哪些类型

- -Key-Value 存储 Eg:Amazon S3
- 图表 Eg:Neo4J
- 文档存储 Eg:MongoDB
- 基于列存储 Eg:Cassandra

65、为什么用MOngoDB？

- 架构简单
- 没有复杂的连接
- 深度查询能力,MongoDB支持动态查询。
- 容易调试
- 容易扩展
- 不需要转化/映射应用对象到数据库对象
- 使用内部内存作为存储工作区,以便更快的存取数据。

66、在哪些场景使用MongoDB

- 大数据
- 内容管理系统
- 移动端Apps
- 数据管理

67、MongoDB中的命名空间是什么意思?

MongoDB内部有预分配空间的机制，每个预分配的文件都用0进行填充。

数据文件每新分配一次，它的大小都是上一个数据文件大小的2倍，每个数据文件最大2G。

MongoDB每个集合和每个索引都对应一个命名空间，这些命名空间的元数据集中在16M的*.ns文件中，平均每个命名占用约 628 字节，也即整个数据库的命名空间的上限约为24000。

如果每个集合有一个索引（比如默认的_id索引），那么最多可以创建12000个集合。如果索引数更多，则可创建的集合数就更少了。同时，如果集合数太多，一些操作也会变慢。

要建立更多的集合的话，MongoDB 也是支持的，只需要在启动时加上“--nssize”参数，这样对应数据库的命名空间文件就可以变得更大以便保存更多的命名。这个命名空间文件（.ns文件）最大可以为 2G。

每个命名空间对应的盘区不一定是连续的。与数据文件增长相同，每个命名空间对应的盘区大小都是随分配次数不断增长的。目的是为了平衡命名空间浪费的空间与保持一个命名空间数据的连续性。

需要注意的一个命名空间$freelist，这个命名空间用于记录不再使用的盘区（被删除的Collection或索引）。每当命名空间需要分配新盘区时，会先查看$freelist是否有大小合适的盘区可以使用，如果有就回收空闲的磁盘空间。

68、哪些语言支持MongoDB?

- C
- C++
- C#
- Java
- Node.js
- Perl
- Php 等

69、在MongoDB中如何创建一个新的数据库

MongoDB 用 `use` + **数据库名称** 的方式来创建数据库。`use` 会创建一个新的数据库，如果该数据库存在，则返回这个数据库。

```
>use mydb
switched to db mydb
```

70、在MongoDB中如何查看数据库列表

使用命令"show dbs"

```
>show dbs
```

71、MongoDB中的分片是什么意思

分片是将数据水平切分到不同的物理节点。当应用数据越来越大的时候，数据量也会越来越大。当数据量增长时，单台机器有可能无法存储数据或可接受的读取写入吞吐量。利用分片技术可以添加更多的机器来应对数据量增加以及读写操作的要求。

72、如何查看使用MongoDB的连接Sharding - MongoDB Manual21.如何查看使用MongoDB的连接

使用命令"db.adminCommand(“connPoolStats”)"

```
>db.adminCommand(“connPoolStats”)
```

73、什么是复制

复制是将数据同步到多个服务器的过程，通过多个数据副本存储到多个服务器上增加数据可用性。复制可以保障数据的安全性，灾难恢复，无需停机维护（如备份，重建索引，压缩），分布式读取数据。

74、在MongoDB中如何在集合中插入一个文档

要想将数据插入 MongoDB 集合中，需要使用 `insert()` 或 `save()` 方法。

```
>db.collectionName.insert({"key":"value"})
>db.collectionName.save({"key":"value"})
```

75、在MongoDB中如何除去一个数据库Collection Methods24.在MongoDB中如何除去一个数据库

MongoDB 的`dropDatabase()`命令用于删除已有数据库。

```
>db.dropDatabase()
```

76、在MongoDB中如何创建一个集合。

在 MongoDB 中，创建集合采用**db.createCollection(name, options)**方法。`options`是一个用来指定集合配置的文档。

```
>db.createCollection("collectionName")db.createCollection() - MongoDB Manual>db.createCollection("
```

77、在MongoDB中如何查看一个已经创建的集合

可以使用show collections 查看当前数据库中的所有集合清单

```
>show collections
```

78、在MongoDB中如何删除一个集合

MongoDB 利用`db.collection.drop()`来删除数据库中的集合。

```
>db.CollectionName.drop()
```

79、为什么要在MongoDB中使用分析器

数据库分析工具(Database Profiler)会针对正在运行的mongod实例收集数据库命令执行的相关信息。包括增删改查的命令以及配置和管理命令。分析器(profiler)会写入所有收集的数据到 system.profile集合，一个capped集合在管理员数据库。分析器默认是关闭的你能通过per数据库或per实例开启。

80、MongoDB支持主键外键关系吗

默认MongoDB不支持主键和外键关系。 用Mongodb本身的API需要硬编码才能实现外键关联，不够直观且难度较大。可以参考以下链接

81、MongoDB支持哪些数据类型

- String
- Integer
- Double
- Boolean
- Object
- Object ID
- Arrays
- Min/Max Keys
- Datetime
- Code
- Regular Expression等

82、为什么要在MongoDB中用"Code"数据类型

"Code"类型用于在文档中存储 JavaScript 代码。

83、为什么要在MongoDB中用"Regular Expression"数据类型

"Regular Expression"类型用于在文档中存储正则表达式

84、为什么在MongoDB中使用"Object ID"数据类型

"ObjectID"数据类型用于存储文档id

85、如何在集合中插入一个文档

要想将数据插入 MongoDB 集合中，需要使用insert()或save()方法。

```
>db.collectionName.insert({"key":"value"})
>db.collectionName.save({"key":"value"})
```

86、"ObjectID"有哪些部分组成

一共有四部分组成:时间戳、客户端ID、客户进程ID、三个字节的增量计数器

_id是一个 12 字节长的十六进制数，它保证了每一个文档的唯一性。在插入文档时，需要提供`_id`。如果你不提供，那么 MongoDB 就会为每一文档提供一个唯一的 id。`_id`的头 4 个字节代表的是当前的时间戳，接着的后 3 个字节表示的是机器 id 号，接着的 2 个字节表示 MongoDB 服务器进程 id，最后的 3 个字节代表递增值。

87、在MongoDb中什么是索引

索引用于高效的执行查询.没有索引MongoDB将扫描查询整个集合中的所有文档这种扫描效率很低，需要处理大量数据。索引是一种特殊的数据结构，将一小块数据集保存为容易遍历的形式。索引能够存储某种特殊字段或字段集的值，并按照索引指定的方式将字段值进行排序。

88、如何添加索引

使用`db.collection.createIndex()`在集合中创建一个索引

```
>db.collectionName.createIndex({columnName:1})
```

89、用什么方法可以格式化输出结果

使用pretty() 方法可以格式化显示结果

```
>db.collectionName.find().pretty()
```

90、如何使用"AND"或"OR"条件循环查询集合中的文档

在`find()`方法中，如果传入多个键，并用逗号(`,`)分隔它们，那么 MongoDB 会把它看成是**AND**条件。

```
>db.mycol.find({key1:value1, key2:value2}).pretty()
```

若基于**OR**条件来查询文档，可以使用关键字**$or**。

```
>db.mycol.find(
   {
      $or: [
         {key1: value1}, {key2:value2}
      ]
   }
).pretty()
```

91、在MongoDB中如何更新数据

`update()`与`save()`方法都能用于更新集合中的文档。`update()`方法更新已有文档中的值，而`save()`方法则是用传入该方法的文档来替换已有文档。

```
>db.collectionName.update({key:value},{$set:{newkey:newValue}})
```

92、如何删除文档

MongoDB 利用`remove()`方法 清除集合中的文档。它有 2 个可选参数：

- deletion criteria：（可选）删除文档的标准。
- justOne：（可选）如果设为 true 或 1，则只删除一个文档。

```
>db.collectionName.remove({key:value})
```

93、在MongoDB中如何排序

MongoDB 中的文档排序是通过 `sort()` 方法来实现的。 `sort()` 方法可以通过一些参数来指定要进行排序的字段，并使用 1 和 -1 来指定排序方式，其中 1 表示升序，而 -1 表示降序。

```
>db.connectionName.find({key:value}).sort({columnName:1})
```

94、什么是聚合

聚合操作能够处理数据记录并返回计算结果。聚合操作能将多个文档中的值组合起来，对成组数据执行各种操作，返回单一的结果。它相当于 SQL 中的 count(*) 组合 group by。对于 MongoDB 中的聚合操作，应该使用 `aggregate()` 方法。

```
>db.COLLECTION_NAME.aggregate(AGGREGATE_OPERATION)
```

95、在MongoDB中什么是副本集

在MongoDB中副本集由一组MongoDB实例组成，包括一个主节点多个次节点，MongoDB客户端的所有数据都写入主节点(Primary),副节点从主节点同步写入数据，以保持所有复制集内存储相同的数据，提高数据可用性。

String面试题

1、不同版本的 Spring Framework 有哪些主要功能？

Version	Feature
Spring 2.5	发布于 2007 年。这是第一个支持注解的版本。
Spring 3.0	发布于 2009 年。它完全利用了 Java5 中的改进，并为 JEE6 提供了支 持
Spring 4.0	发布于 2013 年。这是第一个完全支持 JAVA8 的版本。
Spring 5.0	Spring Framework 5.0的最大特点之一是**响应式编程（Reactive Programming）**。 响应式编程核心功能和对响应式endpoints的支持可通过Spring Framework 5.0中获得。

2、什么是 Spring Framework？

Spring 是一个开源应用框架，旨在降低应用程序开发的复杂度。它是轻量级、松散耦合的。它具有分层体系结构，允许用户选择组件，同时还为 J2EE 应用程序开发提供了一个有凝聚力的框架。它可以集成其他框架，如 Structs、Hibernate、EJB 等，所以又称为框架的框架。

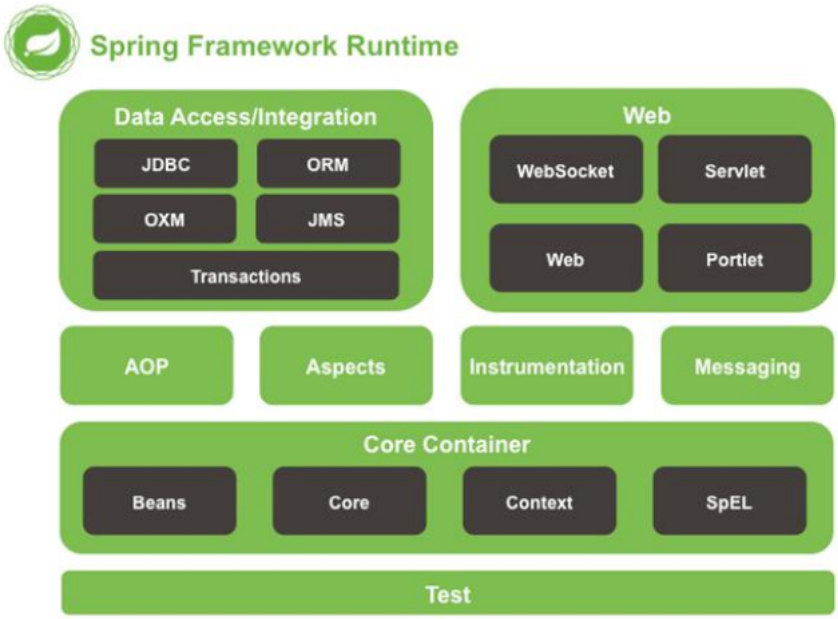
3、列举 Spring Framework 的优点。

由于 Spring Frameworks 的分层架构，用户可以自由选择自己需要的组件。Spring Framework 支持 POJO(Plain Old Java Object) 编程，从而具备持续集成和可测试性。由于依赖注入和控制反转，JDBC 得以简化。它是开源免费的。

4、Spring Framework 有哪些不同的功能？

轻量级 - Spring 在代码量和透明度方面都很轻便。IOC - 控制反转 AOP - 面向切面编程可以将应用业务逻辑和系统服务分离，以实现高内聚。容器 - Spring 负责创建和管理对象（Bean）的生命周期和配置。MVC - 对 web 应用提供了高度可配置性，其他框架的集成也十分方便。事务管理 - 提供了用于事务管理的通用抽象层。Spring 的事务支持也可用于容器较少的环境。JDBC 异常 - Spring的 JDBC 抽象层提供了一个异常层次结构，简化了错误处理策略。

5、Spring Framework 中有多少个模块，它们分别是什么？



Spring 核心容器 – 该层基本上是 Spring Framework 的核心。它包含以下模块：
- Spring Core
- Spring Bean
- SpEL (Spring Expression Language)
- Spring Context

数据访问/集成 – 该层提供与数据库交互的支持。它包含以下模块：
- JDBC (Java DataBase Connectivity)
- ORM (Object Relational Mapping)
- OXM (Object XML Mappers)
- JMS (Java Messaging Service)
- Transaction

Web – 该层提供了创建 Web 应用程序的支持。它包含以下模块：
Web
Web – Servlet
Web – Socket
Web – Portlet

AOP
该层支持面向切面编程

Instrumentation
该层为类检测和类加载器实现提供支持。

Test
该层为使用 JUnit 和 TestNG 进行测试提供支持。

几个杂项模块:

Messaging – 该模块为 STOMP 提供支持。它还支持注解编程模型，该模型用于从 WebSocket 客户端路由和处理 STOMP 消息。Aspects – 该模块为与 AspectJ 的集成提供支持。

6、什么是 Spring 配置文件？

Spring 配置文件是 XML 文件。该文件主要包含类信息。它描述了这些类是如何配置以及相互引入的。但是，XML 配置文件冗长且更加干净。如果没有正确规划和编写，那么在大项目中管理变得非常困难

7、Spring 应用程序有哪些不同组件？

Spring 应用一般有以下组件：
- 接口 - 定义功能。
- Bean 类 - 它包含属性，setter 和 getter 方法，函数等。
- Spring 面向切面编程（AOP） - 提供面向切面编程的功能。

□ Bean 配置文件 - 包含类的信息以及如何配置它们。
□ 用户程序 - 它使用接口。

8、使用 Spring 有哪些方式？

使用 Spring 有以下方式：
□ 作为一个成熟的 Spring Web 应用程序。

作为第三方 Web 框架，使用 Spring Frameworks 中间层。
□ 用于远程使用。
□ 作为企业级 Java Bean，它可以包装现有的 POJO（Plain Old Java Objects）。

9、什么是 Spring IOC 容器？

Spring 框架的核心是 Spring 容器。容器创建对象，将它们装配在一起，配置它们并管理它们的完整生命周期。Spring 容器使用依赖注入来管理组成应用程序的组件。容器通过读取提供的配置元数据来接收对象进行实例化，配置和组装的指令。该元数据可以通过 XML，Java 注解或 Java 代码提供。

10、什么是依赖注入？

在依赖注入中，您不必创建对象，但必须描述如何创建它们。您不是直接在代码中将组件和服务连接在一起，而是描述配置文件中哪些组件需要哪些服务。由 IoC容器将它们装配在一起。

11、可以通过多少种方式完成依赖注入？

通常，依赖注入可以通过三种方式完成，即：
□ 构造函数注入
□ setter 注入
□ 接口注入
在 Spring Framework 中，仅使用构造函数和 setter 注入。

12、区分构造函数注入和 setter 注入

构造函数注入	setter 注入
没有部分注入	有部分注入
不会覆盖 setter 属性	会覆盖 setter 属性
任意修改都会创建一个新实例	任意修改不会创建一个新实例
适用于设置很多属性	适用于设置少量属性

13、spring 中有多少种 IOC 容器？

BeanFactory - BeanFactory 就像一个包含 bean 集合的工厂类。它会在客户端要求时实例化 bean。
ApplicationContext - ApplicationContext 接口扩展了 BeanFactory 接口。它在 BeanFactory 基础上提供了一些额外的功能。

14、区分 BeanFactory 和 ApplicationContext。

BeanFactory	ApplicationContext
它使用懒加载	它使用即时加载
它使用语法显式提供资源对象	它自己创建和管理资源对象
不支持国际化	支持国际化
不支持基于依赖的注解	支持基于依赖的注解

15、列举 IoC 的一些好处。

IoC 的一些好处是：
□ 它将最小化应用程序中的代码量。
□ 它将使您的应用程序易于测试，因为它不需要单元测试用例中的任何单例或 JNDI 查找机制。
□ 它以最小的影响和最少的侵入机制促进松耦合。
□ 它支持即时的实例化和延迟加载服务。

16、Spring IoC 的实现机制。

Spring 中的 IoC 的实现原理就是工厂模式加反射机制。

实例:

```
interface Fruit {
    public abstract void eat();
}

class Apple implements Fruit {
    public void eat(){
    System.out.println("Apple");
    }
}

class Orange implements Fruit {
    public void eat(){
    System.out.println("Orange");
    }
}

class Factory {
    public static Fruit getInstance(String ClassName) {
        Fruit f=null;
        try {
            f=(Fruit)Class.forName(ClassName).newInstance();
        } catch (Exception e) {
            e.printStackTrace();
        }
        return f;
    }
}
class Client {
    public static void main(String[] a) {
        Fruit f=Factory.getInstance("io.github.dunwu.spring.Apple");
        if(f!=null){
            f.eat();
        }
    }
}
```

17、什么是 spring bean？

- 它们是构成用户应用程序主干的对象。
- Bean 由 Spring IoC 容器管理。
- 它们由 Spring IoC 容器实例化，配置，装配和管理。
- Bean 是基于用户提供给容器的配置元数据创建。

18、spring 提供了哪些配置方式？

基于 xml 配置

bean 所需的依赖项和服务在 XML 格式的配置文件中指定。这些配置文件通常包含许多 bean 定义和特定于应用程序的配置选项。它们通常以 bean 标签开头。例如：

```
<bean id="studentbean" class="org.edureka.firstSpring.StudentBean">
    <property name="name" value="Edureka"></property>
</bean>
```

基于注解配置

您可以通过在相关的类，方法或字段声明上使用注解，将 bean 配置为组件类本身，而不是使用 XML 来描述 bean 装配。默认情况下，Spring 容器中未打开注解装配。因此，您需要在使用它之前在 Spring 配置文件中启用它。例如：

```
<beans>
    <context:annotation-config/>
    <!-- bean definitions go here -->
</beans>
```

基于 Java API 配置

Spring 的 Java 配置是通过使用 @Bean 和 @Configuration 来实现。

1、 @Bean 注解扮演与 元素相同的角色。

2、 @Configuration 类允许通过简单地调用同一个类中的其他 @Bean 方法来定义 bean 间依赖关系。

例如：

```
@Configuration
public class StudentConfig {
    @Bean
    public StudentBean myStudent() {
        return new StudentBean();
    }
}
```

19、spring 支持集中 bean scope？

Spring bean 支持 5 种 scope：
Singleton - 每个 Spring IoC 容器仅有一个单实例。Prototype - 每次请求都会产生一个新的实例。Request - 每一次 HTTP 请求都会产生一个新的实例，并且该 bean 仅在当前 HTTP 请求内有效。Session - 每一次 HTTP 请求都会产生一个新的 bean，同时该 bean 仅在当前 HTTP session 内有效。Global-session - 类似于标准的 HTTP Session 作用域，不过它仅仅在基于portlet 的 web 应用中才有意义。Portlet 规范定义了全局 Session 的概念，
它被所有构成某个 portlet web 应用的各种不同的 portlet 所共享。在 globalsession 作用域中定义的 bean 被限定于全局 portlet Session 的生命周期范围内。如果你在 web 中使用 global session 作用域来标识 bean，那么 web会自动当成 session 类型来使用。

仅当用户使用支持 Web 的 ApplicationContext 时，最后三个才可用。

20、spring bean 容器的生命周期是什么样的？

spring bean 容器的生命周期流程如下：
1、Spring 容器根据配置中的 bean 定义中实例化 bean。

2、Spring 使用依赖注入填充所有属性，如 bean 中所定义的配置。

3、如果 bean 实现BeanNameAware 接口，则工厂通过传递 bean 的 ID 来调用setBeanName()。

4、如果 bean 实现 BeanFactoryAware 接口，工厂通过传递自身的实例来调用 setBeanFactory()。

5、如果存在与 bean 关联的任何BeanPostProcessors，则调用 preProcessBeforeInitialization() 方法。

6、如果为 bean 指定了 init 方法（ 的 init-method 属性），那么将调用它。

7、最后，如果存在与 bean 关联的任何 BeanPostProcessors，则将调用 postProcessAfterInitialization() 方法。8、如果 bean 实现 DisposableBean 接口，当 spring 容器关闭时，会调用 destory()。

9、如果为bean 指定了 destroy 方法（ 的 destroy-method 属性），那么将调用它。

21、什么是 spring 的内部 bean？

只有将 bean 用作另一个 bean 的属性时，才能将 bean 声明为内部 bean。为了定义 bean，Spring 的基于 XML 的配置元数据在 或 中提供了 元素的使用。内部 bean 总是匿名
的，它们总是作为原型。
例如，假设我们有一个 Student 类，其中引用了 Person 类。这里我们将只创建一个 Person 类实例并在 Student 中使用它。
Student.java

```
public class Student {
    private Person person;
    //Setters and Getters
}
public class Person {
    private String name;
    private String address;
    //Setters and Getters
}
```

bean.xml

```
<bean id="StudentBean" class="com.edureka.Student">
    <property name="person">
        <!--This is inner bean -->
        <bean class="com.edureka.Person">
        <property name="name" value="Scott"></property>
        <property name="address" value=
        "Bangalore"></property>
        </bean>
    </property>
</bean>
```

22、什么是 spring 装配

当 bean 在 Spring 容器中组合在一起时，它被称为装配或 bean 装配。Spring容器需要知道需要什么 bean 以及容器应该如何使用依赖注入来将 bean 绑定在一起，同时装配 bean。

23、自动装配有哪些方式？

Spring 容器能够自动装配 bean。也就是说，可以通过检查 BeanFactory 的内容让 Spring 自动解析 bean 的协作者。
自动装配的不同模式：
no - 这是默认设置，表示没有自动装配。应使用显式 bean 引用进行装配。byName - 它根据 bean 的名称注入对象依赖项。它匹配并装配其属性与 XML文件中由相同名称定义的 bean。byType - 它根据类型注入对象依赖项。如果属性的类型与 XML 文件中的一个 bean 名称匹配，则匹配并装配属性。构造函数- 它通过调用类的构造函数来注入依赖项。它有大量的参数。autodetect - 首先容器尝试通过构造函数使用 autowire 装配，如果不能，则尝试通过 byType 自动装配

24、自动装配有什么局限？

覆盖的可能性 - 您始终可以使用 和 设置指定依赖项，这将覆盖自动装配。基本元数据类型 - 简单属性（如原数据类型，字符串和类）无法自动装配。令人困惑的性质 - 总是喜欢使用明确的装配，因为自动装配不太精确。

25、什么是基于注解的容器配置

不使用 XML 来描述 bean 装配，开发人员通过在相关的类，方法或字段声明上使用注解将配置移动到组件类本身。它可以作为 XML 设置的替代方案。例如：Spring 的 Java 配置是通过使用 @Bean 和 @Configuration 来实现。 @Bean 注解扮演与 元素相同的角色。
@Configuration 类允许通过简单地调
用同一个类中的其他 @Bean 方法来定义 bean 间依赖关系。
例如

```
@Configuration
public class StudentConfig {
    @Bean
    public StudentBean myStudent() {
        return new StudentBean();
    }
}
```

26、如何在 spring 中启动注解装配？

默认情况下，Spring 容器中未打开注解装配。因此，要使用基于注解装配，我们必须通过配置 <context：annotation-config/> 元素在 Spring 配置文件中启用它。

27、@Component, @Controller, @Repository

@Service 有何区别？
@Component ：这将 java 类标记为 bean。它是任何 Spring 管理组件的通用构造型。spring 的组件扫描机制现在可以将其拾取并将其拉入应用程序环境中。

@Controller ：这将一个类标记为 Spring Web MVC 控制器。标有它的Bean 会自动导入到 IoC 容器中。

@Service ：此注解是组件注解的特化。它不会对 @Component 注解提供任何其他行为。您可以在服务层类中使用@Service 而不是 @Component，因为它以更好的方式指定了意图。

@Repository ：这个注解是具有类似用途和功能的 @Component 注解的特化。它为 DAO 提供了额外的好处。它将 DAO 导入 IoC 容器，并使未经检查的异常有资格转换为 Spring DataAccessException。

28、@Required 注解有什么用？

@Required 应用于 bean 属性 setter 方法。此注解仅指示必须在配置时使用bean 定义中的显式属性值或使用自动装配填充受影响的 bean 属性。如果尚未填充受影响的 bean 属性，则容器将抛出 eanInitializationException。

示例：

```
public class Employee {
    private String name;
    @Required
    public void setName(String name){
        this.name=name;
    }
    public string getName(){
        return name;
    }
}
```

29、@Autowired 注解有什么用？

@Autowired 可以更准确地控制应该在何处以及如何进行自动装配。此注解用于在 setter 方法，构造函数，具有任意名称或多个参数的属性或方法上自动装配bean。默认情况下，它是类型驱动的注入。

```
public class Employee {
    private String name;
    @Autowired
    public void setName(String name) {
        this.name=name;
    }
    public string getName(){
        return name;
    }
}
```

30、@Qualifier 注解有什么用？

当您创建多个相同类型的 bean 并希望仅使用属性装配其中一个 bean 时，您可以使用@Qualifier 注解和 @Autowired 通过指定应该装配哪个确切的 bean
来消除歧义。

例如，这里我们分别有两个类，Employee 和 EmpAccount。在 EmpAccount中，使用@Qualifier 指定了必须装配 id 为 emp1 的 bean。

Employee.java

```
public class Employee {
    private String name;
    @Autowired
    public void setName(String name) {
        this.name=name;
    }
    public string getName() {
        return name;
    }
}
```

EmpAccount.java

```
public class EmpAccount {
    private Employee emp;
    @Autowired
    @Qualifier(emp1)
    public void showName() {
        System.out.println("Employee name : "+emp.getName);
    }
}
```

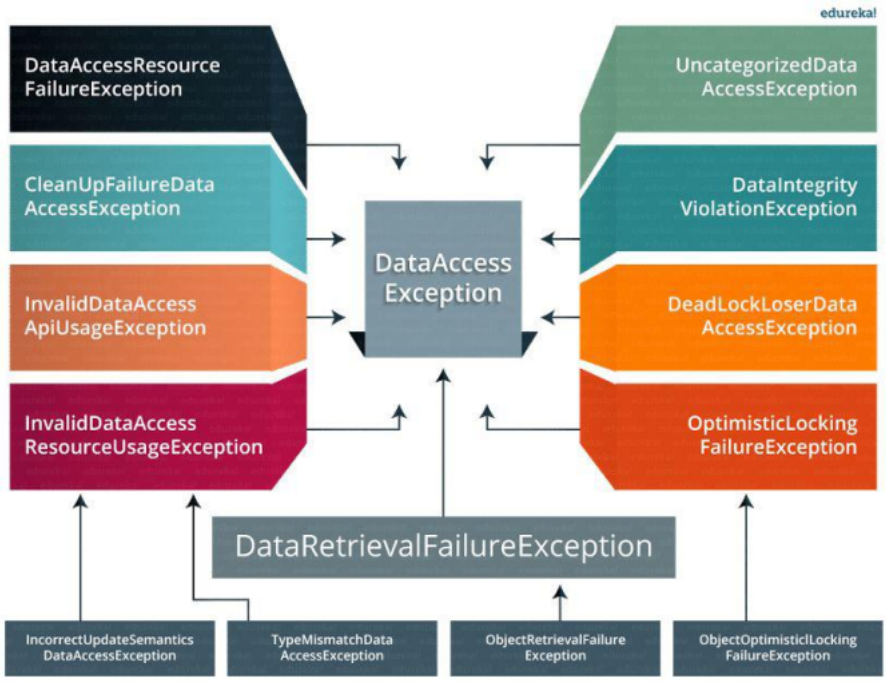
31、@RequestMapping 注解有什么用？

@RequestMapping 注解用于将特定 HTTP 请求方法映射到将处理相应请求的
控制器中的特定类/方法。此注释可应用于两个级别：
类级别：映射请求的 URL 方法级别：映射 URL 以及 HTTP 请求方法

32、spring DAO 有什么用？

Spring DAO 使得 JDBC，Hibernate 或 JDO 这样的数据访问技术更容易以一种统一的方式工作。这使得用户容易在持久性技术之间切换。
它还允许您在编写代码时，无需考虑捕获每种技术不同的异常。

33、列举 Spring DAO 抛出的异常。



34、spring JDBC API 中存在哪些类？

- JdbcTemplate
- SimpleJdbcTemplate
- NamedParameterJdbcTemplate
- SimpleJdbcInsert
- SimpleJdbcCall

35、使用 Spring 访问 Hibernate 的方法有哪些？

我们可以通过两种方式使用 Spring 访问 Hibernate：
1、 使用 Hibernate 模板和回调进行控制反转
2、 扩展 HibernateDAOSupport 并应用 AOP 拦截器节点

36、列举 spring 支持的事务管理类型

Spring 支持两种类型的事务管理：
1、 程序化事务管理：在此过程中，在编程的帮助下管理事务。它为您提供极大的灵活性，但维护起来非常困难。
2、 声明式事务管理：在此，事务管理与业务代码分离。仅使用注解或基于 XML的配置来管理事务。

37、spring 支持哪些 ORM 框架

- Hibernate
- iBatis
- JPA
- JDO
- OJB

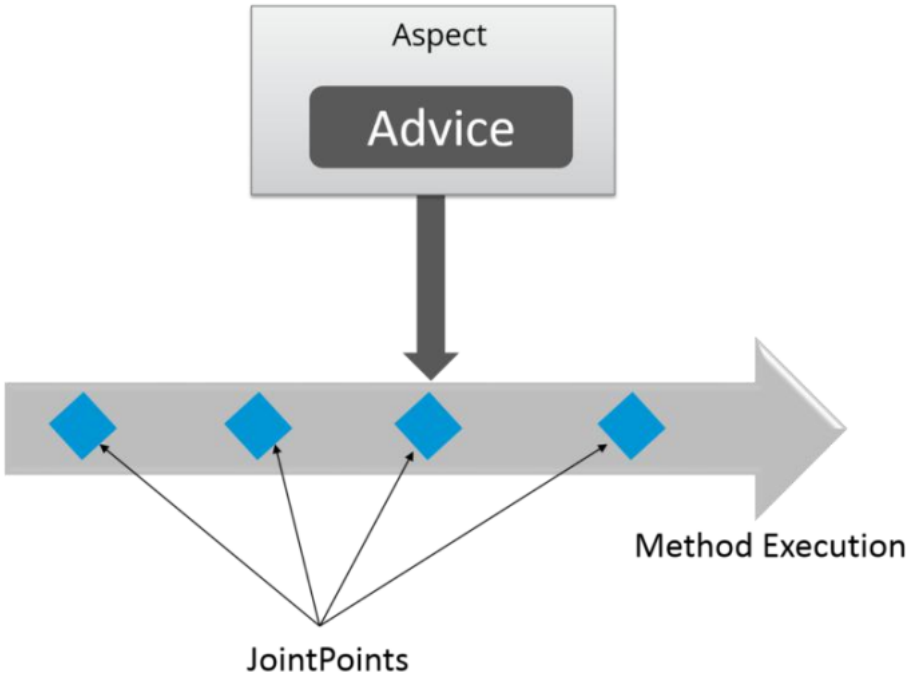
38、什么是 AOP？

AOP(Aspect-Oriented Programming), 即 面向切面编程, 它与OOP(Object-Oriented Programming, 面向对象编程) 相辅相成, 提供了与 OOP 不同的抽象软件结构的视角. 在 OOP 中, 我们以类(class)作为我们的基本单元, 而 AOP 中的基本单元是 Aspect(切面)

39、什么是 Aspect？

aspect 由 pointcount 和 advice 组成, 它既包含了横切逻辑的定义, 也包括了连接点的定义. Spring AOP 就是负责实施切面的框架, 它将切面所定义的横切逻辑编织到切面所指定的连接点中. AOP 的工作重心在于如何将增强编织目标对象的连接点上, 这里包含两个工作:
1、如何通过 pointcut 和 advice 定位到特定的 joinpoint 上

2、如何在advice 中编写切面代码.



以简单地认为, 使用 @Aspect 注解的类就是切面.

40、什么是切点（JoinPoint）

程序运行中的一些时间点, 例如一个方法的执行, 或者是一个异常的处理.在 Spring AOP 中, join point 总是方法的执行点。

41、什么是通知（Advice）？

特定 JoinPoint 处的 Aspect 所采取的动作称为 Advice。Spring AOP 使用一个 Advice 作为拦截器，在 JoinPoint “周围”维护一系列的拦截器。

42、有哪些类型的通知（Advice）？

- Before - 这些类型的 Advice 在 joinpoint 方法之前执行，并使用@Before 注解标记进行配置。
- After Returning - 这些类型的 Advice 在连接点方法正常执行后执行，并使用@AfterReturning 注解标记进行配置。
- After Throwing - 这些类型的 Advice 仅在 joinpoint 方法通过抛出异常退出并使用 @AfterThrowing 注解标记配置时执行。
- After (finally) - 这些类型的 Advice 在连接点方法之后执行，无论方法退出是正常还是异常返回，并使用 @After 注解标记进行配置。
- Around - 这些类型的 Advice 在连接点之前和之后执行，并使用@Around 注解标记进行配置。

43、指出在 spring aop 中 concern 和 cross-cuttingconcern 的不同之处。

concern 是我们想要在应用程序的特定模块中定义的行为。它可以定义为我们想要实现的功能。

cross-cutting concern 是一个适用于整个应用的行为，这会影响整个应用程序。例如，日志记录，安全性和数据传输是应用程序几乎每个模块都需要关注的问题，因此它们是跨领域的问题

44、AOP 有哪些实现方式？

实现 AOP 的技术，主要分为两大类：

静态代理
指使用 AOP 框架提供的命令进行编译，从而在编译阶段就可生成 AOP 代理类，因此也称为编译时增强；
- 编译时编织（特殊编译器实现）
- 类加载时编织（特殊的类加载器实现）。

动态代理
在运行时在内存中“临时”生成 AOP 动态代理类，因此也被称为运行时增强。
- JDK 动态代理
- CGLIB

45、Spring AOP and AspectJ AOP 有什么区别？

Spring AOP 基于动态代理方式实现；AspectJ 基于静态代理方式实现。SpringAOP 仅支持方法级别的 PointCut；提供了完全的 AOP 支持，它还支持属性级别的 PointCut。

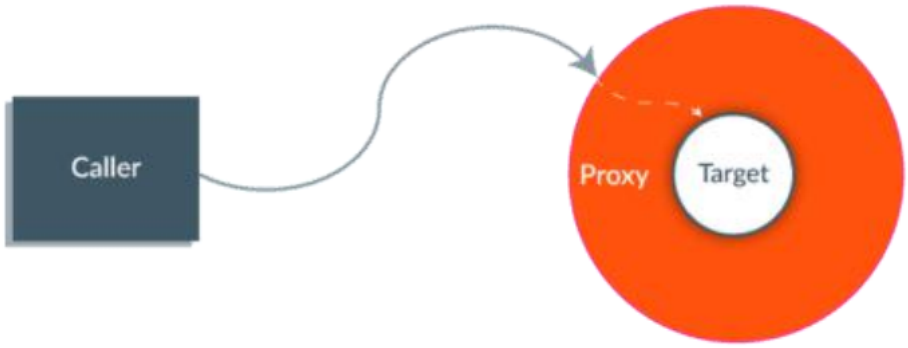
46、如何理解 Spring 中的代理？

将 Advice 应用于目标对象后创建的对象称为代理。在客户端对象的情况下，目标对象和代理对象是相同的。

```
Advice + Target Object = Proxy
```

47、什么是编织（Weaving）？

为了创建一个 advice 对象而链接一个 aspect 和其它应用类型或对象，称为编织（Weaving）。在 Spring AOP 中，编织在运行时执行。请参考下图：

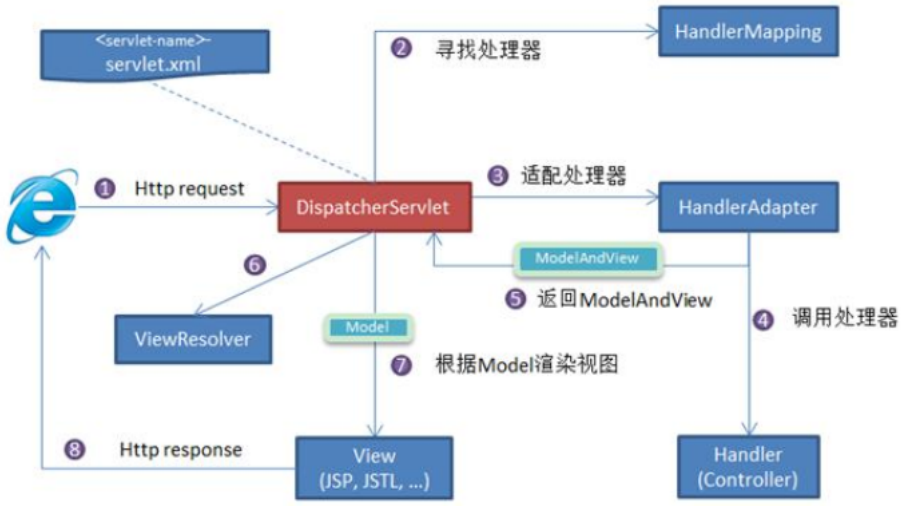


48、Spring MVC 框架有什么用？

Spring Web MVC 框架提供 模型-视图-控制器 架构和随时可用的组件，用于开发灵活且松散耦合的 Web 应用程序。MVC 模式有助于分离应用程序的不同方面，如输入逻辑，业务逻辑和 UI 逻辑，同时在所有这些元素之间提供松散耦合。

49、描述一下 DispatcherServlet 的工作流程

DispatcherServlet 的工作流程可以用一幅图来说明：



1、向服务器发送 HTTP 请求，请求被前端控制器 DispatcherServlet 捕获。

2、 DispatcherServlet 根据 -servlet.xml 中的配置对请求的 URL 进行解析，得到请求资源标识符（URI）。然后根据该 URI，调用 HandlerMapping获得该 Handler 配置的所有相关的对象（包括 Handler 对象以及 Handler 对象对应的拦截器），最后以 HandlerExecutionChain 对象的形式返回。

3、 DispatcherServlet 根据获得的 Handler，选择一个合适的HandlerAdapter。（附注：如果成功获得 HandlerAdapter 后，此时将开始执行拦截器的 preHandler(...)方法）。

4、提取 Request 中的模型数据，填充 Handler 入参，开始执行 Handler（ Controller)。在填充 Handler 的入参过程中，根据你的配置，Spring 将帮你做一些额外的工作：

- HttpMessageConveter：将请求消息（如 Json、xml 等数据）转换成一个对象，将对象转换为指定的响应信息。
- 数据转换：对请求消息进行数据转换。如 String 转换成 Integer、Double 等。
- 数据根式化：对请求消息进行数据格式化。如将字符串转换成格式化数字或格式化日期等。
- 数据验证：验证数据的有效性（长度、格式等），验证结果存储到BindingResult 或 Error 中。

5、Handler(Controller)执行完成后，向 DispatcherServlet 返回一个ModelAndView 对象；
6、根据返回的 ModelAndView，选择一个适合的 ViewResolver（必须是已经注册到 Spring 容器中的 ViewResolver)返回给 DispatcherServlet。
7、 ViewResolver 结合 Model 和 View，来渲染视图。
8、视图负责将渲染结果返回给客户端。

50、介绍一下 WebApplicationContext

WebApplicationContext 是 ApplicationContext 的扩展。它具有 Web 应用程序所需的一些额外功能。它与普通的 ApplicationContext 在解析主题和决定与哪个 servlet 关联的能力方面有所不同。
英文原文链接：
https://www.edureka.co/blog/interview-questions/spring-interview-questions/

51、什么是 spring?

Spring 是个 java 企业级应用的开源开发框架。Spring 主要用来开发 Java 应用，但是有些扩展是针对构建 J2EE 平台的 web 应用。Spring 框架目标是简化 Java企业级应用开发，并通过 POJO 为基础的编程模型促进良好的编程习惯。

52、使用 Spring 框架的好处是什么？

□ 轻量：Spring 是轻量的，基本的版本大约 2MB。
□ 控制反转：Spring 通过控制反转实现了松散耦合，对象们给出它们的依赖，而不是创建或查找依赖的对象们。
□ 面向切面的编程(AOP)：Spring 支持面向切面的编程，并且把应用业务逻辑和系统服务分开。
□ 容器：Spring 包含并管理应用中对象的生命周期和配置。
□ MVC 框架：Spring 的 WEB 框架是个精心设计的框架，是 Web 框架的一个很好的替代品。
□ 事务管理：Spring 提供一个持续的事务管理接口，可以扩展到上至本地事务下至全局事务（JTA）。
□ 异常处理：Spring 提供方便的API把具体技术相关的异常（比如由JDBC，Hibernate or JDO 抛出的）转化为一致的 unchecked 异常。

53、Spring 由哪些模块组成?

以下是 Spring 框架的基本模块：

□ Core module
□ Bean module
□ Context module
□ Expression Language module
□ JDBC module
□ ORM module
□ OXM module
□ Java Messaging Service(JMS) module
□ Transaction module
□ Web module
□ Web-Servlet module
□ Web-Struts module
□ Web-Portlet module

54、Spring的IOC和AOP机制

我们是在使用Spring框架的过程中，其实就是为了使用IOC，依赖注入，和AOP，面向切面编程，这两个是Spring的灵魂。
主要用到的设计模式有工厂模式和代理模式。
IOC就是典型的工厂模式，通过sessionfactory去注入实例。
AOP就是典型的代理模式的体现。
代理模式是常用的java设计模式，他的特征是代理类与委托类有同样的接口，代理类主要负责为委托类预处理消息、过滤消息、把消息转发给委托类，以及事后处理消息等。代理类与委托类之间通常会存在关联关系，一个代理类的对象与一个委托类的对象关联，代理类的对象本身并不真正实现服务，而是通过调用委托类的对象的相关方法，来提供特定的服务

spring的IoC容器是spring的核心，spring AOP是spring框架的重要组成部分。
在传统的程序设计中，当调用者需要被调用者的协助时，通常由调用者来创建被调用者的实例。但在spring里创建被调用者的工作不再由调用者来完成，因此控制反转（IoC）；创建被调用者实例的工作通常由spring容器来完成，然后注入调用者，因此也被称为依赖注入（DI），依赖注入和控制反转是同一个概念。

面向方面编程（AOP)是以另一个角度来考虑程序结构，通过分析程序结构的关注点来完善面向对象编程（OOP）。OOP将应用程序分解成各个层次的对象，而AOP将程序分解成多个切面。spring AOP 只实现了方法级别的连接点，在J2EE应用中，AOP拦截到方法级别的操作就已经足够。在spring中，未来使IoC方便地使用健壮、灵活的企业服务，需要利用spring AOP实现为IoC和企业服务之间建立联系。

IOC:控制反转也叫依赖注入。利用了工厂模式
将对象交给容器管理，你只需要在spring配置文件总配置相应的bean，以及设置相关的属性，让spring容器来生成类的实例对象以及管理对象。在spring容器启动的时候，spring会把你在配置文件中配置的bean都初始化好，然后在你需要调用的时候，就把它已经初始化好的那些bean分配给你需要调用这些bean的类（假设这个类名是A），分配的方法就是调用A的setter方法来注入，而不需要你在A里面new这些bean了。

> 注意：面试的时候，如果有条件，画图，这样更加显得你懂了

AOP:面向切面编程。（Aspect-Oriented Programming）
AOP可以说是对OOP的补充和完善。OOP引入封装、继承和多态性等概念来建立一种对象层次结构，用以模拟公共行为的一个集合。当我们需要为分散的对象引入公共行为的时候，OOP则显得无能为力。也就是说，OOP允许你定义从上到下的关系，但并不适合定义从左到右的关系。例如日志功能。日志代码往往水平地散布在所有对象层次中，而与它所散布到的对象的核心功能毫无关系。在OOP设计中，它导致了大量代码的重复，而不利于各个模块的重用。将程序中的交叉业务逻辑（比如安全，日志，事务等），封装成一个切面，然后注入到目标对象（具体
业务逻辑）中去。

实现AOP的技术，主要分为两大类：一是采用动态代理技术，利用截取消息的方式，对该消息进行装饰，以取代原有对象行为的执行；二是采用静态织入的方式，引入特定的语法创建“方面”，从而使得编译器可以在编译期间织入有关“方面”的代码.

简单点解释，比方说你想在你的biz层所有类中都加上一个打印‘你好’的功能,这时就可以用aop思想来做.你先写个类写个类方法，方法经实现打印‘你好’,然后loc这个类 ref = “biz.*”让每个类都注入即可实现

55、Spring中Autowired和Resource关键字的区别

@Resource和@Autowired都是做bean的注入时使用，其实@Resource并不是Spring的注解，它的包是javax.annotation.Resource，需要导入，但是Spring支持该注解的注入。
1、共同点
两者都可以写在字段和setter方法上。两者如果都写在字段上，那么就不需要再写setter方法。
2、不同点
（1）@Autowired

@Autowired为Spring提供的注解，需要导入包
org.springframework.beans.factory.annotation.Autowired;只按照byType注入

```
public class TestServiceImpl {
    // 下面两种@Autowired只要使用一种即可
    @Autowired
    private UserDao userDao; // 用于字段上
    @Autowired
    public void setUserDao(UserDao userDao) { // 用于属性的方法上
        this.userDao = userDao;
    }
}
```

@Autowired注解是按照类型（byType）装配依赖对象，默认情况下它要求依赖对象必须存在，如果允许null值，可以设置它的required属性为false。如果我们想使用按照名称（byName）来装配，可以结合@Qualifier注解一起使用。如下：

```
public class TestServiceImpl {
    @Autowired
    @Qualifier("userDao")
    private UserDao userDao;
}
```

（2）@Resource

@Resource默认按照ByName自动注入，由J2EE提供，需要导入包javax.annotation.Resource。@Resource有两个重要的属性：name和type，而Spring将@Resource注解的name属性解析为bean的名字，而type属性则解析为bean的类型。所以，如果使用name属性，则使用byName的自动注入策略，而使用type属性时则使用byType自动注入策略。如果既不制定name也不制定type属性，这时将通过反射机制使用byName自动注入策略

```
public class TestServiceImpl {
    // 下面两种@Resource只要使用一种即可
    @Resource(name="userDao")
    private UserDao userDao; // 用于字段上
    @Resource(name="userDao")
    public void setUserDao(UserDao userDao) { // 用于属性的setter方法上
        this.userDao = userDao;
    }
}
```

> 注：最好是将@Resource放在setter方法上，因为这样更符合面向对象的思想，通过set、get去操作属性，而不是直接去操作属性。

@Resource装配顺序：
①如果同时指定了name和type，则从Spring上下文中找到唯一匹配的bean进行装配，找不到则抛出异常。
②如果指定了name，则从上下文中查找名称（id）匹配的bean进行装配，找不到则抛出异常。
③如果指定了type，则从上下文中找到类似匹配的唯一bean进行装配，找不到或是找到多个，都会抛出异常

④如果既没有指定name，又没有指定type，则自动按照byName方式进行装配；如果没有匹配，则回退为一个原始类型进行匹配，如果匹配则自动装配。@Resource的作用相当于@Autowired，只不过@Autowired按照byType自动注入

56、依赖注入的方式有几种，各是什么?

一、构造器注入
将被依赖对象通过构造函数的参数注入给依赖对象，并且在初始化对象的时候注入。
优点：
对象初始化完成后便可获得可使用的对象。
缺点：
当需要注入的对象很多时，构造器参数列表将会很长；不够灵活。若有多种注入方式，每种方式只需注入指定几个依赖，那么就需要提供多个重载的构造函数，麻烦。
二、setter方法注入
IoC Service Provider通过调用成员变量提供的setter函数将被依赖对象注入给依赖类。
优点：
灵活。可以选择性地注入需要的对象。
缺点：
依赖对象初始化完成后由于尚未注入被依赖对象，因此还不能使用。

三、接口注入
依赖类必须要实现指定的接口，然后实现该接口中的一个函数，该函数就是用于依赖注入。该函数的参数就是要注入的对象

优点
接口注入中，接口的名字、函数的名字都不重要，只要保证函数的参数是要注入的对象类型即可。
缺点：
侵入行太强，不建议使用。
PS：**什么是侵入行？**
如果类A要使用别人提供的一个功能，若为了使用这功能，需要在自己的类中增加额外的代码，这就是侵入性

57、讲一下什么是Spring

Spring是一个轻量级的IoC和AOP容器框架。是为Java应用程序提供基础性服务的一套框架，目的是用于简化企业应用程序的开发，它使得开发者只需要关心业务需求。常见的配置方式有三种：基于XML的配置、基于注解的配置、基于Java的配置。
主要由以下几个模块组成：
Spring Core：核心类库，提供IOC服务；
Spring Context：提供框架式的Bean访问方式，以及企业级功能（JNDI、定时任务等）；
Spring AOP：AOP服务；
Spring DAO：对JDBC的抽象，简化了数据访问异常的处理

Spring ORM：对现有的ORM框架的支持；
Spring Web：提供了基本的面向Web的综合特性，例如多方文件上传；
Spring MVC：提供面向Web应用的Model-View-Controller实现。

58、Spring MVC流程

1、 用户发送请求至前端控制器DispatcherServlet。
2、 DispatcherServlet收到请求调用HandlerMapping处理器映射器。
3、 处理器映射器找到具体的处理器(可以根据xml配置、注解进行查找)，生成处理器对象及处理器拦截器(如果有则生成)一并返回给DispatcherServlet。
4、 DispatcherServlet调用HandlerAdapter处理器适配器。
5、 HandlerAdapter经过适配调用具体的处理器(Controller，也叫后端控制器)。
6、 Controller执行完成返回ModelAndView。
7、 HandlerAdapter将controller执行结果ModelAndView返回给DispatcherServlet。
8、 DispatcherServlet将ModelAndView传给ViewReslover视图解析器。
9、 ViewReslover解析后返回具体View。
10、DispatcherServlet根据View进行渲染视图（即将模型数据填充至视图中）。
11、 DispatcherServlet响应用户

组件说明：
以下组件通常使用框架提供实现：
DispatcherServlet：作为前端控制器，整个流程控制的中心，控制其它组件执行，统一调度，降低组件之间的耦合性，提高每个组件的扩展性

HandlerMapping：通过扩展处理器映射器实现不同的映射方式，例如：配置文件方式，实现接口方式，注解方式等。
HandlAdapter：通过扩展处理器适配器，支持更多类型的处理器。
ViewResolver：通过扩展视图解析器，支持更多类型的视图解析，例如：jsp、freemarker、pdf、excel等。

组件：
1、前端控制器DispatcherServlet（不需要工程师开发）,由框架提供
作用：接收请求，响应结果，相当于转发器，中央处理器。有了dispatcherServlet减少了其它组件之间的耦合度。

用户请求到达前端控制器，它就相当于mvc模式中的c，dispatcherServlet是整个流程控制的中心，由它调用其它组件处理用户的请求，dispatcherServlet的存在降低了组件之间的耦合性。

2、处理器映射器HandlerMapping(不需要工程师开发),由框架提供
作用：根据请求的url查找Handler
HandlerMapping负责根据用户请求找到Handler即处理器，springmvc提供了不同的映射器实现不同的映射方式，例如：配置文件方式，实现接口方式，注解方式等。

3、处理器适配器HandlerAdapter
作用：按照特定规则（HandlerAdapter要求的规则）去执行Handler
通过HandlerAdapter对处理器进行执行，这是适配器模式的应用，通过扩展适配器可以对更多类型的处理器进行执行。

4、处理器Handler(需要工程师开发)
注意：编写Handler时按照HandlerAdapter的要求去做，这样适配器才可以去正确执行Handler
Handler 是继DispatcherServlet前端控制器的后端控制器，在DispatcherServlet的控制下Handler对具体的用户请求进行处理。
由于Handler涉及到具体的用户业务请求，所以一般情况需要工程师根据业务需求开发Handler。

5、视图解析器View resolver(不需要工程师开发),由框架提供
作用：进行视图解析，根据逻辑视图名解析成真正的视图（view）
View Resolver负责将处理结果生成View视图，View Resolver首先根据逻辑视图名解析成物理视图名即具体的页面地址，再生成View视图对象，最后对View进行渲染将处理结果通过页面展示给用户。springmvc框架提供了很多的View视图类型，包括：jstlView、freemarkerView、pdfView等。一般情况下需要通过页面标签或页面模版技术将模型数据通过页面展示给用户，需要由工程师根据业务需求开发具体的页面。

6、视图View(需要工程师开发jsp...)
View是一个接口，实现类支持不同的View类型（jsp、freemarker、pdf...）核心架构的具体流程步骤如下：
1、首先用户发送请求——>DispatcherServlet，前端控制器收到请求后自己不进行处理，而是委托给其他的解析器进行处理，作为统一访问点，进行全局的流程控制；

2、DispatcherServlet——>HandlerMapping， HandlerMapping 将会把请求映射为HandlerExecutionChain 对象（包含一个Handler 处理器（页面控制器）对象、多个HandlerInterceptor 拦截器）对象，通过这种策略模式，很容易添加新的映射策略；

3、DispatcherServlet——>HandlerAdapter，HandlerAdapter 将会把处理器包装为适配器，从而支持多种类型的处理器，即适配器设计模式的应用，从而很容易支持很多类型的处理器；

4、HandlerAdapter——>处理器功能处理方法的调用，HandlerAdapter 将会根据适配的结果调用真正的处理器的功能处理方法，完成功能处理；并返回一个ModelAndView 对象（包含模型数据、逻辑视图名）；

5、ModelAndView的逻辑视图名——> ViewResolver， ViewResolver 将把逻辑视图名解析为具体的View，通过这种策略模式，很容易更换其他视图技术；

6、View——>渲染，View会根据传进来的Model模型数据进行渲染，此处的Model实际是一个Map数据结构，因此很容易支持其他视图技术；

7、返回控制权给DispatcherServlet，由DispatcherServlet返回响应给用户，到此一个流程结束。下边两个组件通常情况下需要开发：
Handler：处理器，即后端控制器用controller表示。
View：视图，即展示给用户的界面，视图中通常需要标签语言展示模型数据。

59、springMVC是什么

springMVC是一个MVC的开源框架，springMVC=struts2+spring，springMVC就相当于是Struts2加上sring的整合，但是这里有一个疑惑就是，springMVC和spring是什么样的关系呢？这个在百度百科上有一个很好的解释：意思是说，springMVC是spring的一个后续产品，其实就是spring在原有基础上，又提供了web应用的MVC模块，可以简单的把springMVC理解为是spring的一个模块（类似AOP，IOC这样的模块），网络上经常会说springMVC和spring无缝集成，其实springMVC就是spring的一个子模块，所以根本不需要同spring进行整合

60、SpringMVC怎么样设定重定向和转发的？

（1）转发：在返回值前面加"forward:"，譬如"forward:user.do?name=method4
（2）重定向：在返回值前面加"redirect:"，譬如"redirect:http://www.baidu.com"

61、SpringMVC常用的注解有哪些

@RequestMapping：用于处理请求 url 映射的注解，可用于类或方法上。用于类上，则表示类中的所有响应请求的方法都是以该地址作为父路径。
@RequestBody：注解实现接收http请求的json数据，将json转换为java对象。
@ResponseBody：注解实现将conreoller方法返回对象转化为json对象响应给客户

62、Spring的AOP理解

OOP面向对象，允许开发者定义纵向的关系，但并适用于定义横向的关系，导致了大量代码的重复，而不利于各个模块的重用。
AOP，一般称为面向切面，作为面向对象的一种补充，用于将那些与业务无关，但却对多个对象产生影响的公共行为和逻辑，抽取并封装为一个可重用的模块，这个模块被命名为“切面”（Aspect），减少系统中的重复代码，降低了模块间的耦合度，同时提高了系统的可维护性。可用于权限认证、日志、事务处理。
AOP实现的关键在于 代理模式，AOP代理主要分为静态代理和动态代理。静态代理的代表为AspectJ；动态代理则以Spring AOP为代表。
（1）AspectJ是静态代理的增强，所谓静态代理，就是AOP框架会在编译阶段生成AOP代理类，因此也称为编译时增强，他会在编译阶段将AspectJ(切面)织入到Java字节码中，运行的时候就是增强之后的AOP对象。
（2）Spring AOP使用的动态代理，所谓的动态代理就是说AOP框架不会去修改字节码，而是每次运行时在内存中临时为方法生成一个AOP对象，这个AOP对象包含了目标对象的全部方法，并且在特定的切点做了增强处理，并回调原对象的方法。

Spring AOP中的动态代理主要有两种方式，JDK动态代理和CGLIB动态代理 ：

①JDK动态代理只提供接口的代理，不支持类的代理。核心InvocationHandler接口和Proxy类，InvocationHandler 通过invoke()方法反射来调用目标类中的代码，动态地将横切逻辑和业务编织在一起；接着，Proxy利用 InvocationHandler动态创建一个符合某一接口的的实例, 生成目标类的代理对象。
②如果代理类没有实现 InvocationHandler 接口，那么Spring AOP会选择使用CGLIB来动态代理目标类。CGLIB（Code Generation Library），是一个代码生成的类库，可以在运行时动态的生成指定类的一个子类对象，并覆盖其中特定方法并添加增强代码，从而实现AOP。CGLIB是通过继承的方式做的动态代理，因此如果某个类被标记为final，那么它是无法使用CGLIB做动态代理的

3）静态代理与动态代理区别在于生成AOP代理对象的时机不同，相对来说AspectJ的静态代理方式具
有更好的性能，但是AspectJ需要特定的编译器进行处理，而Spring AOP则无需特定的编译器处理。

63、Spring的IOC理解

1）IOC就是控制反转，是指创建对象的控制权的转移，以前创建对象的主动权和时机是由自己把控的，而现在这种权力转移到Spring容器中，并由容器根据配置文件去创建实例和管理各个实例之间的依赖关系，对象与对象之间松散耦合，也利于功能的复用。DI依赖注入，和控制反转是同一个概念的不同角度的描述，即 应用程序在运行时依赖IoC容器来动态注入对象需要的外部资源。

（2）最直观的表达就是，IOC让对象的创建不用去new了，可以由spring自动生产，使用java的反射机制，根据配置文件在运行时动态的去创建对象以及管理对象，并调用对象的方法的。

（3）Spring的IOC有三种注入方式 ：构造器注入、setter方法注入、根据注解注入。

> IoC让相互协作的组件保持松散的耦合，而AOP编程允许你把遍布于应用各层的功能分离出来形成可重用的功能组件

64、解释一下spring bean的生命周期

首先说一下Servlet的生命周期：实例化，初始init，接收请求service，销毁destroy；Spring上下文中的Bean生命周期也类似，如下：
（1）实例化Bean：
对于BeanFactory容器，当客户向容器请求一个尚未初始化的bean时，或初始化bean的时候需要注入另一个尚未初始化的依赖时，容器就会调用createBean进行实例化。对于ApplicationContext容器，当容器启动结束后，通过获取BeanDefinition对象中的信息，实例化所有的bean。

（2）设置对象属性（依赖注入）：
实例化后的对象被封装在BeanWrapper对象中，紧接着，Spring根据BeanDefinition中的信息 以及 通过BeanWrapper提供的设置属性的接口完成依赖注入。

（3）处理Aware接口：
接着，Spring会检测该对象是否实现了xxxAware接口，并将相关的xxxAware实例注入给Bean：

①如果这个Bean已经实现了BeanNameAware接口，会调用它实现的setBeanName(String beanId)方法，此处传递的就是Spring配置文件中Bean的id值；
②如果这个Bean已经实现了BeanFactoryAware接口，会调用它实现的setBeanFactory()方法，传递的是Spring工厂自身。
③如果这个Bean已经实现了ApplicationContextAware接口，会调用setApplicationContext(ApplicationContext)方法，传入Spring上下文；
（4）BeanPostProcessor：

如果想对Bean进行一些自定义的处理，那么可以让Bean实现了BeanPostProcessor接口，那将会调用postProcessBeforeInitialization(Object obj, String s)方法。
（5）InitializingBean 与 init-method：
如果Bean在Spring配置文件中配置了 init-method 属性，则会自动调用其配置的初始化方法。
（6）如果这个Bean实现了BeanPostProcessor接口，将会调用postProcessAfterInitialization(Object obj, String s)方法；由于这个方法是在Bean初始化结束时调用的，所以可以被应用于内存或缓存技术

> 以上几个步骤完成后，Bean就已经被正确创建了，之后就可以使用这个Bean了

7）DisposableBean：
当Bean不再需要时，会经过清理阶段，如果Bean实现了DisposableBean这个接口，会调用其实现的destroy()方法；
（8）destroy-method：
最后，如果这个Bean的Spring配置中配置了destroy-method属性，会自动调用其配置的销毁方法

65、解释Spring支持的几种bean的作用域。

Spring容器中的bean可以分为5个范围：
（1）singleton：默认，每个容器中只有一个bean的实例，单例的模式由BeanFactory自身来维护。
（2）prototype：为每一个bean请求提供一个实例。
（3）request：为每一个网络请求创建一个实例，在请求完成以后，bean会失效并被垃圾回收器回收。
（4）session：与request范围类似，确保每个session中有一个bean的实例，在session过期后，bean会随之失效。
（5）global-session：全局作用域，global-session和Portlet应用相关。当你的应用部署在Portlet容器中工作时，它包含很多portlet。如果你想要声明让所有的portlet共用全局的存储变量的话，那么这全局变量需要存储在global-session中。全局作用域与Servlet中的session作用域效果相同

66、Spring基于xml注入bean的几种方式

1）Set方法注入；
（2）构造器注入：①通过index设置参数的位置；②通过type设置参数类型；
（3）静态工厂注入；
（4）实例工厂；
详细内容可以阅读：https://blog.csdn.net/a745233700/article/details/89307518

67、Spring框架中都用到了哪些设计模式

（1）工厂模式：BeanFactory就是简单工厂模式的体现，用来创建对象的实例；
（2）单例模式：Bean默认为单例模式。
（3）代理模式：Spring的AOP功能用到了JDK的动态代理和CGLIB字节码生成技术；
（4）模板方法：用来解决代码重复的问题。比如. RestTemplate, JmsTemplate, JpaTemplate。
（5）观察者模式：定义对象键一种一对多的依赖关系，当一个对象的状态发生改变时，所有依赖于它的对象都会得到通知被制动更新，如Spring中listener的实现--ApplicationListener

68、核心容器（应用上下文) 模块

这是基本的 Spring 模块，提供 spring 框架的基础功能，BeanFactory 是 任何以 spring 为基础的应用的核心。Spring 框架建立在此模块之上，它使 Spring 成为一个容器。

69、BeanFactory – BeanFactory 实现举例。

Bean 工厂是工厂模式的一个实现，提供了控制反转功能，用来把应用的配置和依赖从正真的应用代码中分离。
最常用的 BeanFactory 实现是 XmlBeanFactory 类。

70、XMLBeanFactory

最常用的就是 org.springframework.beans.factory.xml.XmlBeanFactory ，它根据 XML 文件中的定义加载 beans。该容器从 XML 文件读取配置元数据并用它去创建一个完全配置的系统或应用。

71、解释 AOP 模块

AOP 模块用于发给我们的 Spring 应用做面向切面的开发， 很多支持由 AOP 联盟提供，这样就确保了 Spring 和其他 AOP 框架的共通性。这个模块将元数据编程引入 Spring。

72、解释 JDBC 抽象和 DAO 模块。

通过使用 JDBC 抽象和 DAO 模块，保证数据库代码的简洁，并能避免数据库资源错误关闭导致的问题，它在各种不同的数据库的错误信息之上，提供了一个统一的异常访问层。它还利用 Spring 的 AOP 模块给 Spring 应用中的对象提供事务管理服务。

72、解释对象/关系映射集成模块。

Spring 通过提供 ORM 模块，支持我们在直接 JDBC 之上使用一个对象/关系映射映射(ORM)工具，Spring 支持集成主流的 ORM 框架，如 Hiberate,JDO 和 iBATISSQL Maps。Spring 的事务管理同样支持以上所有 ORM 框架及 JDBC。

73、解释 WEB 模块。

Spring 的 WEB 模块是构建在 application context 模块基础之上，提供一个适合 web 应用的上下文。这个模块也包括支持多种面向 web 的任务，如透明地处理多个文件上传请求和程序级请求参数的绑定到你的业务对象。它也有对 Jakarta Struts 的支持。

74、Spring 配置文件

Spring 配置文件是个 XML 文件，这个文件包含了类信息，描述了如何配置它们，以及如何相互调用

75、什么是 Spring IOC 容器？

Spring IOC 负责创建对象，管理对象（通过依赖注入（DI），装配对象，配置对象，并且管理这些对象的整个生命周期。

76、IOC 的优点是什么？

IOC 或 依赖注入把应用的代码量降到最低。它使应用容易测试，单元测试不再需要单例和 JNDI 查找机制。最小的代价和最小的侵入性使松散耦合得以实现。IOC容器支持加载服务时的饿汉式初始化和懒加载。

77、ApplicationContext 通常的实现是什么?

 FileSystemXmlApplicationContext ：此容器从一个 XML 文件中加载 beans 的定义，XML Bean 配置文件的全路径名必须提供给它的构造函数。

 ClassPathXmlApplicationContext：此容器也从一个 XML 文件中加载 beans 的定义，这里，你需要正确设置 classpath 因为这个容器将在 classpath里找 bean 配置。

 WebXmlApplicationContext：此容器加载一个 XML 文件，此文件定义了一个 WEB 应用的所有 bean。

78、Bean 工厂和 Application contexts 有什么区别？

Application contexts 提供一种方法处理文本消息，一个通常的做法是加载文件资源（比如镜像），它们可以向注册为监听器的 bean 发布事件。另外，在容器或容器内的对象上执行的那些不得不由 bean 工厂以程序化方式处理的操作，可以在Application contexts 中以声明的方式处理。Application contexts 实现了MessageSource 接口，该接口的实现以可插拔的方式提供获取本地化消息的方法。

79、一个 Spring 的应用看起来象什么？

 一个定义了一些功能的接口。
 这实现包括属性，它的 Setter ， getter 方法和函数等。
 Spring AOP。
 Spring 的 XML 配置文件。
 使用以上功能的客户端程序。

80、什么是 Spring 的依赖注入？

依赖注入，是 IOC 的一个方面，是个通常的概念，它有多种解释。这概念是说你不用创建对象，而只需要描述它如何被创建。你不在代码里直接组装你的组件和服务，但是要在配置文件里描述哪些组件需要哪些服务，之后一个容器（IOC 容器）负责把他们组装起来。

81、有哪些不同类型的 IOC（依赖注入）方式？

 构造器依赖注入：构造器依赖注入通过容器触发一个类的构造器来实现的，该类有一系列参数，每个参数代表一个对其他类的依赖。
 Setter 方法注入：Setter 方法注入是容器通过调用无参构造器或无参static 工厂 方法实例化 bean 之后，调用该 bean 的 setter 方法，即实现了基于 setter 的依赖注入。

82、哪种依赖注入方式你建议使用，构造器注入，还是 Setter方法注入？

你两种依赖方式都可以使用，构造器注入和 Setter 方法注入。最好的解决方案是用构造器参数实现强制依赖，setter 方法实现可选依赖

83、什么是 Spring beans?

Spring beans 是那些形成 Spring 应用的主干的 java 对象。它们被 Spring IOC容器初始化，装配，和管理。这些 beans 通过容器中配置的元数据创建。比如，以 XML 文件中 的形式定义。

Spring 框架定义的 beans 都是单件 beans。在 bean tag 中有个属性"singleton"，如果它被赋为 TRUE，bean 就是单件，否则就是一个 prototype bean。默认是 TRUE，所以所有在 Spring 框架中的 beans 缺省都是单件。

84、一个 Spring Bean 定义 包含什么？

一个 Spring Bean 的定义包含容器必知的所有配置元数据，包括如何创建一个bean，它的生命周期详情及它的依赖。

85、如何给 Spring 容器提供配置元数据?

这里有三种重要的方法给 Spring 容器提供配置元数据。
XML 配置文件。
基于注解的配置。
基于 java 的配置。

86、你怎样定义类的作用域?

当定义一个 在 Spring 里，我们还能给这个 bean 声明一个作用域。它可以通过bean 定义中的 scope 属性来定义。如，当 Spring 要在需要的时候每次生产一个新的 bean 实例，bean 的 scope 属性被指定为 prototype。另一方面，一个 bean 每次使用的时候必须返回同一个实例，这个 bean 的 scope 属性 必须设为singleton

87、解释 Spring 支持的几种 bean 的作用域。

Spring 框架支持以下五种 bean 的作用域：
 singleton : bean 在每个 Spring ioc 容器中只有一个实例。
 prototype：一个 bean 的定义可以有多个实例。
 request：每次 http 请求都会创建一个 bean，该作用域仅在基于 web的 Spring ApplicationContext 情形下有效。
 session：在一个 HTTP Session 中，一个 bean 定义对应一个实例。该作用域仅在基于 web 的 Spring ApplicationContext 情形下有效。
 global-session：在一个全局的 HTTP Session 中，一个 bean 定义对应一个实例。该作用域仅在基于 web 的 Spring ApplicationContext 情形下有效。缺省的 Spring bean 的作用域是 Singleton.

88、Spring 框架中的单例 bean 是线程安全的吗?

不，Spring 框架中的单例 bean 不是线程安全的。

89、解释 Spring 框架中 bean 的生命周期。

 Spring 容器 从 XML 文件中读取 bean 的定义，并实例化 bean。
 Spring 根据 bean 的定义填充所有的属性。

 如果 bean 实现了 BeanNameAware 接口，Spring 传递 bean 的 ID 到setBeanName 方法。
 如果 Bean 实现了 BeanFactoryAware 接口， Spring 传递beanfactory 给 setBeanFactory 方法。
 如果有任何与 bean 相关联的 BeanPostProcessors，Spring 会在postProcesserBeforeInitialization()方法内调用它们。
 如果 bean 实现 IntializingBean 了，调用它的 afterPropertySet 方法，如果 bean 声明了初始化方法，调用此初始化方法。
 如果有 BeanPostProcessors 和 bean 关联，这些 bean 的postProcessAfterInitialization() 方法将被调用。
 如果 bean 实现了 DisposableBean，它将调用 destroy()方法

90、哪些是重要的 bean 生命周期方法？你能重载它们吗？

有两个重要的 bean 生命周期方法，第一个是 setup ， 它是在容器加载 bean的时候被调用。第二个方法是 teardown 它是在容器卸载类的时候被调用。The bean 标签有两个重要的属性（init-method 和 destroy-method）。用它们你可以自己定制初始化和注销方法。它们也有相应的注解（@PostConstruct 和
@PreDestroy）。

91、什么是 Spring 的内部 bean ?

当一个 bean 仅被用作另一个 bean 的属性时，它能被声明为一个内部 bean，为了定义 inner bean，在 Spring 的 基于 XML 的 配置元数据中，可以在 或 元素内使用 元素，内部 bean 通常是匿名的，它们的 Scope 一般是 prototype。

92、在 Spring 中如何注入一个 java 集合 ?

Spring 提供以下几种集合的配置元素：
□ 类型用于注入一列值，允许有相同的值。
□ 类型用于注入一组值，不允许有相同的值。
□ 类型用于注入一组键值对，键和值都可以为任意类型。
□ 类型用于注入一组键值对，键和值都只能为 String 类型。

93、什么是 bean 装配?

装配，或 bean 装配是指在 Spring 容器中把 bean 组装到一起，前提是容器需要知道 bean 的依赖关系，如何通过依赖注入来把它们装配到一起。

94、什么是 bean 的自动装配 ?

Spring 容器能够自动装配相互合作的 bean，这意味着容器不需要和配置，能通过 Bean 工厂自动处理 bean 之间的协作。

95、解释不同方式的自动装配 。

有五种自动装配的方式，可以用来指导 Spring 容器用自动装配方式来进行依赖注入。

□ no：默认的方式是不进行自动装配，通过显式设置 ref 属性来进行装配。

□ byName：通过参数名 自动装配，Spring 容器在配置文件中发现 bean的 autowire 属性被设置成 byname，之后容器试图匹配、装配和该 bean 的属性具有相同名字的 bean。
□ byType:：通过参数类型自动装配，Spring 容器在配置文件中发现 bean的 autowire 属性被设置成 byType，之后容器试图匹配、装配和该 bean 的属性具有相同类型的 bean。如果有多个 bean 符合条件，则抛出错误。
□ constructor：这个方式类似于 byType， 但是要提供给构造器参数，如果没有确定的带参数的构造器参数类型，将会抛出异常。
□ autodetect：首先尝试使用 constructor 来自动装配，如果无法工作，则使用 byType 方式。

96、自动装配有哪些局限性

自动装配的局限性是：
□ 重写：你仍需用 和 配置来定义依赖，意味着总要重写自动装配。
□ 基本数据类型：你不能自动装配简单的属性，如基本数据类型，String字符串，和类。
□ 模糊特性：自动装配不如显式装配精确，如果有可能，建议使用显式装配。

97、你可以在 Spring 中注入一个 null 和一个空字符串吗 ?

可以

98、什么是基于 Java 的 Spring 注解配置? 给一些注解的例子.

基于 Java 的配置，允许你在少量的 Java 注解的帮助下，进行你的大部分 Spring配置而非通过 XML 文件。
以@Configuration 注解为例，它用来标记类可以当做一个 bean 的定义，被Spring IOC 容器使用。另一个例子是@Bean 注解，它表示此方法将要返回一个对象，作为一个 bean 注册进 Spring 应用上下文。

99、什么是基于注解的容器配置?

相对于 XML 文件，注解型的配置依赖于通过字节码元数据装配组件，而非尖括号的声明。
开发者通过在相应的类，方法或属性上使用注解的方式，直接组件类中进行配置，而不是使用 xml 表述 bean 的装配关系。

100、怎样开启注解装配 ?

注解装配在默认情况下是不开启的，为了使用注解装配，我们必须在 Spring 配置文件中配置 context:annotation-config/元素

101、@Required 注解

这个注解表明 bean 的属性必须在配置的时候设置，通过一个 bean 定义的显式的属性值或通过自动装配，若@Required 注解的 bean 属性未被设置，容器将抛出BeanInitializationException。

102、@Autowired 注解

@Autowired 注解提供了更细粒度的控制，包括在何处以及如何完成自动装配。它的用法和@Required 一样，修饰 setter 方法、构造器、属性或者具有任意名称和/或多个参数的 PN 方法。

103、@Qualifier 注解

当有多个相同类型的 bean 却只有一个需要自动装配时，将@Qualifier 注解和@Autowire 注解结合使用以消除这种混淆，指定需要装配的确切的 bean。

104、在 Spring 框架中如何更有效地使用 JDBC?

使用 SpringJDBC 框架，资源管理和错误处理的代价都会被减轻。所以开发者只需写 statements 和 queries 从数据存取数据，JDBC 也可以在 Spring 框架提供的模板类的帮助下更有效地被使用，这个模板叫 JdbcTemplate（例子见这里here）

105、JdbcTemplate

JdbcTemplate 类提供了很多便利的方法解决诸如把数据库数据转变成基本数据类型或对象，执行写好的或可调用的数据库操作语句，提供自定义的数据错误处理。

106、Spring 对 DAO 的支持

Spring 对数据访问对象（DAO）的支持旨在简化它和数据访问技术如 JDBC，Hibernate or JDO 结合使用。这使我们可以方便切换持久层。编码时也不用担心会捕获每种技术特有的异常。

107、使用 Spring 通过什么方式访问 Hibernate?

在 Spring 中有两种方式访问 Hibernate：
□ 控制反转 Hibernate Template 和 Callback。
□ 继承 HibernateDAOSupport 提供一个 AOP 拦截器。

108、Spring 支持的 ORM

Spring 支持以下 ORM：
□ Hibernate
□ iBatis
□ JPA (Java Persistence API)
□ TopLink
□ JDO (Java Data Objects)
□ OJB

109、如何通过 HibernateDaoSupport 将 Spring 和 Hibernate结合起来？

用 Spring 的 SessionFactory 调用 LocalSessionFactory。集成过程分三步：
□ 配置 the Hibernate SessionFactory。
□ 继承 HibernateDaoSupport 实现一个 DAO。
□ 在 AOP 支持的事务中装配。

110、Spring 支持的事务管理类型

Spring 支持两种类型的事务管理：
□ 编程式事务管理：这意味你通过编程的方式管理事务，给你带来极大的灵活性，但是难维护。
□ 声明式事务管理：这意味着你可以将业务代码和事务管理分离，你只需用注解和 XML 配置来管理事务。

111、Spring 框架的事务管理有哪些优点？

□ 它为不同的事务 API 如 JTA，JDBC，Hibernate，JPA 和 JDO，提供一个不变的编程模式。
□ 它为编程式事务管理提供了一套简单的 API 而不是一些复杂的事务 API
如

□ 它支持声明式事务管理。
□ 它和 Spring 各种数据访问抽象层很好得集成。

112、你更倾向用那种事务管理类型？

大多数 Spring 框架的用户选择声明式事务管理，因为它对应用代码的影响最小，因此更符合一个无侵入的轻量级容器的思想。声明式事务管理要优于编程式事务管理，虽然比编程式事务管理（这种方式允许你通过代码控制事务）少了一点灵活性

113、解释 AOP

面向切面的编程，或 AOP， 是一种编程技术，允许程序模块化横向切割关注点，或横切典型的责任划分，如日志和事务管理。

114、Aspect 切面

AOP 核心就是切面，它将多个类的通用行为封装成可重用的模块，该模块含有一组 API 提供横切功能。比如，一个日志模块可以被称作日志的 AOP 切面。根据需求的不同，一个应用程序可以有若干切面。在 Spring AOP 中，切面通过带有@Aspect 注解的类实现。

115、在 Spring AOP 中，关注点和横切关注的区别是什么？

关注点是应用中一个模块的行为，一个关注点可能会被定义成一个我们想实现的一个功能。横切关注点是一个关注点，此关注点是整个应用都会使用的功能，并影响整个应用，比如日志，安全和数据传输，几乎应用的每个模块都需要的功能。因此这些都属于横切关注点。

116、连接点

连接点代表一个应用程序的某个位置，在这个位置我们可以插入一个 AOP 切面，它实际上是个应用程序执行 Spring AOP 的位置。

117、通知

通知是个在方法执行前或执行后要做的动作，实际上是程序执行时要通过SpringAOP 框架触发的代码段。
Spring 切面可以应用五种类型的通知：
 before：前置通知，在一个方法执行前被调用。
 after: 在方法执行之后调用的通知，无论方法执行是否成功。
 after-returning: 仅当方法成功完成后执行的通知。
 after-throwing: 在方法抛出异常退出时执行的通知。
 around: 在方法执行之前和之后调用的通知。

118、切点

切入点是一个或一组连接点，通知将在这些位置执行。可以通过表达式或匹配的方式指明切入点。

119、什么是引入?

引入允许我们在已存在的类中增加新的方法和属性。

120、什么是目标对象?

被一个或者多个切面所通知的对象。它通常是一个代理对象。也指被通知（advised）对象。

121、什么是代理?

代理是通知目标对象后创建的对象。从客户端的角度看，代理对象和目标对象是一样的。

122、有几种不同类型的自动代理？

BeanNameAutoProxyCreator
DefaultAdvisorAutoProxyCreator
Metadata autoproxying

123、什么是织入。什么是织入应用的不同点？

织入是将切面和到其他应用类型或对象连接或创建一个被通知对象的过程。织入可以在编译时，加载时，或运行时完成。

124、解释基于 XML Schema 方式的切面实现。

在这种情况下，切面由常规类以及基于 XML 的配置实现。

125、解释基于注解的切面实现

在这种情况下(基于@AspectJ 的实现)，涉及到的切面声明的风格与带有 java5 标注的普通 java 类一致。
Spring 的 MVC

126、什么是 Spring 的 MVC 框架？

Spring 配备构建 Web 应用的全功能 MVC 框架。Spring 可以很便捷地和其他MVC 框架集成，如 Struts，Spring 的 MVC 框架用控制反转把业务对象和控制逻辑清晰地隔离。它也允许以声明的方式把请求参数和业务对象绑定。

127、DispatcherServlet

Spring 的 MVC 框架是围绕 DispatcherServlet 来设计的，它用来处理所有的 HTTP请求和响应。

128、WebApplicationContext

WebApplicationContext 继承了 ApplicationContext 并增加了一些 WEB 应用必备的特有功能，它不同于一般的 ApplicationContext ，因为它能处理主题，并找到被关联的 servlet。

129、什么是 Spring MVC 框架的控制器？

控制器提供一个访问应用程序的行为，此行为通常通过服务接口实现。控制器解析用户输入并将其转换为一个由视图呈现给用户的模型。Spring 用一个非常抽象的方式实现了一个控制层，允许用户创建多种用途的控制器。

130、@Controller 注解

该注解表明该类扮演控制器的角色，Spring 不需要你继承任何其他控制器基类或引用 Servlet API。

131、@RequestMapping 注解

该注解是用来映射一个 URL 到一个类或一个特定的方处理法上

Spring Boot面试题

1、什么是 Spring Boot？

多年 来， 随着 新功 能的 增加 ，spring 变得 越来 越复 杂。 只需 访问https://spring.io/projects 页面 ，我们 就会 看到 可以 在我 们的 应用 程序 中使 用的所有 Spring 项目 的不 同功 能。 如果 必须 启动 一个 新的 Spring 项目 ，我 们必 须添加构 建路 径或 添加 Maven 依赖关系 ，配 置应 用程 序服 务器 ，添 加 spring 配置 。因此 ，开始 一个 新的 spring 项目 需要 很多 努力 ，因为 我们 现在 必须 从头 开始做所有事 情。

Spring Boot 是解 决这 个问 题的 方法 。Spring Boot 已经 建立 在现 有 spring 框架之上 。使用 spring 启动 ，我们 避免 了之 前我 们必 须做 的所 有样 板代 码和 配置 。因此， Spring Boot 可以 帮助 我们 以最 少的 工作 量， 更加 健壮 地使 用现 有的 Spring功能

2、为什么要用SpringBoot

Spring Boot 优点非常多，如：

一、独立运行

Spring Boot而且内嵌了各种servlet容器，Tomcat、Jetty等，现在不再需要打成war包部署到容器中，Spring Boot只要打成一个可执行的jar包就能独立运行，所有的依赖包都在一个jar包内。

二、简化配置

spring-boot-starter-web启动器自动依赖其他组件，简少了maven的配置。

三、自动配置
Spring Boot能根据当前类路径下的类、jar包来自动配置bean，如添加一个spring-boot-starter-web启动器就能拥有web的功能，无需其他配置。
四、无代码生成和XML配置
Spring Boot配置过程中无代码生成，也无需XML配置文件就能完成所有配置工作，这一切都是借助于条件注解完成的，这也是Spring4.x的核心功能之一。
五、应用监控
Spring Boot提供一系列端点可以监控服务及应用，做健康检测

3、Spring Boot 有哪些优点？

Spring Boot 的优点有：
1、减少开发，测试时间和努力。
2、使用 JavaConfig 有助于避免使用 XML。
3、避免大量的 Maven 导入和各种版本冲突。
4、提供意见发展方法。
5、通过提供默认值快速开始开发。
6、没有单独的 Web 服务器需要。这意味着你不再需要启动 Tomcat，Glassfish或其他任何东西。
7、需要更少的配置 因为没有 web.xml 文件。只需添加用@ Configuration 注释的类，然后添加用@Bean 注释的方法，Spring 将自动加载对象并像以前一样对其进行管理。您甚至可以将@Autowired 添加到 bean 方法中，以使 Spring 自动装入需要的依赖关系中。
8、基于环境的配置 使用这些属性，您可以将您正在使用的环境传递到应用程序：-Dspring.profiles.active = {enviornment}。在加载主应用程序属性文件后，Spring 将在（application{environment} .properties）中加载后续的应用程序属性文件。

4、Spring Boot 的核心注解是哪个？它主要由哪几个注解组成的？

启动类上面的注解是@SpringBootApplication，它也是 Spring Boot 的核心注解，主要组合包含了以下
3 个注解：
@SpringBootConfiguration：组合了 @Configuration 注解，实现配置文件的功能。
@EnableAutoConfiguration：打开自动配置的功能，也可以关闭某个自动配置的选项，如关闭数据源自动配置功能：
@SpringBootApplication(exclude = { DataSourceAutoConfiguration.class })。
@ComponentScan：Spring组件扫描

5、运行Spring Boot有哪几种方式

1）打包用命令或者放到容器中运行
2）用 Maven/Gradle 插件运行
3）直接执行 main 方法运行

6、如何理解 Spring Boot 中的 Starters？

Starters是什么：
Starters可以理解为启动器，它包含了一系列可以集成到应用里面的依赖包，你可以一站式集成Spring及其他技术，而不需要到处找示例代码和依赖包。如你想使用Spring JPA访问数据库，只要加入springboot-starter-data-jpa启动器依赖就能使用了。Starters包含了许多项目中需要用到的依赖，它们能快速持续的运行，都是一系列得到支持的管理传递性依赖。
Starters命名：
Spring Boot官方的启动器都是以spring-boot-starter-命名的，代表了一个特定的应用类型。第三方的启动器不能以spring-boot开头命名，它们都被Spring Boot官方保留。一般一个第三方的应该这样命名，像mybatis的mybatis-spring-boot-starter。
Starters分类：

1. Spring Boot应用类启动器

启动器名称	功能描述
spring-boot-starter	包含自动配置、日志、YAML的支持。
spring-boot-starter-web	使用Spring MVC构建web 工程，包含restful，默认使用Tomcat容器。
...	...

https://blog.csdn.net/Kevin_Gu6

2.Spring Boot生产启动器

启动器名称	功能描述
spring-boot-starter-actuator	提供生产环境特性，能监控管理应用。

3. Spring Boot技术类启动器

启动器名称	功能描述
spring-boot-starter-json	提供对JSON的读写支持。
spring-boot-starter-logging	默认的日志启动器，默认使用Logback。
...	...

4. 其他第三方启动器

7、如何在Spring Boot启动的时候运行一些特定的代码？

如果你想在Spring Boot启动的时候运行一些特定的代码，你可以实现接口ApplicationRunner或者CommandLineRunner，这两个接口实现方式一样，它们都只提供了一个run方法。CommandLineRunner：启动获取命令行参数

8、Spring Boot 需要独立的容器运行吗？

可以不需要，内置了 Tomcat/ Jetty 等容器

9、Spring Boot中的监视器是什么？

Spring boot actuator是spring启动框架中的重要功能之一。Spring boot监视器可帮助您访问生产环境中正在运行的应用程序的当前状态。有几个指标必须在生产环境中进行检查和监控。即使一些外部应用程序可能正在使用这些服务来向相关人员触发警报消息。监视器模块公开了一组可直接作为HTTP URL访问的REST端点来检查状态

10、如何使用Spring Boot实现异常处理？

Spring提供了一种使用ControllerAdvice处理异常的非常有用的方法。 我们通过实现一个ControlerAdvice类，来处理控制器类抛出的所有异常

11、你如何理解 Spring Boot 中的 Starters

Starters可以理解为启动器，它包含了一系列可以集成到应用里面的依赖包，你可以一站式集成 Spring及其他技术，而不需要到处找示例代码和依赖包。如你想使用 Spring JPA 访问数据库，只要加入spring-boot-starter-data-jpa 启动器依赖就能使用了

12、springboot常用的starter有哪些

spring-boot-starter-web 嵌入tomcat和web开发需要servlet与jsp支持
spring-boot-starter-data-jpa 数据库支持
spring-boot-starter-data-redis redis数据库支持
spring-boot-starter-data-solr solr支持
mybatis-spring-boot-starter 第三方的mybatis集成starter

13、SpringBoot 实现热部署有哪几种方式

主要有两种方式：
Spring Loaded
Spring-boot-devtools

14、如何理解 Spring Boot 配置加载顺序

在 Spring Boot 里面，可以使用以下几种方式来加载配置。
1）properties文件；
2）YAML文件；
3）系统环境变量；

4）命令行参数；
等等.....

15、Spring Boot 的核心配置文件有哪几个？它们的区别是什么？

Spring Boot 的核心配置文件是 application 和 bootstrap 配置文件。
application 配置文件这个容易理解，主要用于 Spring Boot 项目的自动化配置。
bootstrap 配置文件有以下几个应用场景。

1. 使用 Spring Cloud Config 配置中心时，这时需要在 bootstrap 配置文件中添加连接到配置中心的配置属性来加载外部配置中心的配置信息；
2. 一些固定的不能被覆盖的属性；
3. 一些加密/解密的场景

16、如何集成 Spring Boot 和 ActiveMQ

对于集成 Spring Boot 和 ActiveMQ，我们使用spring-boot-starter-activemq依赖关系。 它只需要很少的配置，并且不需要样板代码

17、什么是 JavaConfig？

Spring JavaConfig 是 Spring 社区的产品，它提供了配置 Spring IoC 容器的纯Java 方法。因此它有助于避免使用 XML 配置。使用 JavaConfig 的优点在于：

1、面向对象的配置。由于配置被定义为 JavaConfig 中的类，因此用户可以充分利用 Java 中的面向对象功能。一个配置类可以继承另一个，重写它的@Bean 方法等。

2、减少或消除 XML 配置。基于依赖注入原则的外化配置的好处已被证明。但是，许多开发人员不希望在 XML 和 Java 之间来回切换。JavaConfig 为开发人员提供了一种纯 Java 方法来配置与 XML 配置概念相似的 Spring 容器。从技术角度来讲，只使用 JavaConfig 配置类来配置容器是可行的，但实际上很多人认为将JavaConfig 与 XML 混合匹配是理想的。

3、类型安全和重构友好。JavaConfig 提供了一种类型安全的方法来配置 Spring容器。由于 Java 5.0 对泛型的支持，现在可以按类型而不是按名称检索 bean，不需要任何强制转换或基于字符串的查找。

18、如何重新加载 Spring Boot 上的更改，而无需重新启动服务器？

这可以使用 DEV 工具来实现。通过这种依赖关系，您可以节省任何更改，嵌入式tomcat 将重新启动。Spring Boot 有一个开发工具（ DevTools ）模块，它有助于提高开发人员的生产力。Java 开发人员面临的一个主要挑战是将文件更改自动部署到服务器并自动重启服务器。开发人员可以重新加载 Spring Boot 上的更改，而无需重新启动服务器。这将消除每次手动部署更改的需要。Spring Boot 在发布它的第一个版本时没有这个功能。这是开发人员最需要的功能。DevTools 模块完全满足开发人员的需求。该模块将在生产环境中被禁用。它还提供 H2 数据库控制台以更好地测试应用程序。

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <optional>true</optional>
</dependency>
```

19、Spring Boot 中的监视器是什么？

Spring boot actuator 是 spring 启动框架中的重要功能之一。Spring boot 监视器可帮助您访问生产环境中正在运行的应用程序的当前状态。有几个指标必须在生产环境中进行检查和监控。即使一些外部应用程序可能正在使用这些服务来向相关人员触发警报消息。监视器模块公开了一组可直接作为 HTTP URL 访问的REST 端点来检查状态。

20、如何在 Spring Boot 中禁用 Actuator 端点安全性？

默认情况下，所有敏感的 HTTP 端点都是安全的，只有具有 ACTUATOR 角色的用户才能访问它们。安全性是使用标准的 HttpServletRequest.isUserInRole 方法实施的。 我们可以使用来禁用安全性。只有在执行机构端点在防火墙后访问时，才建议禁用安全性。

21、如何在自定义端口上运行 Spring Boot 应用程序？

为了在自定义端口上运行 Spring Boot 应用程序，您可以在application.properties 中指定端口。

```
server.port = 8090
```

22、什么是 YAML？

YAML 是一种人类可读的数据序列化语言。它通常用于配置文件。

与属性文件相比，如果我们想要在配置文件中添加复杂的属性，YAML 文件就更加结构化，而且更少混淆。可以看出 YAML 具有分层配置数据。

23、如何实现 Spring Boot 应用程序的安全性？

为了实现 Spring Boot 的安全性，我们使用 spring-boot-starter-security 依赖项，并且必须添加安全配置。它只需要很少的代码。配置类将必须扩展WebSecurityConfigurerAdapter 并覆盖其方法。

24、如何集成 Spring Boot 和 ActiveMQ？

对于集成 Spring Boot 和 ActiveMQ，我们使用
依赖关系。 它只需要很少的配置，并且不需要样板代码。

25、如何使用 Spring Boot 实现分页和排序？

使用 Spring Boot 实现分页非常简单。使用 Spring Data-JPA 可以实现将可分页的传递给存储库方法。

26、什么是 Swagger？你用 Spring Boot 实现了它吗？

Swagger 广泛用于可视化 API，使用 Swagger UI 为前端开发人员提供在线沙箱。Swagger 是用于生成 RESTful Web 服务的可视化表示的工具，规范和完整框架实现。它使文档能够以与服务器相同的速度更新。当通过 Swagger 正确定义时，消费者可以使用最少量的实现逻辑来理解远程服务并与其进行交互。因此，Swagger
消除了调用服务时的猜测。

27、什么是 Spring Profiles？

Spring Profiles 允许用户根据配置文件（dev，test，prod 等）来注册 bean。因此，当应用程序在开发中运行时，只有某些 bean 可以加载，而在 PRODUCTION中，某些其他 bean 可以加载。假设我们的要求是 Swagger 文档仅适用于 QA 环境，并且禁用所有其他文档。这可以使用配置文件来完成。Spring Boot 使得使用配置文件非常简单。

28、什么是 Spring Batch？

Spring Boot Batch 提供可重用的函数，这些函数在处理大量记录时非常重要，包括日志/跟踪，事务管理，作业处理统计信息，作业重新启动，跳过和资源管理。它还提供了更先进的技术服务和功能，通过优化和分区技术，可以实现极高批量和高性能批处理作业。简单以及复杂的大批量批处理作业可以高度可扩展的方式利用框架处理重要大量的信息。

29、什么是 FreeMarker 模板？

FreeMarker 是一个基于 Java 的模板引擎，最初专注于使用 MVC 软件架构进行动态网页生成。使用 Freemarker 的主要优点是表示层和业务层的完全分离。程序员可以处理应用程序代码，而设计人员可以处理 html 页面设计。最后使用freemarker 可以将这些结合起来，给出最终的输出页面。

30、如何使用 Spring Boot 实现异常处理？

Spring 提供了一种使用 ControllerAdvice 处理异常的非常有用的方法。 我们通过实现一个 ControlerAdvice 类，来处理控制器类抛出的所有异常。

31、您使用了哪些 starter maven 依赖项？

使用了下面的一些依赖项

```
spring-boot-starter-activemq
spring-boot-starter-security
```

这有助于增加更少的依赖关系，并减少版本的冲突。

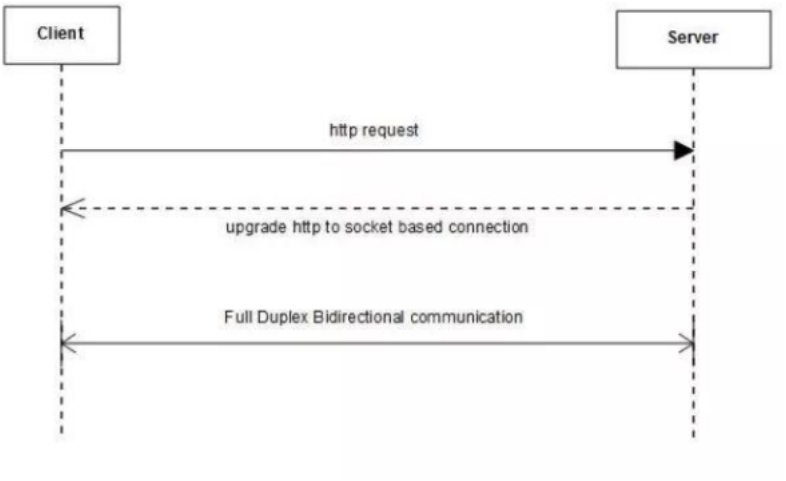
32、什么是 CSRF 攻击？

CSRF 代表跨站请求伪造。这是一种攻击，迫使最终用户在当前通过身份验证的Web 应用程序上执行不需要的操作。CSRF 攻击专门针对状态改变请求，而不是数据窃取，因为攻击者无法查看对伪造请求的响应。

这有助于增加更少的依赖关系，并减少版本的冲突。

33、什么是 WebSockets？

WebSocket 是一种计算机通信协议，通过单个 TCP 连接提供全双工通信信道。



1、WebSocket 是双向的 -使用 WebSocket 客户端或服务器可以发起消息发送。
2、WebSocket 是全双工的 -客户端和服务器通信是相互独立的。
3、单个 TCP 连接 -初始连接使用 HTTP，然后将此连接升级到基于套接字的连接。然后这个单一连接用于所有未来的通信
4、Light -与 http 相比，WebSocket 消息数据交换要轻得多。

34、什么是 AOP？

在软件开发过程中，跨越应用程序多个点的功能称为交叉问题。这些交叉问题与应用程序的主要业务逻辑不同。因此，将这些横切关注与业务逻辑分开是面向方面编程（AOP）的地方。

35、什么是 Apache Kafka？

Apache Kafka 是一个分布式发布 - 订阅消息系统。它是一个可扩展的，容错的发布 - 订阅消息系统，它使我们能够构建分布式应用程序。这是一个 Apache 顶级项目。Kafka 适合离线和在线消息消费。

36、我们如何监视所有 Spring Boot 微服务？

Spring Boot 提供监视器端点以监控各个微服务的度量。这些端点对于获取有关应用程序的信息（如它们是否已启动）以及它们的组件（如数据库等）是否正常运行很有帮助。但是，使用监视器的一个主要缺点或困难是，我们必须单独打开应用程序的知识点以了解其状态或健康状况。想象一下涉及 50 个应用程序的微服务，管理员将不得不击中所有 50 个应用程序的执行终端。为了帮助我们处理这种情况，我们将使用位于的开源项目。 它建立在 Spring Boot Actuator 之上，它提供了一个 Web UI，使我们能够可视化多个应用程序的度量。

37、Spring Boot 的配置文件有哪几种格式？它们有什么区别？

.properties 和 .yml，它们的区别主要是书写格式不同。

1).properties

```
app.user.name = javastack
```

2).yml

```
app:
  user:
    name: javastack
```

另外，.yml 格式不支持 `@PropertySource` 注解导入配置。

38、开启 Spring Boot 特性有哪几种方式？

1）继承spring-boot-starter-parent项目

2）导入spring-boot-dependencies项目依赖

39、Spring Boot 的目录结构是怎样的？

```
cn
 +- javastack
     +- MyApplication.java
     |
```

```
+- customer
|   +- Customer.java
|   +- CustomerController.java
|   +- CustomerService.java
|   +- CustomerRepository.java
|
+- order
    +- Order.java
    +- OrderController.java
    +- OrderService.java
    +- OrderRepository.java
```

这个目录结构是主流及推荐的做法，而在主入口类上加上 @SpringBootApplication 注解来开启 Spring Boot 的各项能力，如自动配置、组件扫描等。

40、运行 Spring Boot 有哪几种方式？

1) 打包用命令或者放到容器中运行

2) 用 Maven/ Gradle 插件运行

3) 直接执行 main 方法运行

41、Spring Boot 自动配置原理是什么？

注解 @EnableAutoConfiguration, @Configuration, @ConditionalOnClass 就是自动配置的核心，首先它得是一个配置文件，其次根据类路径下是否有这个类去自动配置。

42、如何在 Spring Boot 启动的时候运行一些特定的代码？

可以实现接口 ApplicationRunner 或者 CommandLineRunner，这两个接口实现方式一样，它们都只提供了一个 run 方法

43、Spring Boot 有哪几种读取配置的方式？

Spring Boot 可以通过 @PropertySource,@Value,@Environment, @ConfigurationProperties 来绑定变量

44、Spring Boot 支持哪些日志框架？推荐和默认的日志框架是哪个？

Spring Boot 支持 Java Util Logging, Log4j2, Lockback 作为日志框架，如果你使用 Starters 启动器，Spring Boot 将使用 Logback 作为默认日志框架

45、Spring Boot 如何定义多套不同环境配置？

提供多套配置文件，如：

```
applcation.properties

application-dev.properties

application-test.properties

application-prod.properties
```

运行时指定具体的配置文件

46、Spring Boot 可以兼容老 Spring 项目吗，如何做？

可以兼容，使用 `@ImportResource` 注解导入老 Spring 项目配置文件。

47、保护 Spring Boot 应用有哪些方法？

- 在生产中使用HTTPS
- 使用Snyk检查你的依赖关系
- 升级到最新版本
- 启用CSRF保护
- 使用内容安全策略防止XSS攻击
- ...

48、Spring Boot 2.X 有什么新特性？与 1.X 有什么区别？

- 配置变更
- JDK 版本升级
- 第三方类库升级
- 响应式 Spring 编程支持
- HTTP/2 支持
- 配置属性绑定
- 更多改进与加强...

49、如何重新加载Spring Boot上的更改，而无需重新启动服务器？

这可以使用DEV工具来实现。通过这种依赖关系，您可以节省任何更改，嵌入式tomcat将重新启动。
Spring Boot有一个开发工具（DevTools）模块，它有助于提高开发人员的生产力。Java开发人员面临的一个主要挑战是将文件更改自动部署到服务器并自动重启服务器。
开发人员可以重新加载Spring Boot上的更改，而无需重新启动服务器。这将消除每次手动部署更改的需要。Spring Boot在发布它的第一个版本时没有这个功能。
这是开发人员最需要的功能。DevTools模块完全满足开发人员的需求。该模块将在生产环境中被禁用。它还提供H2数据库控制台以更好地测试应用程序。

org.springframework.boot
spring-boot-devtools
true

50、springboot集成mybatis的过程

添加mybatis的starter maven依赖

org.mybatis.spring.boot
mybatis-spring-boot-starter
1.2.0

在mybatis的接口中 添加@Mapper注解
在application.yml配置数据源信息

51、Spring Boot、Spring MVC 和 Spring 有什么区别？

SpringFrame

> SpringFramework 最重要的特征是依赖注入。所有 SpringModules 不是依赖注入就是 IOC 控制反转。

当我们恰当的使用 DI 或者是 IOC 的时候，我们可以开发松耦合应用。松耦合应用的单元测试可以很容易的进行。

SpringMVC

Spring MVC 提供了一种分离式的方法来开发 Web 应用。通过运用像 DispatcherServelet，MoudlAndView 和 ViewResolver 等一些简单的概念，开发 Web 应用将会变的非常简单。

SpringBoot

Spring 和 SpringMVC 的问题在于需要配置大量的参数。

```
<bean
 class="org.springframework.web.servlet.view.InternalResourceViewResolver">
<property name="prefix">
        <value>/WEB-INF/views/</value>
    </property>
    <property name="suffix">
        <value>.jsp</value>
    </property>
  </bean>

  <mvc:resources mapping="/webjars/**" location="/webjars/"/>
```

Spring Boot 通过一个自动配置和启动的项来目解决这个问题。为了更快的构建产品就绪应用程序，Spring Boot 提供了一些非功能性特征。

52、什么是 Spring Boot Stater ？

> 启动器是一套方便的依赖没描述符，它可以放在自己的程序中。你可以一站式的获取你所需要的 Spring 和相关技术，而不需要依赖描述符的通过示例代码搜索和复制黏贴的负载。

例如，如果你想使用 Sping 和 JPA 访问数据库，只需要你的项目包含 spring-boot-starter-data-jpa 依赖项，你就可以完美进行。

问题四 **你能否举一个例子来解释更多 Staters 的内容？**

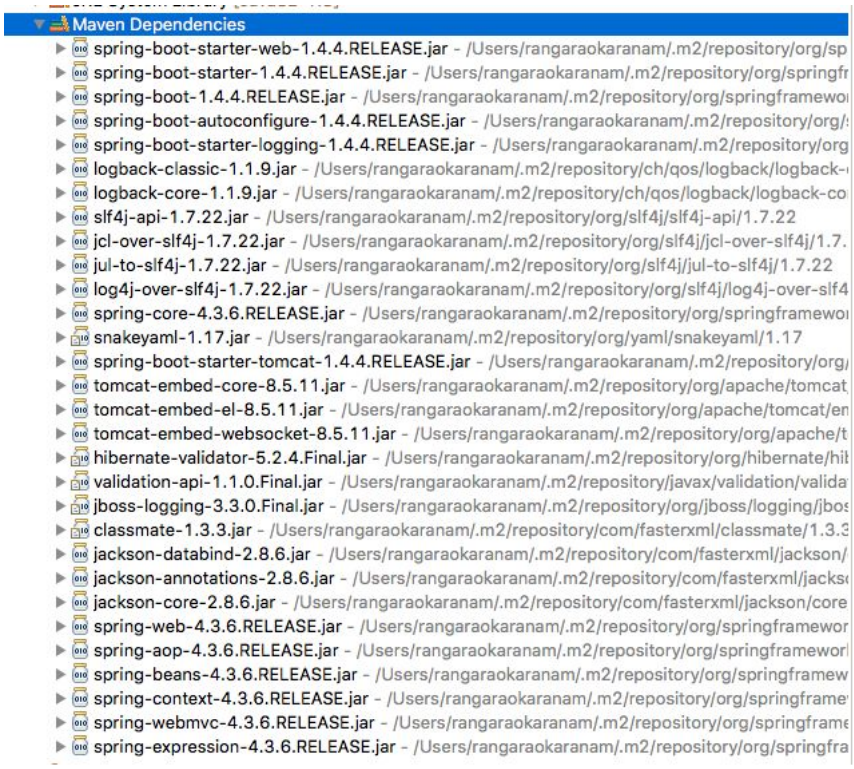
让我们来思考一个 Stater 的例子 -Spring Boot Stater Web。

如果你想开发一个 web 应用程序或者是公开 REST 服务的应用程序。Spring Boot Start Web 是首选。让我们使用 Spring Initializr 创建一个 Spring Boot Start Web 的快速项目。

Spring Boot Start Web 的依赖项

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

下面的截图是添加进我们应用程序的不同的依赖项



依赖项可以被分为

```
Spring - core, beans, context, aop

Web MVC - (Spring MVC)

Jackson - for JSON Binding

Validation - Hibernate,Validation API

Enbedded Servlet Container - Tomcat

Logging - logback,slf4j
```

任何经典的 Web 应用程序都会使用所有这些依赖项。Spring Boot Starter Web 预先打包了这些依赖项。

> 作为一个开发者，我不需要再担心这些依赖项和它们的兼容版本。

53、Spring Boot 还提供了其它的哪些 Starter Project Options？

Spring Boot 也提供了其它的启动器项目包括，包括用于开发特定类型应用程序的典型依赖项。

spring-boot-starter-web-services - SOAP Web Services

spring-boot-starter-web - Web 和 RESTful 应用程序

spring-boot-starter-test - 单元测试和集成测试

spring-boot-starter-jdbc - 传统的 JDBC

spring-boot-starter-hateoas - 为服务添加 HATEOAS 功能

spring-boot-starter-security - 使用 SpringSecurity 进行身份验证和授权

spring-boot-starter-data-jpa - 带有 Hibeernate 的 Spring Data JPA

spring-boot-starter-data-rest - 使用 Spring Data REST 公布简单的 REST 服务

54、Spring 是如何快速创建产品就绪应用程序的？

Spring Boot 致力于快速产品就绪应用程序。为此，它提供了一些譬如高速缓存，日志记录，监控和嵌入式服务器等开箱即用的非功能性特征。

spring-boot-starter-actuator - 使用一些如监控和跟踪应用的高级功能

spring-boot-starter-undertow, spring-boot-starter-jetty, spring-boot-starter-tomcat - 选择您的特定嵌入式 Servlet 容器

spring-boot-starter-logging - 使用 logback 进行日志记录

spring-boot-starter-cache - 启用 Spring Framework 的缓存支持

###Spring2 和 Spring5 所需要的最低 Java 版本是什么？

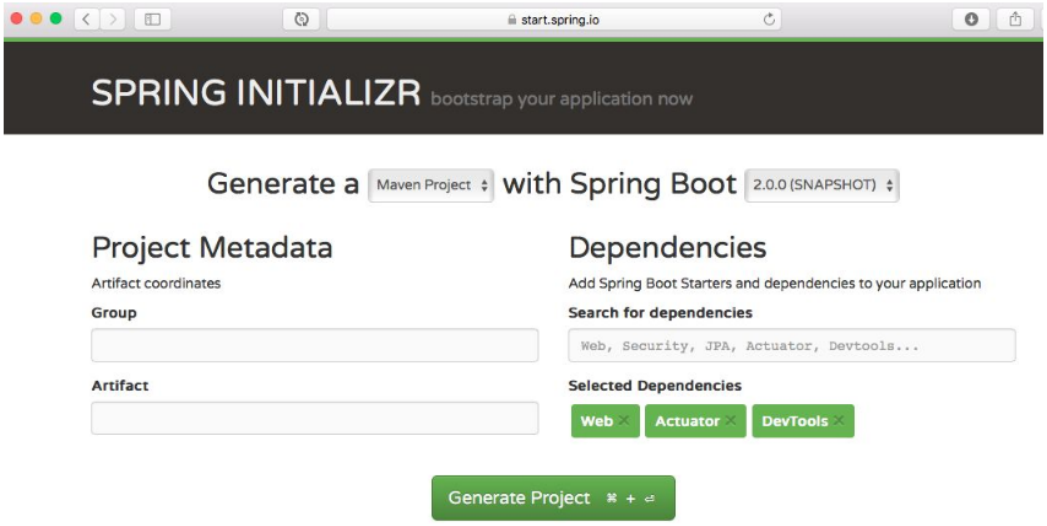
Spring Boot 2.0 需要 Java8 或者更新的版本。Java6 和 Java7 已经不再支持。

推荐阅读：

https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-2.0.0-M1-Release-Notes

55、创建一个 Spring Boot Project 的最简单的方法是什么？

Spring Initializr是启动 Spring Boot Projects 的一个很好的工具。



- 就像上图中所展示的一样，我们需要做一下几步：
- 登录 Spring Initializr，按照以下方式进行选择：
- 选择 com.in28minutes.springboot 为组
- 选择 studet-services 为组件
- 选择下面的依赖项
- Web
- Actuator
- DevTools
- 点击生 GenerateProject
- 将项目导入 Eclipse。文件 - 导入 - 现有的 Maven 项目

56、Spring Initializr 是创建 Spring Boot Projects 的唯一方法吗？

不是的。

Spring Initiatlizr 让创建 Spring Boot 项目变的很容易，但是，你也可以通过设置一个 maven 项目并添加正确的依赖项来开始一个项目。

在我们的 Spring 课程中，我们使用两种方法来创建项目。

第一种方法是 start.spring.io 。
另外一种方法是在项目的标题为“Basic Web Application”处进行手动设置。

手动设置一个 maven 项目

这里有几个重要的步骤：

- 在 Eclipse 中，使用文件 - 新建 Maven 项目来创建一个新项目
- 添加依赖项。
- 添加 maven 插件。
- 添加 Spring Boot 应用程序类。

到这里，准备工作已经做好！

57、如何使用 SpringBoot 自动重装我的应用程序？

使用 Spring Boot 开发工具。

把 Spring Boot 开发工具添加进入你的项目是简单的。

把下面的依赖项添加至你的 Spring Boot Project pom.xml 中

```
<dependency>
     <groupId>org.springframework.boot</groupId>
     <artifactId>spring-boot-devtools</artifactId>
     <scope>runtime</scope>
</dependency>
```

重启应用程序，然后就可以了。

同样的，如果你想自动装载页面，有可以看看 FiveReload

- http://www.logicbig.com/tutorials/spring-framework/spring-boot/boot-live-reload/.

在我测试的时候，发现了 LiveReload 漏洞，如果你测试时也发现了，请一定要告诉我们。

58、 什么是嵌入式服务器？我们为什么要使用嵌入式服务器呢?

思考一下在你的虚拟机上部署应用程序需要些什么。

第一步： 安装 Java

第二部： 安装 Web 或者是应用程序的服务器（Tomat/Wbesphere/Weblogic 等等）

第三部： 部署应用程序 war 包

如果我们想简化这些步骤，应该如何做呢？

让我们来思考如何使服务器成为应用程序的一部分？

> 你只需要一个安装了 Java 的虚拟机，就可以直接在上面部署应用程序了，
> 是不是很爽？

这个想法是嵌入式服务器的起源。

当我们创建一个可以部署的应用程序的时候，我们将会把服务器（例如，tomcat）嵌入到可部署的服务器中。

> 例如，对于一个 Spring Boot 应用程序来说，你可以生成一个包含 Embedded Tomcat 的应用程序 jar。你就可以想运行正常 Java 应用程序一样来运行 web 应用程序了。

嵌入式服务器就是我们的可执行单元包含服务器的二进制文件（例如，tomcat.jar）。

59、如何在 Spring Boot 中添加通用的 JS 代码？

在源文件夹下，创建一个名为 static 的文件夹。然后，你可以把你的静态的内容放在这里面。

例如，myapp.js 的路径是 resources\static\js\myapp.js

你可以参考它在 jsp 中的使用方法

```
<csript src="/js/myapp.js"></script>
```

错误：HAL browser gives me unauthorized error - Full authenticaition is required to access this resource.

该如何来修复这个错误呢？

```
{
  "timestamp": 1488656019562,
  "status": 401,
  "error": "Unauthorized",
  "message": "Full authentication is required to access this resource.",
  "path": "/beans"
}
```

两种方法：

方法 1：关闭安全验证

application.properties

```
management.security.enabled:FALSE
```

方法二：在日志中搜索密码并传递至请求标头中

60、什么是 Spring Date？

来自：//projects.spring.io/spring- data/

> Spring Data 的使命是在保证底层数据存储特殊性的前提下，为数据访问提供一个熟悉的，一致性的，基于 Spring 的编程模型。这使得使用数据访问技术，关系数据库和非关系数据库，map-reduce 框架以及基于云的数据服务变得很容易。

为了让它更简单一些，Spring Data 提供了不受底层数据源限制的 Abstractions 接口。

下面来举一个例子

```
interface TodoRepository extends CrudRepository<Todo, Long> {
```

你可以定义一简单的库，用来插入，更新，删除和检索代办事项，而不需要编写大量的代码。

61、什么是 Spring Data REST?

Spring Data TEST 可以用来发布关于 Spring 数据库的 HATEOAS RESTful 资源。

下面是一个使用 JPA 的例子

```
@RepositoryRestResource(collectionResourceRel = "todos", path = "todos")
public interface TodoRepository
        extends PagingAndSortingRepository<Todo, Long> {
```

不需要写太多代码，我们可以发布关于 Spring 数据库的 RESTful API。

下面展示的是一些关于 TEST 服务器的例子

POST

- URL:http：//localhost：8080/todos
- Use Header:Content-Type:Type:application/json
- Request Content

代码如下

```
{
  "user": "Jill",
  "desc": "Learn Hibernate",
  "done": false
}
```

响应内容

```
{
  "user": "Jill",
  "desc": "Learn Hibernate",
  "done": false,
  "_links": {
"self": {
  "href": "http://localhost:8080/todos/1"
},
"todo": {
  "href": "http://localhost:8080/todos/1"
}
  }
}
```

响应包含新创建资源的 href。

62、path=”users”, collectionResourceRel=”users” 如何与 Spring Data Rest 一起使用？

```
@RepositoryRestResource(collectionResourceRel = "users", path = "users")

public interface UserRestRepository extends
PagingAndSortingRepository<User, Long>
```

- path- 这个资源要导出的路径段。
- collectionResourceRel- 生成指向集合资源的链接时使用的 rel 值。在生成 HATEOAS 链接时使用。

63、当 Spring Boot 应用程序作为 Java 应用程序运行时，后台会发生什么？

如果你使用 Eclipse IDE，Eclipse maven 插件确保依赖项或者类文件的改变一经添加，就会被编译并在目标文件中准备好！在这之后，就和其它的 Java 应用程序一样了。

当你启动 java 应用程序的时候，spring boot 自动配置文件就会魔法般的启用了。

- 当 Spring Boot 应用程序检测到你正在开发一个 web 应用程序的时候，它就会启动 tomcat。

64、我们能否在 spring-boot-starter-web 中用 jetty 代替 tomcat？

在 spring-boot-starter-web 移除现有的依赖项，并把下面这些添加进去。

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
    <exclusions>
        <exclusion>
```

```
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-tomcat</artifactId>
        </exclusion>
    </exclusions>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-jetty</artifactId>
</dependency>
```

65、如何使用 Spring Boot 生成一个 WAR 文件？

推荐阅读:

- https://spring.io/guides/gs/convert-jar-to-war/

下面有 spring 说明文档直接的链接地址：

- https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/#build-tool-plugins-maven-packaging

66、如何使用 Spring Boot 部署到不同的服务器？

你需要做下面两个步骤：

- 在一个项目中生成一个 war 文件。
- 将它部署到你最喜欢的服务器（websphere 或者 Weblogic 或者 Tomcat and so on）。

第一步：这本入门指南应该有所帮助：
https://spring.io/guides/gs/convert-jar-to-war/

第二步：取决于你的服务器。

67、RequestMapping 和 GetMapping 的不同之处在哪里？

- RequestMapping 具有类属性的，可以进行 GET,POST,PUT 或者其它的注释中具有的请求方法。
- GetMapping 是 GET 请求方法中的一个特例。它只是 ResquestMapping 的一个延伸，目的是为了提高清晰度。

68、为什么我们不建议在实际的应用程序中使用 Spring Data Rest?

我们认为 Spring Data Rest 很适合快速原型制造！在大型应用程序中使用需要谨慎。

通过 Spring Data REST 你可以把你的数据实体作为 RESTful 服务直接发布。

当你设计 RESTful 服务器的时候，最佳实践表明，你的接口应该考虑到两件重要的事情：

- 你的模型范围。
- 你的客户。

通过 With Spring Data REST，你不需要再考虑这两个方面，只需要作为 TEST 服务发布实体。

这就是为什么我们建议使用 Spring Data Rest 在快速原型构造上面，或者作为项目的初始解决方法。对于完整演变项目来说，这并不是一个好的注意。

69、在 Spring Initializer 中，如何改变一个项目的包名字？

好消息是你可以定制它。点击链接“转到完整版本”。你可以配置你想要修改的包名称！

70、可以配置 application.propertierde 的完整的属性列表在哪里可以找到？

这里是完整的指南：

- https://docs.spring.io/spring-boot/docs/current/reference/html/common-application-properties.html

71、JPA 和 Hibernate 有哪些区别？

简而言之

- JPA 是一个规范或者接口
- Hibernate 是 JPA 的一个实现

当我们使用 JPA 的时候，我们使用 javax.persistence 包中的注释和接口时，不需要使用 hibernate 的导入包。

我们建议使用 JPA 注释，因为哦我们没有将其绑定到 Hibernate 作为实现。后来（我知道 - 小于百分之一的几率），我们可以使用另一种 JPA 实现。

72、使用 Spring Boot 启动连接到内存数据库 H2 的 JPA 应用程序需要哪些依赖项？

在 Spring Boot 项目中，当你确保下面的依赖项都在类路里面的时候，你可以加载 H2 控制台。

- web 启动器
- h2
- jpa 数据启动器

其它的依赖项在下面：

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
</dependency>
```

需要注意的一些地方：

- 一个内部数据内存只在应用程序执行期间存在。这是学习框架的有效方式。
- 这不是你希望的真是世界应用程序的方式。
- 在问题“如何连接一个外部数据库？”中，我们解释了如何连接一个你所选择的数据库。

73、如何不通过任何配置来选择 Hibernate 作为 JPA 的默认实现？

因为 Spring Boot 是自动配置的。

下面是我们添加的依赖项

```
<dependency>
           <groupId>org.springframework.boot</groupId>
           <artifactId>spring-boot-starter-data-jpa</artifactId>
      </dependency>
```

spring-boot-stater-data-jpa 对于 Hibernate 和 JPA 有过渡依赖性。

当 Spring Boot 在类路径中检测到 Hibernate 中，将会自动配置它为默认的 JPA 实现。

74、指定的数据库连接信息在哪里？它是如何知道自动连接至 H2 的？

这就是 Spring Boot 自动配置的魔力。

来自：https://docs.spring.io/spring-boot/docs/current/reference/html/using-boot-auto-configuration.html

Spring Boot auto-configuration 试图自动配置你已经添加的基于 jar 依赖项的 Spring 应用程序。比如说，如果 HSQLDBis 存在你的类路径中，并且，数据库连接 bean 还没有手动配置，那么我们可以自动配置一个内存数据库。

进一步的阅读：

http://www.springboottutorial.com/spring-boot-auto-configuration

75、我们如何连接一个像 MSSQL 或者 orcale 一样的外部数据库？

让我们以 MySQL 为例来思考这个问题：

第一步 - 把 mysql 连接器的依赖项添加至 pom.xml

```
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
</dependency>
```

第二步 - 从 pom.xml 中移除 H2 的依赖项

或者至少把它作为测试的范围。

```
<!--
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
<scope>test</scope>
</dependency>
-->
```

第三步 - 安装你的 MySQL 数据库

更多的来看看这里 -https://github.com/in28minutes/jpa-with-hibernate#installing-and-setting-up-mysql

第四步 - 配置你的 MySQL 数据库连接

配置 application.properties

```
spring.jpa.hibernate.ddl-auto=none
spring.datasource.url=jdbc:mysql://localhost:3306/todo_example
spring.datasource.username=todouser
spring.datasource.password=YOUR_PASSWORD
```

第五步 - 重新启动，你就准备好了！

就是这么简单！

76、Spring Boot 配置的默认 H2 数据库的名字是上面？为什么默认的数据库名字是 testdb？

在 application.properties 里面，列出了所有的默认值

- https://docs.spring.io/spring-boot/docs/current/reference/html/common-application-properties.html

找到下面的属性

```
spring.datasource.name=testdb # Name of the datasource.
```

如果你使用了 H2 内部存储数据库，它里面确定了 Spring Boot 用来安装你的 H2 数据库的名字。

77、如果 H2 不在类路径里面，会出现上面情况？

将会报下面的错误

```
Cannot determine embedded database driver class for database type NONE
```

把 H2 添加至 pom.xml 中，然后重启你的服务器

```
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
<scope>runtime</scope>
</dependency>
```

78、你能否举一个以 ReadOnly 为事务管理的例子？

当你从数据库读取内容的时候，你想把事物中的用户描述或者是其它描述设置为只读模式，以便于 Hebernate 不需要再次检查实体的变化。这是非常高效的。

79、发布 Spring Boot 用户应用程序自定义配置的最好方法是什么？

@Value 的问题在于，您可以通过应用程序分配你配置值。更好的操作是采取集中的方法。
你可以使用 @ConfigurationProperties 定义一个配置组件。

```
@Component
@ConfigurationProperties("basic")
public class BasicConfiguration {
    private boolean value;
    private String message;
    private int number;
```

你可以在 application.properties 中配置参数。

```
basic.value: true
basic.message: Dynamic Message
basic.number: 100
```

80、配置文件的需求是什么？

企业应用程序的开发是复杂的，你需要混合的环境：

- Dev
- QA

- Stage
- Production

在每个环境中，你想要不同的应用程序配置。

> 配置文件有助于在不同的环境中进行不同的应用程序配置。

Spring 和 Spring Boot 提供了你可以制定的功能。

- 不同配置文件中，不同环境的配置是什么？
- 为一个制定的环境设置活动的配置文件。

Spring Boot 将会根据特定环境中设置的活动配置文件来选择应用程序的配置。

81、如何使用配置文件通过 Spring Boot 配置特定环境的配置？

配置文件不是设别环境的关键。

在下面的例子中，我们将会用到两个配置文件

- dev
- prod

缺省的应用程序配置在 application.properties 中。让我们来看下面的例子：

application.properties

```
basic.value= true
basic.message= Dynamic Message
basic.number= 100
```

我们想要为 dev 文件自定义 application.properties 属性。我们需要创建一个名为 application-dev.properties 的文件，并且重写我们想要自定义的属性。

application-dev.properties

```
basic.message: Dynamic Message in DEV
```

一旦你特定配置了配置文件，你需要在环境中设定一个活动的配置文件。

有多种方法可以做到这一点：

- 在 VM 参数中使用 Dspring.profiles.active=prod
- 在 application.properties 中使用 spring.profiles.active=prod

文章来源： http://www.3xmq.com/article/1522809264295

82、我们如何使用Maven设置Spring Boot应用程序？

我们可以像在任何其他库中一样在Maven项目中包含Spring Boot。但是，最好的方法是从 `spring-boot-starter-parent` 项目继承并声明依赖于Spring Boot启动器。这样做可以让我们的项目重用Spring Boot的默认设置。 继承 `spring-boot-starter-parent` 项目非常简单 - 我们只需要在 `pom.xml` 中指定一个 `parent` 元素：

```
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>2.1.6.RELEASE</version>
</parent>
```

我们可以在Maven 中央仓库找到最新版本的 `spring-boot-starter-parent`。 上面的方式很方便但是并不一定符合实际需要。例如公司要求所有项目依赖构建从一个标准BOM开始，我们就不能按上面的方式进行。 在这种情况下，我们可以进行如下引用：

```
<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-dependencies</artifactId>
            <version>2.1.6.RELEASE</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
```

83、如何禁用特定的自动配置？

如果我们要禁用特定的自动配置，我们可以使用 `@EnableAutoConfiguration` 注解的 `exclude` 属性来指示它。如下禁用了数据源自动配置 `DataSourceAutoConfiguration` ：

```
// other annotations
@EnableAutoConfiguration(exclude = DataSourceAutoConfiguration.class)
public class MyConfiguration { }
```

如果我们使用 `@SpringBootApplication` 注解。 它具有 `@EnableAutoConfiguration` 作为元注解 - 我们同样可以配置exclude属性来禁用自动配置：

```
// other annotations
@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)
public class MyConfiguration { }
```

我们还可以使用 `spring.autoconfigure.exclude` 环境属性禁用自动配置。在 `application.properties` (也可以是 `application.yml`)配置文件设置如下也可以达到同样的目的：

```
spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
```

84、Spring boot支持哪些外部配置？

Spring Boot支持外部配置，允许我们在各种环境中运行相同的应用程序。我们可以使用 `properties` 文件， `YAML文件` ，环境变量，系统属性和命令行选项参数来指定配置属性。 然后，我们可以访问使用这些属性@Value注释，经由绑定对象 的 `@ConfigurationProperties` 注释或 `Environment` 环境抽象类注入。 以下是最常见的外部配置来源：

- 命令行属性：命令行选项参数是以双连字符开头的程序参数，例如 `-server.port = 8080` 。Spring Boot将所有参数转换为属性，并将它们添加到环境属性集中。
- 应用程序属性：应用程序属性是从 `application.properties` 文件或其 `YAML` 对应文件加载的属性。默认情况下，Spring Boot会在当前目录，类路径根或其 `config` 子目录中搜索此文件。 特定于配置文件的属性：特定于配置文件的属性从 `application- {profile} .properties` 文件或其 `YAML` 对应文件加载。 `{profile}` 占位符是指活性轮廓。这些文件与非特定属性文件位于相同位置，并且优先于非特定属性文件。

85、如何对Spring Boot应用进行测试？

在为Spring应用程序运行集成测试时，我们必须有一个 `ApplicationContext` 。 为了简化测试，Spring Boot为测试提供了一个特殊的注释 `@SpringBootTest` 。此批注从其 `classes` 属性指示的配置类创建 `ApplicationContext` 。 如果未设置classes属性，Spring Boot将搜索主配置类。搜索从包含测试的包开始，直到找到使用@SpringBootApplication或@SpringBootConfiguration注释的类。 请注意，如果我们使用 `JUnit 4` ，我们必须用 `@RunWith（SpringRunner.class）` 装饰测试类。

86、Spring Boot Actuator有什么用？

`Spring Boot Actuator` 可以帮助你监控和管理Spring Boot应用，比如健康检查、审计、统计和HTTP追踪等。所有的这些特性可以通过 `JMX` 或者 `HTTP endpoints` 来获得。 Actuator同时还可以与外部应用监控系统整合，比如 `Prometheus`, `Graphite`, `DataDog`, `Influx`, `Wavefront`, `New Relic` 等。这些系统提供了非常好的仪表盘、图标、分析和告警等功能，使得你可以通过统一的接口轻松的监控和管理你的应用。 `Actuator` 使用 `Micrometer` 来整合上面提到的外部应用监控系统。这使得只要通过非常小的配置就可以集成任何应用监控系统。将Spring Boot Actuator集成到一个项目中非常简单。我们需要做的就是在 `pom.xml` 文件中包含 `spring-boot-starter-actuator` 启动器：

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

`Spring Boot Actuator` 可以使用 `HTTP` 或 `JMX` 端点公开操作信息。但是，大多数应用程序都使用 `HTTP` ，其中端点的标识和/执行器前缀形成 `URL` 路径。 以下是Actuator提供的一些最常见的内置端点：

- `auditevents` ： 公开审计事件信息
- `env` ： 公开环境属性
- `health` ： 显示应用程序运行状况信息
- `httptrace` ： 显示HTTP跟踪信息

- `info`：显示任意应用程序信息
- `metrics`：显示指标信息
- `mappings`：显示所有@RequestMapping路径的列表
- `scheduledtasks`：显示应用程序中的计划任务
- `threaddump`：执行线程转储
- `beans`：所有加载的spring bean

更多关于`Spring Boot Actuator`的信息可查看Spring Boot 2.x 中的 Actuator 。 **请注意：生产使用Actuator务必保护好这些端点，避免未授权的访问请求。**

87、SpringBoot 中静态首页默认位置可以放在哪里？

当我们应用根目录时，可以直接映射，将 index.html 放入下面的位置：

```
classpath:/META-INF/resources/index.html
classpath:/resources/index.html
classpath:/static/index.html
classpath:/public/index.html
```

89、SpringBoot 中静态资源直接映射的优先级是怎样的？

SpringBoot 静态资源直接映射为/**，可以通过根目录来访问。/META-INF/resources/webjars/映射为/webjars/，通过访问 /webjar 访问。优先级顺序为：META-INF/resources > resources > static > public。

90、继承 WebMvcConfigurerAdapter 抽象类，常用的重写方法列举几个？

WebMvcConfigurerAdapter 实现 WebMvcConfigurer 接口，常用的可能需要重写的方法有下面几个：

```
/** 解决跨域问题 **/
public void addCorsMappings(CorsRegistry registry) ;
/** 添加拦截器 **/
void addInterceptors(InterceptorRegistry registry);
/** 这里配置视图解析器 **/
void configureViewResolvers(ViewResolverRegistry registry);
/** 配置内容裁决的一些选项 **/
void configureContentNegotiation(ContentNegotiationConfigurer configurer);
/** 视图跳转控制器 **/
void addViewControllers(ViewControllerRegistry registry);
/** 静态资源处理 **/
void addResourceHandlers(ResourceHandlerRegistry registry);
/** 默认静态资源处理器 **/
void configureDefaultServletHandling(DefaultServletHandlerConfigurer configurer);
```

91、@SpringBootApplication 引入了哪3个重要的注解？

@SpringBootConfiguration、@EnableAutoConfiguration、@ComponentScan。其它的 4 个 @Target、@Retention、@Documented、@Inherited，也重要，但应该不是本题想问的知识点。

92、@SpringBootApplication 注解中的属性相当于哪几个注解？

等价于以默认属性使用 @Configuration，@EnableAutoConfiguration 和 @ComponentScan。

Spring Cloud面试题

1、什么是 Spring Cloud？

Spring cloud 流应用程序启动器是基于 Spring Boot 的 Spring 集成应用程序，提供与外部系统的集成。Spring cloud Task，一个生命周期短暂的微服务框架，用于快速构建执行有限数据处理的应用程序。

2、使用 Spring Cloud 有什么优势？

使用 Spring Boot 开发分布式微服务时，我们面临以下问题
1、与分布式系统相关的复杂性-这种开销包括网络问题，延迟开销，带宽问题，安全问题。
2、服务发现-服务发现工具管理群集中的流程和服务如何查找和互相交谈。它涉及一个服务目录，在该目录中注册服务，然后能够查找并连接到该目录中的服务。
3、冗余-分布式系统中的冗余问题。
4、负载平衡 --负载平衡改善跨多个计算资源的工作负荷，诸如计算机，计算机集群，网络链路，中央处理单元，或磁盘驱动器的分布。
5、性能-问题 由于各种运营开销导致的性能问题。
6、部署复杂性-Devops 技能的要求。

3、服务注册和发现是什么意思？Spring Cloud 如何实现？

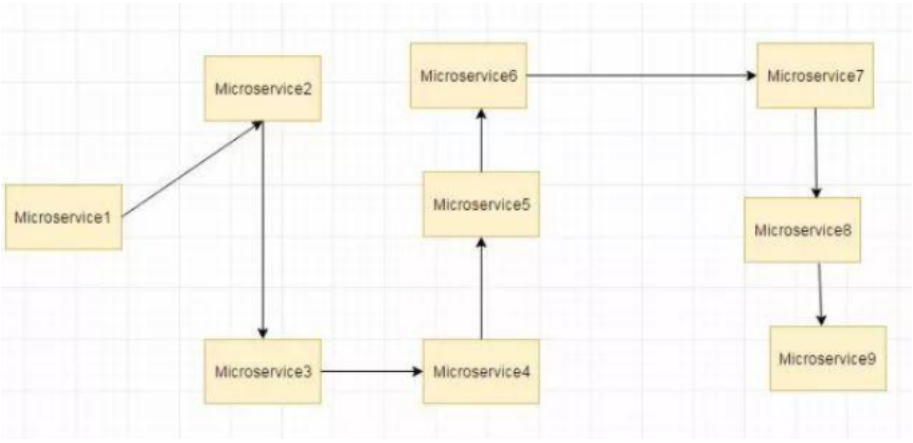
当我们开始一个项目时，我们通常在属性文件中进行所有的配置。随着越来越多的服务开发和部署，添加和修改这些属性变得更加复杂。有些服务可能会下降，而某些位置可能会发生变化。手动更改属性可能会产生问题。 Eureka 服务注册和发现可以在这种情况下提供帮助。由于所有服务都在 Eureka 服务器上注册并通过调用 Eureka 服务器完成查找，因此无需处理服务地点的任何更改和处理。

4、负载平衡的意义什么？

在计算中，负载平衡可以改善跨计算机，计算机集群，网络链接，中央处理单元或磁盘驱动器等多种计算资源的工作负载分布。负载平衡旨在优化资源使用，最大化吞吐量，最小化响应时间并避免任何单一资源的过载。使用多个组件进行负载平衡而不是单个组件可能会通过冗余来提高可靠性和可用性。负载平衡通常涉及专用软件或硬件，例如多层交换机或域名系统服务器进程。

5、什么是 Hystrix？它如何实现容错？

Hystrix 是一个延迟和容错库，旨在隔离远程系统，服务和第三方库的访问点，当出现故障是不可避免的故障时，停止级联故障并在复杂的分布式系统中实现弹性。通常对于使用微服务架构开发的系统，涉及到许多微服务。这些微服务彼此协作。思考以下微服务

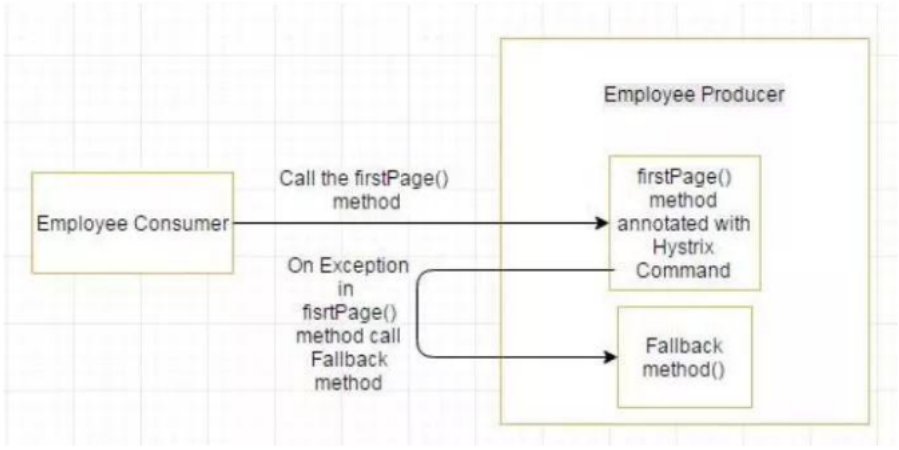


假设如果上图中的微服务 9 失败了，那么使用传统方法我们将传播一个异常。但这仍然会导致整个系统崩溃。 随着微服务数量的增加，这个问题变得更加复杂。微服务的数量可以高达 1000.这是 hystrix 出现的地方 我们将使用 Hystrix 在这种情况下的 Fallback 方法功能。我们有两个服务 employee-consumer 使用由 employee-consumer 公开的服务。简化图如下所示

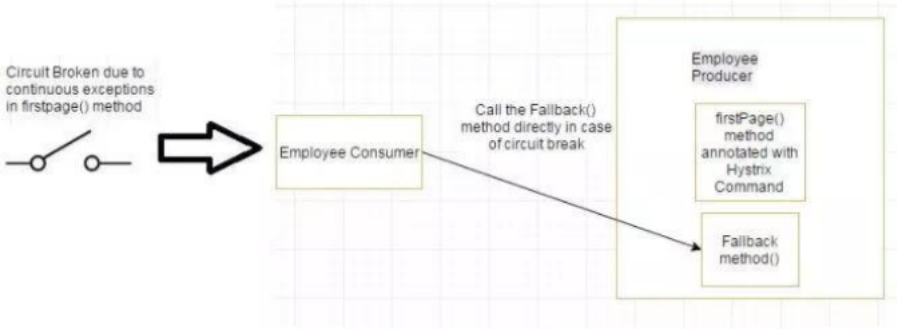
现在假设由于某种原因，employee-producer 公开的服务会抛出异常。我们在这种情况下使用 Hystrix 定义了一个回退方法。这种后备方法应该具有与公开服务相同的返回类型。如果暴露服务中出现异常，则回退方法将返回一些值。

6、什么是 Hystrix 断路器？我们需要它吗？

由于某些原因，employee-consumer 公开服务会引发异常。在这种情况下使用Hystrix 我们定义了一个回退方法。如果在公开服务中发生异常，则回退方法返回一些默认值。



如果 firstPage method() 中的异常继续发生，则 Hystrix 电路将中断，并且员工使用者将一起跳过 firtsPage 方法，并直接调用回退方法。断路器的目的是给第一页方法或第一页方法可能调用的其他方法留出时间，并导致异常恢复。可能发生的情况是，在负载较小的情况下，导致异常的问题有更好的恢复机会 。



7、什么是 Netflix Feign？它的优点是什么？

Feign 是受到 Retrofit，JAXRS-2.0 和 WebSocket 启发的 java 客户端联编程序。Feign 的第一个目标是将约束分母的复杂性统一到 http apis，而不考虑其稳定性。在 employee-consumer 的例子中，我们使用了 employee-producer 使用 REST模板公开的 REST 服务。

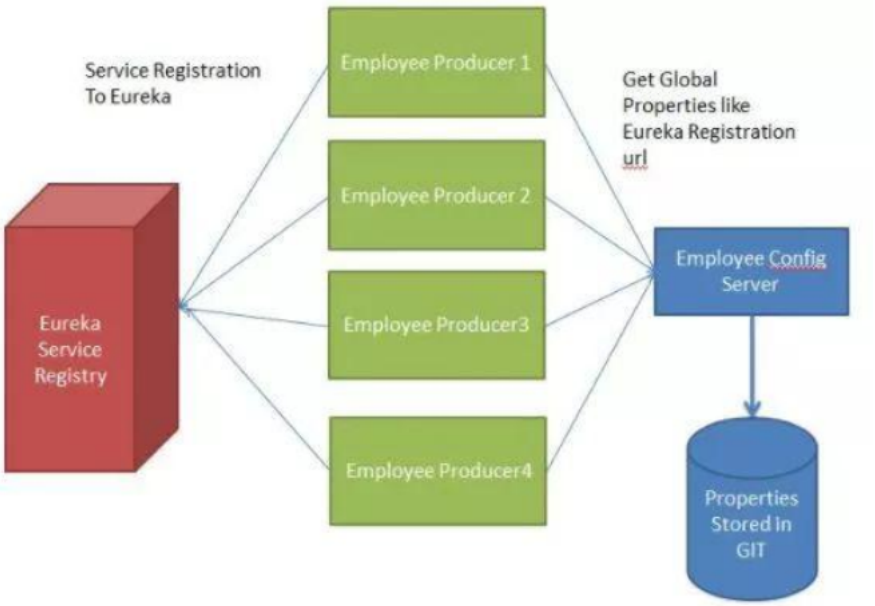
但是我们必须编写大量代码才能执行以下步骤
1、使用功能区进行负载平衡。
2、获取服务实例，然后获取基本 URL。
3、利用 REST 模板来使用服务。 前面的代码如下

```
@Controller
public class ConsumerControllerClient {
    @Autowired
    private LoadBalancerClient loadBalancer;
    public void getEmployee() throws RestClientException, IOException {
        ServiceInstance serviceInstance=loadBalancer.choose("employee-producer");
        System.out.println(serviceInstance.getUri());
        String baseUrl=serviceInstance.getUri().toString();
        baseUrl=baseUrl+"/employee";
        RestTemplate restTemplate = new RestTemplate();
        ResponseEntity<String> response=null;
        try{
                response=restTemplate.exchange(baseUrl,
                HttpMethod.GET, getHeaders(),String.class);
            }catch (Exception ex)
            {
                System.out.println(ex);
            }
        System.out.println(response.getBody());
    }
}
```

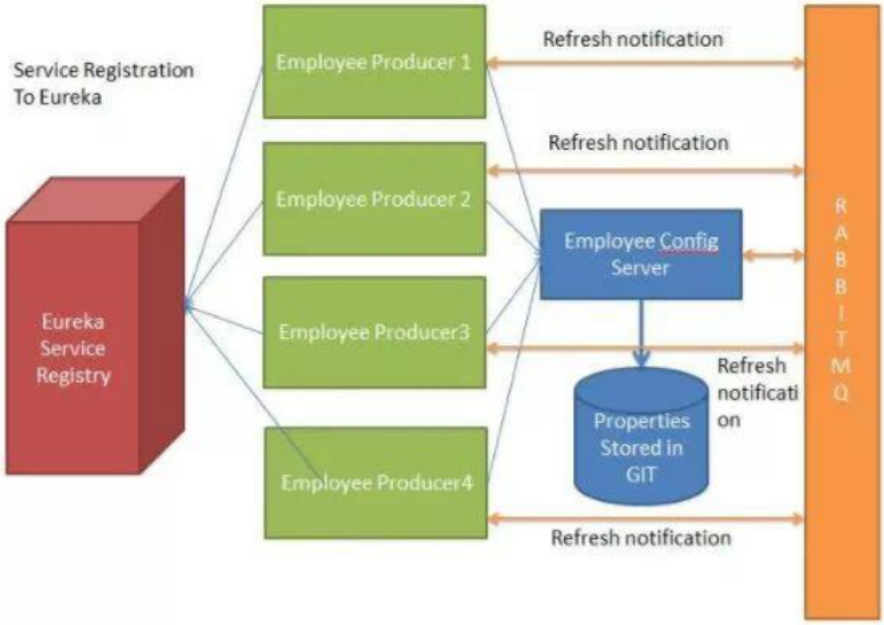
之前的代码，有像 NullPointer 这样的例外的机会，并不是最优的。我们将看到如何使用 Netflix Feign 使呼叫变得更加轻松和清洁。如果 Netflix Ribbon 依赖关系也在类路径中，那么 Feign 默认也会负责负载平衡。

8、什么是 Spring Cloud Bus？我们需要它吗？

考虑以下情况：我们有多个应用程序使用 Spring Cloud Config 读取属性，而Spring Cloud Config 从 GIT 读取这些属性。
下面的例子中多个员工生产者模块从 Employee Config Module 获取 Eureka 注册的财产



如果假设 GIT 中的 Eureka 注册属性更改为指向另一台 Eureka 服务器，会发生什么情况。在这种情况下，我们将不得不重新启动服务以获取更新的属性。还有另一种使用执行器端点/刷新的方式。但是我们将不得不为每个模块单独调用这个 url。例如，如果 Employee Producer1 部署在端口 8080 上，则调用 http：// localhost：8080 / refresh。同样对于 Employee Producer2 http：//localhost：8081 / refresh 等等。这又很麻烦。这就是 Spring Cloud Bus 发挥作用的地方。



Spring Cloud Bus 提供了跨多个实例刷新配置的功能。因此，在上面的示例中，如果我们刷新 Employee Producer1，则会自动刷新所有其他必需的模块。如果我们有多个微服务启动并运行，这特别有用。这是通过将所有微服务连接到单个消息代理来实现的。无论何时刷新实例，此事件都会订阅到侦听此代理的所有微服务，并且它们也会刷新。可以通过使用端点/总线/刷新来实现对任何单个实例的刷新

9、什么是微服务

微服务架构是一种架构模式或者说是一种架构风格，它提倡将单一应用程序划分为一组小的服务，每个服务运行在其独立的自己的进程中，服务之间相互协调、互相配合，为用户提供最终价值。服务之间采用轻量级的通信机制互相沟通（通常是基于HTTP的RESTful API）,每个服务都围绕着具体的业务进行构建，并且能够被独立的构建在生产环境、类生产环境等。另外，应避免统一的、集中式的服务管理机制，对具体的一个服务而言，应根据业务上下文，选择合适的语言、工具对其进行构建，可以有一个非常轻量级的集中式管理来协调这些服务，可以使用不同的语言来编写服务，也可以使用不同的数据存储。

10、什么是服务熔断？什么是服务降级

熔断机制是应对雪崩效应的一种微服务链路保护机制。当某个微服务不可用或者响应时间太长时，会进行服务降级，进而熔断该节点微服务的调用，快速返回"错误"的响应信息。当检测到该节点微服务调用响应正常后恢复调用链路。在SpringCloud框架里熔断机制通过Hystrix实现，Hystrix会监控微服务间调用的状况，当失败的调用到一定阈值，缺省是5秒内调用20次，如果失败，就会启动熔断机制。

服务降级，一般是从整体负荷考虑。就是当某个服务熔断之后，服务器将不再被调用，此时客户端可以自己准备一个本地的fallback回调，返回一个缺省值。这样做，虽然水平下降，但好歹可用，比直接挂掉强。

Hystrix相关注解
@EnableHystrix：开启熔断
@HystrixCommand(fallbackMethod="XXX")：声明一个失败回滚处理函数XXX，当被注解的方法执行超时（默认是1000毫秒），就会执行fallback函数，返回错误提示

微服务条目	落地的技术
服务开发	SpringBoot、Spring、SpringMVC
服务配置管理	Netflix公司的Archaius、阿里的Diamond等
服务注册与发现	Eureka、Consul、Zookeeper
服务调用	RPC、Rest、gRPC
服务熔断器	Hystrix、Envoy等
负载均衡	Nginx、Ribbon
服务接口调用（客户端调用服务的简化工具）	Feign
消息队列	Kafka、RabbitMQ、ActiveMQ等
服务配置中心配置管理	SpringCloudConfig、Chef等
服务路由（API网关）	Zuul
服务监控	Zabbix、Naggios、Metrics、Spectator等
全链路追踪	Zipkin、Brave、Dapper等
服务部署	Docker、OpenStack、Kubernetes等
数据流操作开发包	SpringCloud Stream
事件消息总线	Spring Cloud Bus

11、Eureka和zookeeper都可以提供服务注册与发现的功能，请说说两个的区别？

Zookeeper保证了CP（C：一致性，P：分区容错性），Eureka保证了AP（A：高可用）1.当向注册中心查询服务列表时，我们可以容忍注册中心返回的是几分钟以前的信息，但不能容忍直接down掉不可用。也就是说，服务注册功能对高可用性要求比较高，但zk会出现这样一种情况，当master节点因为网络故障与其他节点失去联系时，剩余节点会重新选leader。问题在于，选取leader时间过长，30 ~ 120s，且选取期间zk集群都不可用，这样就会导致选取期间注册服务瘫痪。在云部署的环境下，因网络问题使得zk集群失去master节点是较大概率会发生的事，虽然服务能够恢复，但是漫长的选取时间导致的注册长期不可用是不能容忍的。

2.Eureka保证了可用性，Eureka各个节点是平等的，几个节点挂掉不会影响正常节点的工作，剩余的节点仍然可以提供注册和查询服务。而Eureka的客户端向某个Eureka注册或发现时发生连接失败，则会自动切换到其他节点，只要有一台Eureka还在，就能保证注册服务可用，只是查到的信息可能不是最新的。除此之外，Eureka还有自我保护机制，如果在15分钟内超过85%的节点没有正常的心跳，那么Eureka就认为客户端与注册中心发生了网络故障，此时会出现以下几种情况：

①、Eureka不在从注册列表中移除因为长时间没有收到心跳而应该过期的服务。
②、Eureka仍然能够接受新服务的注册和查询请求，但是不会被同步到其他节点上（即保证当前节点仍然可用）
③、当网络稳定时，当前实例新的注册信息会被同步到其他节点。因此，Eureka可以很好的应对因网络故障导致部分节点失去联系的情况，而不会像Zookeeper那样使整个微服务瘫痪

12、SpringBoot和SpringCloud的区别？

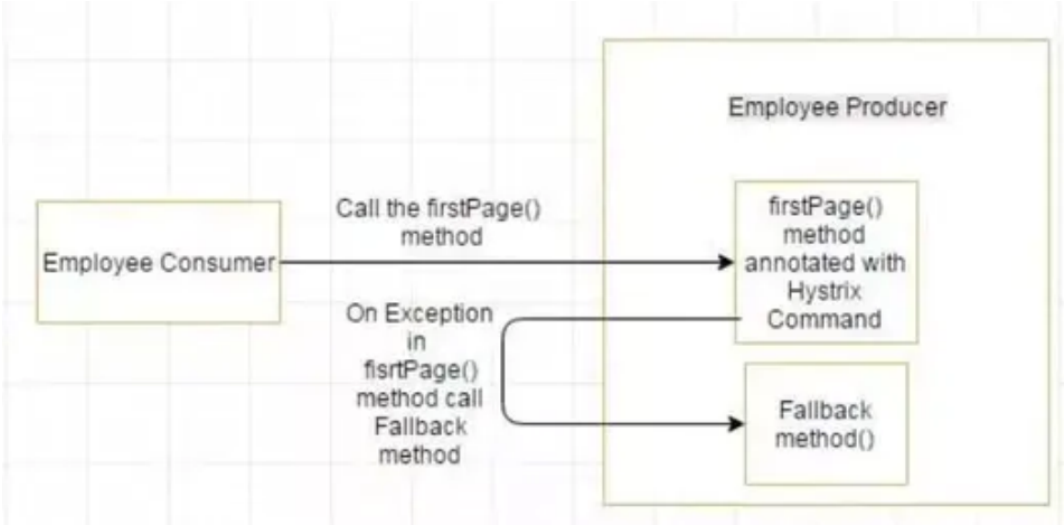
SpringBoot专注于快速方便的开发单个个体微服务。

SpringCloud是关注全局的微服务协调整理治理框架，它将SpringBoot开发的一个个单体微服务整合并
管理起来，为各个微服务之间提供，配置管理、服务发现、断路器、路由、微代理、事件总线、全局锁、决策竞
选、分布式会话等等集成服务SpringBoot可以离开SpringCloud独立使用开发项目， 但是SpringCloud离不开SpringBoot ，属于依赖的关系.

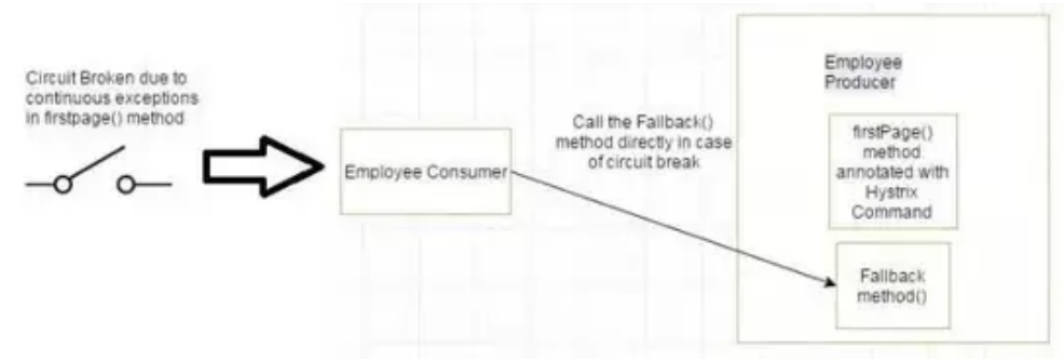
SpringBoot专注于快速、方便的开发单个微服务个体，SpringCloud关注全局的服务治理框架

13、什么是Hystrix断路器？我们需要它吗

由于某些原因，employee-consumer公开服务会引发异常。在这种情况下使用Hystrix我们定义了一个回退方法。如果在公开服务中发生异常，则回退方法返回一些默认值。



如果firstPage method() 中的异常继续发生，则Hystrix电路将中断，并且员工使用者将一起跳过firtsPage方法，并直接调用回退方法。 断路器的目的是给第一页方法或第一页方法可能调用的其他方法留出时间，并导致异常恢复。可能发生的情况是，在负载较小的情况下，导致异常的问题有更好的恢复机会 。



14、说说 RPC 的实现原理

首先需要有处理网络连接通讯的模块，负责连接建立、管理和消息的传输。其次需要有编解码的模块，因为网络通讯都是传输的字节码，需要将我们使用的对象序列化和反序列化。剩下的就是客户端和服务器端的部分，服务器端暴露要开放的服务接口，客户调用服务接口的一个代理实现，这个代理实现负责收集数据、编码并传输给服务器然后等待结果返回。

15、微服务的优点缺点?说下开发项目中遇到的坑?

优点:

1.每个服务直接足够内聚，代码容易理解
2.开发效率高，一个服务只做一件事，适合小团队开发
3.松耦合，有功能意义的服务。
4.可以用不同语言开发，面向接口编程。
5.易于第三方集成
6.微服务只是业务逻辑的代码，不会和HTML,CSS或其他界面结合.
7.可以灵活搭配，连接公共库/连接独立库

缺点:
1.分布式系统的责任性
2.多服务运维难度加大。
3.系统部署依赖，服务间通信成本，数据一致性，系统集成测试，性能监控。

16、spring cloud 和dubbo区别?

1.服务调用方式 dubbo是RPC springcloud Rest Api
2.注册中心,dubbo 是zookeeper springcloud是eureka，也可以是zookeeper
3.服务网关,dubbo本身没有实现，只能通过其他第三方技术整合，springcloud有Zuul路由网关，作为路由服务器，进行消费者的请求分发,springcloud支持断路器，与git完美集成配置文件支持版本控制，事物总线实现配置文件的更新与服务自动装配等等一系列的微服务架构要素。

17、REST 和RPC对比

1.RPC主要的缺陷是服务提供方和调用方式之间的依赖太强，需要对每一个微服务进行接口的定义，并通过持续继承发布，严格版本控制才不会出现冲突。
2.REST是轻量级的接口，服务的提供和调用不存在代码之间的耦合，只需要一个约定进行规范。

18、你所知道的微服务技术栈？

维度(springcloud)
服务开发：springboot spring springmvc
服务配置与管理:Netfix公司的Archaiusm ,阿里的Diamond
服务注册与发现:Eureka,Zookeeper
服务调用:Rest RPC gRpc
服务熔断器:Hystrix
服务负载均衡:Ribbon Nginx
服务接口调用:Fegin
消息队列:Kafka Rabbitmq activemq
服务配置中心管理:SpringCloudConfig
服务路由（API网关）Zuul
事件消息总线:SpringCloud Bus

19、微服务之间是如何独立通讯的?

1.远程调用，比如feign调用，直接通过远程过程调用来访问别的service。
2.消息中间件

20、springcloud如何实现服务的注册?

1.服务发布时，指定对应的服务名,将服务注册到 注册中心(eureka zookeeper)
2.注册中心加@EnableEurekaServer,服务用@EnableDiscoveryClient，然后用ribbon或feign进行服务直接的调用发现。

21、Eureka和Zookeeper区别

1.Eureka取CAP的AP，注重可用性，Zookeeper取CAP的CP注重一致性。
2.Zookeeper在选举期间注册服务瘫痪，虽然服务最终会恢复，但选举期间不可用。
3.eureka的自我保护机制，会导致一个结果就是不会再从注册列表移除因长时间没收到心跳而过期的服务。依然能接受新服务的注册和查询请求，但不会被同步到其他节点。不会服务瘫痪。
4.Zookeeper有Leader和Follower角色，Eureka各个节点平等。
5.Zookeeper采用过半数存活原则，Eureka采用自我保护机制解决分区问题。
6.eureka本质是一个工程，Zookeeper只是一个进程。

22、eureka自我保护机制是什么?

1.当Eureka Server 节点在短时间内丢失了过多实例的连接时（比如网络故障或频繁启动关闭客户端）节点会进入自我保护模式，保护注册信息，不再删除注册数据，故障恢复时，自动退出自我保护模式。

23、什么是Ribbon？

ribbon是一个负载均衡客户端，可以很好的控制htt和tcp的一些行为。feign默认集成了ribbon。

24、什么是feigin？它的优点是什么？

1.feign采用的是基于接口的注解
2.feign整合了ribbon，具有负载均衡的能力
3.整合了Hystrix，具有熔断的能力

使用:
1.添加pom依赖。
2.启动类添加@EnableFeignClients
3.定义一个接口@FeignClient(name=“xxx”)指定调用哪个服务

25、Ribbon和Feign的区别？

1.Ribbon都是调用其他服务的，但方式不同。
2.启动类注解不同，Ribbon是@RibbonClient feign的是@EnableFeignClients
3.服务指定的位置不同，Ribbon是在@RibbonClient注解上声明，Feign则是在定义抽象方法的接口中使用@FeignClient声明。
4.调用方式不同，Ribbon需要自己构建http请求，模拟http请求然后使用RestTemplate发送给其他服务，步骤相当繁琐。Feign需要将调用的方法定义成抽象方法即可。

26、什么是Spring Cloud Bus?

spring cloud bus 将分布式的节点用轻量的消息代理连接起来，它可以用于广播配置文件的更改或者服务直接的通讯，也可用于监控。
如果修改了配置文件，发送一次请求，所有的客户端便会重新读取配置文件。
使用:
1.添加依赖
2.配置rabbimq

27、springcloud断路器作用?

当一个服务调用另一个服务由于网络原因或自身原因出现问题，调用者就会等待被调用者的响应 当更多的服务请求到这些资源导致更多的请求等待，发生连锁效应（雪崩效应）
断路器有完全打开状态:一段时间内 达到一定的次数无法调用 并且多次监测没有恢复的迹象 断路器完全打开 那么下次请求就不会请求到该服务
半开:短时间内 有恢复迹象 断路器会将部分请求发给该服务，正常调用时 断路器关闭
关闭：当服务一直处于正常状态 能正常调用

28、Spring Cloud Gateway?

Spring Cloud Gateway是Spring Cloud官方推出的第二代网关框架，取代Zuul网关。网关作为流量的，在微服务系统中有着非常作用，网关常见的功能有路由转发、权限校验、限流控制等作用。

使用了一个RouteLocatorBuilder的bean去创建路由，除了创建路由RouteLocatorBuilder可以让你添加各种predicates和filters，predicates断言的意思，顾名思义就是根据具体的请求的规则，由具体的route去处理，filters是各种过滤器，用来对请求做各种判断和修改。

29、作为服务注册中心，Eureka比Zookeeper好在哪里?

1.Eureka保证的是可用性和分区容错性，Zookeeper 保证的是一致性和分区容错性 。
2.Eureka还有一种自我保护机制，如果在15分钟内超过85%的节点都没有正常的心跳，那么Eureka就认为客户端与注册中心出现了网络故障。而不会像zookeeper那样使整个注册服务瘫痪。

30、什么是 Ribbon负载均衡？

1.Spring Cloud Ribbon是基于Netflix Ribbon实现的一套客户端 负载均衡的工具。

2. Ribbon客户端组件提供一系列完善的配置项如连接超时，重试等。简单的说，就是在配置文件中列出Load Balancer（简称LB）后面所有的机器，Ribbon会自动的帮助你基于某种规则（如简单轮询，随机连接等）去连接这些机器。我们也很容易使用Ribbon实现自定义的负载均衡算法。

31、Ribbon负载均衡能干什么？

1.将用户的请求平摊的分配到多个服务上
2.集中式LB即在服务的消费方和提供方之间使用独立的LB设施(可以是硬件，如F5, 也可以是软件，如nginx), 由该设施负责把访问请求通过某种策略转发至服务的提供方；
3.进程内LB将LB逻辑集成到消费方，消费方从服务注册中心获知有哪些地址可用，然后自己再从这些地址中选择出一个合适的服务器。

注意： Ribbon就属于进程内LB，它只是一个类库，集成于消费方进程，消费方通过它来获取到服务提供方的地址。

32、什么是 zuul路由网关

1.Zuul 包含了对请求的路由和过滤两个最主要的功能:其中路由功能负责将外部请求转发到具体的微服务实例上，是实现外部访问统一入口的基础而过滤器功能则负责对请求的处理过程进行干预，是实现请求校验、服务聚合等功能的基础、
2.Zuul和Eureka进行整合，将Zuul自身注册为Eureka服务治理下的应用，同时从Eureka中获得其他微服务的消息，也即以后的访问微服务都是通过Zuul跳转后获得。

> 注意： Zuul服务最终还是会注册进Eureka 提供=代理+路由+过滤 三大功能。

33、分布式配置中心能干嘛？

1.集中管理配置文件不同环境不同配置，动态化的配置更新，分环境部署比如dev/test/prod/beta/release
2.运行期间动态调整配置，不再需要在每个服务部署的机器上编写配置文件，服务会向配置中心统一拉取配置自己的信息
3.当配置发生变动时，服务不需要重启即可感知到配置的变化并应用新的配置将配置信息以REST接口的形式暴露

34、Hystrix相关注解

@EnableHystrix：开启熔断
@HystrixCommand(fallbackMethod=”XXX”)：声明一个失败回滚处理函数XXX，当被注解的方法执行超时（默认是1000毫秒），就会执行fallback函数，返回错误提示。

微服务条目	落地的技术
服务开发	SpringBoot、Spring、SpringMVC
服务配置管理	Netflix公司的Archaius、阿里的Diamond等
服务注册与发现	Eureka、Consul、Zookeeper
服务调用	RPC、Rest、gRPC
服务熔断器	Hystrix、Envoy等
负载均衡	Nginx、Ribbon
服务接口调用（客户端调用服务的简化工具）	Feign
消息队列	Kafka、RabbitMQ、ActiveMQ等
服务配置中心配置管理	SpringCloudConfig、Chef等
服务路由（API网关）	Zuul
服务监控	Zabbix、Naggios、Metrics、Spectator等
全链路追踪	Zipkin、Brave、Dapper等
服务部署	Docker、OpenStack、Kubernetes等
数据流操作开发包	SpringCloud Stream
事件消息总线	Spring Cloud Bus

35、Eureka和zookeeper都可以提供服务注册与发现的功能，请说说两个的区别？

Zookeeper保证了CP（C：一致性，P：分区容错性），Eureka保证了AP（A：高可用）

1.当向注册中心查询服务列表时，我们可以容忍注册中心返回的是几分钟以前的信息，但不能容忍直接down掉不可用。也就是说，服务注册功能对高可用性要求比较高，但zk会出现这样一种情况，当master节点因为网络故障与其他节点失去联系时，剩余节点会重新选leader。问题在于，选取leader时间过长，30 ~ 120s，且选取期间zk集群都不可用，这样就会导致选取期间注册服务瘫痪。在云部署的环境下，因网络问题使得zk集群失去master节点是较大概率会发生的事，虽然服务能够恢复，但是漫长的选取时间导致的注册长期不可用是不能容忍的。

2.Eureka保证了可用性，Eureka各个节点是平等的，几个节点挂掉不会影响正常节点的工作，剩余的节点仍然可以提供注册和查询服务。而Eureka的客户端向某个Eureka注册或发现时发生连接失败，则会自动切换到其他节点，只要有一台Eureka还在，就能保证注册服务可用，只是查到的信息可能不是最新的。除此之外，Eureka还有自我保护机制，如果在15分钟内超过85%的节点没有正常的心跳，那么Eureka就认为客户端与注册中心发生了网络故障，此时会出现以下几种情况：

①、Eureka不在从注册列表中移除因为长时间没有收到心跳而应该过期的服务。

②、Eureka仍然能够接受新服务的注册和查询请求，但是不会被同步到其他节点上（即保证当前节点仍然可用）

③、当网络稳定时，当前实例新的注册信息会被同步到其他节点。

因此，Eureka可以很好的应对因网络故障导致部分节点失去联系的情况，而不会像Zookeeper那样使整个微服务瘫痪。

RabbitMQ面试题

1、什么是 rabbitmq

采用 AMQP 高级消息队列协议的一种消息队列技术,最大的特点就是消费并不需要确保提供方存在,实现了服务之间的高度解耦

2、为什么要使用 rabbitmq

1、在分布式系统下具备异步,削峰,负载均衡等一系列高级功能;

2、拥有持久化的机制，进程消息，队列中的信息也可以保存下来。

3、实现消费者和生产者之间的解耦。

4、对于高并发场景下，利用消息队列可以使得同步访问变为串行访问达到一定量的限流，利于数据库的操作。

5.可以使用消息队列达到异步下单的效果，排队中，后台进行逻辑下单。

3、使用 rabbitmq 的场景

1、服务间异步通信
2、顺序消费
3、定时任务
4、请求削峰

4、如何确保消息正确地发送至 RabbitMQ？ 如何确保消息接收方消费了消息？

发送方确认模式
将信道设置成 confirm 模式（发送方确认模式），则所有在信道上发布的消息都会被指派一个唯一的 ID。
一旦消息被投递到目的队列后，或者消息被写入磁盘后（可持久化的消息），信道会发送一个确认给生产者（包含消息唯一 ID）。
如果 RabbitMQ 发生内部错误从而导致消息丢失，会发送一条 nack（notacknowledged，未确认）消息。发送方确认模式是异步的，生产者应用程序在等待确认的同时，可以继续发送消息。当确认消息到达生产者应用程序，生产者应用程序的回调方法就会被触发来处理确认消息。

接收方确认机制
接收方消息确认机制
消费者接收每一条消息后都必须进行确认（消息接收和消息确认是两个不同操作）。只有消费者确认了消息，RabbitMQ 才能安全地把消息从队列中删除。这里并没有用到超时机制，RabbitMQ 仅通过 Consumer 的连接中断来确认是否需要重新发送消息。也就是说，只要连接不中断，RabbitMQ 给了 Consumer 足够长的时间来处理消息。保证数据的最终一致性；

下面罗列几种特殊情况

如果消费者接收到消息，在确认之前断开了连接或取消订阅，RabbitMQ 会认为消息没有被分发，然后重新分发给下一个订阅的消费者。（可能存在消息重复消费的隐患，需要去重）如果消费者接收到消息却没有确认消息，连接也未断开，则 RabbitMQ 认为该消费者繁忙，将不会给该消费者分发更多的消息。

5、如何避免消息重复投递或重复消费？

在消息生产时，MQ 内部针对每条生产者发送的消息生成一个 inner-msg-id，作为去重的依据（消息投递失败并重传），避免重复的消息进入队列；

在消息消费时，要求消息体中必须要有一个 bizId（对于同一业务全局唯一，如支付 ID、订单 ID、帖子 ID 等）作为去重的依据，避免同一条消息被重复消费。

6、消息基于什么传输？

由于 TCP 连接的创建和销毁开销较大，且并发数受系统资源限制，会造成性能瓶颈。RabbitMQ 使用信道的方式来传输数据。信道是建立在真实的 TCP 连接内的虚拟连接，且每条 TCP 连接上的信道数量没有限制

7、消息如何分发？

若该队列至少有一个消费者订阅，消息将以循环（round-robin）的方式发送给消费者。每条消息只会分发给一个订阅的消费者（前提是消费者能够正常处理消息并进行确认）。
通过路由可实现多消费的功能

8、消息怎么路由？

消息提供方->路由->一至多个队列
消息发布到交换器时，消息将拥有一个路由键（routing key），在消息创建时设定。

通过队列路由键，可以把队列绑定到交换器上。
消息到达交换器后，RabbitMQ 会将消息的路由键与队列的路由键进行匹配（针对不同的交换器有不同的路由规则）；
常用的交换器主要分为一下三种
fanout：如果交换器收到消息，将会广播到所有绑定的队列上
direct：如果路由键完全匹配，消息就被投递到相应的队列
topic：可以使来自不同源头的消息能够到达同一个队列。 使用 topic 交换器时，可以使用通配符

9、如何确保消息不丢失？

消息持久化，当然前提是队列必须持久化
RabbitMQ 确保持久性消息能从服务器重启中恢复的方式是，将它们写入磁盘上的一个持久化日志文件，当发布一条持久性消息到持久交换器上时，Rabbit 会在消息提交到日志文件后才发送响应。

一旦消费者从持久队列中消费了一条持久化消息，RabbitMQ 会在持久化日志中把这条消息标记为等待垃圾收集。如果持久化消息在被消费之前 RabbitMQ 重启，那么 Rabbit 会自动重建交换器和队列（以及绑定），并重新发布持久化日志文件中的消息到合适的队列。

10、使用 RabbitMQ 有什么好处？

1、服务间高度解耦
2、异步通信性能高
3、流量削峰

11、RabbitMQ 的集群

镜像集群模式
你创建的 queue，无论元数据还是 queue 里的消息都会存在于多个实例上，然后每次你写消息到 queue 的时候，都会自动把消息到多个实例的 queue 里进行消息同步。

好处在于，你任何一个机器宕机了，没事儿，别的机器都可以用。坏处在于，第一，这个性能开销也太大了吧，消息同步所有机器，导致网络带宽压力和消耗很重！第二，这么玩儿，就没有扩展性可言了，如果某个 queue 负载很重，你加机器，新增的机器也包含了这个 queue 的所有数据，并没有办法线性扩展你的 queue

12、mq 的缺点

系统可用性降低
系统引入的外部依赖越多，越容易挂掉，本来你就是 A 系统调用 BCD 三个系统的接口就好了，人 ABCD 四个系统好好的，没啥问题，你偏加个 MQ 进来，万一MQ 挂了咋整？MQ 挂了，整套系统崩溃了，你不就完了么。

系统复杂性提高
硬生生加个 MQ 进来，你怎么保证消息没有重复消费？怎么处理消息丢失的情况？怎么保证消息传递的顺序性？头大头大，问题一大堆，痛苦不已

一致性问题
A 系统处理完了直接返回成功了，人都以为你这个请求就成功了；但是问题是，要是 BCD 三个系统那里，BD 两个系统写库成功了，结果 C 系统写库失败了，咋整？你这数据就不一致了。

所以消息队列实际是一种非常复杂的架构，你引入它有很多好处，但是也得针对它带来的坏处做各种额外的技术方案和架构来规避掉，最好之后，你会发现，妈呀，系统复杂度提升了一个数量级，也许是复杂了 10 倍。但是关键时刻，用，还是得用的

13、Kafka、ActiveMQ、RabbitMQ、RocketMQ 都有什么区别？

对于吞吐量来说kafka和RocketMQ支撑高吞吐，ActiveMQ和RabbitMQ比他们低一个数量级。对于延迟量来说RabbitMQ是最低的。

1.从社区活跃度
按照目前网络上的资料，RabbitMQ 、activeM 、ZeroMQ 三者中，综合来看，RabbitMQ 是首选。

2.持久化消息比较

ActiveMq 和RabbitMq 都支持。持久化消息主要是指我们机器在不可抗力因素等情况下挂掉了，消息不会丢失的机制。

3.综合技术实现
可靠性、灵活的路由、集群、事务、高可用的队列、消息排序、问题追踪、可视化管理工具、插件系统等等。
RabbitMq / Kafka 最好，ActiveMq 次之，ZeroMq 最差。当然ZeroMq 也可以做到，不过自己必须手动写代码实现，代码量不小。尤其是可靠性中的：持久性、投递确认、发布者证实和高可用性。

4.高并发
毋庸置疑，RabbitMQ 最高，原因是它的实现语言是天生具备高并发高可用的erlang 语言。

5.比较关注的比较， RabbitMQ 和 Kafka
RabbitMq 比Kafka 成熟，在可用性上，稳定性上，可靠性上， RabbitMq 胜于 Kafka （理论上）。另外，Kafka 的定位主要在日志等方面， 因为Kafka 设计的初衷就是处理日志的，可以看做是一个日志（消息）系统一个重要组件，针对性很强，所以 如果业务方面还是建议选择 RabbitMq 。还有就是，Kafka 的性能（吞吐量、TPS ）比RabbitMq 要高出来很多

14、如何保证高可用的？

RabbitMQ 是比较有代表性的，因为是基于主从（非分布式）做高可用性的，我们就以 RabbitMQ 为例子讲解第一种 MQ 的高可用性怎么实现。RabbitMQ 有三种模式：单机模式、普通集群模式、镜像集群模式。
单机模式，就是 Demo 级别的，一般就是你本地启动了玩玩儿的?，没人生产用单机模式普通集群模式，意思就是在多台机器上启动多个 RabbitMQ 实例，每个机器启动一个。你创建的queue，只会放在一个 RabbitMQ 实例上，但是每个实例都同步 queue 的元数据（元数据可以认为是queue 的一些配置信息，通过元数据，可以找到 queue 所在实例）。你消费的时候，实际上如果连接到了另外一个实例，那么那个实例会从 queue 所在实例上拉取数据过来。这方案主要是提高吞吐量的，就是说让集群中多个节点来服务某个 queue 的读写操作。

镜像集群模式：这种模式，才是所谓的 RabbitMQ 的高可用模式。跟普通集群模式不一样的是，在镜像集群模式下，你创建的 queue，无论元数据还是 queue 里的消息都会存在于多个实例上，就是说，每个 RabbitMQ 节点都有这个 queue 的一个完整镜像，包含 queue 的全部数据的意思。然后每次你写消息到 queue 的时候，都会自动把消息同步到多个实例的 queue 上。RabbitMQ 有很好的管理控制台，就是在后台新增一个策略，这个策略是镜像集群模式的策略，指定的时候是可以要求数据同步到所有节点的，也可以要求同步到指定数量的节点，再次创建 queue 的时候，应用这个策略，就会自动将数据同步到其他的节点上去了。这样的话，好处在于，你任何一个机器宕机了，没事儿，其它机器（节点）还包含了这个 queue 的完整数据，别的 consumer 都可以到其它节点上去消费数据。坏处在于，第一，这个性能开销也太大了吧，消息需要同步到所有机器上，导致网络带宽压力和消耗很重！RabbitMQ 一个 queue 的数据都是放在一个节点里的，镜像集群下，也是每个节点都放这个 queue 的完整数据

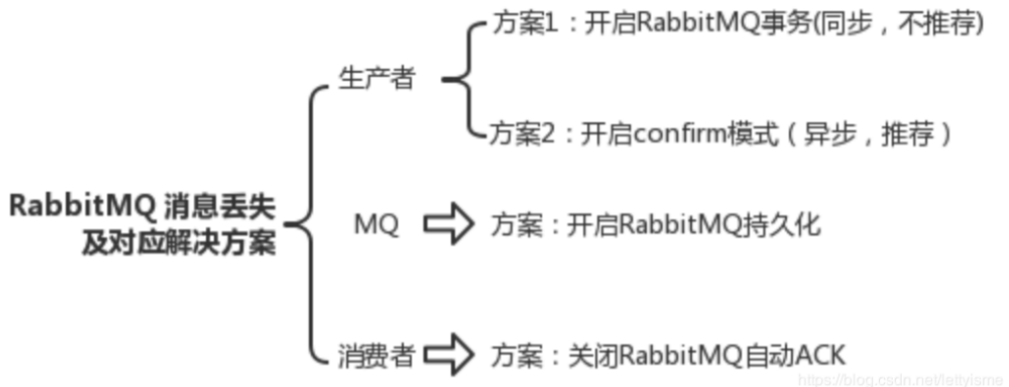
Kafka 一个最基本的架构认识：由多个 broker 组成，每个 broker 是一个节点；你创建一个 topic，这个 topic 可以划分为多个 partition，每个 partition 可以存在于不同的 broker 上，每个 partition 就放一部分数据。这就是天然的分布式消息队列，就是说一个 topic 的数据，是分散放在多个机器上的，每个机器就放一部分数据。Kafka 0.8 以后，提供了 HA 机制，就是 replica（复制品）副本机制。每个partition 的数据都会同步到其它机器上，形成自己的多个 replica 副本。所有 replica 会选举一个leader 出来，那么生产和消费都跟这个 leader 打交道，然后其他 replica 就是 follower。写的时候，leader 会负责把数据同步到所有 follower 上去，读的时候就直接读 leader 上的数据即可。只能读写leader？很简单，要是你可以随意读写每个 follower，那么就要 care 数据一致性的问题，系统复杂度太高，很容易出问题。Kafka 会均匀地将一个 partition 的所有 replica 分布在不同的机器上，这样才可以提高容错性。因为如果某个 broker 宕机了，没事儿，那个 broker上面的 partition 在其他机器上都
有副本的，如果这上面有某个 partition 的 leader，那么此时会从 follower 中重新选举一个新的 leader出来，大家继续读写那个新的 leader 即可。这就有所谓的高可用性了。写数据的时候，生产者就写leader，然后 leader 将数据落地写本地磁盘，接着其他 follower 自己主动从 leader 来 pull 数据。一旦所有 follower 同步好数据了，就会发送 ack 给 leader，leader 收到所有 follower 的 ack 之后，就会返回写成功的消息给生产者。（当然，这只是其中一种模式，还可以适当调整这个行为）消费的时候，只会从 leader 去读，但是只有当一个消息已经被所有 follower 都同步成功返回 ack 的时候，这个消息才会被消费者读到

15、如何保证消息的可靠传输？如果消息丢了怎么办

数据的丢失问题，可能出现在生产者、MQ、消费者中
生产者丢失：生产者将数据发送到 RabbitMQ 的时候，可能数据就在半路给搞丢了，因为网络问题啥的，都有可能。此时可以选择用 RabbitMQ 提供的事务功能，就是生产者发送数据之前开启 RabbitMQ事务channel.txSelect，然后发送消息，如果消息没有成功被 RabbitMQ 接收到，那么生产者会收到异常报错，此时就可以回滚事务channel.txRollback，然后重试发送消息；如果收到了消息，那么可以提交事务channel.txCommit。吞吐量会下来，因为太耗性能。所以一般来说，如果你要确保说写RabbitMQ 的消息别丢，可以开启 confirm模式，在生产者那里设置开启confirm模式之后，你每次写的消息都会分配一个唯一的 id，然后如果写入了 RabbitMQ 中，RabbitMQ 会给你回传一个ack消息，告诉你说这个消息 ok 了。如果 RabbitMQ 没能处理这个消息，会回调你一个nack接口，告诉你这个消息接收失败，你可以重试。而且你可以结合这个机制自己在内存里维护每个消息 id 的状态，如果超过一定时间还没接收到这个消息的回调，那么你可以重发。事务机制和cnofirm机制最大的不同在于，事务机制是同步的，你提交一个事务之后会阻塞在那儿，但是confirm机制是异步的，你发送个消息之后就可以发送下一个消息，然后那个消息RabbitMQ 接收了之后会异步回调你一个接口通知你这个消息接收到了。所以一般在生产者这块避免数据丢失，都是用confirm机制的

MQ中丢失：就是 RabbitMQ 自己弄丢了数据，这个你必须开启 RabbitMQ 的持久化，就是消息写入之后会持久化到磁盘，哪怕是 RabbitMQ 自己挂了，恢复之后会自动读取之前存储的数据，一般数据不会丢。设置持久化有两个步骤：创建 queue 的时候将其设置为持久化，这样就可以保证 RabbitMQ 持久化 queue 的元数据，但是不会持久化 queue 里的数据。第二个是发送消息的时候将消息的 deliveryMode 设置为 2，就是将消息设置为持久化的，此时 RabbitMQ 就会将消息持久化到磁盘上去。必须要同时设置这两个持久化才行，RabbitMQ 哪怕是挂了，再次重启，也会从磁盘上重启恢复queue，恢复这个 queue 里的数据。持久化可以跟生产者那边的confirm机制配合起来，只有消息被持久化到磁盘之后，才会通知生产者ack了，所以哪怕是在持久化到磁盘之前，RabbitMQ 挂了，数据丢了，生产者收不到ack，你也是可以自己重发的。注意，哪怕是你给 RabbitMQ 开启了持久化机制，也有一种可能，就是这个消息写到了 RabbitMQ 中，但是还没来得及持久化到磁盘上，结果不巧，此时RabbitMQ 挂了，就会导致内存里的一点点数据丢失

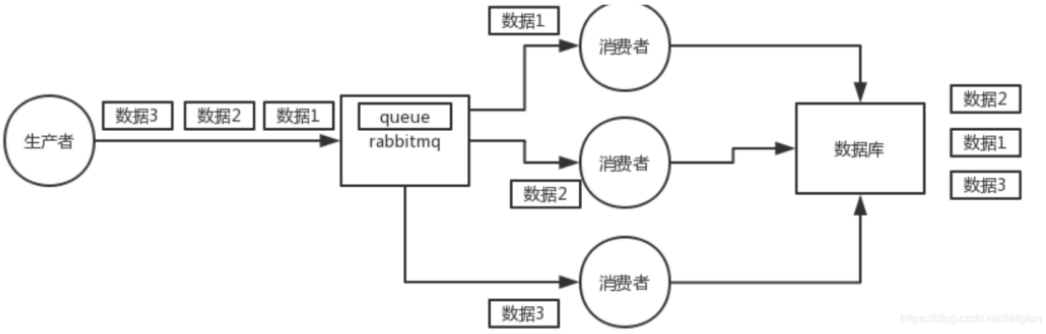
消费端丢失：你消费的时候，刚消费到，还没处理，结果进程挂了，比如重启了，那么就尴尬了，RabbitMQ 认为你都消费了，这数据就丢了。这个时候得用 RabbitMQ 提供的ack机制，简单来说，就是你关闭 RabbitMQ 的自动ack，可以通过一个 api 来调用就行，然后每次你自己代码里确保处理完的时候，再在程序里ack一把。这样的话，如果你还没处理完，不就没有ack？那 RabbitMQ 就认为你还没处理完，这个时候 RabbitMQ 会把这个消费分配给别的 consumer 去处理，消息是不会丢的



https://blog.csdn.net/lettylsme

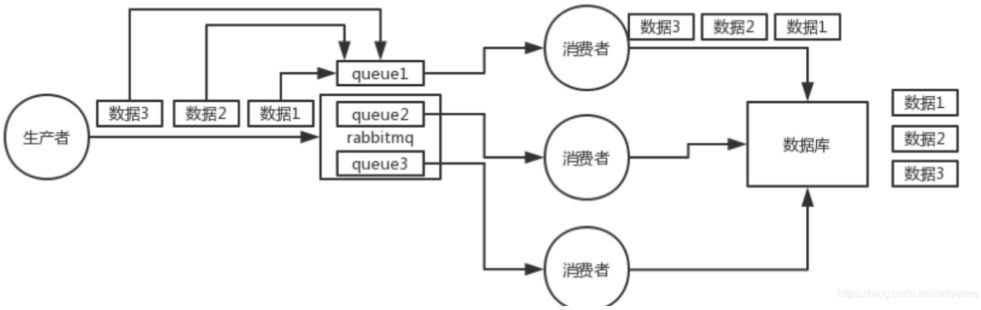
16、如何保证消息的顺序性

先看看顺序会错乱的场景：RabbitMQ：一个 queue，多个 consumer，这不明显乱了；



解决：

拆分多个 queue，每个 queue 一个 consumer，就是多一些 queue 而已，确实是麻烦点；或者就一个 queue 但是对应一个 consumer，然后这个 consumer 内部用内存队列做排队，然后分发给底层不同的 worker 来处理。



17、如何解决消息队列的延时以及过期失效问题？消息队列满了以后该怎么处理？有几百万消息持续积压几小时，说说怎么解决

消息积压处理办法：临时紧急扩容：
先修复 consumer 的问题，确保其恢复消费速度，然后将现有 cnosumer 都停掉。新建一个 topic，partition 是原来的 10 倍，临时建立好原先 10 倍的 queue 数量。然后写一个临时的分发数据的 consumer 程序，这个程序部署上去消费积压的数据，消费之后不做耗时的处理，直接均匀轮询写入临时建立好的 10 倍数量的 queue。

接着临时征用 10 倍的机器来部署 consumer，每一批 consumer 消费一个临时 queue 的数据。这种做法相当于是临时将 queue 资源和 consumer 资源扩大 10 倍，以正常的 10 倍速度来消费数据。等快速消费完积压数据之后，得恢复原先部署的架构，重新用原先的 consumer 机器来消费消息。MQ中消息失效：假设你用的是 RabbitMQ，RabbtiMQ 是可以设置过期时间的，也就是 TTL。如果消息在 queue 中积压超过一定的时间就会被 RabbitMQ 给清理掉，这个数据就没了。那这就是第二个坑了。这就不是说数据会大量积压在 mq 里，而是大量的数据会直接搞丢。我们可以采取一个方案，就是批量重导，这个我们之前线上也有类似的场景干过。就是大量积压的时候，我们当时就直接丢弃数据了，然后等过了高峰期以后，比如大家一起喝咖啡熬夜到晚上12点以后，用户都睡觉了。这个时候我们就开始写程序，将丢失的那批数据，写个临时程序，一点一点的查出来，然后重新灌入 mq 里面去，把白天丢的数据给他补回来。也只能是这样了。假设 1 万个订单积压在 mq 里面，没有处理，其中 1000个订单都丢了，你只能手动写程序把那 1000 个订单给查出来，手动发到 mq 里去再补一次

mq消息队列块满了：如果消息积压在 mq 里，你很长时间都没有处理掉，此时导致 mq 都快写满了，咋办？这个还有别的办法吗？没有，谁让你第一个方案执行的太慢了，你临时写程序，接入数据来消费，消费一个丢弃一个，都不要了，快速消费掉所有的消息。然后走第二个方案，到了晚上再补数据吧

18、设计MQ的思路

比如说这个消息队列系统，我们从以下几个角度来考虑一下：
首先这个 mq 得支持可伸缩性吧，就是需要的时候快速扩容，就可以增加吞吐量和容量，那怎么搞？设计个分布式的系统呗，参照一下 kafka 的设计理念，broker -> topic -> partition，每个 partition 放一个机器，就存一部分数据。如果现在资源不够了，简单啊，给 topic 增加 partition，然后做数据迁移，增加机器，不就可以存放更多数据，提供更高的吞吐量了？

其次你得考虑一下这个 mq 的数据要不要落地磁盘吧？那肯定要了，落磁盘才能保证别进程挂了数据就丢了。那落磁盘的时候怎么落啊？顺序写，这样就没有磁盘随机读写的寻址开销，磁盘顺序读写的性能是很高的，这就是 kafka 的思路。

其次你考虑一下你的 mq 的可用性啊？这个事儿，具体参考之前可用性那个环节讲解的 kafka 的高可用保障机制。多副本 -> leader & follower -> broker 挂了重新选举 leader 即可对外服务。能不能支持数据 0 丢失啊？可以的，参考我们之前说的那个 kafka 数据零丢失方案

19、什么是Message？

消息，消息是不具名的，它由消息头和消息体组成。消息体是不透明的，而消息头则由一系列的可选属性组成，这些属性包括 routing-key（路由键）、 priority（相对于其他消息的优先权）、 delivery-mode（指出该消息可能需要持久性存储）等。

20、什么是Publisher ？

消息的生产者，也是一个向交换器发布消息的客户端应用程序。

21、什么是Exchange（将消息路由给队列 ）

交换器，用来接收生产者发送的消息并将这些消息路由给服务器中的队列

22、什么是Binding（消息队列和交换器之间的关联）

绑定，用于消息队列和交换器之间的关联。一个绑定就是基于路由键将交换器和消息队列连接起来的路由规则，所以可以将交换器理解成一个由绑定构成的路由表

23、什么是Queue？

消息队列，用来保存消息直到发送给消费者。它是消息的容器，也是消息的终点。 一个消息可投入一个或多个队列。消息一直在队列里面，等待消费者连接到这个队列将其取走

24、什么是Connection ？

网络连接，比如一个 TCP 连接。

25、什么是Channel？

信道， 多路复用连接中的一条独立的双向数据流通道。信道是建立在真实的 TCP 连接内地虚拟连接， AMQP 命令都是通过信道发出去的，不管是发布消息、订阅队列还是接收消息，这些动作都是通过信道完成。因为对于操作系统来说建立和销毁 TCP 都是非常昂贵的开销，所以引入了信道的概念，以复用一条 TCP 连接

26、什么是Consumer ？

消息的消费者，表示一个从消息队列中取得消息的客户端应用程序

27、什么是Virtual Host ？

虚拟主机，表示一批交换器、消息队列和相关对象。虚拟主机是共享相同的身份认证和加密环境的独立服务器域。

28、什么是Broker？

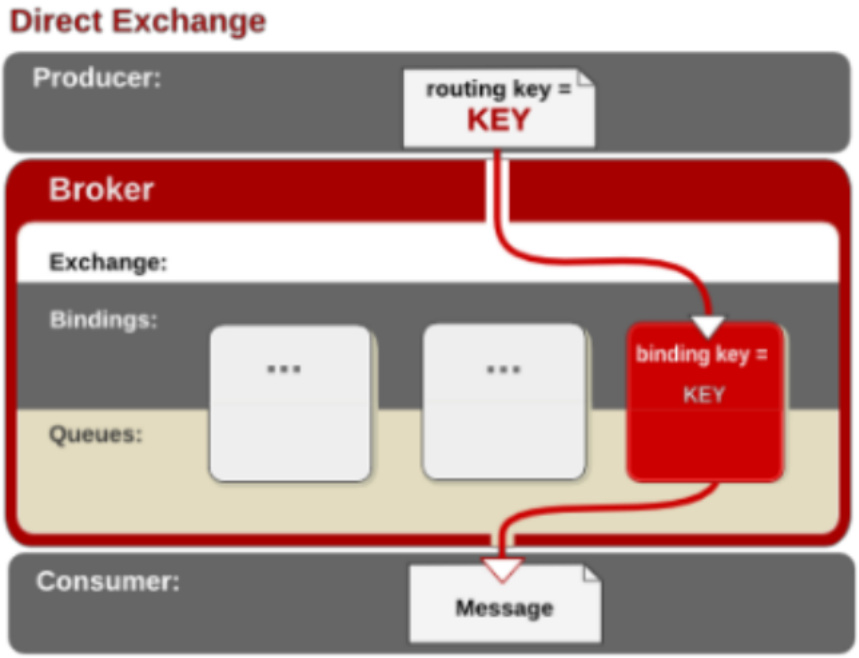
表示消息队列服务器实体

29、Exchange 类型 ？

Exchange 分发消息时根据类型的不同分发策略有区别， 目前共四种类型： direct、 fanout、topic、 headers 。 headers 匹配 AMQP 消息的 header 而不是路由键，此外 headers 交换器和direct 交换器完全一致，但性能差很多，目前几乎用不到了。

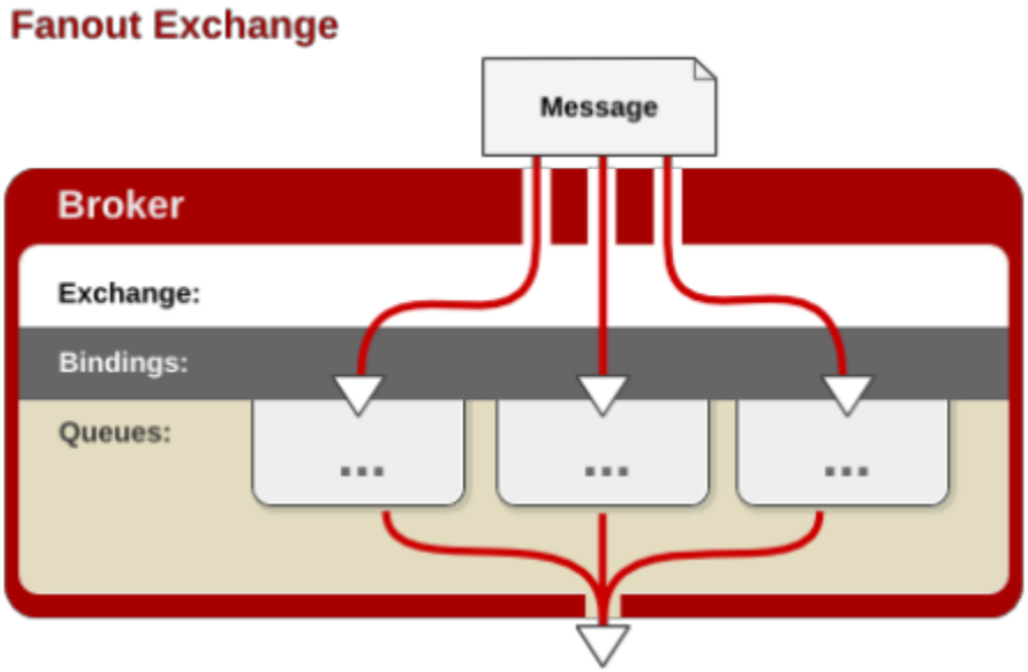
30、Direct 键（routing key）分布 ？

Direct： 消息中的路由键（routing key）如果和 Binding 中的 binding key 一致，交换器就将消息发到对应的队列中。它是完全匹配、单播的模式。



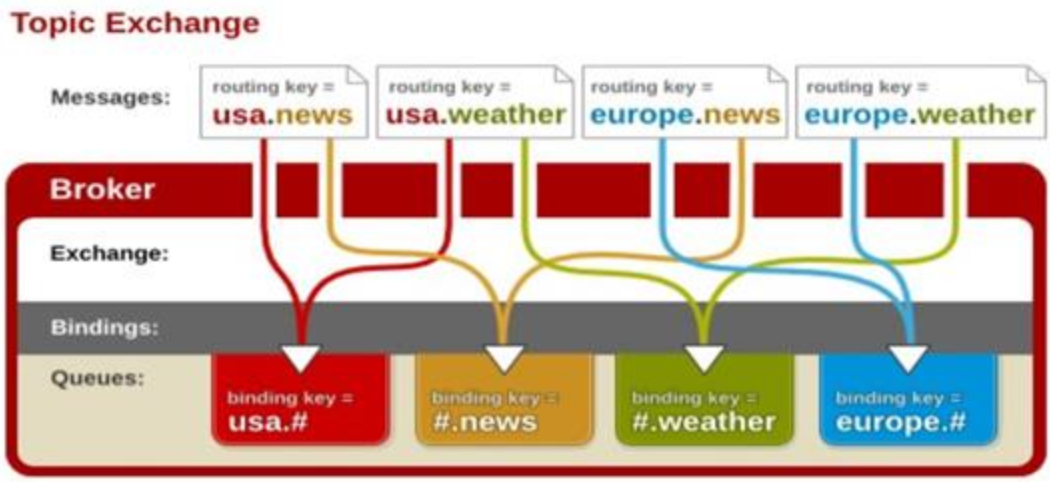
31、Fanout（广播分发）？

Fanout： 每个发到 fanout 类型交换器的消息都会分到所有绑定的队列上去。很像子网广播，每台子网内的主机都获得了一份复制的消息。fanout 类型转发消息是最快的。



32、topic 交换器（模式匹配）？

topic 交换器： topic 交换器通过模式匹配分配消息的路由键属性，将路由键和某个模式进行匹配，此时队列需要绑定到一个模式上。它将路由键和绑定键的字符串切分成单词，这些单词之间用点隔开。它同样也会识别两个通配符：符号“#”和符号“”。 #匹配 0 个或多个单词，匹配不多不少一个单词。



Dubbo 面试题

1、为什么要用 Dubbo？

随着服务化的进一步发展，服务越来越多，服务之间的调用和依赖关系也越来越复杂，诞生了面向服务的架构体系(SOA)，也因此衍生出了一系列相应的技术，如对服务提供、服务调用、连接处理、通信协议、序列化方式、服务发现、服务路由、日志输出等行为进行封装的服务框架。就这样为分布式系统的服务治理框架就出现了，Dubbo 也就这样产生了。

2、Dubbo 的整体架构设计有哪些分层?

接口服务层（Service）：该层与业务逻辑相关，根据 provider 和 consumer 的业务设计对应的接口和实现

配置层（Config）：对外配置接口，以 ServiceConfig 和 ReferenceConfig 为中心

服务代理层（Proxy）：服务接口透明代理，生成服务的客户端 Stub 和 服务端的 Skeleton，以 ServiceProxy 为中心，扩展接口为 ProxyFactory

服务注册层（Registry）：封装服务地址的注册和发现，以服务 URL 为中心，扩展接口为 RegistryFactory、Registry、RegistryService

路由层（Cluster）：封装多个提供者的路由和负载均衡，并桥接注册中心，以Invoker 为中心，扩展接口为 Cluster、Directory、Router 和 LoadBlancce

监控层（Monitor）：RPC 调用次数和调用时间监控，以 Statistics 为中心，扩展接口为 MonitorFactory、Monitor 和 MonitorService

远程调用层（Protocal）：封装 RPC 调用，以 Invocation 和 Result 为中心，扩展接口为 Protocal、Invoker 和 Exporter

信息交换层（Exchange）：封装请求响应模式，同步转异步。以 Request 和Response 为中心，扩展接口为 Exchanger、ExchangeChannel、ExchangeClient 和 ExchangeServer

网络传输层（Transport）：抽象 mina 和 netty 为统一接口，以 Message 为中心，扩展接口为 Channel、Transporter、Client、Server 和 Codec

数据序列化层（Serialize）：可复用的一些工具，扩展接口为 Serialization、ObjectInput、ObjectOutput 和 ThreadPool

3、默认使用的是什么通信框架，还有别的选择吗?

默认也推荐使用 netty 框架，还有 mina

4、服务调用是阻塞的吗？

默认是阻塞的，可以异步调用，没有返回值的可以这么做。
Dubbo 是基于 NIO 的非阻塞实现并行调用，客户端不需要启动多线程即可完成并行调用多个远程服务，相对多线程开销较小，异步调用会返回一个 Future 对象。

5、一般使用什么注册中心？还有别的选择吗？

推荐使用 Zookeeper 作为注册中心，还有 Redis、Multicast、Simple 注册中心，但不推荐。

6、默认使用什么序列化框架，你知道的还有哪些？

推荐使用 Hessian 序列化，还有 Duddo、FastJson、Java 自带序列化。

7、服务提供者能实现失效踢出是什么原理？

服务失效踢出基于 zookeeper 的临时节点原理。

8、服务上线怎么不影响旧版本？

采用多版本开发，不影响旧版本。

9、如何解决服务调用链过长的问题？

可以结合 zipkin 实现分布式服务追踪。

10、说说核心的配置有哪些？

配置	配置说明
dubbo:service	服务配置
dubbo:reference	引用配置
dubbo:protocol	协议配置
dubbo:applicatio n	应用配置
dubbo:module	模块配置
dubbo:registry	注册中心配置
dubbo:monitor	监控中心配置
dubbo:provider	提供方配置
dubbo:consumer	消费方配置
dubbo:method	方法配置
dubbo:argument	参数配置

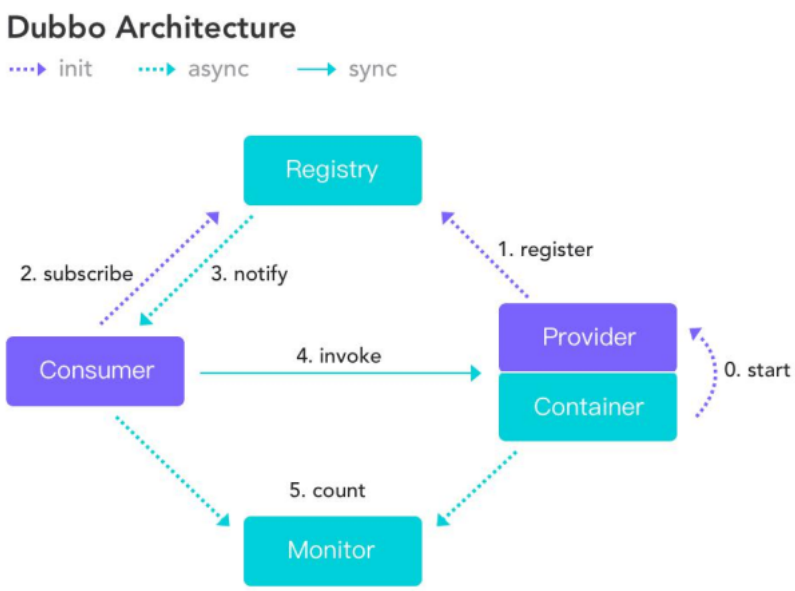
11、Dubbo 推荐用什么协议？

□ dubbo:// (推荐)
□ rmi://
□ hessian://
□ http://
□ webservice://
□ thrift://
□ memcached://
□ redis://
□ rest://

12、同一个服务多个注册的情况下可以直连某一个服务吗？

可以点对点直连，修改配置即可，也可以通过 telnet 直接某个服务。

13、画一画服务注册与发现的流程图？



14、Dubbo 集群容错有几种方案？

集群容错方案	说明
Failover Cluster	失败自动切换，自动重试其它服务器（默认）
Failfast Cluster	快速失败，立即报错，只发起一次调用
Failsafe Cluster	失败安全，出现异常时，直接忽略
Failback Cluster	失败自动恢复，记录失败请求，定时重发
Forking Cluster	并行调用多个服务器，只要一个成功即返回
Broadcast Cluster	广播逐个调用所有提供者，任意一个报错则报错

15、Dubbo 服务降级，失败重试怎么做？

可以通过 dubbo:reference 中设置 mock="return null"。mock 的值也可以修改为 true，然后再跟接口同一个路径下实现一个 Mock 类，命名规则是 “接口名称+Mock” 后缀。然后在 Mock 类里实现自己的降级逻辑

16、Dubbo 使用过程中都遇到了些什么问题？

在注册中心找不到对应的服务,检查 service 实现类是否添加了@service 注解无法连接到注册中心,检查配置文件中的对应的测试 ip 是否正确

17、Dubbo Monitor 实现原理？

Consumer 端在发起调用之前会先走 filter 链；provider 端在接收到请求时也是先走 filter 链，然后才进行真正的业务逻辑处理。
默认情况下，在 consumer 和 provider 的 filter 链中都会有 Monitorfilter。
1、MonitorFilter 向 DubboMonitor 发送数据
2、DubboMonitor 将数据进行聚合后（默认聚合 1min 中的统计数据）暂存到ConcurrentMap<Statistics, AtomicReference> statisticsMap，然后使用一个含有 3 个线程（线程名字：DubboMonitorSendTimer）的线程池每隔 1min 钟，调用 SimpleMonitorService 遍历发送 statisticsMap 中的统计数据，每发送完毕一个，就重置当前的 Statistics 的 AtomicReference
3、SimpleMonitorService 将这些聚合数据塞入 BlockingQueue queue 中（队列大写为 100000）

4、SimpleMonitorService 使用一个后台线程（线程名为：DubboMonitorAsyncWriteLogThread）将 queue 中的数据写入文件（该线程以死循环的形式来写）
5、SimpleMonitorService 还会使用一个含有 1 个线程（线程名字：DubboMonitorTimer）的线程池每隔 5min 钟，将文件中的统计数据画成图表

18、Dubbo 用到哪些设计模式？

Dubbo 框架在初始化和通信过程中使用了多种设计模式，可灵活控制类加载、权限控制等功能。

工厂模式
Provider 在 export 服务时，会调用 ServiceConfig 的 export 方法。ServiceConfig中有个字段：

```
private static final Protocol protocol =
ExtensionLoader.getExtensionLoader(Protocol.class).getAdaptiveExtension();
```

Dubbo 里有很多这种代码。这也是一种工厂模式，只是实现类的获取采用了 JDKSPI 的机制。这么实现的优点是可扩展性强，想要扩展实现，只需要在 classpath下增加个文件就可以了，代码零侵入。另外，像上面的 Adaptive 实现，可以做到调用时动态决定调用哪个实现，但是由于这种实现采用了动态代理，会造成代码调试比较麻烦，需要分析出实际调用的实现类。

装饰器模式
Dubbo 在启动和调用阶段都大量使用了装饰器模式。以 Provider 提供的调用链为例，具体的调用链代码是在 ProtocolFilterWrapper 的 buildInvokerChain 完成的，具体是将注解中含有 group=provider 的 Filter 实现，按照 order 排序，最后的调用顺序是：

Dubbo 里有很多这种代码。这也是一种工厂模式，只是实现类的获取采用了 JDKSPI 的机制。这么实现的优点是可扩展性强，想要扩展实现，只需要在 classpath下增加个文件就可以了，代码零侵入。另外，像上面的 Adaptive 实现，可以做到调用时动态决定调用哪个实现，但是由于这种实现采用了动态代理，会造成代码调试比较麻烦，需要分析出实际调用的实现类。

装饰器模式
Dubbo 在启动和调用阶段都大量使用了装饰器模式。以 Provider 提供的调用链为例，具体的调用链代码是在 ProtocolFilterWrapper 的 buildInvokerChain 完成的，具体是将注解中含有 group=provider 的 Filter 实现，按照 order 排序，最后的调用顺序是：

```
EchoFilter -> ClassLoaderFilter -> GenericFilter -> ContextFilter ->
ExecuteLimitFilter -> TraceFilter -> TimeoutFilter -> MonitorFilter ->
ExceptionFilter
```

更确切地说，这里是装饰器和责任链模式的混合使用。例如，EchoFilter 的作用是判断是否是回声测试请求，是的话直接返回内容，这是一种责任链的体现。而像ClassLoaderFilter 则只是在主功能上添加了功能，更改当前线程

的 ClassLoader，这是典型的装饰器模式。
观察者模式
Dubbo 的 Provider 启动时，需要与注册中心交互，先注册自己的服务，再订阅自己的服务，订阅时，采用了观察者模式，开启一个 listener。注册中心会每 5 秒定时检查是否有服务更新，如果有更新，向该服务的提供者发送一个 notify 消息，provider 接受到 notify 消息后，即运行 NotifyListener 的 notify 方法，执行监听器方法。

动态代理模式
Dubbo 扩展 JDK SPI 的类 ExtensionLoader 的 Adaptive 实现是典型的动态代理实现。Dubbo 需要灵活地控制实现类，即在调用阶段动态地根据参数决定调用哪个实现类，所以采用先生成代理类的方法，能够做到灵活的调用。生成代理类的代码是 ExtensionLoader 的 createAdaptiveExtensionClassCode 方法。代理类的主要逻辑是，获取 URL 参数中指定参数的值作为获取实现类的 key。

19、Dubbo 配置文件是如何加载到 Spring 中的？

Spring 容器在启动的时候，会读取到 Spring 默认的一些 schema 以及 Dubbo 自定义的 schema，每个 schema 都会对应一个自己的 NamespaceHandler，NamespaceHandler 里面通过 BeanDefinitionParser 来解析配置信息并转化为需要加载的 bean 对象!

20、Dubbo SPI 和 Java SPI 区别？

JDK SPI
JDK 标准的 SPI 会一次性加载所有的扩展实现，如果有的扩展吃实话很耗时，但也没用上，很浪费资源。
所以只希望加载某个的实现，就不现实了

DUBBO SPI
1，对 Dubbo 进行扩展，不需要改动 Dubbo 的源码
2，延迟加载，可以一次只加载自己想要加载的扩展实现。
3，增加了对扩展点 IOC 和 AOP 的支持，一个扩展点可以直接 setter 注入其它扩展点。
4，Dubbo 的扩展机制能很好的支持第三方 IoC 容器，默认支持 Spring Bean。

21、Dubbo 支持分布式事务吗？

目前暂时不支持，可与通过 tcc-transaction 框架实现
介绍：tcc-transaction 是开源的 TCC 补偿性分布式事务框架
Git 地址：https://github.com/changmingxie/tcc-transaction
TCC-Transaction 通过 Dubbo 隐式传参的功能，避免自己对业务代码的入侵。

22、Dubbo 可以对结果进行缓存吗？

为了提高数据访问的速度。Dubbo 提供了声明式缓存，以减少用户加缓存的工作
量

```
<dubbo:reference cache="true" />
```

其实比普通的配置文件就多了一个标签 cache="true"

23、服务上线怎么兼容旧版本？

可以用版本号（version）过渡，多个不同版本的服务注册到注册中心，版本号不同的服务相互间不引用。这个和服务分组的概念有一点类似。

24、Dubbo 必须依赖的包有哪些？

Dubbo 必须依赖 JDK，其他为可选。

25、Dubbo telnet 命令能做什么？

dubbo 服务发布之后，我们可以利用 telnet 命令进行调试、管理。Dubbo2.0.5 以上版本服务提供端口支持 telnet 命令

连接服务
telnet localhost 20880 //键入回车进入 Dubbo 命令模式。
查看服务列表

```
dubbo>ls
com.test.TestService
dubbo>ls com.test.TestService
create
delete
query
```

- ls (list services and methods)
- ls : 显示服务列表。
- ls -l : 显示服务详细信息列表。
- ls XxxService：显示服务的方法列表。
- ls -l XxxService：显示服务的方法详细信息列表。

26、Dubbo 支持服务降级吗？

以通过 dubbo:reference 中设置 mock="return null"。mock 的值也可以修改为 true，然后再跟接口同一个路径下实现一个 Mock 类，命名规则是 “接口名称+Mock” 后缀。然后在 Mock 类里实现自己的降级逻辑

27、Dubbo 如何优雅停机？

Dubbo 是通过 JDK 的 ShutdownHook 来完成优雅停机的，所以如果使用kill -9 PID 等强制关闭指令，是不会执行优雅停机的，只有通过 kill PID 时，才会执行。

28、Dubbo 和 Dubbox 之间的区别？

Dubbox 是继 Dubbo 停止维护后，当当网基于 Dubbo 做的一个扩展项目，如加了服务可 Restful 调用，更新了开源组件等。

29、Dubbo 和 Spring Cloud 的区别？

根据微服务架构在各方面的要素，看看 Spring Cloud 和 Dubbo 都提供了哪些支持。

	Dubbo	Spring Cloud
服务注册中心	Zookeep er	Spring Cloud Netflix Eureka
服务调用方式	RPC	REST API
服务网关	无	Spring Cloud Netflix Zuul
断路器	不完善	Spring Cloud Netflix Hystrix
分布式配置	无	Spring Cloud Config
服务跟踪	无	Spring Cloud Sleuth
消息总线	无	Spring Cloud Bus
数据流	无	Spring Cloud Stream
批量任务	无	Spring Cloud Task
.....	.....	.....

使用 Dubbo 构建的微服务架构就像组装电脑，各环节我们的选择自由度很高，但是最终结果很有可能因为一条内存质量不行就点不亮了，总是让人不怎么放心，但是如果你是一名高手，那这些都不是问题；而 Spring Cloud 就像品牌机，在Spring Source 的整合下，做了大量的兼容性测试，保证了机器拥有更高的稳定性，但是如果要在使用非原装组件外的东西，就需要对其基础有足够的了解。

30、你还了解别的分布式框架吗？

别的还有 spring 的 spring cloud，facebook 的 thrift，twitter 的 finagle 等

31、Dubbo是什么？

Dubbo是阿里巴巴开源的基于 Java 的高性能 RPC 分布式服务框架，现已成为 Apache 基金会孵化项目。

面试官问你如果这个都不清楚，那下面的就没必要问了。

> 官网：http://dubbo.apache.org

32、Dubbo默认使用什么注册中心，还有别的选择吗？

推荐使用 Zookeeper 作为注册中心，还有 Redis、Multicast、Simple 注册中心，但不推荐。

33、Dubbo有哪几种配置方式？

1）Spring 配置方式
2）Java API 配置方式

34、在 Provider 上可以配置的 Consumer 端的属性有哪些？

1）timeout：方法调用超时
2）retries：失败重试次数，默认重试 2 次
3）loadbalance：负载均衡算法，默认随机
4）actives 消费者端，最大并发调用限制

35、Dubbo启动时如果依赖的服务不可用会怎样？

Dubbo 缺省会在启动时检查依赖的服务是否可用，不可用时会抛出异常，阻止 Spring 初始化完成，默认 check="true"，可以通过 check="false" 关闭检查。

36、Dubbo推荐使用什么序列化框架，你知道的还有哪些？

推荐使用Hessian序列化，还有Duddo、FastJson、Java自带序列化。

37、Dubbo有哪几种负载均衡策略，默认是哪种？

负载均衡策略	说明
Random LoadBalance	随机，按权重设置随机概率（默认）
RoundRobin LoadBalance	轮询，按公约后的权重设置轮询比率
LeastActive LoadBalance	最少活跃调用数，相同活跃数的随机
ConsistentHash LoadBalance	一致性 Hash，相同参数的请求总是发到同一提供者

https://blog.csdn.net/moakun

38、注册了多个同一样的服务，如果测试指定的某一个服务呢？

可以配置环境点对点直连，绕过注册中心，将以服务接口为单位，忽略注册中心的提供者列表。

39、Dubbo支持服务多协议吗？

Dubbo 允许配置多协议，在不同服务上支持不同协议或者同一服务上同时支持多种协议。

40、当一个服务接口有多种实现时怎么做？

当一个接口有多种实现时，可以用 group 属性来分组，服务提供方和消费方都指定同一个 group 即可。

41、服务上线怎么兼容旧版本？

可以用版本号（version）过渡，多个不同版本的服务注册到注册中心，版本号不同的服务相互间不引用。这个和服务分组的概念有一点类似。

42、Dubbo可以对结果进行缓存吗？

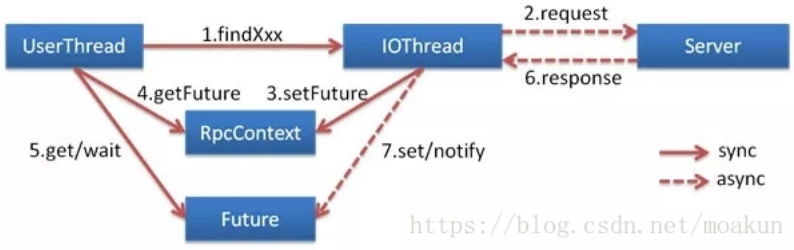
可以，Dubbo 提供了声明式缓存，用于加速热门数据的访问速度，以减少用户加缓存的工作量。

43、Dubbo服务之间的调用是阻塞的吗？

默认是同步等待结果阻塞的，支持异步调用。

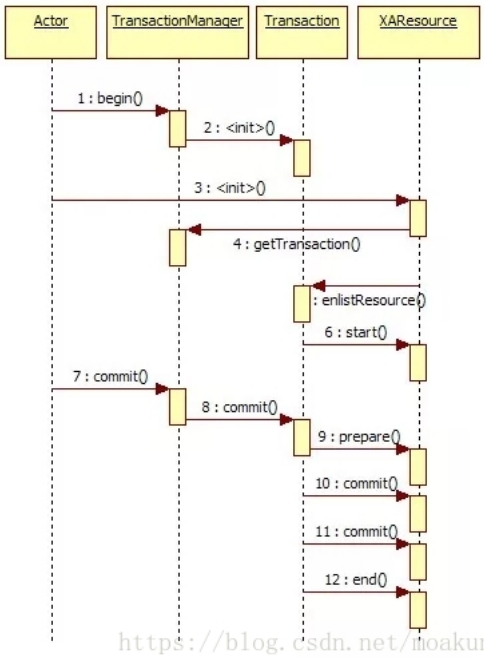
Dubbo 是基于 NIO 的非阻塞实现并行调用，客户端不需要启动多线程即可完成并行调用多个远程服务，相对多线程开销较小，异步调用会返回一个 Future 对象。

异步调用流程图如下。



44、Dubbo支持分布式事务吗？

目前暂时不支持，后续可能采用基于 JTA/XA 规范实现，如以图所示。



45、Dubbo支持服务降级吗？

Dubbo 2.2.0 以上版本支持。

46、Dubbo如何优雅停机？

Dubbo 是通过 JDK 的 ShutdownHook 来完成优雅停机的，所以如果使用 kill -9 PID 等强制关闭指令，是不会执行优雅停机的，只有通过 kill PID 时，才会执行。

47、服务提供者能实现失效踢出是什么原理？

服务失效踢出基于 Zookeeper 的临时节点原理。

29、如何解决服务调用链过长的问题？

Dubbo 可以使用 Pinpoint 和 Apache Skywalking(Incubator) 实现分布式服务追踪，当然还有其他很多方案。

30、服务读写推荐的容错策略是怎样的？

读操作建议使用 Failover 失败自动切换，默认重试两次其他服务器。

写操作建议使用 Failfast 快速失败，发一次调用失败就立即报错。

31、Dubbo必须依赖的包有哪些？

Dubbo 必须依赖 JDK，其他为可选。

32、Dubbo的管理控制台能做什么？

管理控制台主要包含：路由规则，动态配置，服务降级，访问控制，权重调整，负载均衡，等管理功能。

33、说说 Dubbo 服务暴露的过程。

Dubbo 会在 Spring 实例化完 bean 之后，在刷新容器最后一步发布 ContextRefreshEvent 事件的时候，通知实现了 ApplicationListener 的 ServiceBean 类进行回调 onApplicationEvent 事件方法，Dubbo 会在这个方法中调用 ServiceBean 父类 ServiceConfig 的 export 方法，而该方法真正实现了服务的（异步或者非异步）发布。

34、Dubbo 停止维护了吗？

2014 年开始停止维护过几年，17 年开始重新维护，并进入了 Apache 项目。

35、Dubbo 和 Dubbox 有什么区别？

Dubbox 是继 Dubbo 停止维护后，当当网基于 Dubbo 做的一个扩展项目，如加了服务可 Restful 调用，更新了开源组件等。

36、你还了解别的分布式框架吗？

别的还有 Spring cloud、Facebook 的 Thrift、Twitter 的 Finagle 等。

37、Dubbo 能集成 Spring Boot 吗？

可以的，项目地址如下。

> https://github.com/apache/incubator-dubbo-spring-boot-project

38、在使用过程中都遇到了些什么问题？

Dubbo 的设计目的是为了满足高并发小数据量的 rpc 调用，在大数据量下的性能表现并不好，建议使用 rmi 或 http 协议。

39、你读过 Dubbo 的源码吗？

要了解 Dubbo 就必须看其源码，了解其原理，花点时间看下吧，网上也有很多教程，后续有时间我也会在公众号上分享 Dubbo 的源码。

40、你觉得用 Dubbo 好还是 Spring Cloud 好？

扩展性的问题，没有好坏，只有适合不适合，不过我好像更倾向于使用 Dubbo, Spring Cloud 版本升级太快，组件更新替换太频繁，配置太繁琐，还有很多我觉得是没有 Dubbo 顺手的地方……

MyBatis 面试题

1、什么是 Mybatis？

1、Mybatis 是一个半 ORM（对象关系映射）框架，它内部封装了 JDBC，开发时只需要关注 SQL 语句本身，不需要花费精力去处理加载驱动、创建连接、创建statement 等繁杂的过程。程序员直接编写原生态 sql，可以严格控制 sql 执行性能，灵活度高。

2、MyBatis 可以使用 XML 或注解来配置和映射原生信息，将 POJO 映射成数据库中的记录，避免了几乎所有的 JDBC 代码和手动设置参数以及获取结果集。3、通过 xml 文件或注解的方式将要执行的各种 statement 配置起来，并通过java 对象和 statement 中 sql 的动态参数进行映射生成最终执行的 sql 语句，最后由 mybatis 框架执行 sql 并将结果映射为 java 对象并返回。（从执行 sql 到返回 result 的过程）。

2、Mybaits 的优点

1、基于 SQL 语句编程，相当灵活，不会对应用程序或者数据库的现有设计造成任何影响，SQL 写在 XML 里，解除 sql 与程序代码的耦合，便于统一管理；提供 XML标签，支持编写动态 SQL 语句，并可重用。

2、与 JDBC 相比，减少了 50%以上的代码量，消除了 JDBC 大量冗余的代码，不需要手动开关连接；

3、很好的与各种数据库兼容（因为 MyBatis 使用 JDBC 来连接数据库，所以只要JDBC 支持的数据库 MyBatis 都支持）。

4、能够与 Spring 很好的集成；

5、提供映射标签，支持对象与数据库的 ORM 字段关系映射；提供对象关系映射标签，支持对象关系组件维护。

3、MyBatis 框架的缺点

1、SQL 语句的编写工作量较大，尤其当字段多、关联表多时，对开发人员编写SQL 语句的功底有一定要求。
2、SQL 语句依赖于数据库，导致数据库移植性差，不能随意更换数据库。

4、MyBatis 框架适用场合

1、MyBatis 专注于 SQL 本身，是一个足够灵活的 DAO 层解决方案。
2、对性能的要求很高，或者需求变化较多的项目，如互联网项目，MyBatis 将是不错的选择。

5、MyBatis 与 Hibernate 有哪些不同？

1、Mybatis 和 hibernate 不同，它不完全是一个 ORM 框架，因为 MyBatis 需要程序员自己编写 Sql 语句。

2、Mybatis 直接编写原生态 sql，可以严格控制 sql 执行性能，灵活度高，非常适合对关系数据模型要求不高的软件开发，因为这类软件需求变化频繁，一但需求变化要求迅速输出成果。但是灵活的前提是 mybatis 无法做到数据库无关性，如果需要实现支持多种数据库的软件，则需要自定义多套 sql 映射文件，工作量大。

3、Hibernate 对象/关系映射能力强，数据库无关性好，对于关系模型要求高的软件，如果用 hibernate 开发可以节省很多代码，提高效率。

6、#{}和${}的区别是什么？

#{}是预编译处理，${}是字符串替换。
Mybatis 在处理#{}时，会将 sql 中的#{}替换为?号，调用 PreparedStatement 的set 方法来赋值；
Mybatis 在处理${}时，就是把${}替换成变量的值。
使用#{}可以有效的防止 SQL 注入，提高系统安全性

7、当实体类中的属性名和表中的字段名不一样 ，怎么办 ？

第 1 种： 通过在查询的 sql 语句中定义字段名的别名，让字段名的别名和实体类的属性名一致。

```
<select id="selectorder" parametertype="int" resultetype="
me.gacl.domain.order">
select order_id id, order_no orderno ,order_price price form
orders where order_id=#{id};
</select>
```

第 2 种： 通过来映射字段名和实体类属性名的一一对应的关系。

```
<select id="getOrder" parameterType="int" resultMap="orderresultmap">
    select * from orders where order_id=#{id}
</select>
<resultMap type="me.gacl.domain.order" id="orderresultmap">
    <!-用 id 属性来映射主键字段->
    <id property="id" column="order_id">
    <!-用 result 属性来映射非主键字段，property 为实体类属性名，column
    为数据表中的属性->
    <result property = "orderno" column ="order_no"/>
    <result property="price" column="order_price" />
</reslutMap>
```

8、 模糊查询 like 语句该怎么写?

第 1 种：在 Java 代码中添加 sql 通配符。

```
string wildcardname = "%smi%";
list<name> names = mapper.selectlike(wildcardname);

<select id="selectlike">
select * from foo where bar like #{value}
</select>
```

第 2 种：在 sql 语句中拼接通配符，会引起 sql 注入

```
tring wildcardname = "smi";
list<name> names = mapper.selectlike(wildcardname);

<select id="selectlike">
select * from foo where bar like "%"#{value}"%"
</select>
```

9、通常一个 Xml 映射文件，都会写一个 Dao 接口与之对应，请问，这个 Dao 接口的工作原理是什么？Dao 接口里的方法，参数不同时，方法能重载吗？

Dao 接口即 Mapper 接口。接口的全限名，就是映射文件中的 namespace 的值；接口的方法名，就是映射文件中 Mapper 的 Statement 的 id 值；接口方法内的参数，就是传递给 sql 的参数。Mapper 接口是没有实现类的，当调用接口方法时，接口全限名+方法名拼接字符串作为 key 值，可唯一定位一个 MapperStatement。在 Mybatis 中，每一个 ▾

```
<insert id="insertname">

insert into names (name) values (#{value})

</insert>
```

然后在 java 代码中像下面这样执行批处理插入:

```
list < string > names = new arraylist();
names.add(“fred”);
names.add(“barney”);
names.add(“betty”);
names.add(“wilma”);
// 注意这里 executortype.batch
sqlsession sqlsession =
sqlsessionfactory.opensession(executortype.batch);
try {
    namemapper mapper = sqlsession.getmapper(namemapper.class);
    for (string name: names) {
    mapper.insertname(name);
    }
    sqlsession.commit();
} catch (Exception e) {
    e.printStackTrace();
    sqlSession.rollback();
    throw e;
}
finally {
    sqlsession.close();
}
```

13、如何获取自动生成的(主)键值?

insert 方法 总是 返回 一个 int 值 ，这 个值 代表 的是 插入 的行 数。
如果 采用 自增 长策 略，自动 生成 的键 值在 insert 方法 执行 完后 可以 被设 置到 传入的参 数对 象中 。

示例 ：

```
<insert id="insertname" usegeneratedkeys="true" keyproperty="id">
insert into names (name) values (#{name})
</insert>


name name = new name();
name.setname("fred");
int rows = mapper.insertname(name);
// 完成后,id 已经被设置到对象中
system.out.println("rows inserted = " + rows);
system.out.println("generated key value = " + name.getid());
```

14、在 mapper 中如何传递多个参数?

1、第一种：
DAO 层的函数

```
public UserselectUser(String name,String area);
```

对应的 xml,#{0}代表接收的是 dao 层中的第一个参数，#{1}代表 dao 层中第二
参数，更多参数一致往后加即可。

```
<select id="selectUser"resultMap="BaseResultMap">
select * fromuser_user_t whereuser_name = #{0} anduser_area=#{1}
</select>
```

2、第二种： 使用 @param 注解

```
public interface usermapper {
user selectuser(@param("username") string
username,@param("hashedpassword") string hashedpassword);
}
```

然后,就可以在 xml 像下面这样使用(推荐封装为一个 map,作为单个参数传递给mapper

```
<select id="selectuser" resulttype="user">
select id, username, hashedpassword from some_table
where username = #{username}
and hashedpassword = #{hashedpassword}
</select>
```

3、第三种：多个参数封装成 map

```
try {
    //映射文件的命名空间.SQL 片段的 ID，就可以调用对应的映射文件中的SQL
    //由于我们的参数超过了两个，而方法中只有一个 Object 参数收集，因此我们使用 Map 集合来装载我们的参数
    Map < String, Object > map = new HashMap();
    map.put("start", start);
    map.put("end", end);
    return sqlSession.selectList("StudentID.pagination", map);
} catch (Exception e) {
    e.printStackTrace();
    sqlSession.rollback();
    throw e;
}
finally {
    MybatisUtil.closeSqlSession();
}
```

15、Mybatis 动态 sql 有什么用？执行原理？有哪些动态 sql？

Mybatis 动态 sql 可以 在 Xml 映射 文件 内，以标 签的 形式 编写 动态 sql，执行 原理是根 据表 达式 的值 完成 逻辑 判断 并动 态拼 接 sql 的功 能。
Mybatis 提供 了 9 种动 态 sql 标签 ：trim | where | set | foreach | if | choose| when | otherwise | bind。

16、Xml 映射文件中，除了常见的 select|insert|updae|delete标签之外，还有哪些标签？

答：、、、、，加上动态 sql 的 9 个标签，其中为 sql 片段标签，通过标签引入 sql 片段，为不支持自增的主键生成策略标签。

17、Mybatis 的 Xml 映射文件中，不同的 Xml 映射文件，id 是否可以重复？

不同的 Xml 映射文件，如果配置了 namespace，那么 id 可以重复；如果没有配置 namespace，那么 id 不能重复；

原因就是 namespace+id 是作为 Map<String, MapperStatement>的 key使用的，如果没有 namespace，就剩下 id，那么，id 重复会导致数据互相覆盖。有了 namespace，自然 id 就可以重复，namespace 不同，namespace+id 自然也就不同。

18、为什么说 Mybatis 是半自动 ORM 映射工具？它与全自动的区别在哪里？

Hibernate 属于全自动 ORM 映射工具，使用 Hibernate 查询关联对象或者关联集合对象时，可以根据对象关系模型直接获取，所以它是全自动的。而 Mybatis在查询关联对象或关联集合对象时，需要手动编写 sql 来完成，所以，称之为半自动 ORM 映射工具。

19、 一对一、一对多的关联查询 ？

```
<mapper namespace="com.lcb.mapping.userMapper">
    <!--association 一对一关联查询 -->
    <select id="getClass" parameterType="int" resultMap="ClassesResultMap">
        select * from class c,teacher t where c.teacher_id=t.t_id and
        c.c_id=#{id}
    </select>
    <resultMap type="com.lcb.user.Classes" id="ClassesResultMap">
        <!-- 实体类的字段名和数据表的字段名映射 -->
        <id property="id" column="c_id"/>
        <result property="name" column="c_name"/>
        <association property="teacher"
        javaType="com.lcb.user.Teacher">
        <id property="id" column="t_id"/>
        <result property="name" column="t_name"/>
        </association>
    </resultMap>
    <!--collection 一对多关联查询 -->
    <select id="getClass2" parameterType="int"
        resultMap="ClassesResultMap2">
        select * from class c,teacher t,student s where c.teacher_id=t.t_id
        and c.c_id=s.class_id and c.c_id=#{id}
    </select>
    <resultMap type="com.lcb.user.Classes" id="ClassesResultMap2">
        <id property="id" column="c_id"/>
        <result property="name" column="c_name"/>
        <association property="teacher"
        javaType="com.lcb.user.Teacher">
        <id property="id" column="t_id"/>
        <result property="name" column="t_name"/>
        </association>
        <collection property="student" ofType="com.lcb.user.Student">
            <id property="id" column="s_id"/>
            <result property="name" column="s_name"/>
        </collection>
    </resultMap>
</mapper>
```

20、MyBatis 实现一对一有几种方式?具体怎么操作的？

有联合查询和嵌套查询,联合查询是几个表联合查询,只查询一次, 通过在resultMap 里面配置 association 节点配置一对一的类就可以完成；

嵌套查询是先查一个表，根据这个表里面的结果的 外键 id，去再另外一个表里面查询数据,也是通过 association 配置，但另外一个表的查询通过 select 属性配置

21、MyBatis 实现一对多有几种方式,怎么操作的？

有联合查询和嵌套查询。联合查询是几个表联合查询,只查询一次,通过在resultMap 里面的 collection 节点配置一对多的类就可以完成；嵌套查询是先查一个表,根据这个表里面的结果的外键 id,去再另外一个表里面查询数据,也是通过配置 collection,但另外一个表的查询通过 select 节点配置。

22、Mybatis 是否支持延迟加载？如果支持，它的实现原理是什么？

答：Mybatis 仅支持 association 关联对象和 collection 关联集合对象的延迟加载，association 指的就是一对一，collection 指的就是一对多查询。在 Mybatis配置文件中，可以配置是否启用延迟加载 lazyLoadingEnabled=true|false。

它的原理是，使用 CGLIB 创建目标对象的代理对象，当调用目标方法时，进入拦截器方法，比如调用 a.getB().getName()，拦截器 invoke()方法发现 a.getB()是null 值，那么就会单独发送事先保存好的查询关联 B 对象的 sql，把 B 查询上来，然后调用 a.setB(b)，于是 a 的对象 b 属性就有值了，接着完成 a.getB().getName() 方法的调用。这就是延迟加载的基本原理。

当然了，不光是 Mybatis，几乎所有的包括 Hibernate，支持延迟加载的原理都是一样的。

23、Mybatis 的一级、二级缓存

1）一级缓存: 基于 PerpetualCache 的 HashMap 本地缓存，其存储作用域为Session，当 Session flush 或 close 之后，该 Session 中的所有 Cache 就将清空，默认打开一级缓存。

2）二级缓存与一级缓存其机制相同，默认也是采用 PerpetualCache，HashMap存储，不同在于其存储作用域为 Mapper(Namespace)，并且可自定义存储源，如 Ehcache。默认不打开二级缓存，要开启二级缓存，使用二级缓存属性类需要实现 Serializable 序列化接口(可用来保存对象的状态),可在它的映射文件中配置 ；

3）对于缓存数据更新机制，当某一个作用域(一级缓存 Session/二级缓存Namespaces)的进行了 C/U/D 操作后，默认该作用域下所有 select 中的缓存将被 clear。

24、什么是 MyBatis 的接口绑定？有哪些实现方式？

接口绑定，就是在 MyBatis 中任意定义接口,然后把接口里面的方法和 SQL 语句绑定, 我们直接调用接口方法就可以,这样比起原来了 SqlSession 提供的方法我们可以有更加灵活的选择和设置。

接口绑定有两种实现方式,一种是通过注解绑定，就是在接口的方法上面加上@Select、@Update 等注解，里面包含 Sql 语句来绑定；另外一种就是通过 xml里面写 SQL 来绑定, 在这种情况下,要指定 xml 映射文件里面的 namespace 必须为接口的全路径名。当 Sql 语句比较简单时候,用注解绑定, 当 SQL 语句比较复杂时候,用 xml 绑定,一般用 xml 绑定的比较多。

25、使用 MyBatis 的 mapper 接口调用时有哪些要求？

1、Mapper 接口方法名和 mapper.xml 中定义的每个 sql 的 id 相同；
2、Mapper 接口方法的输入参数类型和 mapper.xml 中定义的每个 sql 的parameterType 的类型相同；
3、Mapper 接口方法的输出参数类型和 mapper.xml 中定义的每个 sql 的resultType 的类型相同；
4、Mapper.xml 文件中的 namespace 即是 mapper 接口的类路径。

26、Mapper 编写有哪几种方式？

第一种：接口实现类继承 SqlSessionDaoSupport：使用此种方法需要编写mapper 接口，mapper 接口实现类、mapper.xml 文件。
1、在 sqlMapConfig.xml 中配置 mapper.xml 的位置

```
<mappers>
<mapper resource="mapper.xml 文件的地址" />
<mapper resource="mapper.xml 文件的地址" />
</mappers>
```

2、定义 mapper 接口
3、实现类集成 SqlSessionDaoSupport mapper 方法中可以 this.getSqlSession()进行数据增删改查。
4、spring 配置

```
<bean id=" " class="mapper 接口的实现">
<property name="sqlSessionFactory"
ref="sqlSessionFactory"></property>
</bean>
```

第二种：使用 org.mybatis.spring.mapper.MapperFactoryBean：
1、在 sqlMapConfig.xml 中配置 mapper.xml 的位置，如果 mapper.xml 和mappre 接口的名称相同且在同一个目录，这里可以不用配置

```
<mappers>
<mapper resource="mapper.xml 文件的地址" />
<mapper resource="mapper.xml 文件的地址" />
</mappers>
```

2、定义 mapper 接口：

1、mapper.xml 中的 namespace 为 mapper 接口的地址
2、mapper 接口中的方法名和 mapper.xml 中的定义的 statement 的 id 保持一致

3、Spring 中定义

```
<bean id="" class="org.mybatis.spring.mapper.MapperFactoryBean">
<property name="mapperInterface" value="mapper 接口地址" />
<property name="sqlSessionFactory" ref="sqlSessionFactory" />
</bean>
```

第三种：使用 mapper 扫描器：
1、mapper.xml 文件编写：
mapper.xml 中的 namespace 为 mapper 接口的地址；
mapper 接口中的方法名和 mapper.xml 中的定义的 statement 的 id 保持一致；
如果将 mapper.xml 和 mapper 接口的名称保持一致则不用在 sqlMapConfig.xml中进行配置。
2、定义 mapper 接口：
注意 mapper.xml 的文件名和 mapper 的接口名称保持一致，且放在同一个目录
3、配置 mapper 扫描器：

```
<bean class="org.mybatis.spring.mapper.MapperScannerConfigurer">
<property name="basePackage" value="mapper 接口包地址"></property>
<property name="sqlSessionFactoryBeanName" value="sqlSessionFactory"/>
</bean>
```

4、使用扫描器后从 spring 容器中获取 mapper 的实现对象。

27、简述 Mybatis 的插件运行原理，以及如何编写一个插件。

答：Mybatis 仅可以编写针对 ParameterHandler、ResultSetHandler、StatementHandler、Executor 这 4 种接口的插件，Mybatis 使用 JDK 的动态代理，为需要拦截的接口生成代理对象以实现接口方法拦截功能，每当执行这 4 种接口对象的方法时，就会进入拦截方法，具体就是 InvocationHandler 的 invoke()方法，当然，只会拦截那些你指定需要拦截的方法。

编写插件：实现 Mybatis 的 Interceptor 接口并复写 intercept()方法，然后在给插件编写注解，指定要拦截哪一个接口的哪些方法即可，记住，别忘了在配置文件中配置你编写的插件。

28、MyBatis实现一对一有几种方式?具体怎么操作的 ？

有联合查询和嵌套查询,联合查询是几个表联合查询,只查询一次, 通过在resultMap里面配置association节点配置一对一的类就可以完成；
嵌套查询是先查一个表，根据这个表里面的结果的 外键id，去再另外一个表里面查询数据,也是通过association配置，但另外一个表的查询通过select属性配置

ZooKeeper 面试题

1、什么是Zookeeper?

ZooKeeper 是一个开放源码的分布式协调服务，它是集群的管理者，监视着集群中各个节点的状态根据节点提交的反馈进行下一步合理操作。最终，将简单易用的接口和性能高效、功能稳定的系统提供给用户。分布式应用程序可以基于 Zookeeper 实现诸如数据发布/订阅、负载均衡、命名服务、分布式协调/通知、集群管理、Master 选举、分布式锁和分布式队列等功能。

2、Zookeeper 如何保证了分布式一致性特性？

1、顺序一致性
2、原子性
3、单一视图
4、可靠性
5、实时性（最终一致性）
客户端的读请求可以被集群中的任意一台机器处理，如果读请求在节点上注册了监听器，这个监听器也是由所连接的 zookeeper 机器来处理。对于写请求，这些请求会同时发给其他 zookeeper 机器并且达成一致后，请求才会返回成功。因此，随着 zookeeper 的集群机器增多，读请求的吞吐会提高但是写请求的吞吐会下降。有序性是 zookeeper 中非常重要的一个特性，所有的更新都是全局有序的，每个更新都有一个唯一的时间戳，这个时间戳称为 zxid（Zookeeper Transaction Id）。而读请求只会相对于更新有序，也就是读请求的返回结果中会带有这个
zookeeper 最新的 zxid

3、ZooKeeper 提供了什么？

1、文件系统
2、通知机制

4、Zookeeper 文件系统

Zookeeper 提供一个多层级的节点命名空间（节点称为 znode）。与文件系统不同的是，这些节点都可以设置关联的数据，而文件系统中只有文件节点可以存放数据而目录节点不行。

Zookeeper 为了保证高吞吐和低延迟，在内存中维护了这个树状的目录结构，这种特性使得 Zookeeper 不能用于存放大量的数据，每个节点的存放数据上限为1M。

5、ZAB 协议？

ZAB 协议是为分布式协调服务 Zookeeper 专门设计的一种支持崩溃恢复的原子广播协议。

ZAB 协议包括两种基本的模式：崩溃恢复和消息广播 。

当整个 zookeeper 集群刚刚启动或者 Leader 服务器宕机、重启或者网络故障导致不存在过半的服务器与 Leader 服务器保持正常通信时，所有进程（服务器）进入崩溃恢复模式，首先选举产生新的 Leader 服务器，然后集群中 Follower 服务器开始与新的 Leader 服务器进行数据同步，当集群中超过半数机器与该 Leader服务器完成数据同步之后，退出恢复模式进入消息广播模式，Leader 服务器开始接收客户端的事务请求生成事物提案来进行事务请求处理

6、四种类型的数据节点 Znode

1、PERSISTENT-持久节点
除非手动删除，否则节点一直存在于 Zookeeper 上

2、EPHEMERAL-临时节点
临时节点的生命周期与客户端会话绑定，一旦客户端会话失效（客户端与zookeeper 连接断开不一定会话失效），那么这个客户端创建的所有临时节点都会被移除。

3、PERSISTENT_SEQUENTIAL-持久顺序节点
基本特性同持久节点，只是增加了顺序属性，节点名后边会追加一个由父节点维护的自增整型数字。

4、EPHEMERAL_SEQUENTIAL-临时顺序节点
基本特性同临时节点，增加了顺序属性，节点名后边会追加一个由父节点维护的自增整型数字。

7、Zookeeper Watcher 机制 -- 数据变更通知

Zookeeper 允许客户端向服务端的某个 Znode 注册一个 Watcher 监听，当服务端的一些指定事件触发了这个 Watcher，服务端会向指定客户端发送一个事件通知来实现分布式的通知功能，然后客户端根据 Watcher 通知状态和事件类型做出业务上的改变。

工作机制：
1、客户端注册 watcher
2、服务端处理 watcher
3、客户端回调 watcher

Watcher 特性总结

1、一次性
无论是服务端还是客户端，一旦一个 Watcher 被触发，Zookeeper 都会将其从相应的存储中移除。这样的设计有效的减轻了服务端的压力，不然对于更新非常频繁的节点，服务端会不断的向客户端发送事件通知，无论对于网络还是服务端的压力都非常大。
2、客户端串行执行
客户端 Watcher 回调的过程是一个串行同步的过程。
3、轻量

3.1、Watcher 通知非常简单，只会告诉客户端发生了事件，而不会说明事件的具体内容。
3.2、客户端向服务端注册 Watcher 的时候，并不会把客户端真实的 Watcher 对象实体传递到服务端，仅仅是在客户端请求中使用 boolean 类型属性进行了标记。

4、watcher event 异步发送 watcher 的通知事件从 server 发送到 client 是异步的，这就存在一个问题，不同的客户端和服务器之间通过 socket 进行通信，由于网络延迟或其他因素导致客户端在不通的时刻监听到事件，由于 Zookeeper 本身提供了 ordering guarantee，即客户端监听事件后，才会感知它所监视 znode发生了变化。所以我们使用 Zookeeper 不能期望能够监控到节点每次的变化。Zookeeper 只能保证最终的一致性，而无法保证强一致性。
5、注册 watcher getData、exists、getChildren
6、触发 watcher create、delete、setData
7、当一个客户端连接到一个新的服务器上时，watch 将会被以任意会话事件触发。当与一个服务器失去连接的时候，是无法接收到 watch 的。而当 client 重新连接时，如果需要的话，所有先前注册过的 watch，都会被重新注册。通常这是完全透明的。只有在一个特殊情况下，watch 可能会丢失：对于一个未创建的 znode的 exist watch，如果在客户端断开连接期间被创建了，并且随后在客户端连接上之前又删除了，这种情况下，这个 watch 事件可能会被丢失。

8、客户端注册 Watcher 实现

1、调用 getData()/getChildren()/exist()三个 API，传入 Watcher 对象
2、标记请求 request，封装 Watcher 到 WatchRegistration
3、封装成 Packet 对象，发服务端发送 request
4、收到服务端响应后，将 Watcher 注册到 ZKWatcherManager 中进行管理
5、请求返回，完成注册。

9、 服务端处理 Watcher 实现

1、服务端接收 Watcher 并存储
接收到客户端请求，处理请求判断是否需要注册 Watcher，需要的话将数据节点的节点路径和 ServerCnxn（ServerCnxn 代表一个客户端和服务端的连接，实现了 Watcher 的 process 接口，此时可以看成一个 Watcher 对象）存储在WatcherManager 的 WatchTable 和 watch2Paths 中去。

2、Watcher 触发
以服务端接收到 setData() 事务请求触发 NodeDataChanged 事件为例：
2.1 封装 WatchedEvent将通知状态（SyncConnected）、事件类型（NodeDataChanged）以及节点路径封装成一个 WatchedEvent 对象
2.2 查询 Watcher从 WatchTable 中根据节点路径查找 Watcher
2.3 没找到；说明没有客户端在该数据节点上注册过 Watcher
2.4 找到；提取并从 WatchTable 和 Watch2Paths 中删除对应 Watcher（从这里可以看出 Watcher 在服务端是一次性的，触发一次就失效了）
3、调用 process 方法来触发 Watcher

这里 process 主要就是通过 ServerCnxn 对应的 TCP 连接发送 Watcher 事件通知

10、客户端回调 Watcher

客户端 SendThread 线程接收事件通知，交由 EventThread 线程回调 Watcher。客户端的 Watcher 机制同样是一次性的，一旦被触发后，该 Watcher 就失效了。

11、ACL 权限控制机制

UGO（User/Group/Others）
目前在 Linux/Unix 文件系统中使用，也是使用最广泛的权限控制方式。是一种粗粒度的文件系统权限控制模式。

ACL（Access Control List）访问控制列表包括三个方面：
权限模式（Scheme）
1、IP：从 IP 地址粒度进行权限控制
2、Digest：最常用，用类似于 username:password 的权限标识来进行权限配置，便于区分不同应用来进行权限控制
3、World：最开放的权限控制方式，是一种特殊的 digest 模式，只有一个权限标识“world:anyone”
4、Super：超级用户

授权对象
授权对象指的是权限赋予的用户或一个指定实体，例如 IP 地址或是机器灯。

权限 Permission
1、CREATE：数据节点创建权限，允许授权对象在该 Znode 下创建子节点

2、DELETE：子节点删除权限，允许授权对象删除该数据节点的子节点
3、READ：数据节点的读取权限，允许授权对象访问该数据节点并读取其数据内容或子节点列表等
4、WRITE：数据节点更新权限，允许授权对象对该数据节点进行更新操作
5、ADMIN：数据节点管理权限，允许授权对象对该数据节点进行 ACL 相关设置操作

12、Chroot 特性

3.2.0 版本后，添加了 Chroot 特性，该特性允许每个客户端为自己设置一个命名空间。如果一个客户端设置了 Chroot，那么该客户端对服务器的任何操作，都将会被限制在其自己的命名空间下。

通过设置 Chroot，能够将一个客户端应用于 Zookeeper 服务端的一颗子树相对应，在那些多个应用公用一个 Zookeeper 进群的场景下，对实现不同应用间的相互隔离非常有帮助。

13、会话管理

分桶策略：将类似的会话放在同一区块中进行管理，以便于 Zookeeper 对会话进行不同区块的隔离处理以及同一区块的统一处理。
分配原则：每个会话的“下次超时时间点”（ExpirationTime）
计算公式：

```
ExpirationTime_ = currentTime + sessionTimeout
ExpirationTime = (ExpirationTime_ / ExpirationInrerval + 1) *
ExpirationInterval ， ExpirationInterval 是指 Zookeeper 会话超时检查时间
间隔，默认 tickTime
```

14、服务器角色

Leader
1、事务请求的唯一调度和处理者，保证集群事务处理的顺序性
2、集群内部各服务的调度者
Follower
1、处理客户端的非事务请求，转发事务请求给 Leader 服务器
2、参与事务请求 Proposal 的投票
3、参与 Leader 选举投票
Observer
1、3.0 版本以后引入的一个服务器角色，在不影响集群事务处理能力的基础上提升集群的非事务处理能力
2、处理客户端的非事务请求，转发事务请求给 Leader 服务器
3、不参与任何形式的投票

15、Zookeeper 下 Server 工作状态

服务器具有四种状态，分别是 LOOKING、FOLLOWING、LEADING、OBSERVING。

1、LOOKING：寻找 Leader 状态。当服务器处于该状态时，它会认为当前集群中没有 Leader，因此需要进入 Leader 选举状态。
2、FOLLOWING：跟随者状态。表明当前服务器角色是 Follower。
3、LEADING：领导者状态。表明当前服务器角色是 Leader。
4、OBSERVING：观察者状态。表明当前服务器角色是 Observer。

16、数据同步

整个集群完成 Leader 选举之后，Learner（Follower 和 Observer 的统称）回向Leader 服务器进行注册。当 Learner 服务器想 Leader 服务器完成注册后，进入数据同步环节。

数据同步流程：（均以消息传递的方式进行）
Learner 向 Learder 注册
数据同步
同步确认

Zookeeper 的数据同步通常分为四类：

1、直接差异化同步（DIFF 同步）
2、先回滚再差异化同步（TRUNC+DIFF 同步）
3、仅回滚同步（TRUNC 同步）
4、全量同步（SNAP 同步）

在进行数据同步前，Leader 服务器会完成数据同步初始化：
peerLastZxid：

 从 learner 服务器注册时发送的 ACKEPOCH 消息中提取 lastZxid（该Learner 服务器最后处理的 ZXID）
minCommittedLog：
 Leader 服务器 Proposal 缓存队列 committedLog 中最小 ZXID
maxCommittedLog：
 Leader 服务器 Proposal 缓存队列 committedLog 中最大 ZXID

直接差异化同步（DIFF 同步）
 场景：peerLastZxid 介于 minCommittedLog 和 maxCommittedLog之间
先回滚再差异化同步（TRUNC+DIFF 同步）
 场景：当新的 Leader 服务器发现某个 Learner 服务器包含了一条自己没有的事务记录，那么就需要让该 Learner 服务器进行事务回滚--回滚到 Leader服务器上存在的，同时也是最接近于 peerLastZxid 的 ZXID

仅回滚同步（TRUNC 同步）

 场景：peerLastZxid 大于 maxCommittedLog全量同步（SNAP 同步）
 场景一：peerLastZxid 小于 minCommittedLog
 场景二：Leader 服务器上没有 Proposal 缓存队列且 peerLastZxid 不等于 lastProcessZxid

17、zookeeper 是如何保证事务的顺序一致性的？

zookeeper 采用了全局递增的事务 Id 来标识，所有的 proposal（提议）都在被提出的时候加上了 zxid，zxid 实际上是一个 64 位的数字，高 32 位是 epoch（时期; 纪元; 世; 新时代）用来标识 leader 周期，如果有新的 leader 产生出来，epoch会自增，低 32 位用来递增计数。当新产生 proposal 的时候，会依据数据库的两阶段过程，首先会向其他的 server 发出事务执行请求，如果超过半数的机器都能执行并且能够成功，那么就会开始执行。

18、zk 节点宕机如何处理？

Zookeeper 本身也是集群，推荐配置不少于 3 个服务器。Zookeeper 自身也要保证当一个节点宕机时，其他节点会继续提供服务。
如果是一个 Follower 宕机，还有 2 台服务器提供访问，因为 Zookeeper 上的数据是有多个副本的，数据并不会丢失；
如果是一个 Leader 宕机，Zookeeper 会选举出新的 Leader。
ZK 集群的机制是只要超过半数的节点正常，集群就能正常提供服务。只有在 ZK节点挂得太多，只剩一半或不到一半节点能工作，集群才失效。
所以

3 个节点的 cluster 可以挂掉 1 个节点(leader 可以得到 2 票>1.5)
2 个节点的 cluster 就不能挂掉任何 1 个节点了(leader 可以得到 1 票<=1)

19、zookeeper 负载均衡和 nginx 负载均衡区别

zk 的负载均衡是可以调控，nginx 只是能调权重，其他需要可控的都需要自己写插件；但是 nginx 的吞吐量比 zk 大很多，应该说按业务选择用哪种方式。

20、分布式集群中为什么会有 Master？

在分布式环境中，有些业务逻辑只需要集群中的某一台机器进行执行，其他的机器可以共享这个结果，这样可以大大减少重复计算，提高性能，于是就需要进行leader 选举。

21、Zookeeper 有哪几种几种部署模式？

部署模式：单机模式、伪集群模式、集群模式

22、集群最少要几台机器，集群规则是怎样的?

集群规则为 2N+1 台，N>0，即 3 台

23、集群支持动态添加机器吗？

其实就是水平扩容了，Zookeeper 在这方面不太好。两种方式：

全部重启：关闭所有 Zookeeper 服务，修改配置之后启动。不影响之前客户端的会话。
逐个重启：在过半存活即可用的原则下，一台机器重启不影响整个集群对外提供服务。这是比较常用的方式。
3.5 版本开始支持动态扩容

24、Zookeeper 对节点的 watch监听通知是永久的吗？为什么不是永久的?

不是。官方声明：一个 Watch 事件是一个一次性的触发器，当被设置了 Watch的数据发生了改变的时候，则服务器将这个改变发送给设置了 Watch 的客户端，以便通知它们。

为什么不是永久的，举个例子，如果服务端变动频繁，而监听的客户端很多情况下，每次变动都要通知到所有的客户端，给网络和服务器造成很大压力。一般是客户端执行 getData("/节点 A",true)，如果节点 A 发生了变更或删除，客户端会得到它的 watch 事件，但是在之后节点 A 又发生了变更，而客户端又没有设置 watch 事件，就不再给客户端发送。

在实际应用中，很多情况下，我们的客户端不需要知道服务端的每一次变动，我只要最新的数据即可。

25、Zookeeper 的 java 客户端都有哪些？

java 客户端：zk 自带的 zkclient 及 Apache 开源的 Curator。

26、chubby 是什么，和 zookeeper 比你怎么看 ？

chubby 是 google 的， 完全 实现 paxos 算法 ，不 开源 。zookeeper 是 chubby的开 源实 现， 使用 zab 协议 ，paxos 算法 的变 种。

27、说几个 zookeeper 常用的命令。

常用 命令 ：ls get set create delete 等。

28、ZAB 和 Paxos 算法的联系与区别？

相同点：
1、两者 都存 在一 个类 似于 Leader 进程 的角 色 ，由其 负责 协调 多个 Follower 进程的运 行
2、Leader 进程 都会 等待 超过 半数 的 Follower 做出 正确 的反 馈后 ，才会 将一 个提案进 行提 交
3、 ZAB 协议 中， 每个 Proposal 中都 包含 一个 epoch 值来 代表 当前 的 Leader周期 ，Paxos 中名 字为 Ballot

不同点：
ZAB 用来 构建 高可 用的 分布 式数 据主 备系 统（ Zookeeper ）， Paxos 是用 来构 建分布 式一 致性 状态 机系 统。

29、Zookeeper 的典型应用场景

Zookeeper 是一个典型的发布/订阅模式的分布式数据管理与协调框架，开发人员可以使用它来进行分布式数据的发布和订阅。

通过对 Zookeeper 中丰富的数据节点进行交叉使用，配合 Watcher 事件通知机制，可以非常方便的构建一系列分布式应用中年都会涉及的核心功能，如：
1、数据发布/订阅
2、负载均衡
3、命名服务
4、分布式协调/通知
5、集群管理
6、Master 选举
7、分布式锁
8、分布式队列

30、数据发布/订阅

数据发布/订阅系统，即所谓的配置中心，顾名思义就是发布者发布数据供订阅者进行数据订阅。

目的
动态获取数据（配置信息）
实现数据（配置信息）的集中式管理和数据的动态更新

设计模式
Push 模式
Pull 模式

数据（配置信息）特性

1、数据量通常比较小
2、数据内容在运行时会发生动态更新
3、集群中各机器共享，配置一致
如：机器列表信息、运行时开关配置、数据库配置信息等

基于 Zookeeper 的实现方式
□ 数据存储：将数据（配置信息）存储到 Zookeeper 上的一个数据节点
□ 数据获取：应用在启动初始化节点从 Zookeeper 数据节点读取数据，并在该节点上注册一个数据变更 Watcher
□ 数据变更：当变更数据时，更新 Zookeeper 对应节点数据，Zookeeper会将数据变更通知发到各客户端，客户端接到通知后重新读取变更后的数据即可。

31、zk 的命名服务

命名服务是指通过指定的名字来获取资源或者服务的地址，利用 zk 创建一个全局的路径，这个路径就可以作为一个名字，指向集群中的集群，提供的服务的地址，或者一个远程的对象等等。

32、分布式通知和协调

对于系统调度来说：操作人员发送通知实际是通过控制台改变某个节点的状态，然后 zk 将这些变化发送给注册了这个节点的 watcher 的所有客户端。 对于执行情况汇报：每个工作进程都在某个目录下创建一个临时节点。并携带工作的进度数据，这样汇总的进程可以监控目录子节点的变化获得工作进度的实时的全局情况。

33、zk 的命名服务（文件系统）

命名服务是指通过指定的名字来获取资源或者服务的地址，利用 zk 创建一个全局的路径，即是唯一的路径，这个路径就可以作为一个名字，指向集群中的集群，提供的服务的地址，或者一个远程的对象等等。

34、zk 的配置管理（文件系统、通知机制）**

程序分布式的部署在不同的机器上，将程序的配置信息放在 zk 的 znode 下，当有配置发生改变时，也就是 znode 发生变化时，可以通过改变 zk 中某个目录节点的内容，利用 watcher 通知给各个客户端，从而更改配置。

35、Zookeeper 集群管理（文件系统、通知机制）**

所谓集群管理无在乎两点：是否有机器退出和加入、选举 master。对于第一点，所有机器约定在父目录下创建临时目录节点，然后监听父目录节点的子节点变化消息。一旦有机器挂掉，该机器与 zookeeper 的连接断开，其所创建的临时目录节点被删除，所有其他机器都收到通知：某个兄弟目录被删除，于是，所有人都知道：它上船了。
新机器加入也是类似，所有机器收到通知：新兄弟目录加入，highcount 又有了，对于第二点，我们稍微改变一下，所有机器创建临时顺序编号目录节点，每次选取编号最小的机器作为 master 就好。

36、Zookeeper 分布式锁（文件系统、通知机制**）

有了 zookeeper 的一致性文件系统，锁的问题变得容易。锁服务可以分为两类，一个是保持独占，另一个是控制时序 对于第一类，我们将 zookeeper 上的一个 znode 看作是一把锁，通过 createznode的方式来实现。所有客户端都去创建 /distribute_lock 节点，最终成功创建的那个客户端也即拥有了这把锁。用完删除掉自己创建的 distribute_lock 节点就释放出锁。

对于第二类， /distribute_lock 已经预先存在，所有客户端在它下面创建临时顺序编号目录节点，和选 master 一样，编号最小的获得锁，用完删除，依次方便。

37、Zookeeper 队列管理（文件系统、通知机制）

两种类型的队列：
1、同步队列，当一个队列的成员都聚齐时，这个队列才可用，否则一直等待所有成员到达。
2、队列按照 FIFO 方式进行入队和出队操作。

第一类，在约定目录下创建临时目录节点，监听节点数目是否是我们要求的数目。
第二类，和分布式锁服务中的控制时序场景基本原理一致，入列有编号，出列按编号。在特定的目录下创建 PERSISTENT_SEQUENTIAL 节点，创建成功时Watcher 通知等待的队列，队列删除序列号最小的节点用以消费。此场景下Zookeeper 的 znode 用于消息存储，znode 存储的数据就是消息队列中的消息内容，SEQUENTIAL 序列号就是消息的编号，按序取出即可。由于创建的节点是持久化的，所以不必担心队列消息的丢失问题。

38、Zookeeper 角色

Zookeeper 集群是一个基于主从复制的高可用集群，每个服务器承担如下三种角色中的一种

Leader

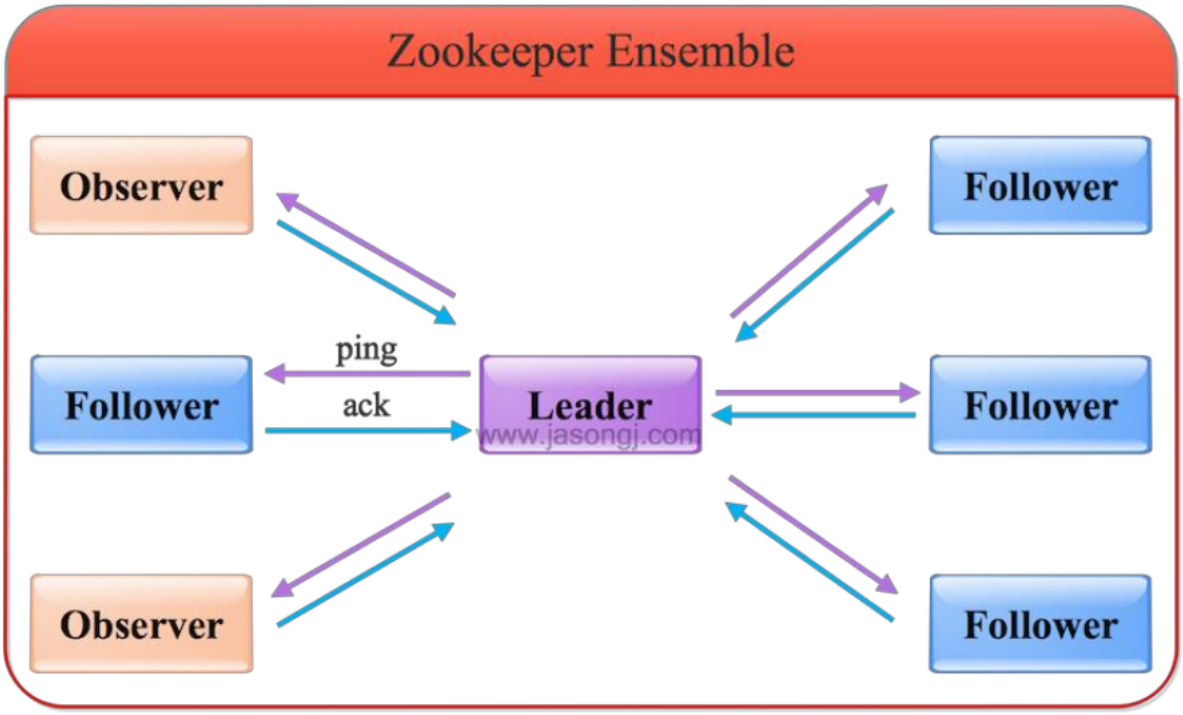
1. 一个 Zookeeper 集群同一时间只会有一个实际工作的 Leader，它会发起并维护与各 Follwer及 Observer 间的心跳。
2. . 所有的写操作必须要通过 Leader 完成再由 Leader 将写操作广播给其它服务器。 只要有超过半数节点（不包括 observeer 节点）写入成功，该写请求就会被提交（类 2PC 协议）。

Follower

1. 一个 Zookeeper 集群可能同时存在多个 Follower，它会响应 Leader 的心跳，
2. Follower 可直接处理并返回客户端的读请求，同时会将写请求转发给 Leader 处理，
3. 并且负责在 Leader 处理写请求时对请求进行投票。

Observer

角色与 Follower 类似，但是无投票权。 Zookeeper 需保证高可用和强一致性，为了支持更多的客户端，需要增加更多 Server； Server 增多，投票阶段延迟增大，影响性能；引入 Observer，Observer 不参与投票； bservers 接受客户端的连接，并将写请求转发给 leader 节点； 加入更多 Observer 节点，提高伸缩性，同时不影响吞吐率。



39、事务编号 Zxid（事务请求计数器+ epoch）

在 ZAB (ZooKeeper Atomic Broadcast , ZooKeeper 原子消息广播协议）协议的事务编号 Zxid设计中， Zxid 是一个 64 位的数字，其中低 32 位是一个简单的单调递增的计数器， 针对客户端每一个事务请求，计数器加 1；而高 32 位则代表 Leader 周期 epoch 的编号， 每个当选产生一个新的 Leader 服务器，就会从这个 Leader 服务器上取出其本地日志中最大事务的 ZXID，并从中读取epoch 值，然后加 1，以此作为新的 epoch，并将低 32 位从 0 开始计数。Zxid（Transaction id）类似于 RDBMS 中的事务 ID，用于标识一次更新操作的 Proposal（提议）
ID。为了保证顺序性，该 zkid 必须单调递增。

40、epoch

epoch：可以理解为当前集群所处的年代或者周期，每个 leader 就像皇帝，都有自己的年号，所以每次改朝换代， leader 变更之后，都会在前一个年代的基础上加 1。这样就算旧的 leader 崩溃恢复之后，也没有人听他的了，因为 follower 只听从当前年代的 leader 的命令。

41、Zab 协议有两种模式-恢复模式（选主）、广播模式（同步）

Zab 协议有两种模式，它们分别是恢复模式（选主）和广播模式（同步）。当服务启动或者在领导者崩溃后， Zab 就进入了恢复模式，当领导者被选举出来，且大多数 Server 完成了和 leader 的状态同步以后，恢复模式就结束了。状态同步保证了 leader 和 Server 具有相同的系统状态。

42、Leader election（选举阶段-选出准 Leader）

Leader election（选举阶段）：节点在一开始都处于选举阶段，只要有一个节点得到超半数节点的票数，它就可以当选准 leader。只有到达 广播阶段（broadcast）准 leader 才会成为真正的 leader。这一阶段的目的是就是为了选出一个准 leader，然后进入下一个阶段

43、Discovery（发现阶段-接受提议、生成 epoch、接受 epoch）

Discovery（发现阶段）：在这个阶段， followers 跟准 leader 进行通信，同步 followers最近接收的事务提议。这个一阶段的主要目的是发现当前大多数节点接收的最新提议，并且准 leader 生成新的 epoch，让 followers 接受，更新它们的 accepted Epoch

一个 follower 只会连接一个 leader， 如果有一个节点 f 认为另一个 follower p 是 leader， f在尝试连接 p 时会被拒绝， f 被拒绝之后，就会进入重新选举阶段

44、Synchronization（同步阶段-同步 follower 副本）

Synchronization（同步阶段）：同步阶段主要是利用 leader 前一阶段获得的最新提议历史，同步集群中所有的副本。 只有当 大多数节点都同步完成，准 leader 才会成为真正的 leader。follower 只会接收 zxid 比自己的 lastZxid 大的提议。

45、Broadcast（广播阶段-leader 消息广播）

Broadcast（广播阶段）：到了这个阶段， Zookeeper 集群才能正式对外提供事务服务，并且 leader 可以进行消息广播。同时如果有新的节点加入，还需要对新节点进行同步

46、ZAB 协议 JAVA 实现（FLE-发现阶段和同步合并为 Recovery Phase（恢复阶段））

协议的 Java 版本实现跟上面的定义有些不同，选举阶段使用的是 Fast Leader Election（FLE），它包含了 选举的发现职责。因为 FLE 会选举拥有最新提议历史的节点作为 leader，这样就省去了发现最新提议的步骤。实际的实现将 发现阶段 和 同步合并为 Recovery Phase（恢复阶段）。所以， ZAB 的实现只有三个阶段： Fast Leader Election； Recovery Phase； Broadcast Phase。

47、投票机制

每个 sever 首先给自己投票， 然后用自己的选票和其他 sever 选票对比， 权重大的胜出，使用权
重较大的更新自身选票箱。 具体选举过程如下：

1. 每个 Server 启动以后都询问其它的 Server 它要投票给谁。对于其他 server 的询问，server 每次根据自己的状态都回复自己推荐的 leader 的 id 和上一次处理事务的 zxid（系统启动时每个 server 都会推荐自己）
2. 收到所有 Server 回复以后，就计算出 zxid 最大的哪个 Server，并将这个 Server 相关信息设置成下一次要投票的 Server。
3. 计算这过程中获得票数最多的的 sever 为获胜者，如果获胜者的票数超过半数，则改server 被选为 leader。否则，继续这个过程，直到 leader 被选举出来
4. leader 就会开始等待 server 连接
5. Follower 连接 leader，将最大的 zxid 发送给 leader
6. Leader 根据 follower 的 zxid 确定同步点，至此选举阶段完成。
7. 选举阶段完成 Leader 同步后通知 follower 已经成为 uptodate 状态
8. Follower 收到 uptodate 消息后，又可以重新接受 client 的请求进行服务了

目前有 5 台服务器，每台服务器均没有数据，它们的编号分别是 1,2,3,4,5,按编号依次启动，它们
的选择举过程如下：

1. 服务器 1 启动，给自己投票，然后发投票信息，由于其它机器还没有启动所以它收不到反馈信息，服务器 1 的状态一直属于 Looking。
2. 服务器 2 启动，给自己投票，同时与之前启动的服务器 1 交换结果，由于服务器 2 的编号大所以服务器 2 胜出，但此时投票数没有大于半数，所以两个服务器的状态依然是 LOOKING。
3. 服务器 3 启动，给自己投票，同时与之前启动的服务器 1,2 交换信息，由于服务器 3 的编号最大所以服务器 3 胜出，此时投票数正好大于半数，所以服务器 3 成为领导者，服务器1,2 成为小弟。
4. 服务器 4 启动，给自己投票，同时与之前启动的服务器 1,2,3 交换信息，尽管服务器 4 的编号大，但之前服务器 3 已经胜出，所以服务器 4 只能成为小弟。
5. 服务器 5 启动，后面的逻辑同服务器 4 成为小弟

48、Zookeeper 工作原理（原子广播）

1. Zookeeper 的核心是原子广播，这个机制保证了各个 server 之间的同步。实现这个机制的协议叫做 Zab 协议。 Zab 协议有两种模式，它们分别是恢复模式和广播模式。
2. 当服务启动或者在领导者崩溃后， Zab 就进入了恢复模式，当领导者被选举出来，且大多数 server 的完成了和 leader 的状态同步以后，恢复模式就结束了。
3. 状态同步保证了 leader 和 server 具有相同的系统状态
4. 一旦 leader 已经和多数的 follower 进行了状态同步后，他就可以开始广播消息了，即进入广播状态。这时候当一个 server 加入 zookeeper 服务中，它会在恢复模式下启动，发现 leader，并和 leader 进行状态同步。待到同步结束，它也参与消息广播。 Zookeeper服务一直维持在 Broadcast 状态，直到 leader 崩溃了或者 leader 失去了大部分的followers 支持。
5. 广播模式需要保证 proposal 被按顺序处理，因此 zk 采用了递增的事务 id 号(zxid)来保证。所有的提议(proposal)都在被提出的时候加上了 zxid。
6. 实现中 zxid 是一个 64 为的数字，它高 32 位是 epoch 用来标识 leader 关系是否改变，每次一个 leader 被选出来，它都会有一个新的 epoch。低 32 位是个递增计数。
7. 当 leader 崩溃或者 leader 失去大多数的 follower，这时候 zk 进入恢复模式，恢复模式需要重新选举出一个新的 leader，让所有的 server 都恢复到一个正确的状态

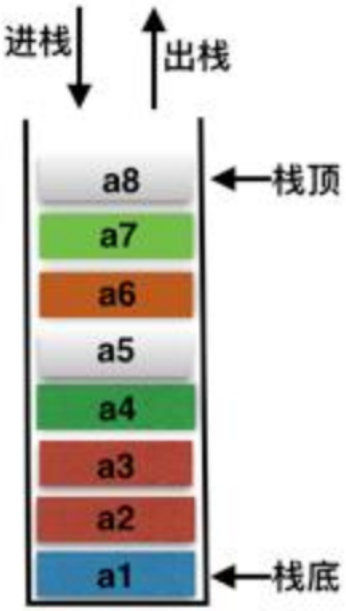
49、Znode 有四种形式的目录节点

1. PERSISTENT：持久的节点。
2. EPHEMERAL： 暂时的节点。
3. PERSISTENT_SEQUENTIAL：持久化顺序编号目录节点。
4. EPHEMERAL_SEQUENTIAL：暂时化顺序编号目录节点。

数据结构面试题

1、栈（stack）

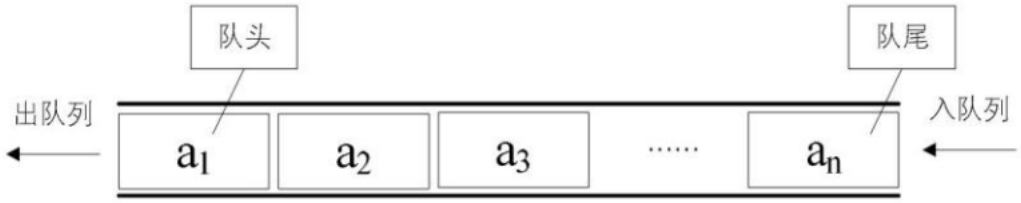
栈（ stack）是限制插入和删除只能在一个位置上进行的表，该位置是表的末端，叫做栈顶（top）。它是后进先出（LIFO）的。对栈的基本操作只有 push（进栈）和 pop（出栈）两种，前者相当于插入，后者相当于删除最后的元素。



图：栈的存储结构示意图

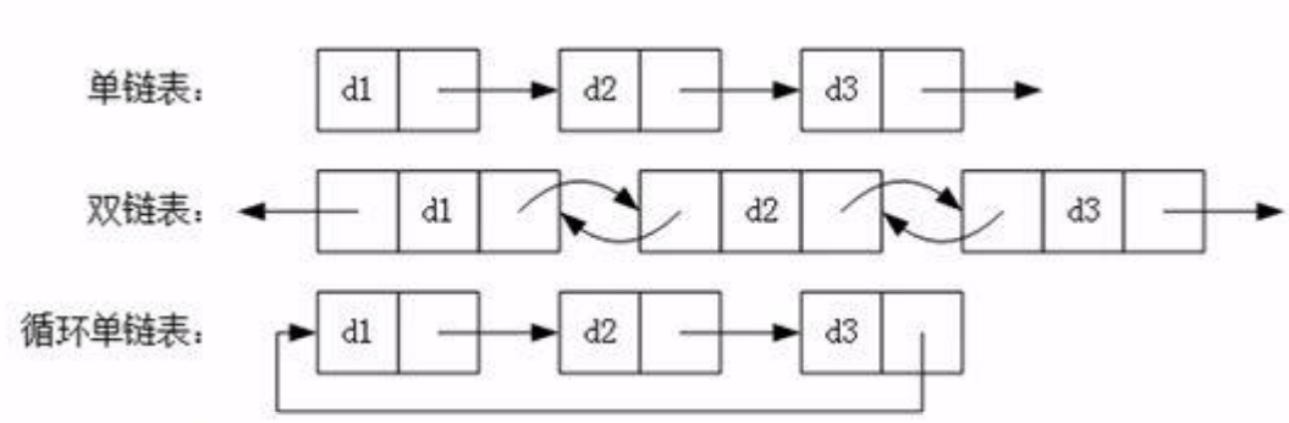
2、队列（queue）

队列是一种特殊的线性表，特殊之处在于它只允许在表的前端（front）进行删除操作，而在表的后端（rear）进行插入操作，和栈一样，队列是一种操作受限制的线性表。进行插入操作的端称为队尾，进行删除操作的端称为队头。



3、链表（Link）

链表是一种数据结构，和数组同级。比如， Java 中我们使用的 ArrayList，其实现原理是数组。而LinkedList 的实现原理就是链表了。链表在进行循环遍历时效率不高，但是插入和删除时优势明显。



4、散列表（Hash Table）

散列表（Hash table，也叫哈希表）是一种查找算法，与链表、树等算法不同的是，散列表算法在查找时不需要进行一系列和关键字（关键字是数据元素中某个数据项的值，用以标识一个数据元素）的比较操作。
散列表算法希望能尽量做到不经过任何比较，通过一次存取就能得到所查找的数据元素，因而必须要在数据元素的存储位置和它的关键字（可用 key 表示）之间建立一个确定的对应关系，使每个关键字和散列表中一个唯一的存储位置相对应。因此在查找时，只要根据这个对应关系找到给定关键字在散列表中的位置即可。这种对应关系被称为散列函数(可用 h(key)表示)。
用的构造散列函数的方法有：

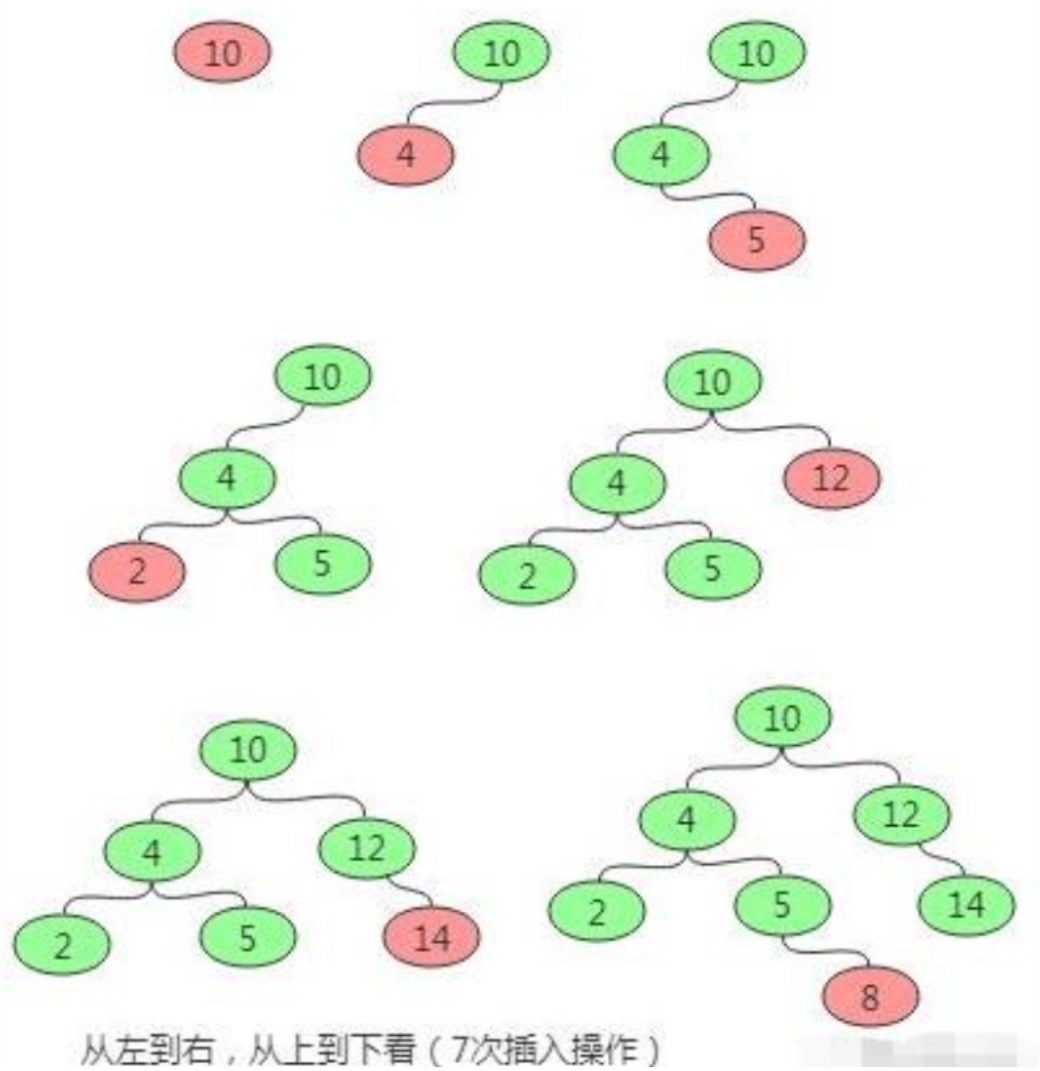
1）直接定址法： 取关键字或关键字的某个线性函数值为散列地址。
即： h(key) = key 或 h(key) = a * key + b， 其中 a 和 b 为常数。
（2）数字分析法
（3）平方取值法： 取关键字平方后的中间几位为散列地址。
（4）折叠法： 将关键字分割成位数相同的几部分，然后取这几部分的叠加和作为散列地址。
（5）除留余数法： 取关键字被某个不大于散列表表长 m 的数 p 除后所得的余数为散列地址，
即： h(key) = key MOD p p ≤ m
（6）随机数法： 选择一个随机函数，取关键字的随机函数值为它的散列地址，
即： h(key) = random(key)

5、排序二叉树

首先如果普通二叉树每个节点满足：左子树所有节点值小于它的根节点值，且右子树所有节点值大于它的根节点值，则这样的二叉树就是排序二叉树。

插入操作

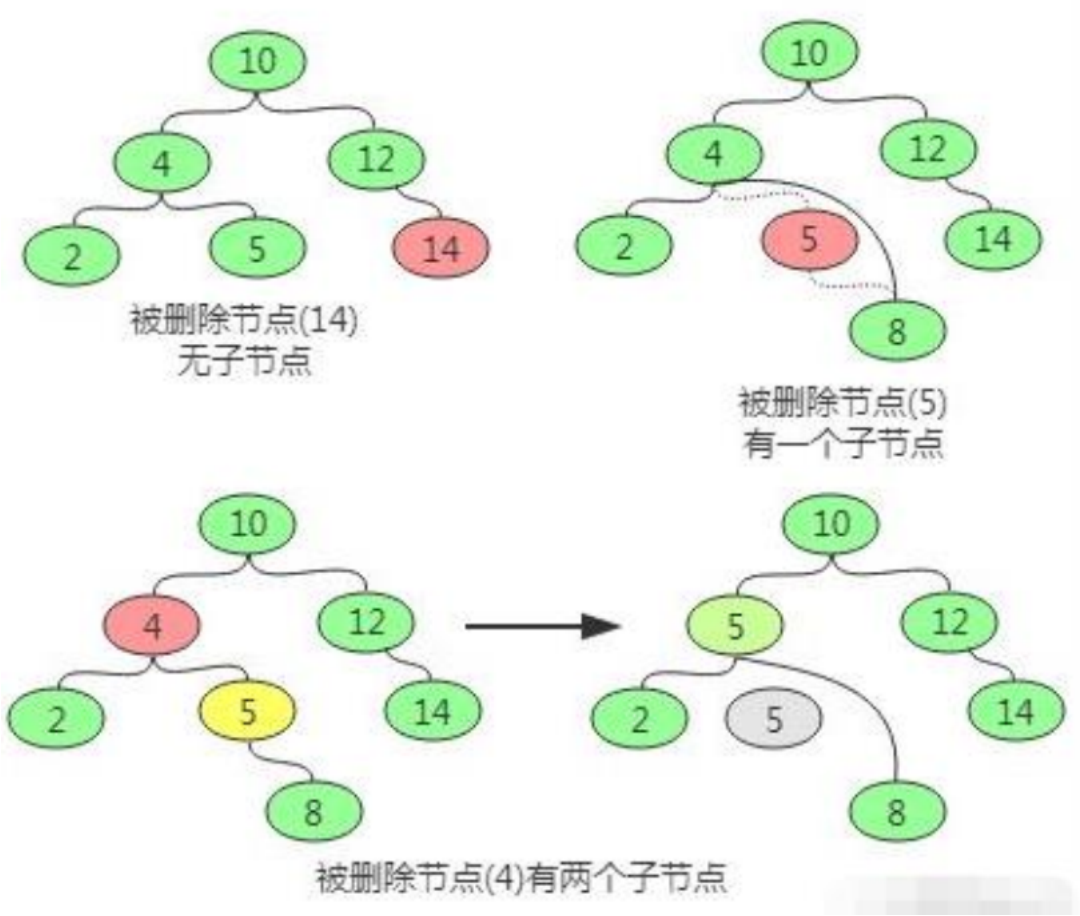
首先要从根节点开始往下找到自己要插入的位置（即新节点的父节点）；具体流程是：新节点与当前节点比较，如果相同则表示已经存在且不能再重复插入；如果小于当前节点，则到左子树中 寻找，如果左子树为空则当前节点为要找的父节点，新节点插入到当前节点的左子树即可；如果大于当前节点，则到右子树中寻找，如果右子树为空则当前节点为要找的父节点，新节点插入到当前节点的右子树即可。



删除操作

删除操作主要分为三种情况， 即要删除的节点无子节点，要删除的节点只有一个子节点，要删除的节点有两个子节点。

1. 对于要删除的节点无子节点可以直接删除，即让其父节点将该子节点置空即可。
2. 对于要删除的节点只有一个子节点，则替换要删除的节点为其子节点。
3. 对于要删除的节点有两个子节点， 则首先找该节点的替换节点（即右子树中最小的节点），接着替换要删除的节点为替换节点，然后删除替换节点。

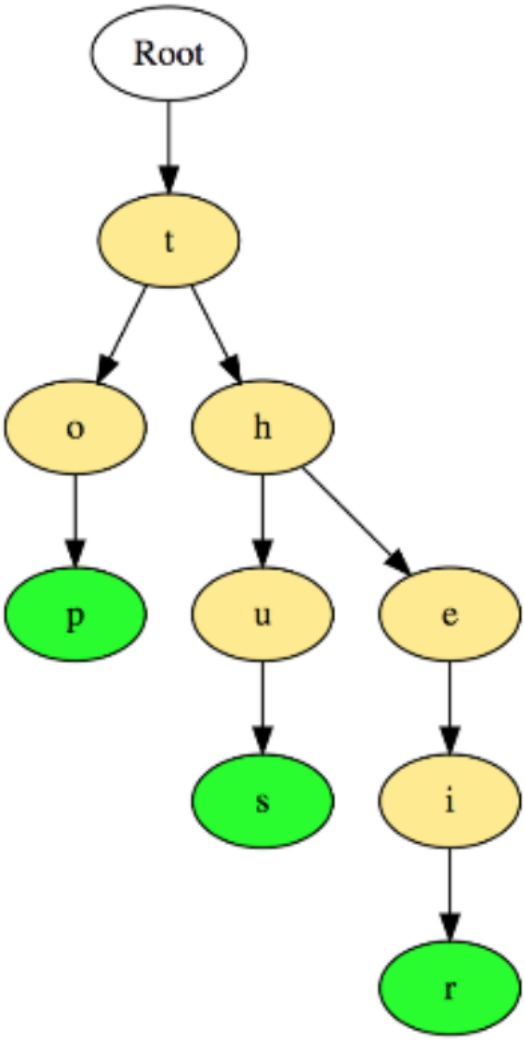


查询操作

查找操作的主要流程为：先和根节点比较，如果相同就返回， 如果小于根节点则到左子树中归查找，如果大于根节点则到右子树中递归查找。因此在排序二叉树中可以很容易获取最大（最右最深子节点）和最小（最左最深子节点）值

6、 前缀树

前缀树(Prefix Trees 或者 Trie)与树类似，用于处理字符串相关的问题时非常高效。它可以实现快速检索，常用于字典中的单词查询，搜索引擎的自动补全甚至 IP 路由。
下图展示了“top”, “thus”和“their”三个单词在前缀树中如何存储的：



7、红黑树

红黑树的特性

（1）每个节点或者是黑色，或者是红色。

（2）根节点是黑色。

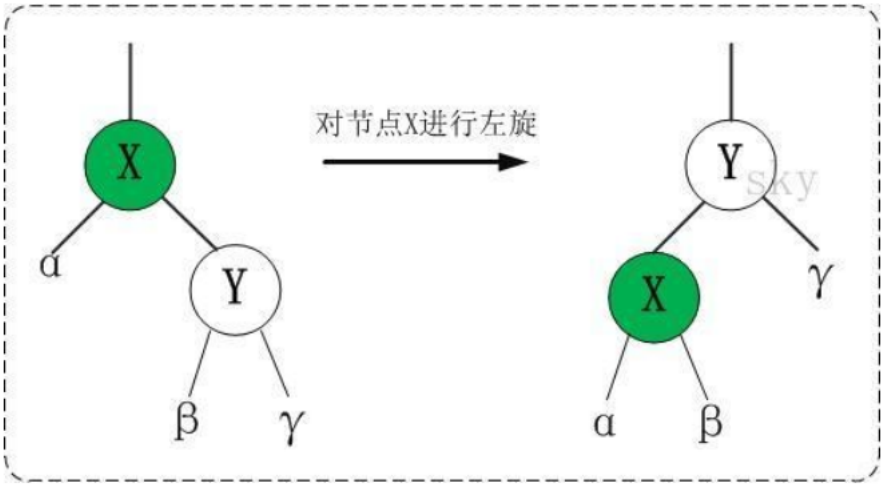
（3）每个叶子节点（NIL）是黑色。 [注意：这里叶子节点，是指为空(NIL 或NULL)的叶子节点！]

（4）如果一个节点是红色的，则它的子节点必须是黑色的。

（5）从一个节点到该节点的子孙节点的所有路径上包含相同数目的黑节点。

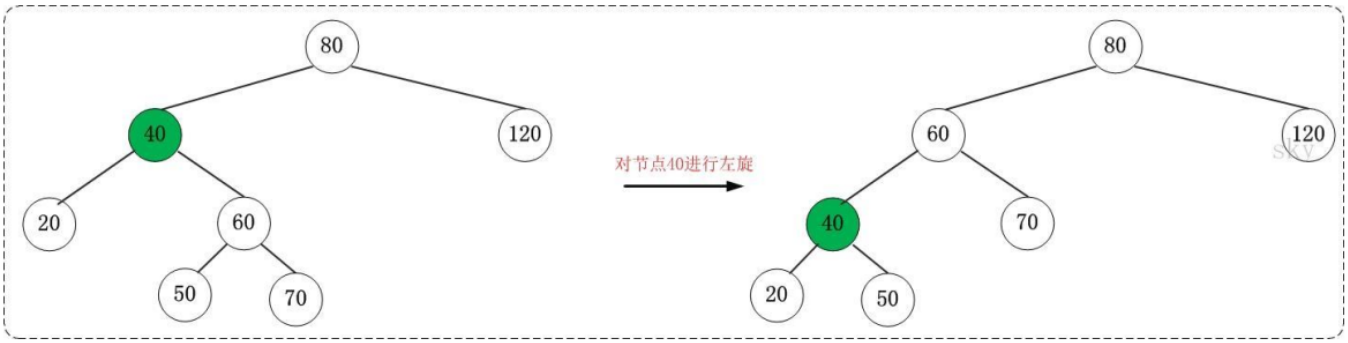
左旋

对 x 进行左旋，意味着，将“x 的右孩子”设为“x 的父亲节点”；即，将 x 变成了一个左节点(x成了为 z 的左孩子)！。 因此，左旋中的“左”，意味着“被旋转的节点将变成一个左节点”。



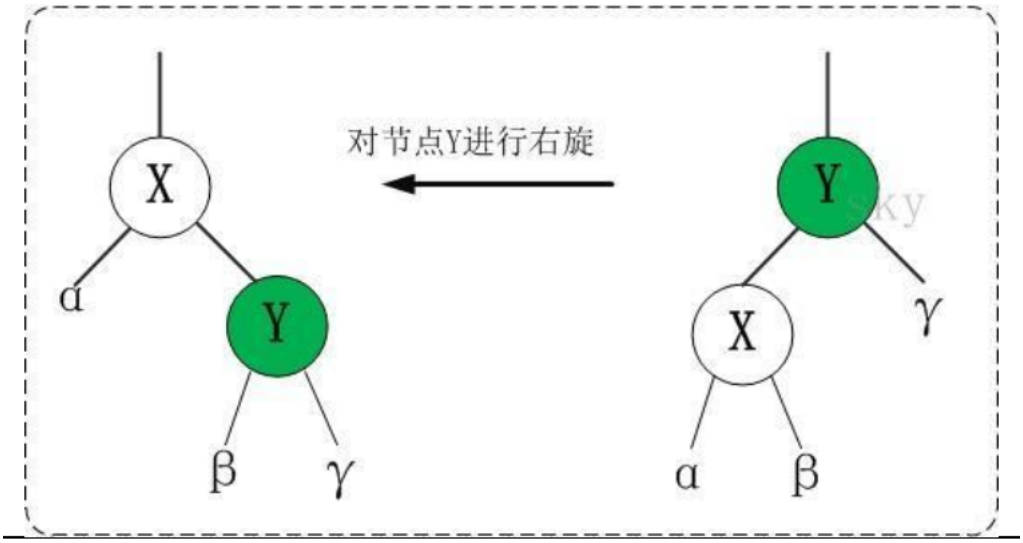

```
LEFT-ROTATE(T, x)
y ← right[x] // 前提：这里假设 x 的右孩子为 y。下面开始正式操作
right[x] ← left[y] // 将 “y 的左孩子” 设为 “x 的右孩子”，即 将β设为 x 的右孩子
p[left[y]] ← x // 将 “x” 设为 “y 的左孩子的父亲”，即 将β的父亲设为 x
```

```
p[y] ← p[x] // 将 “x 的父亲” 设为 “y 的父亲”
if p[x] = nil[T]
then root[T] ← y // 情况 1：如果 “x 的父亲” 是空节点，则将 y 设为根节点
else if x = left[p[x]]
then left[p[x]] ← y // 情况 2：如果 x 是它父节点的左孩子，则将 y 设为“x 的父节点
的左孩子”
else right[p[x]] ← y // 情况 3： (x 是它父节点的右孩子) 将 y 设为“x 的父节点的右孩
子”
left[y] ← x // 将 “x” 设为 “y 的左孩子”
p[x] ← y // 将 “x 的父节点” 设为 “y”
```



右旋

对 x 进行右旋，意味着，将“x 的左孩子”设为“x 的父亲节点”；即，将 x 变成了一个右节点(x成了为 y 的右孩子)！ 因此，右旋中的“右”，意味着“被旋转的节点将变成一个右节点”。




```
RIGHT-ROTATE(T, y)
x ← left[y] // 前提：这里假设 y 的左孩子为 x。下面开始正式操作
left[y] ← right[x] // 将 “x 的右孩子” 设为 “y 的左孩子”，即 将β设为 y 的左孩子
p[right[x]] ← y // 将 “y” 设为 “x 的右孩子的父亲”，即 将β的父亲设为 y
p[x] ← p[y] // 将 “y 的父亲” 设为 “x 的父亲”
if p[y] = nil[T]
then root[T] ← x // 情况 1：如果 “y 的父亲” 是空节点，则将 x 设为根节点
else if y = right[p[y]]
then right[p[y]] ← x // 情况 2：如果 y 是它父节点的右孩子，则将 x 设为“y 的父节
点的左孩子”
else left[p[y]] ← x // 情况 3： (y 是它父节点的左孩子) 将 x 设为“y 的父节点的左孩
子”
right[x] ← y // 将 “y” 设为 “x 的右孩子”
p[y] ← x // 将 “y 的父节点” 设为 “x”
```

添加

第一步: 将红黑树当作一颗二叉查找树，将节点插入。
第二步：将插入的节点着色为"红色"。
根据被插入节点的父节点的情况，可以将"当节点 z 被着色为红色节点，并插入二叉树"划分为三种情况来处理。
① 情况说明：被插入的节点是根节点。
处理方法：直接把此节点涂为黑色。
② 情况说明：被插入的节点的父节点是黑色。
处理方法：什么也不需要做。节点被插入后，仍然是红黑树。

③ 情况说明：被插入的节点的父节点是红色。这种情况下，被插入节点是一定存在非空祖父节点的；进一步的讲，被插入节点也一定存在叔叔节点(即使叔叔节点为空，我们也视之为存在，空节点本身就是黑色节点)。理解这点之后，我们依据"叔叔节点的情况"，将这种情况进一步划分为 3 种情况(Case)

	现象说明	处理策略
Case 1	当前节点的父节点是红色，且当前节点的祖父节点的另一个子节点（叔叔节点）也是红色。	(01) 将“父节点”设为黑色。 (02) 将“叔叔节点”设为黑色。 (03) 将“祖父节点”设为“红色”。 (04) 将“祖父节点”设为“当前节点”(红色节点)；即，之后继续对“当前节点”进行操作。
Case 2	当前节点的父节点是红色，叔叔节点是黑色，且当前节点是其父节点的右孩子	(01) 将“父节点”作为“新的当前节点”。 (02) 以“新的当前节点”为支点进行左旋。
Case 3	当前节点的父节点是红色，叔叔节点是黑色，且当前节点是其父节点的左孩子	(01) 将“父节点”设为“黑色”。 (02) 将“祖父节点”设为“红色”。 (03) 以“祖父节点”为支点进行右旋。

第三步: 通过一系列的旋转或着色等操作，使之重新成为一颗红黑树。

删除

第一步：将红黑树当作一颗二叉查找树， 将节点删除。
这和"删除常规二叉查找树中删除节点的方法是一样的"。分 3 种情况：
① 被删除节点没有儿子，即为叶节点。那么，直接将该节点删除就 OK 了。
② 被删除节点只有一个儿子。那么，直接删除该节点，并用该节点的唯一子节点顶替它的位置。
③ 被删除节点有两个儿子。那么，先找出它的后继节点；然后把“它的后继节点的内容”复制给“该节点的内容”；之后，删除“它的后继节点”。
第二步：通过"旋转和重新着色"等一系列来修正该树，使之重新成为一棵红黑树。

因为"第一步"中删除节点之后，可能会违背红黑树的特性。所以需要通过"旋转和重新着色"来修正
该树，使之重新成为一棵红黑树。
选择重着色 3 种情况。
① 情况说明： x 是“红+黑”节点。
处理方法：直接把 x 设为黑色，结束。此时红黑树性质全部恢复。
② 情况说明： x 是“黑+黑”节点，且 x 是根。
处理方法：什么都不做，结束。此时红黑树性质全部恢复。
③ 情况说明： x 是“黑+黑”节点，且 x 不是根。
处理方法：这种情况又可以划分为 4 种子情况。这 4 种子情况如下表所示：

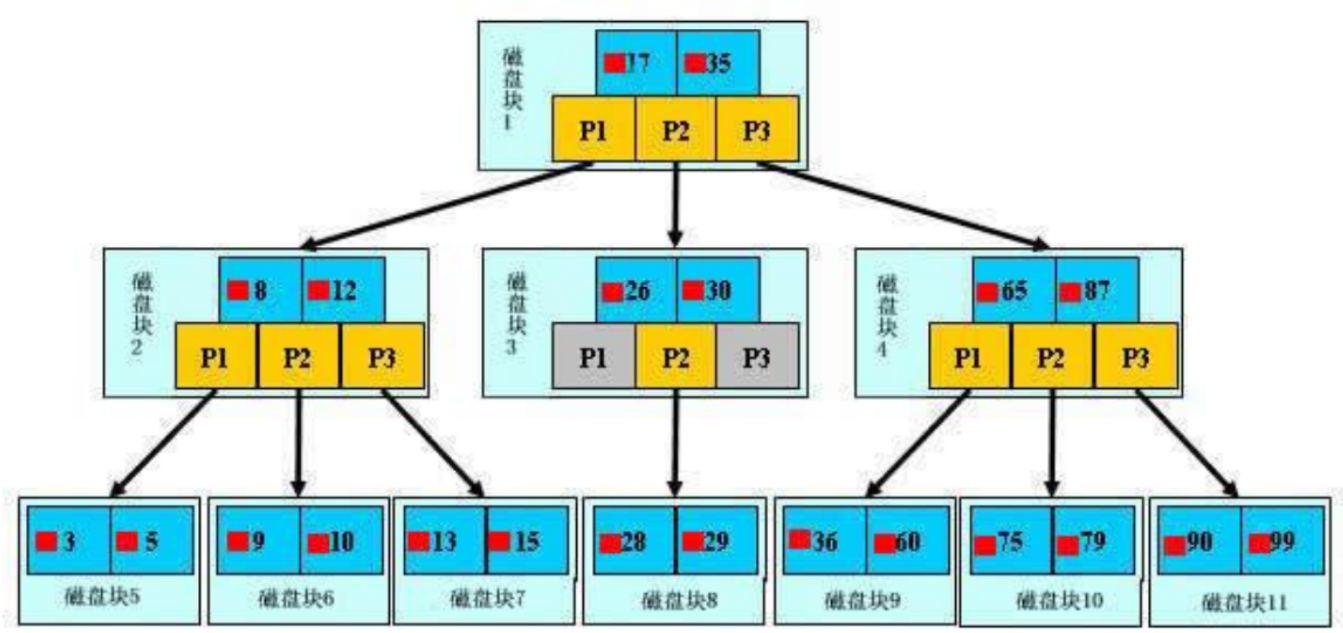
	现象说明	处理策略
Case 1	x是"黑+黑"节点，x的兄弟节点是红色。(此时x的父节点和x的兄弟节点的子节点都是黑节点)。	(01) 将x的兄弟节点设为“黑色”。 (02) 将x的父节点设为“红色”。 (03) 对x的父节点进行左旋。 (04) 左旋后，重新设置x的兄弟节点。
Case 2	x是“黑+黑”节点，x的兄弟节点是黑色，x的兄弟节点的两个孩子都是黑色。	(01) 将x的兄弟节点设为“红色”。 (02) 设置“x的父节点”为“新的x节点”。
Case 3	x是“黑+黑”节点，x的兄弟节点是黑色；x的兄弟节点的左孩子是红色，右孩子是黑色的。	(01) 将x兄弟节点的左孩子设为“黑色”。 (02) 将x兄弟节点设为“红色”。 (03) 对x的兄弟节点进行右旋。 (04) 右旋后，重新设置x的兄弟节点。
Case 4	x是“黑+黑”节点，x的兄弟节点是黑色；x的兄弟节点的右孩子是红色的，x的兄弟节点的左孩子任意颜色。	(01) 将x父节点颜色 赋值给 x的兄弟节点。 (02) 将x父节点设为“黑色”。 (03) 将x兄弟节点的右子节点设为“黑色”。 (04) 对x的父节点进行左旋。 (05) 设置“x”为“根节点”。

参考： https://www.jianshu.com/p/038585421b73
代码实现： https://www.cnblogs.com/skywang12345/p/3624343.html

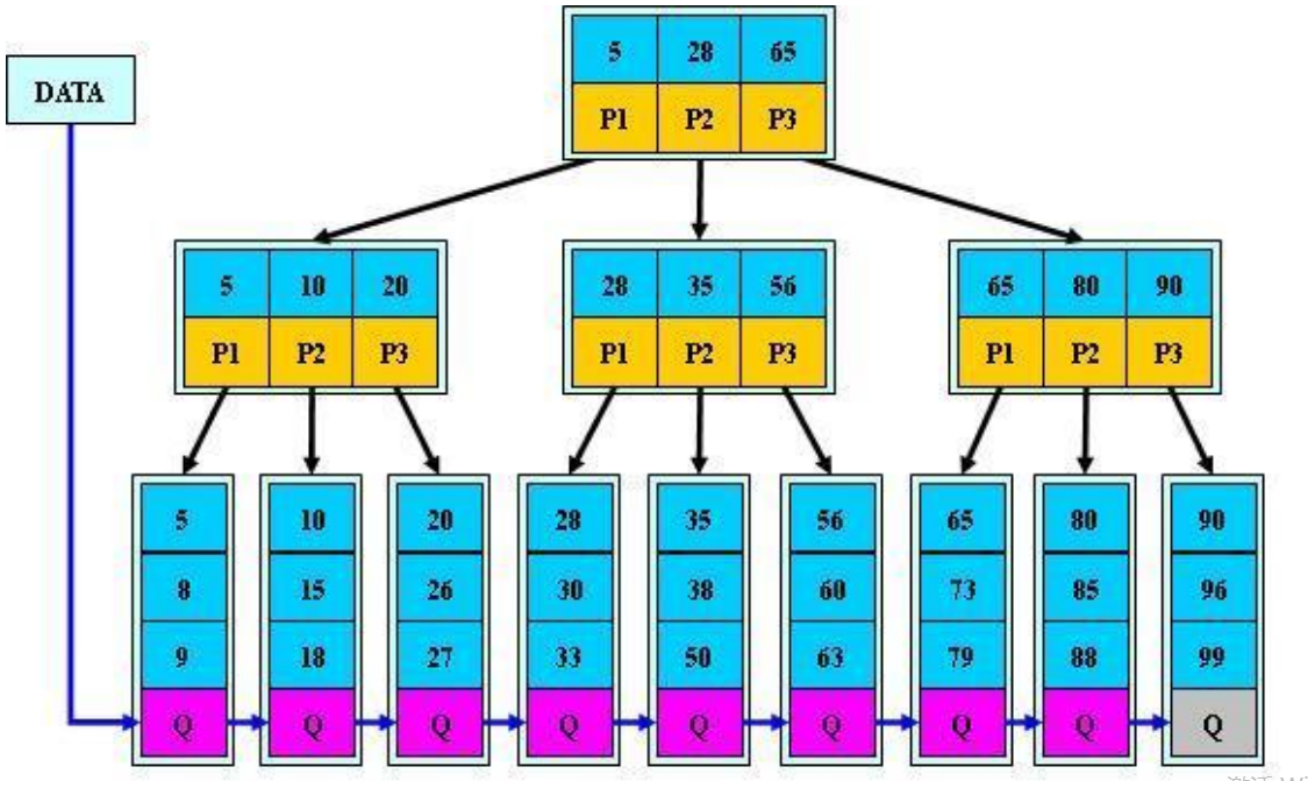
8、B-TREE

B-tree 又叫平衡多路查找树。一棵 m 阶的 B-tree (m 叉树)的特性如下（其中 ceil(x)是一个取上限的函数）：

1. 树中每个结点至多有 m 个孩子；
2. 除根结点和叶子结点外，其它每个结点至少有有 ceil(m / 2)个孩子；
3. 若根结点不是叶子结点，则至少有 2 个孩子（特殊情况：没有孩子的根结点，即根结点为叶子结点，整棵树只有一个根节点）；
4. 所有叶子结点都出现在同一层，叶子结点不包含任何关键字信息(可以看做是外部结点或查询失败的结点，实际上这些结点不存在，指向这些结点的指针都为 null)；
5. 每个非终端结点中包含有 n 个关键字信息：(n， P0， K1， P1， K2， P2， ……， Kn， Pn)。其中：
   a) Ki (i=1…n)为关键字，且关键字按顺序排序 K(i-1)< Ki。
   b) Pi 为指向子树根的接点，且指针 P(i-1)指向子树种所有结点的关键字均小于 Ki，但都大于 K(i-1)。
   c) 关键字的个数 n 必须满足： ceil(m / 2)-1 <= n <= m-1。



一棵 m 阶的 B+tree 和 m 阶的 B-tree 的差异在于：
1.有 n 棵子树的结点中含有 n 个关键字； (B-tree 是 n 棵子树有 n-1 个关键字)
2.所有的叶子结点中包含了全部关键字的信息，及指向含有这些关键字记录的指针，且叶子结点本身依关键字的大小自小而大的顺序链接。 (B-tree 的叶子节点并没有包括全部需要查找的信息)
3.所有的非终端结点可以看成是索引部分，结点中仅含有其子树根结点中最大（或最小）关键字。(B-tree 的非终节点也包含需要查找的有效信息)



参考： https://www.jianshu.com/p/1ed61b4cca12

9、位图

位图的原理就是用一个 bit 来标识一个数字是否存在，采用一个 bit 来存储一个数据，所以这样可以大大的节省空间。 bitmap 是很常用的数据结构， 比如用于 Bloom Filter 中；用于无重复整数的排序等等。 bitmap 通常基于数组来实现，数组中每个元素可以看成是一系列二进制数，所有元素组成更大的二进制集合。
https://www.cnblogs.com/polly333/p/4760275.html

算法面试题

1、数据里有{1,2,3,4,5,6,7,8,9}，请随机打乱顺序，生成一个新的数组（请以代码实现）

```
import java.util.Arrays;
//打乱数组
public class Demo1 {
    //随机打乱
    public static int[] srand(int[] a) {
        int[] b = new int[a.length];
        for(int i = 0; i < a.length;i++) {
            //随机获取下标
            int tmp = (int)(Math.random()*(a.length - i)); //随机数:[ 0 ,
            a.length - i )
            b[i] = a[tmp];
            //将此时a[tmp]的下标移动到靠后的位置
            int change = a[a.length - i - 1];
            a[a.length - i - 1] = a[tmp];
            a[tmp] = change;
        }
        return b;
    }
    public static void main(String[] args) {
        int[] a = {1,2,3,4,5,6,7,8,9};
        System.out.println(Arrays.toString(srand(a)));
    }
}
```

2、写出代码判断一个整数是不是2的阶次方（请代码实现，谢绝调用API方法）

```
import java.util.Scanner;
//判断整数是不是2的阶次方
public class Demo2 {
    public static boolean check(int sum) {
        boolean flag = true; //判断标志
        while(sum > 1) {
            if (sum % 2 == 0) {
                sum = sum/2;
            } else {
                flag = false;
                break;
            }
        }
        return flag;
    }
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.println("请输入一个整数:");
        int sum = scanner.nextInt();
        System.out.println(sum + " 是不是2的阶次方：" + check(sum));
    }
}
```

3、假设今日是2015年3月1日，星期日，请算出13个月零6天后是星期几，距离现在多少天（请用代码实现，谢绝调用API方法）

```
i
mport java.util.Scanner;
//算出星期几
public class Demo4 {
    public static String[] week = {"星期日","星期一","星期二","星期三","星期四","星期五","星期六"};
    public static int i = 0;
    public static int[] monthday1 = {0,31,28,31,30,31,30,31,31,30,31,30,31};
    public static int[] monthday2 = {0,31,29,31,30,31,30,31,31,30,31,30,31};
    //查看距离当前天数的差值
    public static String distance(int year,int month,int day,int newMonth,int newDay) {
        int sum = 0; //设定初始距离天数
        if (month + newMonth >= 12) {
            if (((year + 1) % 4 == 0 && (year + 1) % 100 != 0)||(year + 1) % 400 == 0) {
                sum += 366 + newDay;
                for(int i = 0;i < newMonth - 12;i++) {
                    sum += monthday1[month + i];
                }
            } else {
                sum += 365 + newDay;
                for(int i = 0;i < newMonth - 12;i++) {
                    sum += monthday1[month + i];
                }
            }
        }
        else {
            for(int i = 0;i < newMonth;i++) {
                sum += monthday1[month + i];
            }
            sum += newDay;
        }
        return week[sum%7];
    }
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.println("请输入当前年份");
        int year = scanner.nextInt();
        System.out.println("请输入当前月份");
        int month = scanner.nextInt();
        System.out.println("请输入当前天数");
        int day = scanner.nextInt();
        System.out.println("请输入当前是星期几：以数字表示，如：星期天 为 0");
        int index = scanner.nextInt();
        System.out.println("今天是：" + year + "-" + month + "-" + day + " " +
        week[index]);
        System.err.println("请输入相隔月份");
        int newMonth = scanner.nextInt();
        System.out.println("请输入剩余天数");
        int newDay = scanner.nextInt();
        System.out.println("经过" + newMonth + "月" + newDay + "天后，是" +
        distance(year,month,day,newMonth,newDay));
    }
}
```

4、有两个篮子，分别为A 和 B，篮子A里装有鸡蛋，篮子B里装有苹果，请用面向对象的思想实现两个篮子里的物品交换（请用代码实现）

```
//面向对象思想实现篮子物品交换
public class Demo5 {
    public static void main(String[] args) {
        //创建篮子
        Basket A = new Basket("A");
        Basket B = new Basket("B");
        //装载物品
        A.load("鸡蛋");
        B.load("苹果");
        //交换物品
        A.change(B);
        A.show();
        B.show();
    }
}

class Basket{
    public String name; //篮子名称
    private Goods goods; //篮子中所装物品
    public Basket(String name) {
        // TODO Auto-generated constructor stub
        this.name = name;
        System.out.println(name + "篮子被创建");
```

```
    }
    //装物品函数
    public void load(String name) {
        goods = new Goods(name);
        System.out.println(this.name + "装载了" + name + "物品");
    }
    public void change(Basket B) {
        System.out.println(this.name + " 和 " + B.name + "中的物品发生了交换");
        String tmp = this.goods.getName();
        this.goods.setName(B.goods.getName());
        B.goods.setName(tmp);
    }

    public void show() {
        System.out.println(this.name + "中有" + goods.getName() + "物品");
    }
}

class Goods{
    private String name; //物品名称
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public Goods(String name) {
        // TODO Auto-generated constructor stub
        this.name = name;
    }
}
```

5、二分查找

又叫折半查找，要求待查找的序列有序。每次取中间位置的值与待查关键字比较，如果中间位置的值比待查关键字大，则在前半部分循环这个查找的过程，如果中间位置的值比待查关键字小，则在后半部分循环这个查找的过程。直到查找到了为止，否则序列中没有待查的关键字。

```
public static int biSearch(int []array,int a){
    int lo=0;
    int hi=array.length-1;
    int mid;
    while(lo<=hi){
        mid=(lo+hi)/2;//中间位置
        if(array[mid]==a){
            return mid+1;
        }else if(array[mid]<a){ //向右查找
            lo=mid+1;
        }else{ //向左查找
            hi=mid-1;
        }
    }
    return -1;
}
```

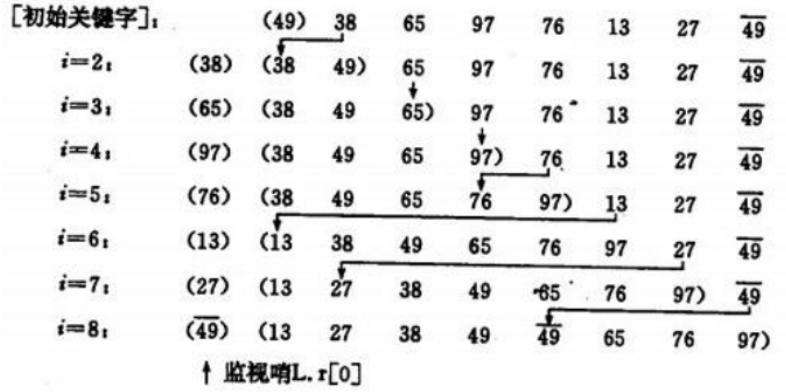
6、冒泡排序算法

1）比较前后相邻的二个数据，如果前面数据大于后面的数据，就将这二个数据交换。
（2）这样对数组的第 0 个数据到 N-1 个数据进行一次遍历后，最大的一个数据就"沉"到数组第N-1 个位置。
（3） N=N-1，如果 N 不为 0 就重复前面二步，否则排序完成。

```
public static void bubbleSort1(int [] a, int n){
    int i, j;
    for(i=0; i<n; i++){//表示 n 次排序过程。
        for(j=1; j<n-i; j++){
            if(a[j-1] > a[j]){//前面的数字大于后面的数字就交换
                //交换 a[j-1]和 a[j]
                int temp;
                temp = a[j-1];
                a[j-1] = a[j];
                a[j]=temp;
            }
        }
    }
}
```

7、插入排序算法

通过构建有序序列，对于未排序数据，在已排序序列中从后向前扫描，找到相应的位置并插入。插入排序非常类似于整扑克牌。在开始摸牌时，左手是空的，牌面朝下放在桌上。接着， 一次从桌上摸起一张牌，并将它插入到左手一把牌中的正确位置上。 为了找到这张牌的正确位置，要将它与手中已有的牌从右到左地进行比较。无论什么时候，左手中的牌都是排好序的。如果输入数组已经是排好序的话，插入排序出现最佳情况，其运行时间是输入规模的一个线性函数。如果输入数组是逆序排列的，将出现最坏情况。平均情况与最坏情况一样，其时间代价是(n2)。




```
public void sort(int arr[]){
    for(int i =1; i<arr.length;i++)
    {
        //插入的数
        int insertVal = arr[i];
        //被插入的位置(准备和前一个数比较)
        int index = i-1;
        //如果插入的数比被插入的数小
        while(index>=0&&insertVal<arr[index])
        {
            //将把 arr[index] 向后移动
            arr[index+1]=arr[index];
            //让 index 向前移动
            index--;
        }
        //把插入的数放入合适位置
        arr[index+1]=insertVal;
    }
}
```

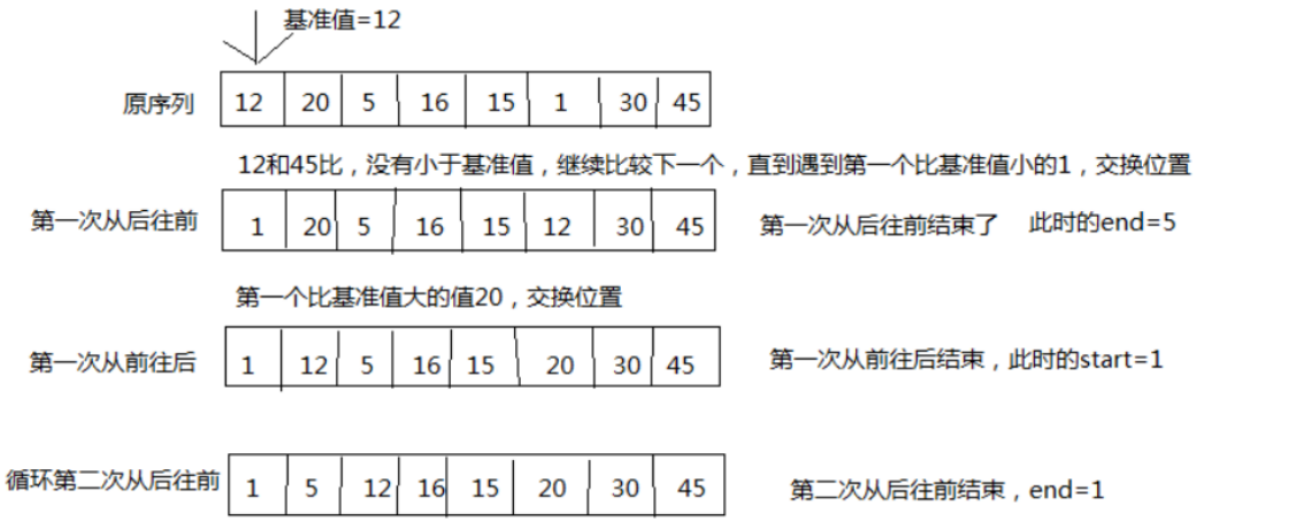
8、快速排序算法

快速排序的原理：选择一个关键值作为基准值。比基准值小的都在左边序列（一般是无序的），比基准值大的都在右边（一般是无序的）。 一般选择序列的第一个元素。

一次循环： 从后往前比较，用基准值和最后一个值比较，如果比基准值小的交换位置，如果没有继续比较下一个，直到找到第一个比基准值小的值才交换。 找到这个值之后，又从前往后开始比较，如果有比基准值大的，交换位置，如果没有继续比较下一个，直到找到第一个比基准值大的值才交换。直到从前往后的比较索引>从后往前比较的索引，结束第一次循环，此时，对于基准值来说，左右两边就是有序的了。

```
public void sort(int[] a,int low,int high){
    int start = low;
    int end = high;
    int key = a[low];
    while(end>start){
        //从后往前比较
        while(end>start&&a[end]>=key)
        //如果没有比关键值小的，比较下一个，直到有比关键值小的交换位置，然后又从前往后比较
        end--;
        if(a[end]<=key){
            int temp = a[end];
            a[end] = a[start];
            a[start] = temp;
        }
        //从前往后比较
        while(end>start&&a[start]<=key){
            //如果没有比关键值大的，比较下一个，直到有比关键值大的交换位置
            start++;
            if(a[start]>=key){
                int temp = a[start];
                a[start] = a[end];
                a[end] = temp;
            }
            //此时第一次循环比较结束，关键值的位置已经确定了。左边的值都比关键值小，右边的值都比关键值大，但是两边的顺序还有可能是不一样的，进行下面的递归调用
        }
        //递归
        if(start>low) sort(a,low,start-1);//左边序列。第一个索引位置到关键值索引-1
        if(end<high) sort(a,end+1,high);//右边序列。从关键值索引+1 到最后一个
```

```
    }
}
```

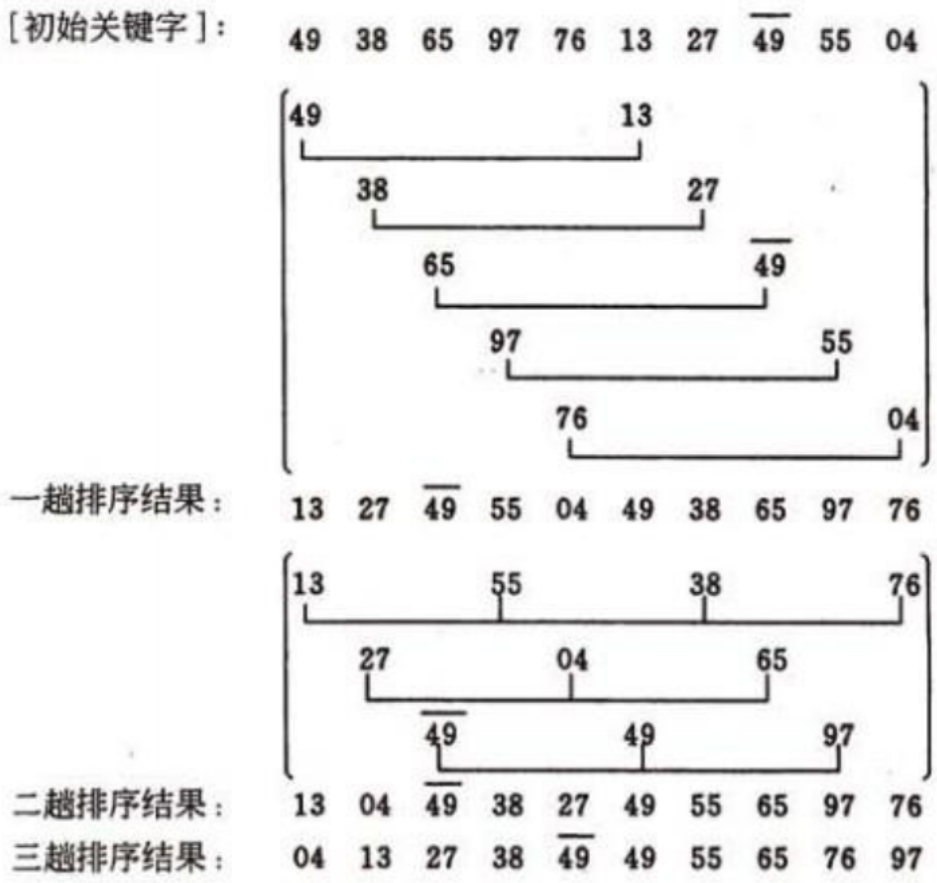


上面的循环之后得到了基准值左右两边的序列了。注意：交换的是起始和结束位置的值，不是基准值

9、希尔排序算法

基本思想：先将整个待排序的记录序列分割成为若干子序列分别进行直接插入排序，待整个序列中的记录"基本有序" 时，再对全体记录进行依次直接插入排序。

1. 操作方法：选择一个增量序列 t1， t2， ... ， tk，其中 ti>tj， tk=1；
2. 按增量序列个数 k，对序列进行 k 趟排序；
3. 每趟排序，根据对应的增量 ti，将待排序列分割成若干长度为 m 的子序列，分别对各子表进行直接插入排序。仅增量因子为1 时，整个序列作为一个表来处理，表长度即为整个序列的长度。

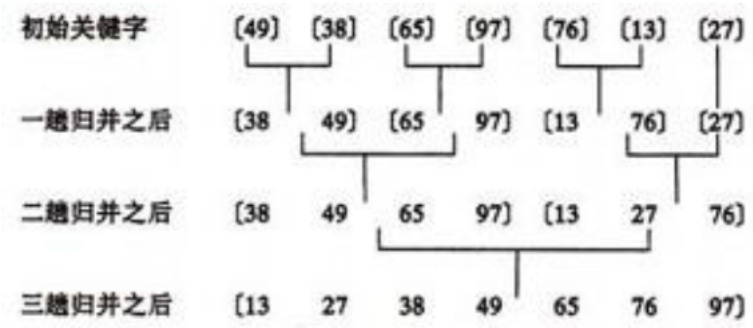



```
private void shellSort(int[] a) {
    int dk = a.length/2;
    while( dk >= 1 ){
```

```
        ShellInsertSort(a, dk);
        dk = dk/2;
    }
}
private void ShellInsertSort(int[] a, int dk) {
    //类似插入排序，只是插入排序增量是 1，这里增量是 dk,把 1 换成 dk 就可以了
    for(int i=dk;i<a.length;i++){
        if(a[i]<a[i-dk]){
            int j;
            int x=a[i];//x 为待插入元素
            a[i]=a[i-dk];
            for(j=i-dk; j>=0 && x<a[j];j=j-dk){
                //通过循环，逐个后移一位找到要插入的位置。
                a[j+dk]=a[j];
            }
            a[j+dk]=x;//插入
        }
    }
}
```

10、归并排序算法

归并（Merge）排序法是将两个（或两个以上）有序表合并成一个新的有序表，即把待排序序列分为若干个子序列，每个子序列是有序的。然后再把有序子序列合并为整体有序序列。




```
public class MergeSortTest {
    public static void main(String[] args) {
        int[] data = new int[] { 5, 3, 6, 2, 1, 9, 4, 8, 7 };
        print(data);
        mergeSort(data);
        System.out.println("排序后的数组： ");
        print(data);
    }
    public static void mergeSort(int[] data) {
        sort(data, 0, data.length - 1);
    }
    public static void sort(int[] data, int left, int right) {
        if (left >= right)
        return;
        // 找出中间索引
        int center = (left + right) / 2;
        // 对左边数组进行递归
        sort(data, left, center);
        // 对右边数组进行递归
        sort(data, center + 1, right);
        // 合并
        merge(data, left, center, right);
        print(data);
    }
    /**
    * 将两个数组进行归并，归并前面 2 个数组已有序，归并后依然有序
    *
    * @param data
    * 数组对象
    * @param left
    * 左数组的第一个元素的索引
    * @param center
    * 左数组的最后一个元素的索引， center+1 是右数组第一个元素的索引
    * @param right
    * 右数组最后一个元素的索引
    */
    public static void merge(int[] data, int left, int center, int right) {
        // 临时数组
        int[] tmpArr = new int[data.length];
        // 右数组第一个元素索引
        int mid = center + 1;
        // third 记录临时数组的索引
        int third = left;
        // 缓存左数组第一个元素的索引
```

```
        int tmp = left;
        while (left <= center && mid <= right) {
            // 从两个数组中取出最小的放入临时数组
            if (data[left] <= data[mid]) {
                tmpArr[third++] = data[left++];
            } else {
                tmpArr[third++] = data[mid++];
            }
        }
        // 剩余部分依次放入临时数组（实际上两个 while 只会执行其中一个）
        while (mid <= right) {
            tmpArr[third++] = data[mid++];
        }
        while (left <= center) {
            tmpArr[third++] = data[left++];
        }
        // 将临时数组中的内容拷贝回原数组中
        // （原 left-right 范围的内容被复制回原数组）
        while (tmp <= right) {
            data[tmp] = tmpArr[tmp++];
        }
    }
    public static void print(int[] data) {
        for (int i = 0; i < data.length; i++) {
            System.out.print(data[i] + "\t");
        }
        System.out.println();
    }
}
```

11、桶排序算法

桶排序的基本思想是： 把数组 arr 划分为 n 个大小相同子区间（桶），每个子区间各自排序，最后合并 。计数排序是桶排序的一种特殊情况，可以把计数排序当成每个桶里只有一个元素的情况。
1.找出待排序数组中的最大值 max、最小值 min
2.我们使用 动态数组 ArrayList 作为桶，桶里放的元素也用 ArrayList 存储。桶的数量为(maxmin)/arr.length+1
3.遍历数组 arr，计算每个元素 arr[i] 放的桶
4.每个桶各自排序

```
public static void bucketSort(int[] arr){
    int max = Integer.MIN_VALUE;
    int min = Integer.MAX_VALUE;
    for(int i = 0; i < arr.length; i++){
        max = Math.max(max, arr[i]);
        min = Math.min(min, arr[i]);
    }
    //创建桶
    int bucketNum = (max - min) / arr.length + 1;
    ArrayList<ArrayList<Integer>> bucketArr = new ArrayList<>(bucketNum);
    for(int i = 0; i < bucketNum; i++){
        bucketArr.add(new ArrayList<Integer>());
    }
    //将每个元素放入桶
    for(int i = 0; i < arr.length; i++){
        int num = (arr[i] - min) / (arr.length);
        bucketArr.get(num).add(arr[i]);
    }
    //对每个桶进行排序
    for(int i = 0; i < bucketArr.size(); i++){
        Collections.sort(bucketArr.get(i));
    }
}
```

12、基数排序算法

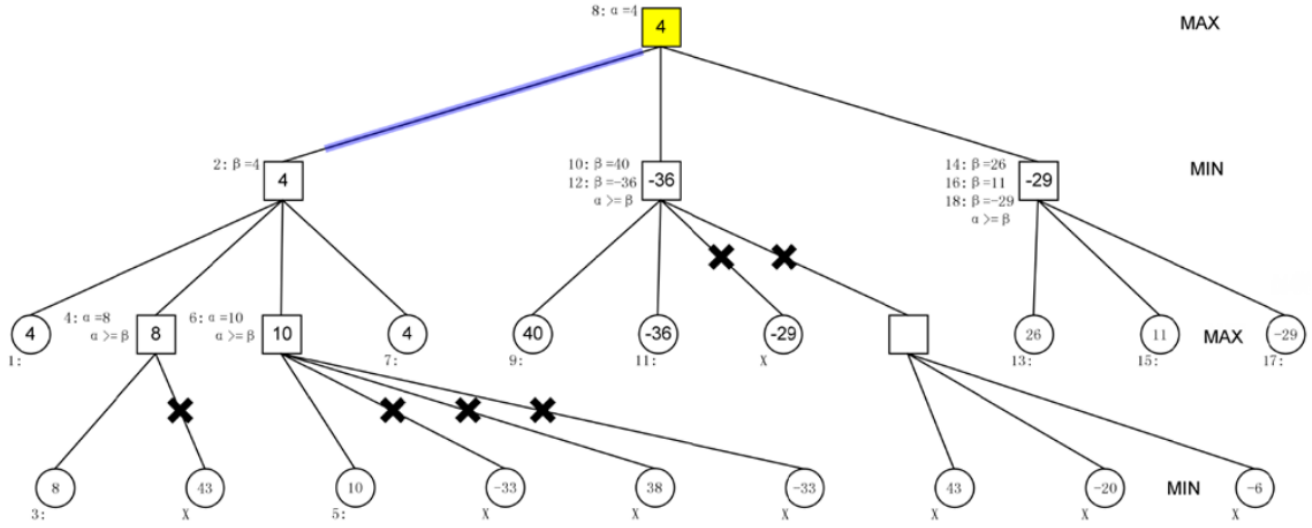
将所有待比较数值（正整数）统一为同样的数位长度，数位较短的数前面补零。然后，从最低位开始，依次进行一次排序。这样从最低位排序一直到最高位排序完成以后,数列就变成一个有序序列。

```
public class radixSort {
    int a[]={49,38,65,97,76,13,27,49,78,34,12,64,5,4,62,99,98,54,101,56,17,18,23,34,15,35,2 5,53,51};
    public radixSort(){
        sort(a);
        for(inti=0;i<a.length;i++){
            System.out.println(a[i]);
        }
    }
    public void sort(int[] array){
        //首先确定排序的趟数;
        int max=array[0];
        for(inti=1;i<array.length;i++){
```

```
            if(array[i]>max){
                max=array[i];
            }
        }
        int time=0;
        //判断位数;
        while(max>0){
            max/=10;
            time++;
        }
        //建立 10 个队列;
        List<ArrayList> queue=newArrayList<ArrayList>();
        for(int i=0;i<10;i++){
            ArrayList<Integer>queue1=new ArrayList<Integer>();
            queue.add(queue1);
        }
        //进行 time 次分配和收集;
        for(int i=0;i<time;i++){
            //分配数组元素;
            for(intj=0;j<array.length;j++){
                //得到数字的第 time+1 位数;
                int x=array[j]%(int)Math.pow(10,i+1)/(int)Math.pow(10, i);
                ArrayList<Integer>queue2=queue.get(x);
                queue2.add(array[j]);
                queue.set(x, queue2);
            }
            int count=0;//元素计数器;
            //收集队列元素;
            for(int k=0;k<10;k++){
                while(queue.get(k).size()>0){
                    ArrayList<Integer>queue3=queue.get(k);
                    array[count]=queue3.get(0);
                    queue3.remove(0);
                    count++;
                }
            }
        }
    }
}
```

13、剪枝算法

在搜索算法中优化中，剪枝，就是通过某种判断，避免一些不必要的遍历过程，形象的说，就是剪去了搜索树中的某些“枝条”，故称剪枝。应用剪枝优化的核心问题是设计剪枝判断方法，即确定哪些枝条应当舍弃，哪些枝条应当保留的方法。



14、回溯算法

回溯算法实际上一个类似枚举的搜索尝试过程，主要是在搜索尝试过程中寻找问题的解，当发现已不满足求解条件时，就“回溯”返回，尝试别的路径。

15、最短路径算法

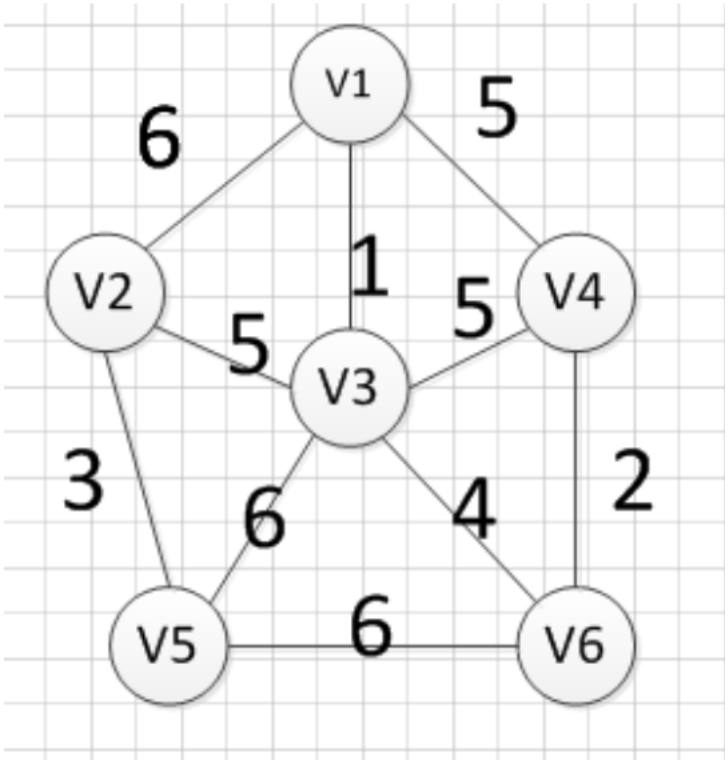
从某顶点出发，沿图的边到达另一顶点所经过的路径中，各边上权值之和最小的一条路径叫做最短路径。解决最短路的问题有以下算法， Dijkstra 算法， Bellman-Ford 算法， Floyd 算法和 SPFA算法等

16、最小生成树算法

现在假设有一个很实际的问题：我们要在 n 个城市中建立一个通信网络，则连通这 n 个城市需要布置 n-1 一条通信线路，这个时候我们需要考虑如何在成本最低的情况下建立这个通信网？

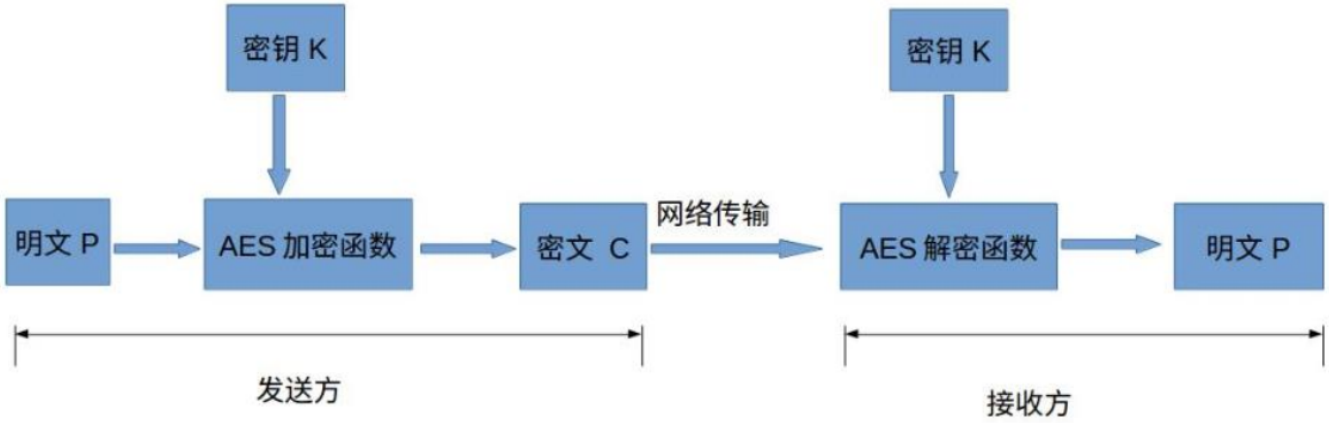
于是我们就可以引入连通图来解决我们遇到的问题， n 个城市就是图上的 n 个顶点，然后，边表示两个城市的通信线路，每条边上的权重就是我们搭建这条线路所需要的成本，所以现在我们有 n 个顶点的连通网可以建立不同的生成树，每一颗生成树都可以作为一个通信网，当我们构造这个连通网所花的成本最小时，搭建该连通网的生成树，就称为最小生成树。

构造最小生成树有很多算法，但是他们都是利用了最小生成树的同一种性质： MST 性质（假设N=(V,{E})是一个连通网， U 是顶点集 V 的一个非空子集，如果（u， v）是一条具有最小权值的边，其中 u 属于 U， v 属于 V-U，则必定存在一颗包含边（u， v）的最小生成树），下面就介绍两种使用 MST 性质生成最小生成树的算法：普里姆算法和克鲁斯卡尔算法。



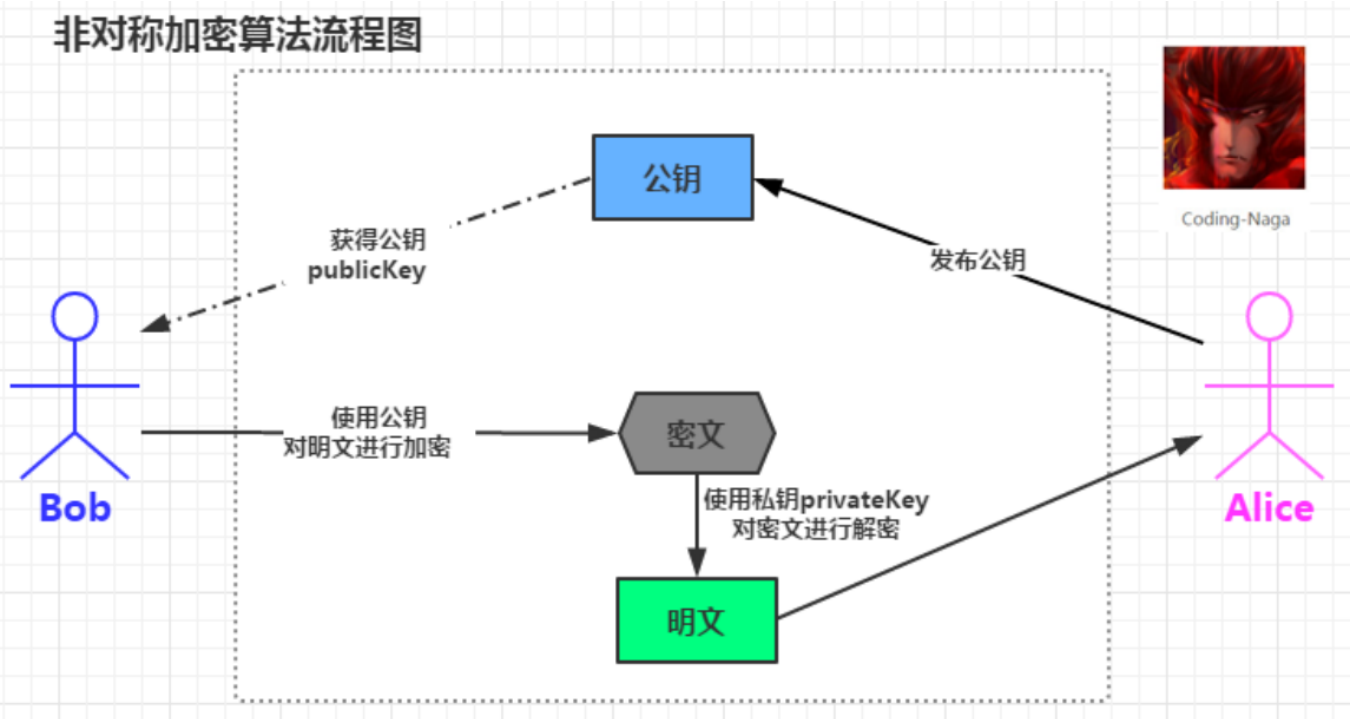
17、AES

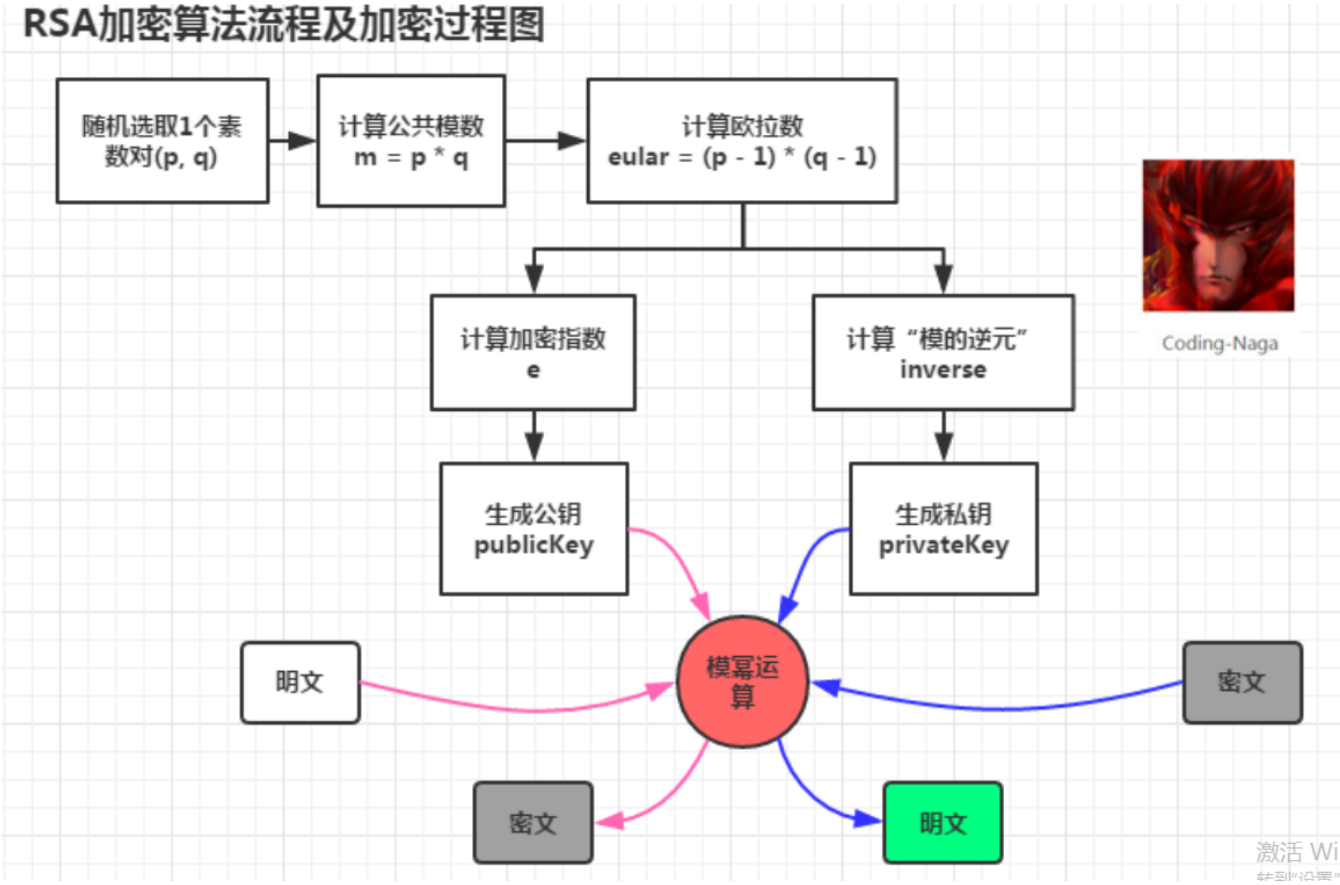
高级加密标准(AES,Advanced Encryption Standard)为最常见的对称加密算法(微信小程序加密传输就是用这个加密算法的)。对称加密算法也就是加密和解密用相同的密钥，具体的加密流程如下图：



18、RSA

RSA 加密算法是一种典型的非对称加密算法，它基于大数的因式分解数学难题，它也是应用最广泛的非对称加密算法。
非对称加密是通过两个密钥（公钥-私钥）来实现对数据的加密和解密的。公钥用于加密，私钥用于解密。





19、CRC

循环冗余校验(Cyclic Redundancy Check, CRC)是一种根据网络数据包或电脑文件等数据产生简短固定位数校验码的一种散列函数，主要用来检测或校验数据传输或者保存后可能出现的错误。它是利用除法及余数的原理来作错误侦测的。

20、MD5

MD5 常常作为文件的签名出现，我们在下载文件的时候，常常会看到文件页面上附带一个扩展名为.MD5 的文本或者一行字符，这行字符就是就是把整个文件当作原数据通过 MD5 计算后的值，我们下载文件后，可以用检查文件 MD5 信息的软件对下载到的文件在进行一次计算。两次结果对比就可以确保下载到文件的准确性。 另一种常见用途就是网站敏感信息加密，比如用户名密码，支付签名等等。随着 https 技术的普及，现在的网站广泛采用前台明文传输到后台， MD5 加密（使用偏移量）的方式保护敏感数据保护站点和数据安全。

21、更多算法练习

更多算法练习题，请访问 https://leetcode-cn.com/problemset/algorithms/

Elasticsearch 面试题

1、elasticsearch 了解多少，说说你们公司 es 的集群架构，索引数据大小，分片有多少，以及一些调优手段 。

面试官：想了解应聘者之前公司接触的 ES 使用场景、规模，有没有做过比较大规模的索引设计、规划、调优。
解答：
如实结合自己的实践场景回答即可。
比如：ES 集群架构 13 个节点，索引根据通道不同共 20+索引，根据日期，每日递增 20+，索引：10 分片，每日递增 1 亿+数据，每个通道每天索引大小控制：150GB 之内。

仅索引层面调优手段：
1.1、设计阶段调优
1、根据业务增量需求，采取基于日期模板创建索引，通过 roll over API 滚动索引；
2、使用别名进行索引管理；
3、每天凌晨定时对索引做 force_merge 操作，以释放空间；

4、采取冷热分离机制，热数据存储到 SSD，提高检索效率；冷数据定期进行 shrink操作，以缩减存储；
5、采取 curator 进行索引的生命周期管理；
6、仅针对需要分词的字段，合理的设置分词器；
7、Mapping 阶段充分结合各个字段的属性，是否需要检索、是否需要存储等。.....

1.2、写入调优

1、写入前副本数设置为 0；
2、写入前关闭 refresh_interval 设置为-1，禁用刷新机制；
3、写入过程中：采取 bulk 批量写入；
4、写入后恢复副本数和刷新间隔；
5、尽量使用自动生成的 id。

1.3、查询调优

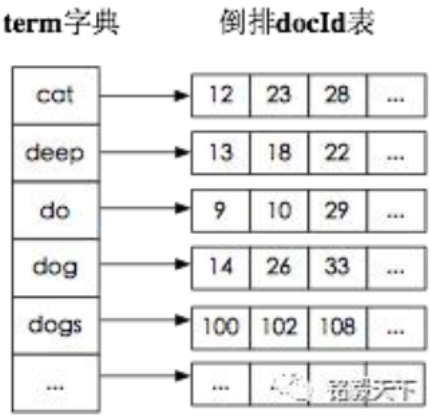
1、禁用 wildcard；
2、禁用批量 terms（成百上千的场景）；
3、充分利用倒排索引机制，能 keyword 类型尽量 keyword；
4、数据量大时候，可以先基于时间敲定索引再检索；

5、设置合理的路由机制。

1.4、其他调优
部署调优，业务调优等。
上面的提及一部分，面试者就基本对你之前的实践或者运维经验有所评估了。

2、elasticsearch 的倒排索引是什么

面试官：想了解你对基础概念的认知。
解答：通俗解释一下就可以。
传统的我们的检索是通过文章，逐个遍历找到对应关键词的位置。
而倒排索引，是通过分词策略，形成了词和文章的映射关系表，这种词典+映射表即为倒排索引。
有了倒排索引，就能实现 o（1）时间复杂度的效率检索文章了，极大的提高了检索效率



学术的解答方式：
倒排索引，相反于一篇文章包含了哪些词，它从词出发，记载了这个词在哪些文档中出现过，由两部分组成——词典和倒排表。
加分项：倒排索引的底层实现是基于：FST（Finite State Transducer）数据结构。
lucene 从 4+版本后开始大量使用的数据结构是 FST。FST 有两个优点：
1、空间占用小。通过对词典中单词前缀和后缀的重复利用，压缩了存储空间；
2、查询速度快。O(len(str))的查询时间复杂度。

3、elasticsearch 索引数据多了怎么办，如何调优，部署

面试官：想了解大数据量的运维能力。
解答：索引数据的规划，应在前期做好规划，正所谓“设计先行，编码在后”，这样才能有效的避免突如其来的数据激增导致集群处理能力不足引发的线上客户检索或者其他业务受到影响。
如何调优，正如问题 1 所说，这里细化一下：

3.1 动态索引层面
基于模板+时间+rollover api 滚动创建索引，举例：设计阶段定义：blog 索引的模板格式为：blog_index_时间戳的形式，每天递增数据。

这样做的好处：不至于数据量激增导致单个索引数据量非常大，接近于上线 2 的32 次幂-1，索引存储达到了 TB+甚至更大。

一旦单个索引很大，存储等各种风险也随之而来，所以要提前考虑+及早避免。

3.2 存储层面
冷热数据分离存储，热数据（比如最近 3 天或者一周的数据），其余为冷数据。对于冷数据不会再写入新数据，可以考虑定期 force_merge 加 shrink 压缩操作，节省存储空间和检索效率。

3.3 部署层面
一旦之前没有规划，这里就属于应急策略。结合 ES 自身的支持动态扩展的特点，动态新增机器的方式可以缓解集群压力，注意：如果之前主节点等规划合理，不需要重启集群也能完成动态新增的。

4、elasticsearch 是如何实现 master 选举的

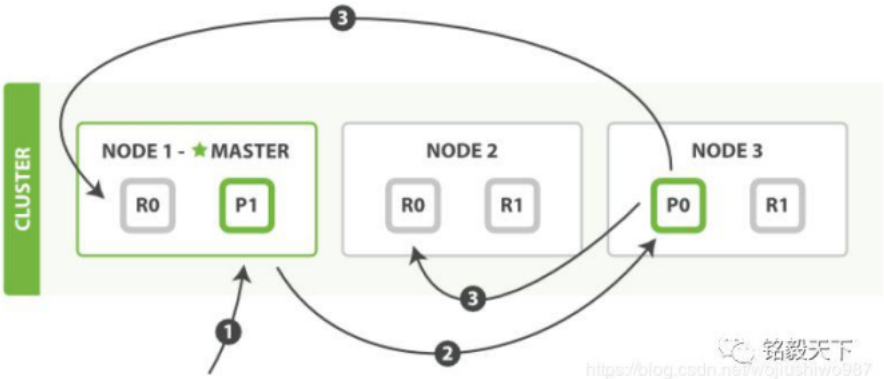
面试官：想了解 ES 集群的底层原理，不再只关注业务层面了。
解答：
前置前提：
1、只有候选主节点（master：true）的节点才能成为主节点。
2、最小主节点数（min_master_nodes）的目的是防止脑裂。

这个我看了各种网上分析的版本和源码分析的书籍，云里雾里。
核对了一下代码，核心入口为 findMaster，选择主节点成功返回对应 Master，否则返回 null。选举流程大致描述如下：
第一步：确认候选主节点数达标，elasticsearch.yml 设置的值
discovery.zen.minimum_master_nodes；

第二步：比较：先判定是否具备 master 资格，具备候选主节点资格的优先返回；若两节点都为候选主节点，则 id 小的值会主节点。注意这里的 id 为 string 类型。题外话：获取节点 id 的方法。
1GET /_cat/nodes?v&h=ip,port,heapPercent,heapMax,id,name
2ip port heapPercent heapMax id name

5、详细描述一下 Elasticsearch 索引文档的过程

面试官：想了解 ES 的底层原理，不再只关注业务层面了。
解答：
这里的索引文档应该理解为文档写入 ES，创建索引的过程。
文档写入包含：单文档写入和批量 bulk 写入，这里只解释一下：单文档写入流程。
记住官方文档中的这个图。



第一步：客户写集群某节点写入数据，发送请求。（如果没有指定路由/协调节点，请求的节点扮演路由节点的角色。）

第二步：节点 1 接受到请求后，使用文档_id 来确定文档属于分片 0。请求会被转到另外的节点，假定节点 3。因此分片 0 的主分片分配到节点 3 上。
第三步：节点 3 在主分片上执行写操作，如果成功，则将请求并行转发到节点 1和节点 2 的副本分片上，等待结果返回。所有的副本分片都报告成功，节点 3 将向协调节点（节点 1）报告成功，节点 1 向请求客户端报告写入成功。

如果面试官再问：第二步中的文档获取分片的过程？
回答：借助路由算法获取，路由算法就是根据路由和文档 id 计算目标的分片 id 的
过程。

```
1shard = hash(_routing) % (num_of_primary_shards)
```

6、详细描述一下 Elasticsearch 搜索的过程？

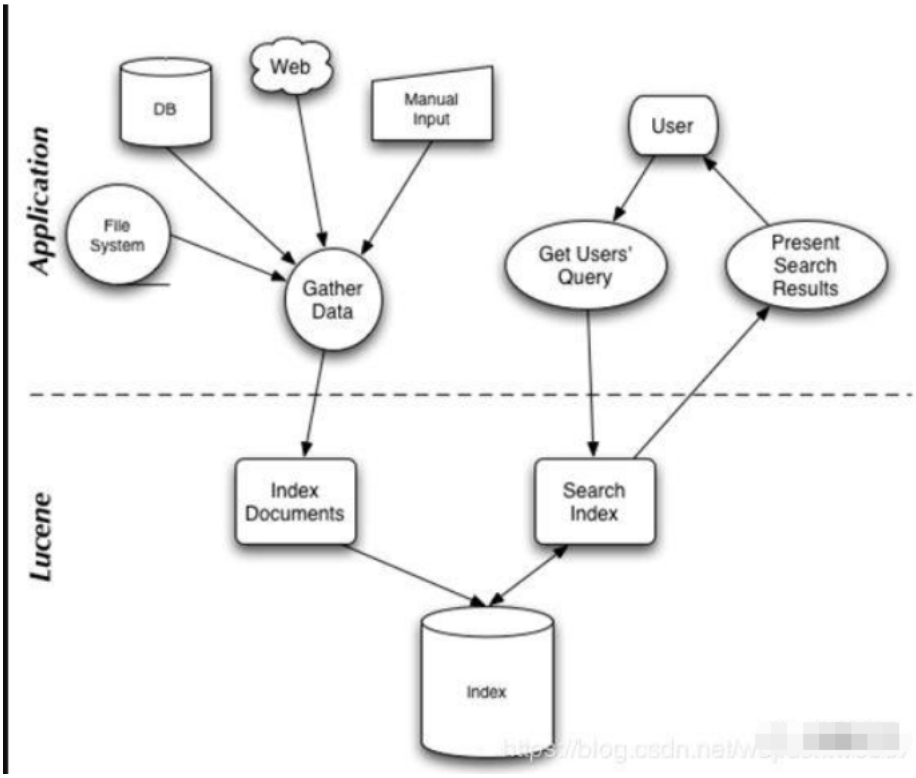
面试官：想了解 ES 搜索的底层原理，不再只关注业务层面了。
解答：
搜索拆解为“query then fetch” 两个阶段。
query 阶段的目的：定位到位置，但不取。
步骤拆解如下：
1、假设一个索引数据有 5 主+1 副本 共 10 分片，一次请求会命中（主或者副本分片中）的一个。
2、每个分片在本地进行查询，结果返回到本地有序的优先队列中。
3、第 2）步骤的结果发送到协调节点，协调节点产生一个全局的排序列表。fetch 阶段的目的：取数据。
路由节点获取所有文档，返回给客户端。

7、Elasticsearch 在部署时，对 Linux 的设置有哪些优化方法

面试官：想了解对 ES 集群的运维能力。
解答：
1、关闭缓存 swap;
2、堆内存设置为：Min（节点内存/2, 32GB）;
3、设置最大文件句柄数；
4、线程池+队列大小根据业务需要做调整；
5、磁盘存储 raid 方式——存储有条件使用 RAID10，增加单节点性能以及避免单节点存储故障。

8、lucence 内部结构是什么？

面试官：想了解你的知识面的广度和深度。
解答：



Lucene 是有索引和搜索的两个过程，包含索引创建，索引，搜索三个要点。可以基于这个脉络展开一些。
最近面试一些公司，被问到的关于 Elasticsearch 和搜索引擎相关的问题，以及自己总结的回答。

9、Elasticsearch 是如何实现 Master 选举的？

1、Elasticsearch 的选主是 ZenDiscovery 模块负责的，主要包含 Ping（节点之间通过这个 RPC 来发现彼此）和 Unicast（单播模块包含一个主机列表以控制哪些节点需要 ping 通）这两部分；

2、对所有可以成为 master 的节点（node.master: true）根据 nodeId 字典排序，每次选举每个节点都把自己所知道节点排一次序，然后选出第一个（第 0 位）节点，暂且认为它是 master 节点。

3、如果对某个节点的投票数达到一定的值（可以成为 master 节点数 n/2+1）并且该节点自己也选举自己，那这个节点就是 master。否则重新选举一直到满足上述条件。

4、补充：master 节点的职责主要包括集群、节点和索引的管理，不负责文档级别的管理；data 节点可以关闭 http 功能*。

10、Elasticsearch 中的节点（比如共 20 个），其中的 10 个选了一个 master，另外 10 个选了另一个 master，怎么办？

1、当集群 master 候选数量不小于 3 个时，可以通过设置最少投票通过数量（discovery.zen.minimum_master_nodes）超过所有候选节点一半以上来解决脑裂问题；
2、当候选数量为两个时，只能修改为唯一的一个 master 候选，其他作为 data节点，避免脑裂问题

11、客户端在和集群连接时，如何选择特定的节点执行请求的？

1、TransportClient 利用 transport 模块远程连接一个 elasticsearch 集群。它并不加入到集群中，只是简单的获得一个或者多个初始化的 transport 地址，并以 轮询 的方式与这些地址进行通信。

12、详细描述一下 Elasticsearch 索引文档的过程。

协调节点默认使用文档 ID 参与计算（也支持通过 routing），以便为路由提供合适的分片。

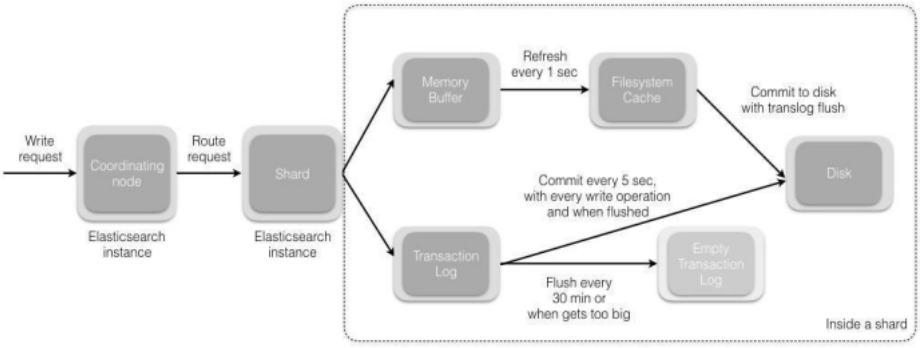
```
shard = hash(document_id) % (num_of_primary_shards)
```

1、当分片所在的节点接收到来自协调节点的请求后，会将请求写入到 Memory Buffer，然后定时（默认是每隔 1 秒）写入到 Filesystem Cache，这个从 MomeryBuffer 到 Filesystem Cache 的过程就叫做 refresh；

2、当然在某些情况下，存在 Momery Buffer 和 Filesystem Cache 的数据可能会丢失，ES 是通过 translog 的机制来保证数据的可靠性的。其实现机制是接收到请求后，同时也会写入到 translog 中，当 Filesystem cache 中的数据写入到磁盘中时，才会清除掉，这个过程叫做 flush；

3、在 flush 过程中，内存中的缓冲将被清除，内容被写入一个新段，段的 fsync将创建一个新的提交点，并将内容刷新到磁盘，旧的 translog 将被删除并开始一个新的 translog。

4、flush 触发的时机是定时触发（默认 30 分钟）或者 translog 变得太大（默认为 512M）时



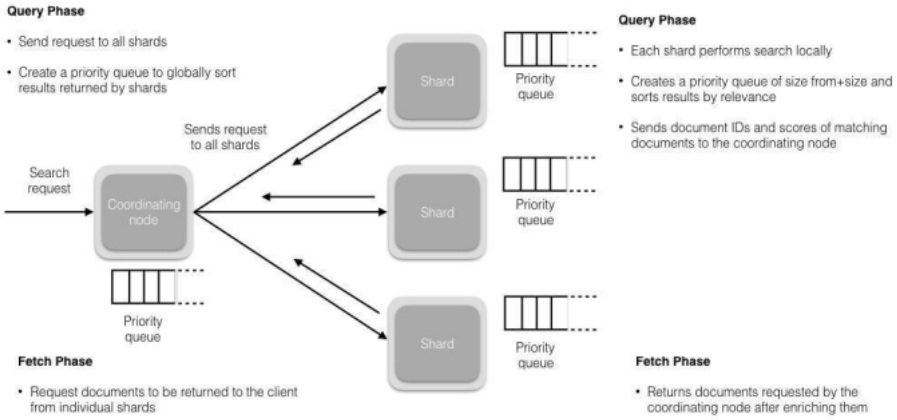
补充：关于 Lucene 的 Segement：
1、Lucene 索引是由多个段组成，段本身是一个功能齐全的倒排索引。
2、段是不可变的，允许 Lucene 将新的文档增量地添加到索引中，而不用从头重建索引。
3、对于每一个搜索请求而言，索引中的所有段都会被搜索，并且每个段会消耗CPU 的时钟周、文件句柄和内存。这意味着段的数量越多，搜索性能会越低。
4、为了解决这个问题，Elasticsearch 会合并小段到一个较大的段，提交新的合并段到磁盘，并删除那些旧的小段。

13、详细描述一下 Elasticsearch 更新和删除文档的过程。

1、删除和更新也都是写操作，但是 Elasticsearch 中的文档是不可变的，因此不能被删除或者改动以展示其变更；
2、磁盘上的每个段都有一个相应的.del 文件。当删除请求发送后，文档并没有真的被删除，而是在.del 文件中被标记为删除。该文档依然能匹配查询，但是会在结果中被过滤掉。当段合并时，在.del 文件中被标记为删除的文档将不会被写入新段。
3、在新的文档被创建时，Elasticsearch 会为该文档指定一个版本号，当执行更新时，旧版本的文档在.del 文件中被标记为删除，新版本的文档被索引到一个新段。旧版本的文档依然能匹配查询，但是会在结果中被过滤掉。

14、详细描述一下 Elasticsearch 搜索的过程

1、搜索被执行成一个两阶段过程，我们称之为 Query Then Fetch；
2、在初始查询阶段时，查询会广播到索引中每一个分片拷贝（主分片或者副本分片）。 每个分片在本地执行搜索并构建一个匹配文档的大小为 from + size 的优先队列。
PS：在搜索的时候是会查询 Filesystem Cache 的，但是有部分数据还在 MemoryBuffer，所以搜索是近实时的。
3、每个分片返回各自优先队列中 所有文档的 ID 和排序值 给协调节点，它合并这些值到自己的优先队列中来产生一个全局排序后的结果列表。
4、接下来就是 取回阶段，协调节点辨别出哪些文档需要被取回并向相关的分片提交多个 GET 请求。每个分片加载并 丰富 文档，如果有需要的话，接着返回文档给协调节点。一旦所有的文档都被取回了，协调节点返回结果给客户端。
5、补充：Query Then Fetch 的搜索类型在文档相关性打分的时候参考的是本分片的数据，这样在文档数量较少的时候可能不够准确，DFS Query Then Fetch 增加了一个预查询的处理，询问 Term 和 Document frequency，这个评分更准确，但是性能会变差。*



15、在 Elasticsearch 中，是怎么根据一个词找到对应的倒排索引的？

SEE：
□ Lucene 的索引文件格式(1)
□ Lucene 的索引文件格式(2)

16、Elasticsearch 在部署时，对 Linux 的设置有哪些优化方法？

1、64 GB 内存的机器是非常理想的， 但是 32 GB 和 16 GB 机器也是很常见的。少于 8 GB 会适得其反。
2、如果你要在更快的 CPUs 和更多的核心之间选择，选择更多的核心更好。多个内核提供的额外并发远胜过稍微快一点点的时钟频率。
3、如果你负担得起 SSD，它将远远超出任何旋转介质。 基于 SSD 的节点，查询和索引性能都有提升。如果你负担得起，SSD 是一个好的选择。
4、即使数据中心们近在咫尺，也要避免集群跨越多个数据中心。绝对要避免集群跨越大的地理距离。
5、请确保运行你应用程序的 JVM 和服务器的 JVM 是完全一样的。 在Elasticsearch 的几个地方，使用 Java 的本地序列化。

6、通过设置 gateway.recover_after_nodes、gateway.expected_nodes、gateway.recover_after_time 可以在集群重启的时候避免过多的分片交换，这可能会让数据恢复从数个小时缩短为几秒钟。
7、Elasticsearch 默认被配置为使用单播发现，以防止节点无意中加入集群。只有在同一台机器上运行的节点才会自动组成集群。最好使用单播代替组播。
8、不要随意修改垃圾回收器（CMS）和各个线程池的大小。
9、把你的内存的（少于）一半给 Lucene（但不要超过 32 GB！），通过ES_HEAP_SIZE 环境变量设置。
10、内存交换到磁盘对服务器性能来说是致命的。如果内存交换到磁盘上，一个100 微秒的操作可能变成 10 毫秒。 再想想那么多 10 微秒的操作时延累加起来。 不难看出 swapping 对于性能是多么可怕。
11、Lucene 使用了大量的文件。同时，Elasticsearch 在节点和 HTTP 客户端之间进行通信也使用了大量的套接字。 所有这一切都需要足够的文件描述符。你应该增加你的文件描述符，设置一个很大的值，如 64,000。

补充：索引阶段性能提升方法
1、使用批量请求并调整其大小：每次批量数据 5–15 MB 大是个不错的起始点。
2、存储：使用 SSD
3、段和合并：Elasticsearch 默认值是 20 MB/s，对机械磁盘应该是个不错的设置。如果你用的是 SSD，可以考虑提高到 100–200 MB/s。如果你在做批量导入，完全不在意搜索，你可以彻底关掉合并限流。另外还可以增加 index.translog.flush_threshold_size 设置，从默认的 512 MB 到更大一些的值，比如 1 GB，这可以在一次清空触发的时候在事务日志里积累出更大的段。

4、如果你的搜索结果不需要近实时的准确度，考虑把每个索引的index.refresh_interval 改到 30s。
5、如果你在做大批量导入，考虑通过设置 index.number_of_replicas: 0 关闭副本。

17、对于 GC 方面，在使用 Elasticsearch 时要注意什么？

1、SEE：https://elasticsearch.cn/article/32
2、倒排词典的索引需要常驻内存，无法 GC，需要监控 data node 上 segmentmemory 增长趋势。
3、各类缓存，field cache, filter cache, indexing cache, bulk queue 等等，要设置合理的大小，并且要应该根据最坏的情况来看 heap 是否够用，也就是各类缓存全部占满的时候，还有 heap 空间可以分配给其他任务吗？避免采用 clear cache等“自欺欺人”的方式来释放内存。
4、避免返回大量结果集的搜索与聚合。确实需要大量拉取数据的场景，可以采用scan & scroll api 来实现。
5、cluster stats 驻留内存并无法水平扩展，超大规模集群可以考虑分拆成多个集群通过 tribe node 连接。
6、想知道 heap 够不够，必须结合实际应用场景，并对集群的 heap 使用情况做持续的监控。

18、Elasticsearch 对于大数据量（上亿量级）的聚合如何实现？

Elasticsearch 提供的首个近似聚合是 cardinality 度量。它提供一个字段的基数，即该字段的 distinct 或者 unique 值的数目。它是基于 HLL 算法的。HLL 会先对我们的输入作哈希运算，然后根据哈希运算的结果中的 bits 做概率估算从而得到基数。其特点是：可配置的精度，用来控制内存的使用（更精确 = 更多内存）；
小的数据集精度是非常高的；我们可以通过配置参数，来设置去重需要的固定内存使用量。无论数千还是数十亿的唯一值，内存使用量只与你配置的精确度相关。

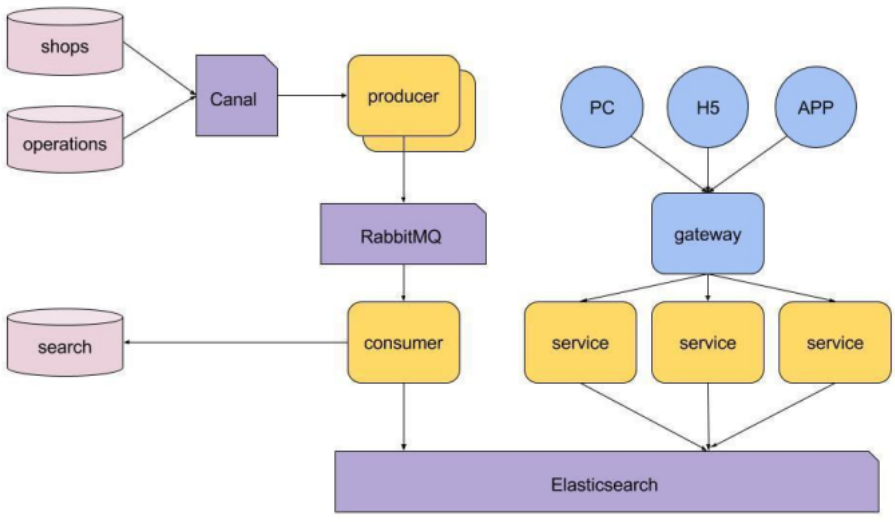
19、在并发情况下，Elasticsearch 如果保证读写一致？

1、可以通过版本号使用乐观并发控制，以确保新版本不会被旧版本覆盖，由应用层来处理具体的冲突；
2、另外对于写操作，一致性级别支持 quorum/one/all，默认为 quorum，即只有当大多数分片可用时才允许写操作。但即使大多数可用，也可能存在因为网络等原因导致写入副本失败，这样该副本被认为故障，分片将会在一个不同的节点上重建。
3、对于读操作，可以设置 replication 为 sync(默认)，这使得操作在主分片和副本分片都完成后才会返回；如果设置 replication 为 async 时，也可以通过设置搜索请求参数 _preference 为 primary 来查询主分片，确保文档是最新版本。

20、如何监控 Elasticsearch 集群状态？

Marvel 让你可以很简单的通过 Kibana 监控 Elasticsearch。你可以实时查看你的集群健康状态和性能，也可以分析过去的集群、索引和节点指标。

21、介绍下你们电商搜索的整体技术架构



22、介绍一下你们的个性化搜索方案？

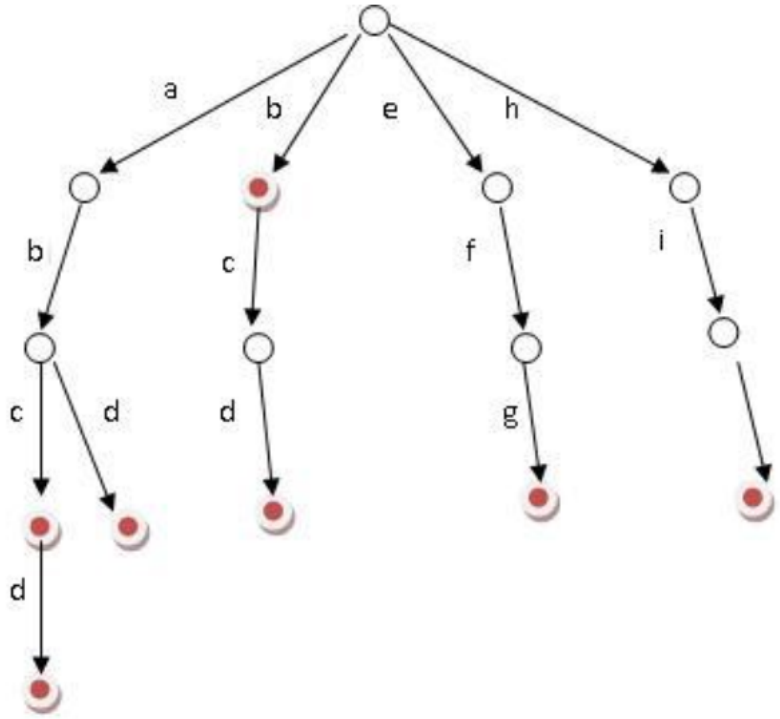
SEE 基于 word2vec 和 Elasticsearch 实现个性化搜索

23、是否了解字典树？

常用字典数据结构如下所示

数据结构	优缺点
排序列表Array/List	使用二分法查找，不平衡
HashMap/TreeMap	性能高，内存消耗大，几乎是原始数据的三倍
Skip List	跳跃表，可快速查找词语，在lucene、redis、Hbase等均有实现。相对于TreeMap等结构，特别适合高并发场景（Skip List介绍）
Trie	适合英文词典，如果系统中存在大量字符串且这些字符串基本没有公共前缀，则相应的trie树将非常消耗内存（数据结构之trie树）
Double Array Trie	适合做中文词典，内存占用小，很多分词工具均采用此种算法（深入双数组Trie）
Ternary Search Tree	三叉树，每一个node有3个节点，兼具省空间和查询快的优点（Ternary Search Tree）
Finite State Transducers (FST)	一种有限状态转移机，Lucene 4有开源实现，并大量使用

Trie 的核心思想是空间换时间，利用字符串的公共前缀来降低查询时间的开销以达到提高效率的目的。它有 3 个基本性质：
1、根节点不包含字符，除根节点外每一个节点都只包含一个字符。
2、从根节点到某一节点，路径上经过的字符连接起来，为该节点对应的字符串。
3、每个节点的所有子节点包含的字符都不相同。



1、可以看到，trie 树每一层的节点数是 26^i 级别的。所以为了节省空间，我们还可以用动态链表，或者用数组来模拟动态。而空间的花费，不会超过单词数×单词长度。
2、实现：对每个结点开一个字母集大小的数组，每个结点挂一个链表，使用左儿子右兄弟表示法记录这棵树；
3、对于中文的字典树，每个节点的子节点用一个哈希表存储，这样就不用浪费太大的空间，而且查询速度上可以保留哈希的复杂度 O(1)

24、拼写纠错是如何实现的？

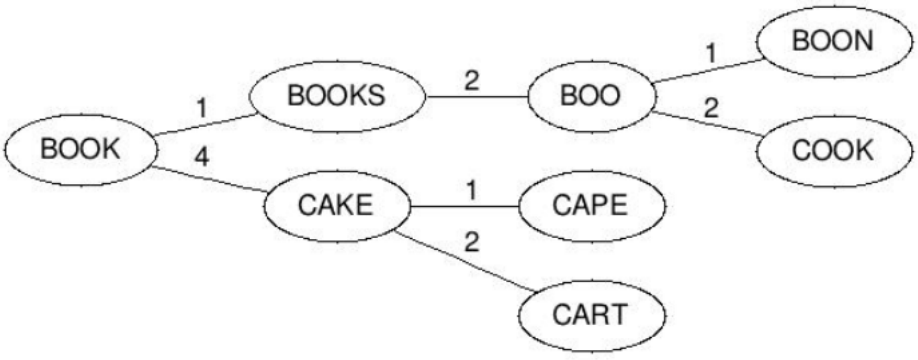
1、拼写纠错是基于编辑距离来实现；编辑距离是一种标准的方法，它用来表示经过插入、删除和替换操作从一个字符串转换到另外一个字符串的最小操作步数；
2、编辑距离的计算过程：比如要计算 batyu 和 beauty 的编辑距离，先创建一个7×8 的表（batyu 长度为 5，coffee 长度为 6，各加 2），接着，在如下位置填入黑色数字。其他格的计算过程是取以下三个值的最小值：
如果最上方的字符等于最左方的字符，则为左上方的数字。否则为左上方的数字
+1。（对于 3,3 来说为 0）
左方数字+1（对于 3,3 格来说为 2）
上方数字+1（对于 3,3 格来说为 2）
最终取右下角的值即为编辑距离的值 3。

		b	e	a	u	t	y
	0	1	2	3	4	5	6
b	1	0	1	2	3	4	5
a	2	1	1	1	2	3	4
t	3	2	2	2	2	2	3
y	4	3	3	3	3	3	2
u	5	4	4	4	3	4	3

对于拼写纠错，我们考虑构造一个度量空间（Metric Space），该空间内任何关系满足以下三条基本条件：
d(x,y) = 0 -- 假如 x 与 y 的距离为 0，则 x=y
d(x,y) = d(y,x) -- x 到 y 的距离等同于 y 到 x 的距离
d(x,y) + d(y,z) >= d(x,z) -- 三角不等式
1、根据三角不等式，则满足与 query 距离在 n 范围内的另一个字符转 B，其与 A的距离最大为 d+n，最小为 d-n。

2、BK 树的构造就过程如下：每个节点有任意个子节点，每条边有个值表示编辑距离。所有子节点到父节点的边上标注 n 表示编辑距离恰好为 n。比如，我们有棵树父节点是”book”和两个子节点”cake”和”books”，”book”到”books”的边标号 1，”book”到”cake”的边上标号 4。从字典里构造好树后，无论何
时你想插入新单词时，计算该单词与根节点的编辑距离，并且查找数值为d(neweord, root)的边。递归得与各子节点进行比较，直到没有子节点，你就可以创建新的子节点并将新单词保存在那。比如，插入”boo”到刚才上述例子的树中，我们先检查根节点，查找 d(“book”, “boo”) = 1 的边，然后检查标号为1 的边的子节点，得到单词”books”。我们再计算距离 d(“books”, “boo”)=2，则将新单词插在”books”之后，边标号为 2。

3、查询相似词如下：计算单词与根节点的编辑距离 d，然后递归查找每个子节点标号为 d-n 到 d+n（包含）的边。假如被检查的节点与搜索单词的距离 d 小于 n，则返回该节点并继续查询。比如输入 cape 且最大容忍距离为 1，则先计算和根的编辑距离 d(“book”, “cape”)=4，然后接着找和根节点之间编辑距离为 3 到5 的，这个就找到了 cake 这个节点，计算 d(“cake”, “cape”)=1，满足条件所以返回 cake，然后再找和 cake 节点编辑距离是 0 到 2 的，分别找到 cape 和cart 节点，这样就得到 cape 这个满足条件的结果。

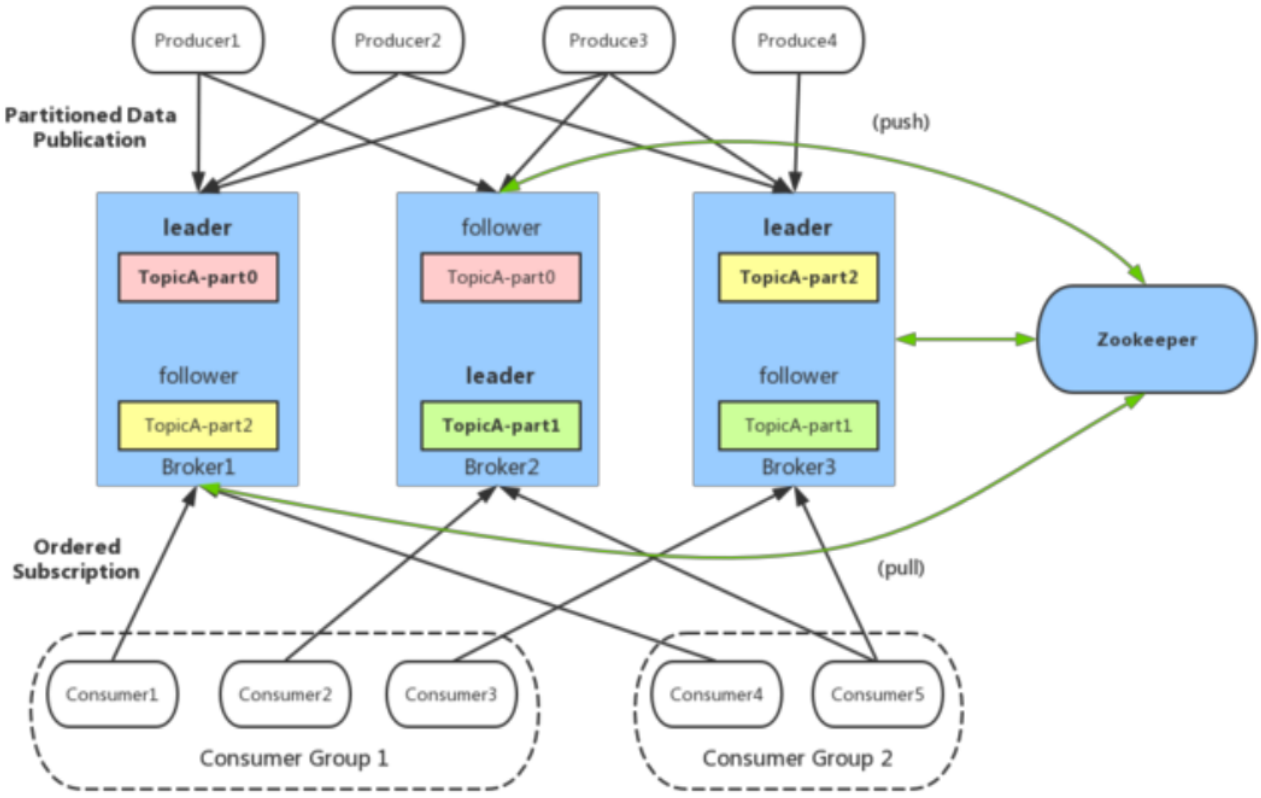


Kafka 面试题

1、Kafka 是什么

Kafka 是一种高吞吐量、分布式、基于发布/订阅的消息系统，最初由 LinkedIn 公司开发，使用Scala 语言编写，目前是 Apache 的开源项目。

1. broker： Kafka 服务器，负责消息存储和转发
2. topic：消息类别， Kafka 按照 topic 来分类消息
3. partition： topic 的分区，一个 topic 可以包含多个 partition， topic 消息保存在各个partition 上
4. offset：消息在日志中的位置，可以理解是消息在 partition 上的偏移量，也是代表该消息的唯一序号
5. Producer：消息生产者
6. Consumer：消息消费者
7. Consumer Group：消费者分组，每个 Consumer 必须属于一个 group
8. Zookeeper：保存着集群 broker、 topic、 partition 等 meta 数据；另外，还负责 broker 故障发现， partition leader 选举，负载均衡等功能

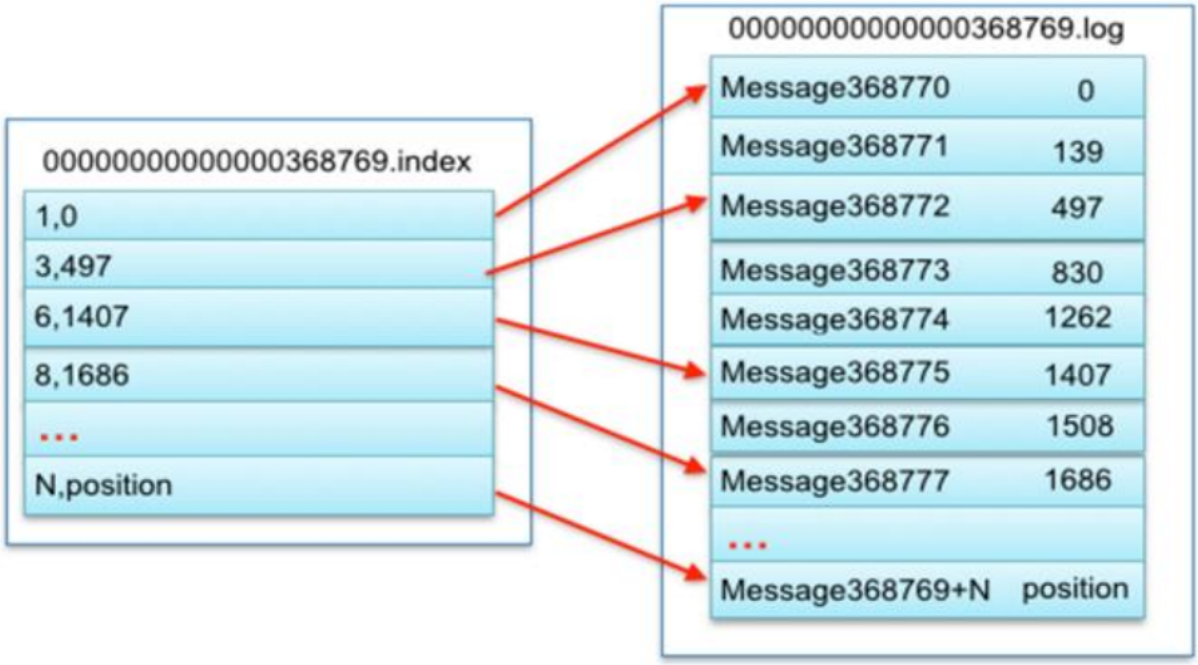


2、partition 的数据文件（offset， MessageSize， data）

partition 中的每条 Message 包含了以下三个属性： offset， MessageSize， data， 其中 offset 表示 Message 在这个 partition 中的偏移量， offset 不是该 Message 在 partition 数据文件中的实际存储位置，而是逻辑上一个值，它唯一确定了 partition 中的一条 Message，可以认为 offset 是partition 中 Message 的 id； MessageSize 表示消息内容 data 的大小； data 为 Message 的具体内容。

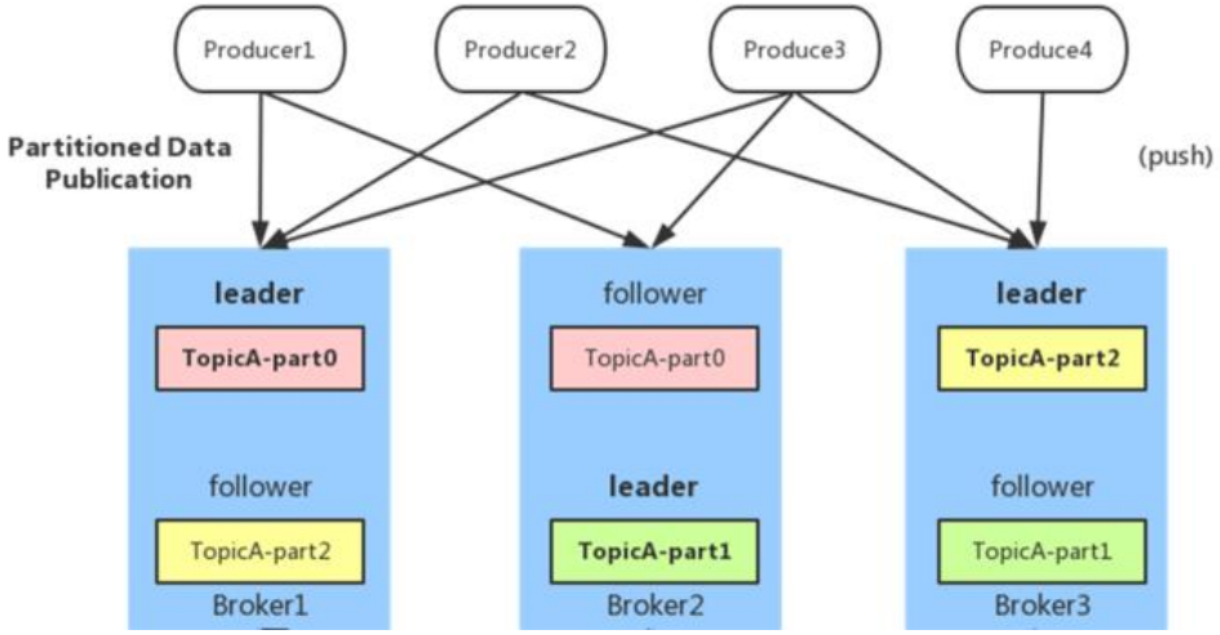
3、数据文件分段 segment（顺序读写、分段命令、二分查找）

Kafka 为每个分段后的数据文件建立了索引文件，文件名与数据文件的名字是一样的，只是文件扩展名为.index。 index 文件中并没有为数据文件中的每条 Message 建立索引，而是采用了稀疏存储的方式，每隔一定字节的数据建立一条索引。这样避免了索引文件占用过多的空间，从而可以将索引文件保留在内存中。



4、负载均衡（partition 会均衡分布到不同 broker 上）

由于消息 topic 由多个 partition 组成， 且 partition 会均衡分布到不同 broker 上，因此，为了有效利用 broker 集群的性能，提高消息的吞吐量， producer 可以通过随机或者 hash 等方式，将消息平均发送到多个 partition 上，以实现负载均衡。



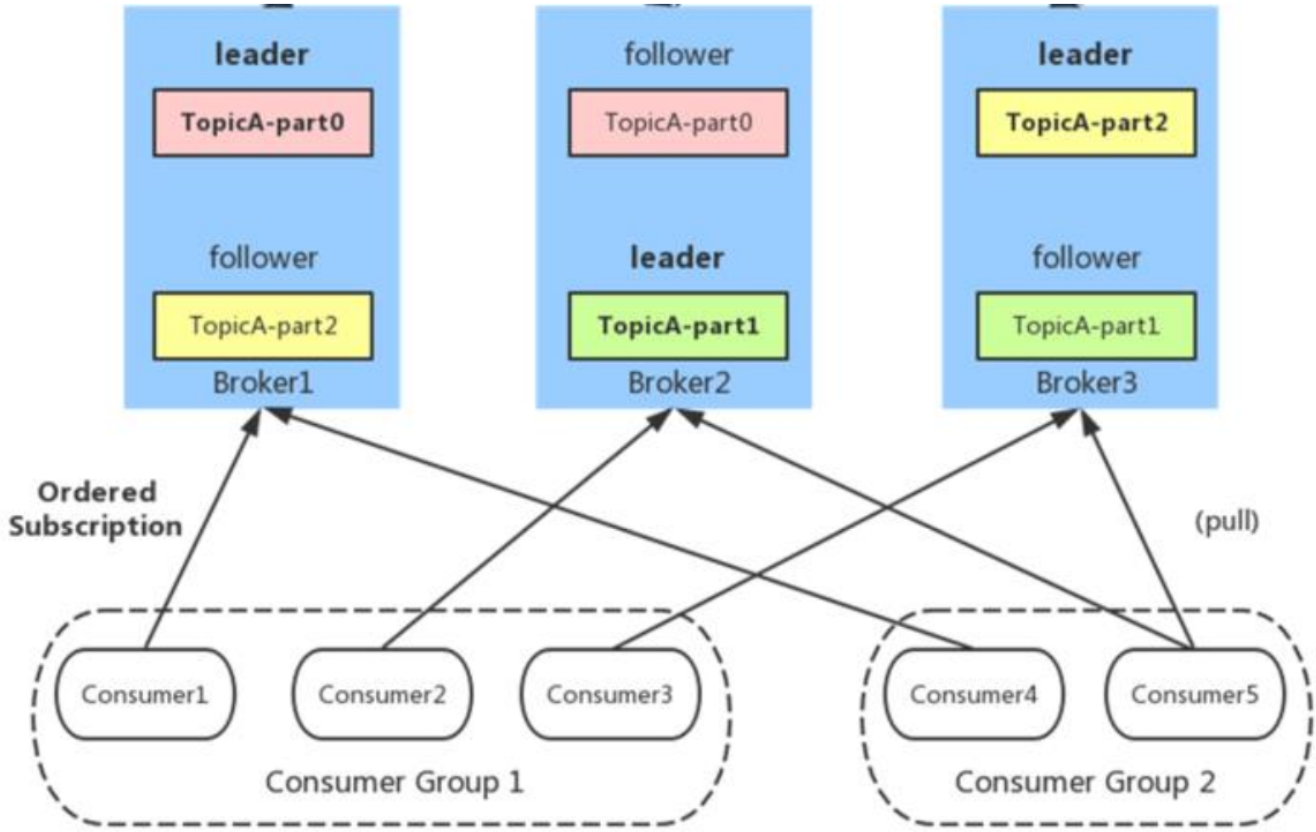
5、批量发送

是提高消息吞吐量重要的方式， Producer 端可以在内存中合并多条消息后， 以一次请求的方式发送了批量的消息给 broker，从而大大减少 broker 存储消息的 IO 操作次数。但也一定程度上影响了消息的实时性，相当于以时延代价，换取更好的吞吐量。

6、压缩（GZIP 或 Snappy）

Producer 端可以通过 GZIP 或 Snappy 格式对消息集合进行压缩。 Producer 端进行压缩之后，在Consumer 端需进行解压。压缩的好处就是减少传输的数据量，减轻对网络传输的压力，在对大数据处理上，瓶颈往往体现在网络上而不是 CPU（压缩和解压会耗掉部分 CPU 资源）。

7、消费者设计



8、Consumer Group

同一 Consumer Group 中的多个 Consumer 实例，不同时消费同一个 partition，等效于队列模式。 partition 内消息是有序的， Consumer 通过 pull 方式消费消息。 Kafka 不删除已消费的消息对于 partition，顺序读写磁盘数据，以时间复杂度 O(1)方式提供消息持久化能力。

9、如何获取 topic 主题的列表

bin/kafka-topics.sh --list --zookeeper localhost:2181

10、生产者和消费者的命令行是什么？

生产者在主题上发布消息：

```
bin/kafka-console-producer.sh --broker-list 192.168.43.49:9092 --topic
Hello-Kafa
```

注意这里的 IP 是 server.properties 中的 listeners 的配置。接下来每个新行就是输入一条新消息。
消费者接受消息：

```
bin/kafka-console-consumer.sh --zookeeper localhost:2181 --topic
Hello-Kafka --from-beginning
```

11、consumer 是推还是拉？

Kafka 最初考虑的问题是，customer 应该从 brokes 拉取消息还是 brokers 将消息推送到 consumer，也就是 pull 还 push。在这方面，Kafka 遵循了一种大部分消息系统共同的传统的设计：producer 将消息推送到 broker，consumer 从broker 拉取消息。

一些消息系统比如 Scribe 和 Apache Flume 采用了 push 模式，将消息推送到下游的 consumer。这样做有好处也有坏处：由 broker 决定消息推送的速率，对于不同消费速率的 consumer 就不太好处理了。消息系统都致力于让 consumer 以最大的速率最快速的消费消息，但不幸的是，push 模式下，当 broker 推送的速率远大于 consumer 消费的速率时，consumer 恐怕就要崩溃了。最终 Kafka 还是选取了传统的 pull 模式。

Pull 模式的另外一个好处是 consumer 可以自主决定是否批量的从 broker 拉取数据。Push 模式必须在不知道下游 consumer 消费能力和消费策略的情况下决定是立即推送每条消息还是缓存之后批量推送。如果为了避免 consumer 崩溃而采用较低的推送速率，将可能导致一次只推送较少的消息而造成浪费。Pull 模式下，
consumer 就可以根据自己的消费能力去决定这些策略。

Pull 有个缺点是，如果 broker 没有可供消费的消息，将导致 consumer 不断在循环中轮询，直到新消息到 t 达。为了避免这点，Kafka 有个参数可以让 consumer阻塞知道新消息到达(当然也可以阻塞知道消息的数量达到某个特定的量这样就可以批量发送)。

12、讲讲 kafka 维护消费状态跟踪的方法

大部分消息系统在 broker 端的维护消息被消费的记录：一个消息被分发到consumer 后 broker 就马上进行标记或者等待 customer 的通知后进行标记。这样也可以在消息在消费后立马就删除以减少空间占用。

但是这样会不会有什么问题呢？如果一条消息发送出去之后就立即被标记为消费过的，一旦 consumer 处理消息时失败了（比如程序崩溃）消息就丢失了。为了解决这个问题，很多消息系统提供了另外一个个功能：当消息被发送出去之后仅仅被标记为已发送状态，当接到 consumer 已经消费成功的通知后才标记为已被消费的状态。这虽然解决了消息丢失的问题，但产生了新问题，首先如果 consumer处理消息成功了但是向 broker 发送响应时失败了，这条消息将被消费两次。第二个问题时，broker 必须维护每条消息的状态，并且每次都要先锁住消息然后更改状态然后释放锁。这样麻烦又来了，且不说要维护大量的状态数据，比如如果消息发送出去但没有收到消费成功的通知，这条消息将一直处于被锁定的状态，Kafka 采用了不同的策略。Topic 被分成了若干分区，每个分区在同一时间只被一个 consumer 消费。这意味着每个分区被消费的消息在日志中的位置仅仅是一个简单的整数：offset。这样就很容易标记每个分区消费状态就很容易了，仅仅需要一个整数而已。这样消费状态的跟踪就很简单了。这带来了另外一个好处：consumer 可以把 offset 调成一个较老的值，去重新消费老的消息。这对传统的消息系统来说看起来有些不可思议，但确实是非常有用的，谁规定了一条消息只能被消费一次呢？

13、讲一下主从同步

https://blog.csdn.net/honglei915/article/details/37565289

14、为什么需要消息系统，mysql 不能满足需求吗？

1.解耦：
允许你独立的扩展或修改两边的处理过程，只要确保它们遵守同样的接口约束。

2.冗余：
消息队列把数据进行持久化直到它们已经被完全处理，通过这一方式规避了数据丢失风险。许多消息队列所采用的"插入-获取-删除"范式中，在把一个消息从队列中删除之前，需要你的处理系统明确的指出该消息已经被处理完毕，从而确保你的数据被安全的保存直到你使用完毕。

3.扩展性：
因为消息队列解耦了你的处理过程，所以增大消息入队和处理的频率是很容易的，只要另外增加处理过程即可。

4.灵活性 & 峰值处理能力：

在访问量剧增的情况下，应用仍然需要继续发挥作用，但是这样的突发流量并不常见。如果为以能处理这类峰值访问为标准来投入资源随时待命无疑是巨大的浪费。使用消息队列能够使关键组件顶住突发的访问压力，而不会因为突发的超负荷的请求而完全崩溃。

5.可恢复性：
系统的一部分组件失效时，不会影响到整个系统。消息队列降低了进程间的耦合度，所以即使一个处理消息的进程挂掉，加入队列中的消息仍然可以在系统恢复后被处理。

6.顺序保证：
在大多使用场景下，数据处理的顺序都很重要。大部分消息队列本来就是排序的，并且能保证数据会按照特定的顺序来处理。（Kafka 保证一个 Partition 内的消息的有序性）

7.缓冲：
有助于控制和优化数据流经过系统的速度，解决生产消息和消费消息的处理速度不一致的情况。

8.异步通信：
很多时候，用户不想也不需要立即处理消息。消息队列提供了异步处理机制，允许用户把一个消息放入队列，但并不立即处理它。想向队列中放入多少消息就放多少，然后在需要的时候再去处理它们。

15、Zookeeper 对于 Kafka 的作用是什么？

Zookeeper 是一个开放源码的、高性能的协调服务，它用于 Kafka 的分布式应用。Zookeeper 主要用于在集群中不同节点之间进行通信

在 Kafka 中，它被用于提交偏移量，因此如果节点在任何情况下都失败了，它都可以从之前提交的偏移量中获取
除此之外，它还执行其他活动，如: leader 检测、分布式同步、配置管理、识别新节点何时离开或连接、集群、节点实时状态等等。

16、数据传输的事务定义有哪三种？
和 MQTT 的事务定义一样都是 3 种。
（1）最多一次: 消息不会被重复发送，最多被传输一次，但也有可能一次不传输
（2）最少一次: 消息不会被漏发送，最少被传输一次，但也有可能被重复传输.
（3）精确的一次（Exactly once）: 不会漏传输也不会重复传输,每个消息都传输被一次而且仅仅被传输一次，这是大家所期望的

16、Kafka 判断一个节点是否还活着有那两个条件？

（1）节点必须可以维护和 ZooKeeper 的连接，Zookeeper 通过心跳机制检查每个节点的连接
（2）如果节点是个 follower,他必须能及时的同步 leader 的写操作，延时不能太久

17、Kafka 与传统 MQ 消息系统之间有三个关键区别

(1).Kafka 持久化日志，这些日志可以被重复读取和无限期保留
(2).Kafka 是一个分布式系统：它以集群的方式运行，可以灵活伸缩，在内部通过复制数据提升容错能力和高可用性
(3).Kafka 支持实时的流式处理

18、讲一讲 kafka 的 ack 的三种机制

request.required.acks 有三个值 0 1 -1(all)
0:生产者不会等待 broker 的 ack，这个延迟最低但是存储的保证最弱当 server 挂掉的时候就会丢数据。
1：服务端会等待 ack 值 leader 副本确认接收到消息后发送 ack 但是如果 leader挂掉后他不确保是否复制完成新 leader 也会导致数据丢失。
-1(all)：服务端会等所有的 follower 的副本受到数据后才会受到 leader 发出的ack，这样数据不会丢失

19、消费者如何不自动提交偏移量，由应用提交？

将 auto.commit.offset 设为 false，然后在处理一批消息后 commitSync() 或者异步提交 commitAsync()
即：

```
ConsumerRecords<> records = consumer.poll();
for (ConsumerRecord<> record : records){
。。。
tyr{
consumer.commitSync()
} 。
。。
}
```

20、消费者故障，出现活锁问题如何解决？

出现“活锁”的情 况，是它 持续 的发 送心 跳，但是 没有 处理 。为 了预 防消 费者 在这种 情况 下一 直持 有分 区，我们 使用 max.poll.interval.ms 活跃 检测 机制 。 在此基础 上， 如果 你调 用的 poll 的频 率大 于最 大间 隔， 则客 户端 将主 动地 离开 组， 以便其 他消 费者 接管 该分 区。 发生 这种 情况 时， 你会 看到 offset 提交 失败 （ 调 用 commitSync（ ） 引发 的 CommitFailedException ）。 这是 一种 安全 机制 ，保 障只有 活动 成员 能够 提交 offset。所 以要 留在 组中 ，你 必须 持续 调用 poll。

消费者提供两个配置设置来控制 poll 循环：
max.poll.interval.ms：增大 poll 的间 隔 ，可以 为消 费者 提供 更多 的时 间去 处理 返回的 消息（ 调用 poll(long)返回 的消 息，通常 返回 的消 息都 是一 批 ）。缺点 是此 值越 大 将会 延迟 组重 新平 衡。

max.poll.records：此 设置 限制 每次 调用 poll 返回 的消 息数 ，这 样可 以更 容易 的预测 每次 poll 间隔 要处 理的 最大 值。通过 调整 此值 ，可以 减少 poll 间隔 ，减少 重新 平 衡分 组的

对于 消息 处理 时间 不可 预测 地的 情况 ，这些 选项 是不 够的 。 处理 这种 情况 的推 荐方法 是将 消息 处理 移到 另一 个线 程中 ，让消 费者 继续 调用 poll。 但是 必须 注意 确保已 提交 的 offset 不超 过实 际位 置。 另外 ，你 必须 禁用 自动 提交 ，并 只有 在线 程完成 处理 后才 为记 录手 动提 交偏 移量（ 取决 于你 ）。 还要 注意 ，你需 要 pause 暂停分 区， 不会 从 poll 接收 到新 消息 ，让 线程 处理 完之 前返 回的 消息 （ 如 果你 的处理能 力比 拉取 消息 的慢 ，那 创建 新线 程将 导致 你机 器内 存溢 出 ）。

21、如何控制消费的位置

kafka 使用 seek(TopicPartition, long)指定新的消费位置。用于查找服务器保留的最早和最新的 offset 的特殊的方法也可用（ seekToBeginning(Collection) 和 seekToEnd(Collection) ）

22、kafka 分布式（ 不是单机 ）的情况下，如何保证消息的顺序消费?

Kafka 分布式的单位是 partition，同一个 partition 用一个 write ahead log 组织，所以可以保证 FIFO 的顺序。不同 partition 之间不能保证顺序。但是绝大多数用户都可以通过 message key 来定义，因为同一个 key 的 Message 可以保证只发送到同一个 partition。

Kafka 中发送 1 条消息的时候，可以指定(topic, partition, key) 3 个参数。partiton 和 key 是可选的。如果你指定了 partition，那就是所有消息发往同 1个 partition，就是有序的。并且在消费端，Kafka 保证，1 个 partition 只能被1 个 consumer 消费。或者你指定 key（ 比如 order id ），具有同 1 个 key 的所有消息，会发往同 1 个 partition。

23、kafka 的高可用机制是什么？

这个问题比较系统，回答出 kafka 的系统特点，leader 和 follower 的关系，消息读写的顺序即可。
https://www.cnblogs.com/qingyunzong/p/9004703.html

https://www.tuicool.com/articles/BNRza2E
https://yq.aliyun.com/articles/64703

24、kafka 如何减少数据丢失

https://www.cnblogs.com/huxi2b/p/6056364.html

25、kafka 如何不消费重复数据？比如扣款，我们不能重复的扣。

其实还是得结合业务来思考，我这里给几个思路：
比如你拿个数据要写库，你先根据主键查一下，如果这数据都有了，你就别插入了，update 一下好吧。

比如你是写 Redis，那没问题了，反正每次都是 set，天然幂等性。
比如你不是上面两个场景，那做的稍微复杂一点，你需要让生产者发送每条数据的时候，里面加一个全局唯一的 id，类似订单 id 之类的东西，然后你这里消费到了之后，先根据这个 id 去比如 Redis 里查一下，之前消费过吗？如果没有消费过，你就处理，然后这个 id 写 Redis。如果消费过了，那你就别处理了，保证别重复处理相同的消息即可。

比如基于数据库的唯一键来保证重复数据不会重复插入多条。因为有唯一键约束了，重复数据插入只会报错，不会导致数据库中出现脏数据

微服务 面试题

微服务，又称微服务 架构，是一种架构风格，它将应用程序构建为以业务领域为模型的小型自治服务集合 。
通俗地说，你必须看到蜜蜂如何通过对齐六角形蜡细胞来构建它们的蜂窝状物。他们最初从使用各种材料的小部分开始，并继续从中构建一个大型蜂箱。这些细胞形成图案，产生坚固的结构，将蜂窝的特定部分固定在一起。这里，每个细胞独立于另一个细胞，但它也与其他细胞相关。这意味着对一个细胞的损害不会损害其他细胞，因此，蜜蜂可以在不影响完整蜂箱的情况下重建这些细胞。

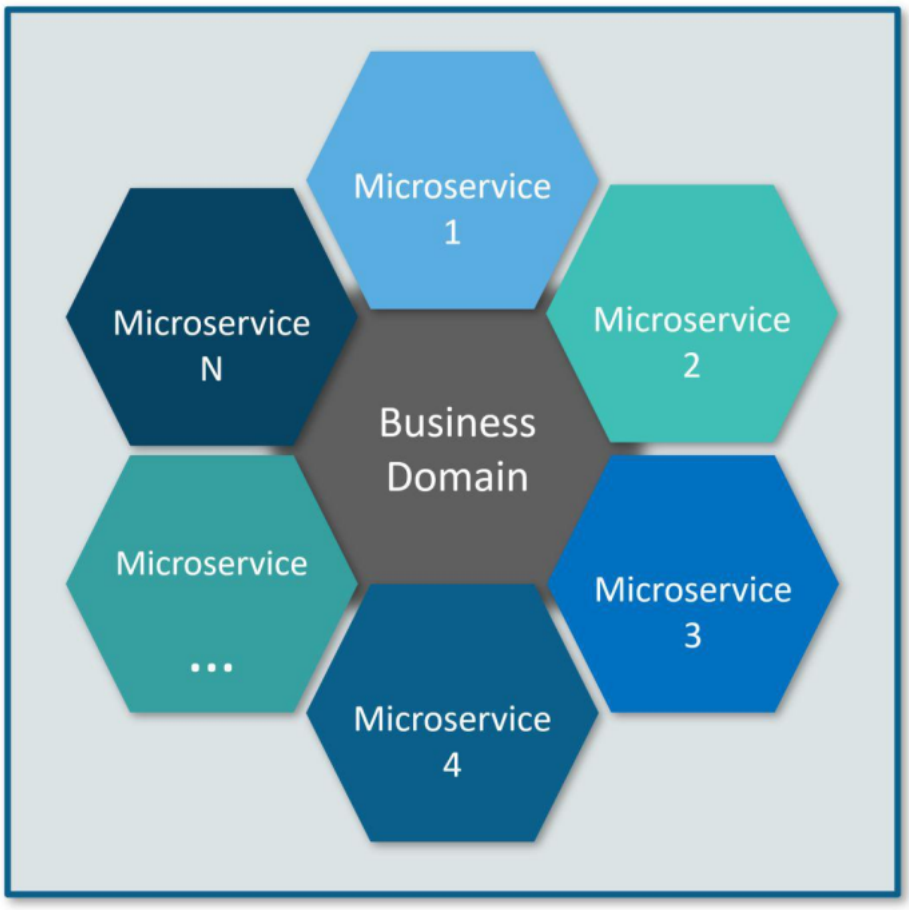


图 1：微服务的蜂窝表示 – 微服务访谈问题
请参考上图。这里，每个六边形形状代表单独的服务组件。与蜜蜂的工作类似，每个敏捷团队都使用可用的框架和所选的技术堆栈构建单独的服务组件。就像在蜂箱中一样，每个服务组件形成一个强大的微服务架构，以提供更好的可扩展性。此外，敏捷团队可以单独处理每个服务组件的问题，而对整个应用程序没有影响或影响最小。

2、微服务架构有哪些优势？

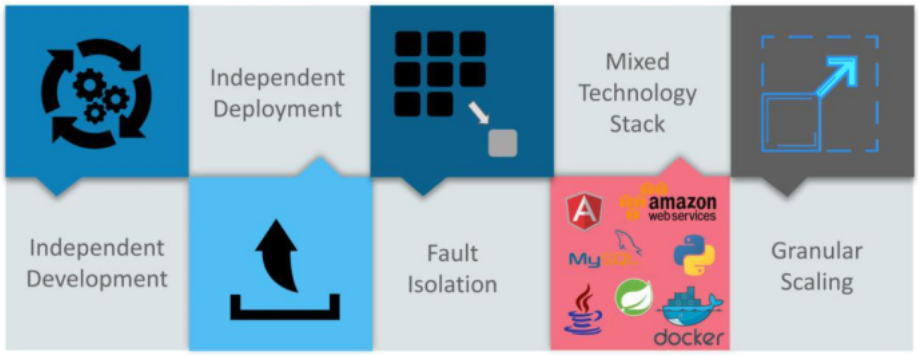


图 2：微服务的 优点 – 微服务访谈问题
□ 独立开发 – 所有微服务都可以根据各自的功能轻松开发
□ 独立部署 – 基于其服务，可以在任何应用程序中单独部署它们
□ 故障隔离 – 即使应用程序的一项服务不起作用，系统仍可继续运行
□ 混合技术堆栈 – 可以使用不同的语言和技术来构建同一应用程序的不同服务
□ 粒度缩放 – 单个组件可根据需要进行缩放，无需将所有组件缩放在一起

3、微服务有哪些特点？

图 3：微服务的 特点 – 微服务访谈问题
□ 解耦 – 系统内的服务很大程度上是分离的。因此，整个应用程序可以轻松构建，更改和扩展
□ 组件化 – 微服务被视为可以轻松更换和升级的独立组件
□ 业务能力 – 微服务非常简单，专注于单一功能
□ 自治 – 开发人员和团队可以彼此独立工作，从而提高速度
□ 持续交付 – 通过软件创建，测试和批准的系统自动化，允许频繁发布软件
□ 责任 – 微服务不关注应用程序作为项目。相反，他们将应用程序视为他们负责的产品
□ 分散治理 – 重点是使用正确的工具来做正确的工作。这意味着没有标准化模式或任何技术模式。开发人员可以自由选择最有用的工具来解决他们的问题
□ 敏捷 – 微服务支持敏捷开发。任何新功能都可以快速开发并再次丢弃

4、设计微服务的最佳实践是什么？

以下是设计微服务的最佳实践：

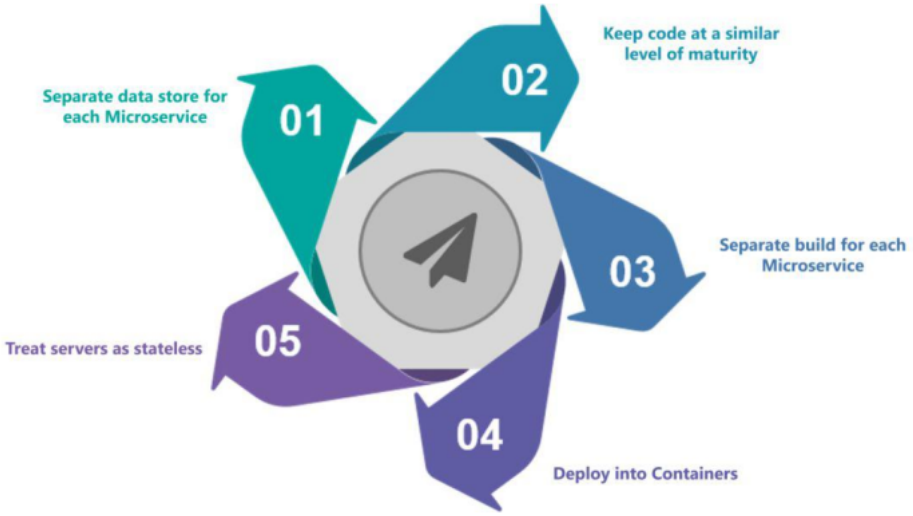


图 4：设计微服务的最佳实践 – 微服务访谈问题

5、微服务架构如何运作？

微服务架构具有以下组件：

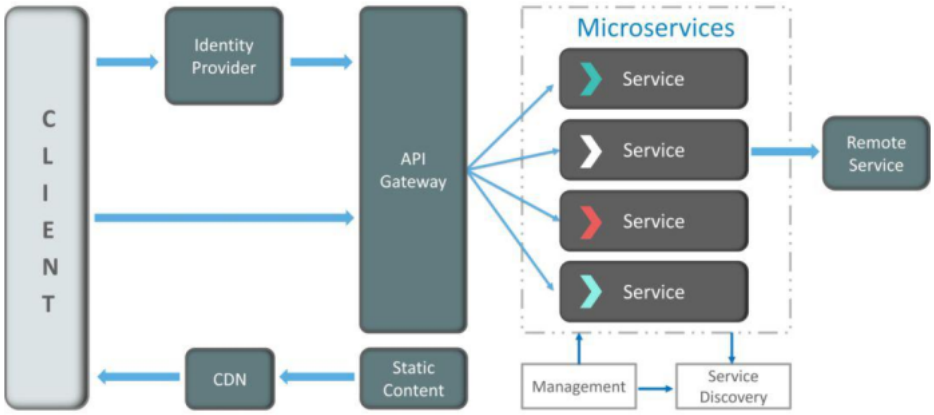


图 5：微服务 架构 – 微服务面试问题

- 客户端 – 来自不同设备的不同用户发送请求。
- 身份提供商 – 验证用户或客户身份并颁发安全令牌。
- API 网关 – 处理客户端请求。
- 静态内容 – 容纳系统的所有内容。
- 管理 – 在节点上平衡服务并识别故障。
- 服务发现 – 查找微服务之间通信路径的指南。
- 内容交付网络 – 代理服务器及其数据中心的分布式网络。
- 远程服务 – 启用驻留在 IT 设备网络上的远程访问信息。

6、微服务架构的优缺点是什么？

微服务架构的优点	微服务架构的缺点
自由使用不同的技术	增加故障排除挑战
每个微服务都侧重于单一功能	由于远程呼叫而增加延迟
支持单个可部署单元	增加了配置和其他操作的工作量
允许经常发布软件	难以保持交易安全
确保每项服务的安全性	艰难地跨越各种边界跟踪数据
多个服务是并行开发和部署的	难以在服务之间进行编码

7、单片，SOA 和微服务架构有什么区别？

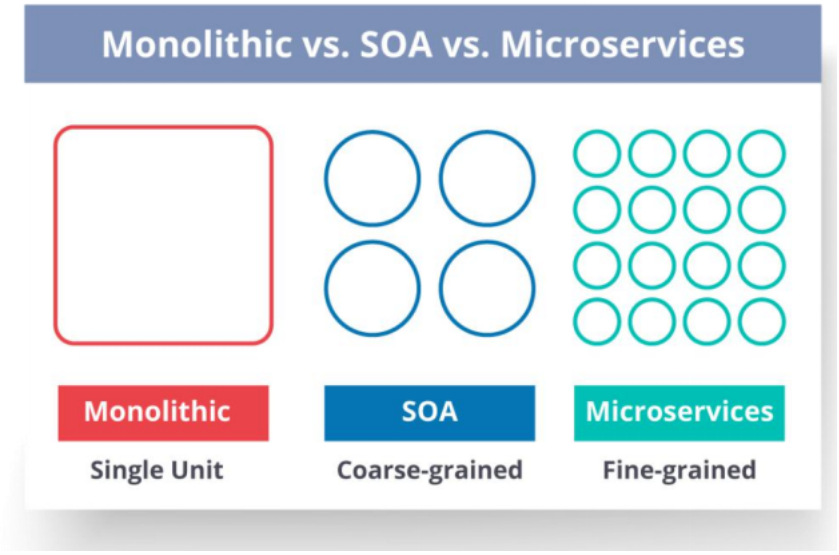


图 6： 单片 SOA 和微服务之间的比较 – 微服务访谈问题

- 单片架构类似于大容器，其中应用程序的所有软件组件组装在一起并紧密封装。

□ 一个面向服务的架构是一种相互通信服务的集合。通信可以涉及简单的数据传递，也可以涉及两个或多个协调某些活动的服务。
□ 微服务架构是一种架构风格，它将应用程序构建为以业务域为模型的小型自治服务集合。

8、在使用微服务架构时，您面临哪些挑战？

开发一些较小的微服务听起来很容易，但开发它们时经常遇到的挑战如下。
□ 自动化组件：难以自动化，因为有许多较小的组件。因此，对于每个组件，我们必须遵循 Build，Deploy 和 Monitor 的各个阶段。
□ 易感性：将大量组件维护在一起变得难以部署，维护，监控和识别问题。它需要在所有组件周围具有很好的感知能力。
□ 配置管理：有时在各种环境中维护组件的配置变得困难。
□ 调试：很难找到错误的每一项服务。维护集中式日志记录和仪表板以调试问题至关重要。

9、SOA 和微服务架构之间的主要区别是什么？

SOA 和微服务之间的主要区别如下：

SOA	微服务
遵循" 尽可能多的共享 "架构方法	遵循" 尽可能少分享 "的架构方法
重要性在于 业务功能 重用	重要性在于" 有界背景 " 的概念
他们有 共同的 治理 和标准	他们专注于 人们的 合作 和其他选择的自由
使用 企业服务总线（ESB） 进行通信	简单的消息系统
它们支持 多种消息协议	他们使用 轻量级协议 ，如 HTTP / REST 等。
多线程， 有更多的开销来处理I / O.	单线程 通常使用Event Loop功能进行非锁定I / O处理
最大化应用程序服务可重用性	专注于 解耦
传统的关系数据库 更常用	现代 关系数据库 更常用
系统的变化需要修改整体	系统的变化是创造一种新的服务
DevOps / Continuous Delivery正在变得流行，但还不是主流	专注于DevOps /持续交付

10、微服务有什么特点？

您可以列出微服务的特征，如下所示：

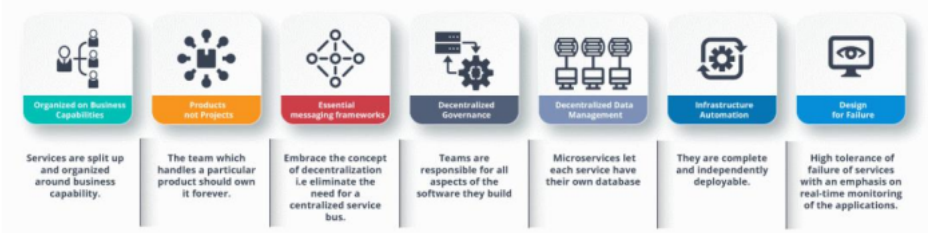


图 7：微服务的特征 – 微服务访谈问题

11、什么是领域驱动设计？

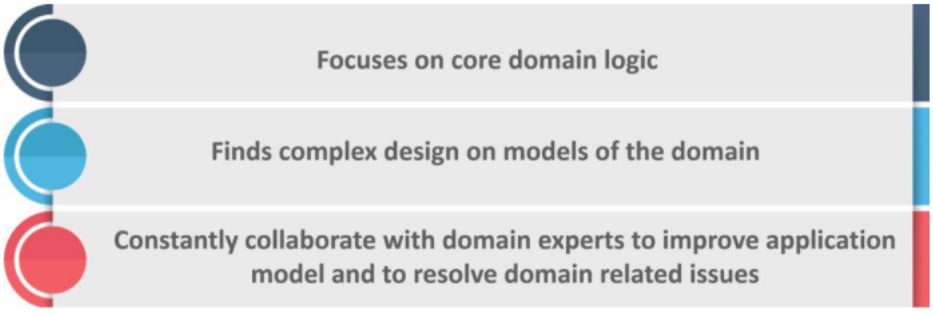


图 8： DDD 原理 – 微服务面试问题

12、为什么需要域驱动设计（DDD）？

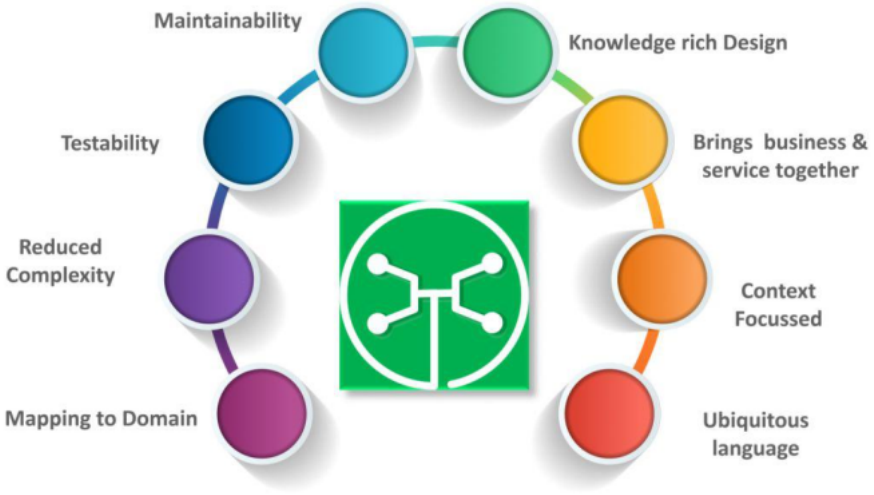


图 9：我们需要 DDD 的因素 – 微服务面试问题

13、什么是无所不在的语言？

如果您必须定义泛在语言（UL），那么它是特定域的开发人员和用户使用的通用语言，通过该语言可以轻松解释域。
无处不在的语言必须非常清晰，以便它将所有团队成员放在同一页面上，并以机器可以理解的方式进行翻译。

14、什么是凝聚力？

模块内部元素所属的程度被认为是凝聚力。

15、什么是耦合？

组件之间依赖关系强度的度量被认为是耦合。一个好的设计总是被认为具有高内聚力和低耦合性。

16、什么是 REST / RESTful 以及它的用途是什么？

Representational State Transfer（REST）/ RESTful Web 服务是一种帮助计算机系统通过 Internet 进行通信的架构风格。这使得微服务更容易理解和实现。微服务可以使用或不使用 RESTful API 实现，但使用 RESTful API 构建松散耦合的微服务总是更容易。

17、你对 Spring Boot 有什么了解？

事实上，随着新功能的增加，弹簧变得越来越复杂。如果必须启动新的 spring 项目，则必须添加构建路径或添加 maven 依赖项，配置应用程序服务器，添加 spring配置。所以一切都必须从头开始。Spring Boot 是解决这个问题的方法。使用 spring boot 可以避免所有样板代码和配置。因此，基本上认为自己就好像你正在烘烤蛋糕一样，春天就像制作蛋糕所需的成分一样，弹簧靴就是你手中的完整蛋糕。

图 10： Spring Boot 的因素 – 微服务面试问题

18、什么是 Spring 引导的执行器？

Spring Boot 执行程序提供了 restful Web 服务，以访问生产环境中运行应用程序的当前状态。在执行器的帮助下，您可以检查各种指标并监控您的应用程序。

19、什么是 Spring Cloud？

根据 Spring Cloud 的官方网站，Spring Cloud 为开发人员提供了快速构建分布式系统中一些常见模式的工具（例如配置管理，服务发现，断路器，智能路由，领导选举，分布式会话，集群状态）。

20、Spring Cloud 解决了哪些问题？

在使用 Spring Boot 开发分布式微服务时，我们面临的问题很少由 Spring Cloud解决。
□ 与分布式系统相关的复杂性 – 包括网络问题，延迟开销，带宽问题，安全问题。
□ 处理服务发现的能力 – 服务发现允许集群中的进程和服务找到彼此并进行通信。
□ 解决冗余问题 – 冗余问题经常发生在分布式系统中。
□ 负载平衡 – 改进跨多个计算资源（例如计算机集群，网络链接，中央处理单元）的工作负载分布。
□ 减少性能问题 – 减少因各种操作开销导致的性能问题。

21、在 Spring MVC 应用程序中使用 WebMvcTest 注释有什么用处？

在测试目标只关注 Spring MVC 组件的情况下，WebMvcTest 注释用于单元测试Spring MVC 应用程序。在上面显示的快照中，我们只想启动 ToTestController。执行此单元测试时，不会启动所有其他控制器和映射。

22、你能否给出关于休息和微服务的要点？

虽然您可以通过多种方式实现微服务，但 REST over HTTP 是实现微服务的一种方式。REST 还可用于其他应用程序，如 Web 应用程序，API 设计和 MVC 应用程序，以提供业务数据。
微服务是一种体系结构，其中系统的所有组件都被放入单独的组件中，这些组件可以单独构建，部署和扩展。微服务的某些原则和最佳实践有助于构建弹性应用程序。简而言之，您可以说 REST 是构建微服务的媒介。

23、什么是不同类型的微服务测试？

在使用微服务时，由于有多个微服务协同工作，测试变得非常复杂。因此，测试分为不同的级别。
□ 在底层，我们有面向技术的测试，如单元测试和性能测试。这些是完全自动化的。
□ 在中间层面，我们进行了诸如压力测试和可用性测试之类的探索性测试。
□ 在顶层， 我们的 验收测试数量很少。这些验收测试有助于利益相关者理解和验证软件功能。

24、您对 Distributed Transaction 有何了解？

分布式事务是指单个事件导致两个或多个不能以原子方式提交的单独数据源的突变的任何情况。在微服务的世界中，它变得更加复杂，因为每个服务都是一个工作单元，并且大多数时候多个服务必须协同工作才能使业务成功。

25、什么是 Idempotence 以及它在哪里使用？

幂等性是能够以这样的方式做两次事情的特性，即最终结果将保持不变，即好像它只做了一次。
用法：在远程服务或数据源中使用 Idempotence，这样当它多次接收指令时，它只处理指令一次。

26、什么是有界上下文？

有界上下文是域驱动设计的核心模式。DDD 战略设计部门的重点是处理大型模型和团队。DDD 通过将大型模型划分为不同的有界上下文并明确其相互关系来处理大型模型。

27、什么是双因素身份验证？

双因素身份验证为帐户登录过程启用第二级身份验证。

图 11： 双因素认证的表示 – 微服务访谈问题
因此，假设用户必须只输入用户名和密码，那么这被认为是单因素身份验证。

28、双因素身份验证的凭据类型有哪些？

这三种凭证是 ：

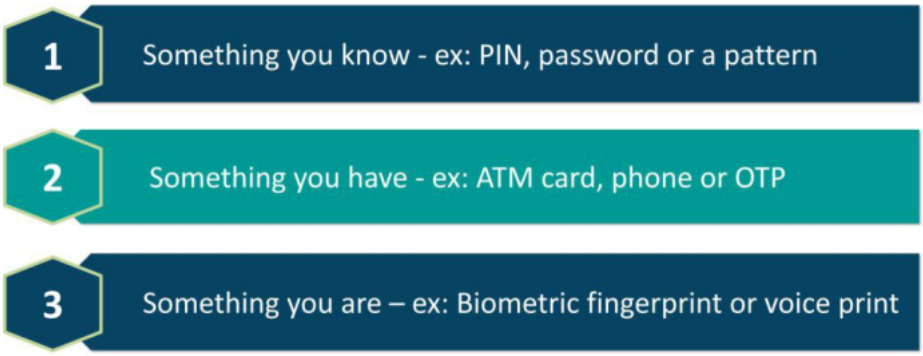


图 12： 双因素认证的证书类型 – 微服务面试问题

29、什么是客户证书？

客户端系统用于向远程服务器发出经过身份验证的请求的一种数字证书称为客户端证书。客户端证书在许多相互认证设计中起着非常重要的作用，为请求者的身份提供了强有力的保证。

30、PACT 在微服务架构中的用途是什么？

PACT 是一个开源工具，允许测试服务提供者和消费者之间的交互，与合同隔离，从而提高微服务集成的可靠性。
微服务中的用法
□ 用于在微服务中实现消费者驱动的合同。
□ 测试微服务的消费者和提供者之间的消费者驱动的合同。
查看即将到来的批次

31、什么是 OAuth？

OAuth 代表开放授权协议。这允许通过在 HTTP 服务上启用客户端应用程序（例如第三方提供商 Facebook，GitHub 等）来访问资源所有者的资源。因此，您可以在不使用其凭据的情况下与另一个站点共享存储在一个站点上的资源。

32、康威定律是什么？

“任何设计系统的组织（广泛定义）都将产生一种设计，其结构是组织通信结构的副本。” – Mel Conway

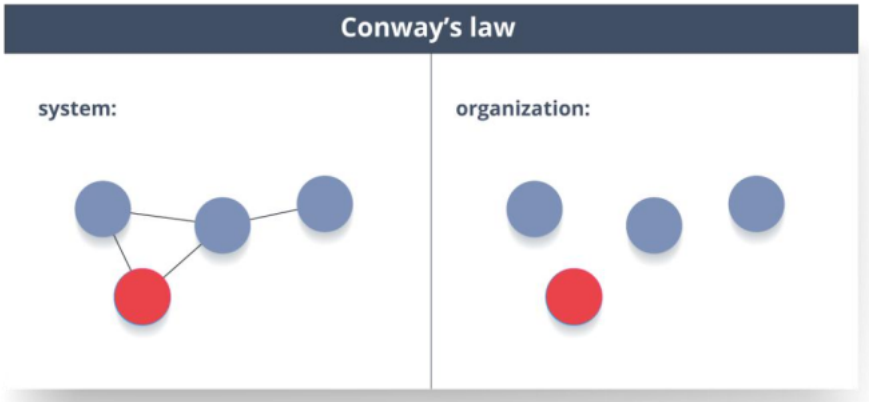


图 13： Conway 定律的表示 – 微服务访谈问题
该法律基本上试图传达这样一个事实：为了使软件模块起作用，整个团队应该进行良好的沟通。因此，系统的结构反映了产生它的组织的社会边界。

33、合同测试你懂什么？

根据 Martin Flower 的说法，合同测试是在外部服务边界进行的测试，用于验证其是否符合消费服务预期的合同。
此外，合同测试不会深入测试服务的行为。更确切地说，它测试该服务调用的输入＆输出包含所需的属性和所述响应延迟，吞吐量是允许的限度内

34、什么是端到端微服务测试？

端到端测试验证了工作流中的每个流程都正常运行。这可确保系统作为一个整体协同工作并满足所有要求。
通俗地说，你可以说端到端测试是一种测试，在特定时期后测试所有东西。

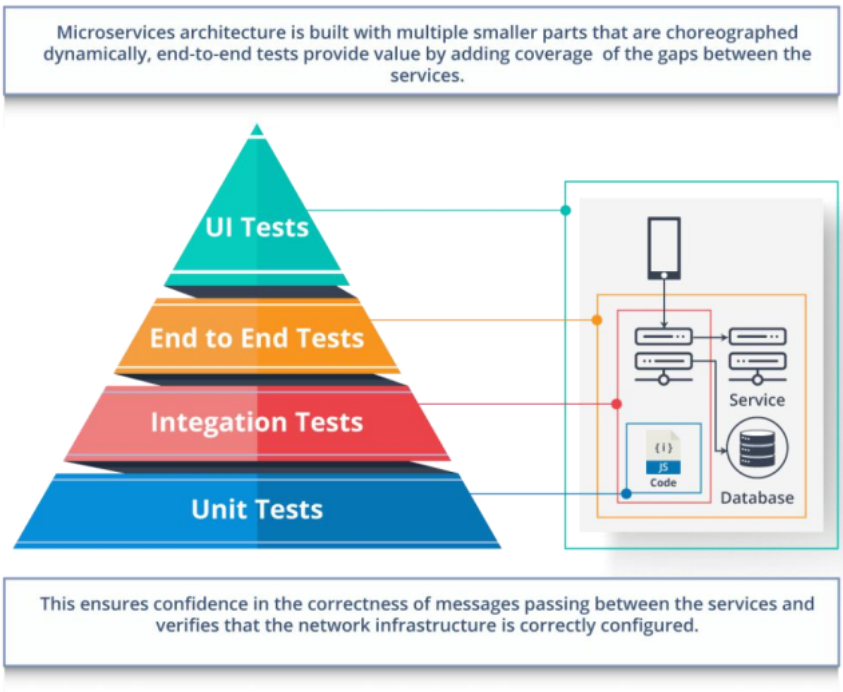


图 14：测试层次 – 微服务面试问题

35、Container 在微服务中的用途是什么？

容器是管理基于微服务的应用程序以便单独开发和部署它们的好方法。您可以将微服务封装在容器映像及其依赖项中，然后可以使用它来滚动按需实例的微服务，而无需任何额外的工作。

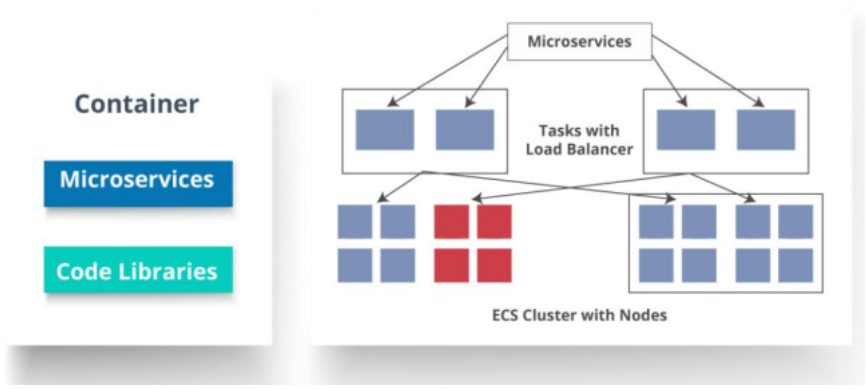


图 15： 容器的表示及其在微服务中的使用方式 – 微服务访谈问题

36、什么是微服务架构中的 DRY？

DRY 代表不要重复自己。它基本上促进了重用代码的概念。这导致开发和共享库，这反过来导致紧密耦合。

37、什么是消费者驱动的合同（CDC）？

这基本上是用于开发微服务的模式，以便它们可以被外部系统使用。当我们处理微服务时，有一个特定的提供者构建它，并且有一个或多个使用微服务的消费者。通常，提供程序在 XML 文档中指定接口。但在消费者驱动的合同中，每个服务消费者都传达了提供商期望的接口。

38、Web，RESTful API 在微服务中的作用是什么？

微服务架构基于一个概念，其中所有服务应该能够彼此交互以构建业务功能。因此，要实现这一点，每个微服务必须具有接口。这使得 Web API 成为微服务的一个非常重要的推动者。RESTful API 基于 Web 的开放网络原则，为构建微服务架构的各个组件之间的接口提供了最合理的模型。

39、您对微服务架构中的语义监控有何了解？

语义监控，也称为 综合监控， 将自动化测试与监控应用程序相结合，以检测业务失败因素。

40、我们如何进行跨功能测试？

跨功能测试是对非功能性需求的验证，即那些无法像普通功能那样实现的需求。

41、我们如何在测试中消除非决定论？

非确定性测试（NDT）基本上是不可靠的测试。所以，有时可能会发生它们通过，显然有时它们也可能会失败。当它们失败时，它们会重新运行通过。

从测试中删除非确定性的一些方法如下：
1、 隔离
2、 异步
3、 远程服务
4、 隔离

5、 时间
6、 资源泄漏

42、Mock 或 Stub 有什么区别？

存根
□ 一个有助于运行测试的虚拟对象。
□ 在某些可以硬编码的条件下提供固定行为。
□ 永远不会测试存根的任何其他行为。
例如，对于空堆栈，您可以创建一个只为 empty（）方法返回 true 的存根。因此，这并不关心堆栈中是否存在元素。
嘲笑
□ 一个虚拟对象，其中最初设置了某些属性。
□ 此对象的行为取决于 set 属性。
□ 也可以测试对象的行为。
例如，对于 Customer 对象，您可以通过设置名称和年龄来模拟它。您可以将 age设置为 12，然后测试 isAdult（）方法，该方法将在年龄大于 18 时返回 true。因此，您的 Mock Customer 对象适用于指定的条件。

43、您对 Mike Cohn 的测试金字塔了解多少？

Mike Cohn 提供了一个名为 Test Pyramid 的模型。这描述了软件开发所需的自动化测试类型。

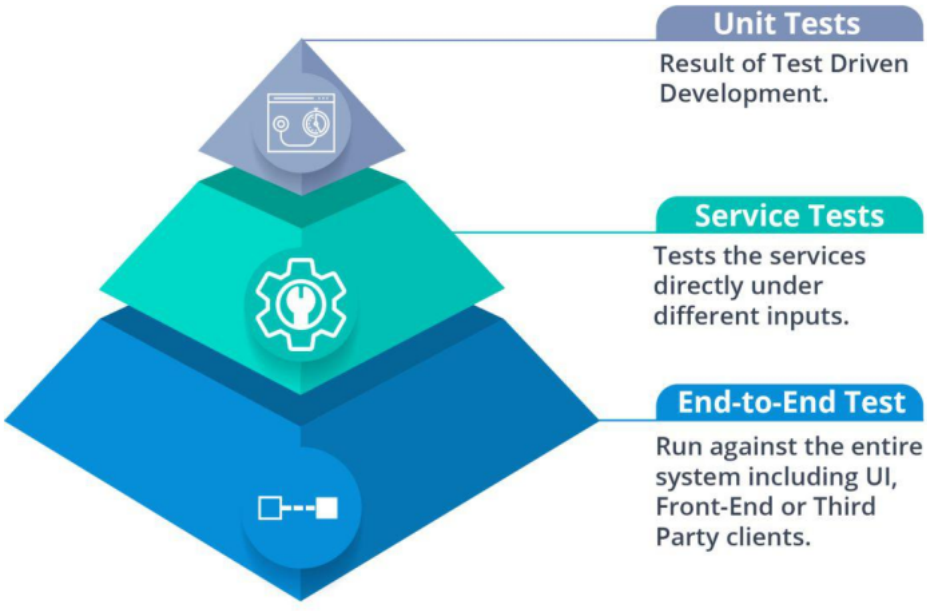


图 16： Mike Cohn 的测试金字塔 – 微服务面试问题
根据金字塔，第一层的测试数量应该最高。在服务层，测试次数应小于单元测试级别，但应大于端到端级别。

44、Docker 的目的是什么？

Docker 提供了一个可用于托管任何应用程序的容器环境。在此，软件应用程序和支持它的依赖项紧密打包在一起。因此，这个打包的产品被称为 Container，因为它是由 Docker 完成的，所以它被称为 Docker 容器！

45、什么是金丝雀释放？

Canary Releasing 是一种降低在生产中引入新软件版本的风险的技术。这是通过将变更缓慢地推广到一小部分用户，然后将其发布到整个基础架构，即将其提供给每个人来完成的。

46、什么是持续集成（CI）？

持续集成（CI）是每次团队成员提交版本控制更改时自动构建和测试代码的过程。这鼓励开发人员通过在每个小任务完成后将更改合并到共享版本控制存储库来共享代码和单元测试。

47、什么是持续监测？

持续监控深入监控覆盖范围，从浏览器内前端性能指标，到应用程序性能，再到主机虚拟化基础架构指标。

48、架构师在微服务架构中的角色是什么？

微服务架构中的架构师扮演以下角色：
□ 决定整个软件系统的布局。
□ 帮助确定组件的分区。因此，他们确保组件相互粘合，但不紧密耦合。
□ 与开发人员共同编写代码，了解日常生活中面临的挑战。
□ 为开发微服务的团队提供某些工具和技术的建议。
□ 提供技术治理，以便技术开发团队遵循微服务原则。

49、我们可以用微服务创建状态机吗？

我们知道拥有自己的数据库的每个微服务都是一个可独立部署的程序单元，这反过来又让我们可以创建一个状态机。因此，我们可以为特定的微服务指定不同的状态和事件。例如，我们可以定义 Order 微服务。订单可以具有不同的状态。Order 状态的转换可以是 Order 微服务中的独立事件。

50、什么是微服务中的反应性扩展？

Reactive Extensions 也称为 Rx。这是一种设计方法，我们通过调用多个服务来收集结果，然后编译组合响应。这些调用可以是同步或异步，阻塞或非阻塞。Rx是分布式系统中非常流行的工具，与传统流程相反。希望这些微服务面试问题可以帮助您进行微服务架构师访谈。
翻译来源：
https://www.edureka.co/blog/interview-questions/microservices-interview-questions/

Linux面试题

1、绝对路径用什么符号表示？当前目录、上层目录用什么表示？主目录用什么表示? 切换目录用什么命令？

答案：
绝对路径： 如/etc/init.d
当前目录和上层目录： ./ ../
主目录： ~/
切换目录： cd

2、怎么查看当前进程？怎么执行退出？怎么查看当前路径？

答案：
查看当前进程： ps
执行退出： exit
查看当前路径： pwd

3、怎么清屏？怎么退出当前命令？怎么执行睡眠？怎么查看当

前用户 id？查看指定帮助用什么命令？
答案：
清屏： clear
退出当前命令： ctrl+c 彻底退出
执行睡眠 ： ctrl+z 挂起当前进程 fg 恢复后台 查看当前用户 id： "id"：查看显示目前登陆账户的 uid 和 gid 及所属分组及用户名

查看指定帮助： 如 man adduser 这个很全 而且有例子； adduser --help 这个告诉你一些常用参数； info adduesr；

4、Ls 命令执行什么功能？ 可以带哪些参数，有什么区别？

答案：
ls 执行的功能： 列出指定目录中的目录，以及文件哪些参数以及区别： a 所有文件 l 详细信息，包括大小字节数，可读可写可执行的权限等

5、查看文件有哪些命令

vi 文件名 #编辑方式查看，可修改
cat 文件名 #显示全部文件内容
more 文件名 #分页显示文件内容
less 文件名 #与 more 相似，更好的是可以往前翻页
tail 文件名 #仅查看尾部，还可以指定行数
head 文件名 #仅查看头部,还可以指定行数

6、列举几个常用的Linux命令

列出文件列表：ls【参数 -a -l】
创建目录和移除目录：mkdir rmdir
用于显示文件后几行内容：tail，例如： tail -n 1000：显示最后1000行
打包：tar -xvf
打包并压缩：tar -zcvf
查找字符串：grep
显示当前所在目录：pwd创建空文件：touch
编辑器：vim vi

7、你平时是怎么查看日志的？

Linux查看日志的命令有多种: tail、cat、tac、head、echo等，本文只介绍几种常用的方法。
1、tail
最常用的一种查看方式
命令格式: tail[必要参数][选择参数][文件]
-f 循环读取
-q 不显示处理信息
-v 显示详细的处理信息
-c<数目> 显示的字节数
-n<行数> 显示行数
-q, --quiet, --silent 从不输出给出文件名的首部
-s, --sleep-interval=S 与-f合用,表示在每次反复的间隔休眠S秒

例如：

```
tail -n 10 test.log 查询日志尾部最后10行的日志;
tail -n +10 test.log 查询10行之后的所有日志;
tail -fn 10 test.log 循环实时查看最后1000行记录(最常用的)
```

一般还会配合着grep搜索用，例如：

```
tail -fn 1000 test.log | grep '关键字'
```

如果一次性查询的数据量太大,可以进行翻页查看，例如 ：

```
tail -n 4700 aa.log |more -1000 可以进行多屏显示(ctrl + f 或者 空格键可以快捷键)
```

2、head
跟tail是相反的head是看前多少行日志

```
head -n 10 test.log 查询日志文件中的头10行日志;
head -n -10 test.log 查询日志文件除了最后10行的其他所有日志;
```

head其他参数参考tail

3、cat
cat 是由第一行到最后一行连续显示在屏幕上
一次显示整个文件：

```
$ cat filename
```

从键盘创建一个文件：

```
$cat > filename
```

将几个文件合并为一个文件：

```
$cat file1 file2 > file 只能创建新文件,不能编辑已有文件
```

将一个日志文件的内容追加到另外一个：

```
$cat -n textfile1 > textfile2
```

清空一个日志文件:

```
$cat : >textfile2
```

注意：> 意思是创建，>>是追加。千万不要弄混了。
cat其他参数参考tail

4、more
more命令是一个基于vi编辑器文本过滤器，它以全屏幕的方式按页显示文本文件的内容，支持vi中的关键字定位操作。more名单中内置了若干快捷键，常用的有H（获得帮助信息），Enter（向下翻滚一行），空格（向下滚动一屏），Q（退出命令）。more命令从前向后读取文件，因此在启动时就加载整个文件。
该命令一次显示一屏文本，满屏后停下来，并且在屏幕的底部出现一个提示信息，给出至今已显示的该文件的百分比：–More–（XX%）
more的语法：more 文件名
Enter 向下n行，需要定义，默认为1行
Ctrl f 向下滚动一屏
空格键 向下滚动一屏
Ctrl b 返回上一屏
= 输出当前行的行号
:f 输出文件名和当前行的行号
v 调用vi编辑器
!命令 调用Shell，并执行命令
q退出more

5、sed
这个命令可以查找日志文件特定的一段，根据时间的一个范围查询，可以按照行号和时间范围查询按照行号

```
sed -n '5,10p' filename 这样你就可以只查看文件的第5行到第10行。
```

按照时间段

```
sed -n '/2014-12-17 16:17:20/,/2014-12-17 16:17:36/p' test.log
```

6、less
less命令在查询日志时，一般流程是这样的

```
less log.log
shift + G 命令到文件尾部 然后输入 ? 加上你要搜索的关键字例如 ? 1213
按 n 向上查找关键字
shift+n 反向查找关键字
less与more类似，使用less可以随意浏览文件，而more仅能向前移动，不能向后移动，而且 less 在查看
之前不会加载整个文件。
less log2013.log 查看文件
ps -ef | less ps查看进程信息并通过less分页显示
history | less 查看命令历史使用记录并通过less分页显示
less log2013.log log2014.log 浏览多个文件
```

常用命令参数：

```
less与more类似，使用less可以随意浏览文件，而more仅能向前移动，不能向后移动，而且 less 在查看
之前不会加载整个文件。
less log2013.log 查看文件
ps -ef | less ps查看进程信息并通过less分页显示
history | less 查看命令历史使用记录并通过less分页显示
less log2013.log log2014.log 浏览多个文件
```

```
常用命令参数：
-b <缓冲区大小> 设置缓冲区的大小
-g 只标志最后搜索的关键词
-i 忽略搜索时的大小写
-m 显示类似more命令的百分比
-N 显示每行的行号
-o <文件名> 将less 输出的内容在指定文件中保存起来
-Q 不使用警告音
-s 显示连续空行为一行
/字符串：向下搜索"字符串"的功能
?字符串：向上搜索"字符串"的功能
n：重复前一个搜索（与 / 或 ? 有关）
N：反向重复前一个搜索（与 / 或 ? 有关）
b 向后翻一页
h 显示帮助界面
q 退出less 命令
```

一般本人查日志配合应用的其他命令

```
history // 所有的历史记录
history | grep XXX // 历史记录中包含某些指令的记录
history | more // 分页查看记录
history -c // 清空所有的历史记录
!! 重复执行上一个命令
查询出来记录后选中 : !323
```

8、建立软链接(快捷方式)，以及硬链接的命令

```
软链接： ln -s slink source
硬链接： ln link source
```

9、目录创建用什么命令？创建文件用什么命令？复制文件用什么命令？

创建目录： mkdir

创建文件：典型的如 touch，vi 也可以创建文件，其实只要向一个不存在的文件输出，都会创建文件
复制文件： cp 7. 文件权限修改用什么命令？格式是怎么样的？
文件权限修改： chmod

格式如下：
chmodu+xfile 给 file 的属主增加执行权限 chmod 751 file 给 file 的属主分配读、写、执行(7)的权限，给 file 的所在组分配读、执行(5)的权限，给其他用户分配执行(1)的权限
chmodu=rwx,g=rx,o=xfile 上例的另一种形式 chmod =r file 为所有用户分配读权限
chmod444file 同上例 chmod a-wx,a+r file 同上例
$ chmod -R u+r directory 递归地给 directory 目录下所有文件和子目录的属主分配读的权限

10、查看文件内容有哪些命令可以使用？

vi 文件名 #编辑方式查看，可修改
cat 文件名 #显示全部文件内容
more 文件名 #分页显示文件内容
less 文件名 #与 more 相似，更好的是可以往前翻页
tail 文件名 #仅查看尾部，还可以指定行数
head 文件名 #仅查看头部,还可以指定行数

11、随意写文件命令？怎么向屏幕输出带空格的字符串，比如”hello world”?

写文件命令：vi
向屏幕输出带空格的字符串:echo hello world

12、终端是哪个文件夹下的哪个文件？黑洞文件是哪个文件夹下的哪个命令？

终端 /dev/tty
黑洞文件 /dev/null

13、移动文件用哪个命令？改名用哪个命令？

mv mv

14、复制文件用哪个命令？如果需要连同文件夹一块复制呢？如果需要有提示功能呢？

cp cp -r ？？ ？？

15、删除文件用哪个命令？如果需要连目录及目录下文件一块删除呢？删除空文件夹用什么命令？

rm rm -r rmdir

16、Linux 下命令有哪几种可使用的通配符？分别代表什么含义?

？“可替 代单 个字 符。
“*” 可替 代任 意多 个字 符。
方括 号“ [charset]” 可替 代 charset 集中 的任 何单 个字 符， 如 [a-z]， [abABC]

17、用什么命令对一个文件的内容进行统计？(行号、单词数、字节数)

wc 命令 - c 统计字节数 - l 统计行数 - w 统计字数。

18、Grep 命令有什么用？ 如何忽略大小写？ 如何查找不含该串的行?

是一种强大的文本搜索工具，它能使用正则表达式搜索文本，并把匹 配的行打印出来。
grep [stringSTRING] filename grep [^string] filename

19、Linux 中进程有哪几种状态？在 ps 显示出来的信息中分别用什么符号表示的？

1、不可中断状态：进程处于睡眠状态，但是此刻进程是不可中断的。不可中断，指进程不响应异步信号。

2、暂停状态/跟踪状态：向进程发送一个 SIGSTOP 信号，它就会因响应该信号 而进入 TASK_STOPPED 状态;当进程正在被跟踪时，它处于 TASK_TRACED 这个特殊的状态。正被跟踪”指的是进程暂停下来，等待跟踪它的进程对它进行操作。
3、就绪状态：在 run_queue 队列里的状态
4、运行状态：在 run_queue 队列里的状态
5、可中断睡眠状态：处于这个状态的进程因为等待某某事件的发生（比如等待socket 连接、等待信号量），而被挂起
6、zombie 状态（僵尸）：父亲没有通过 wait 系列的系统调用会顺便将子进程的尸体（task_struct）也释放掉
7、退出状态
D 不可中断 Uninterruptible（usually IO）
R 正在运行，或在队列中的进程
S 处于休眠状态
T 停止或被追踪
Z 僵尸进程
W 进入内存交换（从内核 2.6 开始无效）
X 死掉的进程

20、怎么使一个命令在后台运行?

一般都是使用 & 在命令结尾来让程序自动运行。(命令后可以不追加空格)

21、利用 ps 怎么显示所有的进程? 怎么利用 ps 查看指定进程的信息？

```
ps -ef (system v 输出)
ps -aux bsd 格式输出
ps -ef | grep pid
```

22、哪个命令专门用来查看后台任务?

job -l

23、把后台任务调到前台执行使用什么命令?把停下的后台任务在后台执行起来用什么命令?

把后台任务调到前台执行 fg
把停下的后台任务在后台执行起来 bg

24、终止进程用什么命令? 带什么参数?

kill [-s <信息名称或编号>][程序] 或 kill [-l <信息编号>]
kill-9 pid

25、怎么查看系统支持的所有信号？

kill -l

26、搜索文件用什么命令? 格式是怎么样的?

find <指定目录> <指定条件> <指定动作>
whereis 加参数与文件名
locate 只加文件名
find 直接搜索磁盘，较慢。
find / -name "string*"

27、查看当前谁在使用该主机用什么命令? 查找自己所在的终端信息用什么命令?

查找自己所在的终端信息：who am i
查看当前谁在使用该主机：who

28、使用什么命令查看用过的命令列表?

history

29、使用什么命令查看磁盘使用空间？ 空闲空间呢?

```
df -hl
```

文件系统 容量 已用 可用 已用% 挂载点

```
Filesystem Size Used Avail Use% Mounted on /dev/hda2 45G 19G 24G
44% /
/dev/hda1 494M 19M 450M 4% /boot
```

30、使用什么命令查看网络是否连通?

```
netstat
```

31、使用什么命令查看 ip 地址及接口信息？

```
ifconfig
```

32、查看各类环境变量用什么命令?

查看所有 env
查看某个，如 home： env $HOME

33、通过什么命令指定命令提示符?

□ \u：显示当前用户账号
□ \h：显示当前主机名

□ \W：只显示当前路径最后一个目录
□ \w：显示当前绝对路径（当前用户目录会以~代替）
□ $PWD：显示当前全路径
□ $：显示命令行'$'或者'#'符号
□ #：下达的第几个命令
□ \d：代表日期，格式为 week day month date，例如："MonAug1"
□ \t：显示时间为 24 小时格式，如：HH：MM：SS
□ \T：显示时间为 12 小时格式
□ \A：显示时间为 24 小时格式：HH：MM
□ \v：BASH 的版本信息 如 export PS1='[\u@\h\w#]$'

34、查找命令的可执行文件是去哪查找的? 怎么对其进行设置及添加?

whereis [-bfmsu][-B <目录>...][-M <目录>...][-S <目录>...][文件...]
补充说明：whereis指令会在特定目录中查找符合条件的文件。这些文件的烈性应属于原始代码，二进制文件，或是帮助文件。

-b 只查找二进制文件。

-B <目录> 只在设置的目录下查找二进制文件。 -f 不显示文件名前的路径名称。

-m 只查找说明文件。

-M <目录> 只在设置的目录下查找说明文件。-s 只查找原始代码文件。
-S <目录> 只在设置的目录下查找原始代码文件。 -u 查找不包含指定类型的文件。

w -h ich 指令会在 PATH 变量指定的路径中，搜索某个系统命令的位置，并且返回第一个搜索结果。
-n 指定文件名长度，指定的长度必须大于或等于所有文件中最长的文件名。
-p 与-n 参数相同，但此处的包括了文件的路径。 -w 指定输出时栏位的宽度。
-V 显示版本信息

35、通过什么命令查找执行命令?

which 只能查可执行文件
whereis 只能查二进制文件、说明文档，源文件等

36、怎么对命令进行取别名？

```
alias la='ls -a
```

37、du 和 df 的定义，以及区别？

du 显示目录或文件的大小

df 显示每个<文件>所在的文件系统的信息，默认是显示所有文件系统。（文件系统分配其中的一些磁盘块用来记录它自身的一些数据，如 i 节点，磁盘分布图，间接块，超级块等。这些数据对大多数用户级的程序来说是不可见的，通常称为 Meta Data。）du 命令是用户级的程序，它不考虑 Meta Data，而 df命令则查看文件系统的磁盘分配图并考虑 Meta Data。

df 命令获得真正的文件系统数据，而 du 命令只查看文件系统的部分情况。

38、awk 详解。

```
awk '{pattern + action}' {filenames}
#cat /etc/passwd |awk -F ':' '{print 1"\t"7}' //-F 的意思是以':'分隔 root
/bin/bash
daemon /bin/sh 搜索/etc/passwd 有 root 关键字的所有行
#awk -F: '/root/' /etc/passwd root:x:0:0:root:/root:/bin/bash
```

39、当你需要给命令绑定一个宏或者按键的时候，应该怎么做呢？

可以使用 bind 命令，bind 可以很方便地在 shell 中实现宏或按键的绑定。
在进行按键绑定的时候，我们需要先获取到绑定按键对应的字符序列。

比如获取 F12 的字符序列获取方法如下：先按下 Ctrl+V,然后按下 F12 .我们就可
以得到 F12 的字符序列 ^[[24~。
接着使用 bind 进行绑定

```
[root@localhost ~]# bind '"\e[24~":"date"'
```

注意：相同的按键在不同的终端或终端模拟器下可能会产生不同的字符序列。
【附】也可以使用 showkey -a 命令查看按键对应的字符序列。

40、如果一个 linux 新手想要知道当前系统支持的所有命令的列表，他需要怎么做？

使用命令 compgen -c，可以打印出所有支持的命令列表。

```
[root@localhost ~]$ compgen -c
l.
ll
ls
which
if
then
else
elif
fi
```

```
case
esac
for
select
while
until
do
done
…
```

41、如果你的助手想要打印出当前的目录栈，你会建议他怎么做？

使用 Linux 命令 dirs 可以将当前的目录栈打印出来。

```
[root@localhost ~]# dirs
/usr/share/X11
```

42、你的系统目前有许多正在运行的任务，在不重启机器的条件下，有什么方法可以把所有正在运行的进程移除呢？

使用 linux 命令 'disown -r '可以将所有正在运行的进程移除。

43、bash shell 中的 hash 命令有什么作用？

linux 命令'hash'管理着一个内置的哈希表，记录了已执行过的命令的完整路径,用该命令可以打印出你所使用过的命令以及执行的次数。

```
[root@localhost ~]# hash
hits command
2 /bin/ls
2 /bin/su
```

44、哪一个 bash 内置命令能够进行数学运算。

bash shell 的内置命令 let 可以进行整型数的数学运算

```
#! /bin/bash
… … le
t c=a+b
… …
```

45、怎样一页一页地查看一个大文件的内容呢？

通过管道将命令"cat file_name.txt" 和 'more' 连接在一起可以实现这个需要

```
[root@localhost ~]# cat file_name.txt | more
```

46、数据字典属于哪一个用户的？

数据字典是属于'SYS'用户的，用户'SYS' 和 'SYSEM'是由系统默认自动创建的

47、怎样查看一个 linux 命令的概要与用法？假设你在/bin 目录中偶然看到一个你从没见过的的命令，怎样才能知道它的作用和用法呢？

使用命令 whatis 可以先出显示出这个命令的用法简要，比如，你可以使用 whatis zcat 去查看'zcat'的介绍以及使用简要。

```
[root@localhost ~]# whatis zcat
zcat [gzip] (1) - compress or expand files
```

48、使用哪一个命令可以查看自己文件系统的磁盘空间配额呢？

使用 命令 repquota 能够 显示 出一 个文 件系 统的 配额 信息
【附 】只 有 root 用户 才能 够查 看其 它用 户的 配额 。